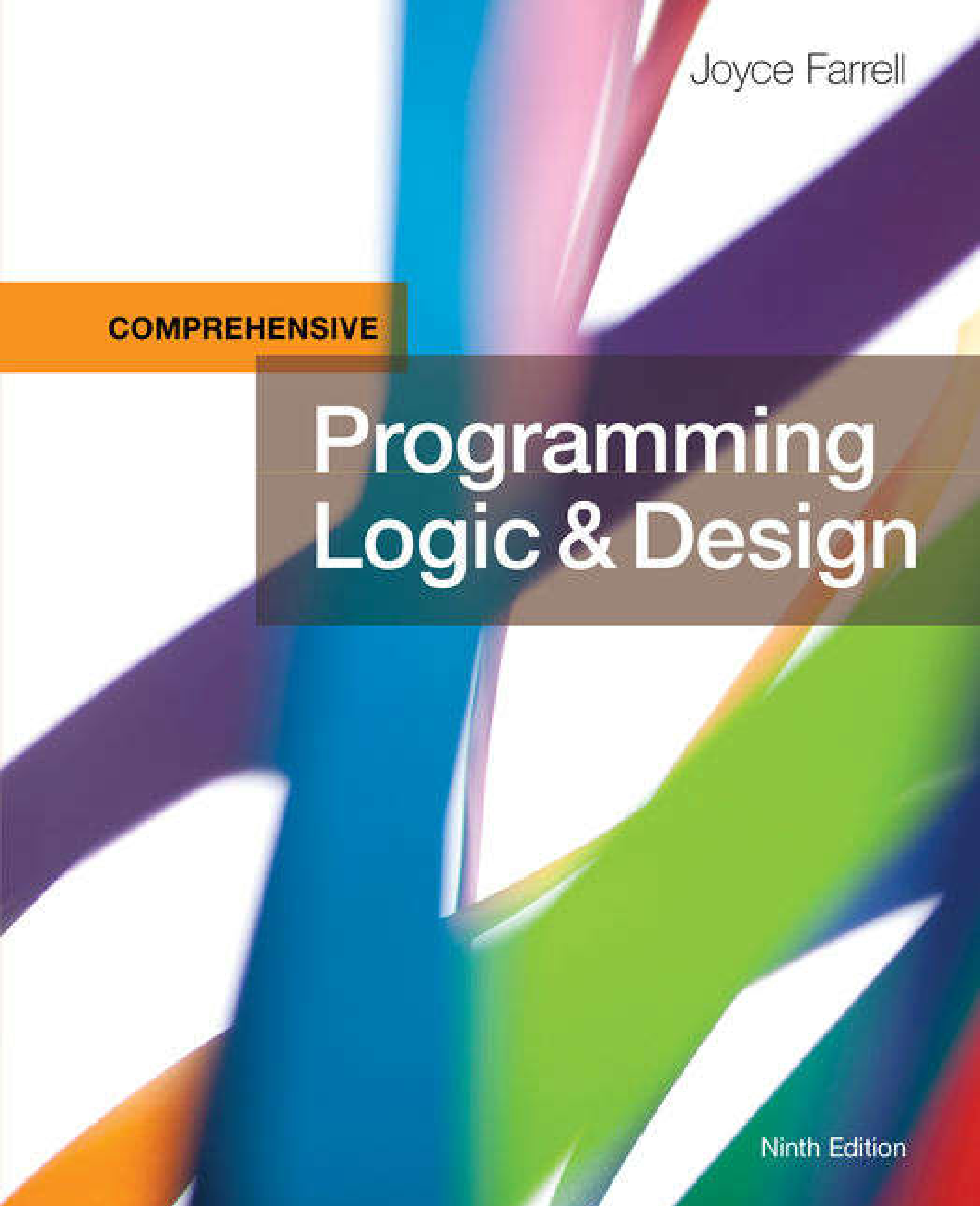

Joyce Farrell
COMPREHENSIVE
Programming Logic & Design
Ninth Edition

PROGRAMMING LOGIC AND DESIGN

COMPREHENSIVE

NINTH EDITION

PROGRAMMING LOGIC AND DESIGN

COMPREHENSIVE

JOYCE FARRELL

CENGAGE Learning®

Australia • Brazil • Japan • Korea • Mexico • Singapore • Spain • United Kingdom • United States



Programming Logic and Design, Comprehensive
Ninth Edition
Joyce Farrell

Senior Product Director:
Kathleen McMahon

Product Team Leader: Kristin McNary

Associate Product Manager: Kate Mason

Senior Content Developer: Alyssa Pratt

Senior Content Project Manager:
Jennifer Feltri-George

Manufacturing Planner: Julio Esperas

Art Director: Diana Graham

Production Service/Composition:
SPi Global

Cover Photo:
Colormos/Photodisc/Getty Images

© 2018 Cengage Learning®

ALL RIGHTS RESERVED. No part of this work covered by the copyright herein may be reproduced or distributed in any form or by any means, except as permitted by U.S. copyright law, without the prior written permission of the copyright owner.

For product information and technology assistance, contact us at **Cengage Learning Customer & Sales Support, 1-800-354-9706**

For permission to use material from this text or product, submit all requests online at **www.cengage.com/permissions**
Further permissions questions can be emailed to **permissionrequest@cengage.com**

Library of Congress Control Number: 2016959742

ISBN: 978-1-337-10207-0

Cengage Learning
20 Channel Center Street
Boston, MA 02210
USA

Unless otherwise noted all items © Cengage Learning.

Cengage Learning is a leading provider of customized learning solutions with employees residing in nearly 40 different countries and sales in more than 125 countries around the world. Find your local representative at **www.cengage.com**.

Cengage Learning products are represented in Canada by Nelson Education, Ltd.

To learn more about Cengage Learning Solutions, visit **www.cengage.com**

Purchase any of our products at your local college store or at our preferred online store **www.cengagebrain.com**

Printed in the United States of America
Print Number: 01 Print Year: 2018

Brief Contents

	Preface	xvi
CHAPTER 1	An Overview of Computers and Programming	1
CHAPTER 2	Elements of High-Quality Programs	38
CHAPTER 3	Understanding Structure	87
CHAPTER 4	Making Decisions	124
CHAPTER 5	Looping	176
CHAPTER 6	Arrays	227
CHAPTER 7	File Handling and Applications	272
CHAPTER 8	Advanced Data Handling Concepts	321
CHAPTER 9	Advanced Modularization Techniques	366
CHAPTER 10	Object-Oriented Programming	420
CHAPTER 11	More Object-Oriented Programming Concepts	464
CHAPTER 12	Event-Driven GUI Programming, Multithreading, and Animation	507
APPENDIX A	Understanding Numbering Systems and Computer Codes	539
APPENDIX B	Solving Difficult Structuring Problems	547
	Glossary	556
	Index	571

Contents

Preface xvi

CHAPTER 1 An Overview of Computers and Programming . 1

Understanding Computer Systems 2
Understanding Simple Program Logic 5
Understanding the Program Development Cycle 8
Understanding the Problem. 8
Planning the Logic. 10
Coding the Program 10
Using Software to Translate the Program into Machine Language 11
Testing the Program 12
Putting the Program into Production. 13
Maintaining the Program 14
Using Pseudocode Statements and Flowchart Symbols . . . 15
Writing Pseudocode 15
Drawing Flowcharts 17
Repeating Instructions 19
Using a Sentinel Value to End a Program 20
Understanding Programming and User Environments 23
Understanding Programming Environments 23
Understanding User Environments 25
Understanding the Evolution of Programming Models 27
Chapter Summary 28
Key Terms 29
Exercises. 32

CHAPTER 2 Elements of High-Quality Programs 38

Declaring and Using Variables and Constants 39
Understanding Data Types 39
Understanding Unnamed, Literal Constants 39
Working with Variables 40
Understanding a Declaration's Data Type 41

Understanding a Declaration's Identifier 42
Assigning Values to Variables. 45
Declaring Named Constants 46
Performing Arithmetic Operations. 47
The Integer Data Type 50
Understanding the Advantages of Modularization 51
Modularization Provides Abstraction. 52
Modularization Helps Multiple Programmers to Work on a Problem 53
Modularization Allows You to Reuse Work 53
Modularizing a Program 54
Declaring Variables and Constants within Modules 58
Understanding the Most Common Configuration for Mainline Logic 60
Creating Hierarchy Charts 64
Features of Good Program Design 66
Using Program Comments 67
Choosing Identifiers 69
Designing Clear Statements 71
Writing Clear Prompts and Echoing Input 72
Maintaining Good Programming Habits 74
Chapter Summary 75
Key Terms 76
Exercises. 79

CHAPTER 3 Understanding Structure 87

The Disadvantages of Unstructured Spaghetti Code 88
Understanding the Three Basic Structures 90
The Sequence Structure 90
The Selection Structure 91
The Loop Structure 92
Combining Structures 93
Using a Priming Input to Structure a Program 99
Understanding the Reasons for Structure 106
Recognizing Structure 107
Structuring and Modularizing Unstructured Logic 110
Chapter Summary 115
Key Terms 115
Exercises. 117

CHAPTER 4 Making Decisions **124**

The Selection Structure 125
Using Relational Comparison Operators 129
Avoiding a Common Error with Relational Operators . . .133
Understanding *AND* Logic 134
Nesting *AND* Decisions for Efficiency 137
Using the AND Operator 139
Avoiding Common Errors in an *AND* Selection 141
Understanding *OR* Logic 143
Writing *OR* Selections for Efficiency 145
Using the OR Operator 147
Avoiding Common Errors in an *OR* Selection 147
Understanding *NOT* Logic 153
Avoiding a Common Error in a *NOT* Expression 154
Making Selections within Ranges 155
Avoiding Common Errors When Using Range Checks . . .157
Understanding Precedence When Combining AND and OR Operators 160
Understanding the `case` Structure 163
Chapter Summary 165
Key Terms 166
Exercises 167

CHAPTER 5 Looping **176**

Appreciating the Advantages of Looping 177
Using a Loop Control Variable 179
Using a Definite Loop with a Counter 179
Using an Indefinite Loop with a Sentinel Value 181
Understanding the Loop in a Program's Mainline Logic . .183
Nested Loops 185
Avoiding Common Loop Mistakes 190
Mistake: Failing to Initialize the Loop Control Variable . .190
Mistake: Neglecting to Alter the Loop Control Variable . .191
Mistake: Using the Wrong Type of Comparison When Testing the Loop Control Variable 192
Mistake: Including Statements Inside the Loop Body that Belong Outside the Loop 194
Using a `for` Loop 199
Using a Posttest Loop 201
Recognizing the Characteristics Shared by Structured Loops 203

Common Loop Applications 205
Using a Loop to Accumulate Totals 205
Using a Loop to Validate Data 209
Limiting a Reprompting Loop 209
Validating a Data Type 212
Validating Reasonableness and Consistency of Data . . .213
Comparing Selections and Loops 214
Chapter Summary 218
Key Terms 218
Exercises. 220

CHAPTER 6 Arrays 227

Storing Data in Arrays 228
How Arrays Occupy Computer Memory 228
How an Array Can Replace Nested Decisions. 231
Using Constants with Arrays 238
Using a Constant as the Size of an Array 238
Using Constants as Array Element Values 239
Using a Constant as an Array Subscript 239
Searching an Array for an Exact Match 240
Using Parallel Arrays 244
Improving Search Efficiency 248
Searching an Array for a Range Match. 250
Remaining within Array Bounds 255
Understanding Array Size 255
Understanding Subscript Bounds 255
Using a `for` Loop to Process an Array 258
Chapter Summary 260
Key Terms 261
Exercises. 261

CHAPTER 7 File Handling and Applications 272

Understanding Computer Files 273
Organizing Files 274
Understanding the Data Hierarchy. 275
Performing File Operations 277
Declaring a File Identifier. 277
Opening a File 278
Reading Data from a File and Processing It 278
Writing Data to a File 281
Closing a File 281

A Program that Performs File Operations 282
Understanding Control Break Logic 285
Merging Sequential Files 290
Master and Transaction File Processing 299
Random Access Files 308
Chapter Summary 309
Key Terms 310
Exercises. 312

CHAPTER 8 Advanced Data Handling Concepts 321

Understanding the Need for Sorting Data 322
Using the Bubble Sort Algorithm 324
Understanding Swapping Values 324
Understanding the Bubble Sort 325
Sorting Multifield Records 340
Sorting Data Stored in Parallel Arrays 340
Sorting Records as a Whole 341
Other Sorting Algorithms. 342
Using Multidimensional Arrays 345
Using Indexed Files and Linked Lists 351
Using Indexed Files 352
Using Linked Lists. 353
Chapter Summary 356
Key Terms 357
Exercises. 358

CHAPTER 9 Advanced Modularization Techniques . . . 366

The Parts of a Method 367
Using Methods with no Parameters 368
Creating Methods that Require Parameters 371
Creating Methods that Require Multiple Parameters. . . .377
Creating Methods that Return a Value 379
Using an IPO Chart 384
Passing an Array to a Method 386
Overloading Methods 394
Avoiding Ambiguous Methods. 397
Using Predefined Methods 400
Method Design Issues: Implementation Hiding, Cohesion, and Coupling 402
Understanding Implementation Hiding 402
Increasing Cohesion 403
Reducing Coupling. 404
Understanding Recursion. 405

Chapter Summary 410
Key Terms 411
Exercises. 412

CHAPTER 10 Object-Oriented Programming. 420

Principles of Object-Oriented Programming 421
Classes and Objects. 421
Polymorphism. 424
Inheritance 426
Encapsulation. 426
Defining Classes and Creating Class Diagrams. 428
Creating Class Diagrams. 430
The Set Methods 433
The Get Methods 434
Work Methods. 435
Understanding Public and Private Access 437
Organizing Classes 440
Understanding Instance Methods 441
Understanding Static Methods 447
Using Objects. 448
Passing an Object to a Method 449
Returning an Object from a Method 450
Using Arrays of Objects 453
Chapter Summary 455
Key Terms 456
Exercises. 458

CHAPTER 11 More Object-Oriented Programming Concepts 464

Understanding Constructors 465
Default Constructors 466
Non-default Constructors. 468
Overloading Instance Methods and Constructors 469
Understanding Destructors. 472
Understanding Composition 474
Understanding Inheritance 475
Understanding Inheritance Terminology 478
Accessing Private Fields and Methods of a Parent Class. 481
Overriding Parent Class Methods in a Child Class. 486

Using Inheritance to Achieve Good Software Design . . .486
An Example of Using Predefined Classes:
Creating GUI Objects 487
Understanding Exception Handling 488
Drawbacks to Traditional Error-Handling Techniques . . .489
The Object-Oriented Exception-Handling Model 491
Using Built-in Exceptions and Creating
Your Own Exceptions 493
Reviewing the Advantages of Object-Oriented Programming 494
Chapter Summary 495
Key Terms 496
Exercises. 497

CHAPTER 12 Event-Driven GUI Programming, Multithreading, and Animation **507**

Understanding Event-Driven Programming 508
User-Initiated Actions and GUI Components 511
Designing Graphical User Interfaces 514
The Interface Should Be Natural and Predictable 514
The Interface Should Be Attractive, Easy to Read,
and Nondistracting 515
To Some Extent, It's Helpful If the User Can Customize
Your Applications. 516
The Program Should Be Forgiving. 516
The GUI Is Only a Means to an End 516
Developing an Event-Driven Application 517
Creating Wireframes. 518
Creating Storyboards 518
Defining the Storyboard Objects in an Object Dictionary. .519
Defining Connections Between the User Screens 520
Planning the Logic. 520
Understanding Threads and Multithreading. 525
Creating Animation 528
Chapter Summary 531
Key Terms 532
Exercises. 533

APPENDIX A Understanding Numbering Systems and Computer Codes 539

APPENDIX B Solving Difficult Structuring Problems . . . 547

Glossary 556

Index 571

Preface

Programming Logic and Design, Comprehensive, Ninth Edition, provides the beginning programmer with a guide to developing structured program logic. This textbook assumes no programming language experience. The writing is nontechnical and emphasizes good programming practices. The examples are business examples; they do not assume mathematical background beyond high school business math.

Additionally, the examples illustrate one or two major points; they do not contain so many features that students become lost following irrelevant and extraneous details. The examples in this book have been created to provide students with a sound background in logic, no matter what programming languages they eventually use to write programs. This book can be used in a stand-alone logic course that students take as a prerequisite to a programming course, or as a companion book to an introductory programming text using any programming language.

Organization and Coverage

Programming Logic and Design, Comprehensive, Ninth Edition, introduces students to programming concepts and enforces good style and logical thinking. General programming concepts are introduced in Chapter 1.

Chapter 2 discusses using data and introduces two important concepts: modularization and creating high-quality programs. It is important to emphasize these topics early so that students start thinking in a modular way and concentrate on making their programs efficient, robust, easy to read, and easy to maintain.

Chapter 3 covers the key concepts of structure, including what structure is, how to recognize it, and most importantly, the advantages to writing structured programs. This chapter's content is unique among programming texts. The early overview of structure presented here provides students a solid foundation for thinking in a structured way.

Chapters 4, 5, and 6 explore the intricacies of decision making, looping, and array manipulation. Chapter 7 provides details of file handling so that students can create programs that process a significant amount of data.

In Chapters 8 and 9, students learn more advanced techniques in array manipulation and modularization. Chapters 10 and 11 provide a thorough, yet accessible, introduction to concepts and terminology used in object-oriented programming. Students learn about classes, objects, instance and static class members, constructors, destructors, inheritance, and the

advantages of object-oriented thinking. Chapter 12 explores some additional object-oriented programming issues: event-driven GUI programming, multithreading, and animation.

Two appendices instruct students on working with numbering systems and providing structure for large programs.

Programming Logic and Design combines text explanation with flowcharts and pseudocode examples to provide students with alternative means of expressing structured logic. Numerous detailed, full-program exercises at the end of each chapter illustrate the concepts explained within the chapter, and reinforce understanding and retention of the material presented.

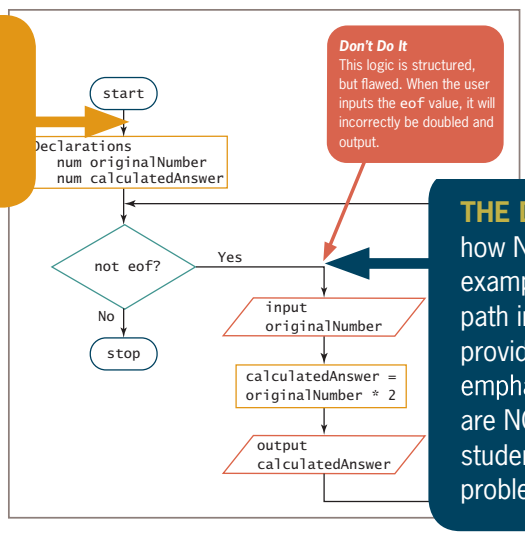
Programming Logic and Design distinguishes itself from other programming logic books in the following ways:

- It is written and designed to be non-language specific. The logic used in this book can be applied to any programming language.
- The examples are everyday business examples: no special knowledge of mathematics, accounting, or other disciplines is assumed.
- The concept of structure is covered earlier than in many other texts. Students are exposed to structure naturally, so that they will automatically create properly designed programs.
- Text explanation is interspersed with both flowcharts and pseudocode so that students can become comfortable with these logic development tools and understand their interrelationship. Screen shots of running programs also are included, providing students with a clear and concrete image of the programs' execution.
- Complex programs are built through the use of complete business examples. Students see how an application is constructed from start to finish, instead of studying only segments of a program.

Features

This text focuses on helping students become better programmers, as well as helping them understand the big picture in program development through a variety of features. Each chapter begins with objectives and ends with a list of key terms and a summary; these useful features will help students organize their learning experience.

FLOWCHARTS, figures, and illustrations provide the reader with a visual learning experience.



THE DON'T DO IT ICON illustrates how NOT to do something—for example, having a dead code path in a program. This icon provides a visual jolt to the student, emphasizing that particular figures are NOT to be emulated and making students more careful to recognize problems in existing code.

Figure 3-17 Structured but incorrect solution to the number-doubling problem

tested. Instead, a result is calculated and displayed one last time before the loop-controlling test is made again. If the program was written to recognize `eof` when `originalNumber` is 0, then an extraneous answer of 0 will be displayed before the program ends. Depending on the language you are using and on the type of input being used, the results might be worse: The program might terminate by displaying an error message or the value output might be indecipherable garbage. In any case, this last output is superfluous—no value should be doubled and output after the `eof` condition is encountered.

As a general rule, a program-ending test should always come immediately after an input statement because that's the earliest point at which it can be evaluated. Therefore, the best solution to the number-doubling problem remains the one shown in Figure 3-16—the structured solution containing the priming input statement.

Understanding Simple Program Logic

- The instruction `myAnswer = myNumber * 2` is an example of a processing operation. In most programming languages, an asterisk is used to indicate multiplication, so this instruction means "Change the value of the memory location `myAnswer` to equal the value at the memory location `myNumber` times two." Mathematical operations are not the only kind of processing operations, but they are very typical. As with input operations, f hardware used for processing is irrelevant—after you write a program, it can computers of different brand names, sizes, and speeds.

7

ber-doubling program, the `output myAnswer` instruction is an example of an ration. Within a particular program, this statement could cause the output n the monitor (which might be a flat-panel plasma screen or a smartphone the output could go to a printer (which could be laser or ink-jet), or the ld be written to a disk or DVD. The logic of the output process is the same no t hardware device you use. When this instruction executes, the value stored at the location named `myAnswer` is sent to an output device. (The output remains in computer memory until something else is stored at the same memory location or power is lost.)

Watch the video *A Simple Program*.

Computer memory consists of millions of numbered locations where data can be stored. The memory location of `myNumber` has a specific numeric address, but when you write programs, you seldom need to be concerned with the value of the memory address; instead, you use the easy-to-remember name you created. Computer programmers often refer to memory addresses using hexadecimal notation, or base 16. Using this system, they might use a value like 42FF01A to refer to a memory address. Despite the use of letters, such an address is still a number. Appendix A contains information about the hexadecimal numbering system.

TWO TRUTHS & A LIE

Understanding Simple Program

1. A program with syntax errors can execute but results.
2. Although the syntax of programming languages logic can be expressed in different languages.
3. Most simple computer programs include steps processing, and output.

A program with syntax errors cannot execute; rors can execute, but might produce incorrect results.

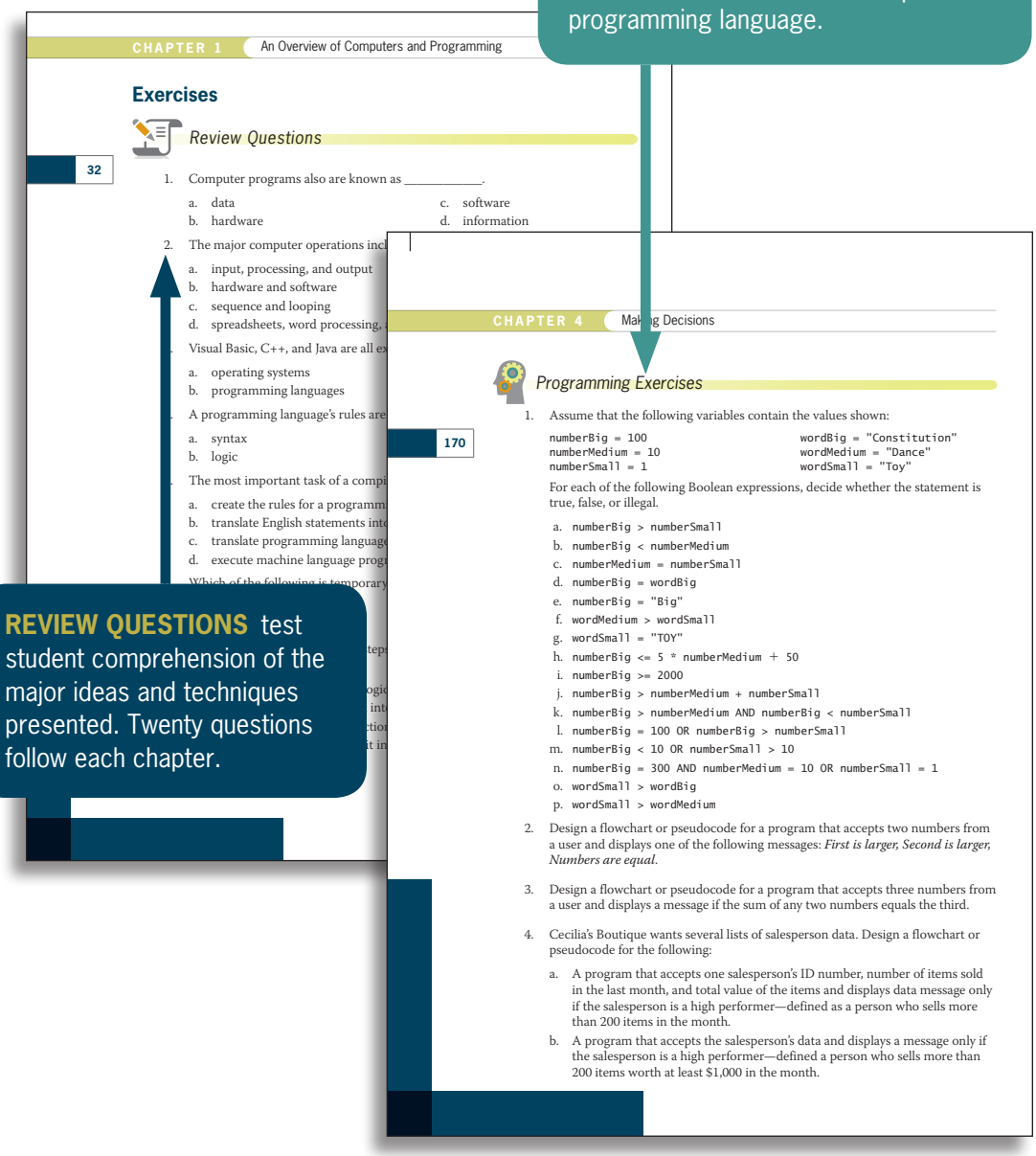
VIDEO LESSONS help explain important chapter concepts. Videos are part of the text's MindTap.

NOTES provide additional information—for example, another location in the book that expands on a topic, or a common error to avoid.

TWO TRUTHS & A LIE mini quizzes appear after each chapter section, with answers provided. The quiz contains three statements based on the preceding section of text—two statements are true and one is false. Answers give immediate feedback without "giving away" answers to the multiple-choice questions and programming problems later in the chapter. Students also have the option to take these quizzes electronically MindTap.

Assessment

PROGRAMMING EXERCISES provide opportunities to practice chapter material. These exercises increase in difficulty and allow students to explore logical programming concepts. Most exercises can be completed using flowcharts, pseudocode, or both. In addition, instructors can assign the exercises as programming problems to be coded and executed in a particular programming language.



CHAPTER 1 An Overview of Computers and Programming

32

Exercises

Review Questions

1. Computer programs also are known as _____.
   a. data
   b. hardware
   c. software
   d. information
2. The major computer operations incl
   a. input, processing, and output
   b. hardware and software
   c. sequence and looping
   d. spreadsheets, word processing,
3. Visual Basic, C++, and Java are all ex
   a. operating systems
   b. programming languages
4. A programming language's rules are
   a. syntax
   b. logic
5. The most important task of a compi
   a. create the rules for a programm
   b. translate English statements int
   c. translate programming languag
   d. execute machine language prog
6. Which of the following is temporary

REVIEW QUESTIONS test student comprehension of the major ideas and techniques presented. Twenty questions follow each chapter.

CHAPTER 4 Making Decisions

170

Programming Exercises

1. Assume that the following variables contain the values shown:

```
numberBig = 100                    wordBig = "Constitution"
numberMedium = 10                  wordMedium = "Dance"
numberSmall = 1                    wordSmall = "Toy"
```

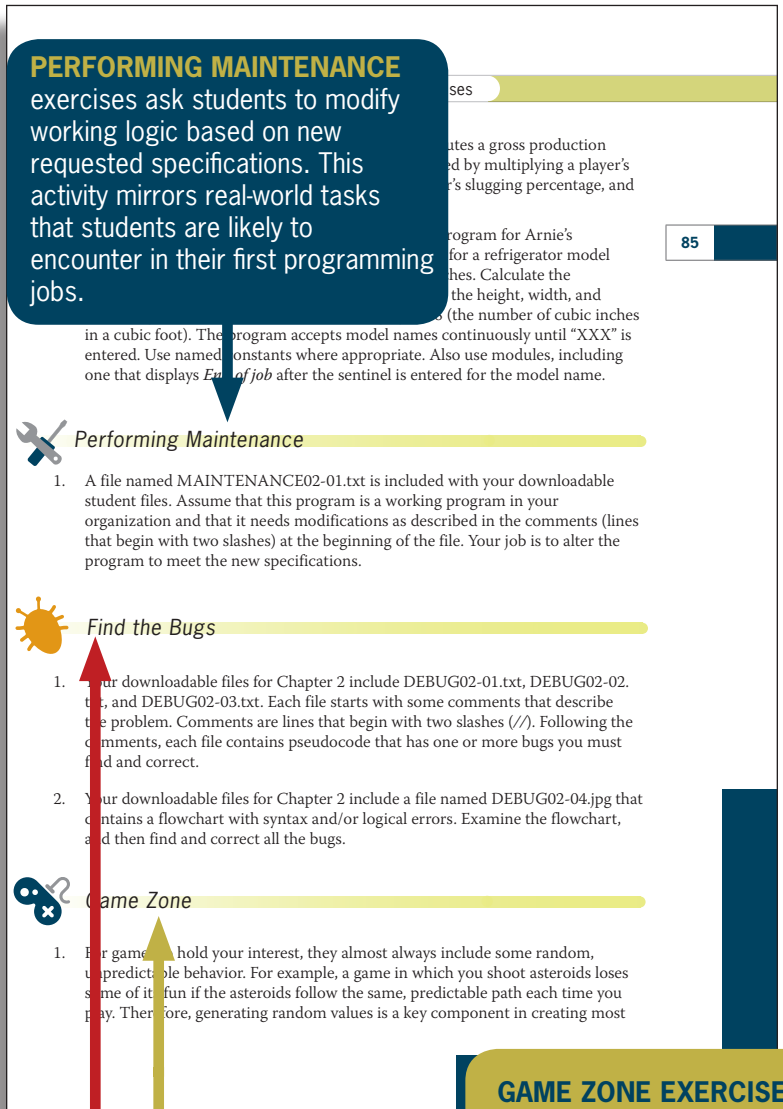
   For each of the following Boolean expressions, decide whether the statement is true, false, or illegal.

   a. `numberBig > numberSmall`
   b. `numberBig < numberMedium`
   c. `numberMedium = numberSmall`
   d. `numberBig = wordBig`
   e. `numberBig = "Big"`
   f. `wordMedium > wordSmall`
   g. `wordSmall = "TOY"`
   h. `numberBig <= 5 * numberMedium + 50`
   i. `numberBig >= 2000`
   j. `numberBig > numberMedium + numberSmall`
   k. `numberBig > numberMedium AND numberBig < numberSmall`
   l. `numberBig = 100 OR numberBig > numberSmall`
   m. `numberBig < 10 OR numberSmall > 10`
   n. `numberBig = 300 AND numberMedium = 10 OR numberSmall = 1`
   o. `wordSmall > wordBig`
   p. `wordSmall > wordMedium`

2. Design a flowchart or pseudocode for a program that accepts two numbers from a user and displays one of the following messages: *First is larger, Second is larger, Numbers are equal.*

3. Design a flowchart or pseudocode for a program that accepts three numbers from a user and displays a message if the sum of any two numbers equals the third.

4. Cecilia's Boutique wants several lists of salesperson data. Design a flowchart or pseudocode for the following:
   a. A program that accepts one salesperson's ID number, number of items sold in the last month, and total value of the items and displays data message only if the salesperson is a high performer—defined as a person who sells more than 200 items in the month.
   b. A program that accepts the salesperson's data and displays a message only if the salesperson is a high performer—defined a person who sells more than 200 items worth at least $1,000 in the month.



ses

utes a gross production
d by multiplying a player's
r's slugging percentage, and

85

rogram for Arnie's
for a refrigerator model
hes. Calculate the
the height, width, and
(the number of cubic inches
in a cubic foot). The program accepts model names continuously until "XXX" is entered. Use named constants where appropriate. Also use modules, including one that displays *End of job* after the sentinel is entered for the model name.

Performing Maintenance

1. A file named MAINTENANCE02-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.

Find the Bugs

1. Your downloadable files for Chapter 2 include DEBUG02-01.txt, DEBUG02-02.txt, and DEBUG02-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 2 include a file named DEBUG02-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.

Game Zone

1. For games to hold your interest, they almost always include some random, unpredictable behavior. For example, a game in which you shoot asteroids loses some of its fun if the asteroids follow the same, predictable path each time you play. Therefore, generating random values is a key component in creating most

PERFORMING MAINTENANCE exercises ask students to modify working logic based on new requested specifications. This activity mirrors real-world tasks that students are likely to encounter in their first programming jobs.

GAME ZONE EXERCISES are included at the end of each chapter. Students can create games as an additional entertaining way to understand key programming concepts.

DEBUGGING EXERCISES are included with each chapter because examining programs critically and closely is a crucial programming skill. Students can download these exercises at *www.cengagebrain.com* and through MindTap. These files are also available to instructors through sso.cengage.com.

Other Features of the Text

This edition of the text includes many features to help students become better programmers and understand the big picture in program development.

- **Clear explanations**. The language and explanations in this book have been refined over eight editions, providing the clearest possible explanations of difficult concepts.
- **Emphasis on structure**. More than its competitors, this book emphasizes structure. Chapter 3 provides an early picture of the major concepts of structured programming.
- **Emphasis on modularity**. From the second chapter onwards, students are encouraged to write code in concise, easily manageable, and reusable modules. Instructors have found that modularization should be encouraged early to instill good habits and a clearer understanding of structure.
- **Objectives**. Each chapter begins with a list of objectives so that the student knows the topics that will be presented in the chapter. In addition to providing a quick reference to topics covered, this feature provides a useful study aid.
- **Chapter summaries**. Following each chapter is a summary that recaps the programming concepts and techniques covered in the chapter.
- **Key terms**. Each chapter lists key terms and their definitions; the list appears in the order that the terms are encountered in the chapter. A glossary at the end of the book lists all the key terms in alphabetical order, along with their working definitions.

MindTap

MindTap is a personalized learning experience with relevant assignments that guide students in analyzing problems, applying what they have learned, and improving their thinking. MindTap allows instructors to measure skills and outcomes with ease.

For instructors: Personalized teaching becomes yours with a learning path that is built with key student objectives. You can control what students see and when they see it. You can use MindTap as-is, or match it to your syllabus by hiding, rearranging, or adding content.

For students: A unique learning path of relevant readings, multimedia, and activities is created to guide you through basic knowledge and comprehension of analysis and application.

For both: Better outcomes empower instructors and motivate students with analytics and reports that provide a snapshot of class progress, the time spent in the course, engagement levels, and completion rates.

The MindTap for Programming Logic and Design includes coding labs in C++, Java, and Python, study tools, videos, and interactive quizzing, all integrated into an eReader that includes the full content of the printed text.

Instructor Resources

The following teaching tools are available to the instructor for download through our Instructor Companion Site at *sso.cengage.com*.

- **Instructor's Manual**. The Instructor's Manual follows the text chapter by chapter to assist in planning and organizing an effective, engaging course. The manual includes learning objectives, chapter overviews, lecture notes, ideas for classroom activities, and abundant additional resources. A sample course syllabus is also available.
- **PowerPoint Presentations**. This text provides PowerPoint slides to accompany each chapter. Slides are included to guide classroom presentations, and can be made available to students for chapter review, or to print as classroom handouts.
- **Solutions**. Solutions to review questions and exercises are provided to assist with grading.
- **Test Bank®**. Cengage Learning Testing Powered by Cognero is a flexible, online system that allows you to:
  - author, edit, and manage test bank content from multiple Cengage Learning solutions,
  - create multiple test versions in an instant, and
  - deliver tests from your LMS, your classroom, or anywhere you want.

Additional Options

- **Visual Logic™ software**. Visual Logic is a simple but powerful tool for teaching programming logic and design without traditional high-level programming language syntax. Visual Logic also interprets and executes flowcharts, providing students with immediate and accurate feedback.

Acknowledgments

I would like to thank all of the people who helped to make this book a reality, especially Alyssa Pratt, Jennifer Feltri-George, Kristin McNary, Kate Mason, and all the other professionals at Cengage Learning who made this book possible. Thanks, too, to my husband, Geoff, and our daughters, Andrea and Audrey, for their support. This book, as were all its previous editions, is dedicated to them.

–Joyce Farrell

CHAPTER 1

An Overview of Computers and Programming

Upon completion of this chapter, you will be able to:

- Describe computer systems
- Understand simple program logic
- List the steps involved in the program development cycle
- Write pseudocode statements and draw flowchart symbols
- Use a sentinel value to end a program
- Understand programming and user environments
- Describe the evolution of programming models

Understanding Computer Systems

A **computer system** is a combination of all the components required to process and store data using a computer. Every computer system is composed of multiple pieces of hardware and software.

- **Hardware** is the equipment, or the physical devices, associated with a computer. For example, keyboards, mice, speakers, and printers are all hardware. The devices are manufactured differently for computers of varying sizes—for example, large mainframes, laptops, and very small devices embedded into products such as telephones, cars, and thermostats. The types of operations performed by different-sized computers, however, are very similar. Computer hardware needs instructions that control its operations, including how and when data items are input, how they are processed, and the form in which they are output or stored.
- **Software** is computer instructions that tells the hardware what to do. Software is **programs**, which are instruction sets written by programmers. You can buy prewritten programs that are stored on a disk or that you download from the Web. For example, businesses use word-processing and accounting programs, and casual computer users enjoy programs that play music and games. Alternatively, you can write your own programs. When you write software instructions, you are **programming**. This book focuses on the programming process.

Software can be classified into two broad types:

- **Application software** comprises all the programs you apply to a task, such as word-processing programs, spreadsheets, payroll and inventory programs, and games. When you hear people say they have "downloaded an **app** onto a mobile device," they are simply using an abbreviation of *application software*.
- **System software** comprises the programs that you use to manage your computer, including operating systems such as Windows, Linux, or UNIX for larger computers and Google Android and Apple iOS for smartphones.

This book focuses on the logic used to write application software programs, although many of the concepts apply to both types of software.

Together, computer hardware and software accomplish three major operations in most programs:

- **Input**: Data items enter the computer system and are placed in memory, where they can be processed. **Data items** include all the text, numbers, and other raw material that are entered into and processed by a computer. Hardware devices that perform input operations include keyboards and mice. In business, many of the data items used are facts and figures about such entities as products, customers, and personnel. Data, however, also can include items such as images, sounds, and a user's mouse or finger-swiping movements.
- **Processing**: Processing data items may involve organizing or sorting them, checking them for accuracy, or performing calculations with them. The hardware component that performs these types of tasks is the **central processing unit**, or **CPU**. Some devices, such as tablets and smartphones, usually contain multiple processors, and efficiently using several CPUs requires special programming techniques.

- **Output**: After data items have been processed, the resulting information usually is sent to a printer, monitor, or some other output device so people can view, interpret, and use the results. Programming professionals often use the term *data* for input items, but use the term **information** for data items that have been processed and output. Sometimes you place output on **storage devices**, such as your hard drive, flash media, or a cloud-based device. (The **cloud** refers to devices at remote locations accessed through the Internet.) People cannot read data directly from these storage devices, but the devices hold information for later retrieval. When you send output to a storage device, sometimes it is used later as input for another program.

You write computer instructions in a computer **programming language** such as Visual Basic, C#, C++, or Java. Just as some people speak English and others speak Japanese, programmers write programs in different languages. Some programmers work exclusively in one language, whereas others know several and use the one that is best suited to the task at hand.

The instructions you write using a programming language are called **program code**; when you write instructions, you are **coding the program**.

Every programming language has rules governing its word usage and punctuation. These rules are called the language's **syntax**. Mistakes in a language's usage are **syntax errors**. If you ask, "How the geet too store do I?" in English, most people can figure out what you probably mean, even though you have not used proper English syntax—you have mixed up the word order, misspelled a word, and used an incorrect word. However, computers are not nearly as smart as most people; in this case, you might as well have asked the computer, "Xpu mxv ort dod nmcad bf B?" Unless the syntax is perfect, the computer cannot interpret the programming language instruction at all.

Figure 1-1 shows how the statement that displays the word *Hello* on a single line on a computer monitor looks in some common programming languages. Notice that the syntax of some languages require that a statement start with an uppercase letter, while the syntax of others does not. Notice that some languages end statements with a semicolon, some with a period, and some with no ending punctuation at all. Also notice that different verbs are used to mean *display*, and that some are spelled like their like English word counterparts, while others like *cout* and *System.out.println* are not regular English words. The different formats you see are just a hint of the various syntaxes used by languages.

Language	Statement that displays Hello on a single line
Java	`System.out.println("Hello);`
C++	`cout << "Hello" << endl;`
Visual Basic	`Console.WriteLine("Hello");`
Python	`print "Hello"`
COBOL	`DISPLAY "Hello".`

Figure 1-1 Displaying the word *Hello* in some common programming languages



After you learn French, you automatically know, or can easily figure out, many Spanish words. Similarly, after you learn one programming language, it is much easier to understand other languages.

When you write a program, you usually type its instructions using a keyboard. When you type program instructions, they are stored in **computer memory**, which is a computer's temporary, internal storage. **Random access memory**, or **RAM**, is a form of internal, volatile memory. Programs that are running and data items that are being used are stored in RAM for quick access. Internal storage is **volatile**—its contents are lost when the computer is turned off or loses power. Usually, you want to be able to retrieve and perhaps modify the stored instructions later, so you also store them on a permanent storage device, such as a disk. Permanent storage devices are **nonvolatile**—that is, their contents are persistent and are retained even when power is lost. If you have had a power loss while working on a computer, but were able to recover your work when power was restored, it's not because the work was still in RAM. Your system has been configured to automatically save your work at regular intervals on a nonvolatile storage device—often your hard drive.

After a computer program is typed using programming language statements and stored in memory, it must be translated to **machine language** that represents the millions of on/off circuits within the computer. Your programming language statements are called **source code**, and the translated machine language statements are **object code.**

Each programming language uses a piece of software, called a **compiler** or an **interpreter**, to translate your source code into machine language. Machine language also is called **binary language**, and is represented as a series of 0s and 1s. The compiler or interpreter that translates your code tells you if any programming language component has been used incorrectly. Syntax errors are relatively easy to locate and correct because your compiler or interpreter highlights them. If you write a computer program using a language such as C++, but spell one of its words incorrectly or reverse the proper order of two words, the software lets you know that it found a mistake by displaying an error message as soon as you try to translate the program.



Although there are differences in how compilers and interpreters work, their basic function is the same—to translate your programming statements into code the computer can use. When you use a compiler, an entire program is translated before it can execute; when you use an interpreter, each instruction is translated just prior to execution. Usually, you do not choose which type of translation to use—it depends on the programming language. However, some languages can use both compilers and interpreters.

After a program's source code is translated successfully to machine language, the computer can carry out the program instructions. When instructions are carried out, a program **runs**, or **executes**. In a typical program, some input will be accepted, some processing will occur, and results will be output.

Besides the popular, comprehensive programming languages such as Java and C++, many programmers use **scripting languages** (also called *scripting programming languages* or *script languages*) such as Python, Lua, Perl, and PHP. Scripts written in these languages usually can be typed directly from a keyboard and are stored as text rather than as binary executable files. Scripting language programs are interpreted line by line each time the program executes, instead of being stored in a compiled (binary) form. Still, with all programming languages, each instruction must be translated to machine language before it can execute.

TWO TRUTHS & A LIE

Understanding Computer Systems

In each Two Truths & a Lie section, two of the numbered statements are true, and one is false. Identify the false statement and explain why it is false.

1. Hardware is the equipment, or the devices, associated with a computer. Software is computer instructions.
2. The grammar rules of a computer programming language are its syntax.
3. You write programs using machine language, and translation software converts the statements to a programming language.

The false statement is #3. You write programs using a programming language such as Visual Basic or Java, and a translation program (called a compiler or interpreter) converts the statements to machine language, which is 0s and 1s.

Understanding Simple Program Logic

For a program to work properly, you must develop correct **logic**; that is, you must write program instructions in a specific sequence, you must not leave out any instructions, and you must not add extraneous instructions. A program with syntax errors cannot be translated fully and cannot execute. A program with no syntax errors is translatable and can execute, but it still might contain **logical errors** and produce incorrect output as a result.

Suppose you instruct someone to make a cake as follows:

```
Get a bowl
Stir
Add two eggs
Add a gallon of gasoline
Bake at 350 degrees for 45 minutes
Add three cups of flour
```

Don't Do It
Don't bake a cake like this!



The dangerous cake-baking instructions are shown with a Don't Do It icon. You will see this icon when the book contains an unrecommended programming practice that is used as an example of what *not* to do.

Even though the cake-baking instructions use English language syntax correctly, the instructions are out of sequence, some are missing, and some instructions belong to procedures other than baking a cake. If you follow these instructions, you will not make an edible cake, and you may end up with a disaster. Many logical errors are more difficult to locate than syntax errors—it is easier for you to determine whether *eggs* is spelled incorrectly in a recipe than it is for you to tell if there are too many eggs or if they are added too soon.

Most simple computer programs include steps that perform input, processing, and output. Suppose you want to write a computer program to double any number you provide. You can write the program in a programming language such as Visual Basic or Java, but if you were to write it using English-like statements, it would look like this:

```
input myNumber
myAnswer = myNumber * 2
output myAnswer
```

The number-doubling process includes three instructions:

- The instruction to `input myNumber` is an example of an input operation. When the computer interprets this instruction, it knows to look to an input device to obtain a number. When you work in a specific programming language, you write instructions that tell the computer which device to access for input. For example, when a user enters a number as data for a program, the user might click on the number with a mouse, type it from a keyboard, or speak it into a microphone. Logically, however, it doesn't matter which hardware device is used, as long as the computer knows to accept a number. When the number is retrieved from an input device, it is placed in the computer's memory in a variable named `myNumber`. A **variable** is a named memory location whose value can vary—that is, hold different values at different points in time. For example, the value of `myNumber` might be 3 when the program is used for the first time and 45 when it is used the next time. In this book, variable names will not contain embedded spaces; for example, the book will use `myNumber` instead of `my Number`.



From a logical perspective, when you input, process, or output a value, the hardware device is irrelevant. The same is true in your daily life. If you follow the instruction "Get eggs for the cake," it does not really matter if you purchase them from a store or harvest them from your own chickens—you get the eggs either way. There might be different practical considerations to getting the eggs, just as there are for getting data from a large database as opposed to getting data from an inexperienced user working at home on a laptop computer. For now, this book is concerned only with the logic of operations, not the minor details.



A college classroom is similar to a named variable in that its name (perhaps 204 Adams Building) can hold different contents at different times. For example, your Logic class might meet there on Monday night, and a math class might meet there on Tuesday morning.

- The instruction `myAnswer = myNumber * 2` is an example of a processing operation. In most programming languages, an asterisk is used to indicate multiplication, so this instruction means "Change the value of the memory location `myAnswer` to equal the value at the memory location `myNumber` times two." Mathematical operations are not the only kind of processing operations, but they are very typical. As with input operations, the type of hardware used for processing is irrelevant—after you write a program, it can be used on computers of different brand names, sizes, and speeds.
- In the number-doubling program, the `output myAnswer` instruction is an example of an output operation. Within a particular program, this statement could cause the output to appear on the monitor (which might be a flat-panel plasma screen or a smartphone display), or the output could go to a printer (which could be laser or ink-jet), or the output could be written to a disk or DVD. The logic of the output process is the same no matter what hardware device you use. When this instruction executes, the value stored in memory at the location named `myAnswer` is sent to an output device. (The output value also remains in computer memory until something else is stored at the same memory location or power is lost.)



Watch the video *A Simple Program*.

Syntax errors: These are errors in the code structure (like missing semicolons or incorrect use of keywords). Programs with syntax errors cannot be executed until those errors are fixed.



Computer memory consists of millions of numbered locations where data can be stored. The memory location of `myNumber` has a specific numeric address, but when you write programs, you seldom need to be concerned with the value of the memory address; instead, you use the easy-to-remember name you created. Computer programmers often refer to memory addresses using hexadecimal notation, or base 16. Using this system, they might use a value like 42FF01A to refer to a memory address. Despite the use of letters, such an address is still a number. Appendix A contains information about the hexadecimal numbering system.

Logic errors or runtime errors: These are errors where the program runs but produces incorrect results because the logic of the code is wrong. For example, using the wrong formula in a calculation. In this case, the program can execute but might produce incorrect results.

TWO TRUTHS & A LIE

Understanding Simple Program Logic

1. A program with syntax errors can execute but might produce incorrect results.
2. Although the syntax of programming languages differs, the same program logic can be expressed in different languages.
3. Most simple computer programs include steps that perform input, processing, and output.

The false statement is #1. A program with syntax errors cannot execute; a program with no syntax errors can execute, but might produce incorrect results.

Understanding the Program Development Cycle

A programmer's job involves writing instructions (such as those in the doubling program in the preceding section), but a professional programmer usually does not just sit down at a computer keyboard and start typing. Figure 1-2 illustrates the **program development cycle,** which can be broken down into at least seven steps:

1. Understand the problem.
2. Plan the logic.
3. Code the program.
4. Use software (a compiler or interpreter) to translate the program into machine language.
5. Test the program.
6. Put the program into production.
7. Maintain the program.

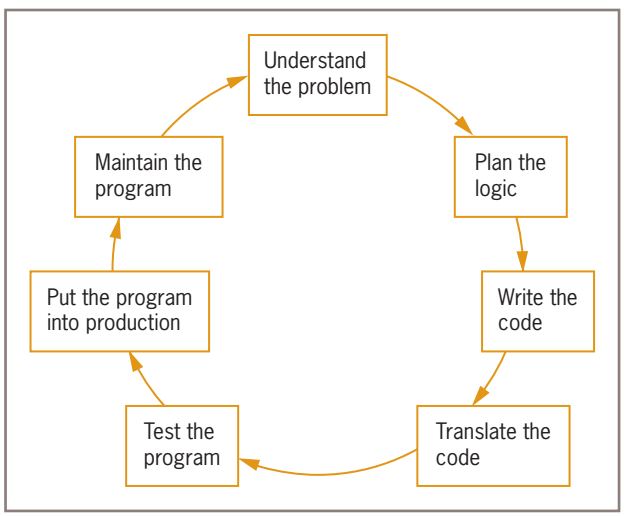


Figure 1-2 The program development cycle

Understanding the Problem

Professional computer programmers write programs to satisfy the needs of others, called **users** or **end users**. Examples of end users include a Human Resources department that needs a printed list of all employees, a Billing department that wants a list of clients who are 30 or more days overdue on their payments, and an Order department that needs a website to provide buyers with an online shopping cart. Because programmers are providing a service to these users, programmers must first understand what the users

want. When a program runs, you usually think of the logic as a cycle of input-processing-output operations, but when you plan a program, you think of the output first. After you understand what the desired result is, you can plan the input and processing steps to achieve it.

Suppose the director of Human Resources says to a programmer, "Our department needs a list of all employees who have been here over five years, because we want to invite them to a special thank-you dinner." On the surface, this seems like a simple request. An experienced programmer, however, will know that the request is incomplete. For example, you might not know the answers to the following questions about which employees to include:

- Does the director want a list of full-time employees only, or a list of full- and part-time employees together?
- Does she want to include people who have worked for the company on a month-to-month contractual basis over the past five years, or only regular, permanent employees?
- Do the listed employees need to have worked for the organization for five years as of today, as of the date of the dinner, or as of some other cutoff date?
- What about an employee who worked three years, took a two-year leave of absence, and has been back for three years?

The programmer cannot make any of these decisions; the user (in this case, the human resources director) must address these questions.

More decisions still might be required. For example:

- What data should be included for each listed employee? Should the list contain both first and last names? Social Security numbers? Phone numbers? Addresses?
- Should the list be in alphabetical order? Employee ID number order? Length-of-service order? Some other order?
- Should the employees be grouped by any criteria, such as department number or years of service?

Several pieces of documentation often are provided to help the programmer understand the problem. **Documentation** consists of all the supporting paperwork for a program; it might include items such as original requests for the program from users, sample output, and descriptions of the data items available for input.

Understanding the problem might be even more difficult if you are writing an app that you hope to market for mobile devices. Business developers usually are approached by a user with a need, but successful developers of mobile apps often try to identify needs that users aren't even aware of yet. For example, no one knew they wanted to play *Angry Birds* or leave messages on Facebook before those applications were developed. Mobile app developers also must consider a wider variety of user skills than programmers who develop applications that are used internally in a corporation. Mobile app developers must make sure their programs work with a range of screen sizes and hardware specifications because software competition is intense and the hardware changes quickly.

Fully understanding the problem may be one of the most difficult aspects of programming. On any job, the description of what the user needs may be vague—worse yet, users may not really know what they want, and users who think they know frequently change their minds after seeing sample output. A good programmer is often part counselor, part detective!



Watch the video *The Program Development Cycle, Part 1*.

Planning the Logic

The heart of the programming process lies in planning the program's logic. During this phase of the process, the programmer plans the steps of the program, deciding what steps to include and how to order them. You can plan the solution to a problem in many ways. Two common planning tools are flowcharts and pseudocode; you will work with many examples of flowcharts and pseudocode throughout this book. Both tools involve writing the steps of the program in English, much as you would plan a trip on paper before getting into the car or plan a party theme before shopping for food and favors.

You may hear programmers refer to planning a program as "developing an algorithm." An **algorithm** is the sequence of steps or rules you follow to solve a problem.

The programmer shouldn't worry about the syntax of any particular language during the planning stage, but should focus on figuring out what sequence of events will lead from the available input to the desired output. Planning the logic includes thinking carefully about all the possible data values a program might encounter and how you want the program to handle each scenario. The process of walking through a program's logic on paper before you actually write the program is called **desk-checking**. You will learn more about planning the logic throughout this book; in fact, the book focuses on this crucial step almost exclusively.

Coding the Program

The programmer can write the source code for a program only after the logic is developed. Hundreds of programming languages are available. Programmers choose particular languages because some have built-in capabilities that make them more efficient than others at handling certain types of operations. Despite their differences, programming languages are quite alike in their basic capabilities—each can handle input operations, arithmetic processing, output operations, and other standard functions. The logic developed to solve a programming problem can be executed using any number of languages. Only after choosing a language must the programmer be concerned with proper punctuation and the correct spelling of commands—in other words, using the correct *syntax*.

Some experienced programmers can successfully combine logic planning and program coding in one step. This may work for planning and writing a very simple program, just

as you can plan and write a postcard to a friend using one step. A good term paper or a Hollywood screenplay, however, needs planning before writing—and so do most programs.

Which step is harder: planning the logic or coding the program? Right now, it may seem to you that writing in a programming language is a very difficult task, considering all the spelling and syntax rules you must learn. However, the planning step is actually more difficult. Which is more difficult: thinking up the twists and turns to the plot of a best-selling mystery novel, or writing a translation of an existing novel from English to Spanish? And who do you think gets paid more, the writer who creates the plot or the translator? (Try asking friends to name any famous translator!)

Using Software to Translate the Program into Machine Language

Even though there are many programming languages, each computer knows only one language—its machine language, which consists of 1s and 0s. Computers understand machine language because they are made up of thousands of tiny electrical switches, each of which can be set in either the on state or the off state, which are represented by a 1 or 0, respectively.

Languages such as Java or Visual Basic are available for programmers because someone has written a translator program (a compiler or interpreter) that changes the programmer's English-like **high-level programming language** into the **low-level machine language** that the computer understands. When you learn the syntax of a programming language, the commands work on any machine on which the language software has been installed. Your commands, however, then are translated to machine language, which differs in various computer makes and models.

If you write a programming statement incorrectly (for example, by misspelling a word, using a word that doesn't exist in the language, or using "illegal" grammar), the translator program doesn't know how to proceed and issues an error message identifying a syntax error. Although making errors is never desirable, syntax errors are not a programmer's deepest concern, because the compiler or interpreter catches every syntax error and displays a message that notifies you about the problem. The computer will not execute a program that contains even one syntax error.

Typically, a programmer develops logic, writes the code, and compiles the program, receiving a list of syntax errors. The programmer then corrects the syntax errors and compiles the program again. Correcting the first set of errors frequently reveals new errors that originally were not apparent to the compiler. For example, if you could use an English compiler and submit the sentence *The dg chase the cat*, the compiler at first might point out only one syntax error. The second word, *dg*, is illegal because it is not part of the English language. Only after you corrected the word to *dog* would the compiler find another syntax error on the third word, *chase*, because it is the wrong verb form for the subject *dog*. This doesn't mean *chase* is necessarily the wrong word. Maybe *dog* is wrong; perhaps the subject should be *dogs*, in which case *chase* is right. Compilers don't always know exactly what you mean, nor do they know what the proper correction should be, but they do know when something is wrong with your syntax.



Watch the video *The Program Development Cycle, Part 2*.

Programmers often compile their code one section at a time. It is far less overwhelming and easier to understand errors that are discovered in 20 lines of code at a time than to try to correct mistakes after 2,000 lines of code have been written. When writing a program, a programmer might need to recompile the code several times. An executable program is created only when the code is free of syntax errors. After a program has been translated into machine language, the machine language program is saved and can be run any number of times without repeating the translation step. You need to retranslate your code only if you make changes to your source code statements. Figure 1-3 shows a diagram of this entire process.

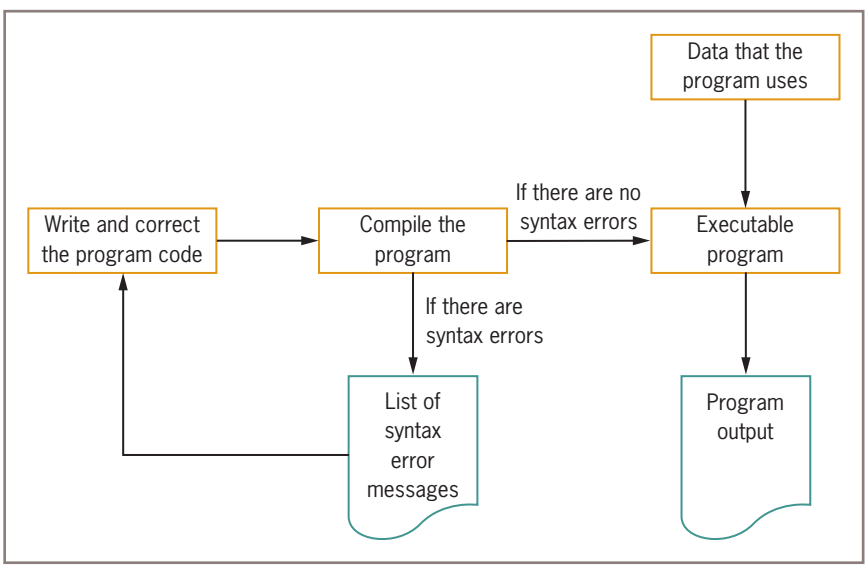


Figure 1-3 Creating an executable program

Testing the Program

A program that is free of syntax errors is not necessarily free of logical errors. A logical error results when you use a syntactically correct statement but use the wrong one for the current context. For example, the English sentence *The dog chases the cat*, although syntactically perfect, is not logically correct if the dog chases a ball or the cat is the aggressor.

After a program is free of syntax errors, the programmer can test it—that is, execute it with some sample data to see whether the results are logically correct. Recall the number-doubling program:

```
input myNumber
myAnswer = myNumber * 2
output myAnswer
```

If you execute the program, provide the value 2 as input to the program, and the answer 4 is displayed, you have executed one successful test run of the program. However, if the answer 40 is displayed, maybe the program contains a logical error. Maybe the second line of code was mistyped with an extra zero, so that the program reads:

```
input myNumber
myAnswer = myNumber * 20
output myAnswer
```

Don't Do It
The programmer typed *20* instead of *2*.

Placing 20 instead of 2 in the multiplication statement caused a logical error. Notice that nothing is syntactically wrong with this second program—it is just as reasonable to multiply a number by 20 as by 2—but if the programmer intends only to double `myNumber`, then a logical error has occurred.

The process of finding and correcting program errors is called **debugging**. You debug a program by testing it using many sets of data. For example, if you write the program to double a number, then enter 2 and get an output value of 4, that doesn't necessarily mean you have a correct program. Perhaps you have typed this program by mistake:

```
input myNumber
myAnswer = myNumber + 2
output myAnswer
```

Don't Do It
The programmer typed "+" instead of "*".

An input of 2 results in an answer of 4, but that doesn't mean your program doubles numbers—it actually only adds 2 to them. If you test your program with additional data and get the wrong answer—for example, if you enter 7 and get an answer of 9—you know there is a problem with your code.

Selecting test data is somewhat of an art in itself, and it should be done carefully. If the Human Resources department wants a list of the names of five-year employees, it would be a mistake to test the program with a small sample file of only long-term employees. If no newer employees are part of the data being used for testing, you do not really know if the program would have eliminated them from the five-year list. Many companies do not know that their software has a problem until an unusual circumstance occurs—for example, the first time an employee has more than nine dependents, the first time a customer orders more than 999 items at a time, or when the Internet runs out of allocated IP addresses, a problem known as *IPV4 exhaustion*.

Putting the Program into Production

After the program is thoroughly tested and debugged, it is ready for the organization to use. Putting the program into production might mean simply running the program once, if it was written to satisfy a user's request for a special list. However, the process might

take months if the program will be run on a regular basis, or if it is one of a large system of programs being developed. Perhaps data-entry people must be trained to prepare the input for the new program, users must be trained to understand the output, or existing data in the company must be changed to an entirely new format to accommodate this program. **Conversion**, the entire set of actions an organization must take to switch over to using a new program or set of programs, can sometimes take months or years to accomplish.

Maintaining the Program

After programs are put into production, making necessary changes is called **maintenance**. Maintenance can be required for many reasons: As examples, new tax rates are legislated, the format of an input file is altered, or the end user requires additional information not included in the original output specifications. Frequently, your first programming job will require maintaining previously written programs. When you maintain the programs others have written, you will appreciate the effort the original programmer put into writing clear code, using reasonable variable names, and documenting his or her work. When you make changes to existing programs, you repeat the development cycle. That is, you must understand the changes, then plan, code, translate, and test them before putting them into production. If a substantial number of program changes are required, the original program might be retired, and the program development cycle might be started for a new program.



Watch the video *The Program Development Cycle, Part 3*.

TWO TRUTHS & A LIE

Understanding the Program Development Cycle

1. Understanding the problem that must be solved can be one of the most difficult aspects of programming.
2. The two most commonly used logic-planning tools are flowcharts and pseudocode.
3. Flowcharting a program is a very different process if you use an older programming language instead of a newer one.

The false statement is #3. Despite their differences, programming languages are quite alike in their basic capabilities—each can handle input operations, arithmetic processing, output operations, and other standard functions. The logic developed to solve a programming problem can be executed using any number of languages.

Using Pseudocode Statements and Flowchart Symbols

When programmers plan the logic for a solution to a programming problem, they often use one of two tools: pseudocode (pronounced *sue-doe-code*) or flowcharts.

- **Pseudocode** is an English-like representation of the logical steps it takes to solve a problem. *Pseudo* is a prefix that means *false*, and to *code* a program means to put it in a programming language; therefore, *pseudocode* simply means *false code*, or sentences that appear to have been written in a computer programming language but do not necessarily follow all the syntax rules of any specific language.
- A **flowchart** is a pictorial representation of the same logical steps.

Writing Pseudocode

You already have seen examples of statements that represent pseudocode earlier in this chapter, and there is nothing mysterious about them. The following five statements constitute a pseudocode representation of a number-doubling problem:

```
start
  input myNumber
  myAnswer = myNumber * 2
  output myAnswer
stop
```

Using pseudocode involves writing down all the steps you will use in a program. Usually, programmers preface their pseudocode with a beginning statement like `start` and end it with a terminating statement like `stop`. The statements between `start` and `stop` look like English and are indented slightly so that `start` and `stop` stand out. Most programmers do not bother with punctuation such as periods at the end of pseudocode statements, although it would not be wrong to use them if you prefer that style. Similarly, there is no need to capitalize the first word in a statement, although you might choose to do so.

Pseudocode is fairly flexible because it is a planning tool, and not the final product. Therefore, for example, you might prefer any of the following:

- Instead of `start` and `stop`, some pseudocode developers would use other terms such as `begin` and `end`.
- Instead of writing `input myNumber`, some developers would write `get myNumber` or `read myNumber`.
- Instead of writing `myAnswer = myNumber * 2`, some developers would write `calculate myAnswer = myNumber times 2` or `myAnswer is assigned myNumber doubled`.
- Instead of writing `output myAnswer`, many pseudocode developers would write `display myAnswer`, `print myAnswer`, or `write myAnswer`.

The point is, the pseudocode statements are instructions to retrieve an original number from an input device and store it in memory where it can be used in a calculation, and then to get the calculated answer from memory and send it to an output device so a person can see it. When you eventually convert your pseudocode to a specific programming language, you do not have such flexibility because specific syntax will be required. For example, if you use the C# programming language and write the statement to output the answer to the monitor, you can code the following:

```
Console.Write(myAnswer);
```

The exact use of words, capitalization, and punctuation are important in the C# statement, but not in the pseudocode statement. Quick Reference 1-1 summarizes the pseudocode standards used in this book. (Note that the Quick Reference mentions modules; you will learn about modules in Chapter 2. Additional pseudocode style features will be discussed as topics are introduced throughout this book.)

QUICK REFERENCE 1-1 Pseudocode Standards

- Programs begin with `start` and `end` with stop; these two words are always aligned.
- Whenever a module name is used, it is followed by a set of parentheses.
- Modules begin with the module name and end with return. The module name and `return` are always aligned.
- Each program statement performs one action—for example, input, processing, or output.
- Program statements are indented a few spaces more than `start` or the module name.
- Each program statement appears on a single line if possible. When this is not possible, continuation lines are indented.
- Program statements begin with lowercase letters.
- No punctuation is used to end statements.



As you learn to create pseudocode and flowchart statements, you will develop a sense for how much detail to include. The statements represent the main steps that must be accomplished without including minute points. The concept is similar to writing an essay outline in which each statement of the outline represents a paragraph.

Drawing Flowcharts

Some professional programmers prefer writing pseudocode to drawing flowcharts, because using pseudocode is more similar to writing the final statements in the programming language. Others prefer drawing flowcharts to represent the logical flow, because flowcharts allow programmers to visualize more easily how the program statements will connect. Especially for beginning programmers, flowcharts are an excellent tool that helps them to visualize how the statements in a program are interrelated.

You can draw a flowchart by hand or use software, such as Microsoft Word and Microsoft PowerPoint, that contains flowcharting tools. You can use several other software programs, such as Visio and Visual Logic, specifically to create flowcharts. When you create a flowchart, you draw geometric shapes that contain the individual statements and that are connected with arrows. You use a parallelogram to represent an **input symbol**, which indicates an input operation. You write an input statement in English inside the parallelogram, as shown in Figure 1-4.

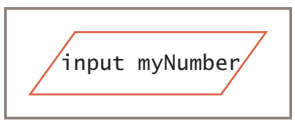


Figure 1-4 Input symbol

Arithmetic operation statements are examples of processing. In a flowchart, you use a rectangle as the **processing symbol** that contains a processing statement, as shown in Figure 1-5.

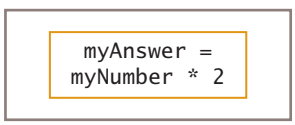


Figure 1-5 Processing symbol

To represent an output statement, you use the same symbol as for input statements—the **output symbol** is a parallelogram, as shown in Figure 1-6. Because the parallelogram is used for both input and output, it is often called the **input/output symbol** or **I/O symbol**.

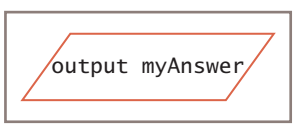


Figure 1-6 Output symbol



Some software programs that use flowcharts (such as Visual Logic) use a right-slanting parallelogram to represent input and a left-slanting parallelogram to represent output. As long as the flowchart creator and the flowchart reader are communicating, the actual shape used is irrelevant. This book will follow the most standard convention of using the right-slanting parallelogram for both input and output.

To show the correct sequence of statements, you use arrows, or **flowlines**, to connect the steps. Whenever possible, most of a flowchart should read from top to bottom or from left to right on a page. That's the way we read English, so when flowcharts follow this convention, they are easier for us to understand.

To be complete, a flowchart should include two more elements: **terminal symbols**, or start/stop symbols, at each end. Often, you place a word like `start` or `begin` in the first terminal symbol and a word like `end` or `stop` in the other. The standard terminal symbol is shaped like a racetrack; many programmers refer to this shape as a lozenge, because it resembles the shape of the medication you might use to soothe a sore throat. Figure 1-7 shows a complete flowchart for the program that doubles a number, and the pseudocode for the same problem. You can see from the figure that the flowchart and pseudocode statements are the same—only the presentation format differs.

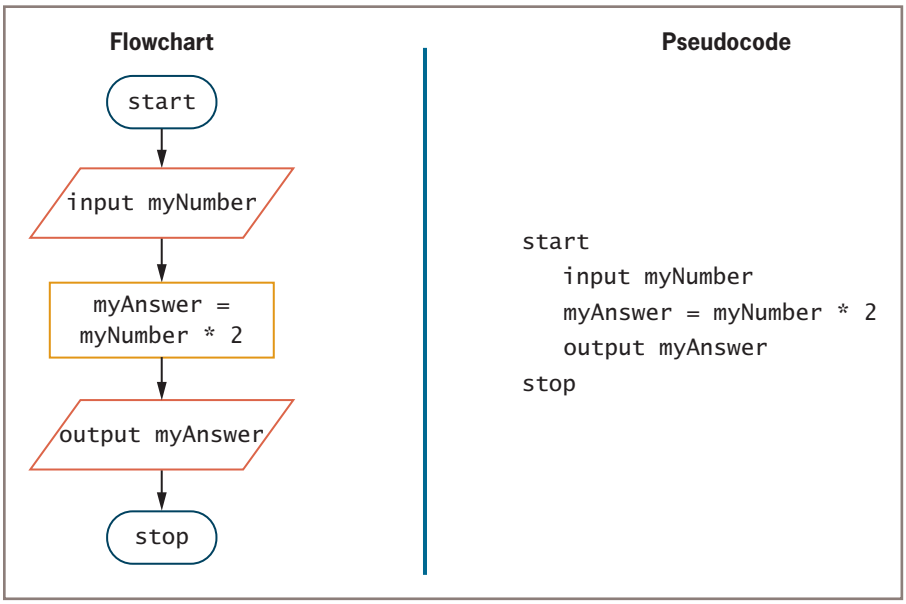
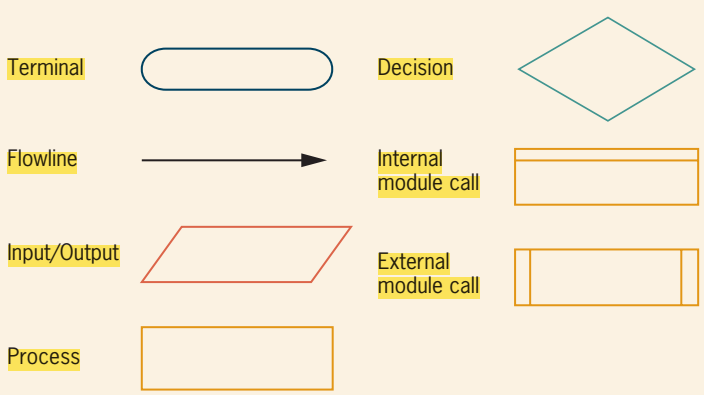


Figure 1-7 Flowchart and pseudocode of program that doubles a number

Programmers seldom create both pseudocode and a flowchart for the same problem; you usually use one or the other. In a large program, you might even prefer to write pseudocode for some parts and to draw a flowchart for others. When you tell a friend how to get to your house, you might write a series of instructions or you might draw a map. Pseudocode is similar to written, step-by-step instructions; a flowchart, like a map, is a visual representation of the same thing. Quick Reference 1-2 summarizes the flowchart symbols used in this book.

QUICK REFERENCE 1-2 Flowchart Symbols



Repeating Instructions

After the flowchart or pseudocode has been developed, the programmer needs only to: (1) buy a computer, (2) buy a language compiler, (3) learn a programming language, (4) code the program, (5) attempt to compile it, (6) fix the syntax errors, (7) compile it again, (8) test it with several sets of data, and (9) put it into production.

"Whoa!" you are probably saying to yourself. "This is simply not worth it! All that work to create a flowchart or pseudocode, and *then* all those other steps? For five dollars, I can buy a pocket calculator that will double any number for me instantly!" You are absolutely right. If this were a real computer program, and all it did was double the value of a number, it would not be worth the effort. Writing a computer program would be worthwhile only if you had many numbers (let's say 10,000) to double in a limited amount of time—let's say the next two minutes.

Unfortunately, the program represented in Figure 1-7 does not double 10,000 numbers; it doubles only one. You could execute the program 10,000 times, of course, but that would require you to sit at the computer and run the program over and over again. You would be better off with a program that could process 10,000 numbers, one after the other.

One solution is to write the program shown in Figure 1-8 and execute the same steps 10,000 times. Of course, writing this program would be very time consuming; you might as well buy the calculator.

```
start
  input myNumber
  myAnswer = myNumber * 2
  output myAnswer
  input myNumber
  myAnswer = myNumber * 2
  output myAnswer
  input myNumber
  myAnswer = myNumber * 2
  output myAnswer
  …and so on for 9,997 more times
```

Don't Do It
You would never want to write such a repetitious list of instructions.

Figure 1-8 Inefficient pseudocode for program that doubles 10,000 numbers

A better solution is to have the computer execute the same set of three instructions over and over again, as shown in Figure 1-9. The repetition of a series of steps is called a **loop**. With this approach, the computer gets a number, doubles it, displays the answer, and then starts again with the first instruction. The same spot in memory, called `myNumber`, is reused for the second number and for any subsequent numbers. The spot in memory named `myAnswer` is reused each time to store the result of the multiplication operation. However, the logic illustrated in the flowchart in Figure 1-9 contains a major problem—the sequence of instructions never ends. This programming situation is known as an **infinite loop**—a repeating flow of logic with no end. You will learn one way to handle this problem later in this chapter; you will learn a superior way in Chapter 3.

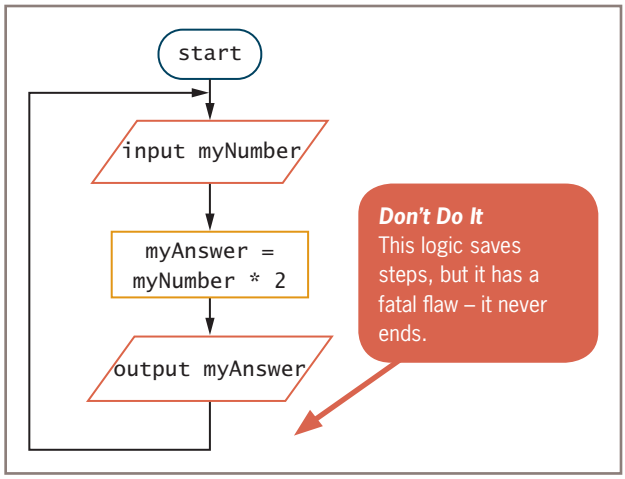


Figure 1-9 Flowchart of infinite number-doubling program

TWO TRUTHS & A LIE

Using Pseudocode Statements and Flowchart Symbols

1. When you draw a flowchart, you use a parallelogram to represent an input operation.
2. When you draw a flowchart, you use a parallelogram to represent a processing operation.
3. When you draw a flowchart, you use a parallelogram to represent an output operation.

The false statement is #2. When you draw a flowchart, you use a rectangle to represent a processing operation.

Using a Sentinel Value to End a Program

The logic in the flowchart for doubling numbers, shown in Figure 1-9, has a major flaw—the program contains an infinite loop. If, for example, the input numbers are being entered at the keyboard, the program will keep accepting numbers and outputting their doubled values forever. Of course, the user could refuse to type any more numbers. But the program cannot progress any further while it is waiting for input; meanwhile, the program is

occupying computer memory and tying up operating system resources. Refusing to enter any more numbers is not a practical solution. Another way to end the program is simply to turn off the computer. But again, that's neither the best solution nor an elegant way for the program to end.

A better way to end the program is to set a predetermined value for `myNumber` that means *Stop the program!* For example, the programmer and the user could agree that the user will never need to know the double of 0 (zero), so the user could enter a 0 to stop. The program then could test any incoming value contained in `myNumber` and, if it is a 0, stop the program.

In a flowchart, you represent testing a value or evaluating an expression by drawing a **decision symbol**, which is shaped like a diamond. The diamond usually contains a question or evaluation, the answer to which is one of two mutually exclusive options. All good computer questions have only two mutually exclusive answers, such as `yes` and `no` or `true` and `false`. For example, "What day of the year is your birthday?" is not a good computer question because there are 366 possible answers. However, "Is your birthday June 24?" is a good computer question because the answer is always either yes or no.

The question to stop the doubling program should be, "Is the value of myNumber just entered equal to 0?" or "`myNumber` = 0?" for short. The complete flowchart will now look like the one shown in Figure 1-10.

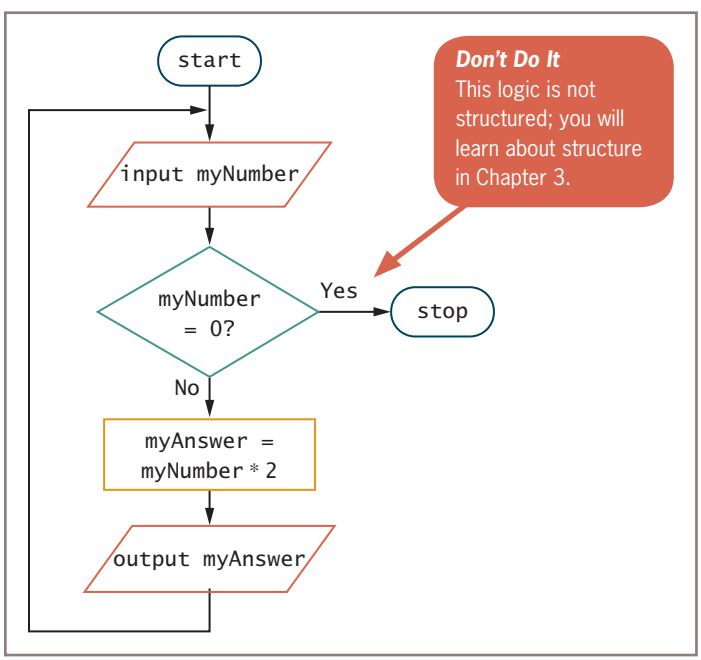


Figure 1-10 Flowchart of number-doubling program with sentinel value of 0

One drawback to using 0 to stop a program, of course, is that it won't work if the user does need to find the double of 0. In that case, some other data-entry value that the user never will need, such as 999 or –1, could be selected to signal that the program should end. A preselected value that stops the execution of a program is often called a **dummy value** because it does not represent real data, but just a signal to stop. Sometimes, such a value is called a **sentinel value** because it represents an entry or exit point, like a sentinel who guards a fortress.

Not all programs rely on user data entry from a keyboard; many read data from an input device, such as a disk. When organizations store data on a disk or other storage device, they do not commonly use a dummy value to signal the end of the file. For one thing, an input record might have hundreds of fields, and if you store a dummy record in every file, you are wasting a large quantity of storage on "nondata." Additionally, it is often difficult to choose sentinel values for fields in a company's data files. Any `balanceDue`, even a zero or a negative number, can be a legitimate value, and any `customerName`, even *ZZ*, could be someone's name. Fortunately, programming languages can recognize the end of data in a file automatically, through a code that is stored at the end of the data. Many programming languages use the term **eof** (for *end of file*) to refer to this marker that automatically acts as a sentinel. This book, therefore, uses `eof` to indicate the end of data whenever using a dummy value is impractical or inconvenient. In the flowchart shown in Figure 1-11, the `eof` evaluation is shaded.

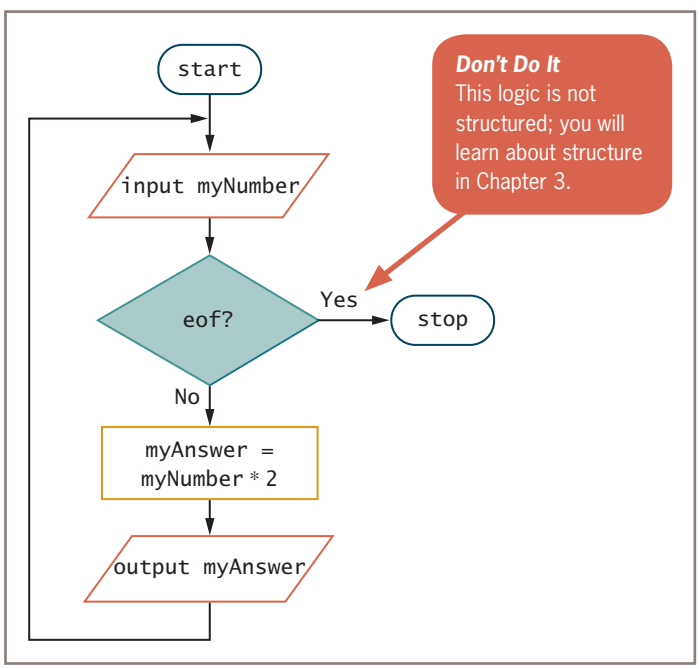


Figure 1-11 Flowchart using `eof`

TWO TRUTHS & A LIE

Using a Sentinel Value to End a Program

1. A program that contains an infinite loop is one that never ends.
2. A preselected value that stops the execution of a program is often called a dummy value or a sentinel value.
3. Many programming languages use the term `fe` (for *file end*) to refer to a marker that automatically acts as a sentinel.

The false statement is #3. The term eof (for *end of file*) is the common term for a file sentinel.

Understanding Programming and User Environments

Many approaches can be used to write and execute a computer program. When you plan a program's logic, you can use a flowchart, pseudocode, or a combination of the two. When you code the program, you can type statements into a variety of text editors. When your program executes, it might accept input from a keyboard, mouse, microphone, or any other input device, and when you provide a program's output, you might use text, images, or sound. This section describes the most common environments you will encounter as a new programmer.

Understanding Programming Environments

When you plan the logic for a computer program, you can use paper and pencil to create a flowchart, or you might use software that allows you to manipulate flowchart shapes. If you choose to write pseudocode, you can do so by hand or by using a word-processing program.

To enter the program into a computer so you can translate and execute it, you usually use a keyboard to type program statements into an editor. You can type a program into one of the following:

- A plain text editor
- A text editor that is part of an integrated development environment.

A **text editor** is a program that you use to create simple text files. It is similar to a word processor, but without as many features. You can use a text editor such as Notepad that is included with Microsoft Windows. Figure 1-12 shows a C# program in Notepad that accepts a number and doubles it. An advantage to using a simple text editor to type and save a program is that the completed program does not require much disk space for storage. For example, the file shown in Figure 1-12 occupies only 314 bytes of storage.

This line contains a prompt that tells the user what to enter. You will learn more about prompts in Chapter 2.

NumberDoublingProgram - Notepad

File Edit Format View Help

```
using static System.Console;
public class NumberDoublingProgram
{
   public static void Main()
   {
      int myNumber;
      int myAnswer;
      Write("Please enter a number >> ");
      myNumber = Convert.ToInt32(Console.ReadLine());
      myAnswer = myNumber * 2;
      WriteLine("\n{0} doubled is {1}", myNumber, myAnswer);
   }
}
```

Figure 1-12 A C# number-doubling program in Notepad

You can use the editor of an **integrated development environment (IDE)** to enter your program. An IDE is a software package that provides an editor, compiler, and other programming tools. For example, Figure 1-13 shows a C# program in the Microsoft Visual Studio IDE, an environment that contains tools useful for creating programs in Visual Basic, C++, and C#.

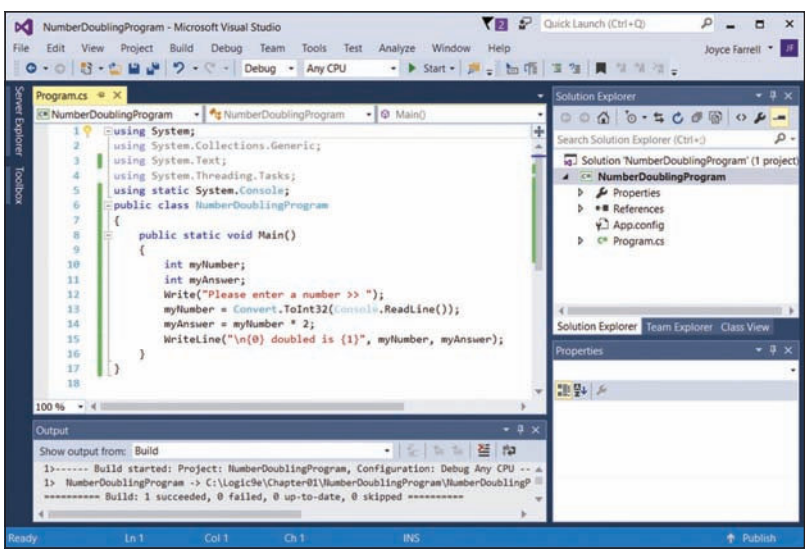


Figure 1-13 A C# number-doubling program in Visual Studio

Using an IDE is helpful to programmers because usually it provides features similar to those you find in many word processors. In particular, an IDE's editor commonly includes such features as the following:

- It uses different colors to display various language components, making elements like data types easier to identify.
- It highlights syntax errors visually for you.
- It employs automatic statement completion; when you start to type a statement, the IDE suggests a likely completion, which you can accept with a keystroke.
- It provides tools that allow you to step through a program's execution one statement at a time so you can more easily follow the program's logic and determine the source of any errors.

When you use the IDE to create and save a program, you occupy much more disk space than when using a plain text editor. For example, the program in Figure 1-13 occupies more than 49,000 bytes of disk space.

Although various programming environments might look different and offer different features, the process of using them is very similar. When you plan the logic for a program using pseudocode or a flowchart, it does not matter which programming environment you will use to write your code, and when you write the code in a programming language, it does not matter which environment you use to write it.

Understanding User Environments

A user might execute a program you have written in any number of environments. For example, a user might execute the number-doubling program from a command line like the one shown in Figure 1-14. A **command line** is a location on your computer screen at which you type text entries to communicate with the computer's operating system. In the program in Figure 1-14, the user is asked for a number, and the results are displayed.

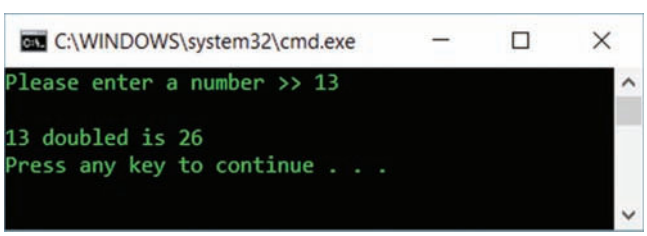


Figure 1-14 Executing a number-doubling program in a command-line environment

Many programs are not run at the command line in a text environment, but are run using a **graphical user interface**, or **GUI** (pronounced *gooey*), which allows users to interact with a program in a graphical environment. When running a GUI program, the user might type input into a text box or use a mouse or other pointing device to select options on the

screen. Figure 1-15 shows a number-doubling program that performs exactly the same task as the one in Figure 1-14, but this program uses a GUI.

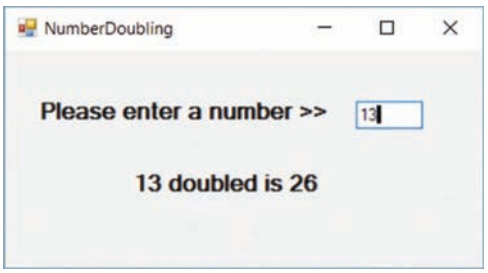


Figure 1-15 Executing a number-doubling program in a GUI environment

A command-line program and a GUI program might be written in the same programming language. (For example, the programs shown in Figures 1-14 and 1-15 were both written using C#.) However, no matter which environment is used to write or execute a program, the logical process is the same. The two programs in Figures 1-14 and 1-15 both accept input, perform multiplication, and perform output. In this book, you will not concentrate on which environment is used to type a program's statements, nor will you care about the type of environment the user will see. Instead, you will be concerned with the logic that applies to all programming situations.

TWO TRUTHS & A LIE

Understanding Programming and User Environments

1. You can type a program into an editor that is part of an integrated development environment, but using a plain text editor provides you with more programming help.
2. When a program runs from the command line, a user types text to provide input.
3. Although GUI and command-line environments look different, the logic of input, processing, and output apply to both program types.

The false statement is #1. An integrated development environment provides more programming help than a plain text editor.

Understanding the Evolution of Programming Models

People have been writing modern computer programs since the 1940s. The oldest programming languages required programmers to work with memory addresses and to memorize awkward codes associated with machine languages. Newer programming languages look much more like natural language and are easier to use, partly because they allow programmers to name variables instead of using unwieldy memory addresses. Also, newer programming languages allow programmers to create self-contained modules or program segments that can be pieced together in a variety of ways.

The oldest computer programs were written in one piece, from start to finish, but modern programs are rarely written that way—they are created by teams of programmers, each developing reusable and connectable program procedures. Writing several small modules is easier than writing one large program, and most large tasks are easier when you break the work into units and get other workers to help with some of the units.



Ada Byron Lovelace predicted the development of software in 1843; she is often regarded as the first programmer. The basis for most modern software was proposed by Alan Turing in 1935.

Currently, programmers use two major models or paradigms to develop programs and their procedures:

- **Procedural programming** focuses on the procedures that programmers create. That is, procedural programmers focus on the actions that are carried out—for example, getting input data for an employee and writing the calculations needed to produce a paycheck from the data. Procedural programmers would approach the job of producing a paycheck by breaking down the process into manageable subtasks.
- **Object-oriented programming** focuses on objects, or "things," and describes their features (also called attributes) and behaviors. For example, object-oriented programmers might design a payroll application by thinking about employees and paychecks, and by describing their attributes. Employees have names and Social Security numbers, and paychecks have names and check amounts. Then the programmers would think about the behaviors of employees and paychecks, such as employees getting raises and adding dependents and paychecks being calculated and output. Object-oriented programmers then would build applications from these entities.

With either approach, procedural or object oriented, you can produce a correct paycheck, and both models employ reusable program modules. The major difference lies in the focus the programmer takes during the earliest planning stages of a project. For now, this book focuses on procedural programming techniques. The skills you gain in programming procedurally—declaring variables, accepting input, making decisions, producing output, and so on—will serve you well whether you eventually write programs using a procedural approach, an object-oriented approach, or both. The programming language in which you write your source code might determine your approach. You can write a procedural program in any language that supports object orientation, but the opposite is not always true.

TWO TRUTHS & A LIE

Understanding the Evolution of Programming Models

1. The oldest computer programs were written in many separate modules.
2. Procedural programmers focus on actions that are carried out by a program.
3. Object-oriented programmers focus on a program's objects and their attributes and behaviors.

The false statement is #1. The oldest programs were written in a single piece; newer programs are divided into modules.

Chapter Summary

- Together, computer hardware (physical devices) and software (instructions) accomplish three major operations: input, processing, and output. You write computer instructions in a computer programming language that requires specific syntax; the instructions are translated into machine language by a compiler or interpreter. When both the syntax and logic of a program are correct, you can run, or execute, the program to produce the desired results.
- For a program to work properly, you must develop correct logic. Logical errors are much more difficult to locate than syntax errors.
- A programmer's job involves understanding the problem, planning the logic, coding the program, translating the program into machine language, testing the program, putting the program into production, and maintaining it.
- When programmers plan the logic for a solution to a programming problem, they often use flowcharts or pseudocode. When you draw a flowchart, you use parallelograms to represent input and output operations, and rectangles to represent processing. Programmers also use decisions to control repetition of instruction sets.
- To avoid creating an infinite loop when you repeat instructions, you can test for a sentinel value. You represent a decision in a flowchart by drawing a diamond-shaped symbol that contains a true-false or yes-no evaluation.
- You can type a program into a plain text editor or one that is part of an integrated development environment. When a program's data values are entered from a keyboard, they

can be entered at the command line in a text environment or in a GUI. Either way, the logic is similar.

- Procedural and object-oriented programmers approach problems differently. Procedural programmers concentrate on the actions performed with data. Object-oriented programmers focus on objects and their behaviors and attributes.

Key Terms

A **computer system** is a combination of all the components required to process and store data using a computer.

Hardware is the collection of physical devices that comprise a computer system.

Software consists of the programs that tell the computer what to do.

Programs are sets of instructions for a computer.

Programming is the act of developing and writing programs.

Application software comprises all the programs you apply to a task.

An **app** is a piece of application software; the term is frequently used for applications on mobile devices.

System software comprises the programs that you use to manage your computer.

Input describes the entry of data items into computer memory using hardware devices such as keyboards and mice.

Data items include all the text, numbers, and other information processed by a computer.

Processing data items may involve organizing them, checking them for accuracy, or performing mathematical operations on them.

The **central processing unit**, or **CPU**, is the computer hardware component that processes data.

Output describes the operation of retrieving information from memory and sending it to a device, such as a monitor or printer, so people can view, interpret, and work with the results.

Information is processed data.

Storage devices are types of hardware equipment, such as disks, that hold information for later retrieval.

The **cloud** refers to remote computers accessed through the Internet.

Programming languages such as Visual Basic, C#, C++, or Java are used to write programs.

Program code is the set of instructions a programmer writes in a programming language.

Coding the program is the act of writing programming language instructions.

The **syntax** of a language is its grammar rules.

Syntax errors are errors in language or grammar.

Computer memory is the temporary, internal storage within a computer.

Random access memory (**RAM**) is temporary, internal computer storage.

Volatile describes storage whose contents are lost when power is lost.

Nonvolatile describes storage whose contents are retained when power is lost.

Machine language is a computer's on/off circuitry language.

Source code is the set of statements a programmer writes in a programming language.

Object code is the set of machine language statements that have been translated from source code.

A **compiler** or **interpreter** translates a high-level language into machine language and indicates if you have used a programming language incorrectly.

Binary language is represented using a series of 0s and 1s.

To **run** or **execute** a program is to carry out its instructions.

Scripting languages such as Python, Lua, Perl, and PHP are used to write programs that are typed directly from a keyboard; they are stored as text rather than as binary executable files.

The **logic** of a computer program is the complete sequence of instructions that lead to a problem's solution

Logical errors occur when incorrect instructions are performed, or when instructions are performed in the wrong order.

A **variable** is a named memory location that can hold different values at different points in time.

The **program development cycle** consists of the steps that occur during a program's lifetime.

Users (or **end users**) are people who employ and benefit from computer programs.

Documentation consists of all the supporting paperwork for a program.

An **algorithm** is the sequence of steps necessary to solve any problem.

Desk-checking is the process of walking through a program solution on paper.

A **high-level programming language** supports English-like syntax.

A **low-level machine language** is made up of 1s and 0s and does not use easily interpreted variable names.

Debugging is the process of finding and correcting program errors.

Conversion is the entire set of actions an organization must take to switch over to using a new program or set of programs.

Maintenance consists of all the improvements and corrections made to a program after it is in production.

Pseudocode is an English-like representation of the logical steps it takes to solve a problem.

A **flowchart** is a pictorial representation of the logical steps it takes to solve a problem.

An **input symbol** indicates an input operation and is represented by a parallelogram in flowcharts.

A **processing symbol** indicates a processing operation and is represented by a rectangle in flowcharts.

An **output symbol** indicates an output operation and is represented by a parallelogram in flowcharts.

An **input/output symbol**, or **I/O symbol**, is represented by a parallelogram in flowcharts.

Flowlines, or arrows, connect the steps in a flowchart.

A **terminal symbol** indicates the beginning or end of a flowchart segment and is represented by a lozenge.

A **loop** is a repetition of a series of steps.

An **infinite loop** occurs when repeating logic cannot end.

A **decision symbol** is shaped like a diamond and used to represent decisions in flowcharts.

A **dummy value** is a preselected value that stops the execution of a program.

A **sentinel value** is a preselected value that stops the execution of a program.

The term **eof** means *end of file*.

A **text editor** is a program that you use to create simple text files; it is similar to a word processor, but without as many features.

An **integrated development environment (IDE)** is a software package that provides an editor, compiler, and other programming tools.

A **command line** is a location on your computer screen at which you type text entries to communicate with the computer's operating system.

A **graphical user interface**, or **GUI** (pronounced *gooey*), allows users to interact with a program in a graphical environment.

Procedural programming is a programming model that focuses on procedures or actions as opposed to object-oriented programming.

Object-oriented programming is a programming model that focuses on objects, or "things," and describes their features (also called attributes) and behaviors, as opposed to procedural programming.

Exercises



Review Questions

1. Computer programs also are known as _____.

   a. data
   b. hardware
   c. software
   d. information

2. The major computer operations include _____.

   a. input, processing, and output
   b. hardware and software
   c. sequence and looping
   d. spreadsheets, word processing, and data communications

3. Visual Basic, C++, and Java are all examples of computer _____.

   a. operating systems
   b. programming languages
   c. hardware
   d. machine languages

4. A programming language's rules are its _____.

   a. syntax
   b. logic
   c. format
   d. options

5. The most important task of a compiler or interpreter is to _____.

   a. create the rules for a programming language
   b. translate English statements into a language such as Java
   c. translate programming language statements into machine language
   d. execute machine language programs to perform useful tasks

6. Which of the following is temporary, internal storage?

   a. CPU
   b. hard disk
   c. keyboard
   d. memory

7. Which of the following pairs of steps in the programming process is in the correct order?

   a. code the program, plan the logic
   b. test the program, translate it into machine language
   c. put the program into production, understand the problem
   d. code the program, translate it into machine language

8. A programmer's most important task before planning the logic of a program is to _____.

   a. decide which programming language to use
   b. code the problem
   c. train the users of the program
   d. understand the problem

9. The two most commonly used tools for planning a program's logic are _____.

   a. ASCII and EBCDIC
   b. Java and Visual Basic
   c. flowcharts and pseudocode
   d. word processors and spreadsheets

10. Writing a program in a language such as C++ or Java is known as _____ the program.

    a. translating
    b. coding
    c. interpreting
    d. compiling

11. An English-like programming language such as Java or Visual Basic is a _____ programming language.

    a. machine-level
    b. low-level
    c. high-level
    d. binary-level

12. Which of the following is an example of a syntax error?

    a. producing output before accepting input
    b. subtracting when you meant to add
    c. misspelling a programming language word
    d. all of the above

13. Which of the following is an example of a logical error?

    a. performing arithmetic with a value before inputting it
    b. accepting two input values when a program requires only one
    c. dividing by 3 when you meant to divide by 30
    d. all of the above

14. The parallelogram is the flowchart symbol representing _____.

    a. input
    b. output
    c. either a or b
    d. none of the above

15. In a flowchart, a rectangle represents _____.

    a. input
    b. a sentinel
    c. a question
    d. processing

16. In flowcharts, the decision symbol is a _____.

    a. parallelogram
    b. rectangle
    c. lozenge
    d. diamond

17. The term *eof* represents _____.

    a. a standard input device
    b. a generic sentinel value
    c. a condition in which no more memory is available for storage
    d. the logical flow in a program

18. When you use an IDE instead of a simple text editor to develop a program, _____.

    a. the logic is more complicated
    b. the logic is simpler
    c. the syntax is different
    d. some help is provided

19. When you write a program that will run in a GUI environment as opposed to a command-line environment, _____.

    a. the logic is very different
    b. some syntax is different
    c. you do not need to plan the logic
    d. users are more confused

20. As compared to procedural programming, with object-oriented programming, _____.

    a. the programmer's focus differs
    b. you cannot use some languages such as Java
    c. you do not accept input
    d. you do not code calculations; they are created automatically



Programming Exercises

1. Match the definition with the appropriate term.

    1. Computer system devices
    2. Another word for *program*
    3. Language rules
    4. Order of instructions
    5. Language translator

    a. compiler
    b. syntax
    c. logic
    d. hardware
    e. software

2. In your own words, describe the steps to writing a computer program.
3. Draw a flowchart or write pseudocode to represent the logic of a program that allows the user to enter a value. The program multiplies the value by 10 and outputs the result.
4. Draw a flowchart or write pseudocode to represent the logic of a program that allows the user to enter a value for one edge of a cube. The program calculates the surface area of one side of the cube, the surface area of the cube, and its volume. The program outputs all the results.
5. Draw a flowchart or write pseudocode to represent the logic of a program that allows the user to enter a value for hours worked in a day. The program calculates the hours worked in a five-day week and the hours worked in a 252-day work year. The program outputs all the results.
6. Draw a flowchart or write pseudocode to represent the logic of a program that allows the user to enter two values. The program outputs the sum of and the difference between the two values.
7. Draw a flowchart or write pseudocode to represent the logic of a program that allows the user to enter values for the current year and the user's birth year. The program outputs the age of the user this year.
8. a. Draw a flowchart or write pseudocode to represent the logic of a program that allows the user to enter an hourly pay rate and hours worked. The program outputs the user's gross pay.

   b. Modify the program that computes gross pay to allow the user to enter the withholding tax rate. The program outputs the net pay after taxes have been withheld.
9. Research current rates of monetary exchange. Draw a flowchart or write pseudocode to represent the logic of a program that allows the user to enter a number of dollars and convert it to Euros and Japanese yen.
10. A mobile phone app allows a user to press a button that starts a timer that counts seconds. When the user presses the button again, the timer stops. Draw a flowchart or write pseudocode that accepts the elapsed time in seconds and displays the value in minutes and seconds. For example, if the elapsed time was 130 seconds, the output would be 2 minutes and 10 seconds.



Performing Maintenance

1. In this chapter you learned that some of the tasks assigned to new programmers frequently involve maintenance—making changes to existing programs because of new requirements. A file named MAINTENANCE01-01.txt is included with

your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

Since the early days of computer programming, program errors have been called bugs. The term is often said to have originated from an actual moth that was discovered trapped in the circuitry of a computer at Harvard University in 1945. Actually, the term *bug* was in use prior to 1945 to mean trouble with any electrical apparatus; even during Thomas Edison's life, it meant an industrial defect. The term *debugging*, however, is more closely associated with correcting program syntax and logic errors than with any other type of trouble.

1. Your downloadable files for Chapter 1 include DEBUG01-01.txt, DEBUG01-02.txt, and DEBUG01-03.txt. Each file starts with some comments (lines that begin with two slashes) that describe the program. Examine the pseudocode that follows the introductory comments, and then find and correct all the bugs.

2. Your downloadable files for Chapter 1 include a file named DEBUG01-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.



Game Zone

1. In 1952, A. S. Douglas wrote his University of Cambridge Ph.D. dissertation on human-computer interaction, and created the first graphical computer game—a version of Tic-Tac-Toe. The game was programmed on an EDSAC vacuum-tube mainframe computer. The first computer game is generally assumed to be *Spacewar!*, developed in 1962 at MIT; the first commercially available video game was *Pong*, introduced by Atari in 1972. In 1980, Atari's *Asteroids* and *Lunar Lander* became the first video games to be registered with the U. S. Copyright Office. Throughout the 1980s, players spent hours with games that now seem very simple and unglamorous; do you recall playing *Adventure*, *Oregon Trail, Where in the World Is Carmen Sandiego?*, or *Myst*?

   Today, commercial computer games are much more complex; they require many programmers, graphic artists, and testers to develop them, and large management and marketing staffs are needed to promote them. A game might cost many

millions of dollars to develop and market, but a successful game might earn hundreds of millions of dollars. Obviously, with the brief introduction to programming you have had in this chapter, you cannot create a very sophisticated game. However, you can get started.

Mad Libs is a children's game in which players provide a few words that are then incorporated into a silly story. The game helps children understand different parts of speech because they are asked to provide specific types of words. For example, you might ask a child for a noun, another noun, an adjective, and a past-tense verb. The child might reply with such answers as *table*, *book*, *silly*, and *studied*.

The newly created Mad Lib might be:

Mary had a little *table.*

Its *book* was *silly* as snow

And everywhere that Mary *studied*

The *table* was sure to go.

Create the logic for a Mad Lib program that accepts five words from input, then creates and displays a short story or nursery rhyme that uses them.

CHAPTER 2

Elements of High-Quality Programs

Upon completion of this chapter, you will be able to:

- Declare and use variables and constants
- Perform arithmetic operations
- Appreciate the advantages of modularization
- Modularize a program
- Create hierarchy charts
- Describe the features of good program design

Declaring and Using Variables and Constants

As you learned in Chapter 1, data items include all the text, numbers, and other material processed by a computer. When you input data items to be used by a program, they are stored in memory where they can be processed and converted to information that is output.

When you write programs, you work with data in two different types: numeric and string, and in three different forms: literals (or unnamed constants), variables, and named constants.

Understanding Data Types

A data item's **data type** is a classification that describes the following:

- What values can be held by the item
- How the item is stored in computer memory
- What operations can be performed on the item

All programming languages support two broad data types:

- **Numeric** describes data that consists of numbers, possibly with a decimal point or a sign; numeric data can be used in arithmetic operations.
- **String** describes data items that are nonnumeric; string data cannot be used in arithmetic operations.

Most programming languages support several additional data types, including multiple types for numeric values that are very large or very small and for those that do and do not have fractional decimal digits. Languages such as C++, C#, Visual Basic, and Java distinguish between **integer** (whole number) numeric variables and **floating-point** (fractional) numeric variables that contain a decimal point. (Floating-point numbers also are called **real numbers**.) Thus, in some languages, the values 4 and 4.3 would be stored in different types of numeric variables. Additionally, many languages allow you to distinguish between very small and very large values that occupy different numbers of bytes in memory. You will learn more about these specialized data types when you study a programming language, but this book uses only the two broadest types: numeric and string.

Understanding Unnamed, Literal Constants

When you use a specific value in a computer program, it is one of two types of constants:

- A **numeric constant** (or **literal numeric constant**) is a number that does not change—for example, 43. When you store a numeric value in computer memory, additional characters such as dollar signs and commas typically are not input or stored. Those characters might be added to output for readability, but they are not part of the number.

- A **string constant** (or **literal string constant**) appears within quotation marks in computer programs. String values are also called **alphanumeric values** because they can contain alphabetic characters as well as numbers and other characters. For example, "Amanda", "51", and "$3,215.99 U.S." all are strings because they are enclosed in quotation marks. Although strings can contain numbers, numeric values cannot contain alphabetic characters.

The numeric constant 43 and the string constant "Amanda" are examples of **unnamed constants**—they do not have identifiers like variables do.



Watch the video *Declaring Variables and Constants*.

Working with Variables

Variables are named memory locations whose contents can vary or differ over time. For example, in the number-doubling program in Figure 2-1, `myNumber` and `myAnswer` are variables. At any moment in time, a variable holds just one value. Sometimes, `myNumber` holds 2 and `myAnswer` holds 4; at other times, `myNumber` holds 6 and `myAnswer` holds 12. The ability of variables to change in value is what makes computers and programming worthwhile. Because one memory location can be used repeatedly with different values, you can write program instructions once and then use them for thousands of separate calculations. *One* set of payroll instructions at your company produces each employee paycheck, and *one* set of instructions at your electric company produces each household's bill.

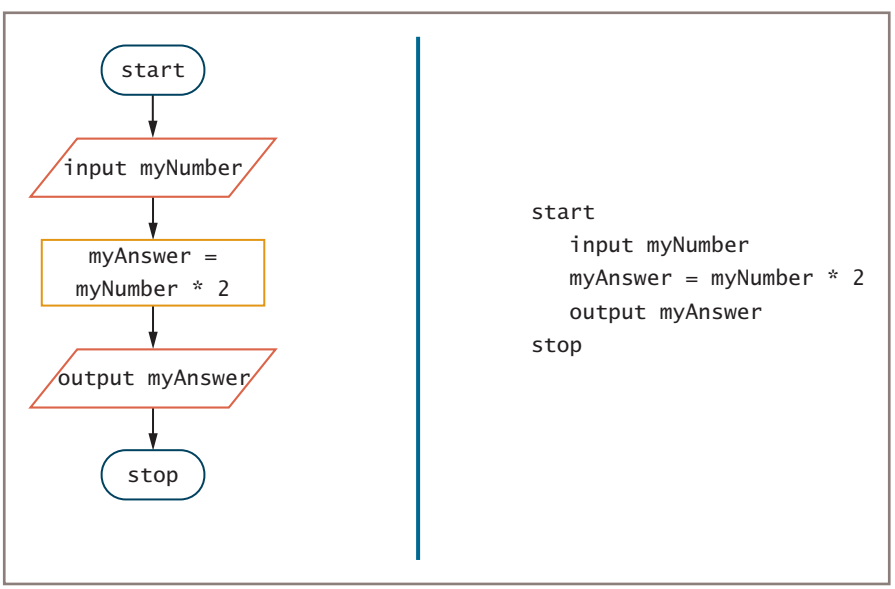


Figure 2-1 Flowchart and pseudocode for the number-doubling program

In most programming languages, before you can use any variable, you must include a declaration for it. A **declaration** is a statement that provides these things for a variable:

- a data type
- an identifier
- optionally, an initial value

Understanding a Declaration's Data Type

Every programming language requires that you specify the correct type for each variable, and that you use each type appropriately.

- A **numeric variable** is one that can hold digits and have mathematical operations performed on it. In the statement `myAnswer = myNumber * 2`, both `myAnswer` and `myNumber` are numeric variables; that is, their intended contents are numeric values, such as 6 and 3, 14.8 and 7.4, or −18 and −9.
- A **string variable** can hold text, such as letters of the alphabet, and other special characters, such as punctuation marks. If a working program contains the statement `lastName = "Lincoln"`, then `lastName` is a string variable. Programmers frequently use strings to hold digits when they will never be used in arithmetic statements—for example, an account number or a zip code.

Type-safety is the feature of some programming languages that prevents assigning values of an incorrect data type. In those languages, you can assign a data item to a variable only if the data item is the correct type. (Such languages are called *strongly typed*.) If you declare `taxRate` as a numeric variable and `inventoryItem` as a string, then the following statements are valid:

```
taxRate = 2.5
inventoryItem = "monitor"
```

The following are invalid because the type of data being assigned does not match the variable type:

```
taxRate = "2.5"
inventoryItem = 2.5
taxRate = inventoryItem
inventoryItem = taxRate
```

Don't Do It
If taxRate is numeric and inventoryItem is a string, then these assignments are invalid.



Watch the video *Understanding Data Types*.

Understanding a Declaration's Identifier

An **identifier** is a program component's name; it is chosen by the programmer. The number-doubling example in Figure 2-1 requires two variable identifiers: `myNumber` and `myAnswer`. Alternatively, these variables could be named `userEntry` and `programSolution`, or `inputValue` and `twiceTheValue`. As a programmer, you choose reasonable and descriptive names for your variables. The language translator (interpreter or compiler) then associates the names you choose with specific memory addresses.

Every computer programming language has its own set of rules for creating identifiers. Most languages allow letters and digits within identifiers. Some languages allow hyphens in variable names, such as `hourly-wage`, and some allow underscores, as in `hourly_wage`. Some languages allow dollar signs or other special characters in variable names (for example, `hourly$`); others allow foreign-alphabet characters, such as π or Ω. Each programming language has a few (perhaps 100 to 200) reserved **keywords** that are not allowed as variable names because they are part of the language's syntax. For example, the data type names in a language, such as `num` and `string`, would not be allowed as variable names. When you learn a programming language, you will learn its list of keywords.

Different languages put different limits on the length of variable names, although in general, the length of identifiers in newer languages is virtually unlimited. In the oldest computer languages, all variable names were written using all uppercase letters because the keypunch machines used at that time created only uppercase letters. In most modern languages, identifiers are case sensitive, so `HoUrLyWaGe`, `hourlywage`, and `hourlyWage` are three separate variable names. Programmers use multiple conventions for naming variables, often depending on the programming language or standards adopted by their employers. Quick Reference 2-1 describes commonly used variable naming conventions.

QUICK REFERENCE 2-1 Variable Naming Conventions

Convention for naming variables	Examples	Languages where commonly used
Camel casing is the convention in which the variable starts with a lowercase letter and any subsequent word begins with an uppercase letter. It is sometimes called **lower camel casing** to emphasize the difference from Pascal casing.	`hourlyWage` `lastName`	Java, C#

(continues)

(continued)

Convention for naming variables	Examples	Languages where commonly used
Pascal casing is a convention in which the first letter of a variable name is uppercase. It is sometimes called **upper camel casing** to distinguish it from lower camel casing.	`HourlyWage` `LastName`	Visual Basic
Hungarian notation is a form of camel casing in which a variable's data type is part of the identifier.	`numHourly-Wage stringLastName`	C for Windows API programming
Snake casing is a convention in which parts of a variable name are separated by underscores.	`hourly_wage` `last_name`	C, C++, Python, Ruby
Mixed case with underscores is a variable naming convention similar to snake casing, but new words start with an uppercase letter.	`Hourly_Wage` `Last_Name`	Ada
Kebob case is sometimes used as the name for the style that uses dashes to separate parts of a variable name. The name derives from the fact that the words look like pieces of food on a skewer.	`hourly-wage` `last-name`	Lisp (with lowercase letters), COBOL (with uppercase letters)

Adopting a naming convention for variables and using it consistently will help make your programs easier to read and understand.

Even though every language has its own rules for naming variables, you should not concern yourself with the specific syntax of any particular computer language when designing the logic of a program. The logic, after all, works with any language. The variable names used throughout this book follow only three rules:

1. *Variable names must be one word.* The name can contain letters, digits, hyphens, or underscores. No language allows embedded spaces in variable names, and most do not allow punctuation such as periods, commas, or colons. This book uses only alphabetic letters, digits, and underscores in variable names. Therefore, `r` is a legal variable name, as are `rate` and `interestRate`. The variable name `interest rate` is not allowed because of the space.
2. *Variable names must start with a letter.* Some programming languages allow variable names to start with a nonalphabetic character such as an underscore. Almost all programming languages prohibit variable names that start with a digit. This book follows the most common convention of starting variable names with a letter.



When you write a program using an editor that is packaged with a compiler in an IDE, the compiler may display variable names in a different color from other program components. This visual aid helps your variable names stand out from words that are part of the programming language.

3. *Variable names should have some appropriate meaning.* This is not a formal rule of any programming language. When computing an interest rate in a program, the computer does not care if you call the variable `g`, `u84`, or `fred`. As long as the correct numeric result is placed in the variable, its actual name doesn't matter. However, it's much easier to follow the logic of a statement like `interestEarned = initialInvestment * interestRate` than a statement like `f = i * r` or `someBanana = j89 * myFriendLinda`. When a program requires changes, which could be months or years after you write the original version, you and your fellow programmers will appreciate clear, descriptive variable names in place of cryptic identifiers. Later in this chapter, you will learn more about selecting good identifiers.

Notice that the flowchart in Figure 2-2 follows the preceding rules for variables: Both variable names, `myNumber` and `myAnswer`, are single words without embedded spaces, and they have appropriate meanings. Some programmers name variables after friends or create puns with them, but computer professionals consider such behavior unprofessional and amateurish.

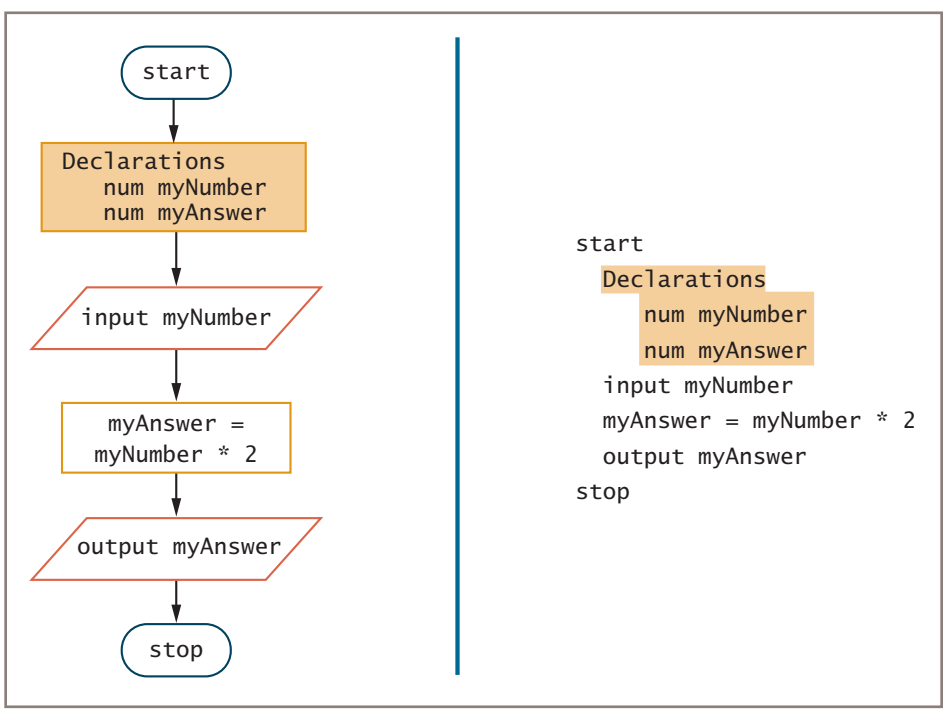


Figure 2-2 Flowchart and pseudocode of number-doubling program with variable declarations

Assigning Values to Variables

When you create a flowchart or pseudocode for a program that doubles numbers, you can include a statement such as the following:

```
myAnswer = myNumber * 2
```

Such a statement is an **assignment statement**. This statement incorporates two actions. First, the computer calculates the arithmetic value of `myNumber * 2`. Second, the computed value is stored in the `myAnswer` memory location.

The equal sign is the **assignment operator**. The assignment operator is an example of a **binary operator**, meaning it requires two operands—one on each side. (An **operand** is simply a value used by an operator.) The assignment operator always operates from right to left, which means that it has **right-associativity** or **right-to-left associativity**. This means that the value of the expression to the right of the assignment operator is evaluated first, and then the result is assigned to the operand on the left. The operand to the right of an assignment operator must be a value—for example, a named or unnamed constant or an arithmetic expression. The operand to the left of an assignment operator must be a name that represents a memory address—the name of the location where the result will be stored.

For example, if you have declared two numeric variables named `someNumber` and `someOtherNumber`, then each of the following is a valid assignment statement:

```
someNumber = 2
someNumber = 3 + 7
someOtherNumber = someNumber
someOtherNumber = someNumber * 5
```

In each case, the expression to the right of the assignment operator is evaluated and stored at the location referenced on the left side. The result to the left of an assignment operator is called an **lvalue**. The *l* is for left. Lvalues are always memory address identifiers.

The following statements, however, are *not* valid:

```
set 2 + 4 = someNumber
set someOtherNumber * 10 = someNumber
set someNumber + someOtherNumber = 10
```

Don't Do It
The operand to the left of an assignment operator must represent a memory address.

The operand to the left of an assignment operator must represent a memory address. In each of these cases, the value to the left of the assignment operator does not represent a memory address, so the statements are invalid.

Initializing a Variable

Besides a data type and an identifier, when you declare a variable you can declare a starting value. Declaring a starting value is known as **initializing the variable**. When you create a variable without assigning it an initial value, your intention is to assign a value later—for example, by receiving one as input or placing the result of

a calculation there. When you assign an initial value, your intention is to use that value before any new value is assigned.

For example, each of the following statements is a valid declaration. Two of the statements include initializations, and two do not:

```
num mySalary
num yourSalary = 14.55
string myName
string yourName = "Juanita"
```

In many programming languages, if you declare a variable and do not initialize it, the variable contains an unknown value until it is assigned a value. A variable's unknown value commonly is called **garbage**. Although some languages use a default value for some variables (such as assigning 0 to any unassigned numeric variable), this book will assume that an unassigned variable holds garbage. In many languages it is illegal to use a garbage-holding variable in an arithmetic or output statement. Even if you work with a language that allows you to display garbage, it serves no purpose to do so and constitutes a logical error.

Where to Declare Variables

Variables must be declared before they are used for the first time in a program. Some languages require all variables to be declared at the beginning of the program, others allow variables to be declared at the beginning of each module, and others allow variables to be declared anywhere at all as long as they are declared before their first use. This book will follow the convention of declaring all variables together. Figure 2-2 shows the number-doubling program from Figure 2-1 with the added declarations shaded.

Declaring Named Constants

Besides variables, most programming languages allow you to create named constants. A **named constant** is similar to a variable, except it can be assigned a value only once. You use a named constant when you want to assign a useful name for a value that will never be changed during a program's execution. Using named constants makes your programs easier to understand by eliminating magic numbers. A **magic number** is an unnamed constant, like 0.06, whose purpose is not immediately apparent.

For example, if a program uses a sales tax rate of 6 percent, you might want to declare a named constant as follows:

```
num SALES_TAX_RATE = 0.06
```

After `SALES_TAX_RATE` is declared, the following statements have identical meaning:

```
taxAmount = price * 0.06
taxAmount = price * SALES_TAX_RATE
```

The way in which named constants are declared differs among programming languages. This book follows the convention of using all uppercase letters in constant identifiers, and using underscores to separate words for readability. Using these conventions makes named

constants easier to recognize. In many languages, a constant must be assigned its value when it is declared, but in some languages, a constant can be assigned its value later. In both cases, however, a constant's value cannot be changed after the first assignment. This book follows the convention of initializing all constants when they are declared.

When you declare a named constant, program maintenance becomes easier. For example, if the value of the sales tax rate changes from 0.06 to 0.07 in the future, and you have declared a named constant SALES_TAX_RATE, you only need to change the value assigned to the named constant at the beginning of the program, then retranslate the program into machine language, and all references to SALES_TAX_RATE are automatically updated. If you used the unnamed literal 0.06 instead, you would have to search for every instance of the value and replace it with the new one. Additionally, if the literal 0.06 was used in other calculations within the program (for example, as a discount rate or price), you would have to carefully select which instances of the value to alter, and you would be likely to make a mistake.



Sometimes, using unnamed literal constants is appropriate in a program, especially if their meaning is clear to most readers. For example, in a program that calculates half of a value by dividing by two, you might choose to use the unnamed literal 2 instead of incurring the extra time and memory costs of creating a named constant HALF and assigning 2 to it. Extra costs that result from adding variables or instructions to a program are known as **overhead**.

TWO TRUTHS & A LIE

Declaring and Using Variables and Constants

1. A variable's data type describes the kind of values the variable can hold and the types of operations that can be performed with it.
2. If `name` is a string variable, then the assignment statement `name = "Ed" is valid`.
3. The operand to the right of an assignment operator must be a name that represents a memory address.

The false statement is #3. The operand to the left of an assignment operator must be a name that represents a memory address—the name of the location where the result will be stored. The value to the right of an assignment operator might be a constant, arithmetic expression, or other value.

Performing Arithmetic Operations

Most programming languages use the following standard arithmetic operators:

+ (plus sign)—addition

− (minus sign)—subtraction

* (asterisk)—multiplication

/ (slash)—division

Many languages also support additional operators that calculate the remainder after division, raise a number to a power, manipulate individual bits stored within a value, and perform other operations.

Like the assignment operator (=), each of the standard arithmetic operators is a binary operator; that is, each requires an expression on both sides. For example, the following statement adds two test scores and assigns the sum to a variable named

```
totalScore:
totalScore = test1 + test2
```

The following adds 10 to `totalScore` and stores the result in `totalScore`:

```
totalScore = totalScore + 10
```

In other words, this example increases the value of `totalScore`. This last example looks odd in algebra because it might appear that the programmer is indicating that the value of `totalScore` and `totalScore` plus 10 are equivalent. You must remember that the equal sign is the assignment operator, and that the statement is actually taking the original value of `totalScore`, adding 10 to it, and assigning the result to the memory address on the left of the operator, which is `totalScore`. In other words, the operation changes the value in `totalScore`.

In programming languages, you can combine arithmetic statements. When you do, every operator follows **rules of precedence** (also called the **order of operations**) that dictate the order in which operations in the same statement are carried out. The rules of precedence for the basic arithmetic statements are as follows:

- Expressions within parentheses are evaluated first. If there are multiple sets of parentheses, the expression within the innermost parentheses is evaluated first.
- Multiplication and division are evaluated next, from left to right.
- Addition and subtraction are evaluated next, from left to right.

The assignment operator has a very low precedence. Therefore, in a statement such as `d = e * f + g`, the operations on the right of the assignment operator in this case, multiplication and addition) are always performed before the final assignment to the variable on the left.



When you learn a specific programming language, you will learn about all the operators that are used in that language. Many programming language books contain a table that specifies the relative precedence of every operator used in the language.

For example, consider the following two arithmetic statements:

```
firstAnswer = 2 + 3 * 4
secondAnswer = (2 + 3) * 4
```

After these statements execute, the value of `firstAnswer` is 14. According to the rules of precedence, multiplication is carried out before addition, so 3 is multiplied by 4, giving 12, and then 2 and 12 are added, and 14 is assigned to `firstAnswer`. The value of `secondAnswer`, however, is 20, because the parentheses force the contained addition operation to be performed first. The 2 and 3 are added, producing 5, and then 5 is multiplied by 4, producing 20.

Forgetting about the rules of arithmetic precedence, or forgetting to add parentheses when you need them, can cause logical errors that are difficult to find in programs. For example, the following statement might appear to average two test scores:

```
average = score1 + score2 / 2
```

However, it does not. Because division has a higher precedence than addition, the preceding statement takes half of `score2`, adds it to `score1`, and stores the result in `average`. The correct statement is:

```
average = (score1 + score2) / 2
```

You are free to add parentheses even when you don't need them to force a different order of operations; sometimes you use them just to make your intentions clearer. For example, the following statements operate identically:

```
totalPriceWithTax = price + price * TAX_RATE
totalPriceWithTax = price + (price * TAX_RATE)
```

In both cases, `price` is multiplied by `TAX_RATE` first, then it is added to `price`, and finally the result is stored in `totalPriceWithTax`. Because multiplication occurs before addition on the right side of the assignment operator, both statements are the same. However, if you feel that the statement with the parentheses makes your intentions clearer to someone reading your program, then you should use them. All the arithmetic operators have **left-to-right associativity**. This means that operations with the same precedence take place from left to right. Consider the following statement:

```
answer = a + b + c * d / e - f
```

Multiplication and division have higher precedence than addition or subtraction, so the multiplication and division are carried out from left to right as follows: `c` is multiplied by `d`, and the result is divided by `e`, giving a new result. Therefore, the statement becomes:

```
answer = a + b + (temporary result just calculated) - f
```

Then, addition and subtraction are carried out from left to right as follows: `a` and `b` are added, the temporary result is added, and then `f` is subtracted. The final result is then assigned to `answer`.

Another way to say this is that the following two statements are equivalent:

```
answer = a + b + c * d / e - f
answer = a + b + ((c * d) / e) - f
```

Quick Reference 2-2 summarizes the precedence and associativity of the five most frequently used operators.

QUICK REFERENCE 2-2 Precedence and Associativity of Five Common Operators

Operator symbol	Operator name	Precedence (compared to other operators in this table)	Associativity
=	Assignment	Lowest	Right-to-left
+	Addition	Medium	Left-to-right
−	Subtraction	Medium	Left-to-right
*	Multiplication	Highest	Left-to-right
/	Division	Highest	Left-to-right



Watch the video *Arithmetic Operator Precedence*.

The Integer Data Type

As mentioned earlier in this chapter, many modern programming languages allow programmers to make fine distinctions between numeric data types. In particular, many languages treat integer numeric values (whole numbers) and floating-point numeric values (numbers with decimal places) differently. In these languages, you can always assign an integer, such as 3, to a floating-point variable or named constant, and it will be converted to 3.0. However, you cannot assign a floating-point value (such as 3.0) directly to an integer variable, because the decimal position values will be lost, even when they are 0.

When you work with a language that makes distinctions between integer and floating-point values, you can combine the different types in arithmetic expressions. When you do, addition, subtraction, and multiplication work as expected. For example, the result of 2.3 + 5 is 7.3, and the result of 4.2 * 2 is 8.4. When you mix types, division works as expected as well. For example, the result of 9.3 / 3 is 3.1.

However, in many languages, dividing an integer by another integer is a special case. In languages such as Java, C++, and C#, dividing two integers results in an integer, and any fractional part of the result is lost. For example, in these languages, the result of 7 / 2 is 3, not 3.5 as you might expect. Programmers say that the decimal portion of the result is cut off, or *truncated*.



When programming in a language that truncates the results of integer division, you must be particularly careful with numbers lower than 1. For example, if you write a program that halves a recipe, you might use an expression such as 1 / 2 * `cupsSugar`. No matter what the value of `cupsSugar` is, the result will always be 0 because 2 goes into 1 zero whole times.

Many programming languages also support a **remainder operator**, which is sometimes called the *modulo operator* or the *modulus operator*. When used with two integer operands, the remainder operator is the value that remains after division. For example, 24 Mod 10 is 4 because when 24 is divided by 10, 4 is the remainder. In Visual Basic, the remainder operator is the keyword `Mod`. In Java, C++, and C#, the operator is the percent sign (%).

The remainder operator can be useful in a variety of situations. For example, you can determine whether a number is even or odd by finding the remainder when the number is divided by 2. Any number that has a remainder of 0 is even, and any number with a remainder of 1 is odd.

Because the remainder operator differs among programming languages, and because the operation itself is handled differently when used with negative operands, the remainder operator will not be used in the rest of this language-independent book. Similarly, this book uses one data type, `num`, for all numeric values, and it is assumed that both integer and floating-point values can be stored in `num` variables and named constants.

TWO TRUTHS & A LIE

Performing Arithmetic Operations

1. Parentheses have higher precedence than any of the common arithmetic operators.
2. Operations in arithmetic statements occur from left to right in the order in which they appear.
3. The following adds 5 to a variable named `points`: `points = points + 5`

The false statement is #2. Operations of equal precedence in an arithmetic statement are carried out from left to right, but operations within parentheses are carried out first, multiplication and division are carried out next, and addition and subtraction take place last.

Understanding the Advantages of Modularization

Programmers seldom write programs as one long series of steps. Instead, they break down their programming problems into smaller units and tackle one cohesive task at a time. These smaller units are **modules**. Programmers also refer to them as **subroutines, procedures, functions,** or **methods**; the name usually reflects the programming language being used. For example, Visual Basic programmers use *procedure* (or *subprocedure*). C and C++ programmers call their modules *functions*, whereas C#, Java, and other object-oriented language programmers are more likely to use *method*. Programmers in COBOL, RPG, and BASIC (all older languages) are most likely to use *subroutine*.



You can learn about modules that receive and return data in Chapter 9 of the comprehensive version of this book.

A main program executes a module by calling it. To **call a module** is to use its name to invoke the module, causing it to execute. When the module's tasks are complete, control returns to the spot from which the module was called. When you access a module, the action is similar to pausing a video. You abandon your primary action (watching a video), take care of some other task (for example, making a sandwich), and then return to the main task exactly where you left off.

The process of breaking down a large program into modules is **modularization**; computer scientists also call it **functional decomposition**. You are never required to modularize a large program to make it run on a computer, but there are at least three reasons for doing so:

- Modularization provides abstraction.
- Modularization helps multiple programmers to work on a problem.
- Modularization allows you to reuse work more easily.

Modularization Provides Abstraction

One reason that modularized programs are easier to understand is that they enable a programmer to see the "big picture." **Abstraction** is the process of paying attention to important properties while ignoring nonessential details. Abstraction is selective ignorance. Life would be tedious without abstraction. For example, you can create a list of things to accomplish today:

```
Do laundry
Call Aunt Nan
Start term paper
```

Without abstraction, the list of chores would begin:

```
Pick up laundry basket
Put laundry basket in car
Drive to Laundromat
Get out of car with basket
Walk into Laundromat
Set basket down
Find quarters for washing machine
… and so on.
```

You might list a dozen more steps before you finish the laundry and move on to the second chore on your original list. If you had to consider every small, low-level detail of every task in your day, you probably would never make it out of bed in the morning. Using a higher-level, more abstract list makes your day manageable. Abstraction makes complex tasks look simple.



Abstract artists create paintings in which they see only the big picture—color and form—and ignore the details. Abstraction has a similar meaning among programmers.

Likewise, some level of abstraction occurs in every computer program. Fifty years ago, a programmer had to understand the low-level circuitry instructions the computer used. But now, newer high-level programming languages allow you to use English-like vocabulary in which one broad statement corresponds to dozens of machine instructions. No matter which high-level programming language you use, if you display a message on the monitor, you are never required to understand how a monitor works to create each pixel on the screen. You write an instruction like `output message` and the details of the hardware operations are handled for you by the operating system.

Modules provide another way to achieve abstraction. For example, a payroll program can call a module named `computeFederalWithholdingTax()`. When you call this module from your program, you use one statement; the module itself might contain dozens of statements. You can write the mathematical details of the module later, someone else can write them, or you can purchase them from an outside source. When you plan your main payroll program, your only concern is that a federal withholding tax will have to be calculated; you save the details for later.

Modularization Helps Multiple Programmers to Work on a Problem

When you divide any large task into modules, you gain the ability to more easily divide the task among various people. Rarely does a single programmer write a commercial program that you buy. Consider any word-processing, spreadsheet, or database program you have used. Each program has so many options, and responds to user selections in so many possible ways, that it would take years for a single programmer to write all the instructions. Professional software developers can write new programs in weeks or months, instead of years, by dividing large programs into modules and assigning each module to an individual programmer or team.

Modularization Allows You to Reuse Work

If a module is useful and well written, you may want to use it more than once within a program or in other programs. For example, a routine that verifies the validity of dates is useful in many programs written for a business. (For example, a month value is valid if it is not lower than 1 or higher than 12, a day value is valid if it is not lower than 1 or higher than 31 if the month is 1, and so on.) If a computerized personnel file contains each employee's birth date, hire date, last promotion date, and termination date, the date-validation module can be used four times with each employee record. Other programs in an organization also can use the module; these programs might ship customer orders, plan employees' birthday parties, or calculate when loan payments should be made. If you write

the date-checking instructions so they are entangled with other statements in a program, they are difficult to isolate and reuse. On the other hand, if you place the instructions in a separate module, the unit is easy to use and portable to other applications. The feature of modular programs that allows individual modules to be used in a variety of applications is **reusability**.

You can find many real-world examples of reusability. When you build a house, you don't invent plumbing and heating systems; you incorporate systems with proven designs. This certainly reduces the time and effort it takes to build a house. The systems you choose are in service in other houses, so they have been tested under a variety of circumstances, increasing their reliability. **Reliability** is the feature of programs that assures you a module has been proven to function correctly. Reliable software saves time and money. If you create the functional components of your programs as stand-alone modules and test them in your current programs, much of the work already will be done when you use the modules in future applications.

TWO TRUTHS & A LIE

Understanding the Advantages of Modularization

1. Modularization eliminates abstraction, a feature that makes programs more confusing.
2. Modularization makes it easier for multiple programmers to work on a problem.
3. Modularization allows you to reuse work more easily.

The false statement is #1. Modularization enables abstraction, which allows you to see the big picture.

Modularizing a Program

Most programs consist of a **main program**, which contains the basic steps, or the **mainline logic**, of the program. The main program then accesses modules that provide more refined details. When you create a module, you include the following:

- A header—The **module header** includes the module identifier and possibly other necessary identifying information.
- A body—The **module body** contains all the statements in the module.
- A `return` statement—The **module `return` statement** marks the end of the module and identifies the point at which control returns to the program or module that called the module. In most programming languages, if you do not include a `return` statement

at the end of a module, the logic will still return. However, this book follows the convention of explicitly including a `return` statement with every module.

Naming a module is similar to naming a variable. The rules and conventions for naming modules are slightly different in every programming language, but in this text, module names follow the same general rules used for variable identifiers:

- Module names must start with a letter and cannot contain spaces.
- Module names should have meaning.



Although it is not a requirement of any programming language, it frequently makes sense to use a verb as all or part of a module's name, because modules perform some action. Typical module names begin with action words such as `get`, `calculate`, and `display`. When you program in visual languages that use screen components such as buttons and text boxes, the module names frequently contain verbs representing user actions, such as `click` or `drag`.

Additionally, in this text, module names are followed by a set of parentheses. This will help you distinguish module names from variable names. This style corresponds to the way modules are named in many programming languages, such as Java, C++, and C#.



As you learn more about modules in specific programming languages, you will find that you sometimes place variable names within the parentheses that follow module names. Any variables enclosed in the parentheses contain information you want to send to the module. For now, the parentheses at the end of module names will be empty in this book.

When a main program wants to use a module, it calls the module. A module can call another module, and the called module can call another. The number of chained calls is limited only by the amount of memory available on your computer. In this book, the flowchart symbol used to call a module is a rectangle with a bar across the top. You place the name of the module you are calling inside the rectangle.



Some programmers use a rectangle with stripes down each side to represent a module in a flowchart, and this book uses that convention if a module is external to a program. For example, prewritten, built-in modules that generate random numbers, compute standard trigonometric functions, and sort values often are external to your programs. However, if the module is being created as part of the program, the book uses a rectangle with a single stripe across the top.

In a flowchart, you draw each module separately with its own sentinel symbols. The beginning sentinel contains the name of the module. This name must be identical to the name used in the calling program or module. The ending sentinel contains `return`, which indicates that when the module ends, the logical progression of statements will exit the module and return to the calling program or module. Similarly, in pseudocode, you start each module with its name and end with a `return` statement; the module name and `return` statements are vertically aligned and all the module statements are indented between them.

For example, consider the program in Figure 2-3, which does not contain any modules. It accepts a customer's name and balance due as input and produces a bill. At the top of the bill, the company's name and address are displayed on three lines, which are followed by the customer's name and balance due. To display the company name and address, you can simply include three `output` statements in the mainline logic of a program, as shown in Figure 2-3, or you can modularize the program by creating both the mainline logic and a `displayAddressInfo()` module, as shown in Figure 2-4.

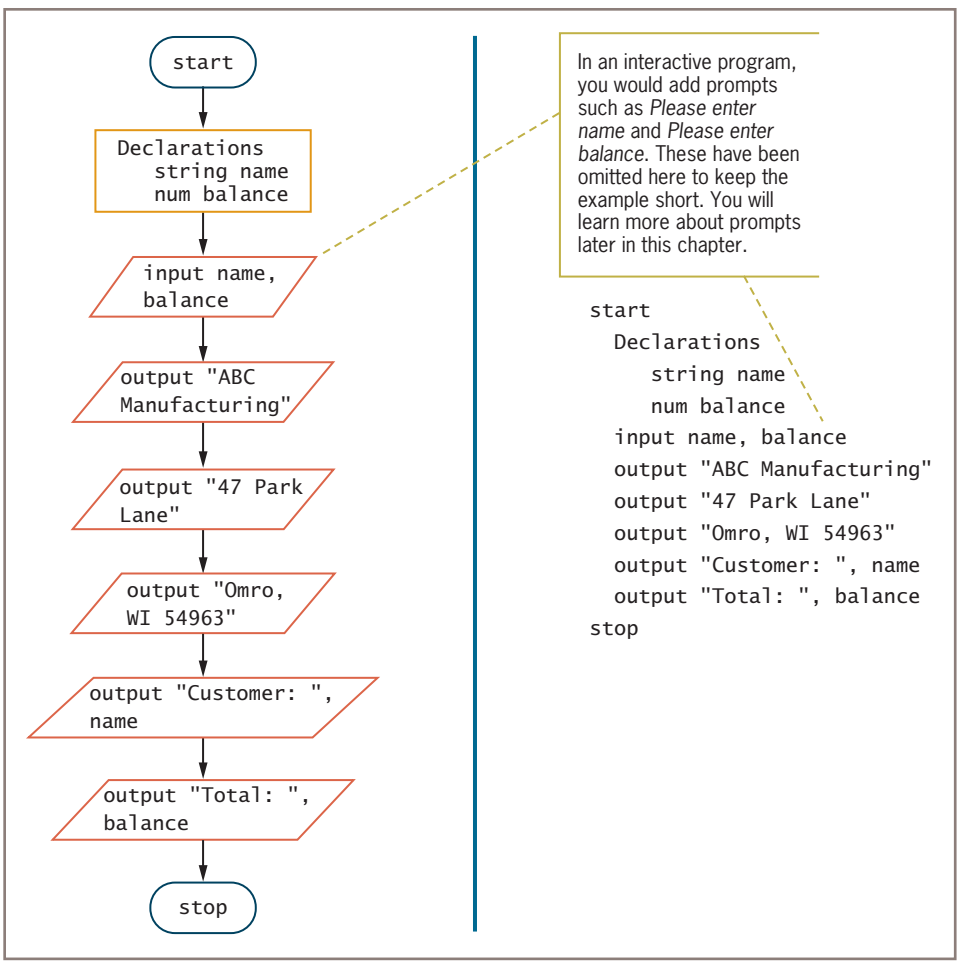


Figure 2-3 Program that produces a bill using only main program

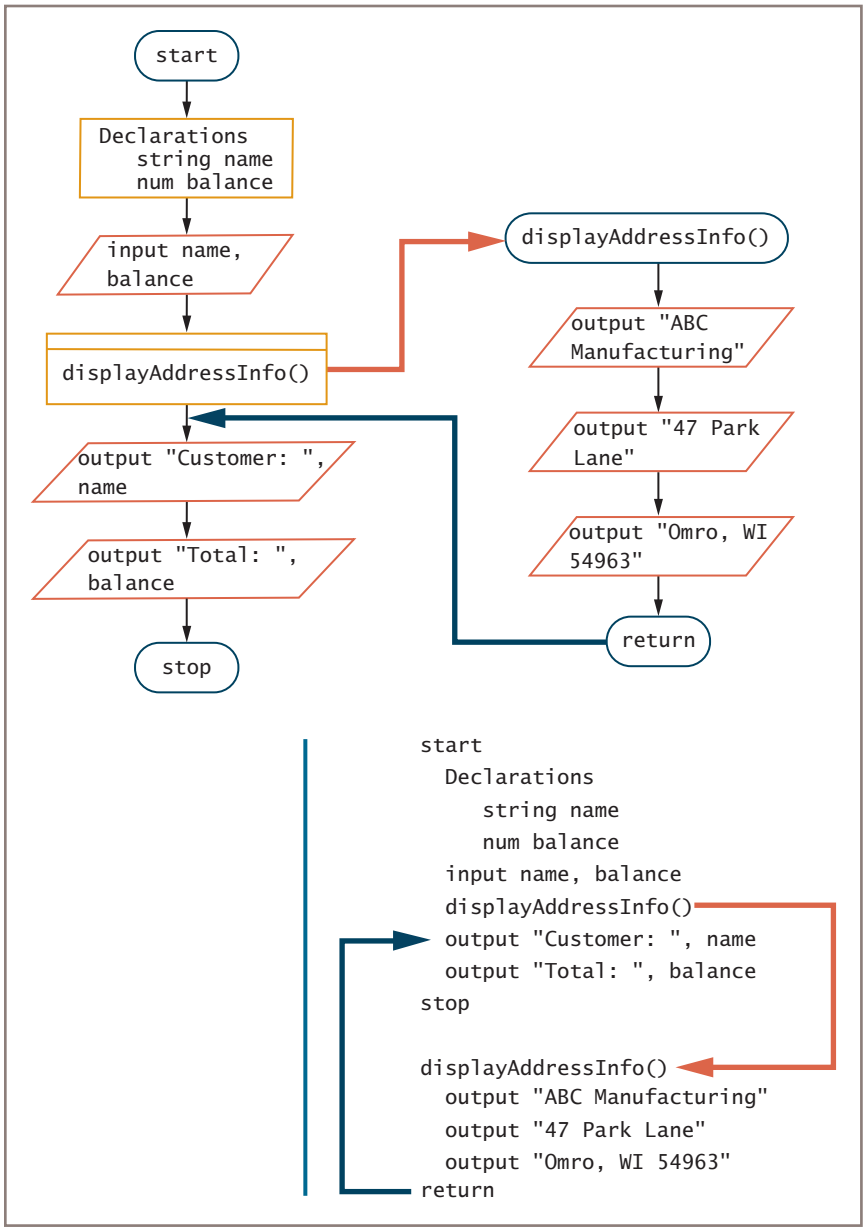


Figure 2-4 Program that produces a bill using main program that calls `displayAddressInfo()` module

When the `displayAddressInfo()` module is called in Figure 2-4, logic transfers from the main program to the `displayAddressInfo()` module, as shown by the large red arrow in both the flowchart and the pseudocode. There, each module statement executes in turn before logical control is transferred back to the main program, where it continues with the

statement that follows the module call, as shown by the large blue arrow. **Encapsulation** is the act of containing instructions in a module.

Neither of the programs in Figures 2-3 and 2-4 is superior to the other in terms of functionality; both perform exactly the same tasks in the same order. However, you may prefer the modularized version of the program for at least two reasons:

- First, the main program remains short and easy to follow because it contains just one statement to call the module, rather than three separate `output` statements to perform the work of the module.
- Second, a module is easy to reuse. After you create the address information module, you can use it in any application that needs the company's name and address. In other words, you do the work once, and then you can use the module many times.



A potential drawback to creating modules and moving between them is the overhead incurred. The computer keeps track of the correct memory address to which it should return after executing a module by recording the memory address in a location known as the **stack**. This process requires a small amount of computer time and resources. In most cases, the advantage to creating modules far outweighs the small amount of overhead required.

Determining when to modularize a program does not depend on a fixed set of rules; it requires experience and insight. Programmers do follow some guidelines when deciding how far to break down modules or how much to put in each of them. Some companies may have arbitrary rules, such as "a module's instructions should never take more than a page," or "a module should never have more than 30 statements," or "never have a module with only one statement." Rather than use such arbitrary rules, a better policy is to place together statements that contribute to one specific task. The more the statements contribute to the same job, the greater the **functional cohesion** of the module. A module that checks the validity of a date's value, or one that displays warranty information for a product, is considered cohesive. A module that checks date validity, deducts insurance premiums, and computes federal withholding tax for an employee would be less cohesive.



Chapter 9 of the comprehensive version of this book provides more information about designing modules for high cohesion. It also explores the topic of *coupling*, which is a measure of how much modules depend on each other.



Watch the video *Modularizing a Program*.

Declaring Variables and Constants within Modules

You can place any statements within modules, including input, processing, and output statements. You also can include variable and constant declarations within modules. For example, you might decide to modify the billing program in Figure 2-4 so it looks like the

one in Figure 2-5. In this version of the program, three named constants that hold the three lines of company data are declared within the `displayAddressInfo()` module. (See shading.)

Variables and constants are usable only in the module in which they are declared. Programmers say the data items are **visible** or **in scope** only within the module in which they are declared. That means the program only recognizes them there. Programmers

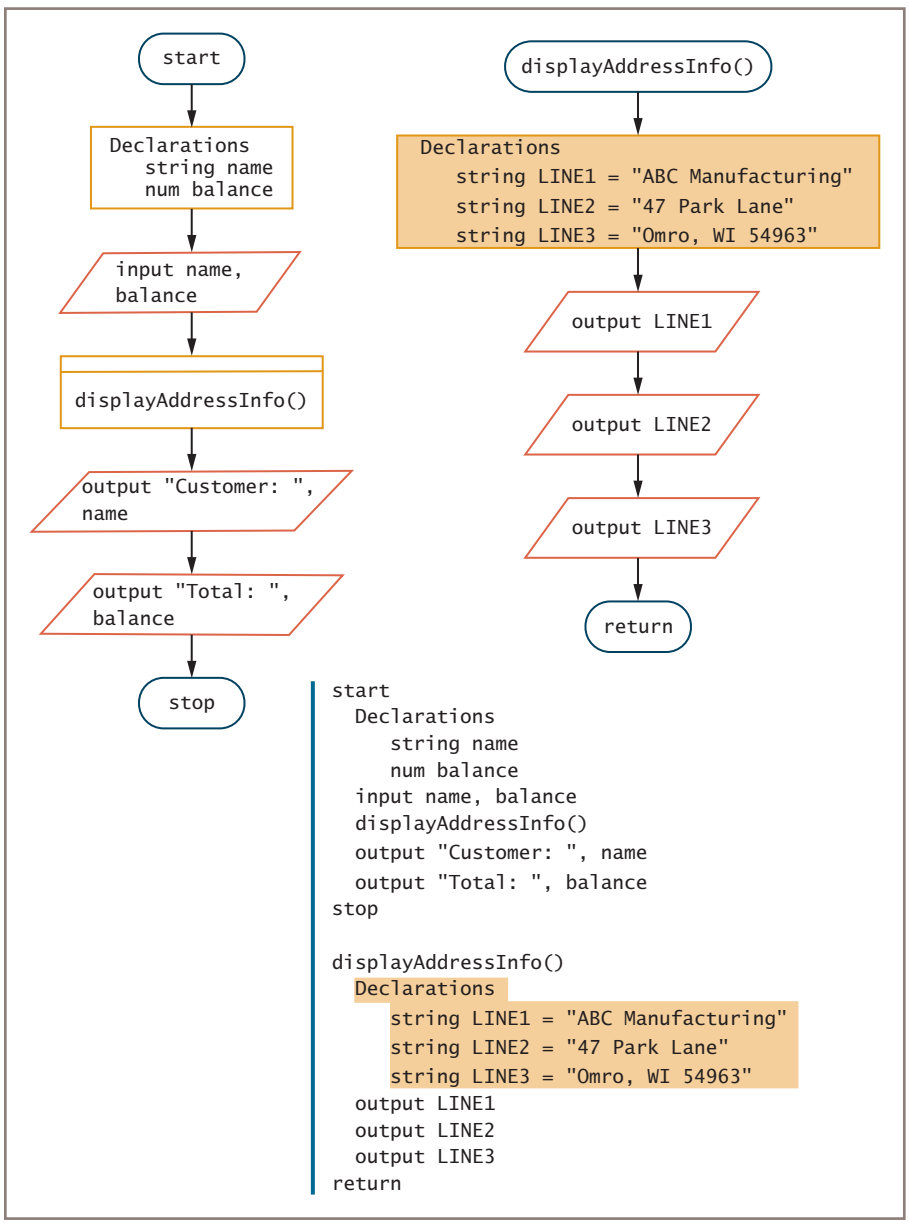


Figure 2-5 The billing program with constants declared within the module

also say that variables and constants are **local** to the module in which they are declared. In other words, when the strings `LINE1`, `LINE2`, and `LINE3` are declared in the `displayAddressInfo()` module in Figure 2-5, they are not recognized and cannot be used by the main module.

One of the motivations for creating modules is that separate modules are easily reusable in multiple programs. If the `displayAddressInfo()` module will be used by several programs within the organization, it makes sense that the definitions for its variables and constants must come with it. This makes the modules more **portable**; that is, they are self-contained units that are transported easily.

Besides local variables and constants, you can create global variables and constants. **Global** variables and constants are known to the entire program; they are said to be declared at the *program level*. That means they are visible to and usable in all the modules called by the program. The opposite is not true—variables and constants declared within a module are not usable elsewhere; they are visible only to that module.

In many modern programming languages, the main program itself is a module, so variables and constants declared there cannot be used elsewhere. To make the examples in this book easier to follow, variables and constants declared at the start of a main program will be considered global and usable in all modules. Until Chapter 9 in the comprehensive version, this book will use only global variables and constants so that you can concentrate on the main logic and not yet be concerned with the techniques necessary to make one module's data available to another. For example, in Figure 2-5, the main program variables `name` and `balance` are global variables and could be used by any module.



Many programmers do not approve of using global variables and constants. They are used here so you can more easily understand modularization before you learn the techniques of sending local variables from one module to another. Chapter 9 of the comprehensive version of this book will describe how you can make every variable local.

Understanding the Most Common Configuration for Mainline Logic

In Chapter 1, you learned that a procedural program contains procedures that follow one another in sequence. The mainline logic of almost every procedural computer program can follow a general structure that consists of three distinct parts:

1. **Housekeeping tasks** include any steps you must perform at the beginning of a program to get ready for the rest of the program. They can include tasks such as variable and constant declarations, displaying instructions to users, displaying report headings, opening any files the program requires, and inputting the first piece of data.



Inputting the first data item is always part of the housekeeping module. You will learn the theory behind this practice in Chapter 3. Chapter 7 covers file handling, including what it means to open and close a file.

2. **Detail loop tasks** do the core work of the program. When a program processes many records, detail loop tasks execute repeatedly for each set of input data until there are no more. For example, in a payroll program, the same set of calculations is executed repeatedly until a check has been produced for each employee.
3. **End-of-job tasks** are the steps you take at the end of the program to finish the application. You can call these finish-up or clean-up tasks. They might include displaying totals or other final messages and closing any open files.

Figure 2-6 shows the relationship of these three typical program parts. Notice how the `housekeeping()` and `endOfJob()` tasks are executed just once, but the `detailLoop()` tasks repeat as long as the `eof` condition has not been met. The flowchart uses a flowline to show how the `detailLoop()` module repeats; the pseudocode uses the words `while` and `endwhile` to contain statements that execute in a loop. You will learn more about the `while` and `endwhile` terms in subsequent chapters; for now, understand that they are a way of expressing repeated actions.

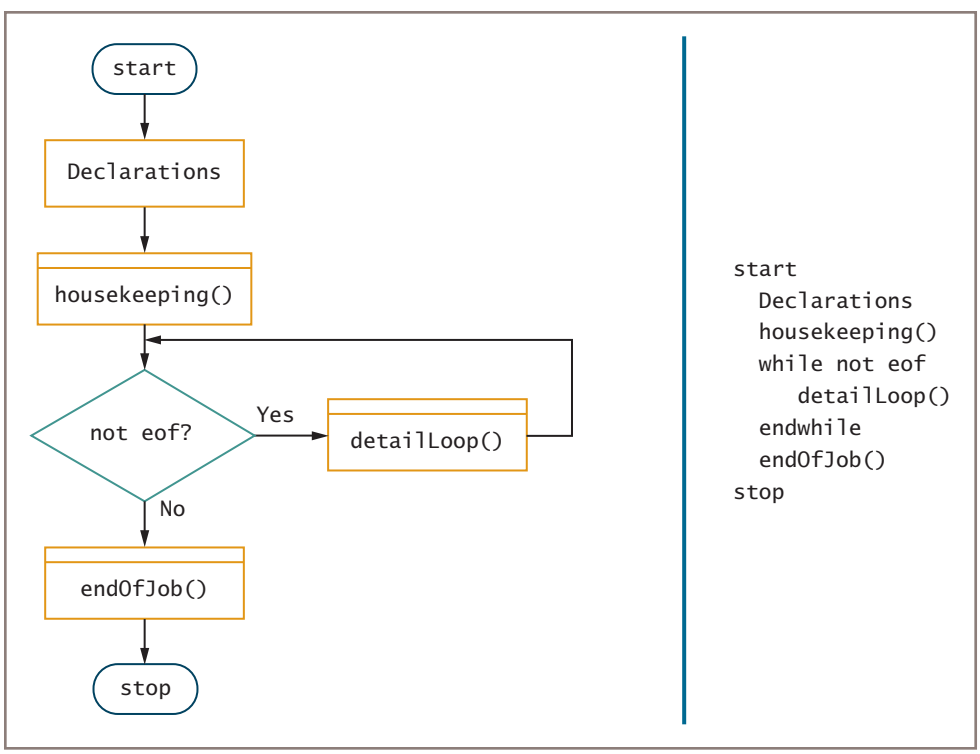


Figure 2-6 Flowchart and pseudocode of mainline logic for a typical procedural program

Many everyday tasks follow the three-module format just described. For example, a candy factory opens in the morning, and the machines are started and filled with ingredients. These housekeeping tasks occur just once at the start of the day. Then, repeatedly during the day, candy is manufactured. This process might take many steps, each of which occurs many times. These are the steps in the detail loop. Then, at the end of the day, the machines are cleaned and shut down. These are the end-of-job tasks.

Not all programs take the format of the logic shown in Figure 2-6, but many do. Keep this general configuration in mind as you think about how you might organize many programs. For example, Figure 2-7 shows a sample payroll report for a small company. A user enters employee names until there are no more to enter, at which point the user enters *XXX*. As long as the entered name is not *XXX*, the user enters the employee's weekly gross pay. Deductions are computed as a flat 25 percent of the gross pay, and the statistics for each employee are output. The user enters another name, and as long as it is not *XXX*, the process continues. Examine the logic in Figure 2-8 to identify the components in the housekeeping, detail loop, and end-of-job tasks. You will learn more about the payroll report program in the next few chapters. For now, concentrate on the big picture of how a typical application works.

Payroll Report

Name	Gross	Deductions	Net
Andrews	1000.00	250.00	750.00
Brown	1400.00	350.00	1050.00
Carter	1275.00	318.75	956.25
Young	1100.00	275.00	825.00

***End of report

Figure 2-7 Sample payroll report

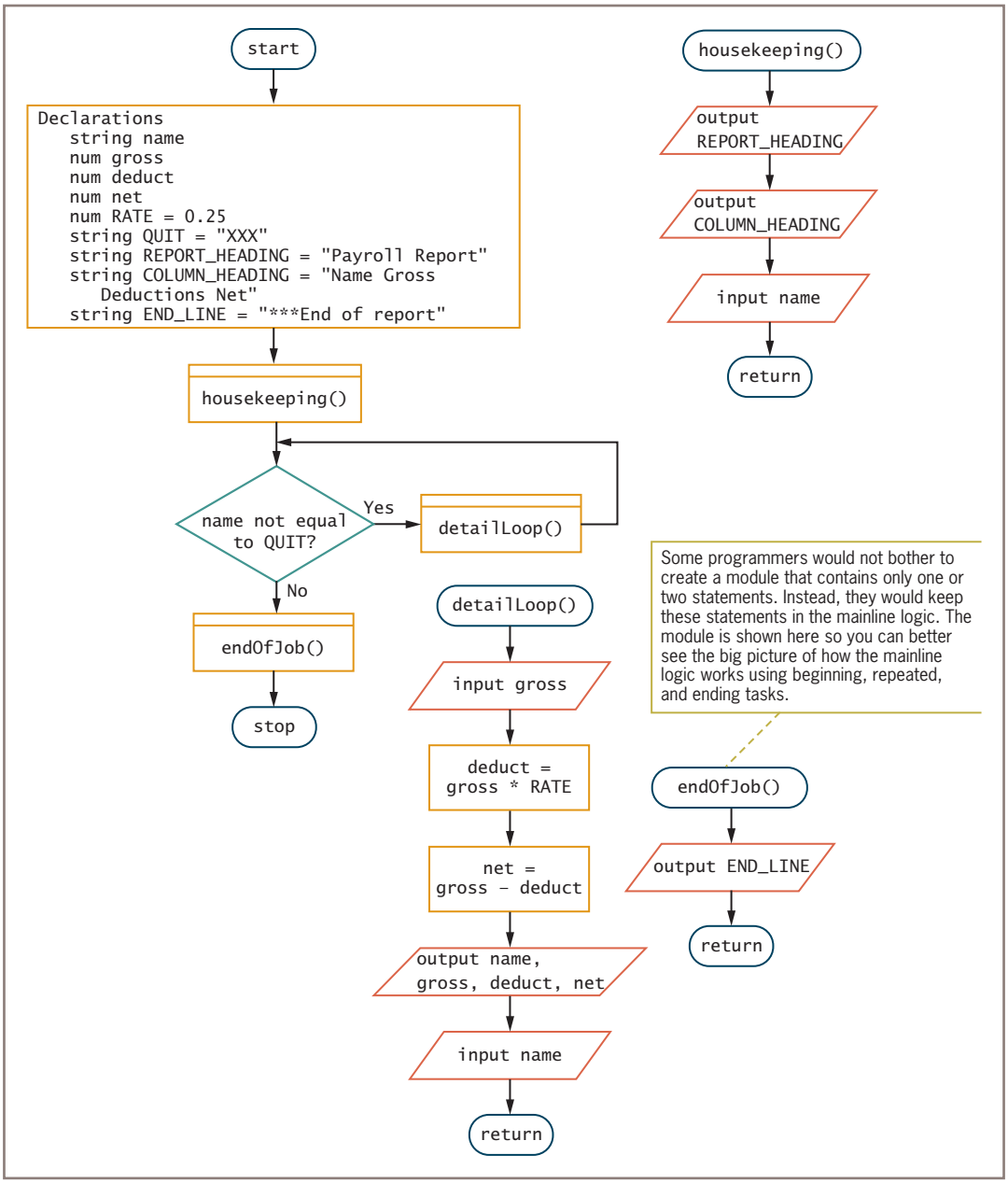


Figure 2-8 Logic for payroll report

TWO TRUTHS & A LIE

Modularizing a Program

1. A calling program calls a module's name when it wants to use the module.
2. Whenever a main program calls a module, the logic transfers to the module; when the module ends, the program ends.
3. Housekeeping tasks include any steps you must perform just once at the beginning of a program to get ready for the rest of the program.

The false statement is #2. When a module ends, the logical flow transfers back to the main calling module and resumes where it left off.

Creating Hierarchy Charts

You may have seen hierarchy charts for organizations, such as the one in Figure 2-9. The chart shows who reports to whom, not when or how often they report.

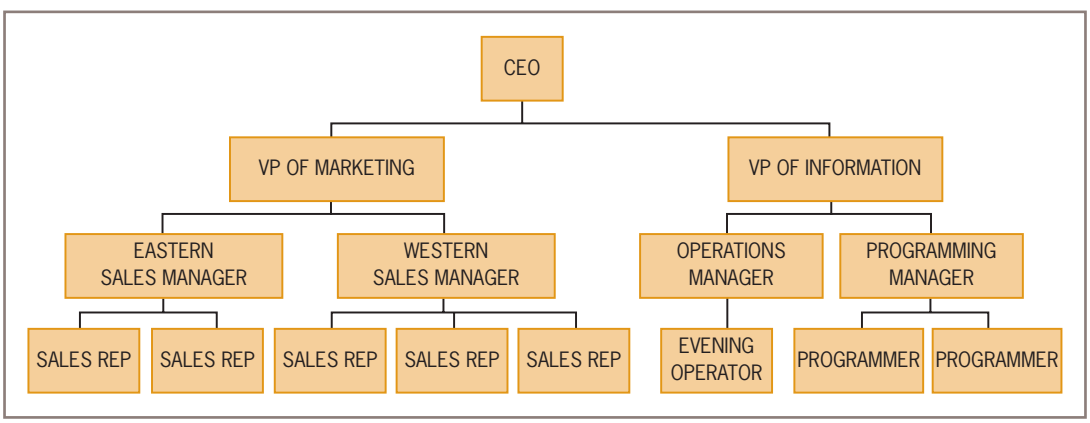


Figure 2-9 An organizational hierarchy chart

When a program has several modules calling other modules, programmers often use a program **hierarchy chart** (sometimes called a *structure chart*) that operates in a similar manner to show the overall picture of how modules are related to one another. A hierarchy chart does not tell you what tasks are to be performed *within* a module, *when* the modules are called, *how* a module executes, or *why* they are called—that information is in the

flowchart or pseudocode. A hierarchy chart tells you only *which* modules exist within a program and *which* modules call others. The hierarchy chart for the program in Figure 2-8 looks like Figure 2-10. It shows that the main module calls three others—`housekeeping()`, `detailLoop()`, and `endOfJob()`.

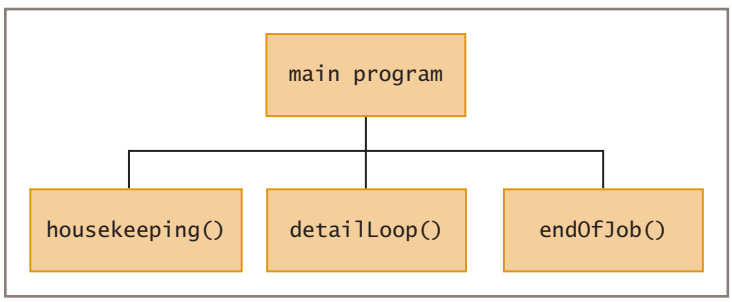


Figure 2-10 Hierarchy chart of payroll report program in Figure 2-8

Figure 2-11 shows an example of a hierarchy chart for the billing program of a mail-order company. The hierarchy chart is for a more complicated program, but like the payroll report chart in Figure 2-10, it supplies module names and a general overview of the tasks to be performed, without specifying any details.

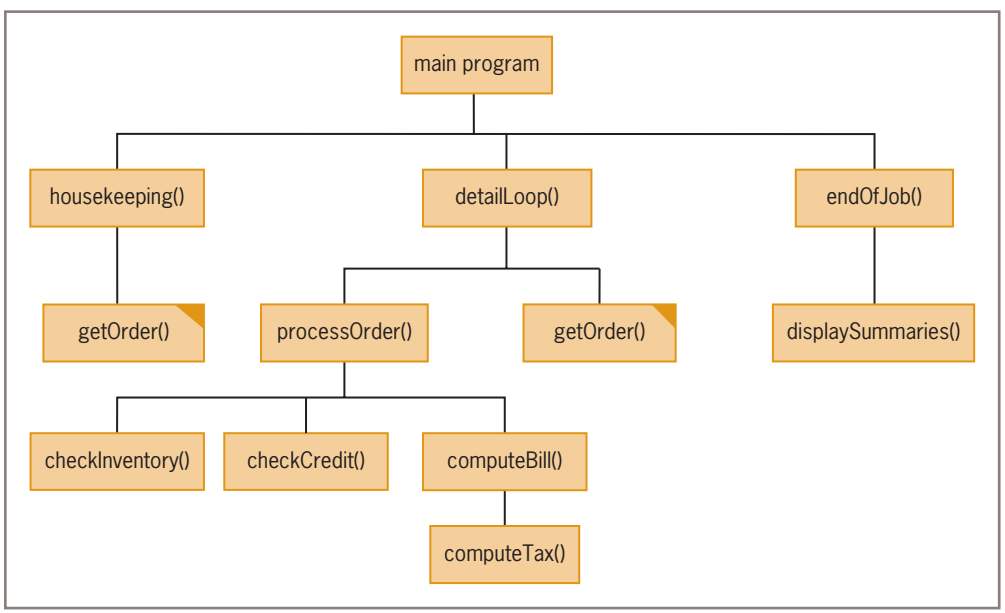


Figure 2-11 Billing program hierarchy chart

Because program modules are reusable, a specific module can be called from several locations within a program. For example, in the billing program hierarchy chart in Figure 2-11, you can see that the `getOrder()` module is used twice. By convention, you blacken a corner of each box that represents a module used more than once. This action alerts readers that any change to this module could have consequences in multiple locations.

A hierarchy chart can be both a planning tool for developing the overall relationship of program modules before you write them and a documentation tool to help others see how modules are related after a program is written. For example, if a tax law changes, a programmer might be asked to rewrite the `computeTax()` module in the billing program diagrammed in Figure 2-11. As the programmer changes the `computeTax()` module, the hierarchy chart shows other dependent modules that might be affected. A hierarchy chart is useful for getting the big picture in a complex program.



Hierarchy charts are used in procedural programming, but other types of diagrams frequently are used in object-oriented environments. For example, the Unified Modeling Language (UML) uses a set of diagrams to describe a system.

TWO TRUTHS & A LIE

Creating Hierarchy Charts

1. You can use a hierarchy chart to illustrate modules' relationships.
2. A hierarchy chart tells you what tasks are to be performed within a module.
3. A hierarchy chart tells you only which modules call other modules.

The false statement is #2. A hierarchy chart tells you nothing about tasks performed within a module; it only depicts how modules are related to each other.

Features of Good Program Design

As your programs become larger and more complicated, the need for good planning and design increases. Think of an application you use, such as a word processor or a spreadsheet. The number and variety of user options are staggering. Not only would it be impossible for a single programmer to write such an application, but without thorough planning and design, the components would never work together properly. Ideally, each program module you design needs to work well as a standalone component and as an element of larger systems. Just as a house with poor plumbing or a car with bad brakes is fatally flawed, a computer-based application can be highly functional only if each component is designed well. Walking through your program's logic on paper (called *desk-checking*, as you learned in Chapter 1) is an important step to achieving superior programs.

Additionally, you can implement several design features while creating programs that are easier to write and maintain. To create good programs, you should do the following:

- Provide program comments where appropriate.
- Choose identifiers thoughtfully.
- Strive to design clear statements within your programs and modules.
- Write clear prompts and echo input.
- Continue to maintain good programming habits as you develop your programming skills.

Using Program Comments

When you write programs, you often might want to insert program comments. **Program comments** are written explanations that are not part of the program logic but that serve as documentation for readers of the program. In other words, they are nonexecuting statements that help readers understand programming statements. Readers might include users who help you test the program and other programmers who might have to modify your programs in the future. Even you, as the program's author, will appreciate comments when you make future modifications and forget why you constructed a statement in a certain way.

The syntax used to create program comments differs among programming languages. This book starts comments in pseudocode with two forward slashes. For example, Figure 2-12 contains comments that explain the origins and purposes of variables in a real estate program.



Program comments are a type of **internal documentation**. This term distinguishes them from supporting documents outside the program, which are called **external documentation**.

```
Declarations
   num sqFeet
      // sqFeet is an estimate provided by the seller of the property
   num pricePerFoot
      // pricePerFoot is determined by current market conditions
   num lotPremium
      // lotPremium depends on amenities such as whether lot is waterfront
```

Figure 2-12 Pseudocode that declares variables and includes comments

In a flowchart, you can use an annotation symbol to hold information that expands on what is stored within another flowchart symbol. An **annotation symbol** is most often represented by a three-sided box that is connected to the step it references by a dashed line. Annotation symbols are used to hold comments or sometimes statements that are too long to fit neatly into a flowchart symbol. For example, Figure 2-13 shows how a programmer might use some annotation symbols in a flowchart for a payroll program.

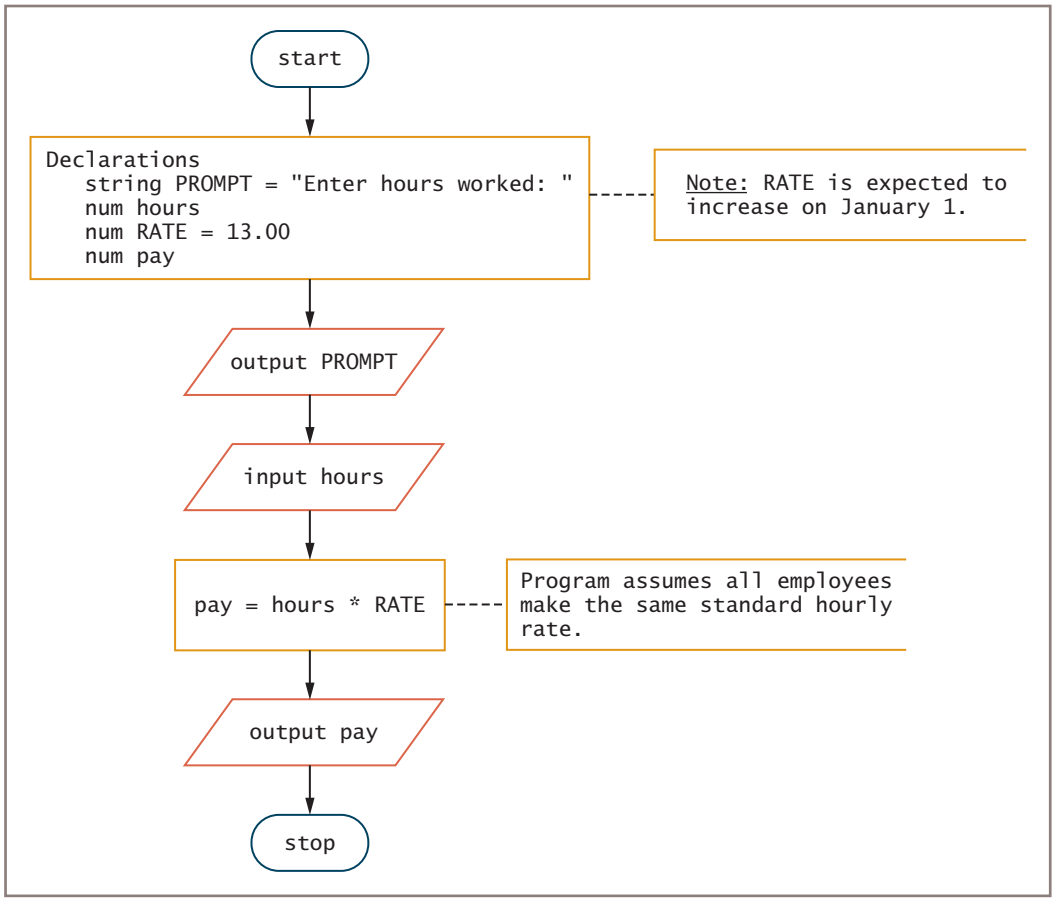


Figure 2-13 Flowchart that includes annotation symbols



You probably will use comments in your coded programs more frequently than you use them in pseudocode or flowcharts. For one thing, flowcharts and pseudocode are more English-like than the code in some languages, so your statements might be less cryptic. Also, your comments will remain in the program as part of the program documentation, but your planning tools are likely to be discarded when the program goes into production.

Including program comments is not necessary to create a working program, but comments can help you to remember the purpose of variables or to explain complicated calculations, especially when you come back to a program months or years after writing it. Some students do not like to include comments in their programs because it takes time to type them and they aren't part of the "real" program, but the programs you write in the future probably will require some comments. When you acquire your first programming job and modify a program written by another programmer, you will appreciate well-placed comments that explain complicated sections of the code.



An additional responsibility regarding comments is that they must be kept current as a program is modified. Outdated comments can provide misleading information about a program's status.

Choosing Identifiers

The selection of good identifiers is an often-overlooked element in program design. When you write programs, you choose identifiers for variables, constants, and modules. You learned the rules for naming variables and modules earlier in this chapter: Each must be a single word with no embedded spaces and must start with a letter. Those simple rules provide a lot of leeway in naming program elements, but not all identifiers are equally good. Choosing good identifiers simplifies your programming job and makes it easier for others to understand your work. Some general guidelines include the following:

- Although not required in any programming language, it usually makes sense to give a variable or constant a name that is a noun (or a combination of an adjective and noun) because it represents a thing—for example, `payRate`. Similarly, it makes sense to give a module an identifier that is a verb, or a combined verb and noun, because a module takes action—for example, `computePayRate()`.
- Use meaningful names. Creating a data item named `someData` or a module named `firstModule()` makes a program cryptic. Not only will others find it hard to read your programs, but also you will forget the purpose of these identifiers even within your own programs. All programmers occasionally use short, nondescriptive names such as `x` or `temp` in a quick program; however, in most cases, data and module names should be meaningful. Programmers refer to programs that contain meaningful names as **self-documenting**. This means that even without further documentation, the program code explains itself to readers.
- Use pronounceable names. A variable name like `pzf` is neither pronounceable nor meaningful. A name that looks meaningful when you write it might not be as meaningful when someone else reads it; for instance, `preparead()` might mean "Prepare ad" to you, but "Prep a read" to others. Look at your names critically to make sure they can be pronounced. Very standard abbreviations do not have to be pronounceable. For example, most businesspeople would interpret ssn as a Social Security number.
- Don't forget that not all programmers share your culture. An abbreviation whose meaning seems obvious to you might be cryptic to someone in a different part of the world, or even a different part of your country. For example, you might name a variable `roi` to hold a value for *return on investment*, but a French-speaking person might interpret the meaning as *king*.
- Be judicious in your use of abbreviations. You can save a few keystrokes when creating a module called `getStat()`, but is the module's purpose to find the state in which a city is located, input some statistics, or determine the status of some variables? Similarly, is a variable named `fn` meant to hold a first name, file number, or something else? Abbreviations can also confuse people in different lines of work: AKA might suggest a sorority (Alpha Kappa Alpha) to a college administrator, a registry (American Kennel Association) to a dog breeder, or an alias (*also known as*) to a police detective.



To save typing time when you develop a program, you can use a short name like `efn`. After the program operates correctly, you can use a text editor's Search and Replace feature to replace your coded name with a more meaningful name such as `employeeFirstName`. When working in an integrated development environment, you can use the technique known as *refactoring* to rename every instance of an identifier.



Many IDEs support an automatic statement-completion feature that saves typing time. After the first time you use a name like `employeeFirstName`, you need to type only the first few letters before the compiler editor offers a list of available names from which to choose. The list is constructed from all the names you have used that begin with the same characters.

- Usually, avoid digits in a name. A zero can be confused with the letter *O*, and the lowercase letter *l* is misread as the numeral 1. Of course, use your judgment: `budgetFor2018` probably will not be misinterpreted.
- Use the rules your language allows to separate words in long, multiword variable names. For example, if the programming language you use allows hyphens or underscores, then use a module name like `initialize-data()` or `initialize_data()`, which is easier to read than `initializedata()`. Another option is to use camel casing to create an identifier such as `initializeData()`. If you use a language that is case sensitive, it is legal but confusing to use variable names that differ only in case. For example, if a single program contains `empName`, `EmpName`, and `Empname`, confusion is sure to follow.
- Consider including a form of the verb *to be*, such as *is* or *are*, in names for variables that are intended to hold a status. For example, use `isFinished` as a string variable that holds a *Y* or *N* to indicate whether a file is exhausted. The shorter name `finished` is more likely to be confused with a module that executes when a program is done. (Many languages support a Boolean data type, which you assign to variables meant to hold only true or false. Using a form of *to be* in identifiers for Boolean variables is appropriate.)
- Many programmers follow the convention of naming constants using all uppercase letters, inserting underscores between words for readability. In this chapter you saw examples such as `SALES_TAX_RATE`.
- Organizations sometimes enforce different rules for programmers to follow when naming program components. It is your responsibility to find out the conventions used in your organization and to adhere to them.



Programmers sometimes create a **data dictionary**, which is a list of every variable name used in a program, along with its type, size, and description. When a data dictionary is created, it becomes part of the program documentation.

When you begin to write programs, the process of determining what variables, constants, and modules you need and what to name them all might seem overwhelming. The design process, however, is crucial. When you acquire your first professional programming assignment, the design process might very well be completed already. Most likely, your first assignment will be to write or modify one small member module of a much larger

application. The more the original programmers adhered to naming guidelines, the better the original design was, and the easier your job of modification will be.

Designing Clear Statements

In addition to using program comments and selecting good identifiers, you can use the following tactics to contribute to the clarity of the statements within your programs:

- Avoid confusing line breaks.
- Use temporary variables to clarify long statements.

Avoiding Confusing Line Breaks

Some older programming languages require that program statements be placed in specific columns. Most modern programming languages are free-form; you can arrange your lines of code any way you see fit. As in real life, with freedom comes responsibility; when you have flexibility in arranging your lines of code, you must take care to make sure your meaning is clear. With free-form code, programmers are allowed to place two or three statements on a line, or, conversely, to spread a single statement across multiple lines. Both make programs harder to read. All the pseudocode examples in this book use appropriate, clear spacing and line breaks.

Using Temporary Variables to Clarify Long Statements

When you need several mathematical operations to determine a result, consider using a series of temporary variables to hold intermediate results. A **temporary variable** (or **work variable**) is not used for input or output, but instead is just a working variable that you use during a program's execution. For example, Figure 2-14 shows two ways to calculate a value for a real estate `salespersonCommission` variable. Each example achieves the same result—the salesperson's commission is based on the square feet multiplied by the price per square foot, plus any premium for a lot with special features, such as a wooded or waterfront lot. However, the second example uses two temporary variables: `basePropertyPrice` and `totalSalePrice`. When the computation is broken down into less complicated, individual steps, it is easier to see how the total price is calculated. In calculations with even more computation steps, performing the arithmetic in stages would become increasingly helpful.

```
//Using a single statement to compute commission
salespersonCommission = (sqFeet * pricePerFoot + lotPremium) * commissionRate

// Using multiple  statements to compute commision
basePropertyPrice = sqFeet * pricePerFoot
totalSalePrice = basePropertyPrice + lotPremium
salespersonCommission = totalSalePrice * commissionRate
```

Figure 2-14 Two ways of achieving the same `salespersonCommission` result



Programmers might say using temporary variables, like the second example in Figure 2-14, is *cheap*. When executing a lengthy arithmetic statement, even if you don't explicitly name temporary variables, the programming language compiler creates them behind the scenes (although without descriptive names), so declaring them yourself does not cost much in terms of program execution time.

Writing Clear Prompts and Echoing Input

When program input should be retrieved from a user, you almost always want to provide a prompt for the user. A **prompt** is a message that is displayed on a monitor to ask the user for a response and perhaps explain how that response should be formatted. Prompts are used both in command-line and GUI interactive programs.

For example, suppose a program asks a user to enter a catalog number for an item the user is ordering. The following prompt is not very helpful:

```
Please enter a number.
```

The following prompt is more helpful:

```
Please enter a five-digit catalog item number.
```

The following prompt is even more helpful:

```
The five-digit catalog item number appears to the right of the item's picture
in the catalog. Please enter it now without any embedded spaces.
```

When program input comes from a stored file instead of a user, prompts are not needed. When a program expects a user response, however, prompts are valuable. For example, Figure 2-15 shows the flowchart and pseudocode for the beginning of the bill-producing program shown earlier in this chapter. If the input was coming from a data file, no prompt would be required, and the logic might look like the logic in Figure 2-15.

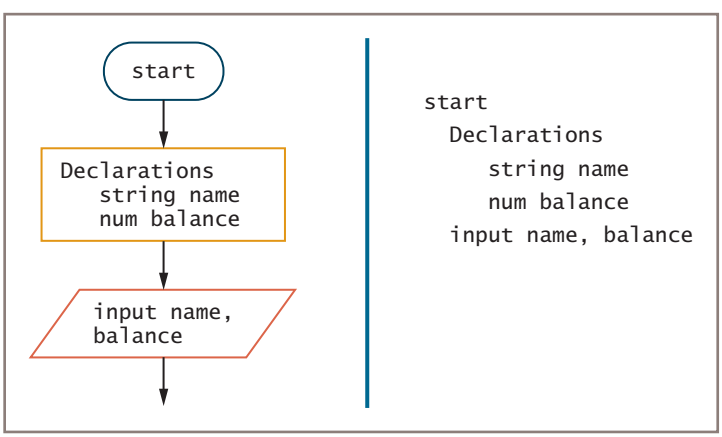


Figure 2-15 Beginning of a program that accepts a name and balance as input

However, if the input was coming from a user, including prompts would be helpful. You could supply a single prompt such as *Please enter a customer's name and balance due*, but inserting more requests into a prompt generally makes it less likely that the user can remember to enter all the parts or enter them in the correct order. It is almost always best to include a separate prompt for each item to be entered. Figure 2-16 shows an example.

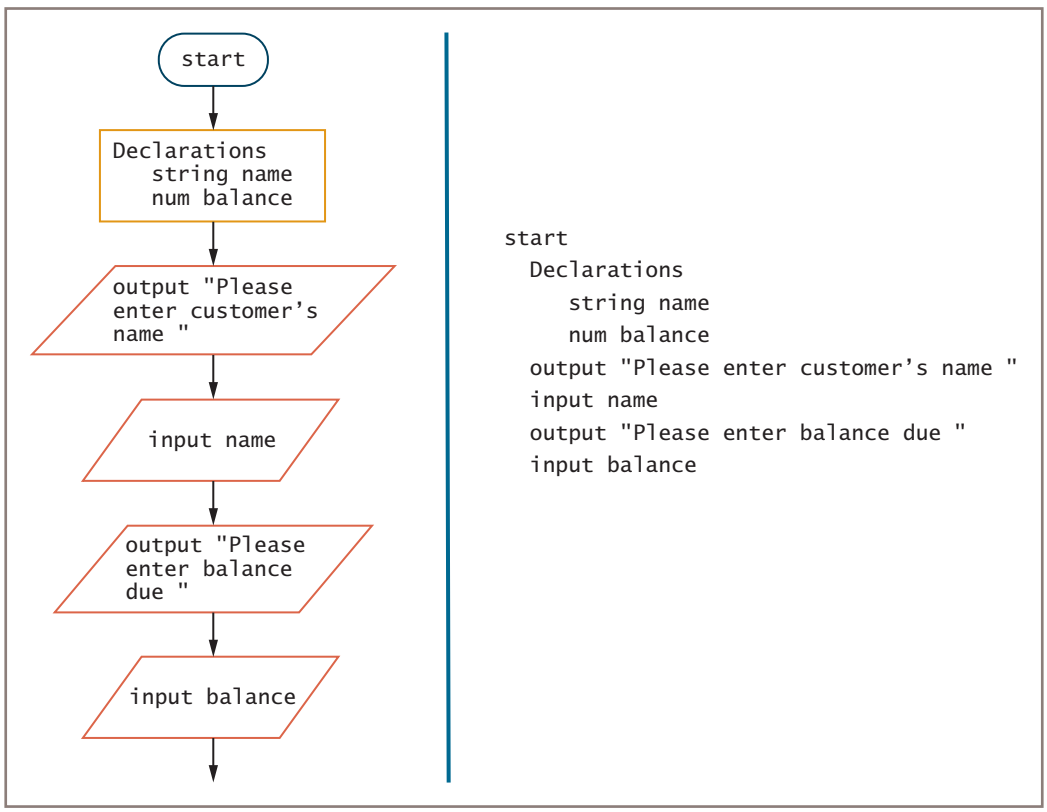


Figure 2-16 Beginning of a program that accepts a name and balance as input and uses a separate prompt for each item

Users also find it helpful when you echo their input. **Echoing input** is the act of repeating input back to a user either in a subsequent prompt or in output. For example, Figure 2-17 shows how the second prompt in Figure 2-16 can be improved by echoing the user's first piece of input data in the second prompt. When a user runs the program that is started in Figure 2-17 and enters `Green` for the customer name, the second prompt will not be *Please enter balance due*. Instead, it will be *Please enter balance due for Green*. For example, if a clerk was about to enter the balance for the wrong customer, the mention of *Green* might be enough to alert the clerk to the potential error.

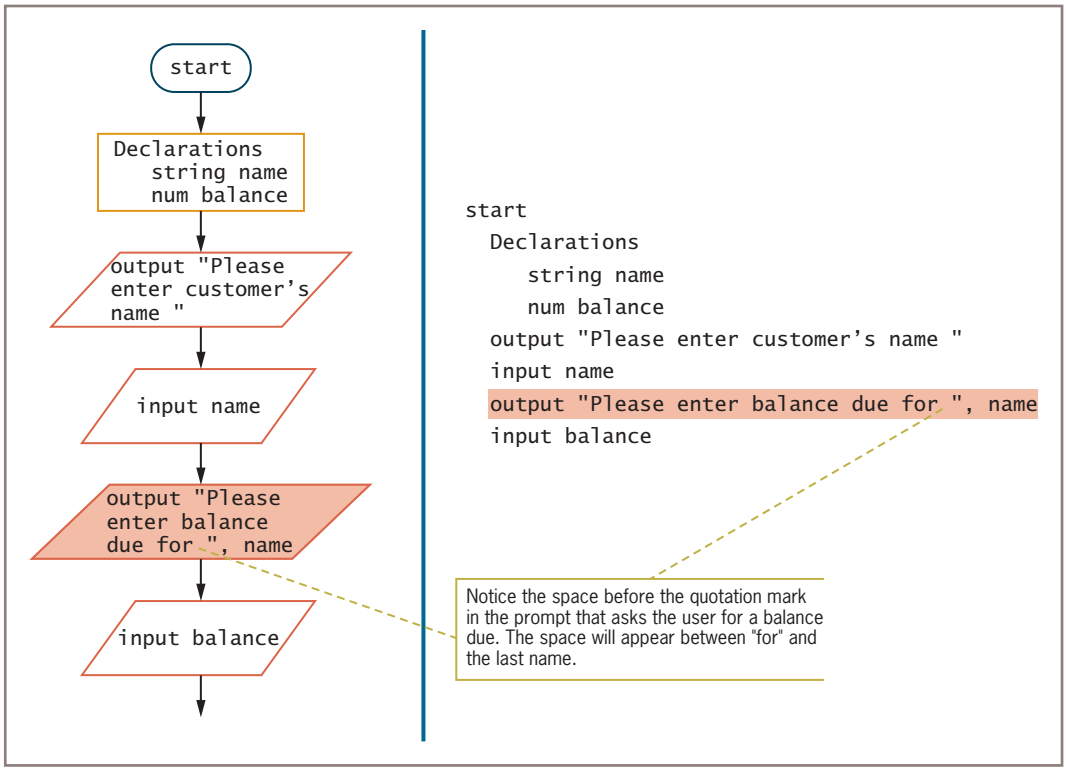


Figure 2-17 Beginning of a program that accepts a customer's name and uses it in the second prompt

Maintaining Good Programming Habits

When you learn a programming language and begin to write lines of program code, it is easy to forget the principles you have learned in this text. Having some programming knowledge and a keyboard at your fingertips can lure you into typing lines of code before you think things through. But every program you write will be better if you plan before you code. Maintaining the habits of first drawing flowcharts or writing pseudocode, as you have learned here, will make your future programming projects go more smoothly. If you desk-check your program logic on paper before coding statements in a programming language, your programs will run correctly sooner. If you think carefully about the variable and module names you choose, and design program statements to be easy to read and use, your programs will be easier to develop and maintain.

TWO TRUTHS & A LIE

Features of Good Program Design

1. A program comment is a message that is displayed on a monitor to ask the user for a response and perhaps explain how that response should be formatted.
2. It usually makes sense to give each variable a name that contains a noun and to give each module a name that contains a verb.
3. Echoing input can help a user to confirm that a data item was entered correctly.

The false statement is #1. A program comment is a written explanation that is not part of the program logic but that serves as documentation for those reading the program. A prompt is a message that is displayed on a monitor to ask the user for a response and perhaps explain how that response should be formatted.

Chapter Summary

- Programs contain data in three different forms: literals (or unnamed constants), variables, and named constants. Each of these types of data can be numeric or string. Variables are named memory locations, the contents of which can vary. A variable declaration includes a data type and an identifier; optionally, it can include an initialization. Every computer programming language has its own set of rules for naming variables; however, all variable names must be written as one word without embedded spaces and should have appropriate meaning. A named constant is similar to a variable, except it can be assigned a value only once.
- Most programming languages use +, −, *, and / as the four standard arithmetic operators. Every operator follows rules of precedence that dictate the order in which operations in the same statement are carried out; multiplication and division always take precedence over addition and subtraction. The rules of precedence can be overridden using parentheses.
- Programmers break down programming problems into smaller, cohesive units called modules, subroutines, procedures, functions, or methods. To execute a module, you call it from another program or module. Any program can contain an unlimited number of modules, and each module can be called an unlimited number of times. Modularization provides abstraction, allows multiple programmers to work on a problem, and makes it easier for you to reuse work.

- When you create a module, you include a header, a body, and a `return` statement. A program or module calls a module's name to execute it. You can place any statements within modules, including declarations, which are local to the module. Global variables and constants are those that are known to the entire program. The mainline logic of almost every procedural computer program can follow a general structure that consists of three distinct parts: housekeeping tasks, detail loop tasks, and end-of-job tasks.
- A hierarchy chart illustrates modules and their relationships; it indicates which modules exist within a program and which modules call others.
- As programs become larger and more complicated, the need for good planning and design increases. You should use program comments where appropriate. Choose identifiers wisely, strive to design clear statements within your programs and modules, write clear prompts and echo input, and continue to maintain good programming habits as you develop your programming skills.

Key Terms

A data item's **data type** is a classification that describes what values can be assigned, how the item is stored, and what types of operations can be performed with the item.

Numeric describes data that consists of numbers and can be used in arithmetic operations.

String describes data that is nonnumeric.

An **integer** is a whole number.

A **floating-point** number is a number with decimal places.

Real numbers are floating-point numbers.

A **numeric constant** (or **literal numeric constant**) is a specific numeric value.

A **string constant** (or **literal string constant**) is a specific group of characters enclosed within quotation marks.

Alphanumeric values can contain alphabetic characters, numbers, and punctuation.

An **unnamed constant** is a literal numeric or string value.

A **declaration** is a statement that provides a data type, an identifier, and, optionally, an initial value.

A **numeric variable** is one that can hold digits, have mathematical operations performed on it, and usually can hold a decimal point and a sign indicating positive or negative.

A **string variable** can hold text that includes letters, digits, and special characters such as punctuation marks.

Type-safety is the feature of some programming languages that prevents assigning values of an incorrect data type.

An **identifier** is a program component's name.

Keywords comprise the limited word set that is reserved in a language.

Camel casing is a naming convention in which the initial letter is lowercase, multiple-word names are run together, and each new word within the name begins with an uppercase letter.

Lower camel casing is another name for the *camel casing* naming convention.

Pascal casing is a naming convention in which the initial letter is uppercase, multiple-word names are run together, and each new word within the name begins with an uppercase letter.

Upper camel casing is another name for the *Pascal casing* naming convention.

Hungarian notation is a naming convention in which a data type or other information is stored as part of a name.

Snake casing is a convention in which parts of a name are separated by underscores.

Mixed case with underscores is a naming convention similar to snake casing, but new words start with an uppercase letter.

Kebob case is sometimes used as the name for the style that uses dashes to separate parts of a name.

An **assignment statement** assigns a value from the right of an assignment operator to the variable or constant on the left of the assignment operator.

The **assignment operator** is the equal sign; it is used to assign a value to the variable or constant on its left.

A **binary operator** is an operator that requires two operands—one on each side.

An **operand** is a value used by an operator.

Right-associativity and right-to-left associativity describe operators that evaluate the expression to the right first.

An **lvalue** is the memory address identifier to the left of an assignment operator.

Initializing a variable is the act of assigning its first value, often at the same time the variable is declared.

Garbage describes the unknown value stored in an unassigned variable.

A **named constant** is similar to a variable, except that its value cannot change after the first assignment.

A **magic number** is an unnamed constant whose purpose is not immediately apparent.

Overhead describes the extra resources a task requires.

Rules of precedence dictate the order in which operations in the same statement are carried out.

The **order of operations** describes the rules of precedence.

Left-to-right associativity describes operators that evaluate the expression to the left first.

The **remainder operator** is an arithmetic operator used in some programming languages; when used with two integer operands, it results in the remainder after division.

Modules are small program units that you can use together to make a program. Programmers also refer to modules as **subroutines**, **procedures**, **functions**, or **methods**.

To **call a module** is to use the module's name to invoke it, causing it to execute.

Modularization is the process of breaking down a program into modules.

Functional decomposition is the act of reducing a large program into more manageable modules.

Abstraction is the process of paying attention to important properties while ignoring nonessential details.

Reusability is the feature of modular programs that allows individual modules to be used in a variety of applications.

Reliability is the feature of modular programs that assures you a module has been tested and proven to function correctly.

A **main program** runs from start to stop and calls other modules.

The **mainline logic** is the logic that appears in a program's main module; it calls other modules.

The **module header** includes the module identifier and possibly other necessary identifying information.

The **module body** contains all the statements in the module.

The **module return statement** marks the end of the module and identifies the point at which control returns to the program or module that called the module.

Encapsulation is the act of containing a task's instructions in a module.

A **stack** is a memory location in which the computer keeps track of the correct memory address to which it should return after executing a module.

The **functional cohesion** of a module is a measure of the degree to which all the module statements contribute to the same task.

Visible describes data items when a module can recognize them. **In scope** describes data that is visible.

Local describes variables that are declared within the module that uses them.

A **portable** module is one that can be reused easily in multiple programs.

Global describes variables that are known to an entire program.

Housekeeping tasks include steps you must perform at the beginning of a program to get ready for the rest of the program.

Detail loop tasks of a program include the steps that are repeated for each set of input data.

End-of-job tasks hold the steps you take at the end of the program to finish the application.

A **hierarchy chart** is a diagram that illustrates modules' relationships to each other.

Program comments are written explanations that are not part of the program logic but that serve as documentation for those reading the program.

Internal documentation is documentation within a coded program.

External documentation is documentation that is outside a coded program.

An **annotation symbol** contains information that expands on what appears in another flowchart symbol; it is most often represented by a three-sided box that is connected to the step it references by a dashed line.

Self-documenting programs are those that contain meaningful identifiers that describe their purpose.

A **data dictionary** is a list of every variable name used in a program, along with its type, size, and description.

A **temporary variable** (or **work variable**) is a variable that you use to hold intermediate results during a program's execution.

A **prompt** is a message that is displayed on a monitor to ask the user for a response and perhaps explain how that response should be formatted.

Echoing input is the act of repeating input back to a user either in a subsequent prompt or in output.

Exercises



Review Questions

1. What does a declaration provide for a variable?

   a. a name
   b. a data type
   c. both of the above
   d. none of the above

2. A variable's data type describes all of the following *except*_____.

   a. what values the variable can hold
   b. the scope of the variable
   c. how the variable is stored in memory
   d. what operations can be performed with the variable

3. The value stored in an uninitialized variable is _____.

   a. null
   b. garbage
   c. compost
   d. its identifier

4. The value 3 is a _____.

   a. numeric variable
   b. string variable
   c. numeric constant
   d. string constant

5. The assignment operator _____.

   a. is a binary operator
   b. has left-to-right associativity
   c. is most often represented by a colon
   d. two of the above

6. Multiplication has a lower precedence than _____.

   a. division
   b. subtraction
   c. assignment
   d. none of the above

7. Which of the following is not a term used as a synonym for *module*?

   a. method
   b. object
   c. procedure
   d. subroutine

8. Modularization _____.

   a. eliminates abstraction
   b. reduces overhead
   c. facilitates reusability
   d. increases the need for correct syntax

9. What is the name for the process of paying attention to important properties while ignoring nonessential details?

   a. abstraction
   b. extraction
   c. extinction
   d. modularization

10. Every module has all of the following *except*_____ .

    a. header
    b. local variables
    c. a body
    d. a `return` statement

11. Programmers say that one module can _____ another, meaning that the first module causes the second module to execute.

    a. declare
    b. define
    c. enact
    d. call

12. The more that a module's statements contribute to the same job, the greater the _____ of the module.

    a. structure
    b. modularity
    c. functional cohesion
    d. size

13. In most modern programming languages, a variable or constant that is declared in a module is _____ in that module.

    a. global
    b. invisible
    c. in scope
    d. undefined

14. Which of the following is *not* a typical housekeeping task?

    a. displaying instructions
    b. printing summaries
    c. opening files
    d. displaying report headings

15. Which module in a typical program will execute the most times?

    a. the housekeeping module
    b. the detail loop
    c. the end-of-job module
    d. It is different in every program.

16. A hierarchy chart tells you _____.

    a. which modules call other modules
    b. what tasks are to be performed within each program module
    c. when a module executes
    d. all of the above

17. What are nonexecuting statements that programmers place within code to explain program statements in English?

    a. pseudocode
    b. trivia
    c. user documentation
    d. comments

18. Program comments are _____.

    a. required to create a runnable program
    b. a form of external documentation
    c. both of the above
    d. none of the above

19. Which of the following is valid advice for naming variables?

    a. To save typing, make most variable names one or two letters.
    b. To avoid conflict with names that others are using, use unusual or unpronounceable names.
    c. To make names easier to read, separate long names by using underscores or capitalization for each new word.
    d. To maintain your independence, shun the conventions of your organization.

20. A message that asks a user for input is a(n) _____.

a. comment
b. echo
c. prompt
d. declaration



Programming Exercises

1. Explain why each of the following names does or does not seem like a good variable name to represent a state sales tax rate.

```
a. stateTaxRate
b. txRt
c. t
d. stateSalesTaxRateValue
e. state tax rate
f. taxRate
g. 1TaxRate
h. moneyCharged
```

2. If `productCost` and `productPrice` are numeric variables, and `productName` is a string variable, which of the following statements are valid assignments? If a statement is not valid, explain why not.

```
a. productCost = 100
b. productPrice = productCost
c. productPrice = productName
d. productPrice = "24.95"
e. 15.67 = productCost
f. productCost = $1,345.52
g. productCost = productPrice - 10
h. productName = "mouse pad"
i. productCost + 20 = productPrice
j. productName = 3-inch nails
k. productName = 43
l. productName = "44"
```

```
m. "99" = productName

n. productName = brush

o. battery = productName

p. productPrice = productPrice

q. productName = productCost
```

3. Assume that `speed = 10` and `miles = 5`. What is the value of each of the following expressions?

```
a. speed + 12 - miles * 2

b. speed + miles * 3

c. (speed + miles) * 3

d. speed + speed * miles + miles

e. (10 - speed) + miles / miles
```

4. Draw a typical hierarchy chart for a program that produces a monthly bill for a cellphone customer. Try to think of at least 10 separate modules that might be included. For example, one module might calculate the charge for daytime phone minutes used.

5. a. Draw the hierarchy chart and then plan the logic for a program needed by Hometown Bank. The program determines a monthly checking account fee. Input includes an account balance and the number of times the account was overdrawn. The output is the fee, which is 1 percent of the balance minus 5 dollars for each time the account was overdrawn. Use three modules. The main program declares global variables and calls housekeeping, detail, and end-of-job modules. The housekeeping module prompts for and accepts a balances. The detail module prompts for and accepts the number of overdrafts, computes the fee, and displays the result. The end-of-job module displays the message *Thanks for using this program.*

   b. Revise the banking program so that it runs continuously for any number of accounts. The detail loop executes continuously while the balance entered is not negative; in addition to calculating the fee, it prompts the user for and gets the balance for the next account. The end-of-job module executes after a number less than 0 is entered for the account balance.

6. a. Draw the hierarchy chart and then plan the logic for a program that calculates a person's body mass index (BMI). BMI is a statistical measure that compares a person's weight and height. The program uses three modules. The first prompts a user for and accepts the user's height in inches. The second module accepts the user's weight in pounds and converts the user's height to meters and weight to kilograms. Then, it calculates BMI as weight

in kilograms divided by height in meters squared, and displays the results. There are 2.54 centimeters in an inch, 100 centimeters in a meter, 453.59 grams in a pound, and 1,000 grams in a kilogram. Use named constants whenever you think they are appropriate. The last module displays the message *End of job*.

b. Revise the BMI-determining program to execute continuously until the user enters 0 for the height in inches.

7. Draw the hierarchy chart and design the logic for a program that calculates service charges for Hazel's Housecleaning service. The program contains housekeeping, detail loop, and end-of-job modules. The main program declares any needed global variables and constants and calls the other modules. The housekeeping module displays a prompt for and accepts a customer's last name. While the user does not enter *ZZZZ* for the name, the detail loop accepts the number of bathrooms and the number of other rooms to be cleaned. The service charge is computed as $40 plus $15 for each bathroom and $10 for each of the other rooms. The detail loop also displays the service charge and then prompts the user for the next customer's name. The end-of-job module, which executes after the user enters the sentinel value for the name, displays a message that indicates the program is complete.

8. Draw the hierarchy chart and design the logic for a program that calculates the projected cost of a remodeling project. Assume that the labor cost is $30 per hour. Design a program that prompts the user for a number hours projected for the job and the wholesale cost of materials. The program computes and displays the cost of the job, which is the number of hours times the hourly rate plus the 120% of the wholesale cost of materials. The program accepts data continuously until 0 is entered for the number of hours. Use appropriate modules, including one that displays *End of program* when the program is finished.

9. a. Draw the hierarchy chart and design the logic for a program needed by the manager of the Stengel County softball team, who wants to compute slugging percentages for his players. A slugging percentage is the total bases earned with base hits divided by the player's number of at-bats. Design a program that prompts the user for a player jersey number, the number of bases earned, and the number of at-bats, and then displays all the data, including the calculated slugging average. The program accepts players continuously until 0 is entered for the jersey number. Use appropriate modules, including one that displays *End of job* after the sentinel is entered for the jersey number.

b. Modify the slugging percentage program to also calculate a player's on-base percentage. An on-base percentage is calculated by adding a player's hits and walks, and then dividing by the sum of at-bats, walks, and sacrifice flies. Prompt the user for all the additional data needed, and display all the data for each player.

c. Modify the softball program so that it also computes a gross production average (GPA) for each player. A GPA is calculated by multiplying a player's on-base percentage by 1.8, then adding the player's slugging percentage, and then dividing by four.

10. Draw the hierarchy chart and design the logic for a program for Arnie's Appliances. Design a program that prompts the user for a refrigerator model name and the interior height, width, and depth in inches. Calculate the refrigerator capacity in cubic feet by first multiplying the height, width, and depth to get cubic inches, and then dividing by 1728 (the number of cubic inches in a cubic foot). The program accepts model names continuously until "XXX" is entered. Use named constants where appropriate. Also use modules, including one that displays *End of job* after the sentinel is entered for the model name.



Performing Maintenance

1. A file named MAINTENANCE02-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 2 include DEBUG02-01.txt, DEBUG02-02.txt, and DEBUG02-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 2 include a file named DEBUG02-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.



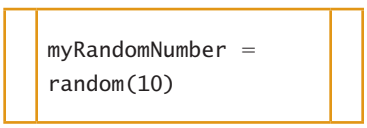
Game Zone

1. For games to hold your interest, they almost always include some random, unpredictable behavior. For example, a game in which you shoot asteroids loses some of its fun if the asteroids follow the same, predictable path each time you play. Therefore, generating random values is a key component in creating most

interesting computer games. Many programming languages come with a built-in module you can use to generate random numbers. The syntax varies in each language, but it is usually something like the following:

```
myRandomNumber = random(10)
```

In this statement, `myRandomNumber` is a numeric variable you have declared and the expression `random`(10) means "call a method that generates and returns a random number between 1 and 10." By convention, in a flowchart, you would place a statement like this in a processing symbol with two vertical stripes at the edges, as shown below.



Create a flowchart or pseudocode that shows the logic for a program that generates a random number, then asks the user to think of a number between 1 and 10. Then display the randomly generated number so the user can see whether his or her guess was accurate. (In future chapters, you will improve this game so that the user can enter a guess and the program can determine whether the user was correct.)

CHAPTER 3

Understanding Structure

Upon completion of this chapter, you will be able to:

- Understand the disadvantages of unstructured spaghetti code
- Describe the three basic structures—sequence, selection, and loop
- Use a priming input to structure a program
- Appreciate the need for structure
- Recognize structure
- Structure and modularize unstructured logic

The Disadvantages of Unstructured Spaghetti Code

Professional business applications usually get far more complicated than the examples you have seen so far in Chapters 1 and 2. Imagine the number of instructions in the computer programs that guide an airplane's flight or audit an income tax return. Even the program that produces your paycheck at work contains many, many instructions. Designing the logic for such a program can be a time-consuming task. When you add hundreds or thousands of instructions to a program, it is easy to create a complicated mess. The descriptive name for logically snarled program statements is **spaghetti code**, because the logic is as hard to follow as one noodle through a plate of spaghetti. Not only is spaghetti code confusing, but also the programs that contain it are prone to error, difficult to reuse, and hard to use as building blocks for larger applications. Programs that use spaghetti code logic are **unstructured programs**; that is, they do not follow the rules of structured logic that you will learn in this chapter. **Structured programs** *do* follow those rules, and eliminate the problems caused by spaghetti code.

For example, suppose that you start a job as a dog washer and that you receive the instructions shown in Figure 3-1. This flowchart is an example of unstructured spaghetti code. A computer program that is organized similarly might "work"—that is, it might produce correct results—but it would be difficult to read and maintain, and its logic would be hard to follow.

You might be able to follow the logic of the dog-washing process in Figure 3-1 for two reasons:

- You might already know how to wash a dog.
- The flowchart contains a limited number of steps.

Imagine, however, that you were not familiar with dog washing, or that the process was far more complicated. (For example, imagine you must wash 100 dogs concurrently while applying flea and tick medication, giving them haircuts, and researching their genealogy.)

Depicting more complicated logic in an unstructured way would be cumbersome. By the end of this chapter, you will understand how to make the unstructured process in Figure 3-1 clearer and less error-prone.



Software developers say that a program that contains spaghetti code has a shorter life than one with structured code. This means that programs developed using spaghetti code exist as production programs in an organization for less time. Such programs are so difficult to alter that when improvements are required, developers often find it easier to abandon the existing program and start from scratch. This takes extra time and costs more money.

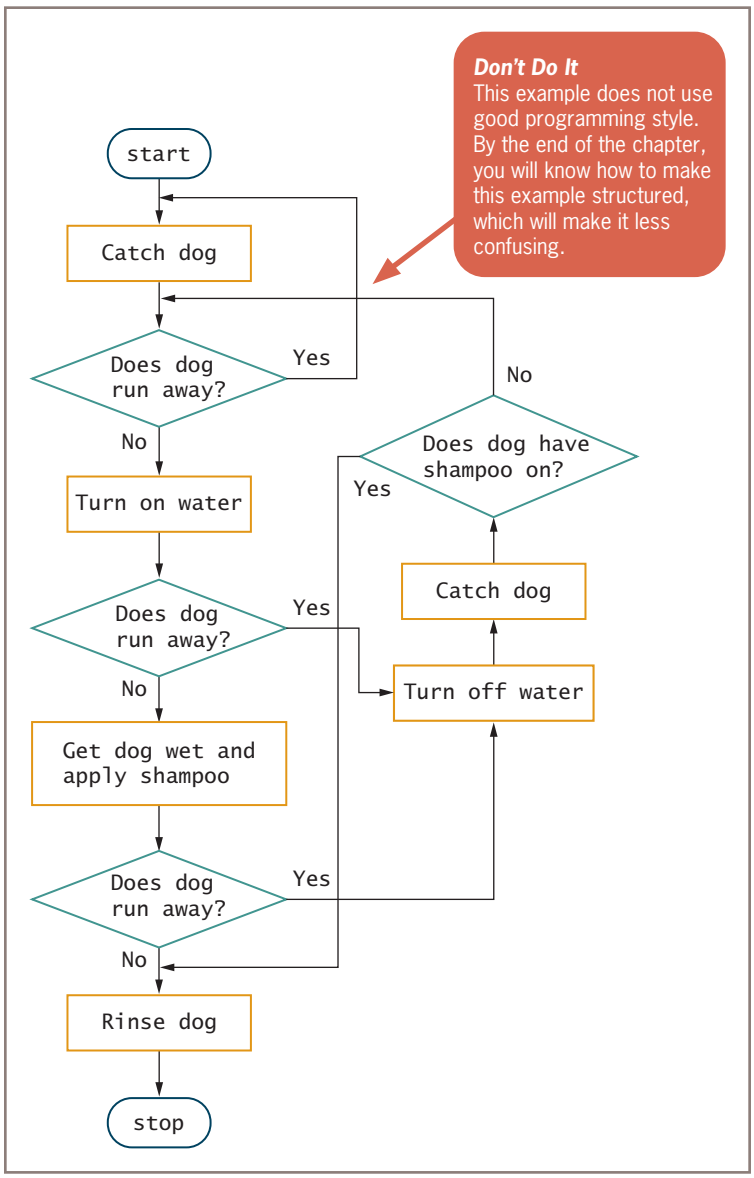


Figure 3-1 Spaghetti code logic for washing a dog

TWO TRUTHS & A LIE

The Disadvantages of Unstructured Spaghetti Code

1. Spaghetti code is the descriptive name for logically snarled programs.
2. Programs written using spaghetti code cannot produce correct results.
3. Programs written using spaghetti code are more difficult to maintain than other programs.

The false statement is #2. Programs written using spaghetti code can produce correct results, but they are more difficult to understand and maintain than programs that use structured techniques.

Understanding the Three Basic Structures

In the mid-1960s, mathematicians proved that any program, no matter how complicated, can be constructed using one or more of only three structures. A **structure** is a basic unit of programming logic; each structure is one of the following:

- sequence
- selection
- loop

With these three structures alone, you can diagram any task, from doubling a number to performing brain surgery. You can diagram each structure with a specific configuration of flowchart symbols.

The Sequence Structure

The **sequence structure** is shown in Figure 3-2. It performs actions or tasks in order, one after the other. A sequence can contain any number of tasks, but there is no option to branch off and skip any of the tasks. Once you start a series of actions in a sequence, you must continue step by step until the sequence ends.

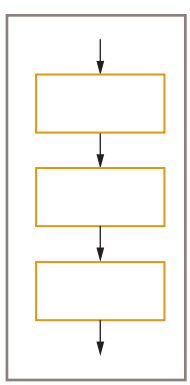


Figure 3-2 Sequence structure

As an example, driving directions often are listed as a sequence. To tell a friend how to get to your house from school, you might provide the following sequence, in which one step follows the other and no steps can be skipped:

```
go north on First Avenue for 3 miles
turn left on Washington Boulevard
go west on Washington for 2 miles
stop at 634 Washington
```

The Selection Structure

The **selection structure,** or **decision structure**, is shown in Figure 3-3. With this structure, one of two courses of action is taken based on the result of testing a condition, or, in other words, evaluating a Boolean expression. A **Boolean expression** is one whose value can be only one of two opposing values, usually expressed as *true* and *false* or *yes* and *no*. True/false evaluation is natural from a computer's standpoint, because computer circuitry consists of two-state on-off switches, often represented by 1 or 0. Every computer decision yields a true-or-false, yes-or-no, 1-or-0 result. A Boolean expression is used to control every selection structure.

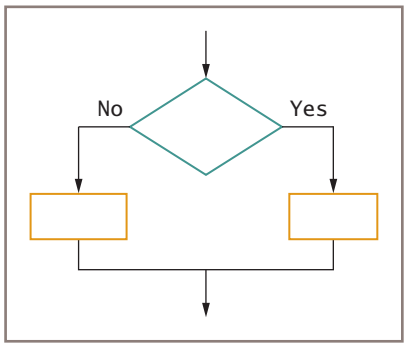


Figure 3-3 Selection structure



Mathematician George Boole (1815–1864) approached logic more simply than his predecessors did, by expressing logical selections with common algebraic symbols. He is considered the founder of mathematical logic, and Boolean (true/false) expressions are named for him.

A flowchart that describes a selection structure begins with a decision symbol that contains a Boolean expression, and the branches of the decision must join at the bottom of the structure. Pseudocode that describes a selection structure starts with if. Pseudocode uses the **end-structure statement** `endif` to clearly show where the structure ends.

Some people call the selection structure an `if-then-else` because it fits the following statement:

```
if someCondition is true then
   do oneProcess
else
   do theOtherProcess
endif
```

For example, you might provide part of the directions to your house as follows:

```
if traffic is backed up on Washington Boulevard then
   continue for 1 block on First Avenue and turn left on Adams Lane
else
   turn left on Washington Boulevard
endif
```

Similarly, a payroll program might include a statement such as:

```
if hoursWorked is more than 40 then
   calculate regularPay and overtimePay
else
   calculate regularPay
endif
```

These `if-else` examples can also be called **dual-alternative `if`s** (or **dual-alternative selections**) because they contain two alternatives—the action taken when the tested condition is true and the action taken when it is false. Note that it is perfectly correct for one branch of the selection to be a "do nothing" branch. In each of the following examples, an action is taken only when the tested condition is true:

```
if it is raining then
   take an umbrella
endif
```

```
if employee participates in the dental plan then
   deduct $40 from employee gross pay
endif
```

The previous examples without `else` clauses are **single-alternative `if`s** (or **single-alternative selections**); a diagram of their structure is shown in Figure 3-4. In these cases, you do not take any special action if it is not raining or if the employee does not belong to the dental plan. The branch in which no action is taken is called the **null case** or **null branch**.

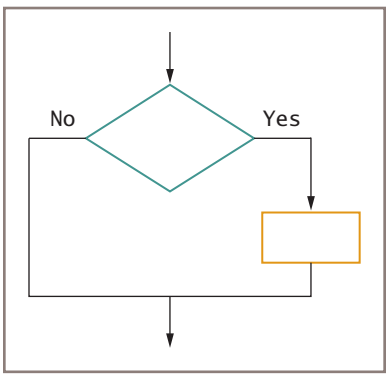


Figure 3-4 Single-alternative selection structure

The Loop Structure

The **loop structure** is shown in Figure 3-5. A loop continues to repeat actions while a tested condition remains true. The action or actions that occur within the loop are the **loop body**. In the most common type of loop, a condition is evaluated; if the answer is true, you execute the loop body and evaluate the condition again. If the condition is still true, you execute the loop body again and then reevaluate the condition. This continues while the condition remains true, or, in other words, until the condition becomes false—then you exit the loop structure. Programmers call this structure a **`while` loop**; pseudocode that describes this type of loop starts with `while` and ends with the end-structure statement `endwhile`. A flowchart that describes the `while` loop structure always begins with a decision symbol that contains a Boolean expression and that has a branch that returns to the evaluation. You might hear programmers refer to looping as **repetition** or **iteration**.

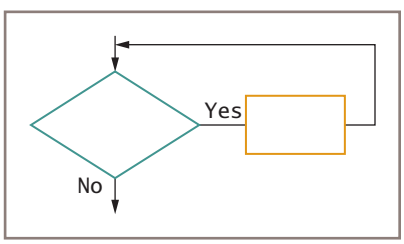


Figure 3-5 Loop structure



The `while` loop tests a condition before executing the loop body even once. Another type of structured loop tests a condition after the first loop body execution. You will learn more about this alternate type of loop in Chapter 5. For the rest of this chapter, assume that all loops are `while` loops that test the controlling condition before the loop body ever executes. All logical problems can be solved using only the three structures—sequence, selection, and `while` loop.

Some programmers call a while loop a **while...do loop**, because it fits the following statement:

```
while testCondition continues to be true
   do someProcess
endwhile
```

When you provide directions to your house which is at street address 634, part of the directions might be:

```
while the address of the house you are passing remains below 634
   travel forward to the next house
   look at the address on the house
endwhile
```

You encounter examples of looping every day, as in each of the following:

```
while you continue to be hungry
   take another bite of food
   determine whether you still feel hungry
endwhile
```

```
while unread pages remain in the reading assignment
   read another unread page
   determine whether there are more pages to read
endwhile
```

Combining Structures

All logic problems can be solved using only these three structures—sequence, selection, and loop—but the structures can be combined in an infinite number of ways. For example, you can have a sequence of tasks followed by a selection, or a loop followed by a sequence. Attaching structures end to end is called **stacking structures**. For example, Figure 3-6 shows a structured flowchart achieved by stacking structures, and shows pseudocode that follows the flowchart logic.



Whether you are drawing a flowchart or writing pseudocode, you can use any opposite, mutually exclusive words to represent decision outcomes—for example, *Yes* and *No* or *true* and *false*. This book follows the convention of using *Yes* and *No* in flowchart diagrams and *true* and *false* in pseudocode.

The pseudocode in Figure 3-6 shows a sequence, followed by a selection, followed by a loop. First `stepA` and `stepB` execute in sequence. Then a selection structure starts with the test of `conditionC` which represents a Boolean expression. The instruction that follows the `if` clause `(stepD)` executes when its tested condition `(conditionC)` is true, the instruction that follows `else (stepE)` executes when the tested condition is false, and any instructions that follow `endif` execute in either case. In other words, statements beyond the `endif`

statement are "outside" the selection structure. Similarly, the `endwhile` statement shows where the loop structure ends. In Figure 3-6, while `conditionF` continues to be true, `stepG` continues to execute. If any statements followed the `endwhile` statement, they would be outside of, and not a part of, the loop.

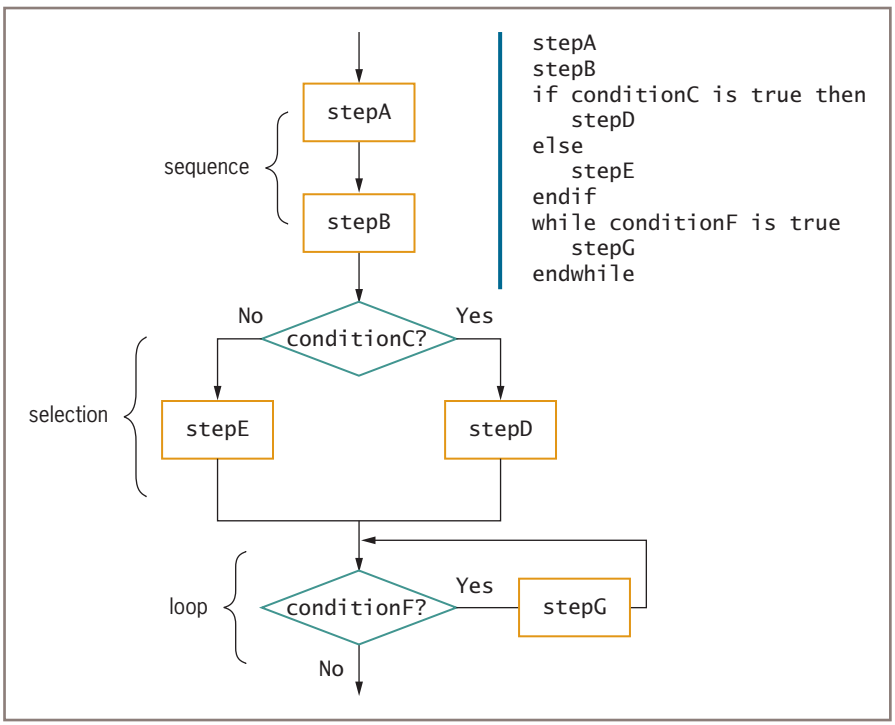


Figure 3-6 Structured flowchart and pseudocode with three stacked structures

Besides stacking structures, you can replace any individual steps in a structured flowchart diagram or pseudocode with additional structures. This means that any sequence, selection, or loop can contain other sequence, selection, or loop structures. For example, you can have a sequence of three tasks on one branch of a selection, as shown in Figure 3-7. Placing a structure within another structure is called **nesting structures**.

In the pseudocode for the logic shown in Figure 3-7, the indentation shows that all three statements (`stepJ`, `stepK`, and `stepL`) must execute if `conditionH` is true. These three statements constitute a **block**, or a group of statements that executes as a single unit.

In place of one of the steps in the sequence in Figure 3-7, you can insert another structure. In Figure 3-8, the process named `stepK` has been replaced with a loop structure that begins with a test of the condition named `conditionM`.

In the pseudocode shown in Figure 3-8, notice that `if` and `endif` are vertically aligned. This shows that they are "on the same level." Similarly, `stepJ`, `while`, `endwhile`, and `stepL`

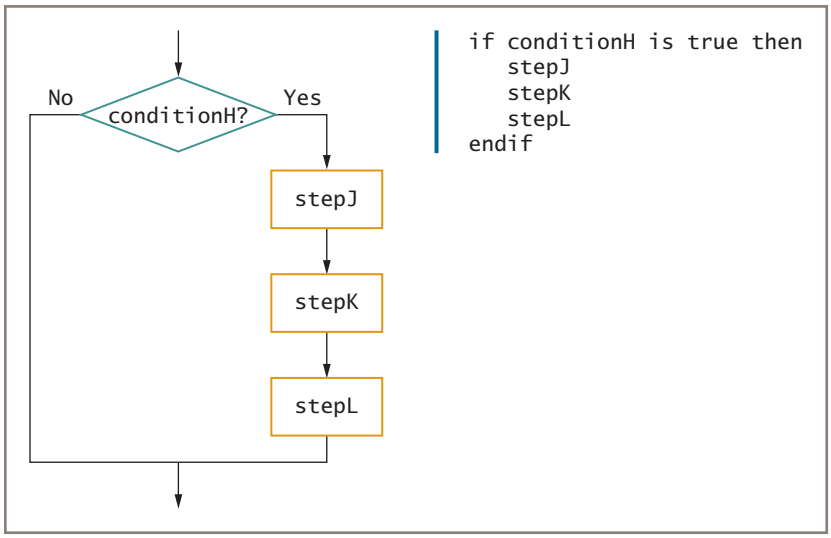


Figure 3-7 Flowchart and pseudocode showing nested structures—a sequence nested within a selection

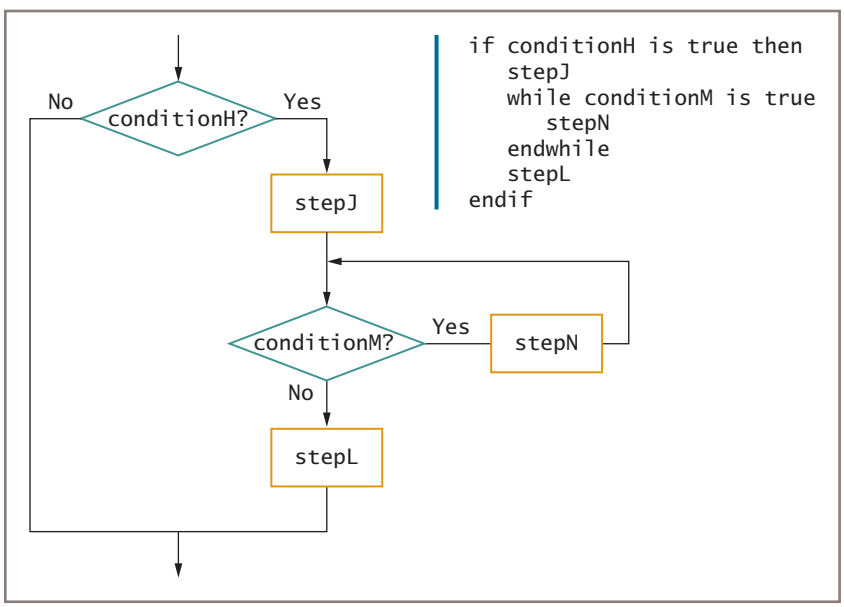


Figure 3-8 Flowchart and pseudocode showing nested structures—a loop nested within a sequence, nested within a selection

are aligned, and they are evenly indented. In the flowchart in Figure 3-8, you could draw a vertical line through the symbols containing `stepJ`, the entry and exit points of the `while` loop, and `stepL`. The flowchart and the pseudocode represent exactly the same logic.

When you nest structures, the statements that start and end a structure are always on the same level and are always in pairs. Structures cannot overlap. For example, if you have an `if` structure that contains a `while` structure, then the `endwhile` statement will come before the `endif`. On the other hand, if you have a `while` that contains an `if`, then the `endif` statement will come before the `endwhile`.

There is no limit to the number of levels you can create when you nest and stack structures. For example, Figure 3-9 shows logic that has been made more complicated by replacing `stepN` with a selection. The structure that performs `stepP` or `stepQ` based on the outcome of `conditionO` is nested within the loop that is controlled by `conditionM`. In the pseudocode in Figure 3-9, notice how the `if`, `else`, and `endif` that describe the condition

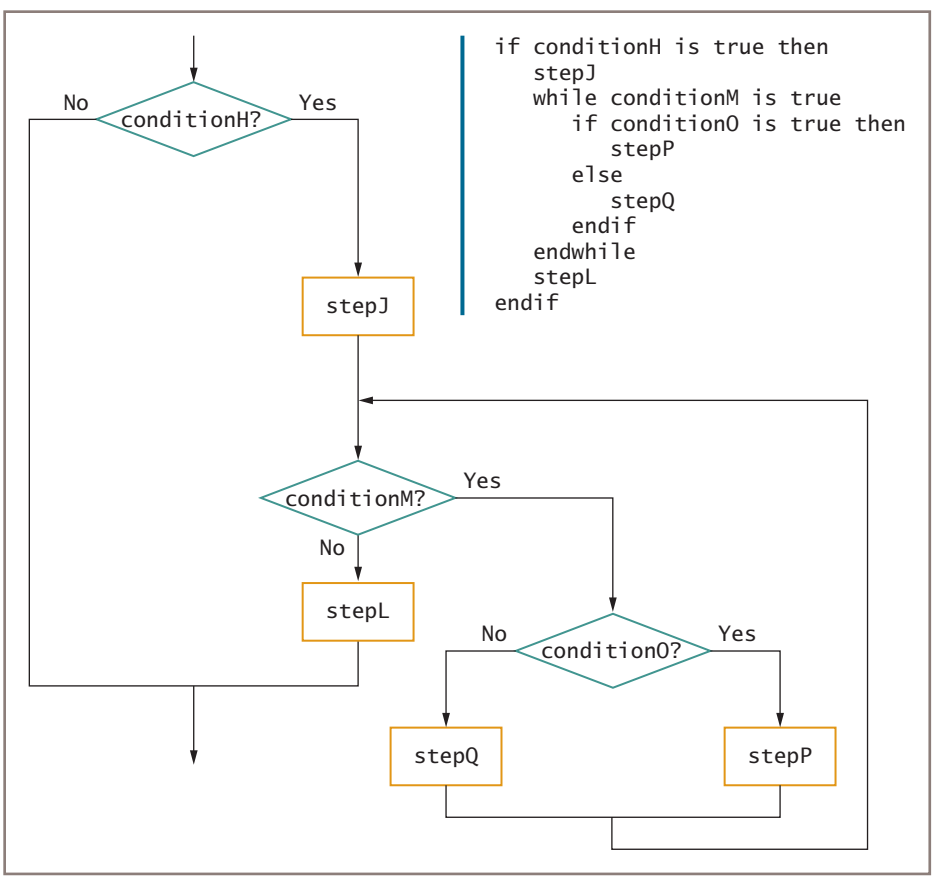


Figure 3-9 Flowchart and pseudocode for a selection within a loop within a sequence within a selection

selection are aligned with each other and within the `while` structure that is controlled by `conditionM`. As before, the indentation used in the pseudocode reflects the logic laid out graphically in the flowchart.

Many of the preceding examples are generic so that you can focus on the relationships of the symbols without worrying what they do. Keep in mind that generic instructions like `stepA` and generic conditions like `conditionC` can stand for anything. For example, Figure 3-10 shows the process of buying and planting flowers outdoors in the spring. The flowchart and pseudocode structures are identical to those in Figure 3-9. In the exercises at the end of this chapter, you will be asked to develop more scenarios that fit the same pattern.

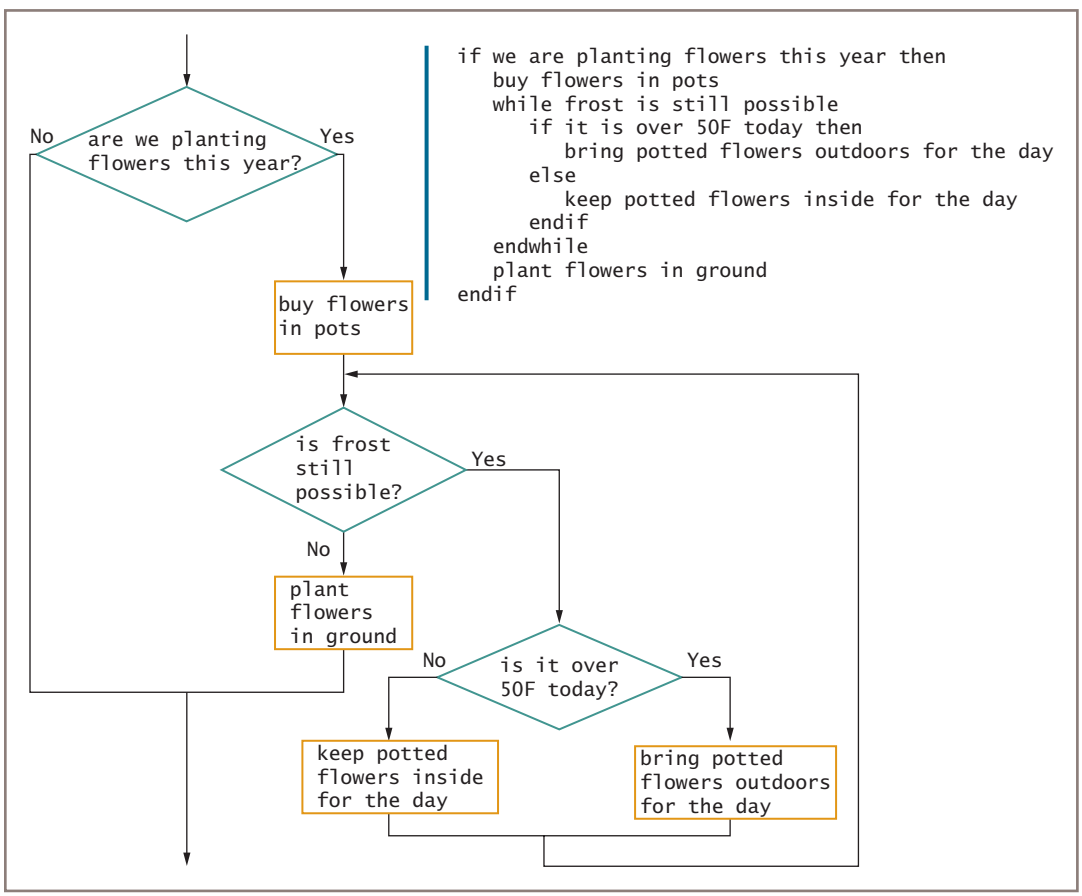
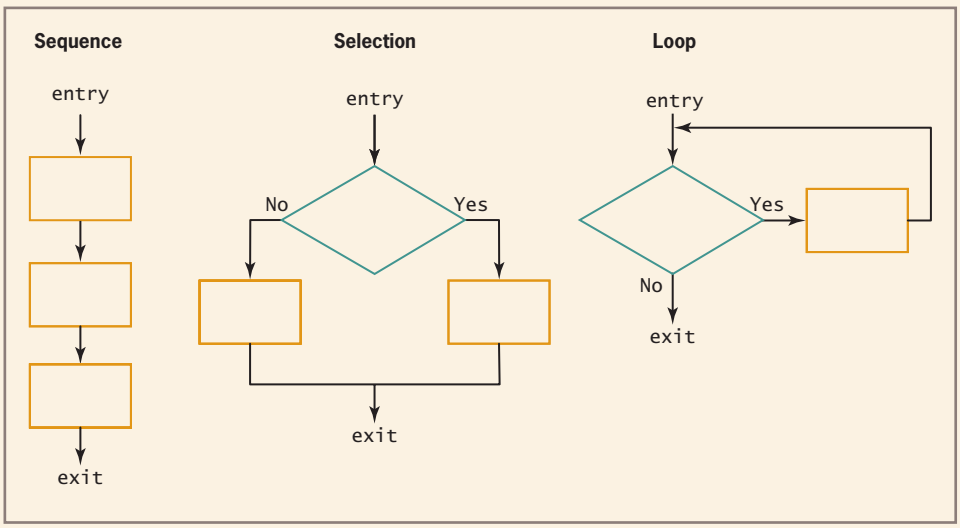


Figure 3-10 The process of buying and planting flowers in the spring

The possible combinations of logical structures are endless, but each segment of a structured program is a sequence, a selection, or a loop. The three structures are shown together in Quick Reference 3-1. Notice that each structure has one entry point and one exit point. One structure can attach to another only at one of these points.

QUICK REFERENCE 3-1 The Three Structures



Try to imagine physically picking up any of the three structures using the entry and exit "handles." These are the spots at which you could connect one structure to another. Similarly, any complete structure, from its entry point to its exit point, can be inserted within the process symbol of any other structure, forming nested structures.

In summary, a structured program has the following characteristics:

- A structured program includes only combinations of the three basic structures—sequence, selection, and loop. Any structured program might contain any number of structures and they can be one, two, or all three types of structures.
- Each of the structures has a single entry point and a single exit point.
- Structures can be stacked or connected to one another only at their entry or exit points.
- Any structure can be nested within another structure.

A structured program is never required to contain examples of all three structures. For example, many simple programs contain only a sequence of several tasks that execute from start to finish without any needed selections or loops. As another example, a program might display a series of numbers, looping to do so, but never making any decisions about the numbers.



Watch the video *Understanding Structure*.

TWO TRUTHS & A LIE

Understanding the Three Basic Structures

1. Each structure in structured programming is a sequence, selection, or loop.
2. All logic problems can be solved using only three structures—sequence, selection, and loop.
3. The three structures cannot be combined in a single program.

The false statement is #3. The three structures can be stacked or nested in an infinite number of ways.

Using a Priming Input to Structure a Program

Recall the number-doubling program discussed in Chapter 2; Figure 3-11 shows a similar program. The program accepts a number as input and checks for the end-of-data condition. If the condition is not met, then the number is doubled, the answer is displayed, and the next number is input.



Recall from Chapter 1 that this book uses `eof` to represent a generic end-of-data condition when the exact tested parameters are not important to the discussion. In this example, the test is for `not eof` because processing will continue while the end of the data has not been reached.

Is the program represented by Figure 3-11 structured? At first, it might be hard to tell. The three allowed structures were illustrated in Quick Reference 3-1, and the flowchart in Figure 3-11 does not look exactly like any of those three shapes. However, because you can stack and nest structures while retaining overall structure, at first glance it might be difficult to determine whether a flowchart as a whole is structured. It is easiest to analyze the flowchart in Figure 3-11 one step at a time.

The beginning of the flowchart looks like Figure 3-12. Is this portion of the flowchart structured? Yes, it is a sequence of two tasks—making declarations and inputting a value.

Adding the next piece of the flowchart looks like Figure 3-13. After a value is input for `originalNumber`, the `not eof?` condition is tested. The sequence is finished; either a selection or a loop is starting. You might not know which one, but you do know that with

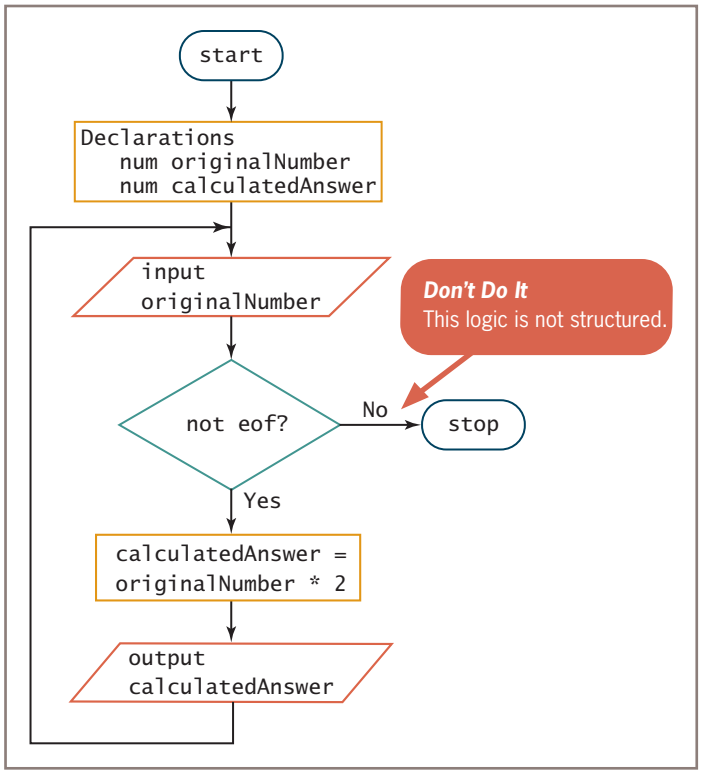


Figure 3-11 Unstructured flowchart of a number-doubling program

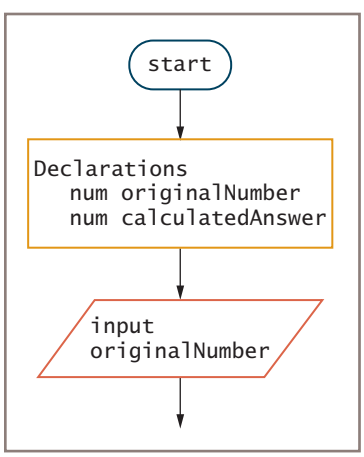


Figure 3-12 Beginning of a number-doubling flowchart

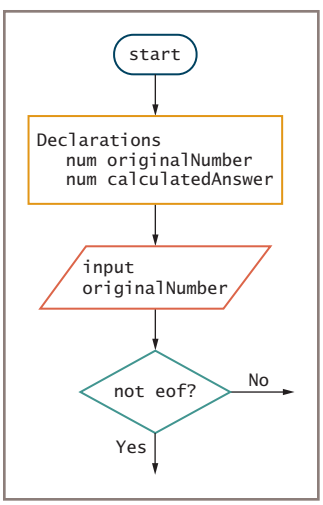


Figure 3-13 Number-doubling flowchart continued

a sequence, each task or step must follow without any opportunity to branch off. So, which type of structure starts with the test in Figure 3-13? Is it a selection or a loop?

Selection and loop structures both start by testing a condition, but they differ as follows:

- In a selection structure, the logic branches in one of two directions after the test, and then the flow comes back together; the condition is not tested a second time within the selection structure.
- In a loop, each time the result of the conditional test results in the execution of the loop body, the flow of logic returns to the test that started the loop. When the body of a loop executes, the controlling condition is always tested again.

If the end-of-data condition is not met in the number-doubling problem in the original Figure 3-11, then the result is calculated and output, a new number is obtained, and the logic returns to the test for the end of the file. In other words, while the answer to the `not eof?` question continues to be *Yes*, a body of two statements continues to execute. Therefore, the `not eof?` test starts a structure that is more likely to be a loop structure than a selection structure.

The number-doubling process *does* contain a loop, but it is not a structured loop. In a structured `while` loop, the rules are:

1. A condition is tested.
2. If the value of the condition indicates the loop body should be executed, then it is.
3. After the loop body executes, the logic must go right back to test the value again—it can't go anywhere else!

The flowchart in Figure 3-11 tests whether the `eof` condition has been met. If the answer is *Yes* (that is, while `not eof?` is true), then the program performs two tasks in the loop body: It does the arithmetic, and it displays the results. Performing two tasks is acceptable because two tasks with no possible branching constitute a sequence, and it is fine to nest one structure (in this case, a sequence) within another structure (in this case, a loop). However, in Figure 3-11, when the sequence ends, the logic does not flow right back to the loop-controlling test. Instead, it goes *above* the test to get another number. For the loop in Figure 3-11 to be a structured loop, the logic must return to the `not eof?` evaluation when the embedded sequence ends.

The flowchart in Figure 3-14 shows the program with the flow of logic returning to the `not eof?` test immediately after the nested two-step sequence. Figure 3-14 shows a structured flowchart, but it has one major flaw—the flowchart does not do the job of continuously doubling different numbers.

Follow the flowchart in Figure 3-14 through a typical program run, assuming the `eof` condition is an input value of 0. Suppose that when the program starts, the user enters *9* for the value of `originalNumber`. That is not `eof`, so the number is multiplied by 2, and 18 is displayed as the value of `calculatedAnswer`. Then the `not eof?` value is tested again. The `not eof?` condition still must be true because a new value representing the sentinel (ending) value has not been entered and cannot be entered. The logic never returns to the `input originalNumber` task, so the value of `originalNumber` never changes. Therefore,

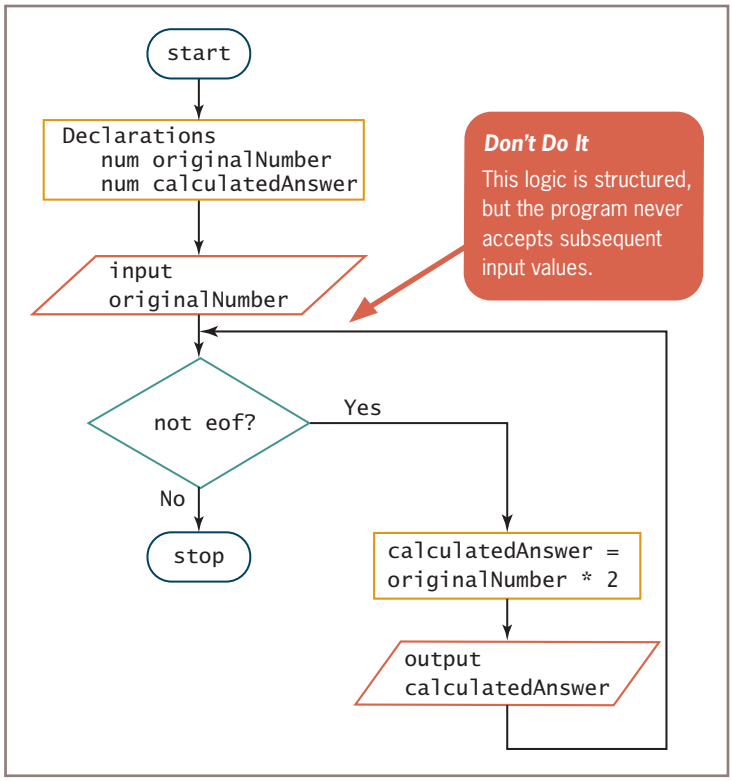


Figure 3-14 Structured, but nonfunctional, flowchart of number-doubling problem

9 doubles again and the answer 18 is displayed again. The `not eof?` result still is true, so the same steps are repeated. This goes on *forever*, with the answer 18 being calculated and output repeatedly. The program logic shown in Figure 3-14 is structured, but it does not work as intended.

Conversely, the program in Figure 3-15 works, but it is not structured because after the tasks execute within a structured loop, the flow of logic must return directly to the loop-controlling test. In Figure 3-15, the logic does not return to this test; instead, it goes "too high" outside the loop to repeat the `input originalNumber` task.

How can the number-doubling problem be both structured and work as intended? Often, for a program to be structured, you must add something extra. In this case, it is a priming input step. A **priming input** or **priming read** is an added statement that gets the first input value in a program. For example, if a program will receive 100 data values as input, you input the first value in a statement that is separate from the other 99. You must do this to keep the program structured.

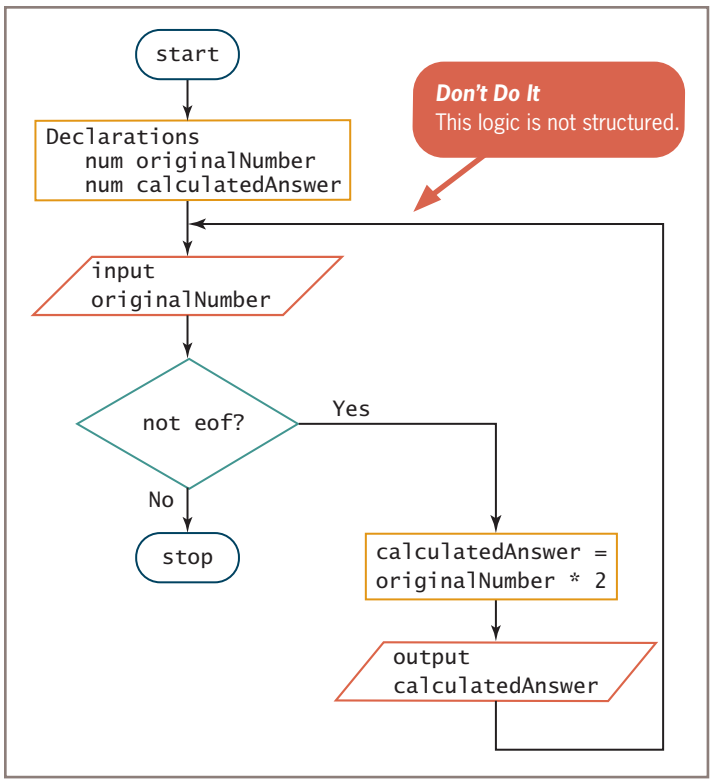


Figure 3-15 Functional but unstructured flowchart

Consider the solution in Figure 3-16; it is structured *and* it does what it is supposed to do. It contains a shaded, additional `input originalNumber` statement. The program logic contains a sequence and a loop. The loop contains another sequence.

The additional `input originalNumber` step shown in Figure 3-16 is typical in structured programs. The first of the two input steps is the priming input. The term *priming* comes from the fact that the input is first, or *primary* (it gets the process going, as in "priming the pump"). The purpose of the priming input step is to control the upcoming loop that begins with the `not eof?` condition test. The last element within the structured loop gets the next, and all subsequent, input values. This is also typical in structured loops—the last step executed within the loop body is one that can alter the value of the condition that controls the loop (which in this case is the `not eof?` test).



In Chapter 2, you learned that the group of preliminary tasks that sets the stage for the main work of a program is called the housekeeping section. The priming input is an example of a housekeeping task.

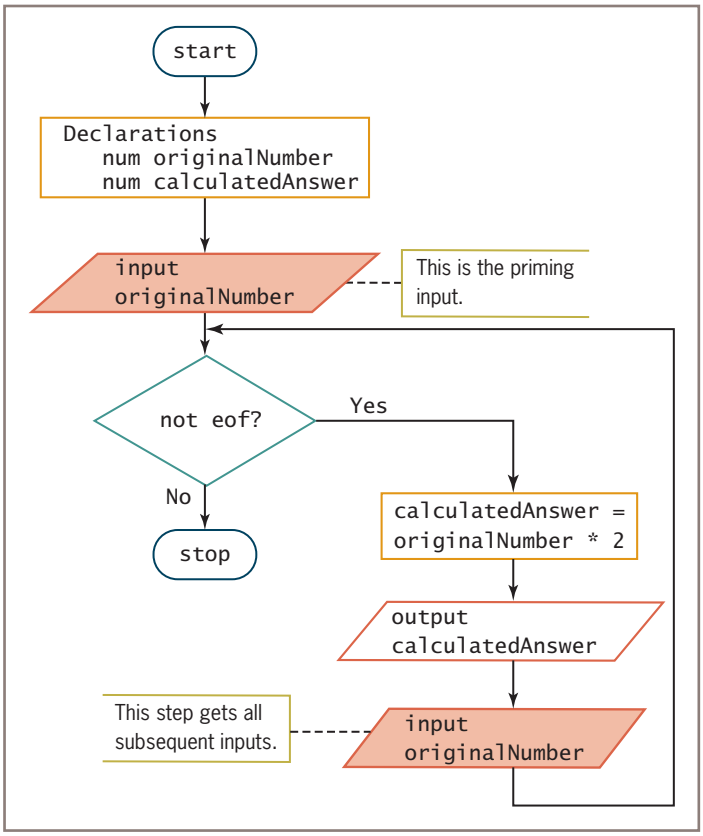


Figure 3-16 Functional, structured flowchart for the number-doubling problem

Figure 3-17 shows another way you might attempt to draw the logic for the number-doubling program. At first glance, the figure might seem to show an acceptable solution to the problem—it is structured, it contains a sequence followed by a single loop with a sequence of three steps nested within it, and it appears to eliminate the need for the priming input statement. When the program starts, the declarations are made and the `not eof?` condition is tested. If it is not the end of input data, then the program gets a number, doubles it, and displays it. Then, if the `not eof?` condition remains true, the program gets another number, doubles it, and displays it. The program might continue while many numbers are input. At some point, the input number will represent the `eof` condition; for example, the program might have been written to recognize the value *0* as the program-terminating value. After the `eof` value is entered, its condition is not immediately

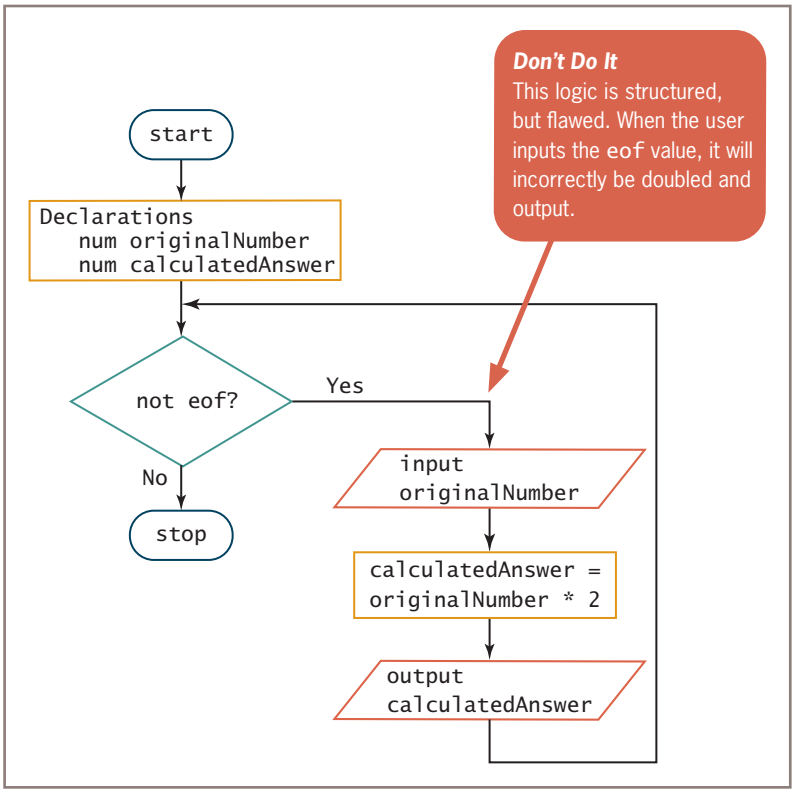


Figure 3-17 Structured but incorrect solution to the number-doubling problem

tested. Instead, a result is calculated and displayed one last time before the loop-controlling test is made again. If the program was written to recognize `eof` when `originalNumber` is 0, then an extraneous answer of 0 will be displayed before the program ends. Depending on the language you are using and on the type of input being used, the results might be worse: The program might terminate by displaying an error message or the value output might be indecipherable garbage. In any case, this last output is superfluous—no value should be doubled and output after the `eof` condition is encountered.

As a general rule, a program-ending test should always come immediately after an input statement because that's the earliest point at which it can be evaluated. Therefore, the best solution to the number-doubling problem remains the one shown in Figure 3-16—the structured solution containing the priming input statement.

TWO TRUTHS & A LIE

Using a Priming Input to Structure a Program

1. A priming input is the statement that repeatedly gets all the data that is input in a program.
2. A structured program might contain more instructions than an unstructured one.
3. A program can be structured yet still be incorrect.

The false statement is #1. A priming input gets the first input.

Understanding the Reasons for Structure

At this point, you might very well be saying, "I liked the original number-doubling program back in Figure 3-11 just fine. I could follow it. Also, the first program had one less step in it, so it was less work. Who cares if a program is structured?"

Until you have some programming experience, it is difficult to appreciate the reasons for using only the three structures—sequence, selection, and loop. However, staying with these three structures is better for the following reasons:

- *Clarity*—The number-doubling program is small. As programs get bigger, they get more confusing if they are not structured.
- *Professionalism*—All other programmers (and programming teachers you might encounter) expect your programs to be structured. It is the way things are done professionally.
- *Efficiency*—Most newer computer languages support structure and use syntax that lets you deal efficiently with sequence, selection, and looping. Older languages, such as assembly languages, COBOL, and RPG, were developed before the principles of structured programming were discovered. However, even programs that use those older languages can be written in a structured form. Newer languages such as C#, C++, and Java tend to enforce structure by their syntax.



In the early days of programming, programmers often created their logic to leave a selection or loop before it was complete by using a "go to" statement. The statement allowed the logic to "go to" any other part of the program breaking out of a structure prematurely. Structured programming is sometimes called **goto-less programming**.

- *Maintenance*—You and other programmers will find it easier to modify and maintain structured programs as changes are required in the future.
- *Modularity*—Structured programs can be broken down easily into modules that can be assigned to any number of programmers. The routines then are pieced back together like modular furniture at each routine's single entry or exit point. Additionally, a module often can be used in multiple programs, saving development time in the new project.

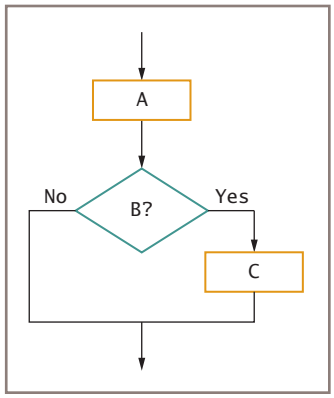


Figure 3-18 Example 1

TWO TRUTHS & A LIE

Understanding the Reasons for Structure

1. Structured programs are clearer than unstructured programs.
2. You and other programmers will find it easier to modify and maintain structured programs as changes are required in the future.
3. Structured programs are not easily divided into parts, making them less prone to error.

The false statement is #3. Structured programs can be broken down easily into modules that can be assigned to any number of programmers.

Recognizing Structure

When you are beginning to learn about structured program design, it is difficult to detect whether a flowchart of a program's logic is structured. For example, is the flowchart segment in Figure 3-18 structured?

Yes, it is. It has a sequence and a selection structure.

Is the flowchart segment in Figure 3-19 structured?

Yes, it is. It has a loop and a selection within the loop.

Is the flowchart segment in the upper-left corner of Figure 3-20 structured?

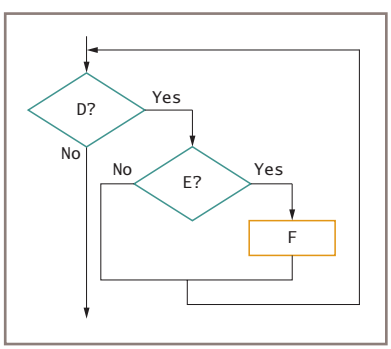


Figure 3-19 Example 2

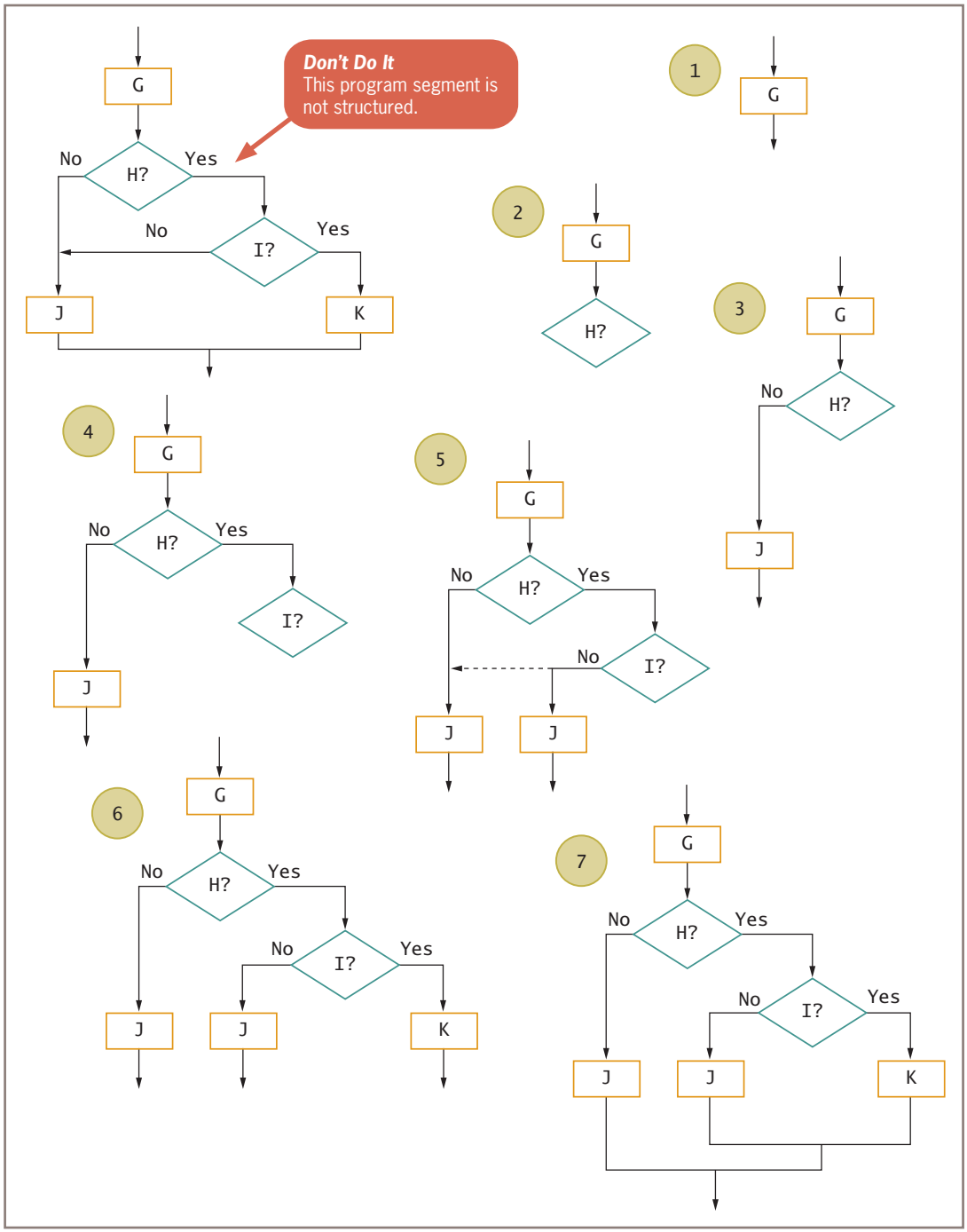


Figure 3-20 Example 3 and process to structure it

No, it is not built from the three basic structures. One way to straighten out an unstructured flowchart segment is to use the "spaghetti bowl" method; that is, picture the flowchart as a bowl of spaghetti that you must untangle. Imagine you can grab one piece of pasta at the top of the bowl and start pulling. As you "pull" each symbol out of the tangled mess, you can untangle the separate paths until the entire segment is structured.

Look at the diagram in the upper-left corner of Figure 3-20. If you could start pulling the arrow at the top, you would encounter a box labeled G. (See Figure 3-20, Step 1.) A single process like G is part of an acceptable structure—it constitutes at least the beginning of a sequence structure.

Imagine that you continue pulling symbols from the tangled segment. The next item in the flowchart is a test of a condition labeled H, as you can see in Figure 3-20, Step 2. At this point, you know the sequence that started with G has ended. Sequences never have conditional tests in them, so the sequence is finished; either a selection or a loop is beginning with the test of H. A loop must return to the loop-controlling conditional test at some later point. You can see from the original logic that whether the value of H is *Yes* or *No*, the logic never returns to H. Therefore, H begins a selection structure, not a loop structure.

To continue detangling the logic, you would pull up on the flowline that emerges from the left side (the *No* side) of the conditional test H. You encounter J, as shown in Step 3 of Figure 3-20. When you continue beyond J, you reach the end of the flowchart.

Now you can turn your attention to the *Yes* side (the right side) of the condition tested in H. When you pull up on the right side, you encounter Question I. (See Step 4 of Figure 3-20.)

In the original version of the flowchart in Figure 3-20, follow the line on the left side of conditional test I. The line emerging from the left side of the test is attached to J, which is outside the selection structure. You might say the I-controlled selection is becoming entangled with the H-controlled selection, so you must untangle the structures by repeating the step that is causing the tangle. (In this example, you repeat Step J to untangle it from the other usage of J.) Continue pulling on the flowline that emerges from J until you reach the end of the program segment, as shown in Step 5 of Figure 3-20.

Now pull on the right side of the condition represented by I. Process K pops up, as shown in Step 6 of Figure 3-20; then you reach the end.

At this point, the untangled flowchart has three loose ends. The loose ends of the selection that starts with I can be brought together to form a selection structure; then the loose ends of the selection that starts with H can be brought together to form another selection structure. The result is the flowchart shown in Step 7 of Figure 3-20. The entire flowchart segment is structured—it has a sequence followed by a selection inside a selection.



If you want to try structuring a more difficult example of an unstructured program, see Appendix B.

TWO TRUTHS & A LIE

Recognizing Structure

1. Some processes cannot be expressed in a structured format.
2. An unstructured flowchart can achieve correct outcomes.
3. Any unstructured flowchart can be "detangled" to become structured.

The false statement is #1. Any set of instructions can be expressed in a structured format.

Structuring and Modularizing Unstructured Logic

Recall the dog-washing process illustrated in Figure 3-1 at the beginning of this chapter. When you look at it now, you should recognize it as an unstructured process. Can this process be reconfigured to perform precisely the same tasks in a structured way? Of course!

Figure 3-21 demonstrates how you might approach structuring the dog-washing logic. Part 1 of the figure shows the beginning of the process. The first step, *Catch dog*, is a simple sequence. This step is followed by testing whether the dog runs away. When a conditional test is encountered, the sequence is over, and either a loop or a selection structure starts. In this case, after the dog runs away, you must catch the dog and determine whether he runs away again, so a loop begins. To create a structured loop like the ones you have seen earlier in this chapter, you can repeat the *Catch dog* process and return immediately to the *Does dog run away?* test.

In the original flowchart in Figure 3-1, you turn on the water when the dog does not run away. This step is a simple sequence, so it can correctly be added after the loop. When the water is turned on, the original logic checks whether the dog runs away after this new development. This starts a loop. In the original flowchart, the lines cross, creating a tangle, so you repeat as many steps as necessary to detangle the lines. After you turn off the water and catch the dog, you encounter the question that tests the condition *Does dog have shampoo on?* Because the logic has not yet reached the shampooing step, there is no need to make this test—the answer at this point always will be *No*. When one of the logical paths emerging from a conditional test can never be traveled, you can eliminate the test. Part 2 of Figure 3-21 shows that if the dog runs away after you turn on the water, but before you've gotten the dog wet and shampooed him, you must turn off the water, catch the dog, and return to testing whether the dog runs away.

The logic in Part 2 of Figure 3-21 is not structured because the second loop that begins with *Does dog run away?* does not immediately return to the loop-controlling test after its body executes. So, to make the loop structured, you can repeat the actions that occur before returning to the loop-controlling test. The flowchart segment in Part 3 of Figure 3-21 is

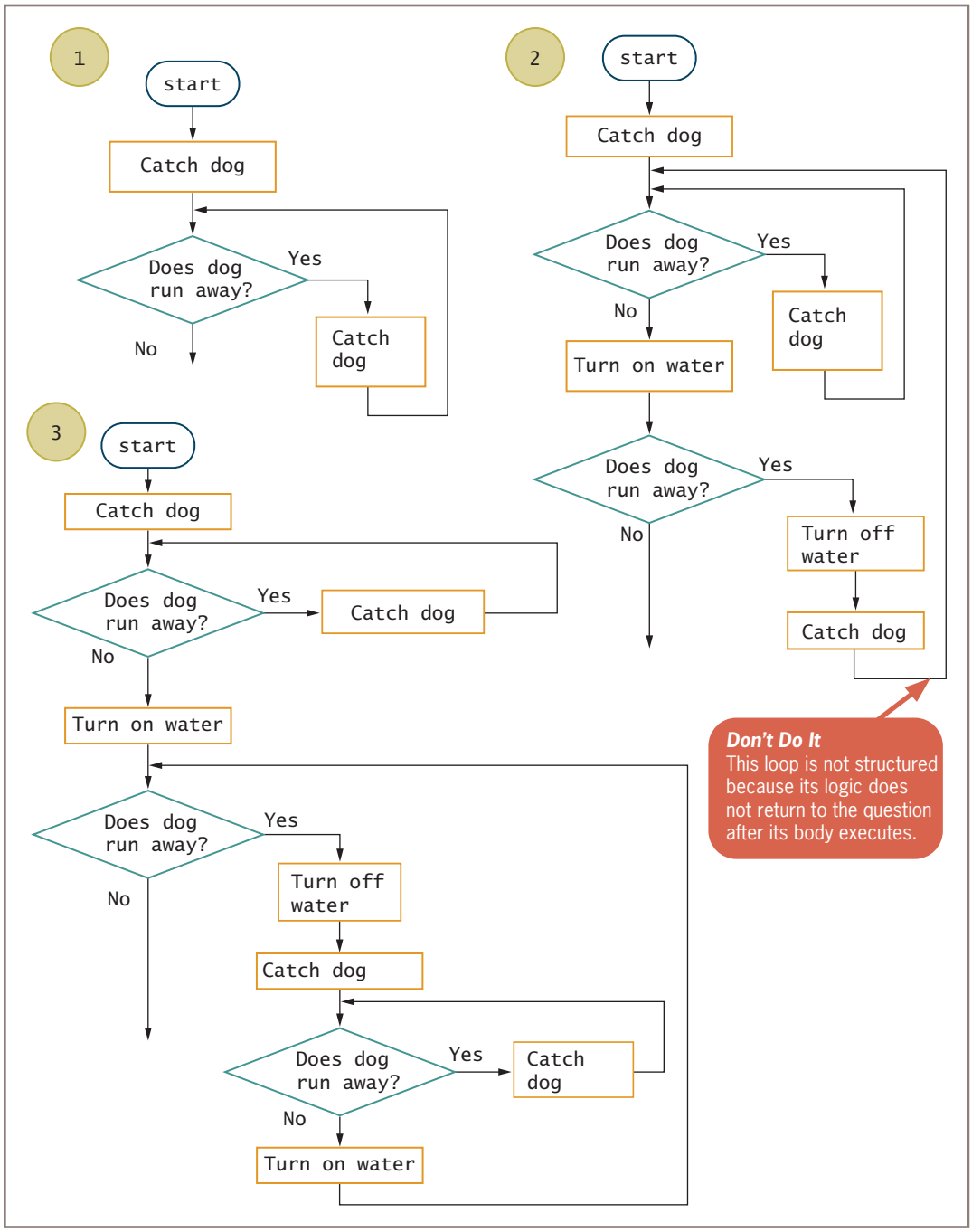


Figure 3-21 Steps to structure the dog-washing process

structured; it contains a sequence, a loop, a sequence, and a final, larger loop. This last loop contains its own sequence, loop, and sequence.

After the dog is caught and the water is on, you wet and shampoo the dog. Then, according to the original flowchart in Figure 3-1, you once again check to see whether the dog has run away. If he has, you turn off the water and catch the dog. From this location in the logic, the value of *Does dog have shampoo on?* will always be *Yes*; as before, there is no need to test a condition when there is only one possible result. So, if the dog runs away, the last loop executes. You turn off the water, continue to catch the dog as he repeatedly escapes, and turn on the water. When the dog is caught at last, you rinse the dog and end the process. Figure 3-22 shows both the complete flowchart and pseudocode. The logic is complete and is structured. It contains alternating sequence and loop structures.

Figure 3-22 includes three places where the sequence-loop-sequence of catching the dog and turning on the water are repeated. If you wanted to, you could modularize the duplicate sections so that their instruction sets are written once and contained in a separate module. Figure 3-23 shows a modularized version of the program; the three module calls are shaded.

One advantage to modularizing the steps needed to catch the dog and start the water is that the main program becomes shorter and easier to understand. Another advantage is that if this process needs to be modified, the changes can be made in just one location. For example, if you decided it was necessary to test the water temperature each time you turned on the water, you would add those instructions only once in the modularized version. In the original version in Figure 3-22, you would have to add those instructions in three places, causing more work and increasing the chance for errors.

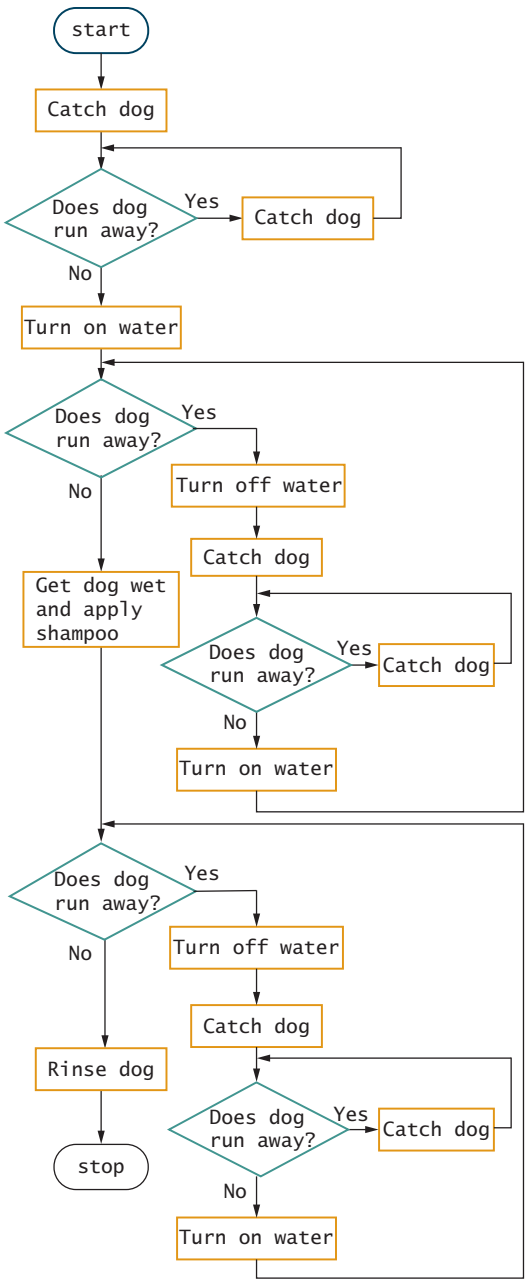
No matter how complicated, any set of steps can always be reduced to combinations of the three basic sequence, selection, and loop structures. These structures can be nested and stacked in an infinite number of ways to describe the logic of any process and to create the logic for every computer program written in the past, present, or future.



For convenience, many programming languages allow two variations of the three basic structures. The `case` structure is a variation of the selection structure and the `do` loop is a variation of the `while` loop. You can learn about these two structures in Chapter 4 and 5. Even though these extra structures can be used in most programming languages, all logical problems can be solved without them.



Watch the video *Structuring Unstructured Logic*.




```
start
   Catch dog
   while dog runs away
      Catch dog
   endwhile
   Turn on water
   while dog runs away
      Turn off water
      Catch dog
      while dog runs away
         Catch dog
      endwhile
      Turn on water
   endwhile
   Get dog wet and apply shampoo
   while dog runs away
      Turn off water
      Catch dog
      while dog runs away
         Catch dog
      endwhile
      Turn on water
   endwhile
   Rinse dog
stop
```

Figure 3-22 Structured dog-washing flowchart and pseudocode

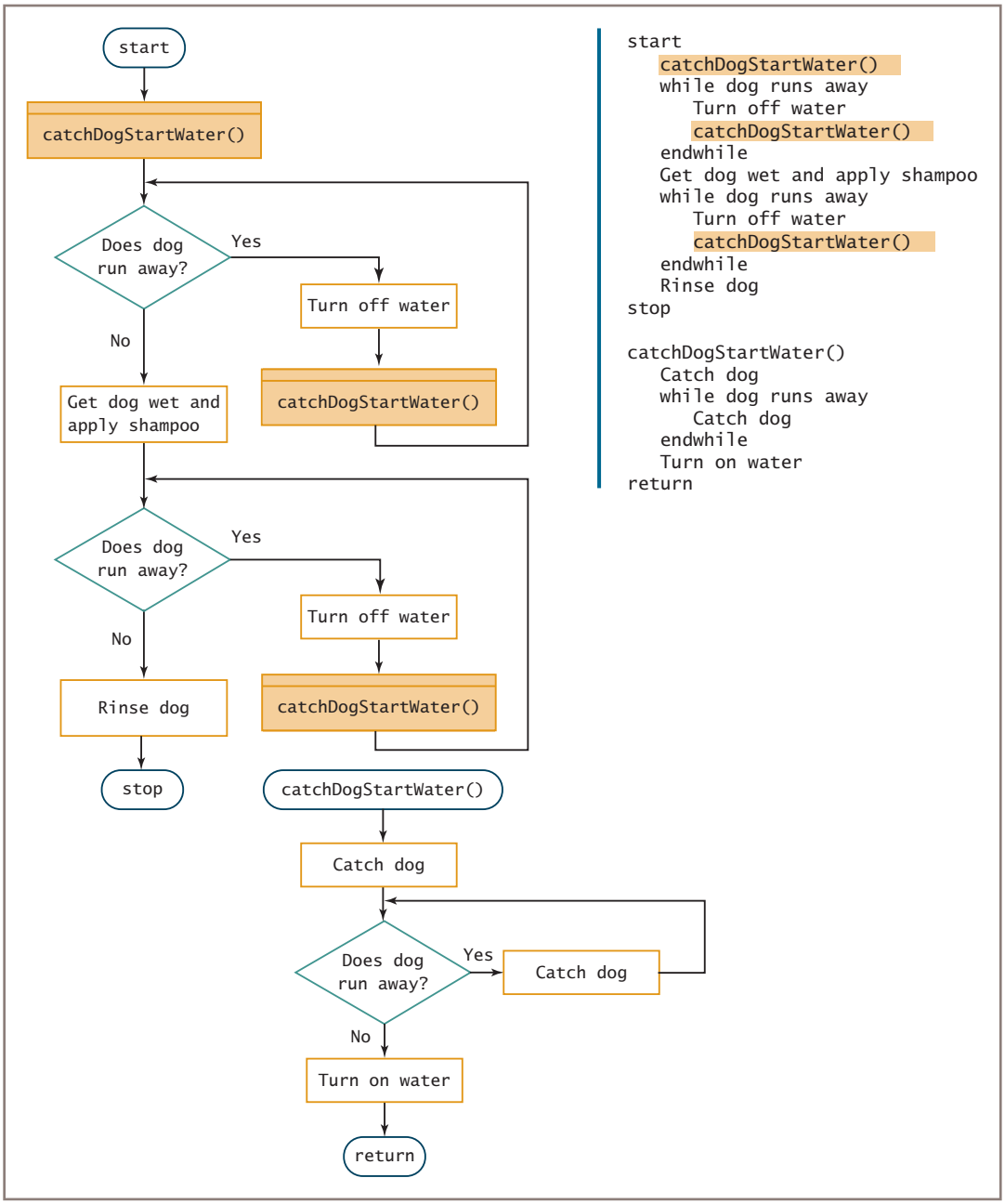


Figure 3-23 Modularized version of the dog-washing program

TWO TRUTHS & A LIE

Structuring and Modularizing Unstructured Logic

1. When you encounter a conditional test in a logical diagram, a sequence should be ending.
2. In a structured loop, the logic returns to the loop-controlling conditional test after the loop body executes.
3. If a flowchart or pseudocode contains a conditional test for which the result never varies, you can eliminate the test.

The false statement is #1. When you encounter a conditional test in a logical diagram, either a selection or a loop should start. Any type of structure might end, however, before the test is encountered.

Chapter Summary

- Spaghetti code is the descriptive name for unstructured program statements that do not follow the rules of structured logic.
- Clearer programs can be constructed using only three basic structures: sequence, selection, and loop. These three structures can be combined in an infinite number of ways by stacking and nesting them. Each structure has a single entry point and a single exit point; one structure can attach to another only at one of these points.
- A priming input is the statement that gets the first input value prior to starting a structured loop. Usually, the last step within the loop body gets the next and all subsequent input values.
- Programmers use structured techniques to promote clarity, professionalism, efficiency, and modularity.
- One way to order an unstructured flowchart segment is to imagine it as a bowl of spaghetti that you must untangle.
- Any set of logical steps can be rewritten to conform to the three structures: sequence, selection, and loop.

Key Terms

Spaghetti code is snarled, unstructured program logic.

Unstructured programs are programs that do *not* follow the rules of structured logic.

Structured programs are programs that do follow the rules of structured logic.

A **structure** is a basic unit of programming logic; each structure is a sequence, selection, or loop.

A **sequence structure** contains series of steps executed in order. A sequence can contain any number of tasks, but there is no option to branch off, skipping any of the tasks.

A **selection structure** or **decision structure** contains a conditional test, and, depending on the result, takes one of two courses of action before continuing with the next task.

A **Boolean expression** is one that represents only one of two states, usually expressed as true or false.

An **end-structure statement** designates the end of a pseudocode structure.

An **`if-then-else`** is another name for a dual-alternative selection structure.

Dual-alternative `if`s (or **dual-alternative selections**) define one action to be taken when the tested condition is true and another action to be taken when it is false.

Single-alternative `if`s (or **single-alternative selections**) take action on just one branch of the decision.

The **null case** or **null branch** is the branch of a decision in which no action is taken.

A **loop structure** continues to repeat actions while a test condition remains true.

A **loop body** is the set of actions that occur within a loop.

A **`while` loop** is a structure that continues to repeat a process while some condition remains true.

Repetition and **iteration** are alternate names for a loop structure.

A **`while...do` loop** is an alternate name for a `while` loop.

Stacking structures is the act of attaching structures end to end.

Nesting structures is the act of placing a structure within another structure.

A **block** is a group of statements that executes as a single unit.

A **priming input** or **priming read** is the statement that reads the first input prior to starting a structured loop that uses the data.

Goto-less programming is a name to describe structured programming, because structured programmers do not use a "go to" statement.

Exercises



Review Questions

1. Snarled program logic is called _____ code.

   a. snake
   b. spaghetti
   c. linguini
   d. gnarly

2. The three structures of structured programming are _____.

   a. sequence, selection, and loop
   b. decision, loop, and iteration
   c. sequence, order, and process
   d. loop, iteration, and refraction

3. A sequence structure can contain _____.

   a. only one task
   b. exactly three tasks
   c. no more than three tasks
   d. any number of tasks

4. Which of the following is *not* another term for a selection structure?

   a. loop structure
   b. decision structure
   c. dual-alternative if structure
   d. `if-then-else` structure

5. A _____ expression has one of two values, often expressed as true or false.

   a. Georgian
   b. Boolean
   c. Selection
   d. Caesarian

6. Placing a structure within another structure is called the _____ structures.

   a. untangling
   b. nesting
   c. building
   d. stacking

7. Attaching structures end to end is called _____.

   a. nesting
   b. untangling
   c. building
   d. stacking

8. When an action is required if a condition is true, but no action is needed if it is false, you use a _____.

   a. sequence
   b. loop
   c. dual-alternative selection
   d. single-alternative selection

9. To take action as long as a condition remains true, you use a _____.

   a. sequence
   b. stack
   c. loop
   d. selection

10. When you must perform one action when a condition is true and a different one when it is false, you use a _____.

    a. sequence
    b. loop
    C. dual-alternative selection
    d. single-alternative selection

11. Which of the following attributes do all three basic structures share?

    a. Their flowcharts all contain exactly three processing symbols.
    b. They all begin with a process.
    c. They all have one entry and one exit point.
    d. They all contain a conditional test.

12. Which is true of stacking structures?

    a. Two incidences of the same structure cannot be stacked adjacently.
    b. When you stack structures, you cannot nest them in the same program.
    c. Each structure has only one point where it can be stacked on top of another.
    d. When you stack structures, the top structure must be a sequence.

13. When you input data in a loop within a program, the input statement that precedes the loop _____.

    a. is the only part of the program allowed to be unstructured
    b. cannot result in `eof`
    c. is called a priming input
    d. executes hundreds or even thousands of times in most business programs

14. A group of statements that executes as a unit is a _____.

    a. block
    b. family
    C. chunk
    d. cohort

15. Which of the following is acceptable in a structured program?

    A. Placing a sequence within the true branch of a dual-alternative decision
    b. Placing a decision within a loop
    c. Placing a loop within one of the steps in a sequence
    d. All of these are acceptable.

16. In a selection structure, the structure-controlling condition is _____.

    a. tested once at the beginning of the structure
    b. tested once at the end of the structure
    c. tested repeatedly until it is false
    d. tested repeatedly until it is true

17. When a loop executes, the structure-controlling condition is _____.

    a. tested exactly once
    b. never tested more than once
    c. tested either before or after the loop body executes
    d. tested only if it is true, and not asked if it is false

18. Which of the following is *not* a reason for enforcing structure rules in computer programs?

    a. Structured programs are clearer to understand than unstructured ones.
    b. Other professional programmers will expect programs to be structured.
    c. Structured programs usually are shorter than unstructured ones.
    d. Structured programs can be broken down into modules easily.

19. Which of the following is *not* a benefit of modularizing programs?

    a. Modular programs are easier to read and understand than nonmodular ones.
    b. If you use modules, you can ignore the rules of structure.
    c. Modular components are reusable in other programs.
    d. Multiple programmers can work on different modules at the same time.

20. Which of the following is true of structured logic?

    a. You can use structured logic with newer programming languages, such as Java and C#, but not with older ones.
    b. Any task can be described using some combination of the three structures: sequence, selection, and loop.
    c. Structured programs require that you break the code into easy-to-handle modules that each contain no more than five actions.
    d. All of these are true.



Programming Exercises

1. In Figure 3-10, the process of buying and planting flowers in the spring was shown using the same structures as the generic example in Figure 3-9. Use the same logical structure as in Figure 3-9 to create a flowchart or pseudocode that describes some other process you know.

2. Each of the flowchart segments in Figure 3-24 is unstructured. Redraw each segment so that it does the same processes under the same conditions, but is structured.

3. Write pseudocode for each example (a through e) in Exercise 2, making sure your pseudocode is structured and accomplishes the same tasks as the flowchart segment.

4. Assume that you have created a mechanical arm that can hold a pen. The arm can perform the following tasks:
   - Lower the pen to a piece of paper.
   - Raise the pen from the paper.

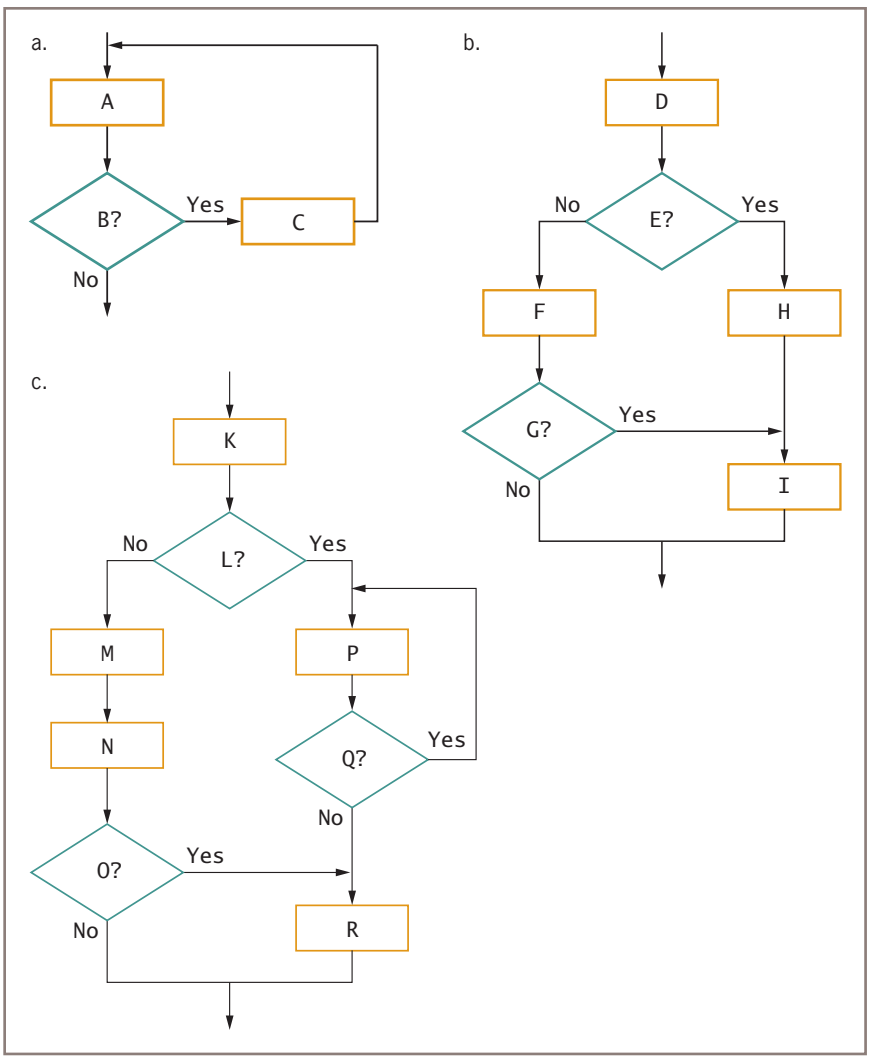


Figure 3-24 Flowcharts for Exercise 2 (*continues*)

(continued)

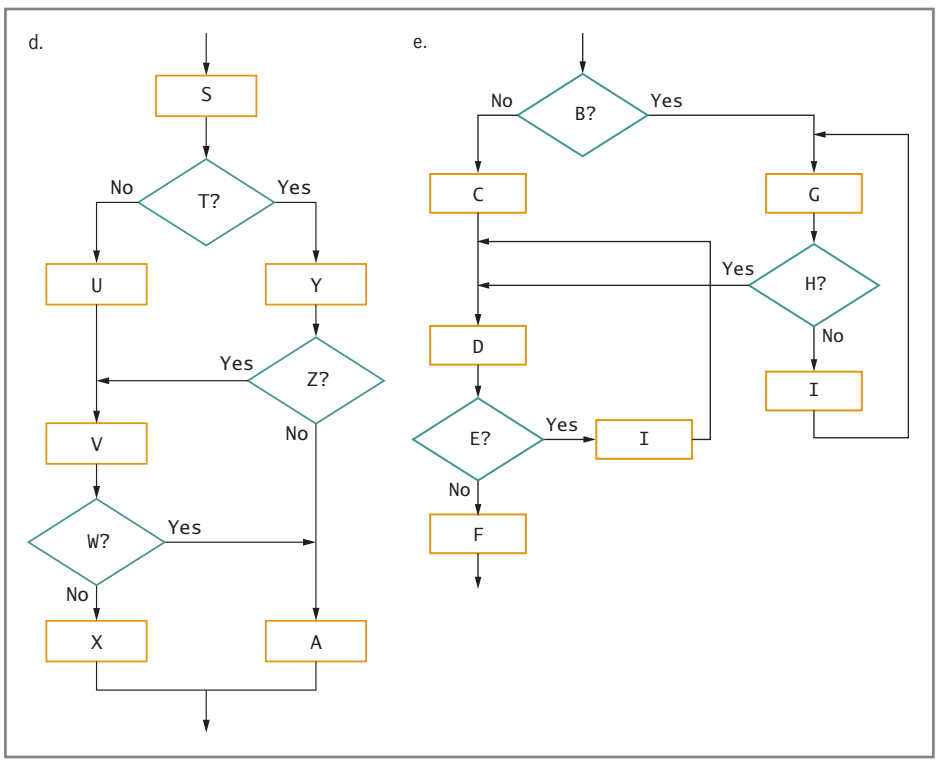


Figure 3-24 Flowcharts for Exercise 2

- Move the pen 1 inch along a straight line. (If the pen is lowered, this action draws a 1-inch line from left to right; if the pen is raised, this action just repositions the pen 1 inch to the right.)
- Turn 90 degrees to the right.
- Draw a circle that is 1 inch in diameter.

Draw a structured flowchart or write structured pseudocode describing the logic that would cause the arm to draw or write the following. Have a fellow student act as the mechanical arm and carry out your instructions. Don't reveal the desired outcome to your partner until the exercise is complete.

a. 1-inch square
b. 2-inch by 1-inch rectangle
c. string of three beads
d. short word (for example, *cat*)
e. four-digit number

5. Assume that you have created a mechanical robot that can perform the following tasks:

   - Stand up.
   - Sit down.
   - Turn left 90 degrees.
   - Turn right 90 degrees.
   - Take a step.

   Additionally, the robot can determine the answer to one test condition:

   - Am I touching something?

   a. Place two chairs 20 feet apart, directly facing each other. Draw a structured flowchart or write pseudocode describing the logic that would allow the robot to start from a sitting position in one chair, cross the room, and end up sitting in the other chair. Have a fellow student act as the robot and carry out your instructions.
   b. Draw a structured flowchart or write pseudocode describing the logic that would allow the robot to start from a sitting position in one chair, stand up and circle the chair, cross the room, circle the other chair, return to the first chair, and sit. Have a fellow student act as the robot and carry out your instructions.

6. Draw a structured flowchart or write pseudocode that describes the process of guessing a number between 1 and 100. After each guess, the player is told that the guess is too high or too low. The process continues until the player guesses the correct number. Pick a number and have a fellow student try to guess it by following your instructions.

7. Looking up a word in a dictionary can be a complicated process. For example, assume that you want to look up *logic*. You might open the dictionary to a random page and see *juice*. You know this word comes alphabetically before *logic*, so you flip forward and see *lamb*. That is still not far enough, so you flip forward and see *monkey*. You have gone too far, so you flip back, and so on. Draw a structured flowchart or write pseudocode that describes the process of looking up a word in a dictionary. Pick a word at random and have a fellow student attempt to carry out your instructions.

8. Draw a structured flowchart or write structured pseudocode describing how to get from your home to your school. Include at least two decisions and two loops.

9. Draw a structured flowchart or write structured pseudocode describing how to decide what college to attend. Include at least two decisions and two loops.

10. Draw a structured flowchart or write structured pseudocode describing how to buy a new shirt. Include at least two decisions and two loops.



Performing Maintenance

1. A file named MAINTENANCE03-01.jpeg is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 3 include DEBUG03-01.txt, DEBUG03-02.txt, and DEBUG03-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 3 include a file named DEBUG03-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.



Game Zone

1. Choose a simple children's game and describe its logic, using a structured flowchart or pseudocode. For example, you might try to explain Rock, Paper, Scissors; Musical Chairs; Duck, Duck, Goose; the card game War; or the elimination game Eenie, Meenie, Minie, Moe.

2. Choose a television game show such as *Wheel of Fortune* or *Jeopardy*! and describe its rules using a structured flowchart or pseudocode.

3. Choose a sport such as baseball or football and describe the actions in one limited play period (such as an at-bat in baseball or a possession in football) using a structured flowchart or pseudocode.

CHAPTER 4

Making Decisions

Upon completion of this chapter, you will be able to:

- Describe the selection structure
- List the relational comparison operators
- Understand AND logic
- Understand OR logic
- Understand NOT logic
- Make selections within ranges
- Describe precedence when combining AND and OR operators
- Appreciate the `case` structure

The Selection Structure

The reason people frequently think computers are smart lies in the ability of computer programs to make decisions. A medical diagnosis program that can decide if your symptoms fit various disease profiles seems quite intelligent, as does a program that can offer different potential driving routes based on your destination. As you learned in Chapter 3, the selection structure is one of the three basic structures; it starts with the evaluation of a Boolean expression and takes one of two paths based on the outcome. See Figures 4-1 and 4-2.

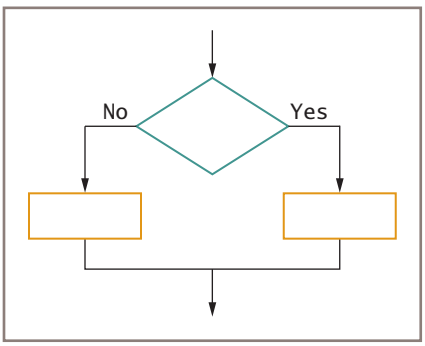


Figure 4-1 The dual-alternative selection structure

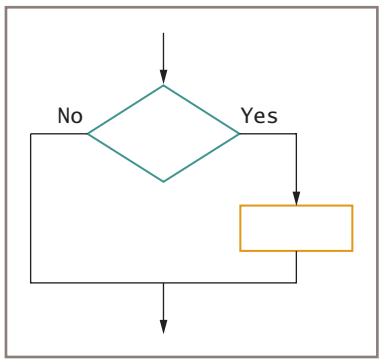


Figure 4-2 The single-alternative selection structure

In Chapter 3, you learned that the structure in Figure 4-1 is a dual-alternative selection because an action is associated with each of two possible outcomes: Depending on the evaluation of the Boolean expression in the decision symbol, the logical flow proceeds either to the left branch of the structure or to the right. The choices are mutually exclusive; that is, the logic can flow to only one of the two alternatives, never to both. This form of the selection structure is an `if-then-else` selection.



This book follows the convention that the two logical paths emerging from a dual-alternative selection are drawn to the right and left of a diamond in a flowchart. Some programmers draw one of the flowlines emerging from the bottom of the diamond. The exact format of the diagram is not as important as the idea that one logical path flows into a selection, and two possible outcomes emerge. Your flowcharts will be easier for readers to follow if you are consistent when you draw selections. For example, if the *Yes* branch flows to the right for one selection, it should flow to the right for all subsequent selections in the same flowchart.

The flowchart segment in Figure 4-2 represents a single-alternative selection in which action is required for only one outcome of the question. This form of the selection structure is called an `if-then` selection, because no alternative or `else` action is necessary.

Quick Reference 4-1 shows the pseudocode standards used to construct `if` statements in this book.

QUICK REFERENCE 4-1 `if` statement pseudocode standards

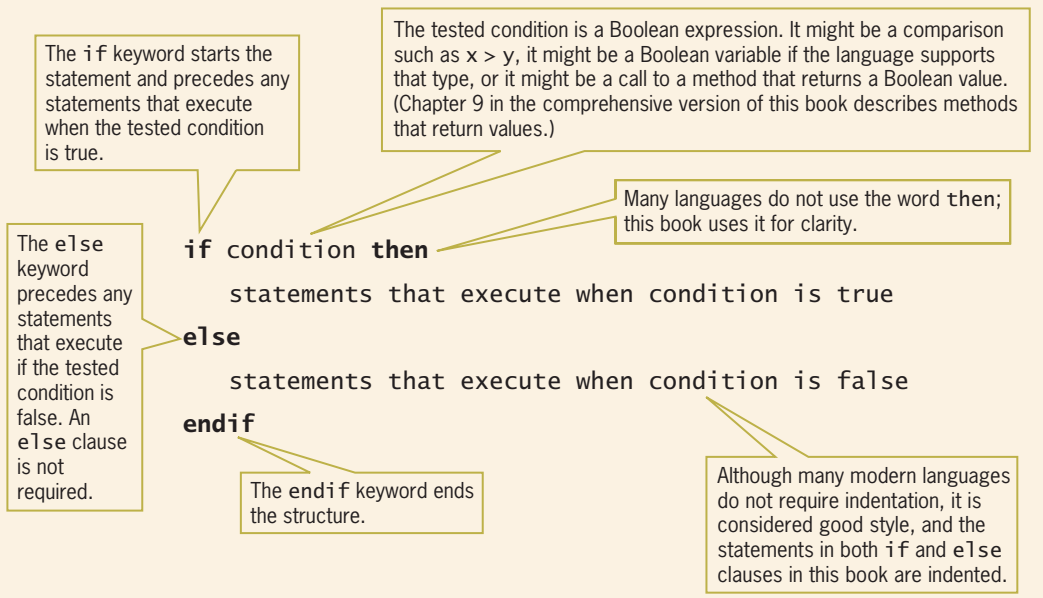


Figure 4-3 shows the flowchart and pseudocode for an interactive program that computes pay for employees. The program displays the weekly pay for each employee at the same hourly rate ($15.00) and assumes that there are no payroll deductions. The mainline logic calls `housekeeping()`, `detailLoop()`, and `finish()` modules. The `detailLoop()` module contains a typical dual-alternative selection that determines whether an employee has worked more than a standard workweek (40 hours), and pays one and one-half times the employee's usual hourly rate for hours worked in excess of 40 per week.



Throughout this book, many examples are presented in both flowchart and pseudocode form. When you analyze a solution, you might find it easier to concentrate on just one of the two design tools at first. When you understand how the program works using one design tool (for example, the flowchart), you can confirm that the solution is identical using the other tool.

In the `detailLoop()` module of the program in Figure 4-3, the decision contains two clauses:

- The **`if-then` clause** is the part of the decision that holds the action or actions that execute when the tested condition in the decision is true. In this example, the clause holds the longer overtime calculation.
- The **`else` clause** of the decision is the part that executes only when the tested condition in the decision is false. In this example, the clause contains the shorter calculation.

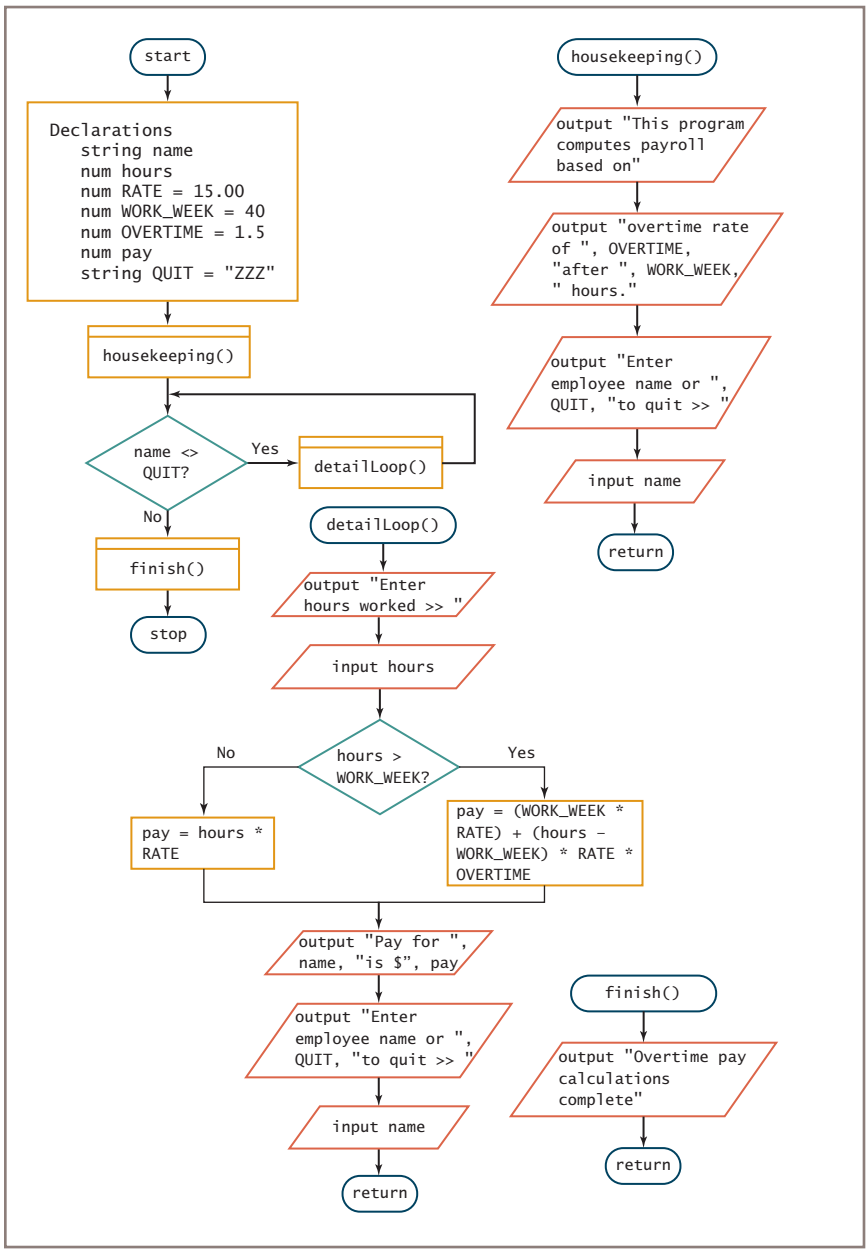


Figure 4-3 Flowchart and pseudocode for overtime payroll program *(continues)*

(continued)

```
start
   Declarations
      string name
      num hours
      num RATE = 15.00
      num WORK_WEEK = 40
      num OVERTIME = 1.5
      num pay
      string QUIT = "ZZZ"
   housekeeping()
   while name <> QUIT
      detailLoop()
   endwhile
   finish()
stop

housekeeping()
   output "This program computes payroll based on"
   output "overtime rate of ", OVERTIME, "after ", WORK_WEEK, " hours."
   output "Enter employee name or ", QUIT, "to quit >> "
   input name
return

detailLoop()
   output "Enter hours worked >> "
   input hours
   if hours > WORK_WEEK then
      pay = (WORK_WEEK * RATE) + (hours - WORK_WEEK) * RATE * OVERTIME
   else
      pay = hours * RATE
   endif
   output "Pay for ", name, "is $", pay
   output "Enter employee name or ", QUIT, "to quit >> "
   input name
return

finish()
   output "Overtime pay calculations complete"
return
```

Figure 4-3 Flowchart and pseudocode for overtime payroll program

Figure 4-4 shows a sample execution of the program in a command-line environment. Data values are entered for three employees. The first two employees do not work more than 40 hours, so their pay is displayed simply as `hours` times 15.00. The third employee, however, has worked one hour of overtime, and so makes $22.50 for the last hour instead of $15.

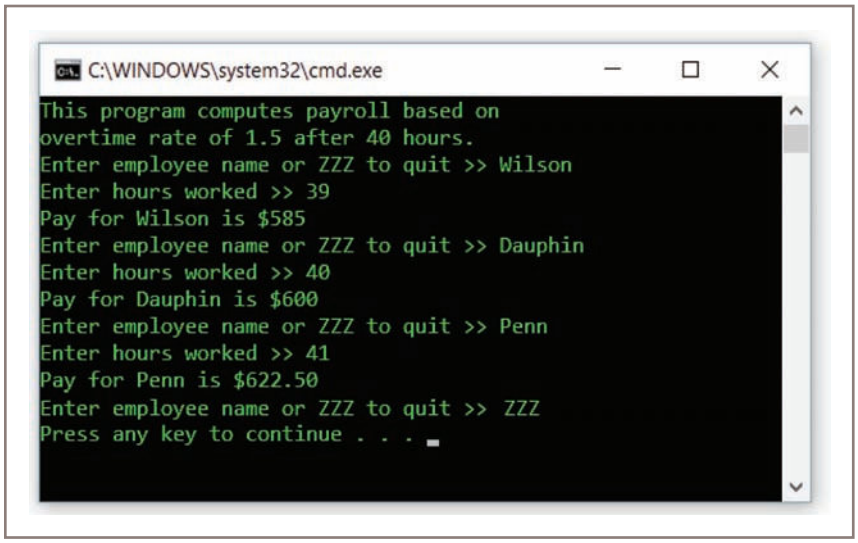


Figure 4-4 Sample execution of the overtime payroll program in Figure 4-3



Watch the video *Boolean Expressions and Decisions*.

TWO TRUTHS & A LIE

The Selection Structure

1. The `if-then` clause is the part of a decision that executes when a tested condition in a decision is true.
2. The `else` clause is the part of a decision that executes when a tested condition in a decision is true.
3. A Boolean expression is one whose value is true or false.

The false statement is #2. The `else` clause is the part of a decision that executes when a tested condition in a decision is false.

Using Relational Comparison Operators

Quick Reference 4-2 describes the six **relational comparison operators** supported by all modern programming languages. Each of these operators is binary—that is, like the arithmetic operators you learned about in Chapter 2, each relational comparison operator

in Quick Reference 4-2 requires two operands, one on each side. Each expression that uses one of these operators evaluates to true or false. Notice that some operators are formed using two characters with no space between them.

QUICK REFERENCE 4-2 Relational Comparison Operators

Operator	Name	Discussion
=	Equivalency operator	Evaluates as true when its operands are equivalent.
>	Greater-than operator	Evaluates as true when the left operand is greater than the right operand.
<	Less-than operator	Evaluates as true when the left operand is less than the right operand.
>=	Greater-than-or-equal-to operator	Evaluates as true when the left operand is greater than or equivalent to the right operand.
<=	Less-than-or-equal-to operator	Evaluates as true when the left operand is less than or equivalent to the right operand.
<>	Not-equal-to operator	Evaluates as true when its operands are not equivalent.

Both operands in a comparison expression must be the same data type; that is, you can compare numeric values to other numeric values, and text strings to other strings. Some programming languages allow exceptions; for example, you can compare a character to a number using the character's numeric code value. Appendix A contains more information about coding systems. In this book, only operands of the same data type will be compared.

In any comparison, the two values can be either variables or constants. For example, the following Boolean expression uses the equivalency operator to compare a variable, `currentTotal`, to a numeric constant, 100:

```
currentTotal = 100
```

Depending on the value of `currentTotal`, the expression is true or false.



An expression that uses the equivalency operator = looks like an assignment statement. For that reason, some languages use a different symbol, for example two equal signs (==), to represent equivalency. This book will use the single equal sign. You can determine whether an expression is a comparison or an assignment by its context. If a statement containing an equal sign appears in a diamond in a flowchart or in an `if` statement in pseudocode, then it is a comparison operator.

As another example, in the following expression both values are variables, and the result is also true or false depending on the values stored in each of the two variables:

```
currentTotal = previousTotal
```

Although it's legal, you would never use expressions in which you compare two literal constants—for example, `20 = 20` or `30 = 40`. Such expressions are **trivial expressions** because each will always evaluate to the same result: true for `20 = 20` and false for `30 = 40`.

Some languages require special operations to compare strings, but this book will assume that the standard comparison operators work correctly with strings based on their alphabetic values. For example, the comparison `"black" < "blue"` would be evaluated as true because `"black"` precedes `"blue"` alphabetically. Usually, string variables are not considered to be equal unless they are identical, including the spacing and whether they appear in uppercase or lowercase. For example, `"black pen"` is not equal to `"blackpen"`, `"BLACK PEN"`, or `"Black Pen"`.

Any decision can be made using combinations of just three types of comparisons: equal, greater than, and less than. You never need the three additional comparisons (greater than or equal, less than or equal, or not equal), but using them often makes decisions more convenient. For example, assume that you need to issue a 10 percent discount to any customer whose age is 65 or greater, and charge full price to other customers. Figure 4-5 shows how you can use either the greater-than-or-equal-to symbol or the less-than symbol to express the same logic that sets a discount when `customerAge` is at least 65.

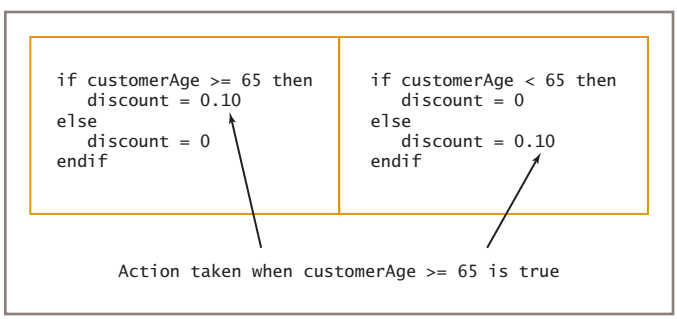


Figure 4-5 Identical logic expressed using >= and <

In any decision for which `a >= b` is true, then `a < b` is false. Conversely, if `a >= b` is false, then `a < b` is true. By rephrasing the question and swapping the actions taken based on the outcome, you can make the same decision in multiple ways. The clearest route is often to ask a question so the positive or true outcome results in the action that was your

motivation for making the test. When your company policy is to "provide a discount for those who are 65 and older," the phrase *greater than or equal to 65* comes to mind, so it is the most natural to use. Conversely, if your policy is to "provide no discount for those under 65," then it is more natural to use the *less than 65* syntax. Either way, the same people receive a discount.

Comparing two amounts to decide if they are *not* equal to each other is the most confusing of all the comparisons. Using *not equal to* in decisions involves thinking in double negatives, which can make you prone to introducing logical errors into your programs. For example, consider the flowchart segment in Figure 4-6.

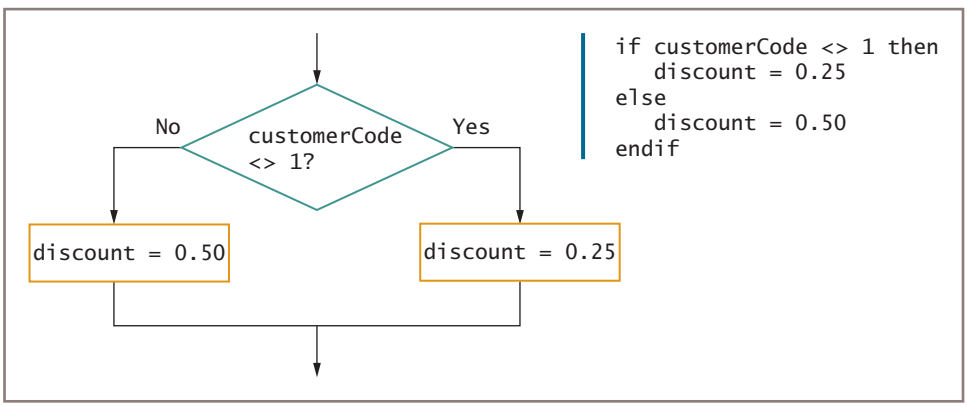


Figure 4-6 Using a negative comparison



Instead of <>, some languages use an exclamation point followed by an equal sign (!=) as the not-equal-to operator. In a flowchart or pseudocode, you might prefer to use the algebraic not-equal-to symbol (≠) or to spell out the words *not equal to*.

In Figure 4-6, if the value of `customerCode` *is* equal to 1, the logical flow follows the false branch of the selection. If `customerCode` <> 1 is true, the discount is 0.25; if `customerCode` <> 1 is not true, it means the `customerCode` *is* 1, and the `discount` is 0.50. Even reading the phrase "if `customerCode` is not equal to 1 is not true" is awkward.

Figure 4-7 shows the same decision, this time asked using positive logic. Making the decision based on what `customerCode` *is* is clearer than trying to determine what `customerCode` is *not*.

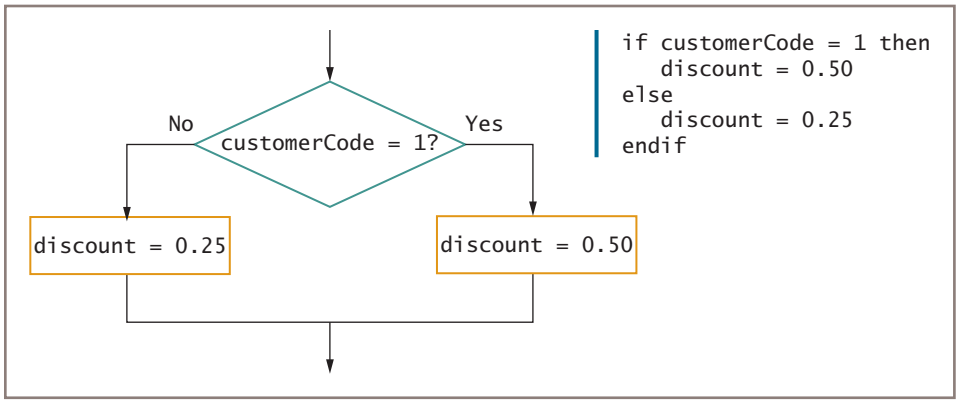


Figure 4-7 Using the positive equivalent of the negative comparison in Figure 4-6



Although negative comparisons can be awkward to use, your meaning is sometimes clearest when using them. Frequently, this occurs when you use an `if` without an `else`, taking action only when some comparison is false. An example would be:

```
if customerZipCode <> LOCAL_ZIP_CODE then
   total = total + deliveryCharge
endif
```

Avoiding a Common Error with Relational Operators

A common error that occurs when programming with relational operators is using the wrong one and missing the boundary or limit required for a selection. If you use the > symbol to make a selection when you should have used >=, all the cases that are equal will go unselected. Unfortunately, people who request programs do not always speak as precisely as a computer. If, for example, your boss says, "Write a program that selects all employees over 65," does she mean to include employees who are 65 or not? In other words, is the comparison `age > 65` or `age >= 65`? Although the phrase *over 65* indicates *greater than 65*, people do not always say what they mean, and the best course of action is to double-check the intended meaning with the person who requested the program—for example, the end user, your supervisor, or your instructor. Similar phrases that can cause misunderstandings are *no more than*, *at least*, and *not under*.



Many modern programming languages support Boolean data types. Instead of holding numbers or words, each Boolean variable can hold only one of two values—true or false. In other words, if `hasWorkedOvertime` is a Boolean variable, then the result of evaluating `hours > 40` can be assigned to it.

TWO TRUTHS & A LIE

Using Relational Comparison Operators

1. Usually, you can compare only values that are of the same data type.
2. A Boolean expression is defined as one that decides whether two values are equal.
3. In any logical comparison expression, the two values compared can be either variables or constants.

The false statement is #2. Although deciding whether two values are equal is a Boolean expression, so is deciding whether one is greater than or less than another. A Boolean expression is one that results in a true or false value.

Understanding *AND* Logic

Often, you need to evaluate more than one expression to determine whether an action should take place. When you make multiple evaluations before an outcome is determined, you create a **compound condition**. For example, suppose you work for a cell phone company that charges customers as follows:

- The basic monthly service bill is $30.
- An additional $20 is billed to customers who make more than 100 calls that last for a total of more than 500 minutes.

The logic needed for this billing program includes an **AND decision**—an expression that tests a condition with two parts that must both evaluate to true for an action to take place. In this case, both a minimum number of calls must be made *and* a minimum number of minutes must be used before the customer is charged the premium amount. A decision that uses an AND expression can be constructed using a **nested decision**, or a **nested `if`**—that is, a decision within the `if-then` or `else` clause of another decision. In Chapter 3, you learned that you can always stack and nest any of the three basic structures. A series of nested `if` statements is also called a **cascading `if` statement**. The flowchart and pseudocode for the program that determines the charges for cell phone customers is shown in Figure 4-8.



The logic for the cell phone billing program assumes that the customer data is retrieved from a file. This eliminates the need for prompts and keeps the program shorter so you can concentrate on the decision-making process. If this were an interactive program, you would use a prompt before each input statement. Chapter 7 covers file processing and explains a few additional steps you can take when working with files.

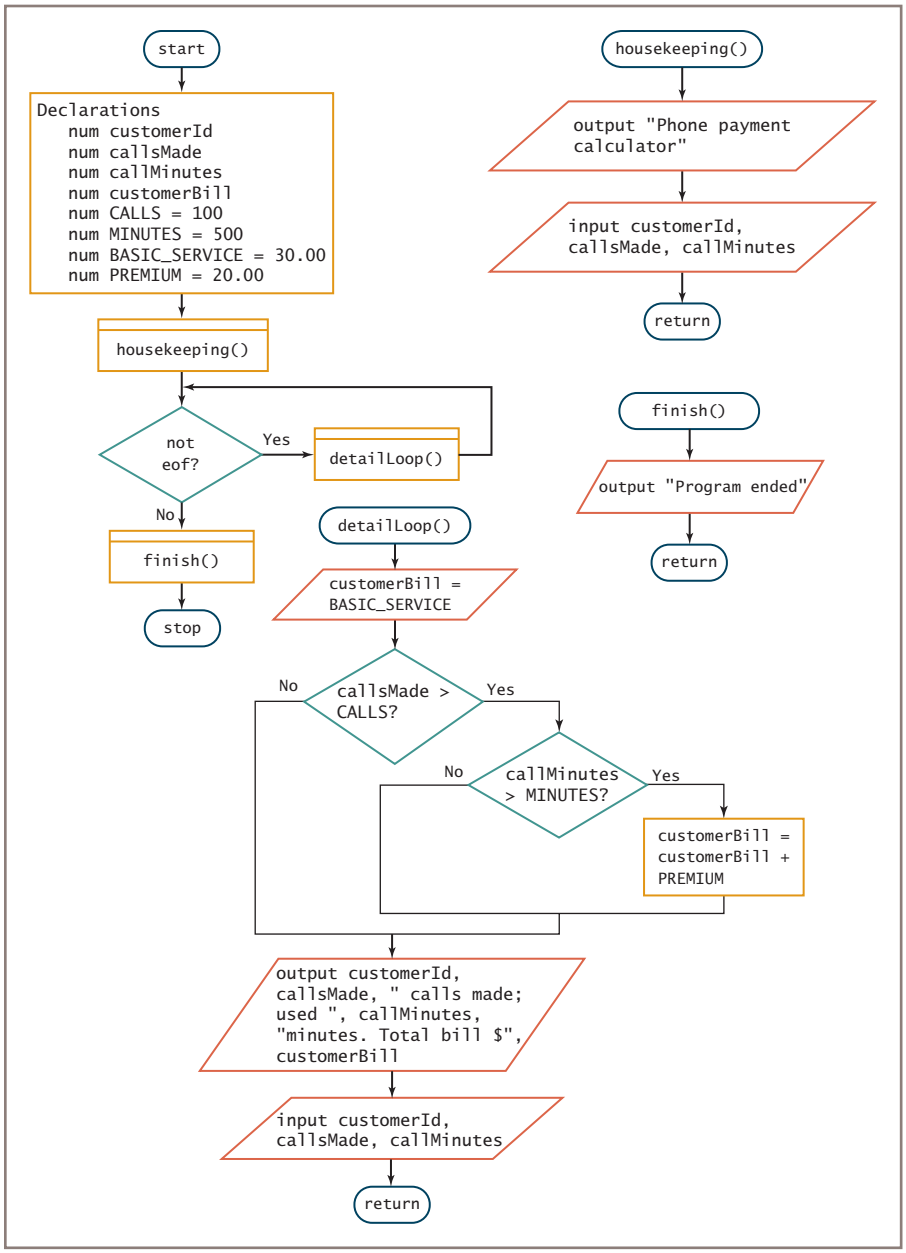


Figure 4-8 Flowchart and pseudocode for cell phone billing program *(continues)*

(continued)

```
start
   Declarations
      num customerId
      num callsMade
      num callMinutes
      num customerBill
      num CALLS = 100
      num MINUTES = 500
      num BASIC_SERVICE = 30.00
      num PREMIUM = 20.00
   housekeeping()
   while not eof
      detailLoop()
   endwhile
   finish()
stop

housekeeping()
   output "Phone payment calculator"
   input customerId, callsMade, callMinutes
return

detailLoop()
   customerBill = BASIC_SERVICE
   if callsMade > CALLS then
      if callMinutes > MINUTES then
         customerBill = customerBill + PREMIUM
      endif
   endif
   output customerId, callsMade, " calls made; used ",
      callMinutes, " minutes. Total bill $", customerBill
   input customerId, callsMade, callMinutes
return

finish()
   output "Program ended"
return
```

Figure 4-8 Flowchart and pseudocode for cell phone billing program

In Figure 4-8, the appropriate variables and constants are declared, and then the `housekeeping()` module displays an introductory heading and gets the first set of input data. After control returns to the mainline logic, the `eof` condition is tested, and if data entry is not complete, the `detailLoop()` module executes. In the `detailLoop()` module, the customer's bill is set to the standard fee, and then the nested decision executes. In the nested `if` structure in Figure 4-8, the expression `callsMade > CALLS` is evaluated first. If this expression is true, only then is the second Boolean expression (`callMinutes > MINUTES`) evaluated. If that expression is also true, then the $20 premium is added to the customer's bill. If either of the tested conditions is false, the customer's bill value is never altered, retaining the initially assigned `BASIC_SERVICE` value of $30.

Nesting *AND* Decisions for Efficiency

When you nest two decisions, you must choose which of the decisions to make first. Logically, either expression in an AND decision can be evaluated first. You often can improve your program's performance, however, by correctly choosing which of two selections to make first.

For example, Figure 4-9 shows two ways to design the nested decision structure that assigns a premium to customers' bills if they make more than 100 cell phone calls and use more

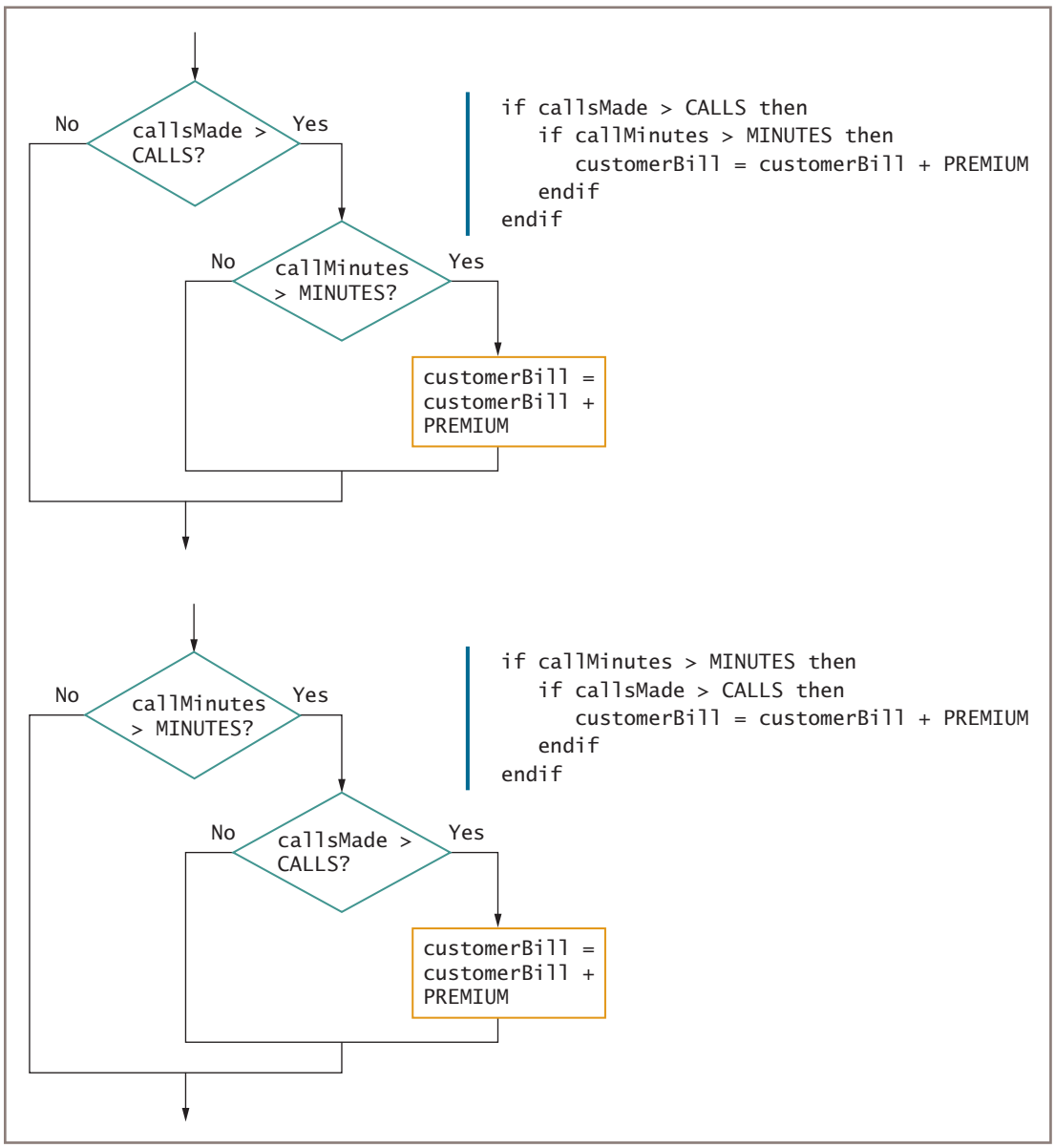


Figure 4-9 Two ways to produce cell phone bills using identical criteria

than 500 minutes in a billing period. The program can ask about calls made first, eliminate customers who have not made more than the minimum, and then ask about the minutes used only for customers who pass (that is, are evaluated as true on) the minimum calls test. Or, the program could ask about the minutes first, eliminate those who do not qualify, and then ask about the number of calls only for customers who pass the minutes test. Either way, only customers who exceed both limits must pay the premium. Does it make a difference which question is asked first? As far as the result goes, no. Either way, the same customers pay the premium—those who qualify on the basis of both criteria. As far as program efficiency goes, however, it *might* make a difference which question is asked first.

Assume that you know that out of 1000 cell phone customers, about 90 percent, or 900, make more than 100 calls in a billing period. Assume that you also know that only about half the 1000 customers, or 500, use more than 500 minutes of call time. If you use the logic shown first in Figure 4-9, and you need to produce 1000 phone bills, the first question, `callsMade > CALLS`, will execute 1000 times. For approximately 90 percent of the customers, or 900 of them, the answer is true, so 100 customers are eliminated from the premium assignment, and 900 proceed to the next question about the minutes used. Only about half the customers use more than 500 minutes, so 450 of the 900 pay the premium, and it takes 1900 decisions to identify them.

Using the alternate logic shown second in Figure 4-9, the first question, `callMinutes > MINUTES`, also will be asked 1000 times—once for each customer. Because only about half the customers use the high number of minutes, only 500 will pass this test and proceed to the question for number of calls made. Then, about 90 percent of the 500, or 450 customers, will pass the second test and be billed the premium amount. In this case, it takes 1500 decisions to identify the 450 premium-paying customers.

Whether you use the first or second decision order in Figure 4-9, the same 450 customers who satisfy both criteria pay the premium. The difference is that when you ask about the number of calls first, the program must make 400 more decisions than when you ask about the minutes used first.

The 400-decision difference between the first and second arrangement in Figure 4-9 doesn't take much time on most computers. But it does take *some* time, and if a corporation has hundreds of thousands of customers instead of only 1000, or if many such decisions have to be made within a program, performance (execution time) can be improved significantly by making decisions in the more efficient order.

Often when you must make nested decisions, you have no idea which event is more likely to occur; in that case, you can legitimately make either comparison first. However, if you do know the probabilities of the conditions, or can make a reasonable guess, the general rule is: *In an AND decision, first evaluate the condition that is less likely to be true.* This eliminates as many instances of the second decision as possible, which speeds up processing time.



Watch the video *Writing Efficient Nested Selections*.

Using the AND Operator

Most programming languages allow you to ask two or more questions in a single comparison by using a **conditional AND operator**, or more simply, an **AND operator** that joins decisions in a single expression. For example, if you want to bill an extra amount to cell phone customers who make more than 100 calls that total more than 500 minutes in a billing period, you can use nested decisions, as shown in the previous section, or you can include both decisions in a single expression by writing an expression such as the following:

```
callsMade > CALLS AND callMinutes > MINUTES
```

When you use one or more AND operators to combine two or more Boolean expressions, each Boolean expression must be true for the entire expression to be evaluated as true. For example, if you ask, "Are you a native-born U.S. citizen and are you at least 35 years old?" the answer to both parts of the question must be *yes* before the response can be a single, summarizing *yes*. If either part of the expression is false, then the entire expression is false.



The conditional AND operator in Java, C++, and C# consists of two ampersands, with no spaces between them (&&). In Visual Basic, you use the keyword **And**. This book uses AND.

One tool that can help you understand the AND operator is a truth table. **Truth tables** are diagrams used in mathematics and logic to help describe the truth of an entire expression based on the truth of its parts. Quick Reference 4-3 contains a truth table that lists all the possibilities with an AND operator. As the table shows, for any two expressions x and y, the expression `x AND y` is true only if both x and y are individually true. If either x or y alone is false, or if both are false, then the expression `x AND y` is false.

QUICK REFERENCE 4-3 Truth table for the AND operator

x?	y?	x AND y?
True	True	True
True	False	False
False	True	False
False	False	False

If the programming language you use allows an AND operator, you must realize that the Boolean expression you place first (to the left of the AND operator) is the one that will be evaluated first, and cases that are eliminated based on the first evaluation will not proceed to the second one. In other words, each part of an expression that uses an AND operator is evaluated only as far as necessary to determine whether the entire expression is true or false. This feature is called **short-circuit evaluation**. A computer can make only one decision at a time, so even when your pseudocode looks like the first example in Figure 4-10, the computer

will execute the logic shown in the second example. Even when you use an AND operator, a computer evaluates expressions one at a time, and in order from left to right. As you can see in the truth table, if the first half of an AND expression is false, then the entire expression is false. In that case, there is no point in evaluating the second half. In other words, evaluating an AND expression is interrupted as soon as either part of it is determined to be false.

When two conditions must be true for an action to take place, you are never required to use the AND operator because using nested `if` statements can always achieve the same result. However, using the AND operator often makes your code more concise, less error-prone, and easier to understand.

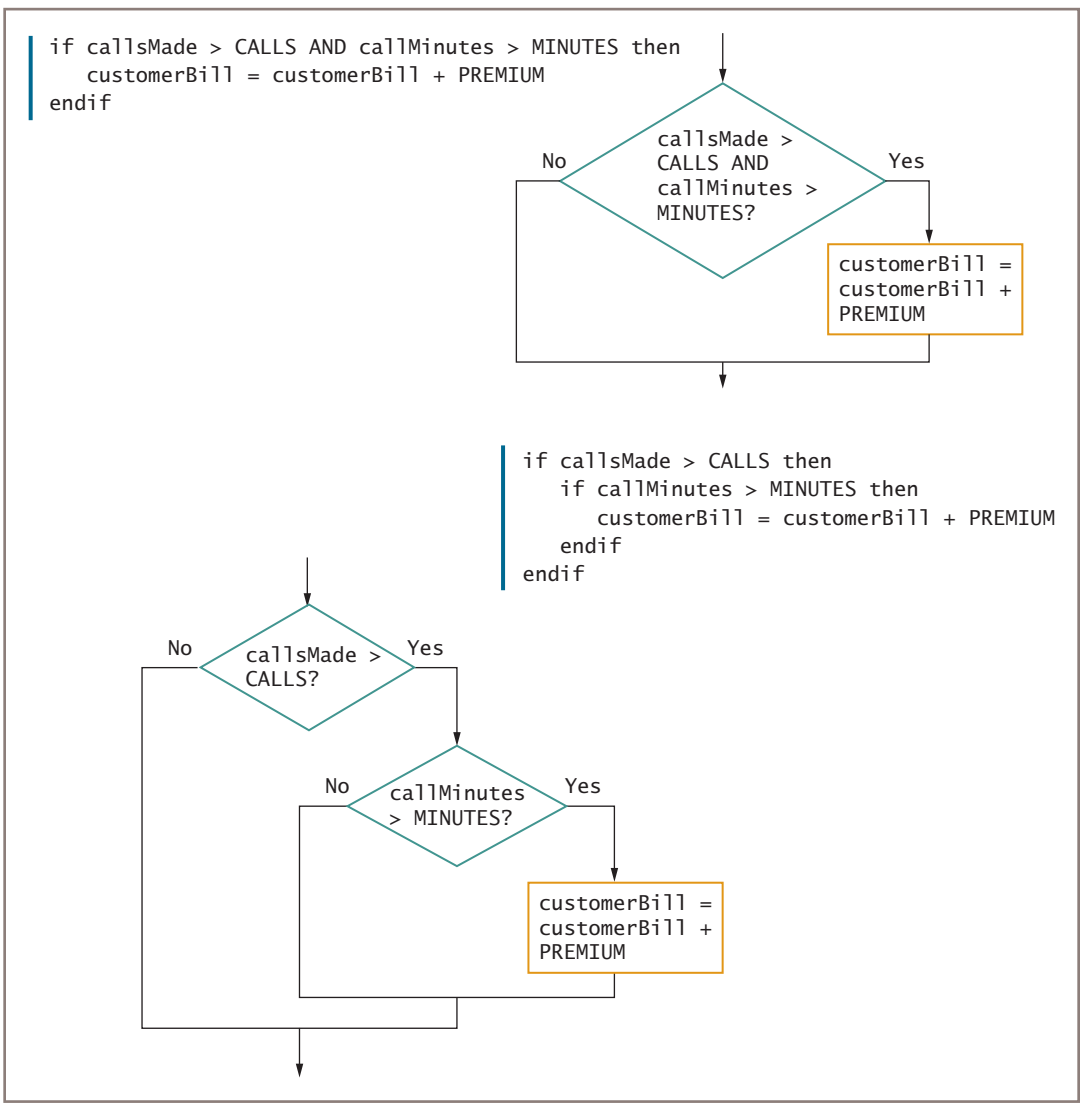


Figure 4-10 Using an AND operator and the logic behind it



There can be confusion between the terms *conditional operator* and *logical operator*. *Conditional operator* most often is used when short-circuit evaluation is in effect, but you sometimes will hear programmers use the two terms interchangeably. To complicate matters, some programmers call the operators *conditional logical operators*.

Avoiding Common Errors in an *AND* Selection

Make Sure Decisions that Should Be Nested Are Nested

When you need to satisfy two or more criteria to initiate an event in a program, you must make sure that the second decision is made entirely within the first decision. For example, if a program's objective is to add a $20 premium to the bill of cell phone customers who exceed 100 calls and 500 minutes in a billing period, then the program segment shown in Figure 4-11 contains three different types of logic errors.

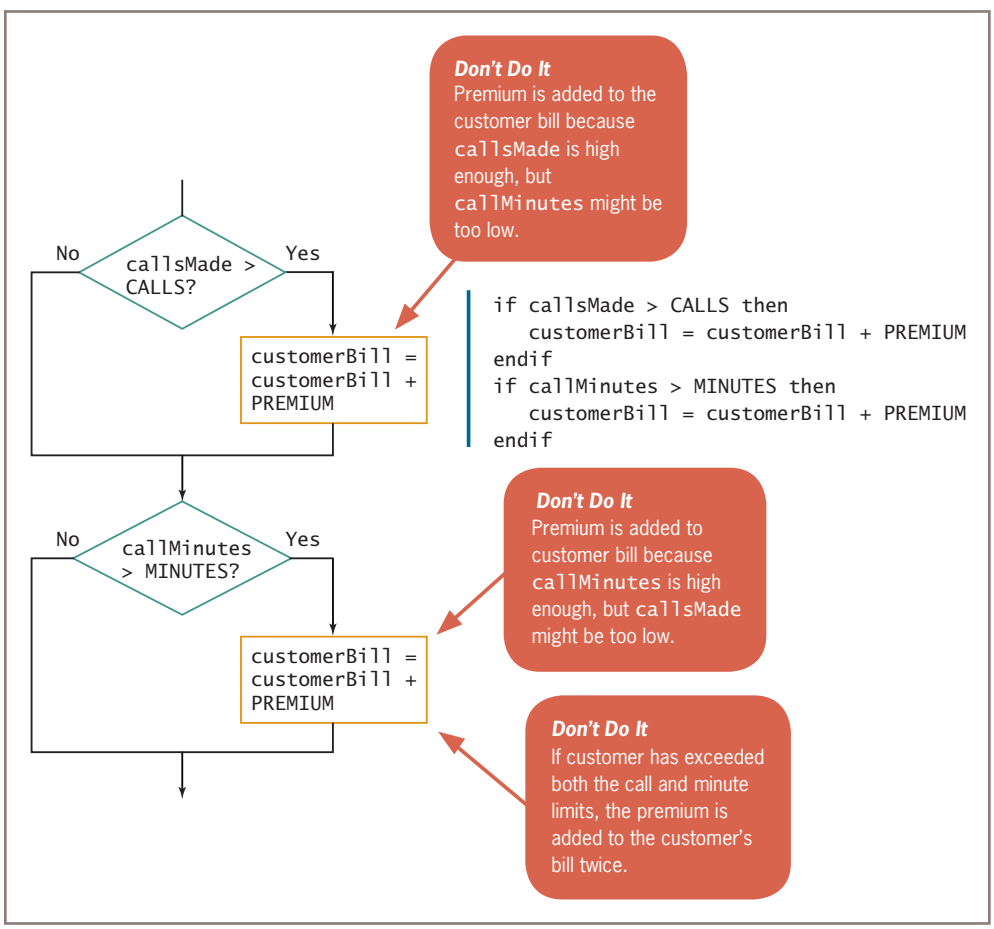


Figure 4-11 Incorrect logic to add a $20 premium to the bills of cell phone customers who meet two criteria

The logic in Figure 4-11 shows that if a customer makes too many calls, $20 is added to his bill. This customer should not necessarily be billed extra because the customer's minutes might be low. In addition, in Figure 4-11, a customer who has made few calls is not eliminated from the second decision. Instead, all customers are subjected to the minutes evaluation, and some are assigned the premium even though they might have made only a few calls. Additionally, any customer who passes both tests because his calls and minutes are both high has the premium added to his bill twice. The decisions in Figure 4-11 are stacked when they should be nested, so the logic they represent is *not* correct for this problem.

Make Sure that Boolean Expressions Are Complete

When you use the `AND` operator in most languages, you must provide a complete Boolean expression on each side of the operator. In other words, the following expression would be valid to use to find `callMinutes` between 100 and 200:

```
callMinutes > 100 AND callMinutes < 200
```

However, `callMinutes > 100 AND < 200` would not be valid because the less-than sign and 200 that follow the `AND` operator do not constitute a complete Boolean expression.

For clarity, you can enclose each Boolean expression in a compound expression in its own set of parentheses. This makes it easier for you to see that each of the `AND` operator's operands is a complete Boolean expression. Use this format if it is clearer to you. For example, you might write the following:

```
if (callMinutes > MINUTES) AND (callsMade > CALLS) then
   customerBill = customerBill + PREMIUM
endif
```

Make Sure that Expressions Are Not Inadvertently Trivial

When you use the `AND` operator, it is easy to inadvertently create trivial expressions that are always true or always false. For example, suppose that you want to display a message if any cell phone customer makes a very low or high number of calls, say fewer than 5 or more than 2000. You might be tempted to write the following expression, but it would be incorrect:

```
if callsMade < 5 AND callsMade > 2000 then
   output "Irregular usage"
  endif
```

Don't Do It
This AND expression is always false.

This `if` statement never results in a displayed message because both parts of the AND expression can never be true at the same time. For example, if the value of `callsMade` is greater than 2000, its value is not less than 5, and if `callsMade` is less than 5, it is not greater than 2000. The programmer intended to use the following code:

```
if callsMade < 5 then
  output "Irregular usage"
else
```

```
  if callsMade > 2000 then
   output "Irregular usage"
  endif
endif
```

Alternately, the programmer might use an `OR` operator, as you will see in the next section.

TWO TRUTHS & A LIE

Understanding *AND* Logic

1. When you nest selection structures because the resulting action requires that two conditions be true, either decision logically can be made first and the results will be the same.
2. When two selections are required for an action to take place, you often can improve your program's performance by appropriately choosing which selection to make first.
3. To improve efficiency in a nested selection in which two conditions must be true for some action to occur, you should first evaluate the condition that is more likely to be true.

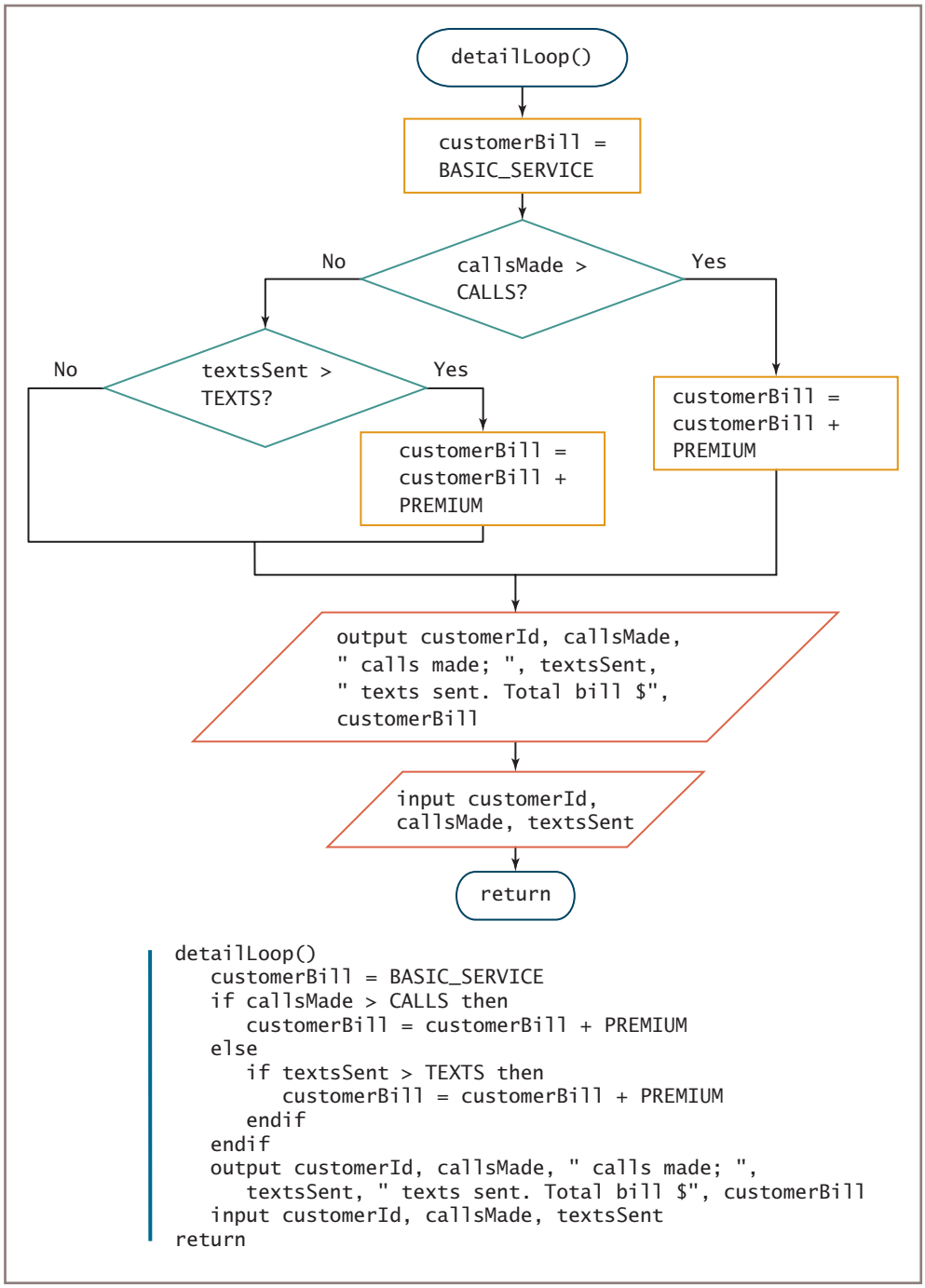
The false statement is #3. For efficiency in a nested selection, you should first evaluate the condition that is less likely to be true.

Understanding *OR* Logic

Sometimes you want to take action when one *or* the other of two conditions is true. This is called an **OR decision** because either one condition *or* some other condition must be met in order for some action to take place. If someone asks, "Are you free for dinner Friday or Saturday?" only one of the two conditions has to be true for the answer to the whole question to be *yes*; only if the answers to both halves of the question are false is the value of the entire expression false. For example, suppose the cell phone company has established a new fee schedule as follows:

- The basic monthly service bill is $30.
- An additional $20 is billed to customers who make more than 100 calls or send more than 200 text messages.

Figure 4-12 shows the altered `detailLoop()` module of the billing program that accomplishes this objective. Assume that new declarations have been made for the `textsSent` variable and a `TEXTS` constant that has been assigned the value 200.




```
detailLoop()
   customerBill = BASIC_SERVICE
   if callsMade > CALLS then
      customerBill = customerBill + PREMIUM
   else
      if textsSent > TEXTS then
         customerBill = customerBill + PREMIUM
      endif
   endif
   output customerId, callsMade, " calls made; ",
      textsSent, " texts sent. Total bill $", customerBill
   input customerId, callsMade, textsSent
return
```

Figure 4-12 Flowchart and pseudocode for cell phone billing program in which a customer must meet one or both of two criteria to be billed a premium

The `detailLoop()` in the program in Figure 4-12 tests the expression `callsMade > CALLS`, and if the result is true, the premium amount is added to the customer's bill. Because making many calls is enough for the customer to incur the premium, there is no need for further questioning. If the customer has not made more than 100 calls, only then does the program need to ask whether `textsSent > TEXTS` is true. If the customer did not make more than 100 calls, but did send more than 200 text messages, then the premium amount is added to the customer's bill.

Writing *OR* Selections for Efficiency

As with an AND condition, when you use an OR condition, you can choose to ask either question first. For example, you can add an extra $20 to the bills of customers who meet one or the other of two criteria using the logic in either part of Figure 4-13.

You might have guessed that one of these solutions is superior to the other when you have some background information about the relative likelihood of each tested condition. For example, let's say you know that out of 1000 cell phone customers, about 90 percent, or 900, make more than 100 calls in a billing period. Suppose that you also know that only about half of the 1000 customers, or 500, send more than 200 text messages.

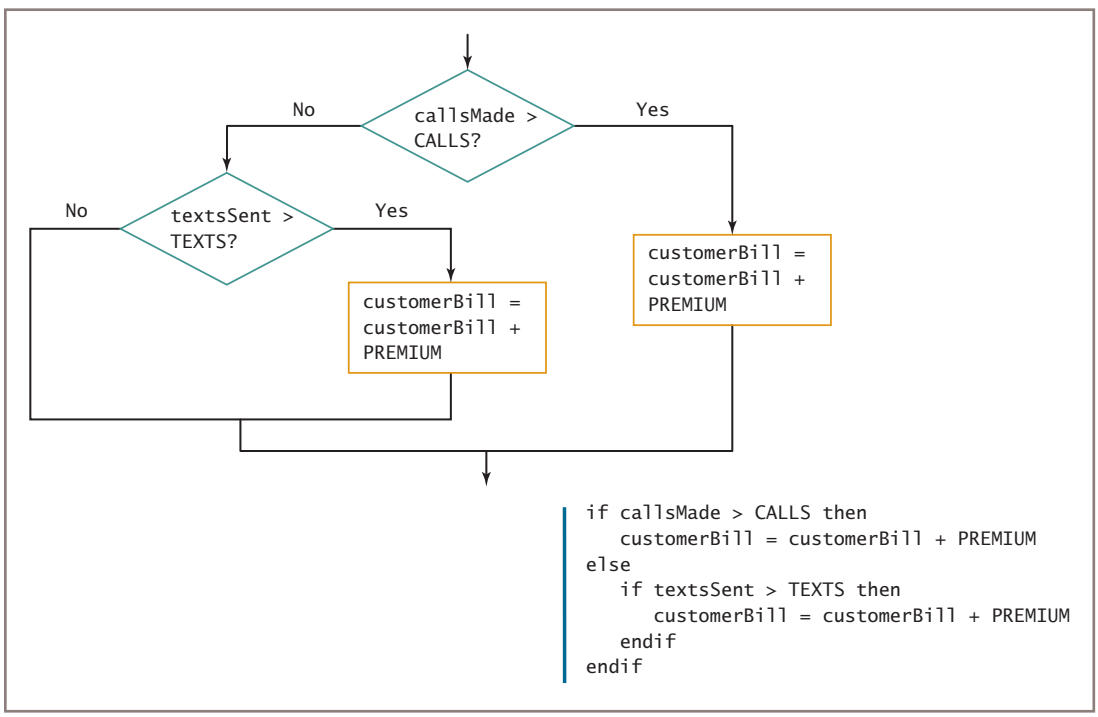


Figure 4-13 Two ways to assign a premium to bills of customers who meet one of two criteria *(continues)*

(continued)

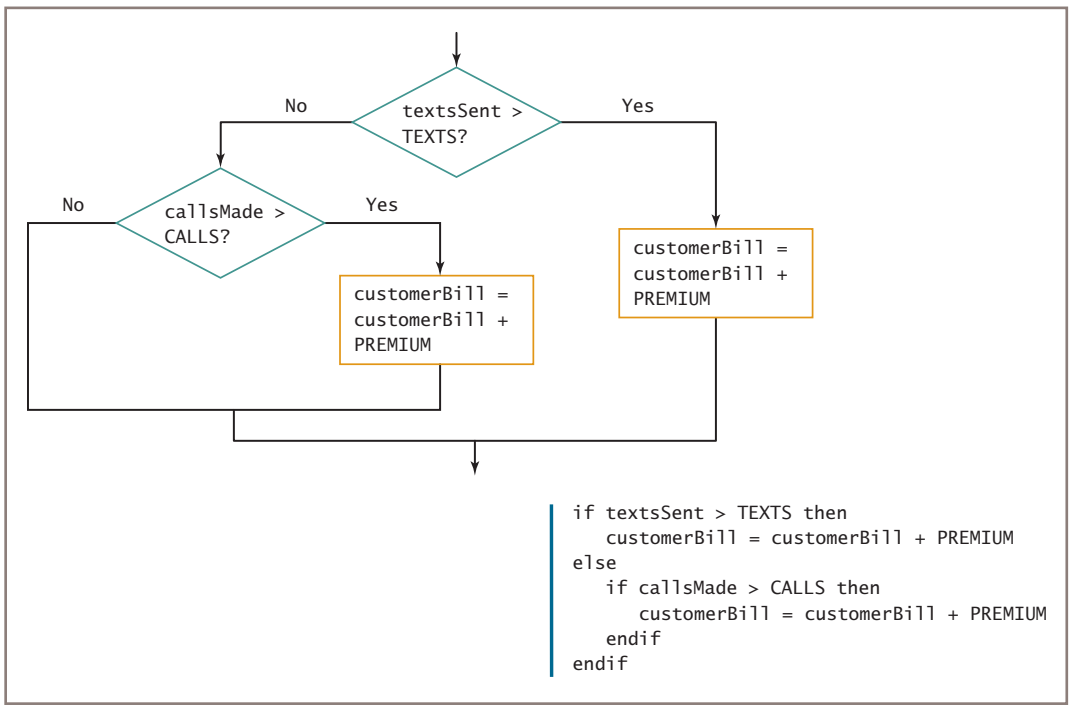


Figure 4-13 Two ways to assign a premium to bills of customers who meet one of two criteria

When you use the logic shown in the first half of Figure 4-13, you first evaluate calls made. For 900 customers, the value of `callsMade` is greater than `CALLS`, so you add the premium to their bills. Only about 100 sets of customer data continue to the next decision regarding the text messages, where about 50 percent of the 100, or 50, are billed the extra amount. In the end, you have made 1100 decisions to correctly add premium amounts for 950 customers.

If you use the OR logic in the second half of Figure 4-13, you ask about text messages first—1000 times, once each for 1000 customers. The result is true for 50 percent, or 500 customers, whose bill is increased. For the other 500 customers, you ask about the number of calls made. For 90 percent of the 500, the result is true, so premiums are added for 450 additional people. In the end, the same 950 customers are billed an extra $20—but this approach requires executing 1500 decisions, 400 more decisions than when using the first decision logic.

The general rule is: *In an OR decision, first evaluate the condition that is more likely to be true*. This approach eliminates as many executions of the second decision as possible, and the time it takes to process all the data is decreased. As with the AND situation, in an OR situation, it is more efficient to eliminate as many extra decisions as possible.

Using the OR Operator

If you need to take action when either one or the other of two conditions is met, you can use two separate, nested selection structures, as in the previous examples. Most programming languages, however, allow you to make two or more decisions in a single comparison by using a **conditional OR operator** (or simply the **OR operator**). For example, you can make the following evaluation:

```
callsMade > CALLS OR textsSent > TEXTS
```

When you use the logical OR operator, only one of the listed conditions must be met for the resulting action to take place. Quick Reference 4-4 contains the truth table for the OR operator. As you can see in the table, the entire expression `x OR y` is false only when `x` and `y` each are false individually.

QUICK REFERENCE 4-4 Truth Table for the OR Operator

x?	y?	x OR y?
True	True	True
True	False	True
False	True	True
False	False	False

If the programming language you use supports an OR operator, you still must realize that the comparison you place first is the expression that will be evaluated first, and cases that pass the test of the first comparison will not proceed to the second comparison. As with the AND operator, this feature is called *short-circuiting*. The computer can make only one decision at a time; even when you write code as shown at the top of Figure 4-14, the computer will execute the logic shown at the bottom.



C#, C++, C, and Java use two pipe symbols (||) as the logical OR operator. In Visual Basic, the keyword used for the operator is `Or`. This book uses OR.

Avoiding Common Errors in an *OR* Selection

Make Sure that Boolean Expressions Are Complete

As with the AND operator, most programming languages require a complete Boolean expression on each side of the OR operator. For example, if you wanted to display a message when customers make either 0 calls or more than 2000 calls, the expression `callsMade = 0 OR callsMade > 2000` is

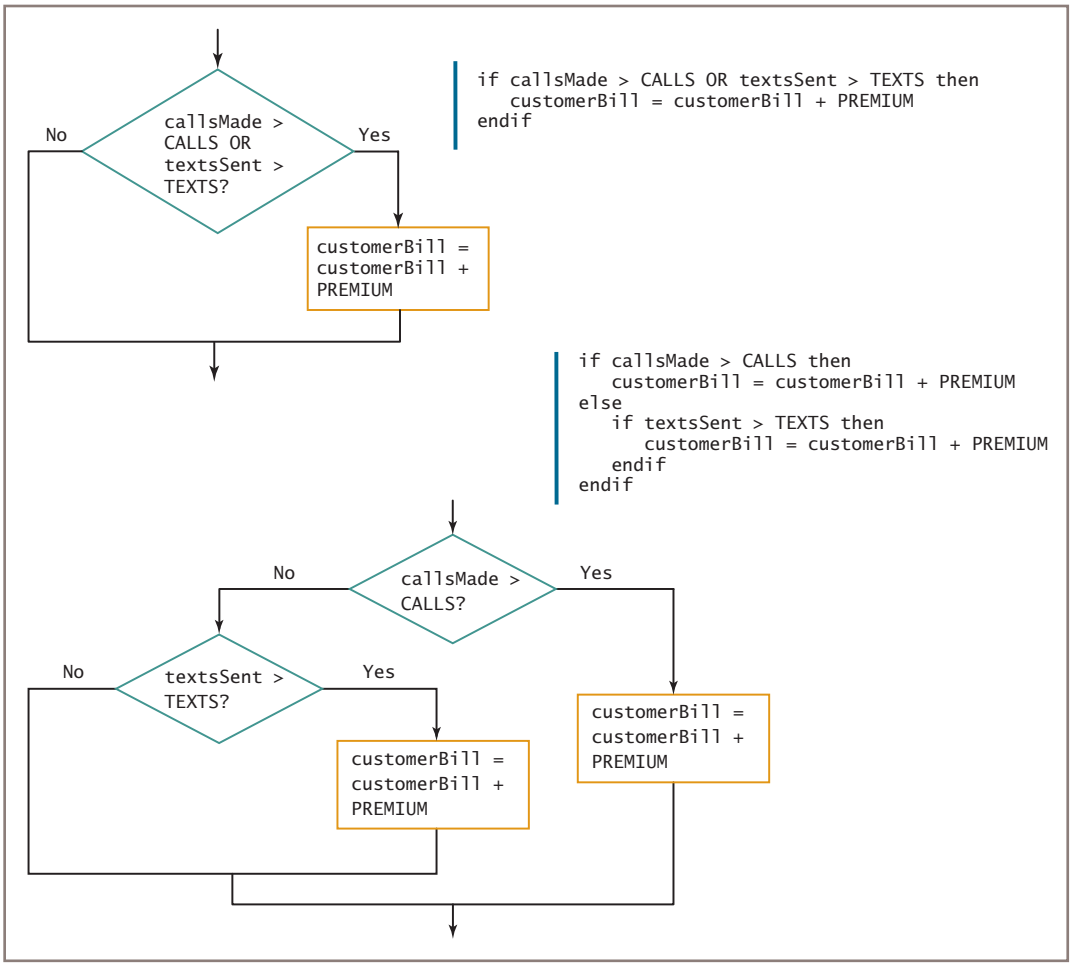


Figure 4-14 Using an OR operator and the logic behind it

appropriate but `callsMade = 0 OR > 2000` is not, because the expression to the right of the `OR` operator is not a complete Boolean expression.

Also, as with the `AND` operator, you can enclose each simple comparison within parentheses for clarity if you want, as in the following statement:

```
if (callsMade = 0) OR (callsMade > 2000) then
   output "Irregular usage"
endif
```

Make Sure that Selections Are Structured

You might have noticed that the assignment statement `customerBill = customerBill + PREMIUM` appears twice in the decision-making processes in

Figures 4-13 and 4-14. When you create a flowchart, the temptation is to draw the logic to look like Figure 4-15. Logically, you might argue that the flowchart in Figure 4-15 is correct because the correct customers are billed the extra $20. This flowchart, however, is not structured. The second question is not a self-contained structure with one entry and exit point; instead, the flowline breaks out of the inner selection structure to join the `Yes` side of the outer selection structure.

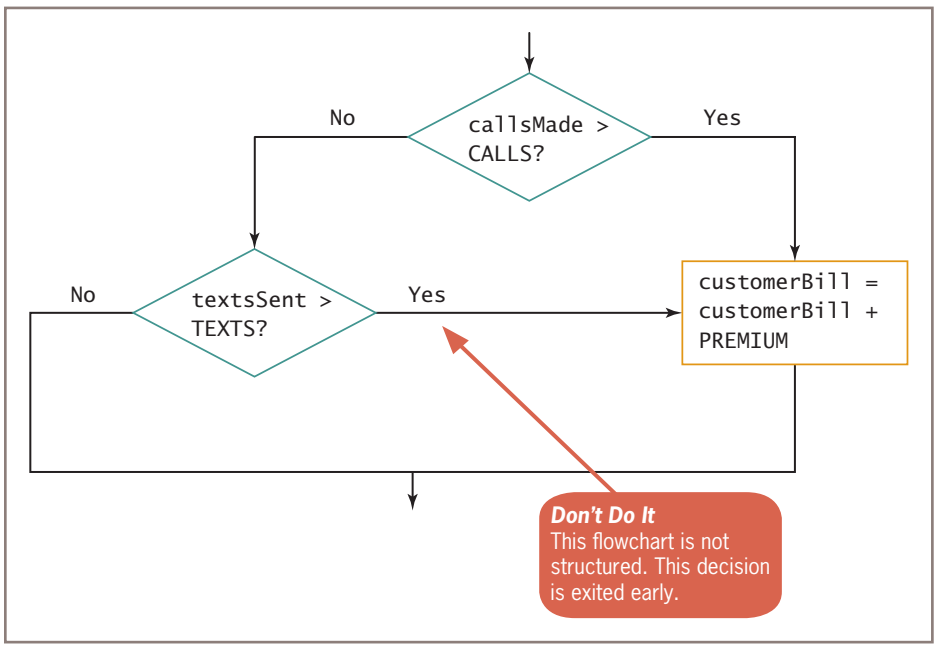


Figure 4-15 Unstructured flowchart for determining customer cell phone bill

Make Sure that You Use OR Selections When They Are Required

The OR selection has additional potential for errors because of the differences in the way people and computers use language. When your boss wants to add an extra amount to the bills of customers who make more than 100 calls *or* send more than 200 texts, she is likely to say, "Add $20 to the bill of anyone who makes more than 100 calls and to anyone who sends more than 200 texts." Her request contains the word *and* between two types of people—those who made many calls and those who sent many texts—placing the emphasis on the people.

However, each decision you make is about the added $20 for a single customer who has met one criterion *or* the other *or* both. In other words, the OR condition is between each customer's attributes, and not between different customers. Instead of the manager's previous statement, it would be clearer if she said, "Add $20 to the bill of anyone who has made more than 100 calls or has sent more than 200 texts," but you can't count on people to speak like computers. As a programmer, you have the job of clarifying what really is being requested. Often, a casual request for A *and* B logically means a request for A *or* B.

Make Sure that Expressions Are Not Inadvertently Trivial

The way we use English can cause another type of error when you are required to find whether a value falls between two other values. For example, a movie theater manager might say, "Provide a discount to patrons who are under 13 years old and to those who are over 64 years old; otherwise, charge the full price." Because the manager has used the word *and* in the request, you might be tempted to create the decision shown in Figure 4-16; however, this logic will not provide a discounted price for any movie patron. You must remember that every time the decision is made in Figure 4-16, it is made for a single movie patron. If `patronAge` contains a value lower than 13, then it cannot possibly contain a value over 64. Similarly, if `patronAge` contains a value over 64, there is no way it can contain a lesser value. Therefore, no value could be stored in `patronAge` for which both parts of the AND condition could be true—and the price will never be set to the discounted price for any patron. In other words, the decision made in Figure 4-16 is trivial. Figure 4-17 shows the correct logic.

A similar error can occur in your logic if the theater manager says something like, "Don't give a discount—that is, do charge full price—if a patron is over 12 or under 65." Because the word *or* appears in the request, you might plan your logic to resemble Figure 4-18. No patron ever receives a discount, because every patron is either over 12 or under 65. Remember, in an OR decision, only one of the conditions needs to be true for the entire

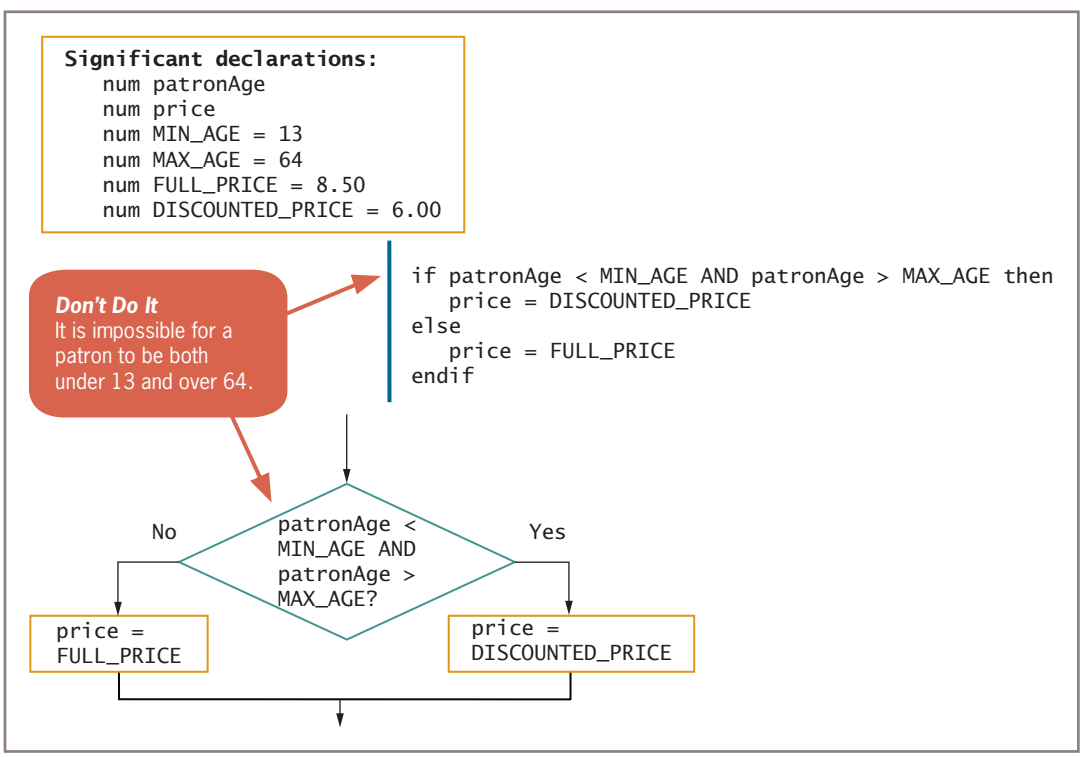


Figure 4-16 Incorrect logic that attempts to provide a discount for young and old movie patrons

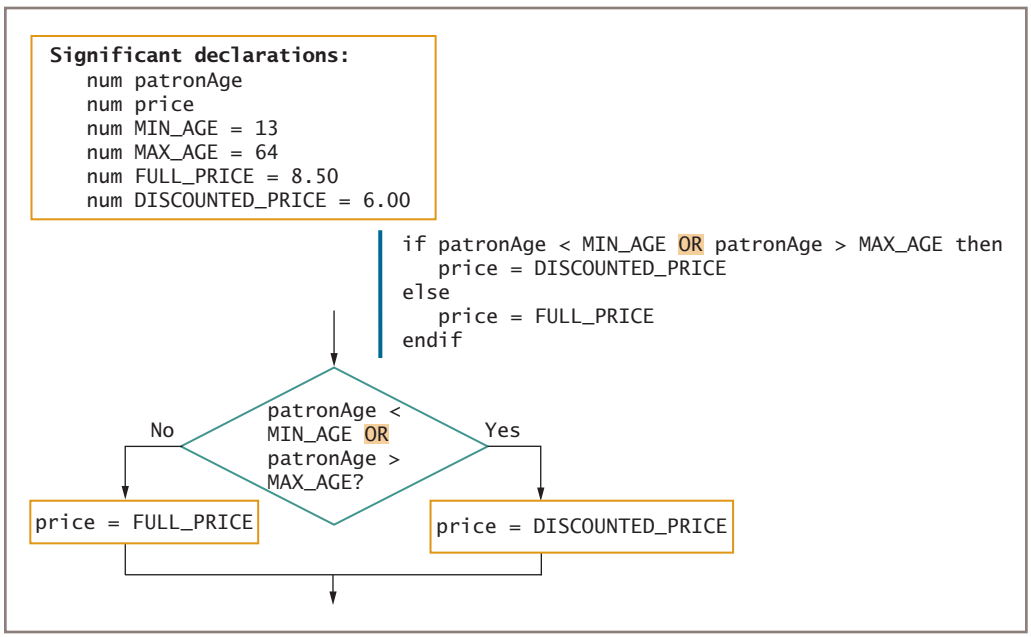


Figure 4-17 Correct logic that provides a discount for young and old movie patrons

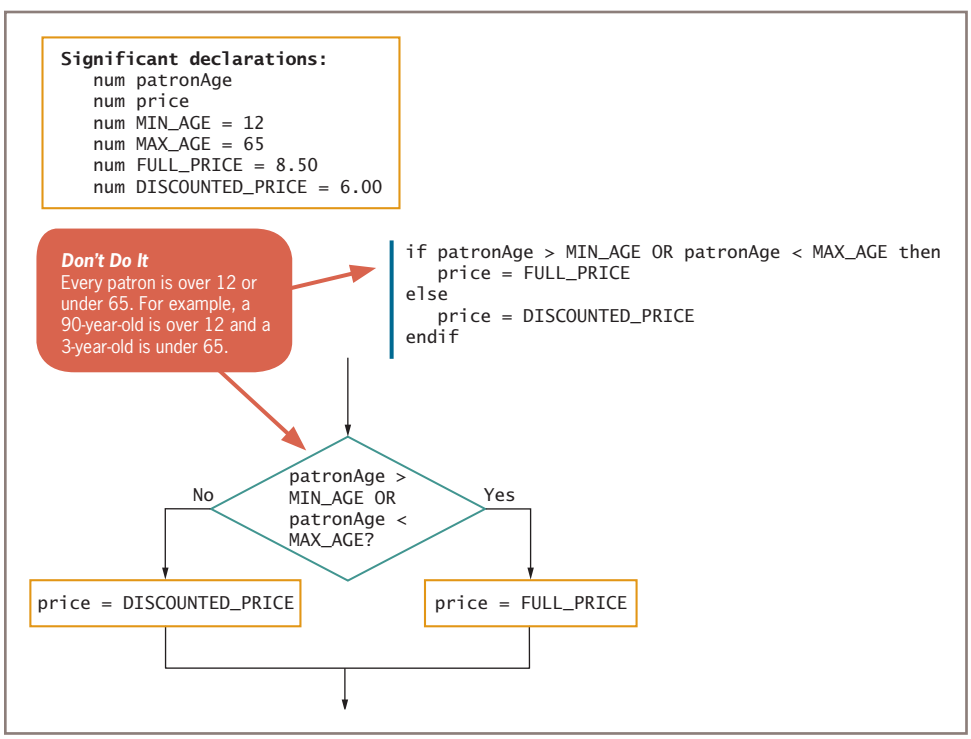


Figure 4-18 Incorrect logic that attempts to charge full price for patrons whose age is over 12 and under 65

expression to be evaluated as true. So, for example, because a patron who is 10 is under 65, the full price is charged, and because a patron who is 70 is over 12, the full price also is charged. Figure 4-19 shows the correct logic for this decision.

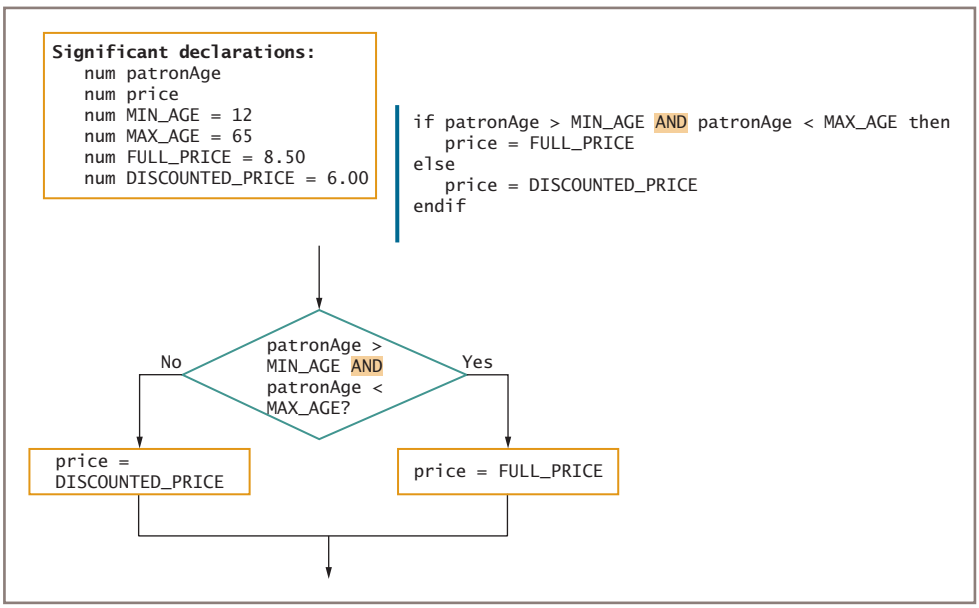


Figure 4-19 Correct logic that charges full price for patrons whose age is over 12 and under 65

Watch the video *Looking in Depth at AND and OR Decisions*.

TWO TRUTHS & A LIE

Understanding *OR* Logic

1. In an OR selection, two or more conditions must be met in order for an event to take place.
2. When you use an OR selection with two conditions, you can choose to evaluate either condition first and still achieve a usable program.
3. The general rule is: In an OR decision, first evaluate the condition that is more likely to be true.

The false statement is #1. In an OR selection, only one of two conditions must be met in order for an event to take place.

Understanding *NOT* Logic

Besides AND and OR operators, most languages support a NOT operator. You use the **NOT operator** to reverse the meaning of a Boolean expression. For example, the following statement outputs *Can register to vote* when age is greater than or equal to 18:

```
if NOT (age < 18) then
   output "Can register to vote"
endif
```

This example uses parentheses around the expression age < 18 to show that the NOT operator applies to the entire Boolean expression age < 18. Without the parentheses, some languages might try to evaluate the expression NOT age before testing the less-than comparison. Depending on the programming language, the result would either be incorrect or the statement would not execute at all.

Quick Reference 4-5 contains the truth table for the NOT operator. As you can see, any expression that would be true without the operator becomes false with it, and any expression that would be false without the operator becomes true with it.

QUICK REFERENCE 4-5 Truth table for the NOT operator

x?	NOT x?
True	False
False	True

You have already learned that arithmetic operators such as + and −, and relational operators such as > and <, are binary operators that require two operands. Unlike those operators, the NOT operator is a **unary operator**, meaning it takes only one operand—that is, you do not use it between two expressions, but you use it in front of a single expression.

As when using the binary not-equal-to comparison operator, using the unary NOT operator can create confusing statements because negative logic is difficult to follow. For example, if your intention is not to allow voter registration for those under 18, then either of the following two statements will accomplish your goal, but the second one is easier to understand:

```
if NOT (age < 18) then
   output "Can register to vote"
endif
if age >= 18 then
   output "Can register to vote"
endif
```

Avoiding a Common Error in a *NOT* Expression

Because thinking with negatives is hard, you need to be careful not to create trivial expressions when using NOT logic. For example, suppose your boss tells you to display a message for all employees *except* those in Departments 1 and 2. You might write the following incorrect code:

```
if NOT (employeeDept = 1) OR NOT (employeeDept = 2) then
   output "Employee is not in Department 1 or 2"
endif
```

Don't Do It
This logic does not eliminate employees in Departments 1 and 2.

Suppose that an employee is in Department 2, and therefore no message should be displayed. For that employee, the following expression is false:

```
employeeDept = 1
```

Therefore, this expression is true:

```
NOT (employeeDept = 1)
```

Because the OR operator's left operand is true, the entire Boolean expression is true, and the message is incorrectly displayed. The correct decision follows:

```
if NOT (employeeDept = 1 OR employeeDept = 2) then
   output "Employee is not in Department 1 or 2"
endif
```



In C++, Java, and C#, the exclamation point is the symbol used for the NOT operator. In those languages, the exclamation point can be used in front of an expression or combined with other comparison operators. For example, the expression *a not equal to b* can be written as `!(a = b)` or as `a != b`. In Visual Basic, the operator is the keyword `Not`. This book uses NOT.

TWO TRUTHS & A LIE

Understanding *NOT* Logic

1. The value of `x <> 0` is the same as the value of NOT (x = 0).
2. The value of `x > y` is the same as the value of NOT (x < y).
3. The value of `x = y OR x > 5` is the same as the value of `x = y OR NOT (x <= 5)`.

The false statement is #2. The value of x > y is not the value of NOT (x < y) because the first expression is false and the second one is true when x and y are equal. The value of x > y is the same as the value of NOT (x <= y).

Items Ordered	Discount Rate (%)
10 or fewer	0
11 to 24	10
25 to 50	15
51 or more	20

Figure 4-20 Discount rates based on items ordered

Making Selections within Ranges

You often need to take action when a variable falls within a range of values. For example, suppose your company provides various customer discounts based on the number of items ordered, as shown in Figure 4-20.

When you write the program that determines a discount rate based on the number of items, you could make hundreds of decisions, evaluating `itemQuantity = 1`, `itemQuantity = 2`, and so on. However, it is more convenient to find the correct discount rate by using a range check.

When you use a **range check,** you compare a variable to a series of values that mark the limiting ends of ranges. To perform a range check, make comparisons using either the lowest or highest value in each range of values. For example, to successfully find each discount rate listed in Figure 4-20, you can make comparisons using either the low end of each range or the high end of each range.

If you want to use the low ends of the ranges (11, 25, and 51), either of these techniques works:

- You can ask: Is `itemQuantity` less than 11? If not, is it less than 25? If not, is it less than 51?
- You can ask: Is `itemQuantity` greater than or equal to 51? If not, is it greater than or equal to 25? If not, is it greater than or equal to 11?

If you want to use the high ends of the ranges (10, 24, and 50), either of these techniques works:

- You can ask: Is `itemQuantity` greater than 50? If not, is it greater than 24? If not, is it greater than 10?
- You can ask: Is `itemQuantity` less than or equal to 10? If not, is it less than or equal to 24? If not, is it less than or equal to 50?

Figure 4-21 shows the flowchart and pseudocode that represent the logic for a program that determines the correct discount for each order quantity using the high ends of the ranges. First, `itemsOrdered` is compared to the high end of the lowest-range group `(RANGE1)`. If `itemsOrdered` is less than or equal to that value (10), then you know the correct discount, `DISCOUNT1`; if not, you continue checking. If `itemsOrdered` is less than or equal to the high end of the next range `(RANGE2)`, then the customer's discount is `DISCOUNT2`; if not, you continue checking, and the customer's discount eventually is set to `DISCOUNT3` or `DISCOUNT4`. In the pseudocode in Figure 4-21, notice how each associated `if`, `else`, and `endif` aligns vertically.



In computer memory, a percent sign (%) is not stored with a numeric value that represents a percentage. Instead, the mathematical equivalent is stored. For example, 15% is stored as 0.15. You can store a percent value as a string, as in "15%", but then you cannot perform arithmetic with it.

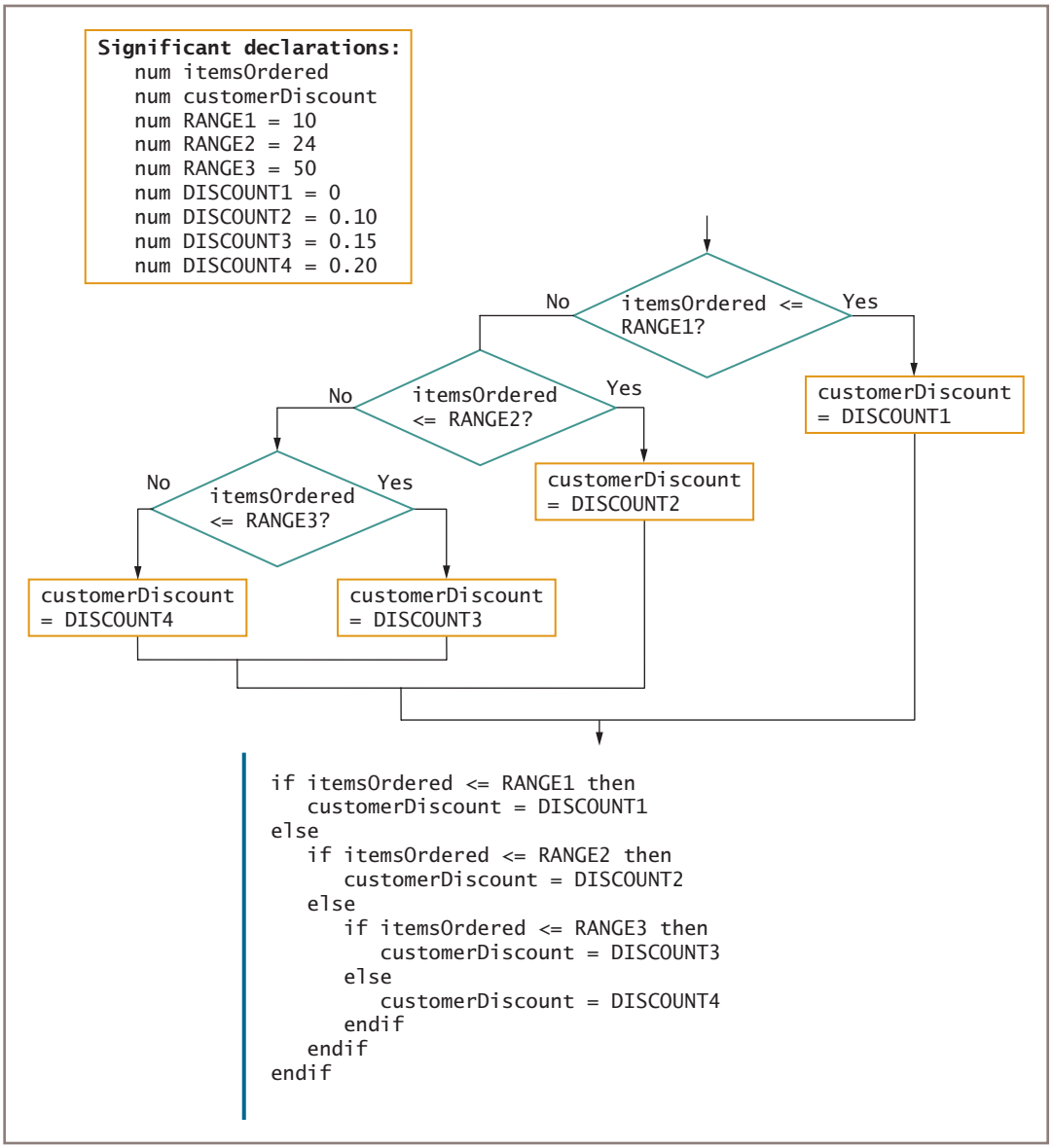


Figure 4-21 Flowchart and pseudocode of logic that selects correct discount based on items ordered

For example, consider an order for 30 items. The expression `itemsOrdered <= RANGE1` evaluates as false, so the else clause of the decision executes. There, `itemsOrdered <= RANGE2` also evaluates to false, so its `else` clause executes. The expression `itemsOrdered <= RANGE3` is true, so customerDiscount becomes `DISCOUNT3`, which is 0.15. Walk through the logic with other values for itemsOrdered and verify for yourself that the correct discount is applied each time.

Avoiding Common Errors When Using Range Checks

To create well-written programs that include range checks, you should be careful to eliminate dead paths and to avoid testing the same range limit multiple times.

Eliminate Dead Paths

When new programmers perform range checks, they are prone to including logic that has too many decisions, entailing more work than is necessary.

Figure 4-22 shows a program segment that contains a range check in which the programmer has asked one question too many—the shaded question in the figure. If you know that `itemsOrdered` is not less than or equal to `RANGE1`, not less than or equal to `RANGE2`, and not less than or equal to `RANGE3`, then `itemsOrdered` must be greater than `RANGE3`. The comparison to `RANGE3` is trivial, so asking whether `itemsOrdered` is greater than `RANGE3` is a waste of time; no customer order can ever travel the logical path on the far left of the flowchart. You might say such a path is a **dead** or **unreachable path**, and that the statements written there constitute dead or unreachable code. Although a program that contains such logic will execute and assign the correct discount to customers who order more than 50 items, providing such a path is inefficient.

In Figure 4-22, it is easier to see the useless path in the flowchart than in the pseudocode representation of the same logic. When you use an `if` without an `else`, however, you are doing nothing when the question's answer is false.



With people, sometimes there are good reasons to ask questions for which you already know the answers. For example, a good trial lawyer seldom asks a question in court if the answer will be a surprise. With computer logic, however, such questions are an inefficient waste of time.

Avoid Testing the Same Range Limit Multiple Times

Another error that programmers make when writing the logic to perform a range check also involves asking unnecessary questions. Figure 4-23 shows an inefficient range selection that asks two unneeded questions. In the figure, if `itemsOrdered` is less than or equal to `RANGE1`, `customerDiscount` is set to `DISCOUNT1`. If `itemsOrdered` is not less than or equal to `RANGE1`, then it must be greater than `RANGE1`, so the next decision (shaded in the figure) is unnecessary. The computer logic will never execute the shaded decision unless `itemsOrdered` is already greater than `RANGE1`—that is, unless the logic follows the false branch of the first selection. If you use the logic in Figure 4-23, you are wasting computer time with a trivial decision that tests a range limit that has already been tested. The same logic applies to the second shaded decision in Figure 4-23. Beginning programmers sometimes justify their use of unnecessary questions as "just making really sure." Such caution is unnecessary when writing computer logic.

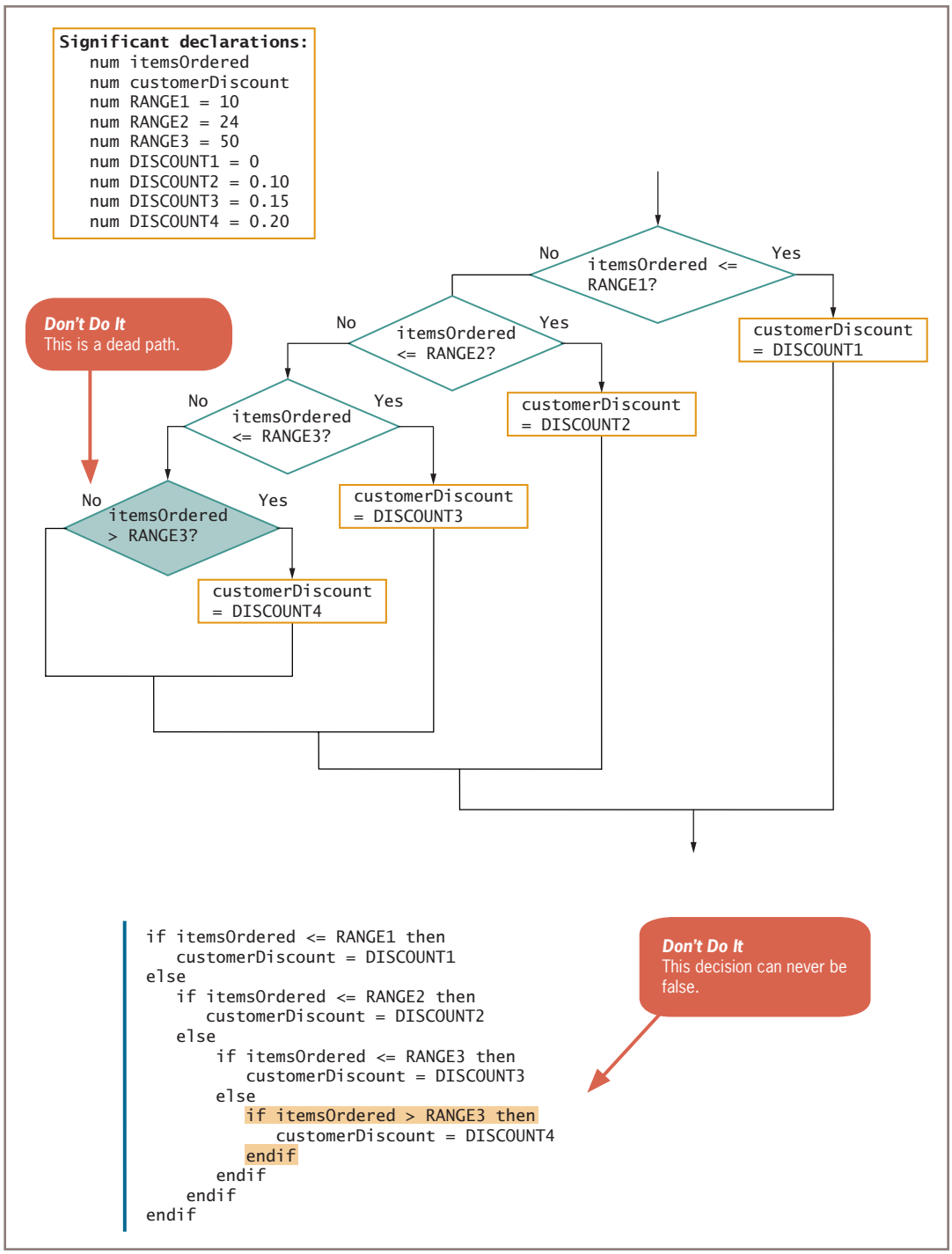


Figure 4-22 Inefficient range selection including unreachable path

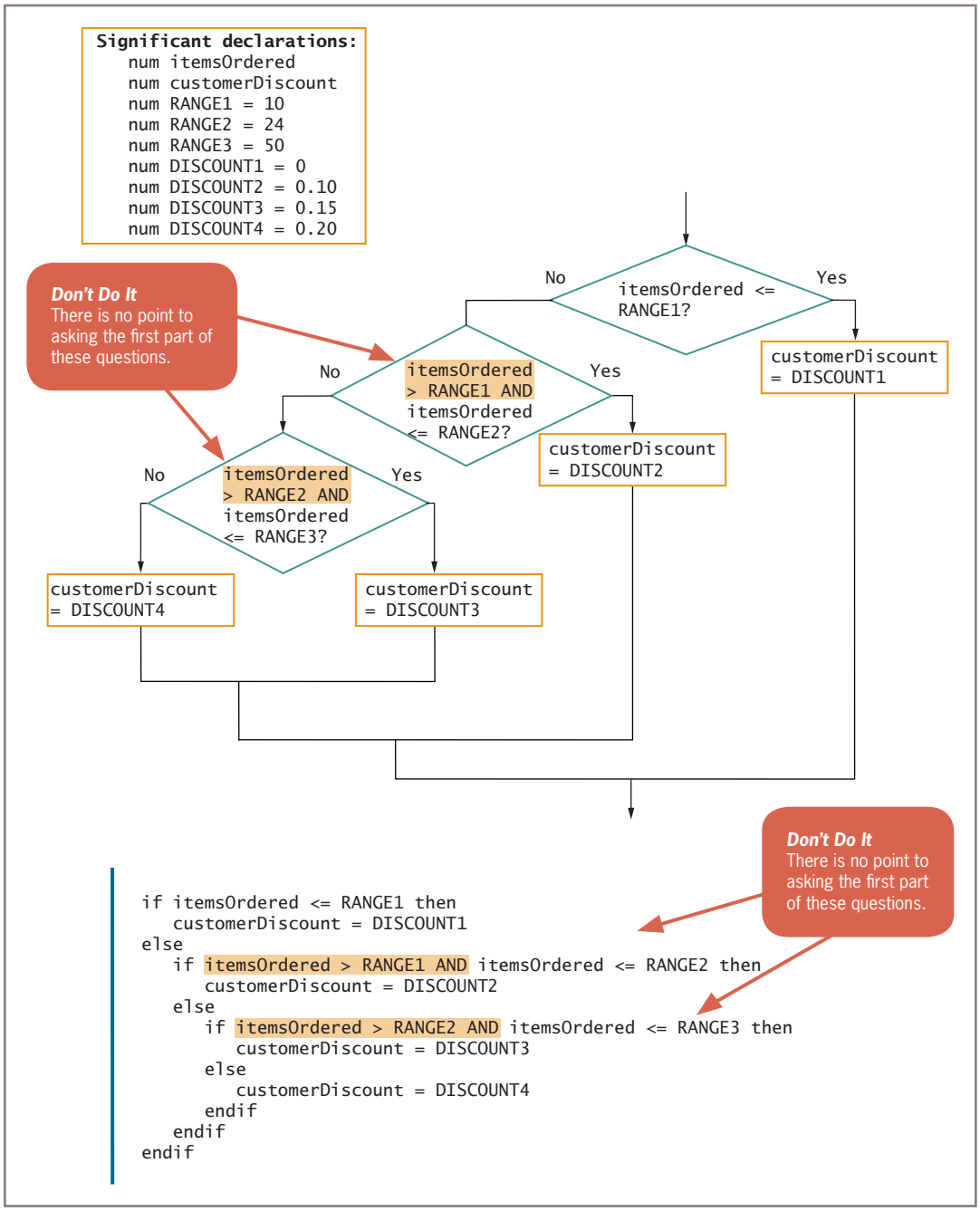


Figure 4-23 Inefficient range selection including unnecessary questions

TWO TRUTHS & A LIE

Making Selections within Ranges

1. When you perform a range check, you compare a variable to every value in a series of ranges.
2. You can perform a range check by making comparisons using the lowest value in each range of values you are using.
3. You can perform a range check by making comparisons using the highest value in each range of values you are using.

The false statement is #1. When you use a range check, you compare a variable to a series of values that represent the ends of ranges. Depending on your logic, you can use either the high or low end of each range.

Understanding Precedence When Combining AND and OR Operators

Most programming languages allow you to combine as many AND and OR operators in an expression as you need. For example, assume that you need to achieve a score of at least 75 on each of three tests to pass a course. You can declare a constant MIN_SCORE equal to 75 and test the multiple conditions with a statement like the following:

```
if score1 >= MIN_SCORE AND score2 >= MIN_SCORE AND score3 >= MIN_SCORE then
   classGrade = "Pass"
else
   classGrade = "Fail"
endif
```

On the other hand, if you need to pass only one of three tests to pass a course, then the logic is as follows:

```
if score1 >= MIN_SCORE OR score2 >= MIN_SCORE OR score3 >= MIN_SCORE then
   classGrade = "Pass"
else
   classGrade = "Fail"
endif
```

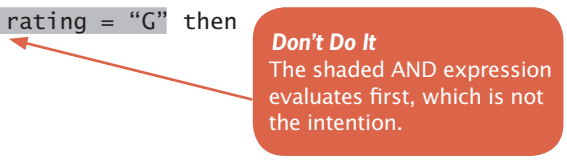
The logic becomes more complicated when you combine AND and OR operators within the same statement. When you do, the AND operators take precedence, meaning the Boolean values of the AND expressions are evaluated first.



In Chapter 2 you learned that in arithmetic statements, multiplication and division have precedence over addition and subtraction. You also learned that precedence is sometimes referred to as *order of operations*.

For example, consider a program that determines whether a movie theater patron can purchase a discounted ticket. Assume that discounts are allowed for children and senior citizens who attend G-rated movies. The following code looks reasonable, but it produces incorrect results because the expression that contains the AND operator (see shading) evaluates before the one that contains the OR operator.

```
if age <= 12 OR age >= 65 AND rating = “G” then
   output “Discount applies”
endif
```



For example, assume that a movie patron is 10 years old and the movie rating is R. The patron should not receive a discount (or be allowed to see the movie!). However, within the `if` statement, the part of the expression that contains the AND operator, `age >= 65 AND rating = "G"`, is evaluated first. For a 10-year-old and an R-rated movie, the question is false (on both counts), so the entire `if` statement becomes the equivalent of the following:

```
if age <= 12 OR aFalseExpression then
   output "Discount applies"
endif
```

Because the patron is 10, `age <= 12` is true, so the original `if` statement becomes the equivalent of:

```
if aTrueExpression OR aFalseExpression then
   output "Discount applies"
endif
```

The combination true OR false evaluates as true. Therefore, the string `"Discount applies"` is output when it should not be.

Many programming languages allow you to use parentheses to correct the logic and force the OR expression to be evaluated first, as shown in the following pseudocode:

```
if (age <= 12 OR age >= 65) AND rating = "G" then
   output "Discount applies"
endif
```

With the added parentheses, if the patron's `age` is 12 or under OR the `age` is 65 or over, the expression is evaluated as:

```
if aTrueExpression AND rating = "G" then
   output "Discount applies"
endif
```

In this statement, when the age value qualifies a patron for a discount, then the rating value also must be acceptable before the discount applies. This was the original intention.

You can use the following techniques to avoid confusion when mixing AND and OR operators:

- You can use parentheses to override the default precedence (order of operations).
- You can use parentheses for clarity even though they do not change what the order of operations would be without them. For example, if a customer should be between

12 and 19 or have a school ID to receive a high school discount, you can use the expression `(age > 12 AND age < 19) OR validId = "Yes"`, even though the evaluation would be the same without the parentheses.

- You can use nested `if` statements instead of using `AND` and `OR` operators. With the flowchart and pseudocode shown in Figure 4-24, it is clear which movie patrons receive the discount. In the flowchart, you can see that the `OR` is nested entirely within the *Yes* branch of the `rating = "G"` decision. Similarly, in the pseudocode in Figure 4-24, you can see by the alignment that if the rating is not G, the logic proceeds directly to the last `endif` statement, bypassing any checking of age at all.

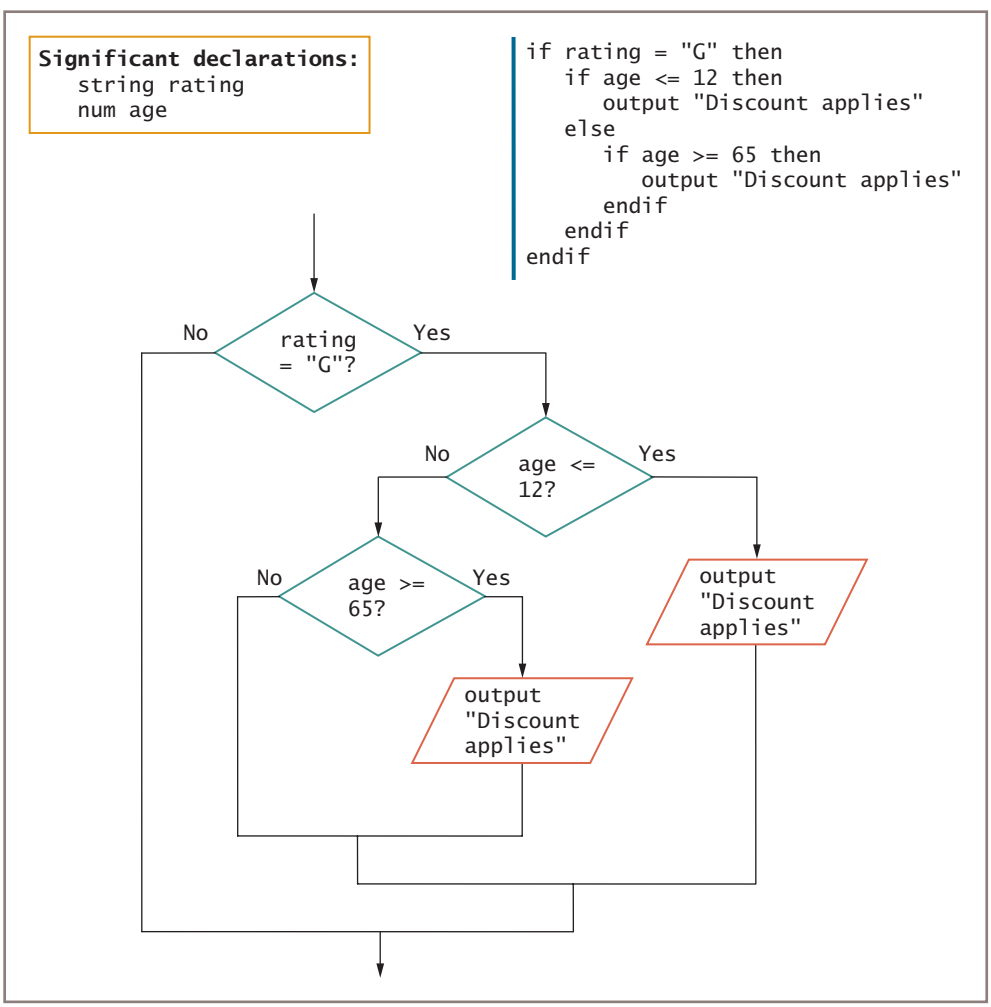


Figure 4-24 Nested decisions that determine movie patron discount

TWO TRUTHS & A LIE

Understanding Precedence When Combining AND and OR Operators

1. Most programming languages allow you to combine as many AND and OR operators in an expression as you need.
2. When you combine AND and OR operators, the OR operators take precedence, meaning their Boolean values are evaluated first.
3. You always can avoid the confusion of mixing AND and OR decisions by nesting `if` statements instead of using AND and OR operators.

The false statement is #2. When you combine AND and OR operators, the AND operators take precedence, meaning the Boolean values of their expressions are evaluated first.

Understanding the case Structure

Most programming languages allow a specialized selection structure called the **case structure** when there are several distinct possible values for a single variable, and each value requires a different subsequent action. Suppose that you work at a school at which tuition varies per credit hour, depending on whether a student is a freshman, sophomore, junior, or senior. The structured flowchart and pseudocode in Figure 4-25 show a series of decisions that assigns different `tuition` values depending on the value of `year`.

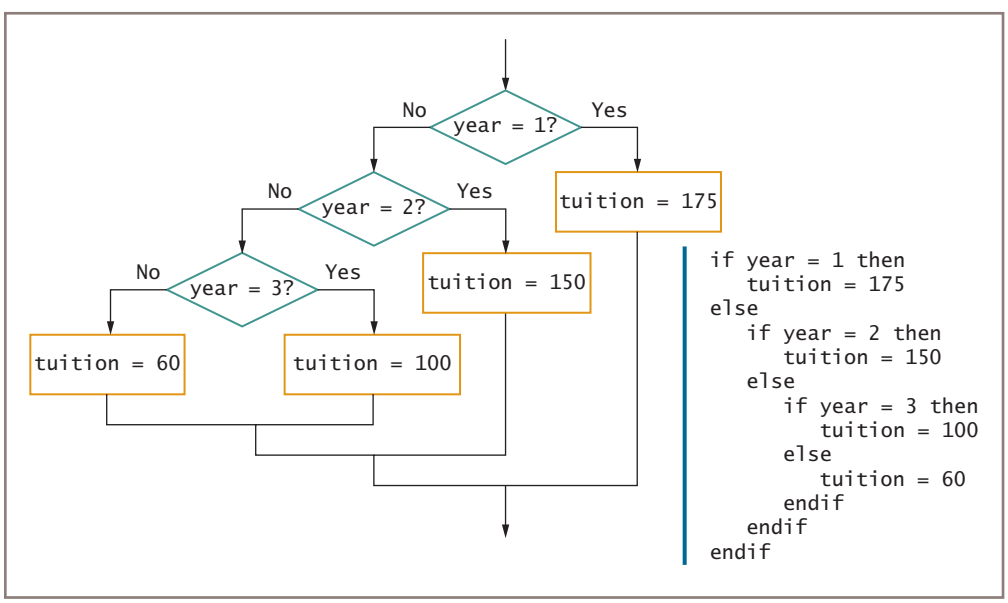


Figure 4-25 Flowchart and pseudocode of tuition decisions

The logic shown in Figure 4-25 is correct and completely structured. The `year = 3?` selection structure is contained within the `year = 2?` structure, which is contained within the `year = 1?` structure. (This example assumes that if `year` is not 1, 2, or 3, the student receives the senior tuition rate.)

Even though the program segments in Figure 4-25 are correct and structured, many programming languages permit using a `case` structure, as shown in Figure 4-26. When using the `case` structure, you test a variable against a series of values, taking appropriate action based on the variable's value. Many people believe programs that contain the `case` structure are easier to read than a program with a long series of decisions, and the `case` structure is allowed because the same results *could* be achieved with a series of structured selections (thus making the program structured). That is, if the first program is structured and the second one reflects the first one point by point, then the second one must be structured as well.

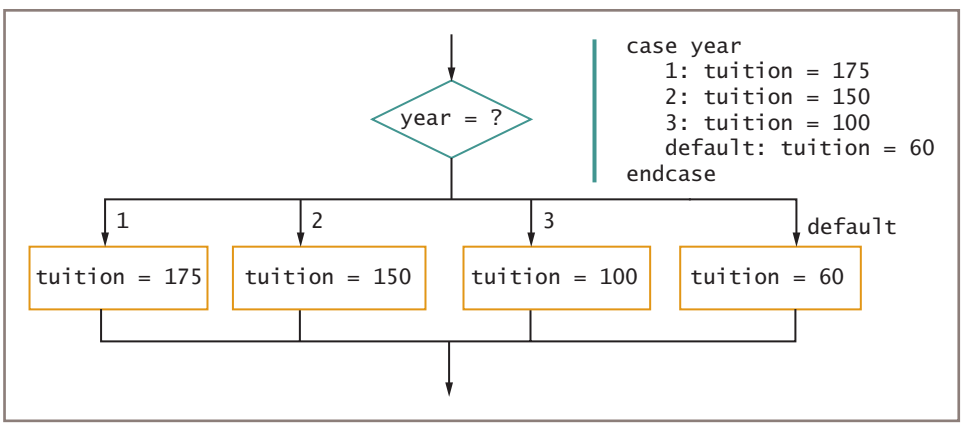


Figure 4-26 Flowchart and pseudocode of `case` structure that determines tuition

The term *default* in Figure 4-26 means *if none of the other cases is true*. Various programming languages use different syntaxes for the default case.

You use the `case` structure only when a series of decisions is based on a single expression. If multiple expressions are tested, then you must use a series of decisions. Besides being easier to read and possibly less prone to error, the `case` structure often executes more quickly in many languages than the series of decisions it represents. The speed of execution depends on a number of technical factors, including how the language compiler was written and how many clauses appear in the `case` statement.

The syntax used in the `case` structure varies widely among languages. Remember that logically, the `case` structure is "extra." All logical problems can be solved using the structures sequence, selection, and loop. When you write your own programs, it is always acceptable to express a complicated decision-making process as a series of individual selections.

TWO TRUTHS & A LIE

Understanding The `case` Structure

1. The `case` structure is used when a series of decisions is based on multiple variables.
2. The `case` structure is used when a series of decisions is based on multiple possible values for a single variable.
3. The syntax of the `case` structure varies among programming languages.

The false statement is #1. The `case` structure is used to test a single variable for multiple values.

Chapter Summary

- Computer program decisions are made by evaluating Boolean expressions. You can use `if-then-else` or `if-then` structures to choose between two possible outcomes.
- You can use relational comparison operators to compare two operands of the same data type. The standard relational comparison operators are =, >, <, >=, <=, and <>.
- In an AND decision, two conditions must be true for a resulting action to take place. An AND decision requires a nested decision or the use of an `AND` operator. In an AND decision, the most efficient approach is to start by evaluating the expression that is less likely to be true.
- In an OR decision, at least one of two conditions must be true for a resulting action to take place. In an OR decision, the most efficient approach is to start by evaluating the expression that is more likely to be true. Most programming languages allow you to ask two or more questions in a single comparison by using a conditional `OR` operator.
- The `NOT` operator reverses the meaning of a Boolean expression.
- To perform a range check, make comparisons with either the lowest or highest value in each range of comparison values. Common errors that occur when programmers perform range checks include asking unnecessary and previously answered questions.
- When you combine `AND` and `OR` operators in an expression, the AND operators take precedence, meaning their Boolean values are evaluated first.
- The `case` structure is a specialized structure that can be used when there are several distinct possible values for a single variable, and each value requires a different subsequent action.

Key Terms

An **`if-then`** decision structure is a single-alternative selection; it contains a tested Boolean expression and an action that is taken only when the expression is true.

An **`if-then` clause** of a decision structure holds the statements that execute when the tested Boolean expression is true.

The **`else` clause** of a decision structure holds the statements that execute only when the tested Boolean expression is false.

Relational comparison operators are the symbols that express Boolean comparisons. Examples include =, >, <, >=, <=, and <>.

A **trivial expression** is one that always evaluates to the same value.

A **compound condition** is an evaluation with multiple parts.

An **AND decision** contains two or more decisions; all conditions must be true for an action to take place.

A **nested decision**, or a **nested `if`**, is a decision within either the `if-then` or `else` clause of another decision.

A **cascading `if` statement** is a series of nested if statements.

A **conditional `AND` operator** (or, more simply, an **AND operator**) is a symbol that you use to combine conditions when they all must be true for an action to occur.

Truth tables are diagrams used in mathematics and logic to help describe the truth of an entire expression based on the truth of its parts.

Short-circuit evaluation is a logical feature in which expressions in each part of a larger expression are evaluated only as far as necessary to determine the final outcome.

An **OR decision** contains two or more decisions; if at least one condition is met, the resulting action takes place.

A **conditional `OR` operator** (or, more simply, an **OR operator**) is a symbol that you use to combine conditions when at least one of them must be true for an action to occur.

The **`NOT` operator** is a symbol that reverses the meaning of a Boolean expression.

A **unary operator** is one that uses only one operand.

A **range check** determines where a variable falls arithmetically when compared to a series of values that mark limiting ends.

A **dead** or **unreachable path** is a logical path that can never be traveled.

A **`case` structure** tests a single variable against multiple values, providing separate actions for each logical path

Exercises



Review Questions

1. A _____expression has one of two values: true or false.

   a. Georgian
   b. Boolean
   c. Barbarian
   d. Selective

2. In a selection, the `else` clause executes _____

   a. only after the `if` clause executes
   b. when the tested condition is true
   c. when the tested condition is false
   d. always

3. The greater-than operator evaluates as true when _____

   a. the right operand is greater than the left operand
   b. the left operand is greater than the right operand
   c. the right operand is equal to the left operand
   d. Both b and c are true.

4. A trivial Boolean expression is one that _____

   a. is uses only two operands
   b. always has the same value
   c. is always true
   d. is always false

5. If `x <= y` is true, then _____

   a. `x = y` is true
   b. `y <= x` is true
   c. `x > y` is false
   d. `x >= y` is false

6. If `j <> k` is true, then _____

   a. `j = k` is true
   b. `j > k` might be true
   c. `j < k` might be true
   d. Both b and c are true.

7. In an AND condition, the most efficient technique is to first ask the question that _____

   a. is most likely to be true
   b. is least likely to be true
   c. that contains the most operands
   d. uses a named constant

8. If `a` is true and `b` is false, then _____

   a. `a AND b` is true
   b. `a AND b` is false
   c. `a OR b` is false
   d. If `a` is true, then `b` must be true.

9. If p is true and q is false, then _____

   a. `p OR q` is true
   b. `p OR q` is false
   c. `p AND q` is true
   d. `p` is greater than `q`

10. Which of the lettered choices is equivalent to the following decision?

```
if x > 10 then
  if y > 10 then
     output "X"
  endif
endif
```

   a. `if x > 10 OR y > 10 then output "X" endif`
   b. `if x > 10 AND x > y then output "X" endif`
   c. `if y > x then output "X" endif`
   d. `if x > 10 AND y > 10 then output "X" endif`

11. If `conditionA` is 30 percent likely to be true and `conditionB` is 10 percent likely to be true, then it is most efficient to test `conditionA` first _____

   a. in an OR decision
   b. in an AND decision
   c. in any decision
   d. never

12. Which of the following is a poorly written, trivial Boolean expression?

   a. `a > b AND b > c`
   b. `a = 100 OR b > 200`
   c. `a < b AND c < 100 AND d <> 5`
   d. `a < 10 AND a > 40`

13. Which of the following must always be false?

   a. `e > 12 AND e < 15`
   b. `e = 10 OR e = 20`
   c. `e > 12 OR f > 12`
   d. `e > 10 AND e < 7`

14. Which of the following must always be true?

   a. `g < 12 AND g < 18`
   b. `g = 12 OR h = 17`
   c. `g <> 12 OR g <> 15`
   d. `g > 12 AND g < 21`

15. In the following pseudocode, what percentage raise will an employee in Department 8 receive?

```
if department < 5 then
   raise = SMALL_RAISE
else
   if department < 14 then
      raise = MEDIUM_RAISE
else
   if department < 9 then
      raise = BIG_RAISE
   endif
  endif
endif
```

a. SMALL_RAISE
b. MEDIUM_RAISE
c. BIG_RAISE
d. impossible to tell

16. In the following pseudocode, what percentage raise will an employee in Department 10 receive?

```
if department < 2 then
   raise = SMALL_RAISE
else
   if department < 6 then
      raise = MEDIUM_RAISE
else
   if department < 10 then
      raise = BIG_RAISE
   endif
  endif
endif
```

a. SMALL_RAISE
b. MEDIUM_RAISE
c. BIG_RAISE
d. impossible to tell

17. When you use a range check, you always compare a variable to _____ value in the range.

a. the lowest
b. the highest
c. an end-of-range value
d. the average

18. If `sales = 100`, `rate = 0.10`, and `expenses = 50`, which of the following expressions is true?

a. `sales >= expenses AND rate < 1`
b. `sales < 200 OR expenses < 100`
c. `expenses = rate OR sales = rate`
d. two of the above

19. If `a` is true, `b` is true, and `c` is false, which of the following expressions is true?

a. `a OR b AND c`
b. `a AND b AND c`
c. `a AND b OR c`
d. two of the above

20. If `d` is true, `e` is false, and `f` is false, which of the following expressions is true?

a. `e OR f AND d`
b. `f AND d OR e`
c. `d OR e AND f`
d. two of the above



Programming Exercises

1. Assume that the following variables contain the values shown:

```
numberBig = 100                    wordBig = "Constitution"
numberMedium = 10                  wordMedium = "Dance"
numberSmall = 1                    wordSmall = "Toy"
```

   For each of the following Boolean expressions, decide whether the statement is true, false, or illegal.

   a. `numberBig > numberSmall`
   b. `numberBig < numberMedium`
   c. `numberMedium = numberSmall`
   d. `numberBig = wordBig`
   e. `numberBig = "Big"`
   f. `wordMedium > wordSmall`
   g. `wordSmall = "TOY"`
   h. `numberBig <= 5 * numberMedium + 50`
   i. `numberBig >= 2000`
   j. `numberBig > numberMedium + numberSmall`
   k. `numberBig > numberMedium AND numberBig < numberSmall`
   l. `numberBig = 100 OR numberBig > numberSmall`
   m. `numberBig < 10 OR numberSmall > 10`
   n. `numberBig = 300 AND numberMedium = 10 OR numberSmall = 1`
   o. `wordSmall > wordBig`
   p. `wordSmall > wordMedium`

2. Design a flowchart or pseudocode for a program that accepts two numbers from a user and displays one of the following messages: *First is larger, Second is larger, Numbers are equal.*

3. Design a flowchart or pseudocode for a program that accepts three numbers from a user and displays a message if the sum of any two numbers equals the third.

4. Cecilia's Boutique wants several lists of salesperson data. Design a flowchart or pseudocode for the following:

   a. A program that accepts one salesperson's ID number, number of items sold in the last month, and total value of the items and displays data message only if the salesperson is a high performer—defined as a person who sells more than 200 items in the month.
   b. A program that accepts the salesperson's data and displays a message only if the salesperson is a high performer—defined a person who sells more than 200 items worth at least $1,000 in the month.

c. A program that accepts salesperson data continuously until a sentinel value is entered and displays a list of the ID numbers of those who sell more than 100 items in the month.

d. A program that accepts salesperson data continuously until a sentinel value is entered and displays a list of salespeople who sold between 50 and 100 items in the month.

5. ShoppingBay is an online auction service that requires several reports. Data for each auctioned item includes an ID number, item description, length of auction in days, and minimum required bid. Design a flowchart or pseudocode for the following:

   a. A program that accepts data for one auctioned item. Display data for an auction only if the minimum required bid is more than $250.00.

   b. A program that continuously accepts auction item data until a sentinel value is entered and displays all data for auctions in which the minimum required bid is more than $300.00.

   c. A program that continuously accepts auction item data and displays data for every auction in which there are no bids yet (in other words, the minimum bid is $0.00) and the length of the auction is seven days or less.

   d. A program that continuously accepts auction data and displays data for every auction in which the length is between 14 and 28 days inclusive.

   e. A program that prompts the user for a maximum required bid, and then continuously accepts auction data and displays data for every auction in which the minimum bid is less than or equal to the amount entered by the user.

6. The Dash Cell Phone Company charges customers a basic rate of $5 per month to send text messages. Additional rates are as follows:

   - The first 100 messages per month, regardless of message length, are included in the basic bill.
   - An additional three cents are charged for each text message after the 100th message, up to and including 300 messages.
   - An additional two cents are charged for each text message after the 300th message.
   - Federal, state, and local taxes add a total of 14 percent to each bill.

   Design a flowchart or pseudocode for the following:

   a. A program that accepts the following data about one customer's messages: area code (three digits), phone number (seven digits), and number of text messages sent. Display all the data, including the month-end bill both before and after taxes are added.

   b. A program that continuously accepts data about text messages until a sentinel value is entered, and displays all the details.

c. A program that continuously accepts data about text messages until a sentinel value is entered, and displays details only about customers who send more than 100 text messages.

d. A program that continuously accepts data about text messages until a sentinel value is entered, and displays details only about customers whose total bill with taxes is over $10.

e. A program that prompts the user for a three-digit area code from which to select bills. Then the program continuously accepts text message data until a sentinel value is entered, and displays data only for messages sent from the specified area code.

7. The Drive-Rite Insurance Company provides automobile insurance policies for drivers. Design a flowchart or pseudocode for the following:

a. A program that accepts insurance policy data, including a policy number, customer last name, customer first name, age, premium due date (month, day, and year), and number of driver accidents in the last three years. If an entered policy number is not between 1000 and 9999 inclusive, set the policy number to 0. If the month is not between 1 and 12 inclusive, or the day is not correct for the month (for example, not between 1 and 31 for January or 1 and 29 for February), set the month, day, and year to 0. Display the policy data after any revisions have been made.

b. A program that continuously accepts policy holders' data until a sentinel value has been entered, and displays the data for any policy holder over 40 years old.

c. A program that continuously accepts policy holders' data until a sentinel value has been entered, and displays the data for any policy holder who is at least 18 years old.

d. A program that continuously accepts policy holders' data and displays the data for any policy holder no more than 60 years old.

e. A program that continuously accepts policy holders' data and displays the data for any policy holder whose premium is due no later than April 30 any year.

f. A program that continuously accepts policy holders' data and displays the data for any policy holder whose premium is due up to and including January 1, 2018.

g. A program that continuously accepts policy holders' data and displays the data for any policy holder whose premium is due by June 1, 2018.

h. A program that continuously accepts policy holders' data and displays the data for anyone who has a policy number between 1000 and 4000 inclusive, whose policy comes due in September or October of any year, and who has had three or fewer accidents.

8. The Barking Lot is a dog day care center. Design a flowchart or pseudocode for the following:

   a. A program that accepts data for an ID number of a dog's owner, and the name, breed, age, and weight of the dog. Display a bill containing all the input data as well as the weekly day care fee, which is $55 for dogs weighing less than 15 pounds, $75 for dogs from 15 to 30 pounds inclusive, $105 for dogs from 31 to 80 pounds inclusive, and $125 for dogs weighing more than 80 pounds.
   b. A program that continuously accepts dogs' data until a sentinel value is entered, and displays billing data for each dog.
   c. A program that continuously accepts dogs' data until a sentinel value is entered, and displays billing data for dog owners who owe more than $100.
   d. A program that continuously accepts dogs' data until a sentinel value is entered, and displays billing data for dogs that weigh less than 20 pounds or more than 100 pounds.

9. Mark Daniels is a carpenter who creates personalized house signs. He wants an application to compute the price of any sign a customer orders, based on the following factors:

   - The minimum charge for all signs is $30.
   - If the sign is made of oak, add $15. No charge is added for pine.
   - The first six letters or numbers are included in the minimum charge; there is a $3 charge for each additional character.
   - Black or white characters are included in the minimum charge; there is an additional $12 charge for gold-leaf lettering.

   Design a flowchart or pseudocode for the following:

   a. A program that accepts data for an order number, customer name, wood type, number of characters, and color of characters. Display all the entered data and the final price for the sign.
   b. A program that continuously accepts sign order data and displays all the relevant information for oak signs with five white letters.
   c. A program that continuously accepts sign order data and displays all the relevant information for pine signs with gold-leaf lettering and more than 10 characters.

10. Black Dot Printing is attempting to organize carpools to save energy. Each input record contains an employee's name and town of residence. Ten percent of the company's employees live in Wonder Lake; 30 percent live in the adjacent town of Woodstock. Black Dot wants to encourage employees who live in either town to drive to work together. Design a flowchart or pseudocode for the following:
    a. A program that accepts an employee's data and displays it with a message that indicates whether the employee is a candidate for the carpool (because he lives in one of the two cities).
    b. A program that continuously accepts employee data until a sentinel value is entered, and displays a list of all employees who are carpool candidates. Make sure the decision-making process is as efficient as possible.
    c. A program that continuously accepts employee data until a sentinel value is entered, and displays a list of all employees who are ineligible to carpool because they do not live in either Wonder Lake or Woodstock. Make sure the decision-making process is as efficient as possible.



Performing Maintenance

1. A file named MAINTENANCE04-01.jpg is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 4 include DEBUG04-01.txt, DEBUG04-02.txt, and DEBUG04-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 4 include a file named DEBUG04-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.



Game Zone

1. In Chapter 2, you learned that many programming languages allow you to generate a random number between 1 and a limiting value named `limit` by using a statement similar to `randomNumber = random(limit)`. Create the logic for a guessing game in which the application generates a random number and the player tries to guess it. Display a message indicating whether the player's guess was correct, too high, or too low. (After you finish Chapter 5, you will be able to modify the application so that the user can continue to guess until the correct answer is entered.)

2. Create a lottery game application. Generate three random numbers, each between 0 and 9. Allow the user to guess three numbers. Compare each of the user's guesses to the three random numbers and display a message that includes the user's guess, the randomly determined three digits, and the amount of money the user has won, as shown in Figure 4-27.

Matching Numbers	Award ($)
Any one matching	10
Two matching	100
Three matching, not in order	1000
Three matching in exact order	1,000,000
No matches	0

Figure 4-27 Awards for matching numbers in lottery game

Make certain that your application accommodates repeating digits. For example, if a user guesses 1, 2, and 3, and the randomly generated digits are 1, 1, and 1, do not give the user credit for three correct guesses—just one.

CHAPTER 5

Looping

Upon completion of this chapter, you will be able to:

- Appreciate the advantages of looping
- Use a loop control variable
- Create nested loops
- Avoid common loop mistakes
- Use a `for` loop
- Use a posttest loop
- Recognize the characteristics shared by all structured loops
- Describe common loop applications
- Appreciate the similarities and differences between selections and loops

Appreciating the Advantages of Looping

Although making decisions is what makes computers seem intelligent, looping makes computer programming both efficient and worthwhile. When you use a loop, one set of instructions can operate on multiple, separate sets of data. Using fewer instructions results in less time required for design and coding, fewer errors, and shorter compile time.

Recall the loop structure that you learned about in Chapter 3; it looks like Figure 5-1. As long as a Boolean expression remains true, the body of a `while` loop executes. Quick Reference 5-1 shows the pseudocode standards this book uses for the `while` statement.

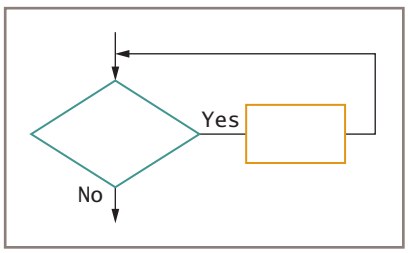


Figure 5-1 The loop structure

QUICK REFERENCE 5-1 `while` Statement Pseudocode Standards

The `while` keyword starts the statement and precedes any statements that execute when the tested condition is true.

The tested condition is a Boolean expression. It might be a comparison such as `x > y`, it might be a Boolean variable if the language supports that type, or it might be a call to a method that returns a Boolean value. (Chapter 9 in the comprehensive version of this book describes methods that return values.)

Although many modern languages do not require indentation, the bodies of `while` loops in this book are indented.

```
while  condition
   statements that execute when condition is true
endwhile
```

The `endwhile` keyword ends the structure. After `endwhile`, the condition is tested again.

You already have learned that many programs use a loop to control repetitive tasks. For example, Figure 5-2 shows the basic structure of many business programs. After some housekeeping tasks are completed, the detail loop repeats once for every data record that must be processed.

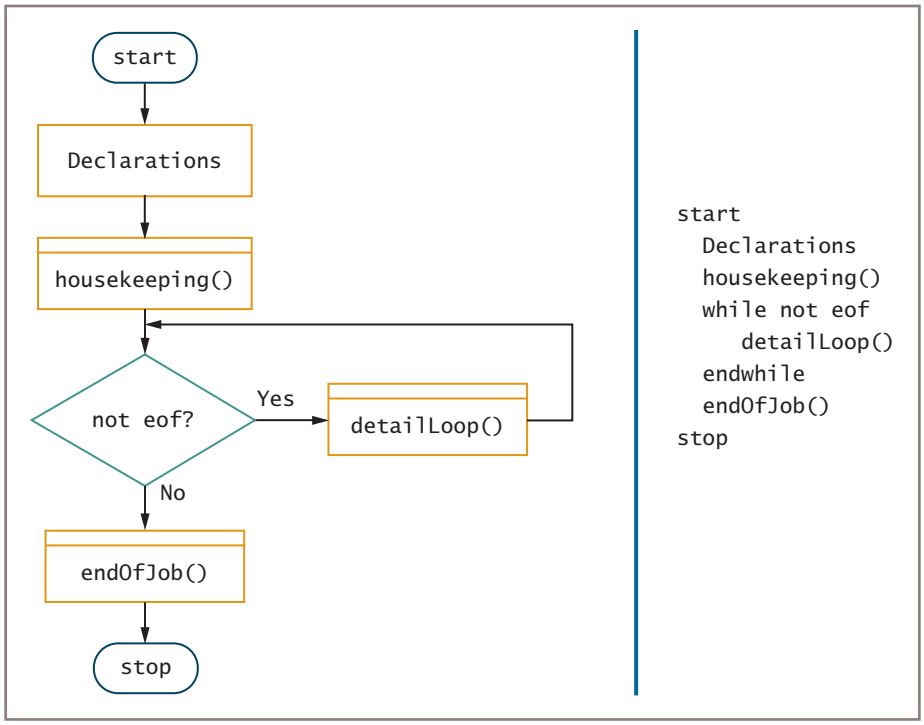


Figure 5-2 The mainline logic common to many business programs

For example, Figure 5-2 might represent the mainline logic of a typical payroll program. The first employee's data would be entered in the `housekeeping()` module, and while the `eof` condition is not met, the `detailLoop()` module would perform such tasks as determining regular and overtime pay and deducting taxes, insurance premiums, charitable contributions, union dues, and other items. Then, after the employee's paycheck is output, the next employee's data would be entered, and the `detailLoop()` module would repeat for the next employee. The advantage to having a computer produce payroll checks is that the calculation instructions need to be written only once and can be repeated indefinitely. At some point, when the program attempts to retrieve data for a new employee, the `eof` condition would be met, and the loop that contains the `detailLoop()` would be exited. Then the `endOfJob()` module would execute; it might contain tasks such as making sure data files are closed or displaying payroll totals.



Watch the video *A Quick Introduction to Loops*.

TWO TRUTHS & A LIE

Appreciating the Advantages of Looping

1. When you use a loop, you can write one set of instructions that operates on multiple, separate sets of data.
2. A major advantage of having a computer perform complicated tasks is the ability to repeat them.
3. A loop is a structure that branches in two logical paths before continuing.

The false statement is #3. A loop is a structure that repeats actions while some condition continues.

Using a Loop Control Variable

You can use a `while` loop to execute a body of statements continuously as long as some condition continues to be true. The body of a loop might contain any number of statements, including input and output statements, arithmetic calculations, module calls, selection structures, and other loops. To make a `while` loop end correctly, you can declare a **loop control variable** to manage the number of repetitions a loop performs. Three separate actions should occur using a loop control variable:

- The loop control variable is initialized before entering the loop.
- The loop control variable's value is tested, and if the result is true, the loop body is entered.
- The loop control variable is altered within the body of the loop so that the tested condition that starts the loop eventually is false.

If you omit any of these actions—initialize, test, alter—or perform them incorrectly, you run the risk of creating an infinite loop. Once your logic enters the body of a structured loop, the entire loop body must execute. Program logic can leave a structured loop only at the evaluation of the loop control variable. Commonly, you can control a loop's repetitions in one of two ways:

- Use a counter to create a definite, counter-controlled loop.
- Use a sentinel value to create an indefinite loop.

Using a Definite Loop with a Counter

Figure 5-3 shows a loop that displays *Hello* four times. The variable `count` is the loop control variable. This loop is a **definite loop** because it executes a definite, predetermined number of times—in this case, four. The loop is a **counted loop**, or **counter-controlled loop**, because the program keeps track of the number of loop repetitions by counting them.

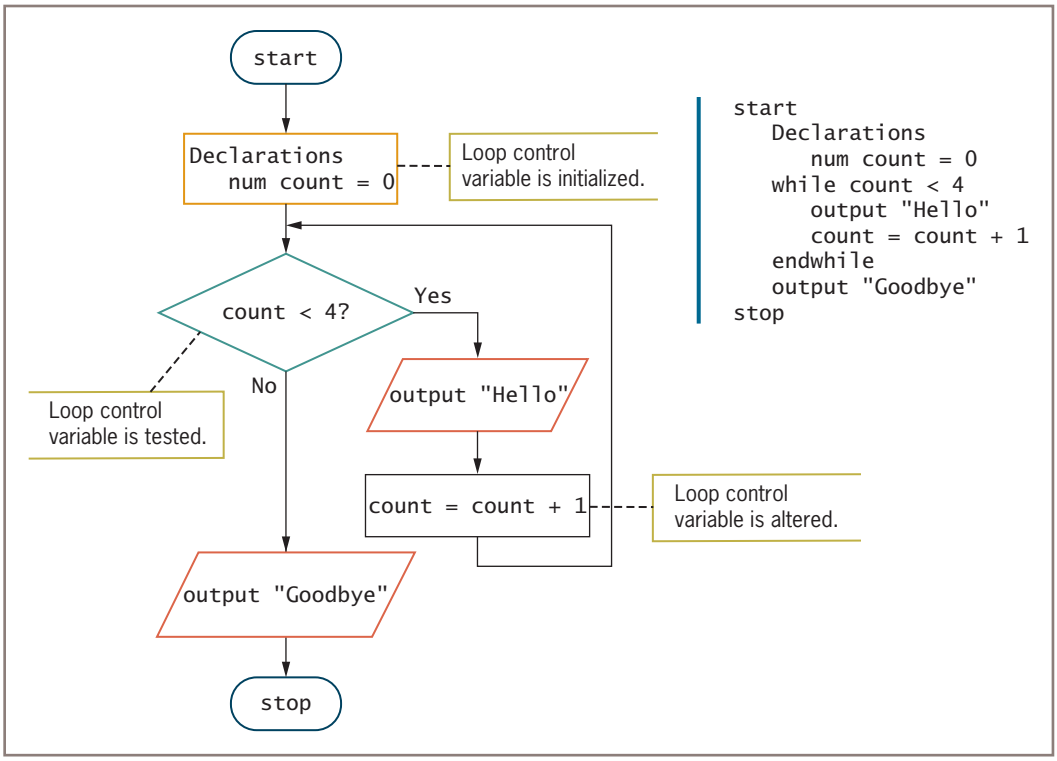


Figure 5-3 A counted `while` loop that outputs *Hello* four times

The loop in Figure 5-3 executes as follows:

- The loop control variable, `count`, is initialized to 0.
- The `while` expression compares `count` to 4.
- The value of `count` is less than 4, and so the loop body executes. The loop body shown in Figure 5-3 consists of two statements that, in sequence, display *Hello* and add 1 to `count`.
- The next time the condition `count < 4` is evaluated, the value of `count` is 1, which is still less than 4, so the loop body executes again. *Hello* is displayed a second time and `count` is incremented to 2.
- *Hello* is displayed a third time and `count` becomes 3, then *Hello* is displayed a fourth time and `count` becomes 4.
- Now when the expression `count < 4?` evaluates, it is `false`, so the loop ends.

Within a loop's body, you can change the value of the loop control variable in a number of ways. For example:

- You might simply assign a new value to the loop control variable.

- You might retrieve a new value from an input device.
- You might **increment**, or increase, the loop control variable, as in the logic in Figure 5-3.
- You might reduce, or **decrement**, the loop control variable. For example, the loop in Figure 5-3 could be rewritten so that `count` is initialized to 4, and reduced by 1 on each pass through the loop. The loop would then continue while `count` remains greater than 0.

The terms *increment* and *decrement* usually refer to small changes; often the value used to increase or decrease the loop control variable is 1. However, loops also are controlled by adding or subtracting values other than 1. For example, to display company profits at five-year intervals for the next 50 years, you would want to add 5 to a loop control variable during each iteration.



Because you frequently need to increment a variable, many programming languages contain a shortcut operator for incrementing. For example, in C++, C#, and Java, the expression `++value` is a shortcut for the expression `value = value + 1`. You will learn about these shortcut operators when you study a programming language that uses them.



Watch the video *Looping*.

The looping logic shown in Figure 5-3 uses a counter. A **counter** is any numeric variable that counts the number of times an event has occurred. In everyday life, people usually count things starting with 1. Many programmers prefer starting their counted loops with a variable containing 0 for two reasons:

- In many computer applications, numbering starts with 0 because of the 0-and-1 nature of computer circuitry.
- When you learn about arrays in Chapter 6, you will discover that array manipulation naturally lends itself to 0-based loops.
- If you initialize a loop control variable to 0 and add one after each loop body execution, the variable always contains the number of times the loop has executed, and you might want to use this number for future comparisons or to display.

Using an Indefinite Loop with a Sentinel Value

Often, the value of a loop control variable is not altered by arithmetic, but instead is altered by user input. For example, perhaps you want to keep performing some task while the user indicates a desire to continue. In that case, you do not know when you write the program whether the loop will be executed two times, 200 times, or at all. This type of loop is an **indefinite loop**.

Consider an interactive program that displays *Hello* repeatedly as long as the user wants to continue. The loop is indefinite because each time the program executes, the loop might be performed a different number of times. The program appears in Figure 5-4.

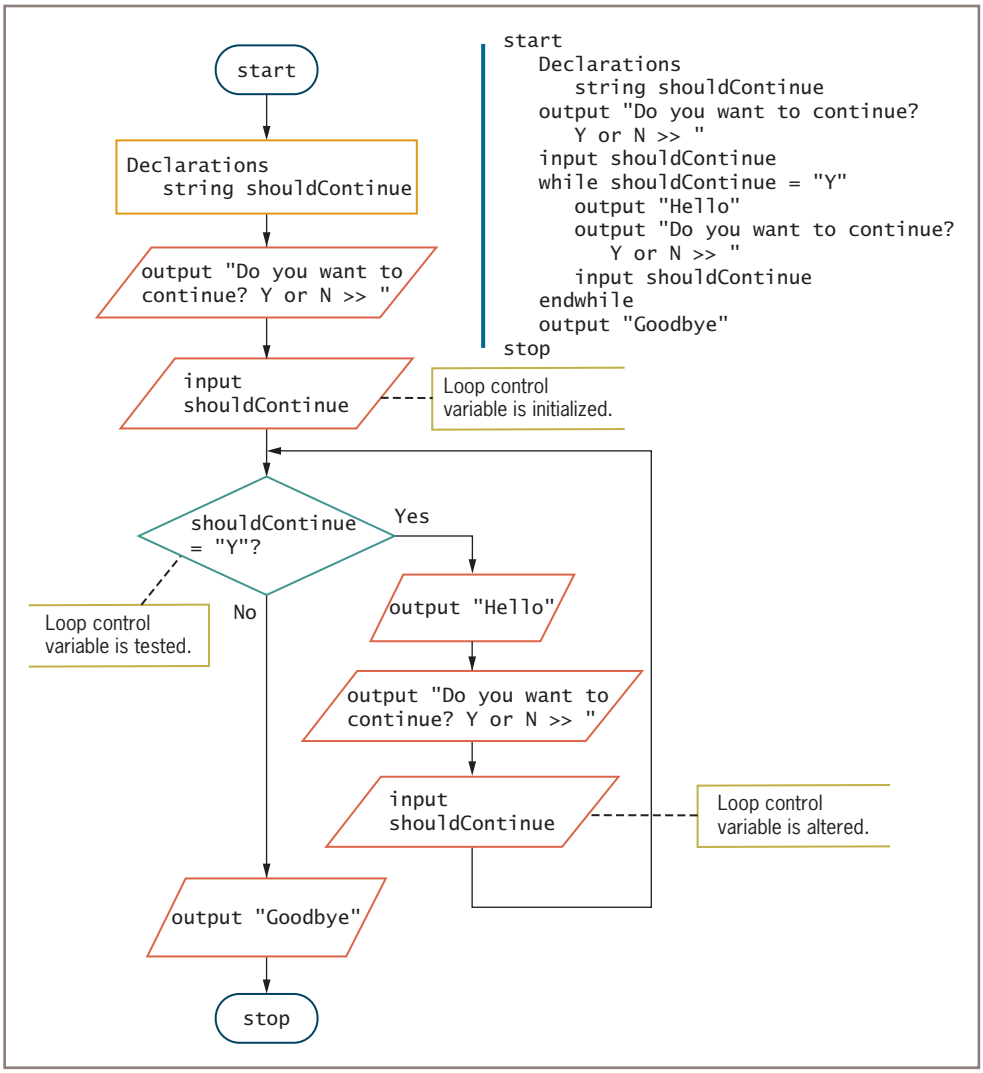


Figure 5-4 An indefinite `while` loop that displays *Hello* as long as the user wants to continue

In the program in Figure 5-4, the loop control variable is `shouldContinue`. The program executes as follows:

- The first `input shouldContinue` statement in the application in Figure 5-4 is a priming input statement. In this statement, the loop control variable is initialized by the user's first response.
- The `while` expression compares the loop control variable to the sentinel value *Y*.

- If the user has entered *Y*, then *Hello* is output and the user is asked whether the program should continue. In this step, the value of `shouldContinue` might change.
- At any point, if the user enters any value other than *Y*, the loop ends. In most programming languages, simple comparisons are case sensitive, so any entry other than *Y*, including *y*, will end the loop.

Figure 5-5 shows how the program might look when it is executed at the command line and in a GUI environment. The screens in Figure 5-5 show programs that perform exactly the same tasks using different environments. In each environment, the user can continue choosing to see *Hello* messages, or can choose to quit the program and display *Goodbye*.

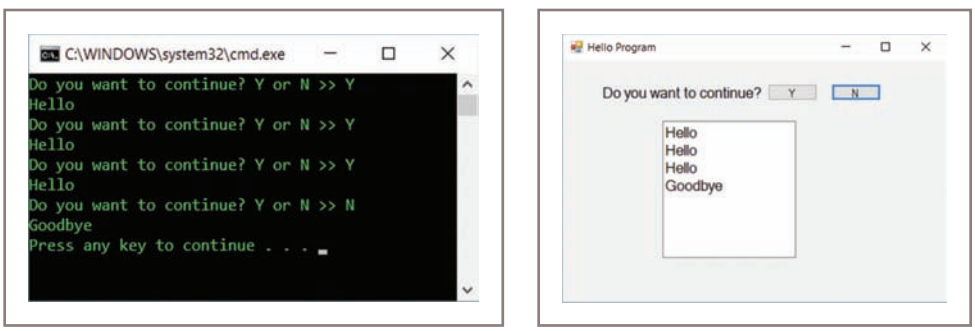


Figure 5-5 Typical executions of the program in Figure 5-4 in two environments

Understanding the Loop in a Program's Mainline Logic

The flowchart and pseudocode segments in Figure 5-4 contain the three steps that should occur in every properly functioning loop:

1. You must provide a starting value for the variable that will control the loop.
2. You must test the loop control variable to determine whether the loop body executes.
3. Within the loop, you must provide a way to alter the loop control variable.

In Chapter 2, you learned that the mainline logic of many business programs follows a standard outline that consists of housekeeping tasks, a loop that repeats, and finishing tasks. The three crucial steps that occur in any loop also occur in standard mainline logic. Figure 5-6 shows the flowchart for the mainline logic of the payroll program that you saw in Figure 2-8. Figure 5-6 points out the three loop-controlling steps. In this case, the three steps—initializing, testing, and altering the loop control variable—are in different modules.

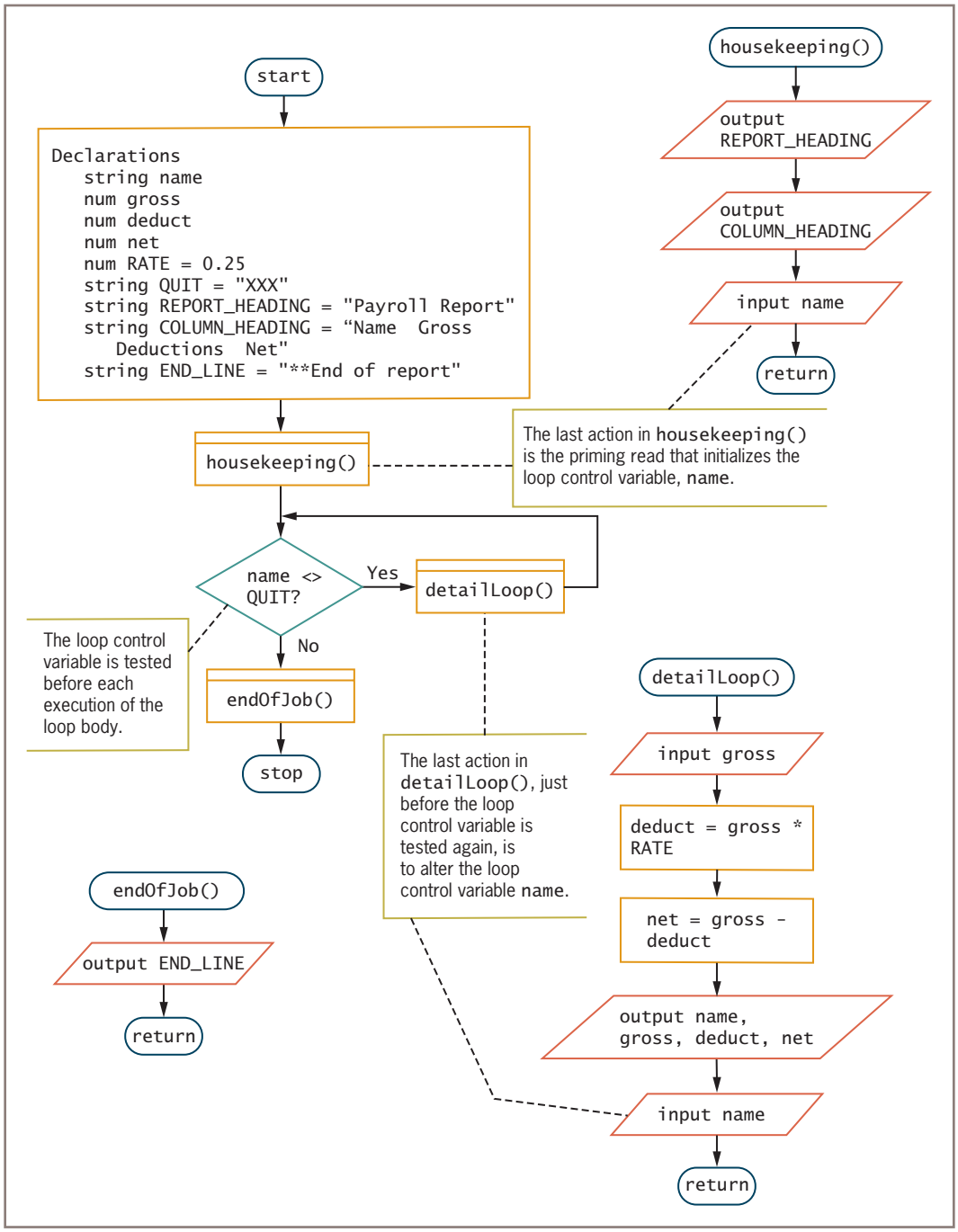


Figure 5-6 A payroll program showing how the loop control variable is used

The steps all occur in the correct places, however, showing that the mainline logic uses a standard and correct loop.

TWO TRUTHS & A LIE

Using a Loop Control Variable

1. To make a `while` loop execute correctly, a loop control variable must be set to 0 before entering the loop.
2. To make a `while` loop execute correctly, a loop control variable should be tested before entering the loop body.
3. To make a `while` loop execute correctly, the body of the loop must take some action that alters the value of the loop control variable.

The false statement is #1. A loop control variable must be initialized, but not necessarily to 0.

Nested Loops

Program logic gets more complicated when you must use loops within loops, or **nested loops**. When loops are nested, the loop that contains the other loop is called the **outer loop**, and the loop that is contained is called the **inner loop**. You need to create nested loops when the values of two or more variables repeat to produce every combination of values. Usually, when you create nested loops, each loop has its own loop control variable.

For example, suppose you want to write a program that produces quiz answer sheets like the ones shown in Figure 5-7. Each answer sheet has a unique heading followed by five parts with three questions in each part, and you want a fill-in-the-blank line for each question. You could write a program that uses 63 separate output statements to produce three sheets (each sheet contains 21 printed lines), but it is more efficient to use nested loops.

Figure 5-8 shows the logic for the program that produces answer sheets. Three loop control variables are declared for the program:

- `quizName` controls the `detailLoop()` module that is called from the mainline logic.
- `partCounter` controls the outer loop within the `detailLoop()` module; it keeps track of the answer sheet parts.
- `questionCounter` controls the inner loop in the `detailLoop()` module; it keeps track of the questions and answer lines within each part section on each answer sheet.

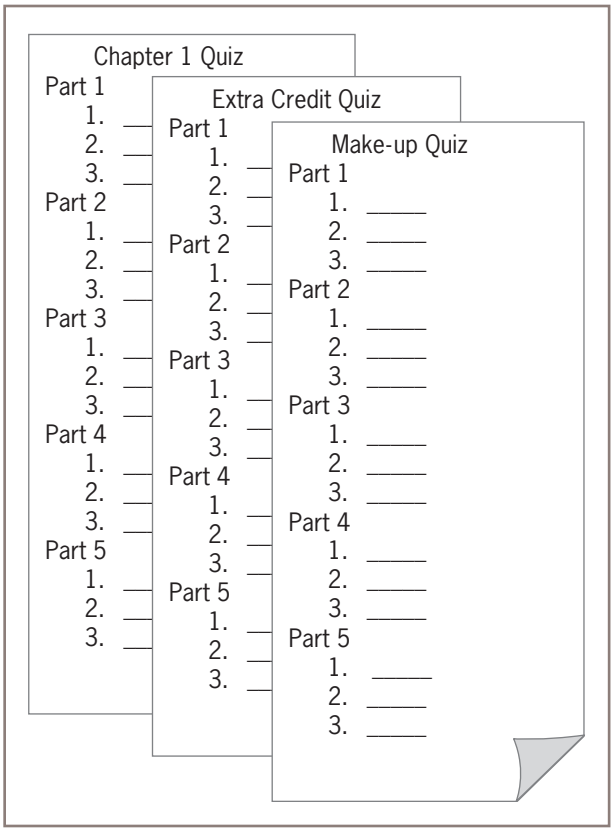


Figure 5-7 Quiz answer sheets

Five named constants are also declared. Three of these constants (`QUIT`, `PARTS`, and `QUESTIONS`) hold the sentinel values for each of the three loops in the program. The other two constants hold the text that will be output (the word *Part* that precedes each part number, and the period-space-underscore combination that forms a fill-in line for each question).



In Figure 5-8, some output (the user prompt) would be sent to one output device, such as a monitor. Other output (the quiz sheet) would be sent to another output device, such as a printer. The statements needed to send output to separate devices differ among languages. The statements to set up the printer would be included in the `housekeeping()` module, and the statements to disengage the printer would be included in the currently empty `endOfJob()` module. Chapter 7 provides more details about sending output to separate files.

When the program in Figure 5-8 starts, the `housekeeping`() module executes and the user enters the name to be output at the top of the first quiz. If the user enters the `QUIT` value, the program ends immediately, but if the user enters anything else, such as *Make-up Quiz*, then the `detailLoop`() module executes.

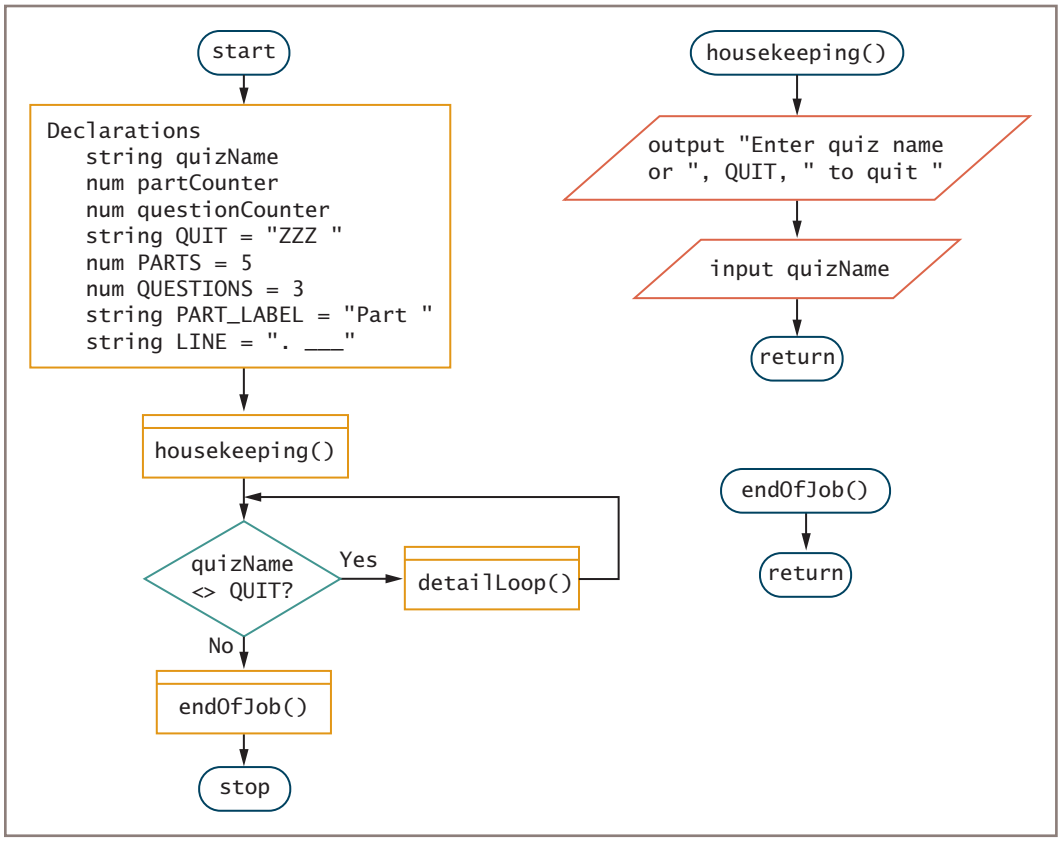


Figure 5-8 Flowchart and pseudocode for `AnswerSheet` program *(continues)*

In the `detailLoop()`, the quiz name is output at the top of the answer sheet. Then `partCounter` is initialized to 1. The `partCounter` variable is the loop control variable for the outer loop in this module. The outer loop continues while `partCounter` is less than or equal to `PARTS`. The last statement in the outer loop adds 1 to `partCounter`. In other words, the outer loop will execute when `partCounter` is 1, 2, 3, 4, and 5.

In the outer loop in the `detailLoop()` module in Figure 5-8, the word *Part* and the current `partCounter` value are output. Then the following steps execute:

- The loop control variable for the inner loop is initialized by setting `questionCounter` to 1.
- The loop control variable `questionCounter` is evaluated. While `questionCounter` does not exceed `QUESTIONS`, the loop body executes: The value of `questionCounter` is output, followed by a period and a fill-in-the-blank line.
- At the end of the loop body, the loop control variable is altered by adding 1 to `questionCounter`, and the `questionCounter` comparison is made again.

(continued)

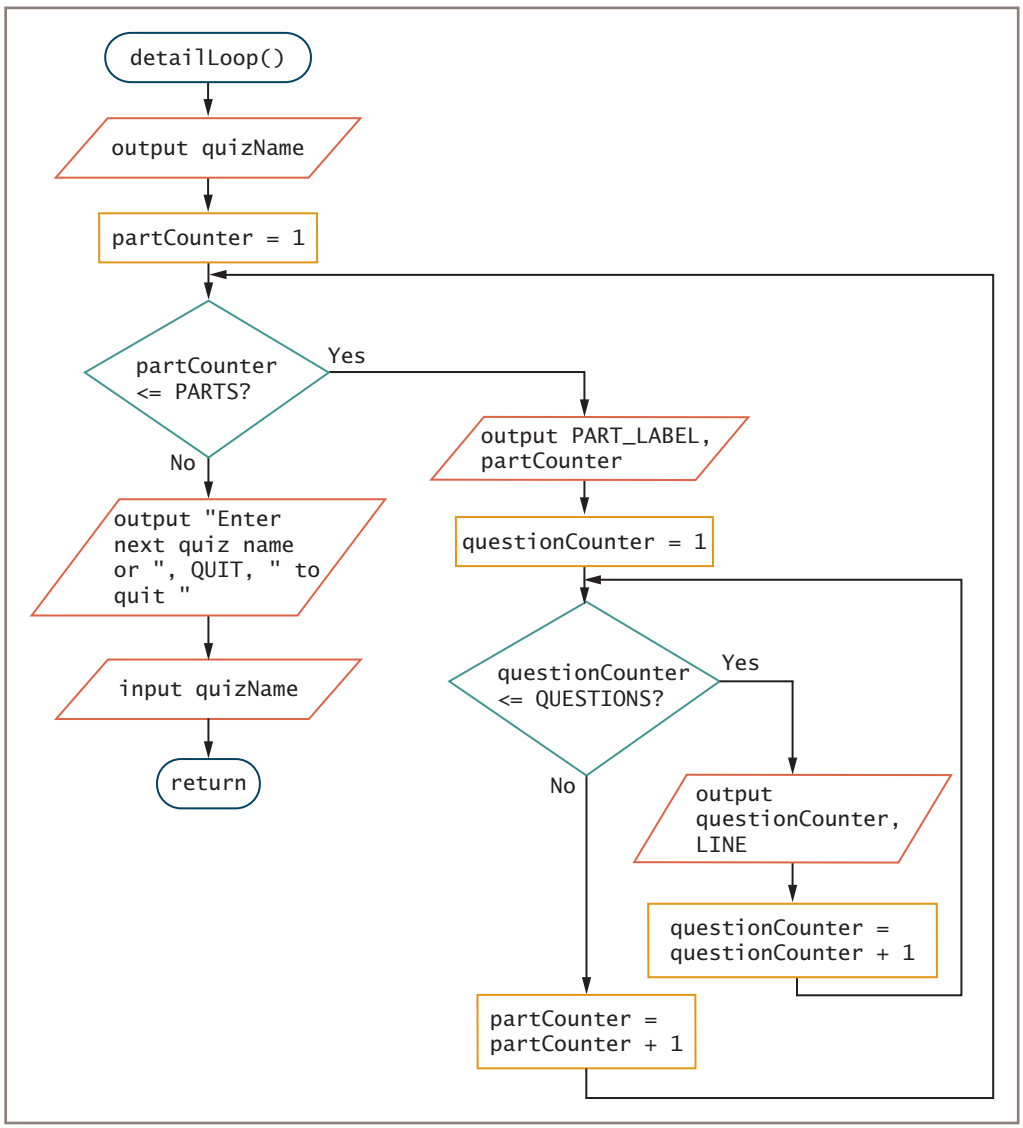


Figure 5-8 Flowchart and pseudocode for `AnswerSheet` program *(continues)*

In other words, when `partCounter` is 1, the part heading is output and underscore lines are output for questions 1, 2, and 3. Then `partCounter` becomes 2, the part heading is output, and underscore lines are created for another set of questions 1, 2, and 3. Then `partCounter` becomes 3, 4, and 5 in turn, and three underscore lines numbered 1, 2, and 3 are created for each part. In all, 15 underscore answer lines are created for each quiz.

In the program in Figure 5-8, it is important that `questionCounter` is reset to 1 within the outer loop, just before entering the inner loop. If this step was omitted, Part 1

(continued)

```
start
   Declarations
      string quizName
      num partCounter
      num questionCounter
      string QUIT = "ZZZ"
      num PARTS = 5
      num QUESTIONS = 3
      string PART_LABEL = "Part "
      string LINE = ". _____"
   housekeeping()
   while quizName <> QUIT
      detailLoop()
   endwhile
   endOfJob()
stop

housekeeping()
   output "Enter quiz name or", QUIT, " to quit "
   input quizName
return

detailLoop()
   output quizName
   partCounter = 1
   while partCounter <= PARTS
      output PART_LABEL, partCounter
      questionCounter = 1
      while questionCounter <= QUESTIONS
         output questionCounter, LINE
         questionCounter = questionCounter + 1
      endwhile
      partCounter = partCounter + 1
   endwhile
   output "Enter next quiz name or ", QUIT, " to quit "
   input quizName
return

endOfJob()
return
```

Figure 5-8 Flowchart and pseudocode for `AnswerSheet` program

would contain questions 1, 2, and 3, but subsequent parts would be empty because `questionCounter` would never again be less than or equal to `QUESTIONS`.

Studying the answer sheet program reveals several facts about nested loops:

- Nested loops never overlap. An inner loop is always completely contained within an outer loop.
- An inner loop goes through all of its iterations each time its outer loop goes through just one iteration.
- The total number of iterations executed by a nested loop is the number of inner loop iterations times the number of outer loop iterations.



Watch the video *Nested Loops*.

TWO TRUTHS & A LIE

Nested Loops

1. When one loop is nested inside another, the loop that contains the other loop is called the outer loop.
2. You need to create nested loops when the values of two or more variables repeat to produce every combination of values.
3. The number of times a loop executes always depends on a constant.

The false statement is #3. The number of times a loop executes might depend on a constant, but it might also depend on a value that varies.

Avoiding Common Loop Mistakes

Programmers make the following common mistakes with loops:

- Failing to initialize the loop control variable
- Neglecting to alter the loop control variable
- Using the wrong type of comparison when testing the loop control variable
- Including statements inside the loop body that belong outside the loop

The following sections explain these common mistakes in more detail.

Mistake: Failing to Initialize the Loop Control Variable

Failing to initialize a loop's control variable is a mistake. For example, consider the program in Figure 5-9. It prompts the user for a name, and while the value of `name` continues not to be the sentinel value *ZZZ*, the program outputs a greeting that uses the name and asks for the next name. This program works correctly.

Figure 5-10 shows an incorrect program in which the loop control variable is not assigned a starting value. If the `name` variable is not set to a starting value, then when the `eof` condition is tested, there is no way to predict whether it will be true. If the user does not enter a value

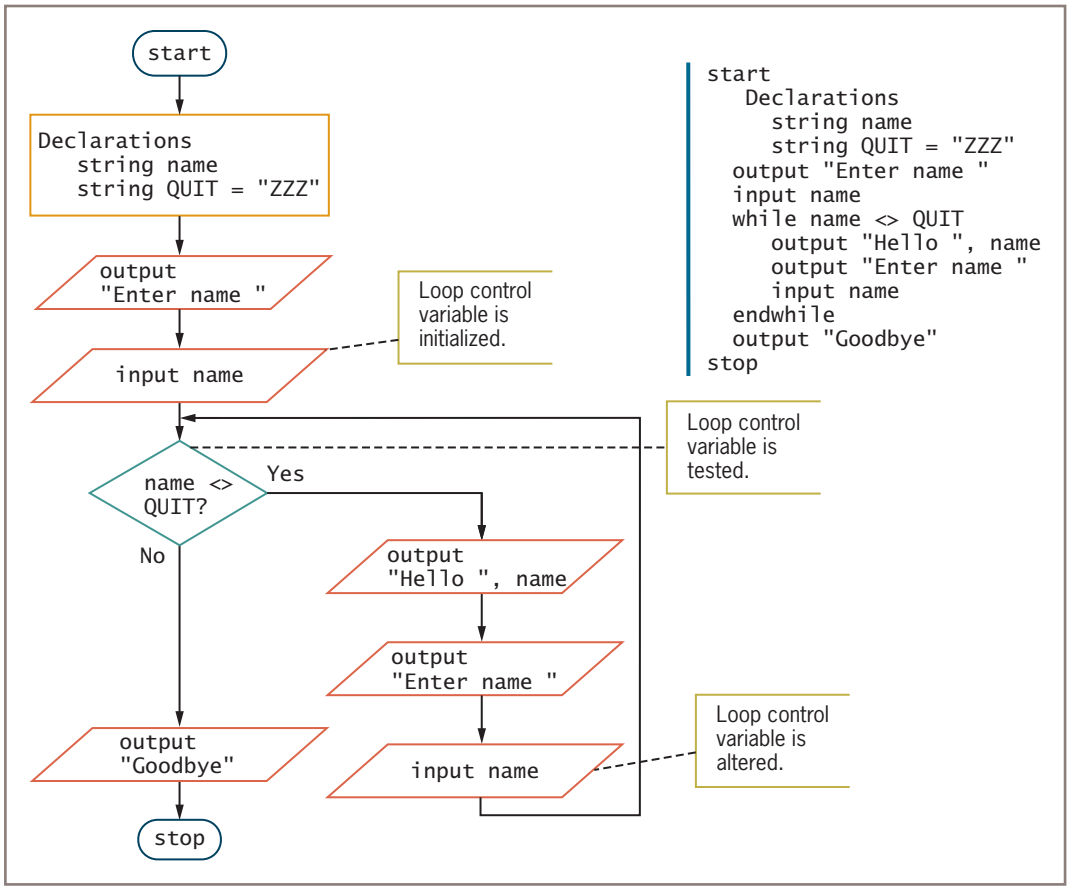


Figure 5-9 Correct logic for greeting program

for `name`, the garbage value originally held by that variable might or might not be *ZZZ*. So, one of two scenarios follows:

- Most likely, the uninitialized value of `name` is not *ZZZ*, so the first greeting output will include garbage—for example, *Hello 12BGr5*.
- By a remote chance, the uninitialized value of `name` is *ZZZ*, so the program ends immediately before the user can enter any names.

Mistake: Neglecting to Alter the Loop Control Variable

Different sorts of errors will occur if you fail to provide a way to alter a loop control variable within a loop. For example, in the program in Figure 5-9 that accepts and displays names, you create such an error if you don't accept names within the loop. Figure 5-11 shows the resulting incorrect logic.

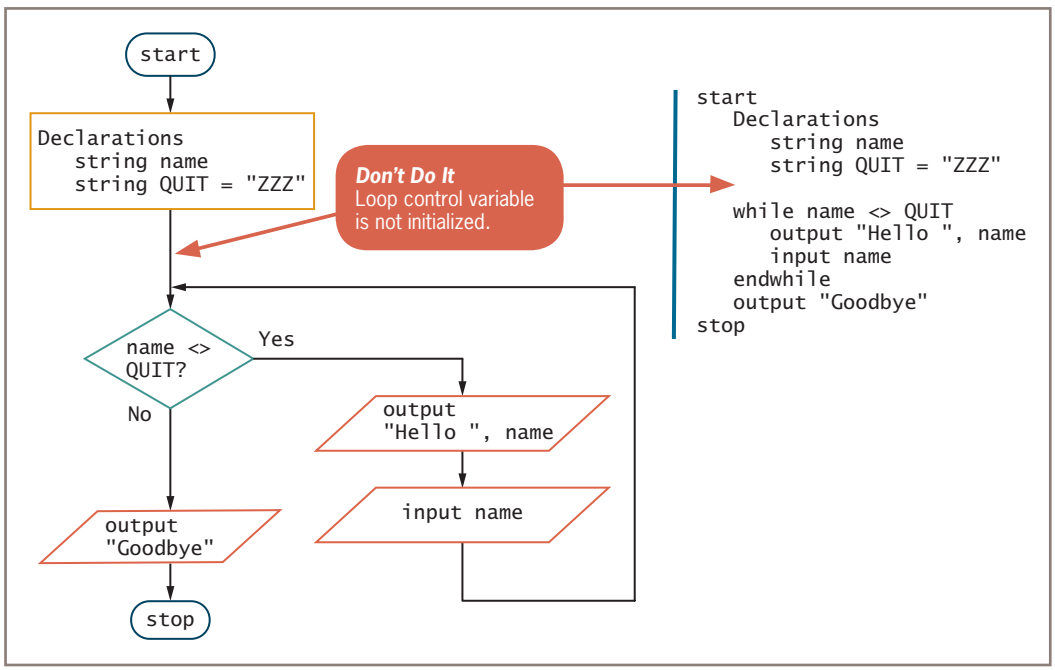


Figure 5-10 Incorrect logic for greeting program because the loop control variable initialization is missing

If you remove the `input name` instruction from the end of the loop in the program in Figure 5-11, no name is ever entered after the first one. For example, assume that when the program starts, the user enters *Fred*. *Fred* is not the `eof` value, and the loop will be entered. After a greeting is output for Fred, no new name is entered, so when the logic returns to the loop-controlling evaluation, the `name` will still not be *ZZZ*, and greetings for Fred will continue to be output infinitely. Under normal conditions, you never want to create a loop that cannot terminate.

Mistake: Using the Wrong Type of Comparison When Testing the Loop Control Variable

Programmers must be careful to use the correct type of comparison in the evaluation that controls a loop. A comparison is correct only when the appropriate operands and operator are used. For example, although only one keystroke differs between the original greeting program in Figure 5-9 and the one in Figure 5-12, the original program correctly produces named greetings and the second one does not.

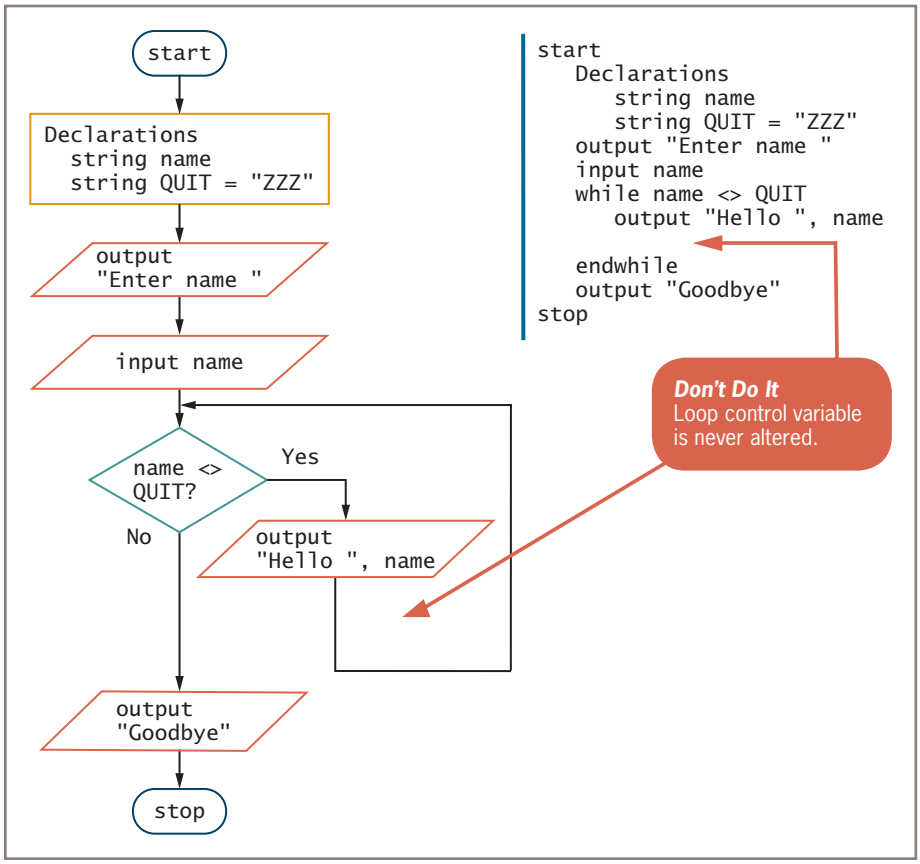


Figure 5-11 Incorrect logic for greeting program because the loop control variable is not altered

In Figure 5-12, a greater-than comparison (>) is made instead of a not-equal-to (<>) comparison. Suppose that when the program executes, the user enters *Fred* as the first name. In most programming languages, when the comparison between *Fred* and *ZZZ* is made, the values are compared alphabetically. *Fred* is not greater than *ZZZ*, so the loop is never entered, and the program ends.

Using the wrong type of comparison in a loop can have serious effects. For example, in a counted loop, if you use <= instead of < to compare a counter to a sentinel value, the program will perform one loop execution too many. If the loop's purpose is only to display greetings, the error might not be critical, but if such an error occurred in a loan company application, each customer might be charged a month's additional interest. If the error occurred in an airline's application, it might overbook a flight. If the error occurred in a pharmacy's drug-dispensing application, each patient might receive one extra (and possibly harmful) unit of medication.

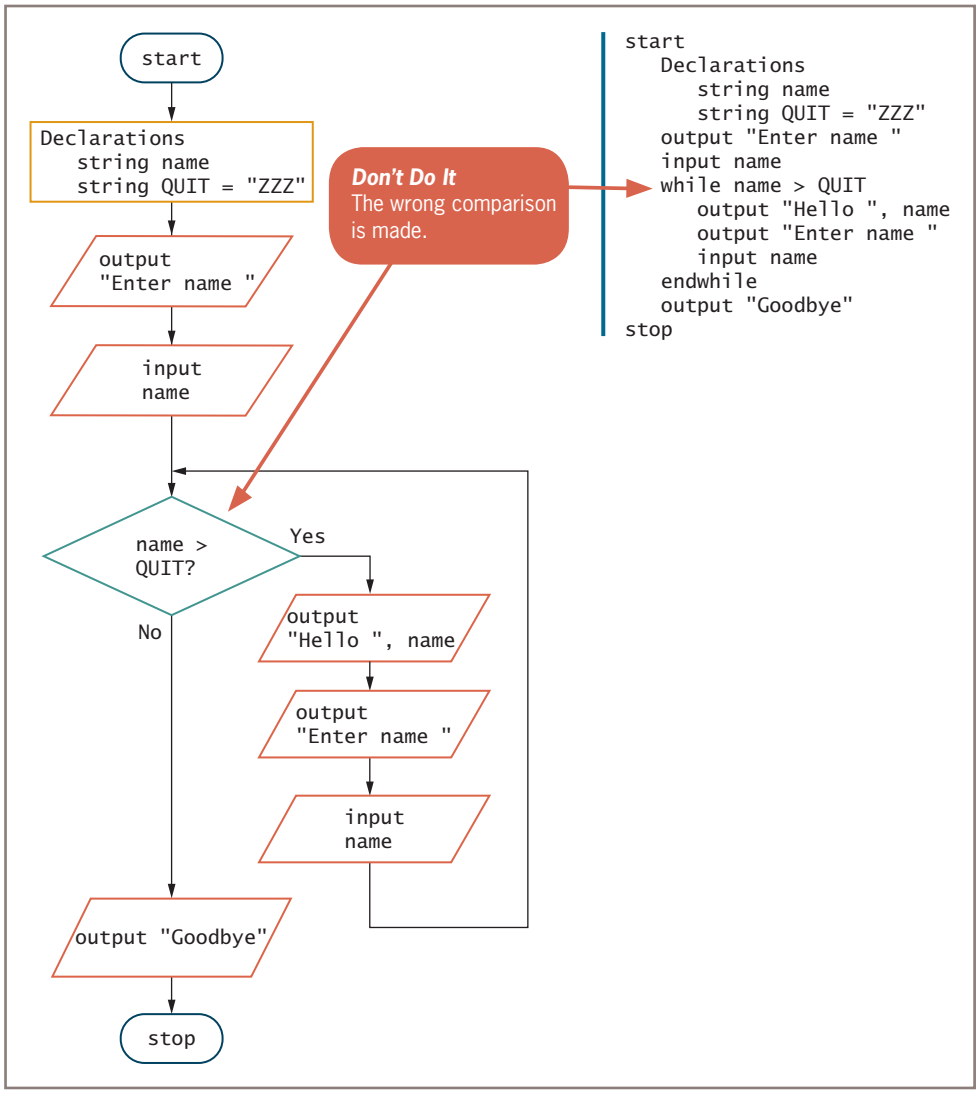


Figure 5-12 Incorrect logic for greeting program because the wrong test is made with the loop control variable

Mistake: Including Statements Inside the Loop Body that Belong Outside the Loop

Suppose that you write a program for a store manager who wants to discount every item he sells by 30 percent. The manager wants 100 new price label stickers for each item. The user enters a price, the new discounted price is calculated, 100 stickers are printed, and the next price is entered. Figure 5-13 shows a program that performs the job inefficiently

because the same value, newPrice, is calculated 100 separate times for each price that is entered.

Figure 5-14 shows the same program, in which the newPrice value that is output on the sticker is calculated only once per new price; the calculation has been moved to a better

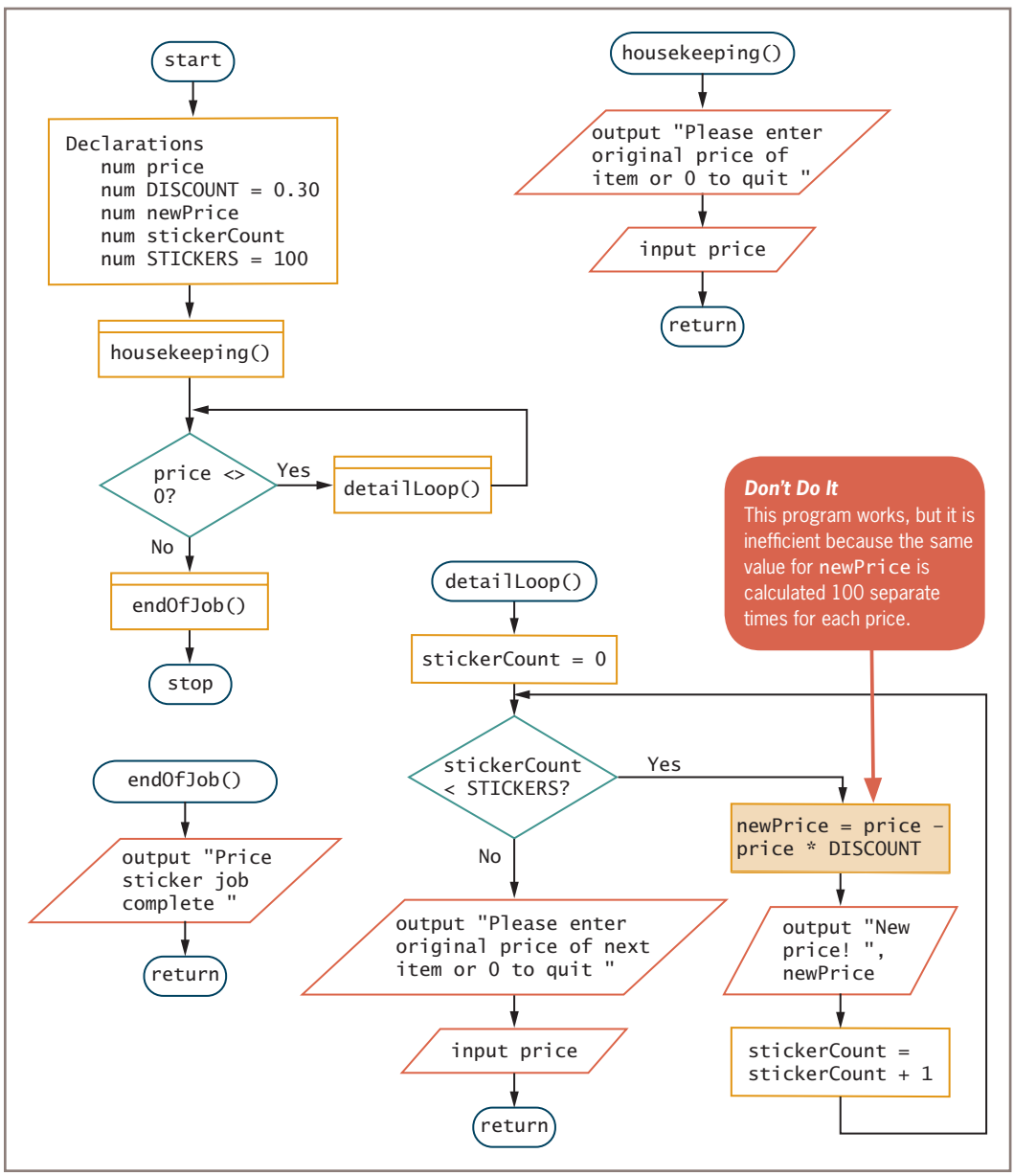


Figure 5-13 Inefficient way to produce 100 discount price stickers for differently priced items *(continues)*

(continued)

```
start
   Declarations
      num price
      num DISCOUNT = 0.30
      num newPrice
      num stickerCount
      num STICKERS = 100
   housekeeping()
   while price <> 0
      detailLoop()
   endwhile
   endOfJob()
stop

housekeeping()
   output "Please enter original price of item or 0 to quit "
   input price
return

detailLoop()
   stickerCount = 0
   while stickerCount < STICKERS
      newPrice = price - price * DISCOUNT
      output "New price! ", newPrice
      stickerCount = stickerCount + 1
   endwhile
   output "Please enter original price of
      next item or 0 to quit "
   input price
return

endOfJob()
   output "Price sticker job complete"
return
```

Don't Do It
This program works, but it is inefficient because the same value for `newPrice` is calculated 100 separate times for each price.

Figure 5-13 Inefficient way to produce 100 discount price stickers for differently priced items

location. The programs in Figures 5-13 and 5-14 do the same thing, but the second program does it more efficiently. As you become more proficient at programming, you will recognize many opportunities to perform the same tasks in alternate, more elegant, and more efficient ways.



When you describe people or events as elegant, you mean they possess a refined gracefulness. Similarly, programmers use the term *elegant* to describe programs that are well designed and easy to understand and maintain.

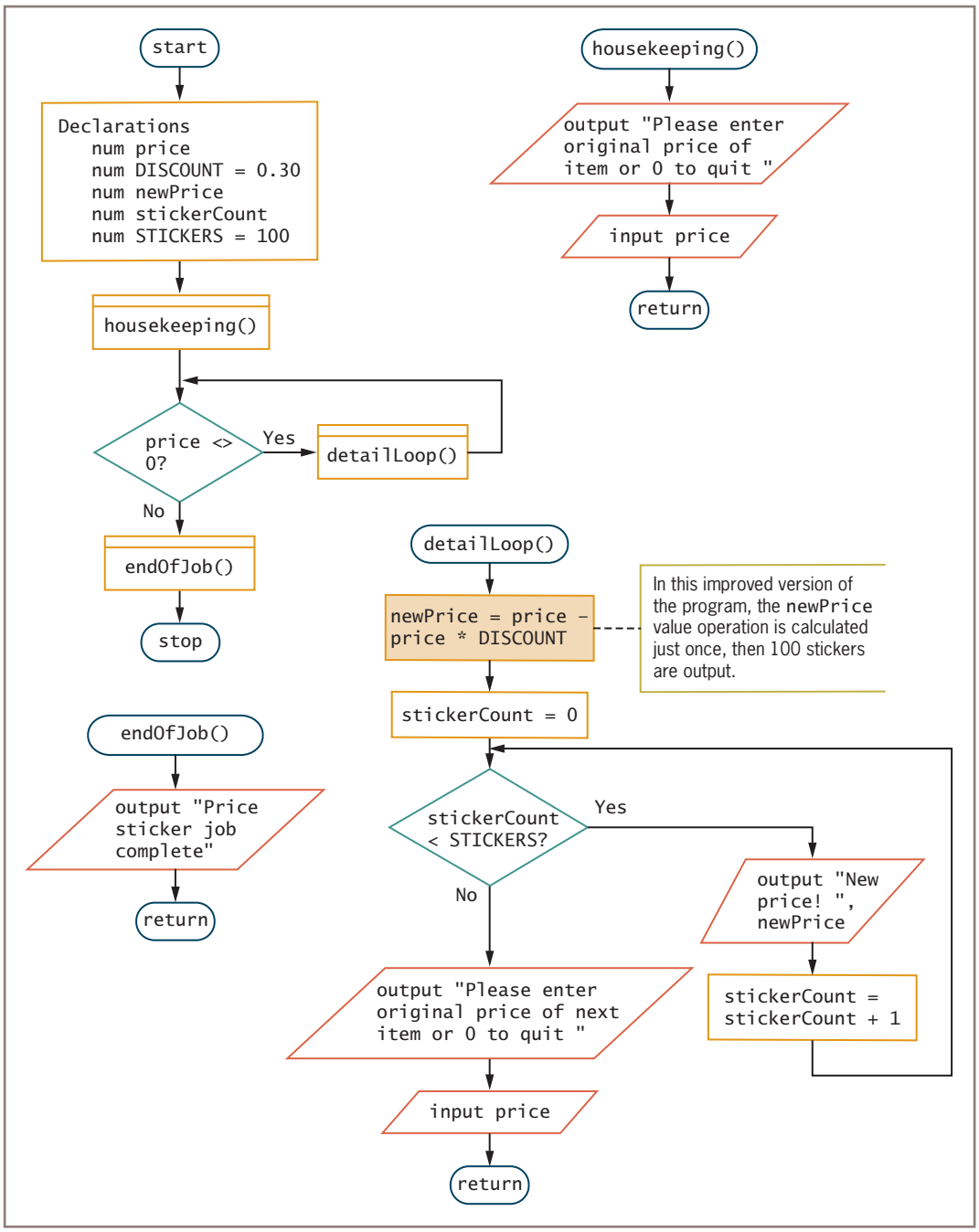


Figure 5-14 Improved discount sticker-making program *(continues)*

(continued)

```
start
   Declarations
      num price
      num DISCOUNT = 0.30
      num newPrice
      num stickerCount
      num STICKERS = 100
   housekeeping()
   while price <> 0
      detailLoop()
   endwhile
   endOfJob()
stop

housekeeping()
   output "Please enter original price of item or 0 to quit "
   input price
return

detailLoop()
   newPrice = price - price * DISCOUNT
   stickerCount = 0
   while stickerCount < STICKERS
      output "New price! ", newPrice
      stickerCount = stickerCount + 1
   endwhile
   output "Please enter original price of next item or 0 to quit "
   input price
return

endOfJob()
   output "Price sticker job complete"
return
```

In this improved version of the program, the newPrice value operation is calculated just once, then 100 stickers are output.

Figure 5-14 Improved discount sticker-making program

TWO TRUTHS & A LIE

Avoiding Common Loop Mistakes

1. In a loop, neglecting to initialize the loop control variable is a mistake.
2. In a loop, neglecting to alter the loop control variable is a mistake.
3. In a loop, comparing the loop control variable using >= or <= is a mistake.

The false statement is #3. Many loops are created correctly using <= or >=.

Using a for Loop

Every high-level programming language contains a `while` statement that you can use to code any loop, including both indefinite and definite loops. In addition to the `while` statement, most computer languages support a `for` statement. You usually use the **`for` statement**, or **`for` loop**, with definite loops—those that will loop a specific number of times—when you know exactly how many times the loop will repeat. The `for` statement provides you with three actions in one compact statement. In a `for` statement, a loop control variable is:

- Initialized
- Tested
- Altered

Quick Reference 5-2 shows pseudocode standards for a `for` statement.

QUICK REFERENCE 5-2 `for` Statement Pseudocode Standards

The `for` keyword starts the statement.

The loop control variable is set to an initial value when the `for` loop starts.

On every iteration, the loop control variable is compared to `finalValue`. When the loop control variable exceeds `finalValue`, the loop ends.

After each execution of the loop body, the loop control variable is altered (usually increased) by the step value, just before the loop control variable is again compared to `finalValue`.

Although many modern languages do not require indentation, the bodies of `for` loops in this book are indented.

```
for loopControlVariable = initialValue to finalValue step stepValue
   statements that execute when loopControlVariable is in range
endfor
```

The `endfor` keyword ends the statement.

The amount by which a `for` loop control variable changes often is called a **step value**. The step value can be any number and can be either positive or negative; that is, it can increment or decrement.

A `for` loop can express the same logic as a `while` statement, but in a more compact form. You never are required to use a `for` statement for any loop; a `while` loop can always be used instead. For example, to display *Hello* four times, you can write either of the sets of statements in Figure 5-15.

```
count = 0
while count <= 3
   output "Hello"
   count = count + 1
endwhile
```

```
for count = 0 to 3 step 1
   output "Hello"
endfor
```

Figure 5-15 Comparable `while` and `for` statements that each output *Hello* four times

The code segments in Figure 5-15 each accomplish the same tasks:

- The variable `count` is initialized to 0.
- The `count` variable is compared to the limit value 3; while `count` is less than or equal to 3, the loop body executes.
- As the last statement in the loop execution, the value of count increases by 1. After the increase, the comparison to the limit value is made again.

A `while` loop can always be used instead of a `for` loop, but when a loop's execution is based on a loop control variable progressing from a known starting value to a known ending value in equal steps, the `for` loop provides a convenient shorthand. It is easy for others to read, and because the loop control variable's initialization, testing, and alteration are all performed in one location, you are less likely to leave out one of these crucial elements.

Although `for` loops are commonly used to control execution of a block of statements a fixed number of times, the programmer doesn't need to know the starting, final, or step value for the loop control variable when the program is written. For example, any of the values might be entered by the user, or might be the result of a calculation.



The `for` loop is particularly useful when processing arrays. You will learn about arrays in Chapter 6.



In Java, C++, and C#, a for loop that displays 21 values (0 through 20) might look similar to the following:

```
for(count = 0; count <= 20; count++)
{
   output count;
}
```

The three actions (initializing, evaluating, and altering the loop control variable) are separated by semicolons within a set of parentheses that follow the keyword `for`. The expression `count++` increases `count` by 1. In each of the three languages, the block of statements that depends on the loop sits between a pair of curly braces, so the `endfor` keyword is not used. None of the three languages uses the keyword `output`, but all of them end output statements with a semicolon.

Both the `while` loop and the `for` loop are examples of pretest loops. In a **pretest loop**, the loop control variable is tested before each iteration. That means the loop body might never execute because the evaluation controlling the loop might be false the first time it is made.



Some books and flowchart programs use a symbol that looks like a hexagon to represent a `for` loop in a flowchart. However, no special symbols are needed to express a `for` loop's logic. A `for` loop is simply a code shortcut, so this book uses standard flowchart symbols to represent initializing the loop control variable, testing it, and altering it.

TWO TRUTHS & A LIE

Using a `for` Loop

1. The `for` statement provides you with three actions in one compact statement: initializing, testing, and altering a loop control variable.
2. A `for` statement body always executes at least one time.
3. In most programming languages, you can provide a `for` loop with any step value.

The false statement is #2. A `for` statement body might not execute depending on the initial value of the loop control variable.

Using a Posttest Loop

Most languages allow you to use a variation of the pretest looping structure that tests the loop control variable after each iteration rather than before. In a **posttest loop**, the loop body executes at least one time because the loop control variable is not tested until after the first iteration.

Recall that a structured loop (often called a `while` loop) looks like Figure 5-16. A posttest loop, sometimes called a `do-while` loop, looks like Figure 5-17.

An important difference exists between the structures in Figure 5-16 and 5-17. In a `while` loop, you evaluate a condition and, depending on the result, you might or might not enter the loop to execute the loop's body. In other words, the condition that controls the loop is tested at the top of the loop.

Conversely, in a **`do-while` loop**, you ensure that the procedure executes at least once; then, depending on the result of the loop-controlling evaluation, the loop body may or may not

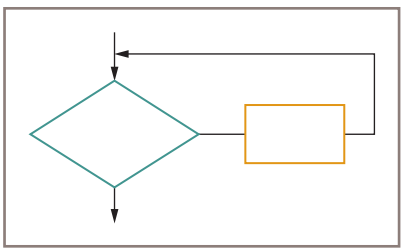


Figure 5-16 The `while` loop, which is a pretest

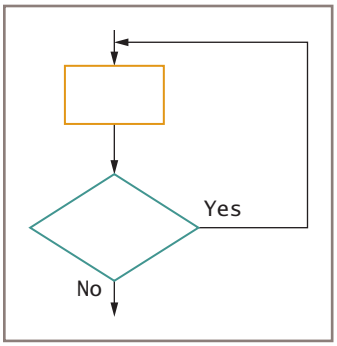


Figure 5-17 Structure of a `do-while` loop, which is a posttest loop

execute additional times. In other words, you test the loop-controlling condition at the bottom of the loop.

Notice that the word *do* begins the name of the `do-while` loop. This should remind you that the action you "do" precedes testing the condition.

You encounter examples of `do-while` looping every day. For example:

```
do
  pay a bill
while more bills remain to be paid
```

As another example:

```
do
  wash a dish
while more dishes remain to be washed
```

In these examples, the activity (paying a bill or washing a dish) must occur at least one time. With a `do-while` loop, you evaluate the condition that determines whether you continue only after the activity has been executed at least once. You never are required to use a posttest loop; you can duplicate the same series of actions by creating a sequence followed by a standard, pretest `while` loop. Consider the flowcharts and pseudocode in Figure 5-18.

On the left side of Figure 5-18, `A` executes, and then `B` is tested. If `B` is true, then `A` executes and `B` is tested again. On the right side of the figure, `A` executes, and then `B` is tested. If `B` is true, then `A` executes and `B` is tested again. In other words, both sets of flowchart and pseudocode segments do exactly the same thing.

Because programmers understand that any posttest loop (`do-while`) can be expressed with a sequence followed by a `while` loop, most languages allow at least one version of the posttest loop for convenience.

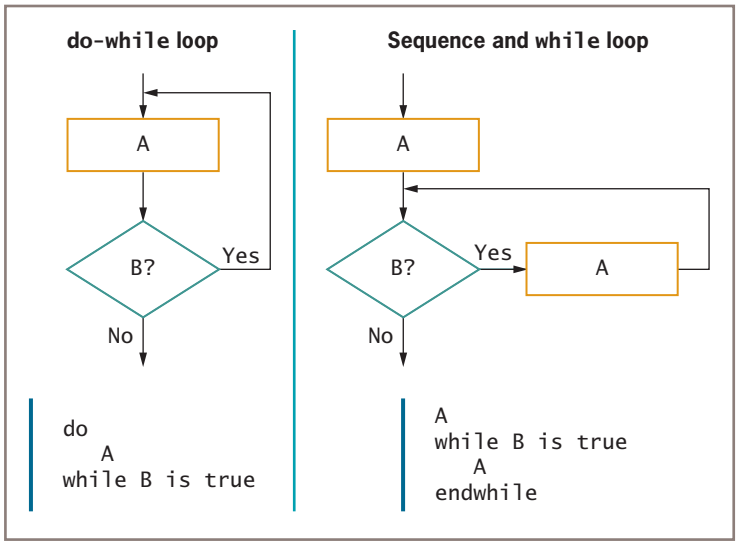


Figure 5-18 Flowchart and pseudocode for `do-while` loop and `while` loop that do the same thing

TWO TRUTHS & A LIE

Using a Posttest Loop

1. In a `while` loop, the loop body might not execute.
2. In a `do-while` loop, the loop body might not execute.
3. The logic expressed by a `do-while` loop can always be expressed using a sequence and a `while` loop.

The false statement is #2. In a `do-while` loop, the loop body executes at least once.

Recognizing the Characteristics Shared by Structured Loops

As you examine Figures 5-16 and 5-17, notice that in the `while` loop, the loop-controlling evaluation is placed at the beginning of the steps that repeat, and in the `do-while` loop, the loop-controlling evaluation is placed at the end of the sequence of steps that repeat.

All structured loops, both pretest and posttest, share these two characteristics:

- The loop-controlling evaluation must provide either the entry to or exit from the structure.
- The loop-controlling evaluation provides the *only* entry to or exit from the structure.

In other words, there is exactly one loop-controlling evaluation, and it provides either the only entrance to or the only exit from the loop.



Some languages support a **`do-until` loop**, which is a posttest loop that iterates until the loop-controlling evaluation is false. The `do-until` loop also follows structured loop rules.

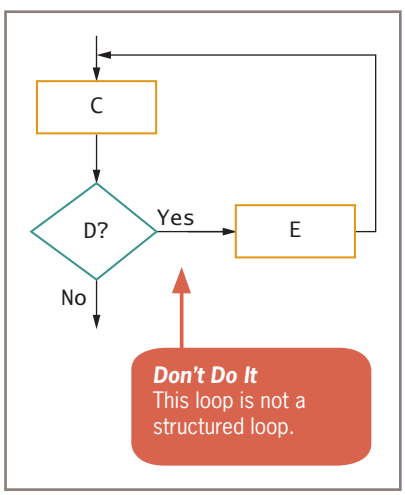


Figure 5-19 Unstructured loop

Figure 5-19 shows an unstructured loop. It is not a `while` loop, which begins with an evaluation and, after an action, returns to the decision. It is also not a `do-while` loop, which begins with an action and ends with a decision that might repeat the action. Instead, it begins like a posttest loop (a `do-while` loop), with a process followed by a decision, but one branch of the decision does not repeat the initial process. Instead, it performs an additional new action before repeating the initial process.

If you need to use the logic shown in Figure 5-19—performing a task, evaluating a condition, and performing an additional task before looping back to the first process—then the way to make the logic structured is to repeat the initial process within the loop at the end of the loop. Figure 5-20 shows the same logic as Figure 5-19, but now it is structured logic, with a sequence of two actions occurring within the loop.

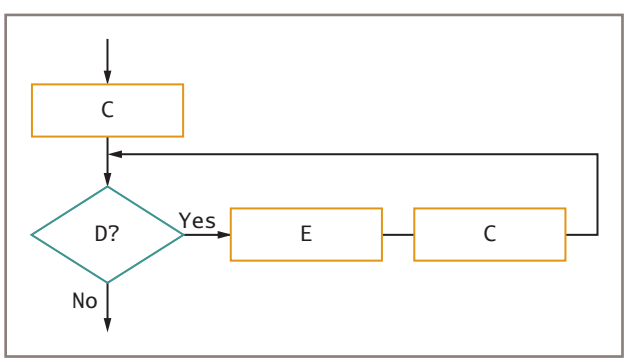


Figure 5-20 Sequence and structured loop that accomplish the same tasks as Figure 5-19



Especially when you are first mastering structured logic, you might prefer to use only the three basic structures—sequence, selection, and `while` loop. Every logical problem can be solved using only these three structures, and you can understand all of the examples in this book using only these three structures.

TWO TRUTHS & A LIE

Recognizing the Characteristics Shared by Structured Loops

1. In a structured loop, the loop-controlling evaluation must provide either the entry to or exit from the repeating structure.
2. In a structured loop, the loop-controlling evaluation provides the *only* entry to or exit from the structure.
3. If you need to perform a task, evaluate a condition, and perform an additional task, you cannot use a structured loop.

The false statement is #3. All looping tasks can be made to be structured.

Common Loop Applications

Although every computer program is different, many techniques are common to a variety of applications. Loops, for example, are frequently used to accumulate totals and to validate data.

Using a Loop to Accumulate Totals

Business reports often include totals. The supervisor who requests a list of employees in the company dental plan is often as interested in the number of participating employees as in who they are. When you receive your telephone bill each month, you usually check the total as well as charges for the individual calls.

Assume that a real estate broker wants to see a list of all properties sold in the last month as well as the total value for all the properties. A program might accept sales data that includes the street address of each property sold and its selling price. The data records might be entered by a clerk as each sale is made, and stored in a file until the end of the month; then they can be used in a monthly report. Figure 5-20 shows an example of such a report.

To create the sales report, you must output the address and price for each property sold and add its value to an accumulator. An **accumulator** is a variable that you use to gather or accumulate values, and is very similar to a counter that you use to count loop iterations. Usually, however, you add just one to a counter, whereas you add some other value to an

```
MONTH-END SALES REPORT

Address              Price

287 Acorn St       150,000
12 Maple Ave       310,000
8723 Marie Ln       65,500
222 Acorn St       127,000
29 Bahama Way      450,000

Total            1,102,500
```

Figure 5-21 Month-end real estate sales report

accumulator. If the real estate broker wants to know how many listings the company holds, you *count* them. When the broker wants to know the total real estate value, you *accumulate* it.

To accumulate total real estate prices, you declare a numeric variable such as `accumPrice` and initialize it to 0. As you get data for each real estate transaction, you output it and add its value to the accumulator `accumPrice`, as shown shaded in Figure 5-22.



Some programming languages assign 0 to a variable you fail to initialize explicitly, but many do not. When you try to add a value to an uninitialized variable, most languages will issue an error message; worse, some languages, such as C and C++, will let you incorrectly start with an accumulator that holds garbage. All of the examples in this book assign the value 0 to each accumulator before using it.



In earlier program examples in this chapter, the modules were named `housekeeping()`, `detailLoop()`, and `endOfJob()`. In the program in Figure 5-22, they are named `getReady()`, `createReport()`, and `finishUp()`. You can assign modules any names that make sense to you, as long as you are consistent with the names within a program.

After the program in Figure 5-22 gets and displays the last real estate transaction, the user enters the sentinel value and loop execution ends. At that point, the accumulator will hold the grand total of all the real estate values. The program displays the word *Total* and the accumulated value `accumPrice`. Then the program ends. Figure 5-22 highlights the three actions you usually must take with accumulators:

- Accumulators are initialized to 0.
- Accumulators are altered, usually once for every data set processed, and most often are altered through addition.
- At the end of processing, accumulators are output.

After outputting the value of `accumPrice`, new programmers often want to reset it to 0. Their argument is that they are "cleaning up after themselves." Although you can take this step without harming the execution of the program, it serves no useful purpose. You cannot set `accumPrice` to 0 in anticipation of having it ready for the next program, or even for the next time you execute the same program. Variables exist only during an execution of the program, and even if a future application happens to contain a variable named `accumPrice`, the variable will not necessarily occupy the same memory location as this one. Even if

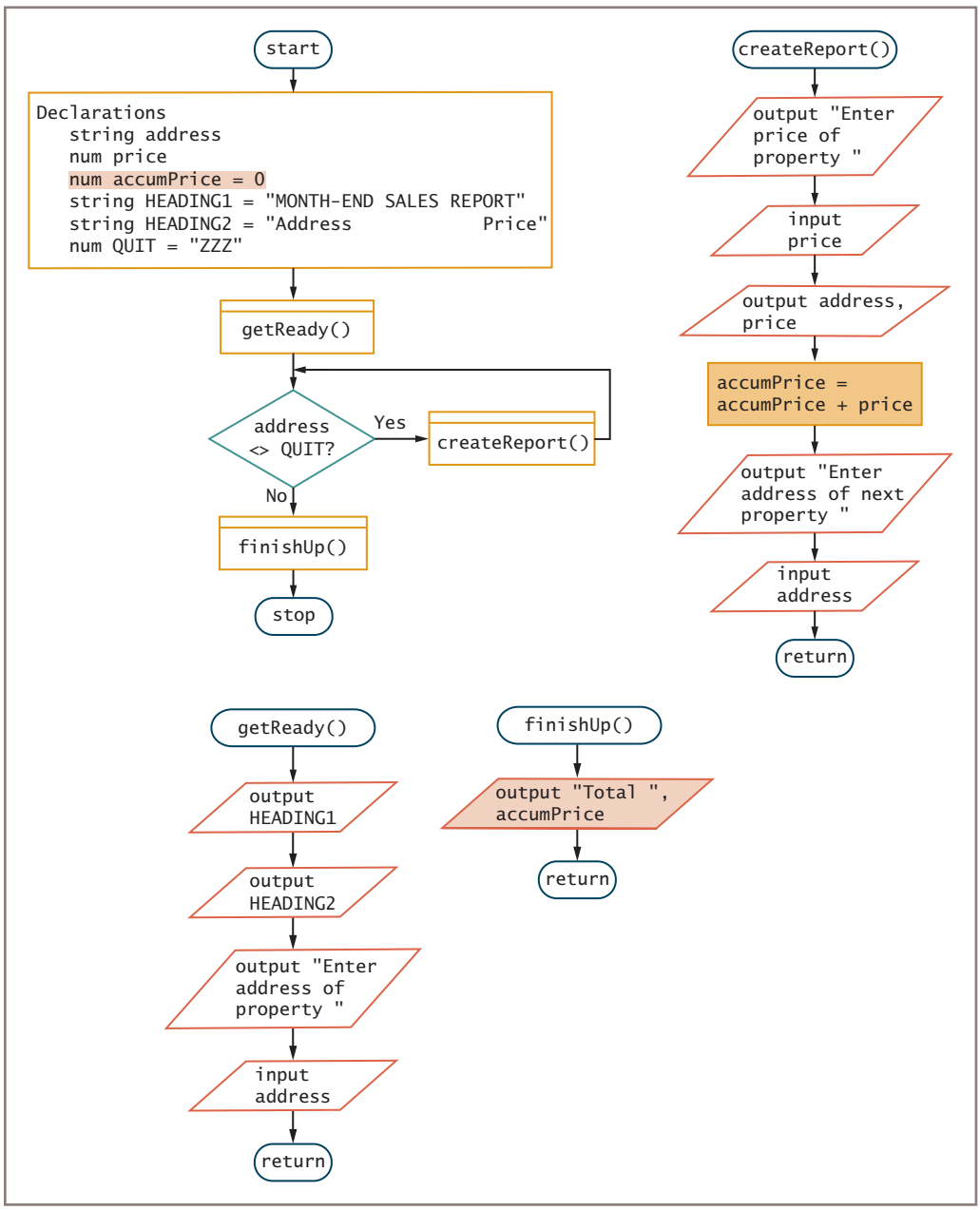


Figure 5-22 Flowchart and pseudocode for real estate sales report program *(continues)*

(continued)

```
start
   Declarations
      string address
      num price
      num accumPrice = 0
      string HEADING1 = "MONTH-END SALES REPORT"
      string HEADING2 = "Address            Price"
      num QUIT = "ZZZ"
   getReady()
   while address <> QUIT
      createReport()
   endwhile
   finishUp()
stop

getReady()
   output HEADING1
   output HEADING2
   output "Enter address of property "
   input address
return

createReport()
   output "Enter price of property "
   input price
   output address, price
   accumPrice = accumPrice + price
   output "Enter address of next property "
   input address
return

finishUp()
   output "Total ", accumPrice
return
```

Figure 5-22 Flowchart and pseudocode for real estate sales report program

you run the same application a second time, the variables might occupy physical memory locations different from those during the first run. At the beginning of any module, it is the programmer's responsibility to initialize all variables that must start with a specific value. There is no benefit to changing a variable's value when it will never be used again during the current execution.

Some business reports are **summary reports**—they contain only totals with no data for individual records. In the example in Figure 5-21, suppose that the broker did not care about details of individual sales, but only about the total for all transactions. You could create a summary report by omitting the step that outputs `address` and `price` from the `createReport()` module. Then you could simply output `accumPrice` at the end of the program.

Using a Loop to Validate Data

When you ask a user to enter data into a computer program, you have no assurance that the data will be accurate. Incorrect user entries are by far the most common source of computer errors. The programs you write will be improved if you employ **defensive programming**, which means trying to prepare for all possible errors before they occur. Loops are frequently used to **validate data**—that is, to make sure it is meaningful and useful. For example, validation might ensure that a value is the correct data type or that it falls within an acceptable range.

Suppose that part of a program you are writing asks a user to enter a number that represents his or her birth month. If the user types a number lower than 1 or greater than 12, you must take some sort of action. For example:

- You could display an error message and stop the program.
- You could choose to assign a default value for the month (for example, 1) before proceeding.
- You could reprompt the user for valid input.

If you choose this last course of action, you then could take at least two approaches. You could use a selection structure, and if the month is invalid, you could ask the user to reenter a number, as shown in Figure 5-23.

The problem with the logic in Figure 5-23 is that the comparisons of `month` to `LOW_MONTH` and `HIGH_MONTH` are made only once, and the user still might not enter valid data on the second attempt. Of course, you could add a third decision, but you still couldn't control what the user enters.

The superior solution is to use a loop to continuously prompt a user for a month until the user enters it correctly. Figure 5-24 shows this approach.

Of course, data validation doesn't prevent all errors; just because a data item is valid does not mean that it is correct. For example, a program can determine that 5 is a valid birth month, but not that your birthday actually falls in month 5. Programmers employ the acronym **GIGO** for *garbage in, garbage out*. It means that if your input is incorrect, your output is worthless.

Limiting a Reprompting Loop

Reprompting a user is a good way to try to ensure valid data, but it can be frustrating to a user if it continues indefinitely. For example, suppose the user must enter a valid birth month, but has used another application in which January was month 0, and keeps entering 0 no matter how many times you repeat the prompt. One helpful addition to the program would be to use the limiting values as part of the prompt. In other words, instead

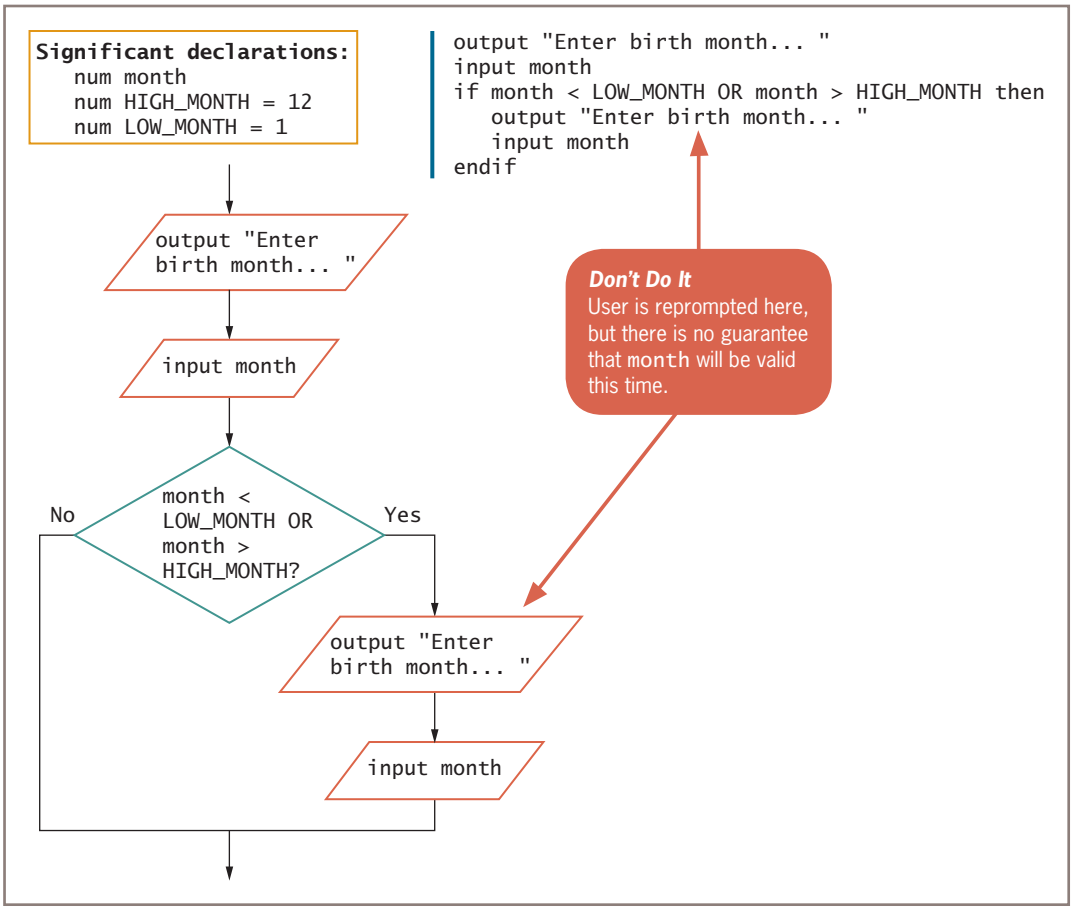


Figure 5-23 Reprompting a user once after an invalid month is entered

of the statement `output "Enter birth month ..."`, the following statement might be more useful:

```
output "Enter birth month between ", LOW_MONTH, ", and " HIGH_MONTH, " ... "
```

The user would see *Enter birth month between 1 and 12 ...*. Still, the user might not understand the prompt or not read it carefully, and might continue to enter unacceptable values, so you might want to employ the tactic used in Figure 5-25, in which the program maintains a count of the number of reprompts. In this example, a constant named `ATTEMPTS` is set to 3. While a count of the user's attempts at correct data entry remains below this limit, and the user enters invalid data, the user continues to be reprompted. If the user exceeds the limited number of allowed attempts, the loop ends.

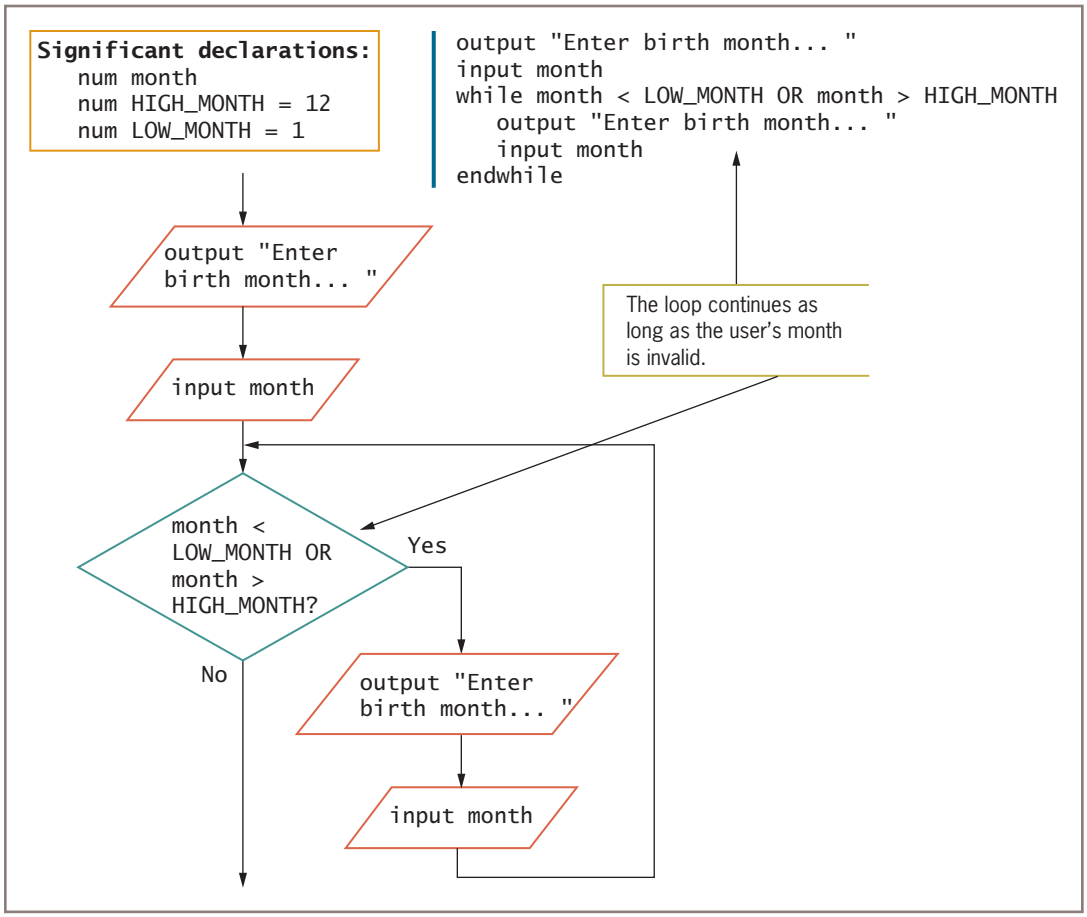


Figure 5-24 Reprompting a user continuously after an invalid month is entered

The action that follows the loop in Figure 5-25 depends on the application. If `count` equals `ATTEMPTS` after the data-entry loop ends, you might want to force the invalid data to a default value. **Forcing** a data item means you override incorrect data by setting the variable to a specific, predetermined value. For example, you might decide that if a month value does not fall between 1 and 12, you will force the month to 0 or to the current month. In a different application, you might just choose to end the program. Ending a program prematurely can frustrate users, and can result in lost revenue for a company. For example, if it is difficult to complete a transaction on a company's website, users might give up and not do business with the organization. In an interactive, Web-based program, if a user has trouble providing valid data, you might choose to have a customer service representative start a chat session with the user to offer help.

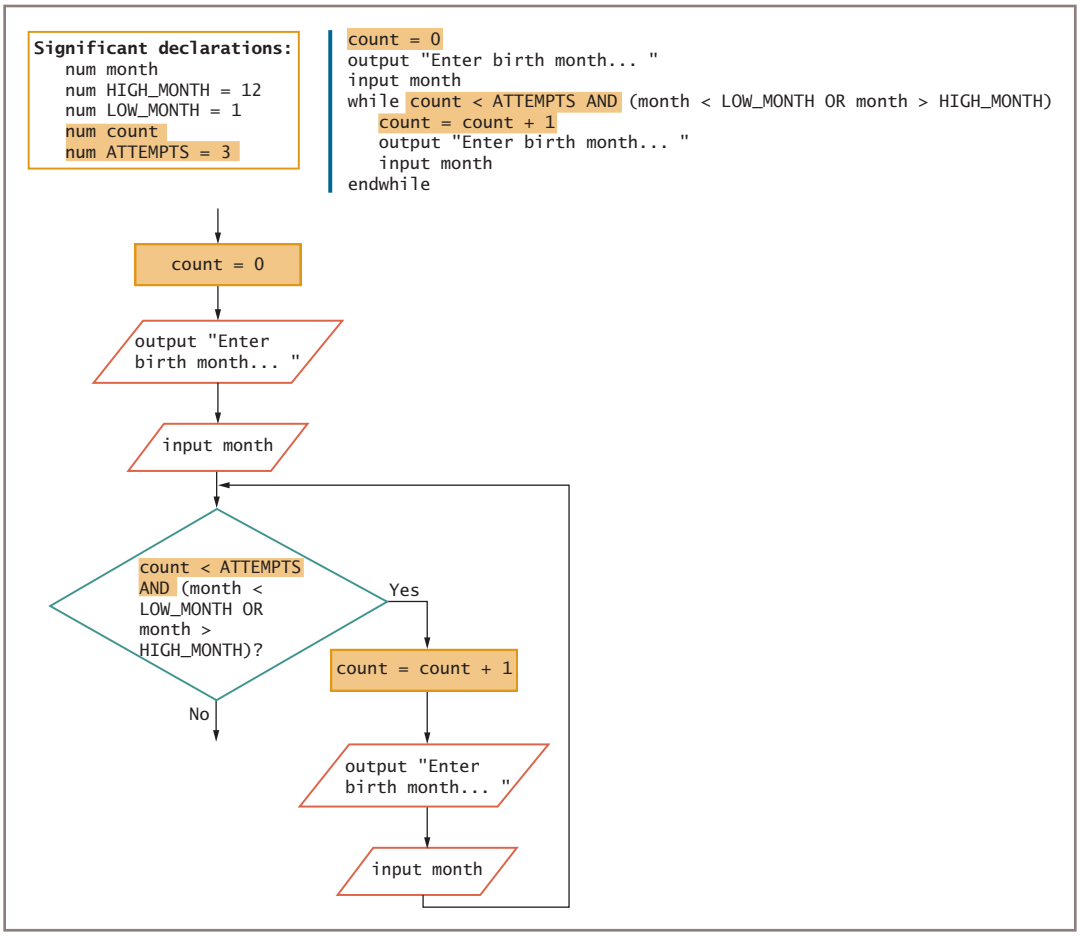


Figure 5-25 Limiting user reprompts

Validating a Data Type

The data you use within computer programs is varied. It stands to reason that validating data requires a variety of methods. For example, some programming languages allow you to check data items to make sure they are the correct data type. Although this technique varies from language to language, you often can make a statement like the one shown in Figure 5-26. In this program segment, `isNumeric()` represents a call to a module; it is used to check whether the entered employee `salary` falls within the category of numeric data. You check to ensure that a value is numeric for many reasons—an important one is that only numeric values can be used correctly in arithmetic statements. A module such as `isNumeric()` most often is provided with the language translator you use to write your programs. Such a module operates as a black box; in other words, you can use the module's results without understanding its internal statements.

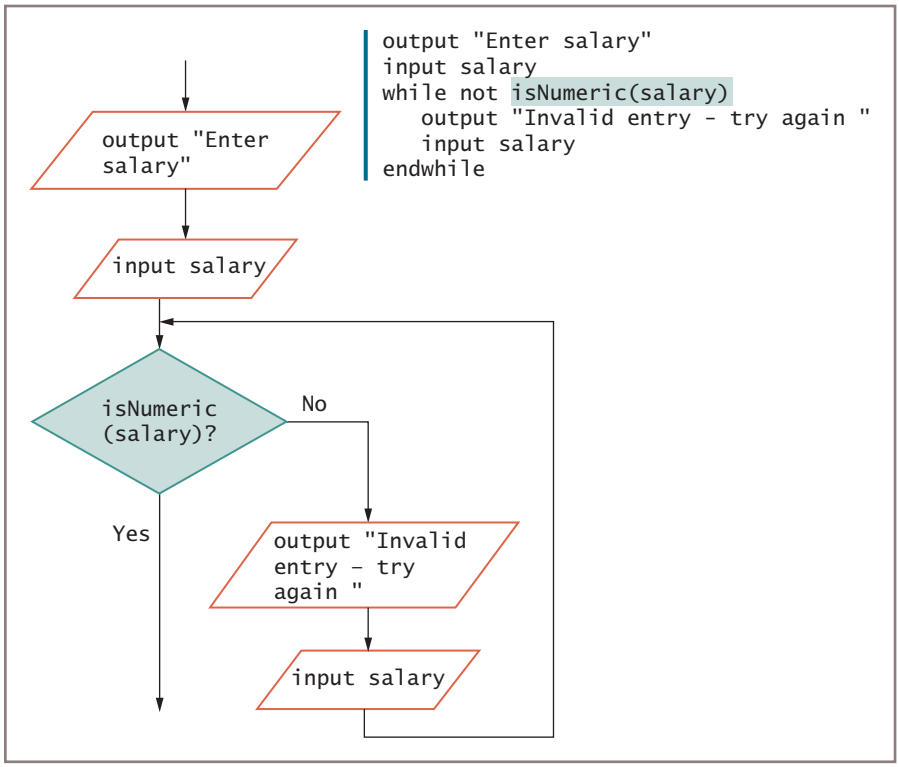


Figure 5-26 Checking data for correct type

Besides allowing you to check whether a value is numeric, some languages contain methods such as `isChar`(), which checks whether a value is a character data type; `isWhitespace`(), which checks whether a value is a nonprinting (whitespace) character, such as a space or tab; and `isUpper`(), which checks whether a value is a capital letter.

In many languages, you accept all user data as a string of characters, and then use built-in methods to attempt to convert the characters to the correct data type for your application. When the conversion methods succeed, you have useful data. When the conversion methods fail because the user has entered the wrong data type, you can take appropriate action, such as issuing an error message, reprompting the user, or forcing the data to a default value.

Validating Reasonableness and Consistency of Data

Data items can be the correct type and within range, but still be incorrect. You have experienced this problem yourself if anyone has ever misspelled your name or overbilled you. The data might have been the correct type—for example, alphabetic letters were used in your name—but the name itself was incorrect. Many data items cannot be checked for reasonableness; for example, the names *Catherine*, *Katherine*, and *Kathryn* are equally reasonable, but only one spelling is correct for a particular woman.

However, many data items can be checked for reasonableness. If you make a purchase on October 3, 2018, then the payment cannot possibly be due prior to that date. Perhaps within your organization, you cannot make more than $20 per hour if you work in Department 12. If your zip code is 90201, your state of residence cannot be New York. If your store's cash on hand was $3,000 when it closed on Tuesday, the amount should not be different when the store opens on Wednesday. If a customer's title is *Ms.*, the customer's gender should be *F*. Each of these examples involves comparing two data items for reasonableness or consistency. You should consider making as many such comparisons as possible when writing your own programs.

Frequently, testing for reasonableness and consistency involves using additional data files. For example, to check that a user has entered a valid county of residence for a state, you might use a file that contains every county name within every state in the United States, and check the user's county against those contained in the file. Good defensive programs try to foresee all possible inconsistencies and errors. The more accurate your data, the more useful information you will produce as output from your programs.



When you become a professional programmer, you want your programs to work correctly as a source of professional pride. On a more basic level, you do not want to be called in to work at 3 a.m. when the overnight run of your program fails because of errors you created.

TWO TRUTHS & A LIE

Common Loop Applications

1. An accumulator is a variable that you use to gather or accumulate values.
2. An accumulator typically is initialized to 0.
3. An accumulator is typically reset to 0 after it is output.

The false statement is #3. There is typically no need to reset an accumulator after it is output.

Comparing Selections and Loops

New programmers sometimes struggle when determining whether to use a selection or a loop to solve some programming problems. Much of the confusion occurs because selections and loops both start by testing conditions and continue by taking action based on the outcome of the test.

An important difference between a selection and a loop, however, is that in the selection structure, the two logical paths that emerge from testing the condition join together following their actions. In the loop structure, the paths that emerge from the test do not join together. Instead, with a loop, one of the logical branches that emerges from the structure-controlling decision eventually returns to the same test. Figure 5-27 compares flowcharts for a selection structure and a loop structure.

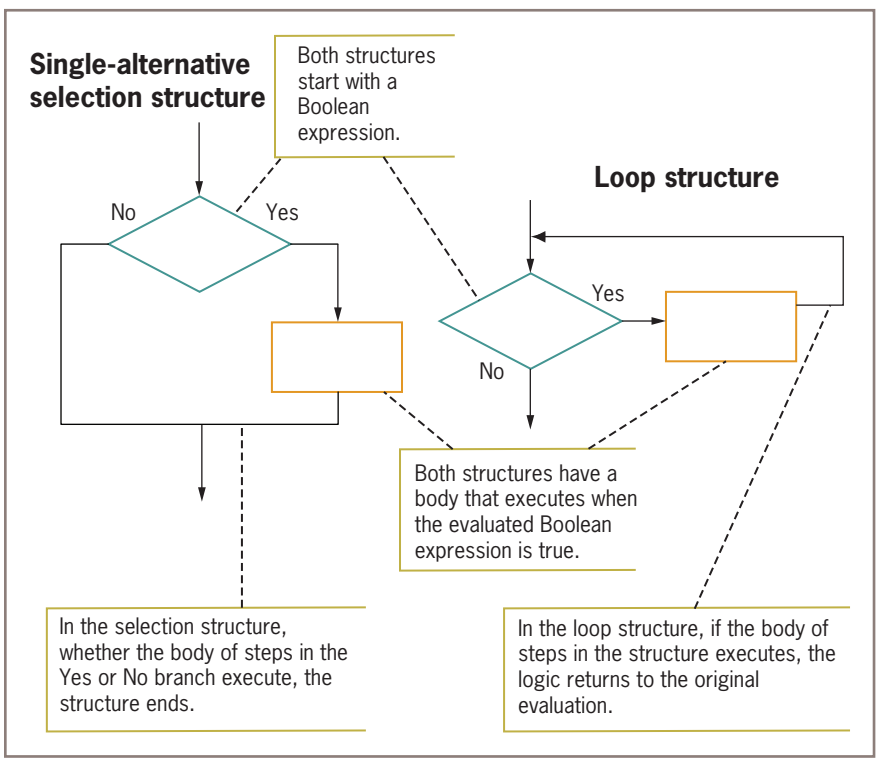


Figure 5-27 Comparing a selection and a loop

When a client describes a programming need, you can listen for certain words to help you decide whether to use a selection or a loop structure. For example, when program requirements contain words like *if, else, unless,* and *otherwise*, the necessary logic might be a selection structure. On the other hand, when program requirements contain words like *while, until, as long as, during, for each, repeat,* and *continue*, the necessary logic might include a loop. As you learned with AND and OR logic in the previous chapter, however, clients do not always use language as precisely as computers, so listening for such keywords is only a guideline.

When you find yourself repeating selection structures that are very similar, you should consider using a loop. For example, suppose your supervisor says, "If you don't reach the end of the file when you are reading an employee record, display the record and read another one. Keep doing this until you run out of records." Because the supervisor used the word *if* to start the request, your first inclination might be to create logic that looks like Figure 5-28.

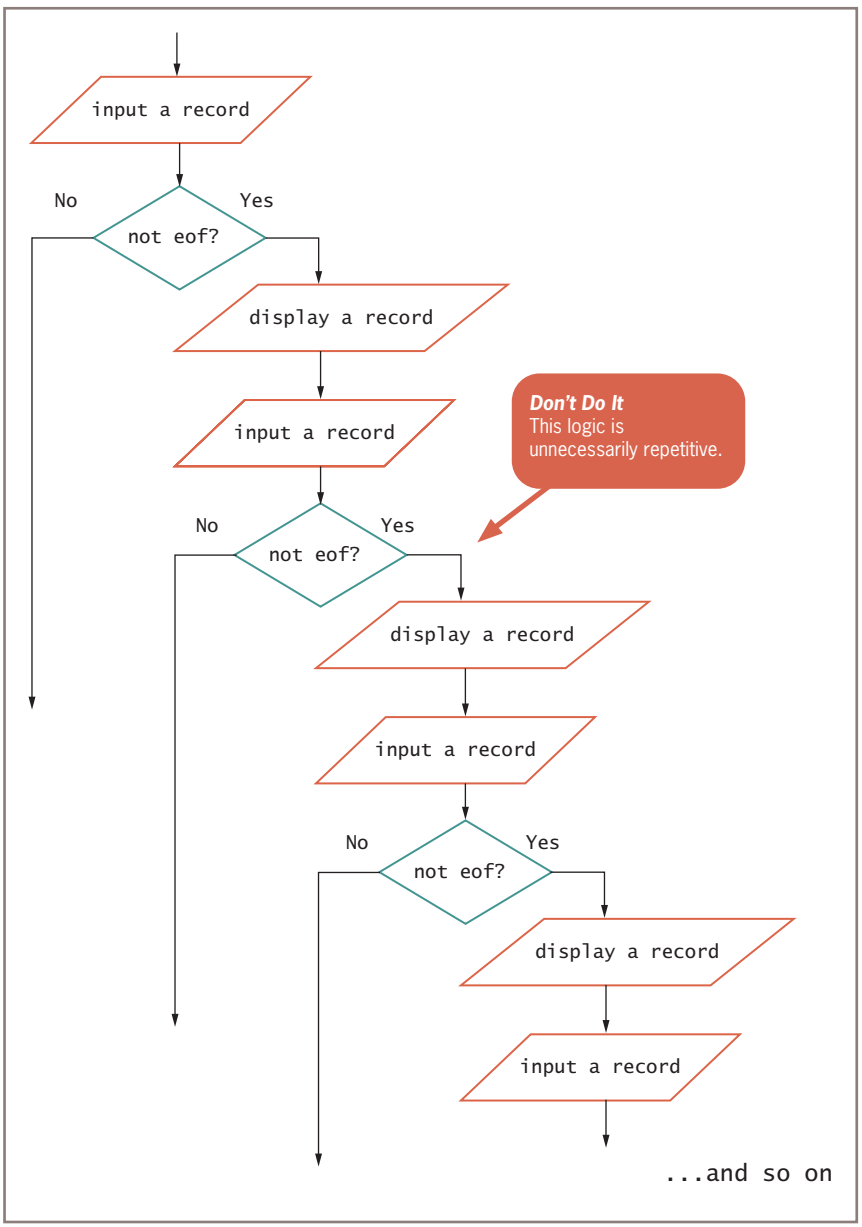


Figure 5-28 Inefficient logic for reading and displaying employee records

The logic in Figure 5-28 works—each time an employee record is input and the `eof` condition is not met, the data is displayed and another record is input. However, the logic in Figure 5-28 is flawed. First, if you do not know how many input records there are, the logic might never end. After each record is input, it is always necessary to check again for the `eof` condition. Second, even if you do know the number of input records, the logic becomes very unwieldy after three or four records. When you examine Figure 5-28, you see that the same set of steps is repeated. Actions that are repeated are best handled in a loop. Figure 5-29 shows a better solution to the problem. In Figure 5-29, the first employee record is input, and as long as the `eof` condition is not met, the program continuously displays and reads additional records.

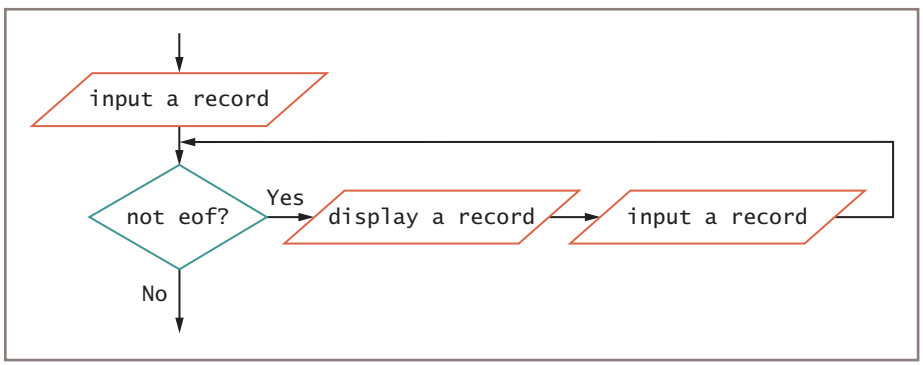


Figure 5-29 Efficient and structured logic for getting and displaying employee records

TWO TRUTHS & A LIE

Comparing Selections and Loops

1. Selection and loop structures differ in that selection structures only take action when a test condition is true.
2. Selection and loop structures are similar in that the tested condition that begins either structure always has two possible outcomes.
3. One difference between selection and loop structures is that the structure-controlling evaluation is repeated in a loop structure.

The false statement is #1. Selection structures can take different actions when a test condition is true and when it is false.

Chapter Summary

- A loop contains one set of instructions that operates on multiple, separate sets of data.
- Three actions are taken with a loop control variable in every `while` loop: You must initialize a loop control variable, compare the variable to some value that controls whether the loop continues or stops, and alter the variable that controls the loop.
- Nested loops are loops that execute within the confines of other loops. When nesting loops, you maintain two separate loop control variables and alter each at the appropriate time.
- Common mistakes that programmers make when writing loops include failing to initialize the loop control variable, neglecting to alter the loop control variable, using the wrong comparison expression with the loop control variable, and including statements inside the loop that belong outside the loop.
- Most computer languages support a `for` statement or `for` loop that you can use with definite loops when you know how many times a loop will repeat. The `for` statement uses a loop control variable that it automatically initializes, tests, and alters.
- In a posttest loop, the loop body executes at least one time because the loop control variable is not tested until after the first iteration.
- In all structured loops, there is exactly one loop-controlling value, and it provides either the only entrance to or the only exit from the loop.
- Loops are used in many applications—for example, to accumulate totals in business reports. Loops also are used to ensure that user data entries are valid by repeatedly reprompting the user.
- In the selection structure, the two logical paths that emerge from a test join together following their actions. In the loop structure, the paths that emerge from the test do not join together; instead, one of the paths eventually returns to the same test.

Key Terms

A **loop control variable** is a variable that determines whether a loop will continue.

A **definite loop** is one for which the number of repetitions is a predetermined value.

A **counted loop**, or **counter-controlled loop**, is a loop whose repetitions are managed by a counter.

To **increment** a variable is to add a constant value to it, frequently 1.

To **decrement** a variable is to decrease it by a constant value, frequently 1.

A **counter** is any numeric variable you use to count the number of times an event has occurred.

An **indefinite loop** is one for which you cannot predetermine the number of executions.

Nested loops occur when a loop structure exists within another loop structure.

An **outer loop** contains another loop when loops are nested.

An **inner loop** is contained within another loop when loops are nested.

A **`for` statement**, or **`for` loop**, can be used to code definite loops and has a loop control variable that it automatically initializes, tests, and alters.

A **step value** is a number by which a loop control variable is altered on each pass through a loop.

A **pretest loop** tests its controlling condition before each iteration, meaning that the loop body might never execute.

A **posttest loop** tests its controlling condition after each iteration, meaning that the loop body executes at least one time.

A **`do-while` loop** is a posttest loop in which the body executes before the loop-controlling condition is tested.

A **`do-until` loop** is a posttest loop that iterates until the loop-controlling condition is false.

An **accumulator** is a variable that you use to gather or accumulate values, such as a running total.

A **summary report** lists only totals, without individual detail records.

Defensive programming is a technique with which you try to prepare for all possible errors before they occur.

To **validate data** is to ensure that data items are meaningful and useful—for example, by ensuring that values are the correct data type, fall within an acceptable range, or are reasonable.

GIGO (garbage in, garbage out) means that if your input is incorrect, your output is worthless.

Forcing a data item means you override incorrect data by setting it to a specific, default value.

Exercises



Review Questions

1. The structure that allows you to write one set of instructions that operates on multiple, separate sets of data is the _____.

   a. sequence
   b. loop
   c. selection
   d. case

2. The loop that frequently appears in a program's mainline logic _____.

   a. always depends on whether a variable equals 0
   b. is an example of an infinite loop
   c. is an unstructured loop
   d. works correctly based on the same logic as other loops

3. Which of the following is *not* a step that must occur with every correctly working loop?

   a. Initialize a loop control variable before the loop starts.
   b. Compare the loop control value to a sentinel during each iteration.
   c. Set the loop control value equal to a sentinel during each iteration.
   d. Allow the loop control variable to be altered during each iteration.

4. The statements executed within a loop are known collectively as the _____.

   a. loop body
   b. loop controls
   c. sequences
   d. sentinels

5. A counter keeps track of _____.

   a. the number of times an event has occurred
   b. the number of machine cycles required by a segment of a program
   c. the number of loop structures within a program
   d. the number of times software has been revised

6. Adding 1 to a variable is also called _____ it.

   a. digesting
   b. resetting
   c. decrementing
   d. incrementing

7. Which of the following is a definite loop?

   a. A loop that executes as long as a user continues to enter valid data
   b. A loop that executes 1,000 times
   c. Both of the above
   d. Neither of the above

8. Which of the following is an indefinite loop?

   a. A loop that executes exactly 10 times
   b. A loop that follows a prompt that asks a user how many repetitions to make and uses the value to control the loop
   c. Both of the above
   d. None of the above

9. When two loops are nested, the loop that is contained by the other is the _____loop.

   a. captive
   b. unstructured
   c. inner
   d. outer

10. When loops are nested, _____.

    a. they typically share a loop control variable
    b. one must end before the other begins
    c. both must be the same type—definite or indefinite
    d. none of the above

11. Most programmers use a `for` loop _____.

    a. for every loop they write
    b. when they know the exact number of times a loop will repeat
    c. when a loop must repeat many times
    d. when a loop will not repeat

12. A report that lists only totals, with no details about individual records, is a(n) _____ report.

    a. accumulator
    b. final
    c. group
    d. summary

13. Typically, the value added to a counter variable is _____.

    a. 0
    b. 1
    c. the same for each iteration
    d. different in each iteration

14. Typically, the value added to an accumulator variable is _____.

    a. 0
    b. 1
    c. the same for each iteration
    d. different in each iteration

15. After an accumulator or counter variable is displayed at the end of a program, it is best to _____.

    a. delete the variable from the program
    b. reset the variable to 0
    c. subtract 1 from the variable
    d. none of the above

16. When you _____, you make sure data items are the correct type and fall within the correct range.

    a. validate data
    b. employ offensive programming
    c. use object orientation
    d. count loop iterations

17. Overriding a user's entered value by setting it to a predetermined value is known as _____.

    a. forcing
    b. accumulating
    c. validating
    d. pushing

18. To ensure that a user's entry is the correct data type, frequently you _____.

    a. prompt the user to verify that the type is correct
    b. use a method built into the programming language
    c. include a statement at the beginning of the program that lists the data types allowed
    d. all of the above

19. A variable might hold an incorrect value even when it is _____.

    a. the correct data type
    b. within a required range
    c. a constant coded by the programmer
    d. all of the above

20. A `do-while` loop _____.

    a. has a body that might never execute
    b. is a type of pretest loop
    c. can be replaced by a sequence and a while loop
    d. is not structured, and therefore obsolete



Programming Exercises

1. What is output by each of the pseudocode segments in Figure 5-30?

2. Design the logic for a program that outputs every number from 1 through 15.

3. Design the logic for a program that outputs every number from 1 through 15 along with its value times 10 and times 100.

4. Design the logic for a program that outputs every even number from 2 through 200.

a.

```
a = 1
b = 2
c = 5
while a < c
   a = a + 1
   b = b + c
endwhile
output a, b, c
```

b.

```
d = 4
e = 6
f = 7
while d > f
   d = d + 1
   e = e - 1
endwhile
output d, e, f
```

c.

```
g = 4
h = 6
while g < h
   g = g + 1
endwhile
output g, h
```

d.

```
j = 2
k = 5
n = 9
while j < k
   m = 6
   while m < n
      output "Goodbye"
      m = m + 1
   endwhile
   j = j + 1
endwhile
```

e.

```
j = 2
k = 5
m = 6
n = 9
while j < k
   while m < n
      output "Hello"
      m = m + 1
   endwhile
   j = j + 1
endwhile
```

f.

```
p = 2
q = 4
while p < q
   output "Adios"
   r = 1
   while r < q
      output "Adios"
      r = r + 1
   endwhile
   p = p + 1
endwhile
```

Figure 5-30 Pseudocode segments for Exercise 1

5. Design the logic for a program that outputs numbers in reverse order from 10 down to 0.

6. Design the logic for a program that allows a user to enter a number. Display the sum of every number from 1 through the entered number.

7. Design the logic for a program that allows a user to continuously enter numbers until the user enters *0*. Display the sum of the numbers entered.

8. Design a program that allows a user to enter any quantity of numbers until a negative number is entered. Then display the highest number and the lowest number.

9. a. Design an application for Bob's E-Z Loans. The application accepts a client's loan amount and monthly payment amount. Output the customer's loan balance each month until the loan is paid off.

   b. Modify the Bob's E-Z Loans application so that after the payment is made each month, a finance charge of 1 percent is added to the balance.

10. a. Design a program for Hunterville College. The current tuition is $20,000 per year, and tuition is expected to increase by 3 percent each year. Display the tuition each year for the next 10 years.

    b. Modify the Hunterville College program so that the user enters the rate of tuition increase instead of having it fixed at 3 percent.

c. Modify the Hunterville College program so that the user enters the rate of tuition increase for the first year. The rate then increases by 0.5 percent each subsequent year.

11. Design a retirement planning calculator for Skulling Financial Services. Allow a user to enter a number of working years remaining in the user's career and the annual amount of money the user can save. Assume that the user earns 3 percent simple interest on savings annually. Program output is a schedule that lists each year number in retirement starting with year 0 and the user's savings at the start of that year. Assume that the user spends $60,000 per year in retirement and then earns 3 percent interest on the remaining balance. End the list after 30 years, or when the user's balance is 0 or less, whichever comes first.

12. a. Design a program for the Hollywood Movie Rating Guide, which can be installed in a kiosk in theaters. Each theater patron enters a value from 0 to 4 indicating the number of stars that the patron awards to the Rating Guide's featured movie of the week. If a user enters a star value that does not fall in the correct range, reprompt the user continuously until a correct value is entered. The program executes continuously until the theater manager enters a negative number to quit. At the end of the program, display the average star rating for the movie.

    b. Modify the movie-rating program so that a user gets three tries to enter a valid rating. After three incorrect entries, the program issues an appropriate message and continues with a new user.

13. Design a program for the Café Noir Coffee Shop to provide some customer market research data. When a customer places an order, a clerk asks for the customer's zip code and age. The clerk enters that data as well as the number of items the customer orders. The program operates continuously until the clerk enters a 0 for zip code at the end of the day. When the clerk enters an invalid zip code (more than 5 digits) or an invalid age (defined as less than 10 or more than 110), the program reprompts the clerk continuously. When the clerk enters fewer than 1 or more than 10 items, the program reprompts the clerk two more times. If the clerk enters a high value on the third attempt, the program accepts the high value, but if the clerk enters a negative value on the third attempt, an error message is displayed and the order is not counted. At the end of the program, display a count of the number of items ordered by customers from the same zip code as the coffee shop (54984), and a count from other zip codes. Also display the average customer age as well as counts of the number of items ordered by customers under 45 and by customers 45 and older.



Performing Maintenance

1. A file named MAINTENANCE05-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 5 include DEBUG05-01.txt, DEBUG05-02.txt, and DEBUG05-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 5 include a file named DEBUG05-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.



Game Zone

1. In Chapter 2, you learned that in many programming languages you can generate a random number between 1 and a limiting value named `LIMIT` by using a statement similar to `randomNumber = random(LIMIT)`. In Chapter 4, you created the logic for a guessing game in which the application generates a random number and the player tries to guess it. Now, create the guessing game itself. After each guess, display a message indicating whether the player's guess was correct, too high, or too low. When the player eventually guesses the correct number, display a count of the number of guesses that were required.

2. Create the logic for a game that simulates rolling two dice by generating two random numbers between 1 and 6 inclusive. The player chooses a number between 2 and 12 (the lowest and highest totals possible for two dice). The player then "rolls" two dice up to three times. If the number chosen by the user comes up, the user wins and the game ends. If the number does not come up within three rolls, the computer wins.

3. Create the logic for the dice game Pig, in which a player can compete with the computer. The object of the game is to be the first to score 100 points. The user and computer take turns "rolling" a pair of dice following these rules:

   - On a turn, each player rolls two dice. If no 1 appears, the dice values are added to a running total for the turn, and the player can choose whether to roll again or pass the turn to the other player. When a player passes, the accumulated turn total is added to the player's game total.
   - If a 1 appears on one of the dice, the player's turn total becomes 0; in other words, nothing more is added to the player's game total for that turn, and it becomes the other player's turn.
   - If a 1 appears on both of the dice, not only is the player's turn over, but also the player's entire accumulated total is reset to 0.
   - When the computer does not roll a 1 and can choose whether to roll again, generate a random value from 1 to 2. The computer then will decide to continue when the value is 1 and decide to quit and pass the turn to the player when the value is not 1.

CHAPTER 6

Arrays

Upon completion of this chapter, you will be able to:

- Store data in arrays
- Appreciate how an array can replace nested decisions
- Use constants with arrays
- Search an array for an exact match
- Use parallel arrays
- Search an array for a range match
- Remain within array bounds
- Use a `for` loop to process an array

Storing Data in Arrays

An **array** is a series or list of values in computer memory. All the values must be the same data type. Usually, all the values in an array have something in common; for example, they might represent a list of employee ID numbers or prices for items sold in a store.

Whenever you require multiple storage locations for objects, you can use a real-life counterpart of a programming array. If you store important papers in a series of file folders and label each folder with a consecutive letter of the alphabet, then you are using the equivalent of an array. If you keep receipts in a stack of shoe boxes and label each box with a month, you are also using the equivalent of an array. Similarly, when you plan courses for the next semester at your school by looking down a list of course offerings, you are using an array.



The arrays discussed in this chapter are single-dimensional arrays, which are similar to lists. Arrays with multiple dimensions are covered in Chapter 8 of the comprehensive version of this book.

Each of these real-life arrays helps you organize objects or information. You *could* store all your papers or receipts in one huge cardboard box, or find courses if they were printed randomly in one large book. However, using an organized storage and display system makes your life easier in each case. Similarly, an array provides an organized storage and display system for a program's data.

How Arrays Occupy Computer Memory

When you declare an array, you declare a structure that contains multiple data items; each data item is one **element** of the array. Each element has the same data type, and each element occupies an area in memory next to, or contiguous to, the others. You can indicate the number of elements an array will hold—the **size of the array**—when you declare the array along with your other variables and constants. For example, you might declare an uninitialized, three-element numeric array named `prices` and an uninitialized string array of 10 employee names as follows:

```
num prices[3]
string employeeNames[10]
```

When naming arrays, programmers follow the same rules as when naming variables. That is, array names must start with a letter and contain no embedded spaces. Additionally, many programmers observe one of the following conventions when naming arrays to make it more obvious that the name represents a group of items:

- Arrays are often named using a plural noun such as `prices` or `employeeNames`.
- Arrays are often named by adding a final word that implies a group, such as `priceList`, `priceTable`, or `priceArray`.

Each array element is differentiated from the others with a unique **subscript**, also called an **index**, which is a number that indicates the position of a particular item within an array. All array elements have the same group name, but each individual element also has a unique subscript indicating how far away it is from the first element. For example, a five-element array uses subscripts 0 through 4, and a ten-element array uses subscripts 0 through 9. In all languages, subscript values must be sequential integers (whole numbers). In most modern languages, such as Visual Basic, Java, C++, and C#, the first array element is accessed using subscript 0, and this book follows that convention.

To use an array element, you place its subscript within square brackets or parentheses (depending on the programming language) after the group name. This book will use square brackets to hold array subscripts so that you don't mistake array names for method names. Many newer programming languages such as C++, Java, and C# also use the square bracket notation.

After you declare an array, you can assign values to some or all of the elements individually. Providing array values sometimes is called **populating the array**. The following code shows a three-element array declaration, followed by three separate statements that populate the array:

```
Declarations
num prices[3]
prices[0] = 25.00
prices[1] = 36.50
prices[2] = 47.99
```

Figure 6-1 shows an array named `prices` that contains three elements, so the elements are `prices[0]`, `prices[1]`, and `prices[2]`. The array elements have been assigned the values 25.00, 36.50, and 47.99, respectively. The element `prices[0]` is zero numbers away from the beginning of the array. The element `prices[1]` is one number away from the beginning of the array, and `prices[2]` is two numbers away.

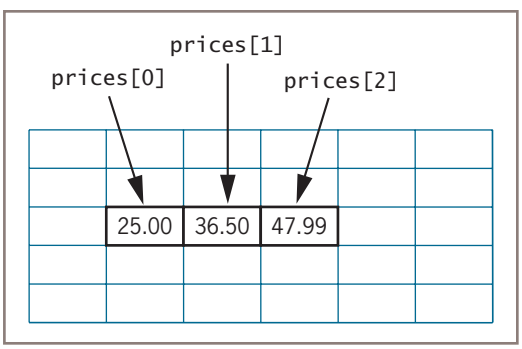


Figure 6-1 Appearance of a three-element array in computer memory



When programmers refer to array element `prices[0]`, they say "`prices` sub 0" or simply "`prices` zero."

If appropriate, you can declare and initialize array elements in one statement. Most programming languages use a statement similar to the following to declare a three-element array and assign a list of values to it:

```
num prices[3] = 25.00, 36.50, 47.99
```



You have learned that you can declare multiple variables of the same data type in a single statement, such as the following:

```
num testScore = 0, age
```

If you want to declare one or more single variables and one or more arrays of the same data type in the same statement, you can achieve better clarity by using notation such as curly braces to set off array element values, as in the following:

```
num prices[3] = { 25.00, 36.50, 47.99 },
   departments[4] = {12, 14, 21, 30}, testScore = 0, age
```

When you use a list of values to initialize an array, the first value you list is assigned to the first array element (element 0), and the subsequent values are assigned to the remaining elements in order. Many programming languages allow you to initialize an array with fewer starting values than there are array elements declared, but no language allows you to initialize an array using more starting values than positions available. When starting values are supplied for an array in this book, each element will be provided with a value.

After an array has been declared and appropriate values have been assigned to specific elements, you can use an individual element in the same way you would use any other data item of the same type. For example, you can assign new values to array elements, you can output the values, and, if the elements are numeric, you can perform arithmetic with them. Quick Reference 6-1 summarizes the characteristics of arrays.

QUICK REFERENCE 6-1 Characteristics of Arrays

- An array is a list of data items in contiguous memory locations.
- Each data item in an array is an *element*.
- Each array element is the same data type; by default, this means that each element is the same size.
- Each element is differentiated from the others by a subscript, which is a whole number.
- Usable subscripts for an array range from 0 to one less than the number of elements in an array.
- Each array element can be used in the same way as a single item of the same data type.



Watch the video *Understanding Arrays*.

TWO TRUTHS & A LIE

Storing Data in Arrays

1. In an array, each element has the same data type.
2. Each array element is accessed using a subscript, which can be a number or a string.
3. Array elements always occupy adjacent memory locations.

The false statement is #2. An array subscript must be a whole number. It can be a named constant, an unnamed constant, or a variable.

How an Array Can Replace Nested Decisions

Consider an application requested by a company's human resources department to produce statistics about employees' claimed dependents. The department wants a report that lists the number of employees who have claimed 0, 1, 2, 3, 4, or 5 dependents. (Assume that you know that no employees have more than five dependents.) For example, Figure 6-2 shows a typical report.

Dependents	Count
0	43
1	35
2	24
3	11
4	5
5	7

Figure 6-2 Typical Dependents report

Without using an array, you could write the application that produces counts for the six categories of dependents (0 through 5) by using a series of decisions. Figure 6-3 shows the pseudocode and flowchart for the decision-making part of such an application. Although this logic works, its length and complexity are unnecessary once you understand how to use an array.



The decision-making process in Figure 6-3 accomplishes its purpose, and the logic is correct, but the process is cumbersome and certainly not recommended. Follow the logic here so that you understand how the application works. In the next pages, you will see how to make the application more elegant.

In Figure 6-3, the variable `dep` is compared to 0. If it is 0, 1 is added to `count0`. If it is not 0, then `dep` is compared to 1. Based on the evaluation, 1 is added to `count1` or `dep` is compared to 2, and so on. Each time the application executes this decision-making process, 1 ultimately is added to one of the six variables that acts as a counter. The dependent-counting logic in Figure 6-3 works, but even with only six categories of dependents, the decision-making process is unwieldy. What if the number of dependents might be any value from 0 to 10, or 0 to 20? With either of these scenarios, the basic logic of the program would remain the same; however, you would need to declare many additional variables to hold the counts, and you would need many additional decisions.

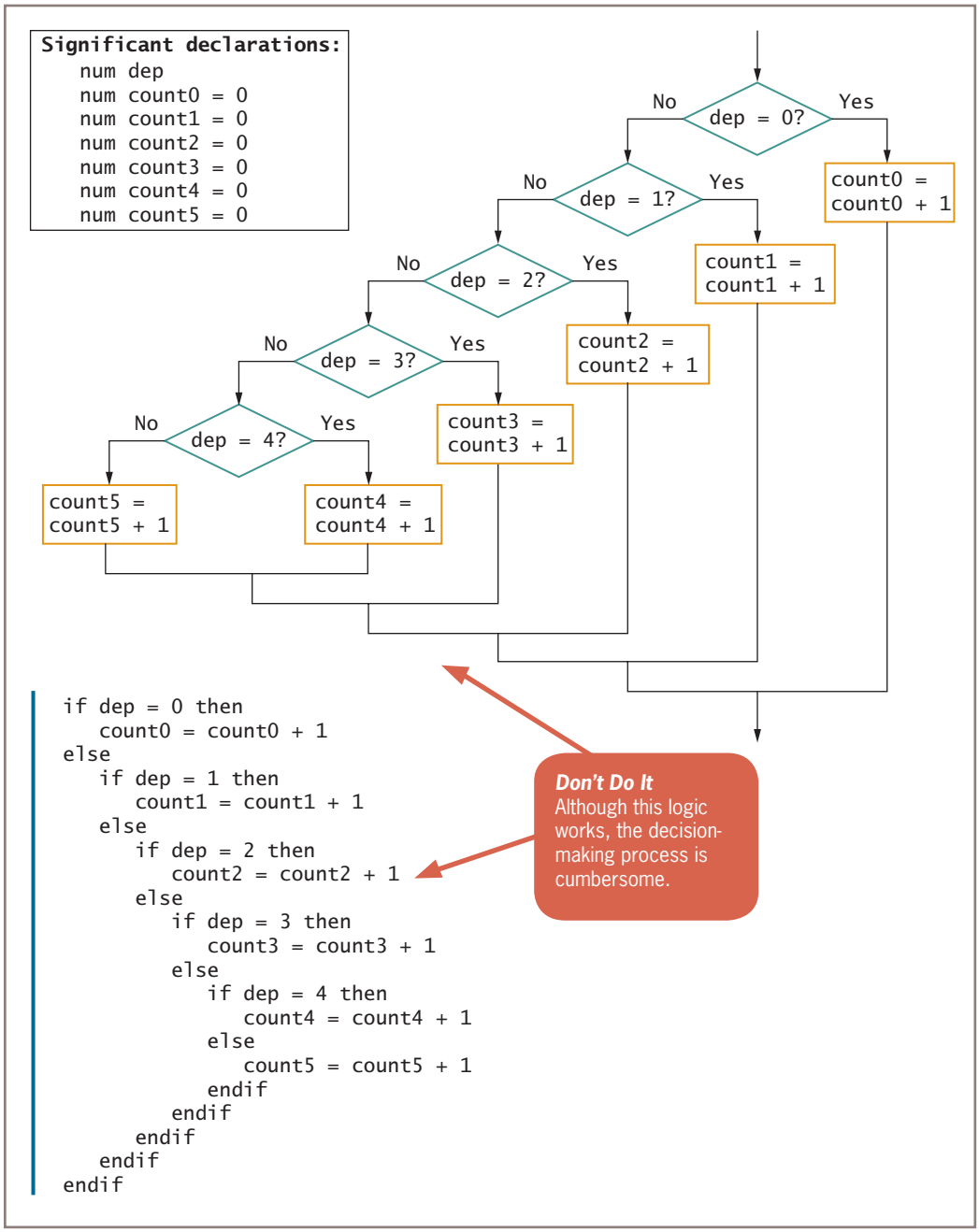


Figure 6-3 Flowchart and pseudocode of decision-making process using a series of decisions—the hard way

Using an array provides an alternate approach to this programming problem and greatly reduces the number of statements you need. When you declare an array, you provide a group name for a number of associated variables in memory. For example, the six dependent count accumulators can be redefined as a single array named `counts`. The individual elements become `counts[0]`, `counts[1]`, `counts[2]`, `counts[3]`, `counts[4]`, and `counts[5]`, as shown in the revised decision-making process in Figure 6-4.

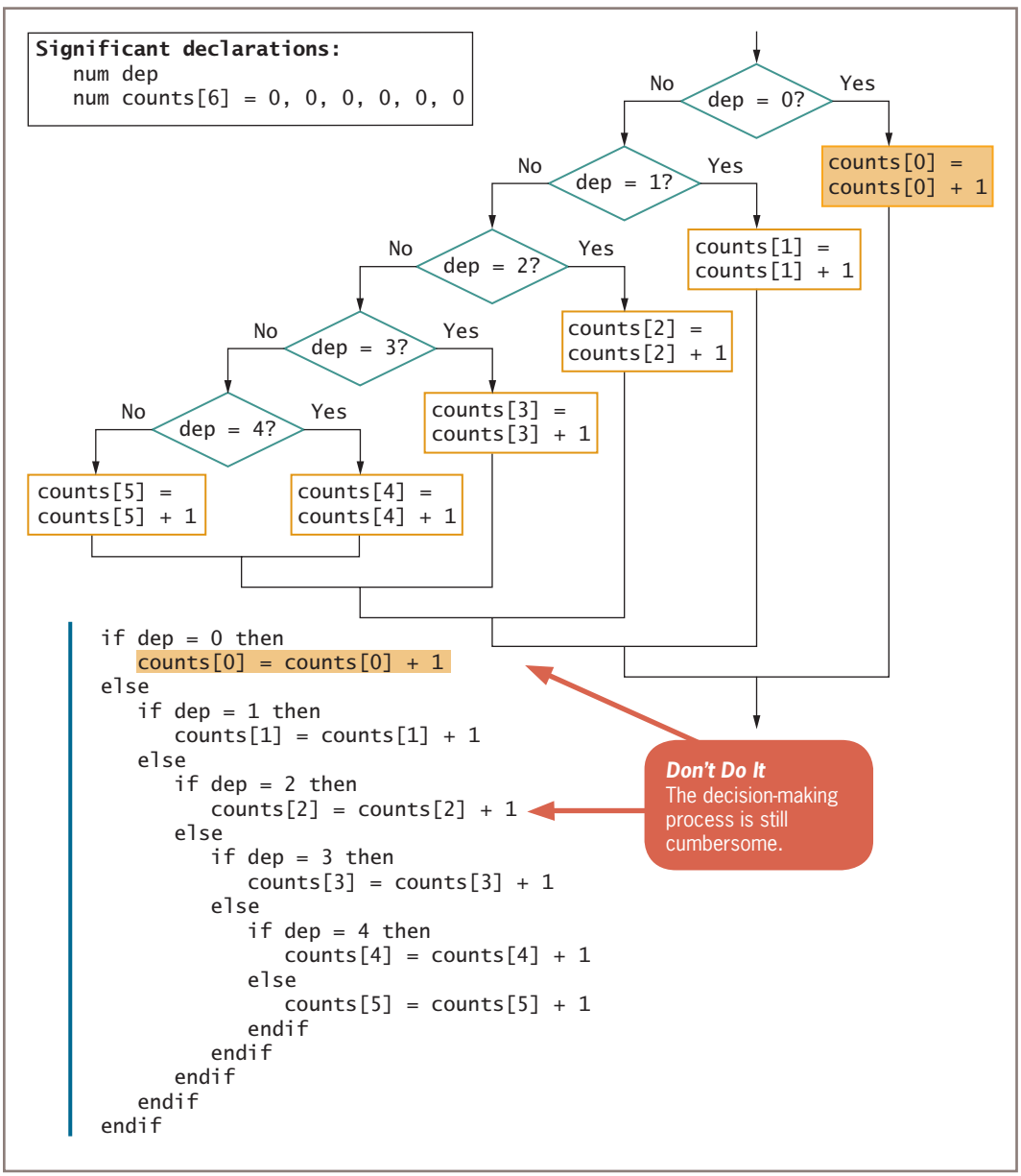


Figure 6-4 Flowchart and pseudocode of decision-making process—but still the hard way

The shaded statement in Figure 6-4 shows that when dep is 0, 1 is added to counts[0]. You can see similar statements for the rest of the counts array elements; when dep is 1, 1 is added to counts[1], when dep is 2, 1 is added to counts[2], and so on. When the dep value is 5, this means it was not 1, 2, 3, or 4, so 1 is added to counts[5]. In other words, 1 is added to one of the elements of the counts array instead of to an individual variable named count0, count1, count2, count3, count4, or count5. Is this version a big improvement over the original in Figure 6-3? Of course it isn't. You still have not taken advantage of the benefits of using the array in this application.

The true benefit of using an array lies in your ability to use a variable as a subscript to the array, instead of using a literal constant such as 0 or 5. Notice in the logic in Figure 6-4 that within each decision, the value compared to dep and the constant that is the subscript in the resulting *Yes* process are always identical. That is, when dep is 0, the subscript used to add 1 to the counts array is 0; when dep is 1, the subscript used for the counts array is 1, and so on. Therefore, you can just use dep as a subscript to the array. You can rewrite the decision-making process as shown in Figure 6-5.

The code segment in Figure 6-5 looks no more efficient than the one in Figure 6-4. However, notice the shaded statements in Figure 6-5—the process that occurs after each decision is exactly the same. In each case, no matter what the value of dep is, you always add 1 to counts[dep]. If you always will take the same action no matter what the answer to a question is, there is no need to ask the question. Instead, you can rewrite the decision-making process as shown in Figure 6-6.

The single statement in Figure 6-6 eliminates the *entire* decision-making process that was the original highlighted section in Figure 6-5! When dep is 2, 1 is added to counts[2]; when dep is 4, 1 is added to counts[4], and so on. *Now* you have significantly improved the original logic. What's more, this process does not change whether there are 20, 30, or any other number of possible categories. To use more than five accumulators, you would declare additional counts elements in the array, but the categorizing logic would remain the same as it is in Figure 6-6.

Figure 6-7 shows an entire program that takes advantage of the array to produce the report that shows counts for dependent categories. Variables and constants are declared and, in the getReady() module, a first value for dep is entered into the program. In the countDependents() module, 1 is added to the appropriate element of the count array and the next value is input. The loop in the mainline logic in Figure 6-7 is an indefinite loop; it continues as long as the user does not enter the sentinel value. When data entry is complete, the finishUp() module displays the report. First, the heading is output, then dep is reset to 0, and then each dep and counts[dep] are output in a loop. The first output statement contains 0 (as the number of dependents) and the value stored in counts[0]. Then, 1 is added to dep and the same set of instructions is used again to display the counts for each number of dependents. The loop in the finishUp() module is a definite loop; it executes precisely six times.

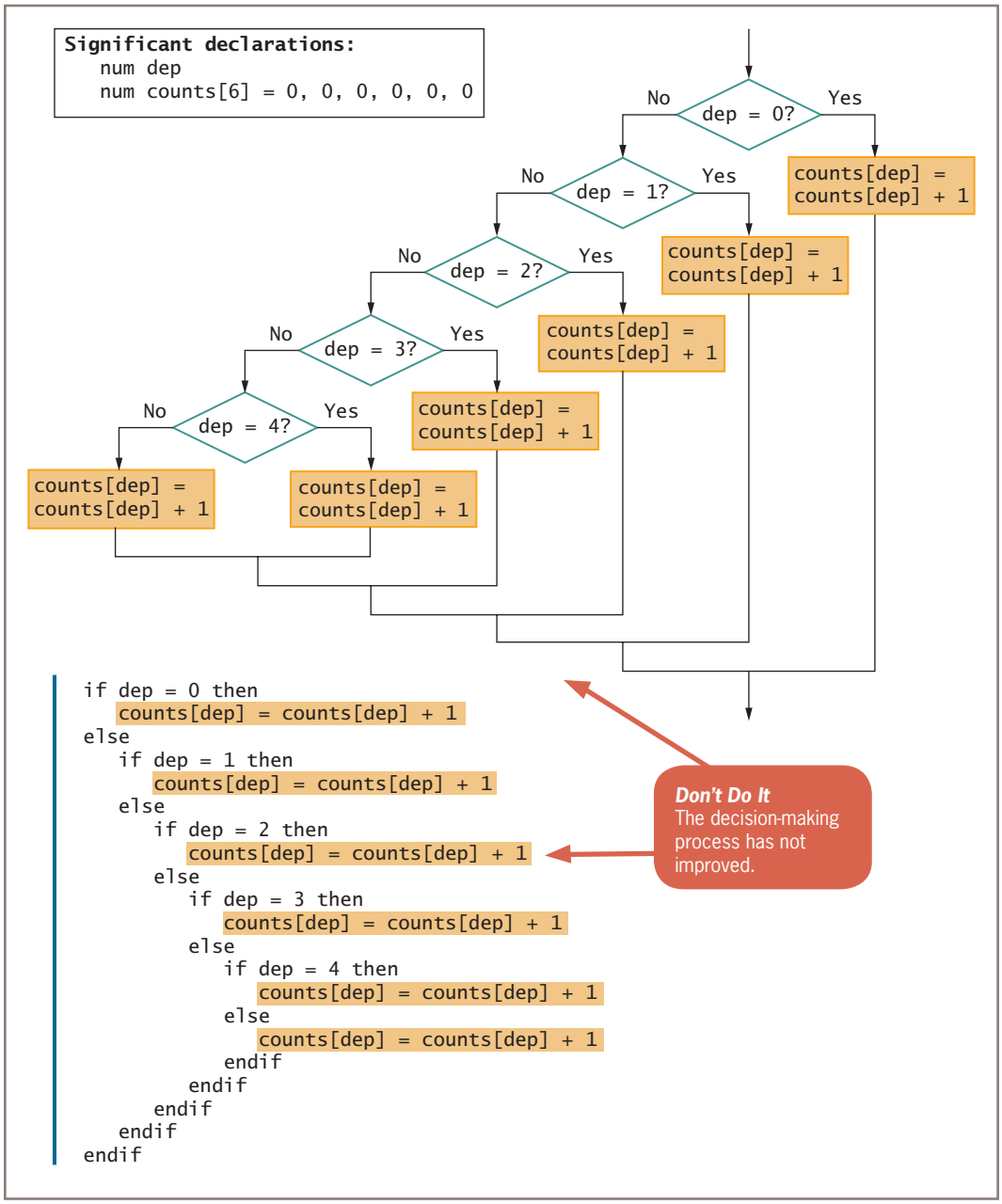


Figure 6-5 Flowchart and pseudocode of decision-making process using an array—but still a hard way

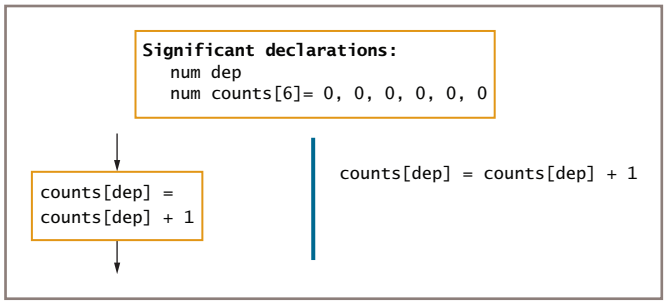


Figure 6-6 Flowchart and pseudocode of efficient decision-making process using an array

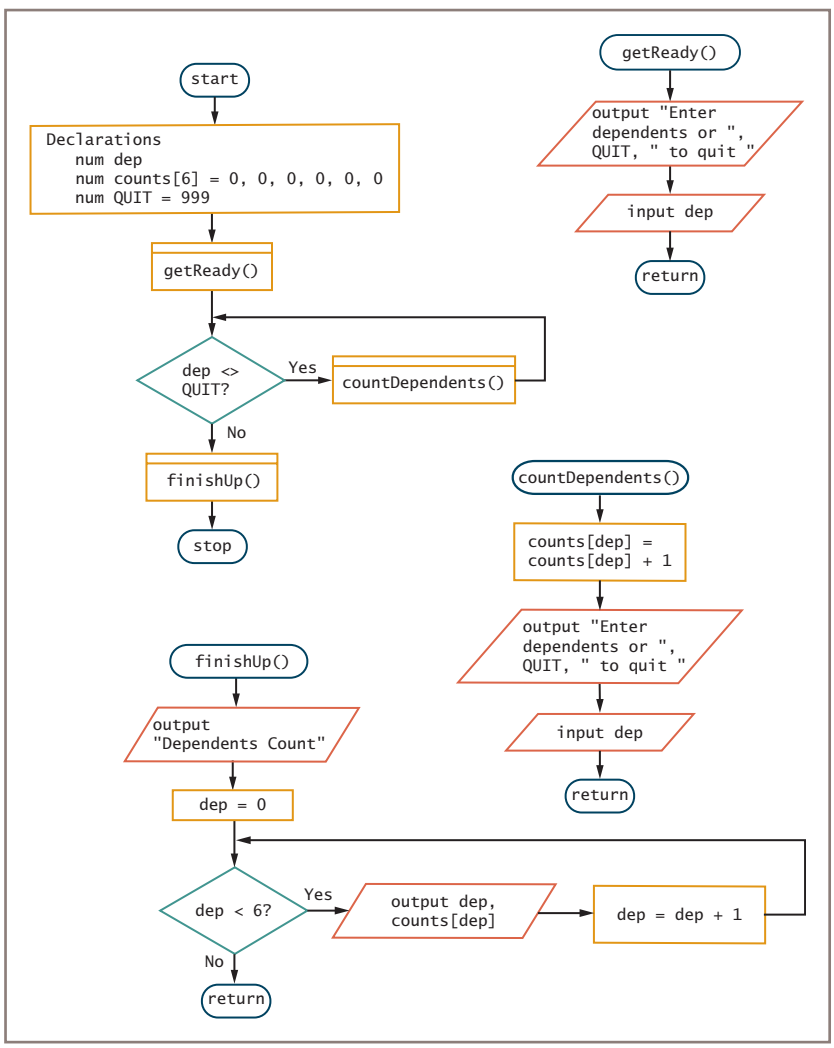


Figure 6-7 Flowchart and pseudocode for Dependents report program *(continues)*

(*continued*)

```
start
   Declarations
      num dep
      num counts[6] = 0, 0, 0, 0, 0, 0
      num QUIT = 999
   getReady()
   while dep <> QUIT
      countDependents()
   endwhile
   finishUp()
stop

getReady()
   output "Enter dependents or ", QUIT, " to quit "
   input dep
return

countDependents()
   counts[dep] = counts[dep] + 1
   output "Enter dependents or ", QUIT, " to quit "
   input dep
return

finishUp()
   output "Dependents Count"
   dep = 0
   while dep < 6
      output dep, counts[dep]
      dep = dep + 1
   endwhile
return
```

Figure 6-7 Flowchart and pseudocode for Dependents report program



The program in Figure 6-7 could be improved by making sure that the value of the subscript `dep` is within range before adding 1 to `counts[dep]`. Later in this chapter, you learn more about ensuring that a subscript falls within the valid range for an array.

The dependent-counting program would have *worked* if it contained a long series of decisions and output statements, but the program is easier to write when you use an array and access its values using the number of dependents as a subscript. Additionally, the new program is more efficient, easier for other programmers to understand, and easier to maintain. Arrays are never mandatory, but often they can drastically cut down on your programming time and make your logic easier to understand.

Learning to use arrays properly can make many programming tasks far more efficient and professional. When you understand how to use arrays, you will be able to provide elegant solutions to problems that otherwise would require tedious programming steps.



Watch the video *Accumulating Values in an Array*.

TWO TRUTHS & A LIE

How an Array Can Replace Nested Decisions

1. You can use an array to replace a long series of decisions.
2. You experience a major benefit of arrays when you use an unnamed numeric constant as a subscript as opposed to using a variable.
3. The process of displaying every element in a 10-element array is basically no different from displaying every element in a 100-element array.

The false statement is #2. You experience a major benefit of arrays when you use a variable as a subscript as opposed to using a constant.

Using Constants with Arrays

In Chapter 2, you learned that named constants hold values that do not change during a program's execution. When working with arrays, you can use constants in several ways:

- To hold the size of an array
- As the array values
- As subscripts

Using a Constant as the Size of an Array

The program in Figure 6-7 still contains one minor flaw. Throughout this book, you have learned to avoid *magic numbers*—that is, unnamed constants. As the totals are output in the loop at the end of the program in Figure 6-7, the array subscript is compared to the constant 6. The program can be improved if you use a named constant instead. Using a named constant makes your code easier to modify and understand. In most programming languages, you can take one of two approaches:

- You can declare a named numeric constant such as `ARRAY_SIZE = 6`. Then you can use this constant in a comparison every time you access the array, always making sure any subscript you use remains less than the constant value.
- In many languages, a value that represents the array size is automatically provided for each array you create. For example, in Java, after you declare an array named `counts`, its size is stored in a field named `counts.length`. In both C# and Visual Basic, the array size is `counts.Length`, with an uppercase *L*. No automatically created value exists in C or C++.

Using Constants as Array Element Values

Sometimes the values stored in arrays should be constants because they are not changed during program execution. For example, suppose that you create an array that holds names for the months of the year. When declaring an array of named constants, programmers conventionally use all uppercase letters with underscores separating words. Don't confuse the array identifier with its contents—the convention in this book is to use all uppercase letters in constant identifiers, but not necessarily in array values. An array named `MONTHS` that holds constant values might be declared as follows:

```
string MONTHS[12] = "January", "February", "March", "April", "May", "June",
   "July", "August", "September", "October", "November", "December"
```

Using a Constant as an Array Subscript

Occasionally you will want to use an unnamed numeric constant as a subscript to an array. For example, to display the first value in an array named `salesArray`, you might write a statement that uses an unnamed literal constant as a subscript, as follows:

```
output salesArray[0]
```

You also might have occasion to use a named constant as a subscript. For example, if `salesArray` holds sales values for each of 20 states covered by your company, and Indiana is state 5, you could output the value for Indiana using an unnamed constant as follows:

```
output salesArray[5]
```

However, if you declare a named constant as `num INDIANA = 5`, then you can display the same value using this statement:

```
output salesArray[INDIANA]
```

An advantage to using a named constant in this case is that the statement becomes self-documenting—anyone who reads your statement more easily understands that your intention is to display the sales value for Indiana.

TWO TRUTHS & A LIE

Using Constants with Arrays

1. If you create a named constant equal to an array size, you can use it as a subscript to the array.
2. If you create a named constant equal to an array size, you can use it as a limit against which to compare subscript values.
3. When you declare an array in Java, C#, or Visual Basic, a constant that represents the array size is automatically provided.

The false statement is #1. If the constant is equal to the array size, then it is larger than any valid array subscript.

Searching an Array for an Exact Match

In the dependent-counting application in this chapter, the array's subscript variable conveniently held small whole numbers—the number of dependents allowed was 0 through 5—and the dep variable directly accessed the array. Unfortunately, real life doesn't always happen in small integers. Sometimes you don't have a variable that conveniently holds an array position; sometimes you have to search through an array to find a value you need.

Consider a mail-order business in which customers place orders that contain a name, address, item number, and quantity ordered. Assume that the item numbers from which a customer can choose are three-digit numbers, but perhaps they are not consecutively numbered 001 through 999. For example, let's say that you offer six items: 106, 108, 307, 405, 457, and 688, as shown in the shaded `VALID_ITEMS` array declaration in Figure 6-8. The array is declared as constant because the item numbers do not change during program

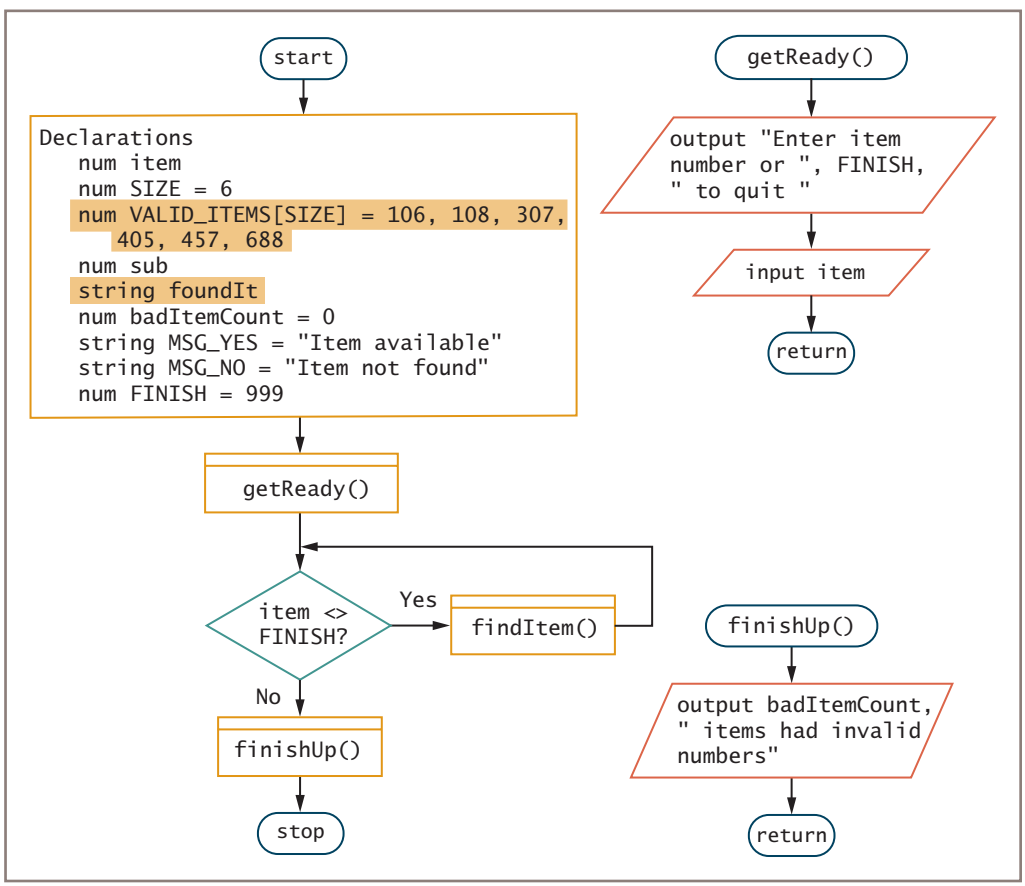


Figure 6-8 Flowchart and pseudocode for program that verifies item availability *(continues)*

(*continued*)

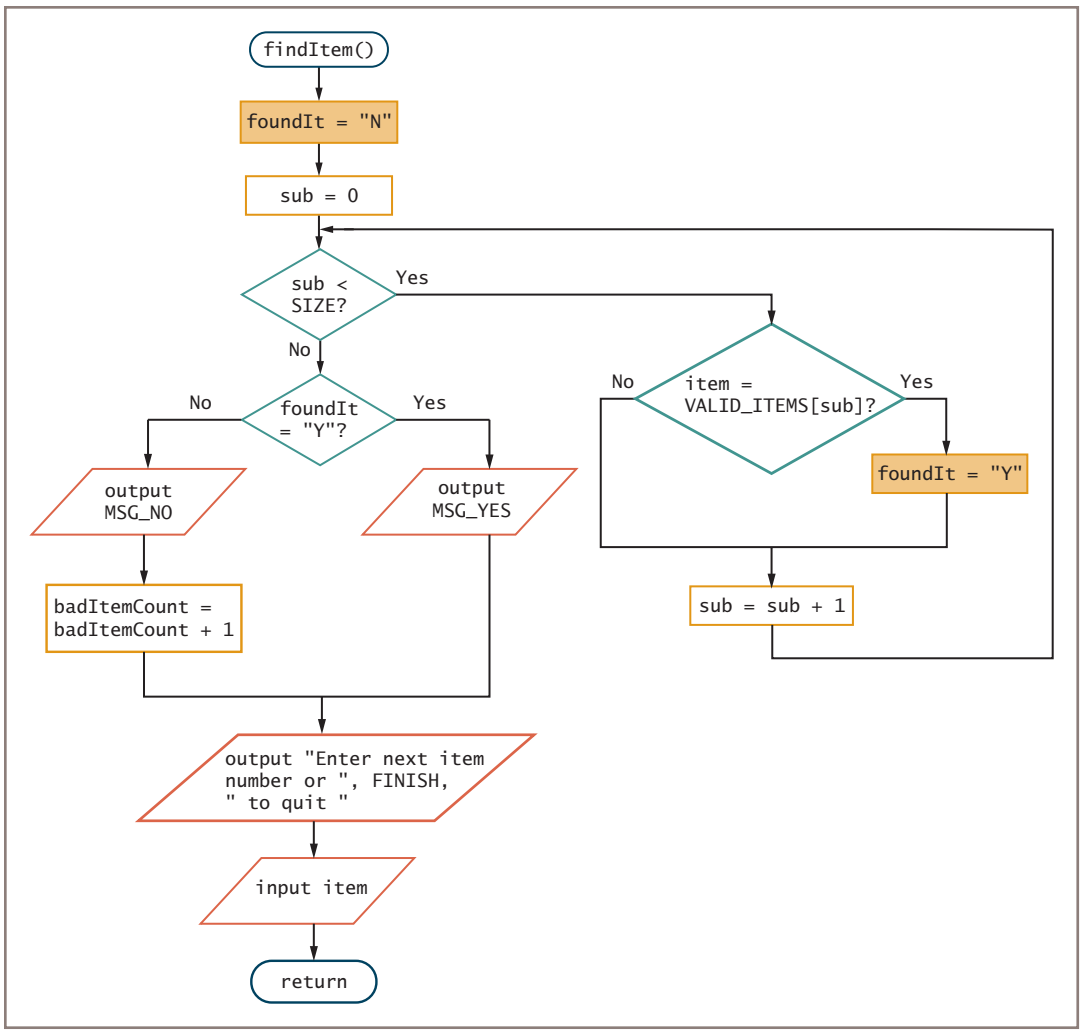


Figure 6-8 Flowchart and pseudocode for program that verifies item availability *(continues)*

execution. When a customer orders an item, a clerical worker can tell whether the order is valid by looking down the list and manually verifying that the ordered item number is on it. In a similar fashion, a computer program can use a loop to test the ordered item number against each `VALID_ITEMS` element, looking for an exact match. When you search through a list from one end to the other, you are performing a **linear search**.

To determine if an ordered item number is valid, you could use a series of six decisions to compare the number to each of the six allowed values. However, the superior approach

(*continued*)

```
start
   Declarations
      num item
      num SIZE = 6
      num VALID_ITEMS[SIZE] = 106, 108, 307,
         405, 457, 688
      num sub
      string foundIt
      num badItemCount = 0
      string MSG_YES = "Item available"
      string MSG_NO = "Item not found"
      num FINISH = 999
   getReady()
   while item <> FINISH
      findItem()
   endwhile
   finishUp()
stop

getReady()
   output "Enter item number or ", FINISH, " to quit "
   input item
return

findItem()
   foundIt = "N"
   sub = 0
   while sub < SIZE
      if item = VALID_ITEMS[sub] then
         foundIt = "Y"
      endif
      sub = sub + 1
   endwhile
   if foundIt = "Y" then
      output MSG_YES
   else
      output MSG_NO
      badItemCount = badItemCount + 1
   endif
   output "Enter next item number or ", FINISH, " to quit "
   input item
return

finishUp()
   output badItemCount, " items had invalid numbers"
return
```

Figure 6-8 Flowchart and pseudocode for program that verifies item availability

shown in Figure 6-8 is to create an array that holds the list of valid item numbers and then to search through the array for an exact match to the ordered item. If you search through the entire array without finding a match for the item the customer ordered, it means the ordered item number is not valid.

The `findItem()` module in Figure 6-8 takes the following steps to verify that an item number exists:

- A flag variable named `foundIt` is set to `"N"`. A **flag** is a variable that is set to indicate whether some event has occurred. In this example, *N* indicates that the item number has not yet been found in the list. (See the first shaded statement in the `findItem()` method in Figure 6-8.)
- A subscript, `sub`, is set to 0. This subscript will be used to access each `VALID_ITEMS` element.
- A loop executes, varying `sub` from 0 through one less than the size of the array. Within the loop, the customer's ordered item number is compared to each item number in the array. If the customer-ordered item matches any item in the array, the flag variable is assigned `"Y"`. (See the last shaded statement in the `findItem()` method in Figure 6-8.) After all six valid item numbers have been compared to the ordered item, if the customer item matches none of them, then the flag variable `foundIt` will still hold the value `"N"`.
- If the flag variable's value is `"Y"` after the entire list has been searched, it means that the item is valid and an appropriate message is displayed, but if the flag has not been assigned `"Y"`, the item was not found in the array of valid items. In this case, an error message is output and 1 is added to a count of bad item numbers.



As an alternative to using the string `foundIt` variable that holds `"Y"` or `"N"` in the method in Figure 6-8, you might prefer to use a numeric variable that you set to 1 or 0. Most programming languages also support a Boolean data type that you can use for `foundIt`; when you declare a variable to be Boolean, you can set its value to `true` or `false`.

TWO TRUTHS & A LIE

Searching an Array for an Exact Match

1. Only whole numbers can be stored in arrays.
2. Only whole numbers can be used as array subscripts.
3. A flag is a variable that indicates whether some event has occurred.

The false statement is #1. Whole numbers can be stored in arrays, but so can many other objects, including strings and numbers with decimal places.

Using Parallel Arrays

When you accept an item number into a mail-order company program, you usually want to accomplish more than simply verifying the item's existence. For example, you might want to determine the name, price, or available quantity of the ordered item. Tasks like these can be completed efficiently using parallel arrays. **Parallel arrays** are two or more arrays in which each element in one array is associated with the element in the same relative position in the other array. Although any array can contain just one data type, each array in a set of parallel arrays might be a different type.

Suppose that you have a list of item numbers and their associated prices. One array named `VALID_ITEMS` contains six elements; each element is a valid item number. Its parallel array also has six elements. The array is named `VALID_PRICES`; each element is a price of an item. Each price in the `VALID_PRICES` array is conveniently and purposely stored in the same position as the corresponding item number in the `VALID_ITEMS` array. Figure 6-9 shows how the parallel arrays might look in computer memory.

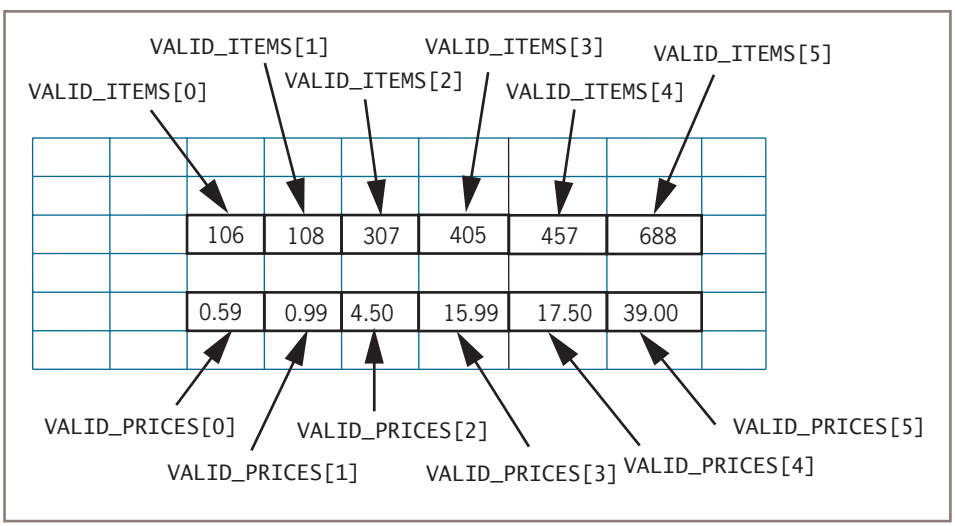


Figure 6-9 Parallel arrays in memory

When you use parallel arrays:

- Two or more arrays contain related data.
- A subscript relates the arrays. That is, elements at the same position in each array are logically related.

Figure 6-10 shows a program that declares parallel arrays. The `VALID_PRICES` array is shaded; each element in it corresponds to a valid item number.

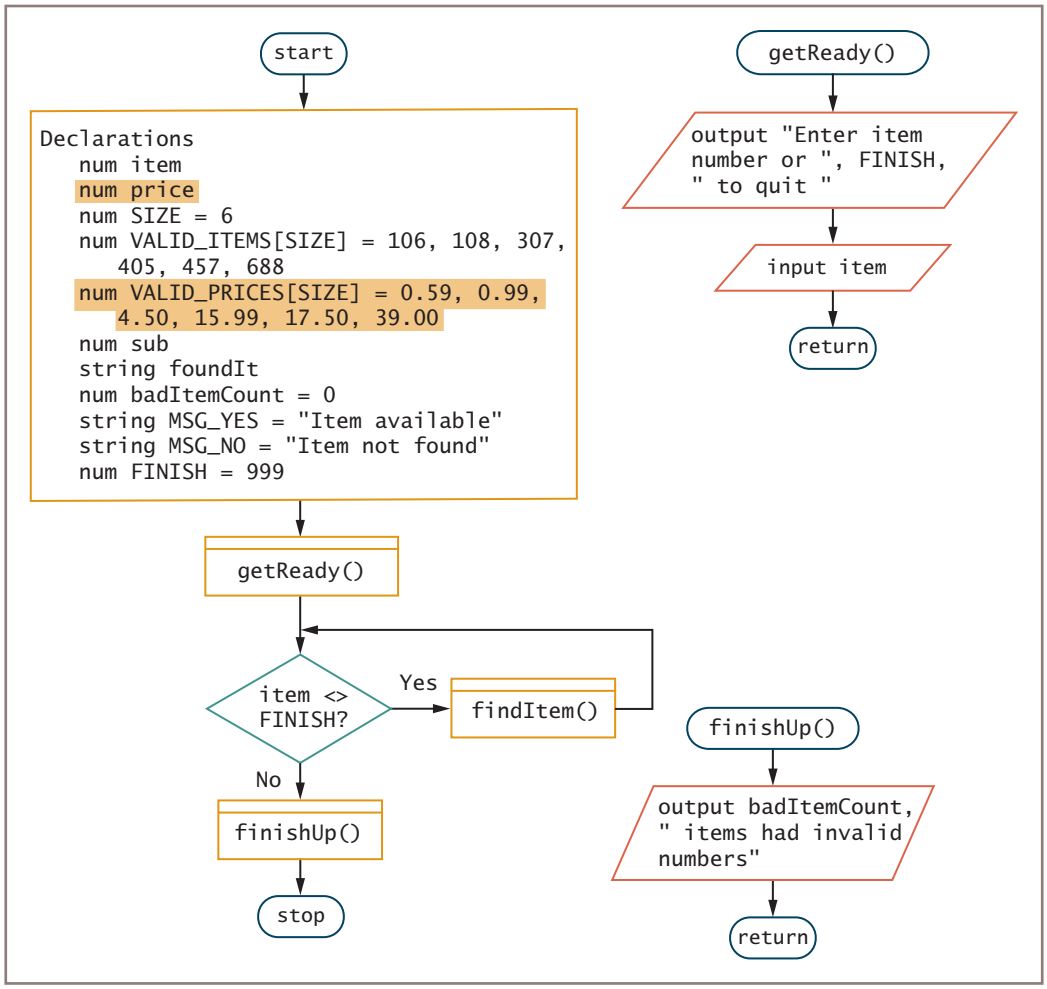


Figure 6-10 Flowchart and pseudocode of program that finds an item price using parallel arrays *(continues)*



Some programmers object to using a cryptic variable name for a subscript, such as `sub` in Figure 6-10, because such names are not descriptive. These programmers would prefer a name like `priceIndex`. Others approve of short names when the variable is used only in a limited area of a program, as it is used here, to step through an array. Programmers disagree about many style issues such as this one. As a programmer, it is your responsibility to find out what conventions are used among your peers in an organization.

As the program in Figure 6-10 receives a customer's order, it looks through each of the `VALID_ITEMS` values separately by varying the subscript `sub` from 0 to the number of items available. When a match for the item number is found, the program pulls the corresponding parallel price out of the list of `VALID_PRICES` values and stores it in the `price` variable. (See shaded statements in Figure 6-10.)

(continued)

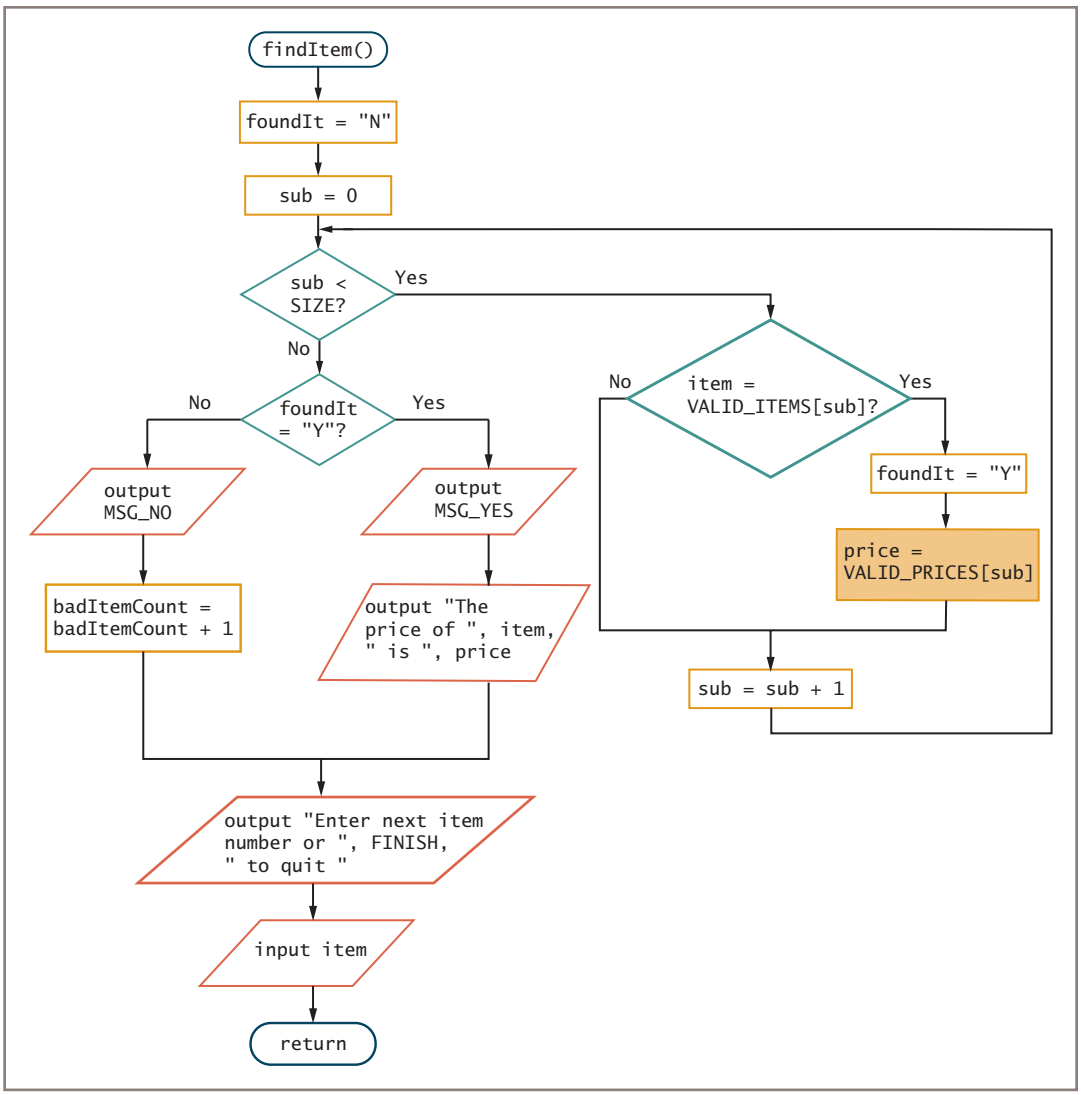


Figure 6-10 Flowchart and pseudocode of program that finds an item price using parallel arrays *(continues)*

(*continued*)

```
start
   Declarations
      num item
      num price
      num SIZE = 6
      num VALID_ITEMS[SIZE] = 106, 108, 307,
         405, 457, 688
      num VALID_PRICES[SIZE] = 0.59, 0.99,
         4.50, 15.99, 17.50, 39.00
      num sub
      string foundIt
      num badItemCount = 0
      string MSG_YES = "Item available"
      string MSG_NO = "Item not found"
      num FINISH = 999
   getReady()
   while item <> FINISH
      findItem()
   endwhile
   finishUp()
stop

getReady()
   output "Enter item number or ", FINISH, " to quit "
   input item
return

findItem()
   foundIt = "N"
   sub = 0
   while sub < SIZE
      if item = VALID_ITEMS[sub] then
         foundIt = "Y"
         price = VALID_PRICES[sub]
      endif
      sub = sub + 1
   endwhile
   if foundIt = "Y" then
      output MSG_YES
      output "The price of ", item, " is ", price
   else
      output MSG_NO
      badItemCount = badItemCount + 1
   endif
   output "Enter next item number or ", FINISH, " to quit "
   input item
return

finishUp()
   output badItemCount, " items had invalid numbers"
return
```

Figure 6-10 Flowchart and pseudocode of program that finds an item price using parallel arrays

Suppose that a customer orders item 457. Using the flowchart or pseudocode in Figure 6-10, walk through the logic yourself to see if you come up with the correct price per item, $17.50. Then, suppose that a customer orders item 458. Walk through the logic and see whether the appropriate *Item not found* message is displayed. The relationship between an item's number and its price is an **indirect relationship**. That means you don't access a price directly by knowing the item number. Instead, you determine the price by knowing an item number's array position. Once you find a match for the ordered item number in the `VALID_ITEMS` array, you know that the price of the item is in the same position in the other array, `VALID_PRICES`. When `VALID_ITEMS[sub]` is the correct item, `VALID_PRICES[sub]` must be the correct price, so `sub` links the parallel arrays. Parallel arrays are most useful when value pairs have an indirect relationship. If values in your program have a direct relationship, you probably don't need parallel arrays. For example, if items were numbered 0, 1, 2, 3, and so on consecutively, you could use the item number itself as a subscript to the price array instead of using a parallel array to hold item numbers. Even if the items were numbered 200, 201, 202, and so on consecutively, you could subtract a constant value (200) from each and use that as a subscript instead of using a parallel array.

Improving Search Efficiency

The mail-order program in Figure 6-10 is still a little inefficient. When a customer orders item 106 or 108, a match is found on the first or second pass through the loop, and continuing to search provides no further benefit. However, even after a match is made, the program in Figure 6-10 continues searching through the item array until `sub` reaches the value `SIZE`. One way to stop the search when the item has been found and `foundIt` is set to `"Y"` is to change the loop-controlling question. Instead of simply continuing the loop while the number of comparisons does not exceed the highest allowed array subscript, you should continue the loop while the searched item is not found *and* the number of comparisons has not exceeded the maximum. Leaving the loop as soon as a match is found improves the program's efficiency. The larger the array, the more beneficial it becomes to exit the searching loop as soon as you find the desired value.

Figure 6-11 shows the improved version of the `findItem()` module with the altered loop-controlling question shaded.

Notice that the price-finding program offers the greatest efficiency when the most frequently ordered items are stored at the beginning of the array, so that only the seldom-ordered items require many loops before finding a match. For example, if you know that item 457 is ordered more frequently than any other item, you can place that item number first in the `VALID_ITEMS` array, and place its price (17.50) first in the `VALID_PRICES` array. That way, the most common scenario is that an item's price is found after only one loop iteration, and the loop can be exited. Often, you can improve search efficiency by rearranging array elements.

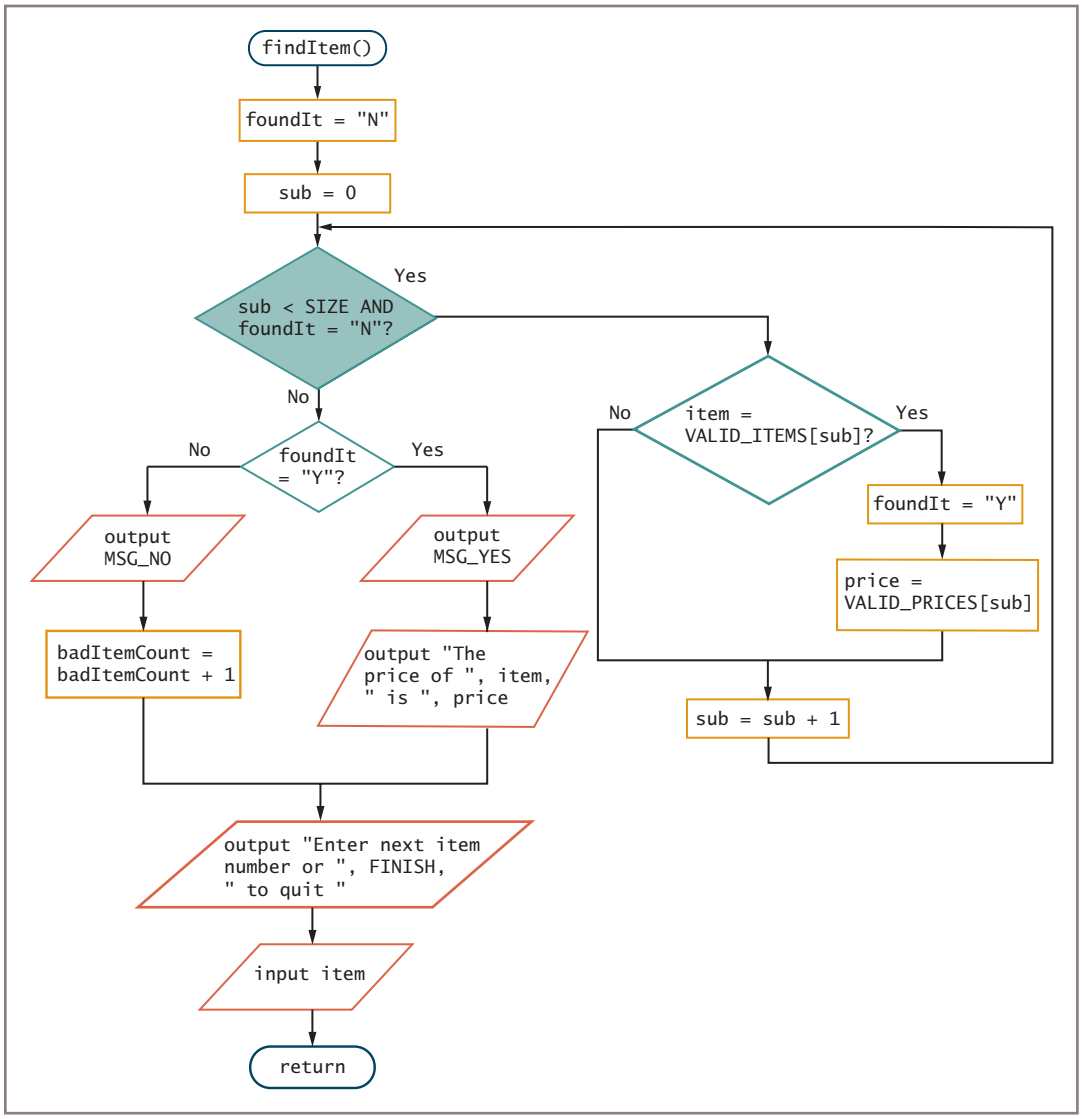


Figure 6-11 Flowchart and pseudocode of the module that finds an item price and exits the loop as soon as it is found (continues)



As you study programming, you will learn other search techniques besides the linear search. For example, with a **binary search** you compare the value you seek with the middle element in a sorted list. If your value is lower, you eliminate the second half of the list, then compare your value to the middle item in the new half-list.



Watch the video *Using Parallel Arrays*.

(*continued*)

```
findItem()
   foundIt = "N"
   sub = 0
   while sub < SIZE AND foundIt = "N"
      if item = VALID_ITEMS[sub] then
         foundIt = "Y"
         price = VALID_PRICES[sub]
      endif
      sub = sub + 1
   endwhile
   if foundIt = "Y" then
      output MSG_YES
      output "The price of ", item, " is ", price
   else
      output MSG_NO
      badItemCount = badItemCount + 1
   endif
   output "Enter next item number or ", FINISH, " to quit "
   input item
return
```

Figure 6-11 Flowchart and pseudocode of the module that finds an item price and exits the loop as soon as it is found

TWO TRUTHS & A LIE

Using Parallel Arrays

1. Parallel arrays must be the same data type.
2. Parallel arrays usually contain the same number of elements.
3. You can improve the efficiency of searching through parallel arrays by using an early exit.

The false statement is #1. Parallel arrays do not need to be the same data type. For example, you might look up a name in a string array to find each person's age in a parallel numeric array.

Searching an Array for a Range Match

In the example in the last section, customer order item numbers needed to match available item numbers exactly to determine the correct price of an item. Sometimes, however, programmers want to work with ranges of values in arrays. In Chapter 4, you learned that a

range of values is any series of values—for example, 1 through 5 or 20 through 30.

Suppose that a company decides to offer quantity discounts when a customer orders multiple items, as shown in Figure 6-12.

Quantity	Discount %
0–8	0
9–12	10
13–25	15
26 or more	20

Figure 6-12 Discounts on orders by quantity

You want to be able to read in customer order data and determine a discount percentage based on the quantity ordered. For example, if a customer has ordered 20 items, you want to be able to output *Your discount is 15 percent*. One ill-advised approach might be to set up an array with as many elements as any customer might ever order, and store the appropriate discount for each possible number, as shown in Figure 6-13. This array is set up to contain the discount for 0 items, 1 item, 2 items, and so on. This approach has at least three drawbacks:

- It requires a very large array that uses a lot of memory.
- You must store the same value repeatedly. For example, each of the first nine elements receives the same value, 0, and each of the next four elements receives the same value, 10.

```
num DISCOUNTS[76]
 = 0, 0, 0, 0, 0, 0, 0, 0, 0,
   0.10, 0.10, 0.10, 0.10,
   0.15, 0.15, 0.15, 0.15, 0.15,
   0.15, 0.15, 0.15, 0.15, 0.15,
   0.15, 0.15, 0.15,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20,
   0.20, 0.20, 0.20, 0.20, 0.20
```

Don't Do It
Although this array correctly lists discounts for each quantity, it is repetitious, prone to error, and difficult to use.

Figure 6-13 Usable—but inefficient—discount array

- How do you know you have enough array elements? Is a customer order quantity of 75 items enough? What if a customer orders 100 or 1,000 items? No matter how many elements you place in the array, there's always a chance that a customer will order more.

A better approach is to create two parallel arrays, each with four elements, as shown in Figure 6-14. Each discount rate is listed once in the DISCOUNTS array, and the low end of each quantity range is listed in the QUAN_LIMITS array. To find the correct discount for any customer's ordered quantity, you can start with the *last* quantity range limit (QUAN_LIMITS[3]). If the quantity ordered is at least that value, 26, the loop is never entered and the customer gets the highest discount rate (DISCOUNTS[3], or 20 percent). If the quantity ordered is not at least QUAN_LIMITS[3]—that is, if it is less than 26—then you reduce the subscript and check to see if the quantity is at least QUAN_LIMITS[2], or 13. If so, the customer receives DISCOUNTS[2], or 15 percent, and so on. Figure 6-15 shows a program that accepts a customer's quantity ordered and determines the appropriate discount rate.

```
num DISCOUNTS[4] =   0, 0.10, 0.15, 0.20
num QUAN_LIMITS[4] = 0,    9,   13,   26
```

Figure 6-14 Parallel arrays to use for determining discount

An alternate approach to the one taken in Figure 6-15 is to store the high end of every range in an array. Then you start with the *lowest* element and check for values *less than or equal to* each array element value.

When using an array to store range limits, you use a loop to make a series of comparisons that otherwise would require many separate decisions. The program that determines customer discount rates in Figure 6-15 requires fewer instructions than one that does not use an array, and modifications to the program will be easier to make in the future.

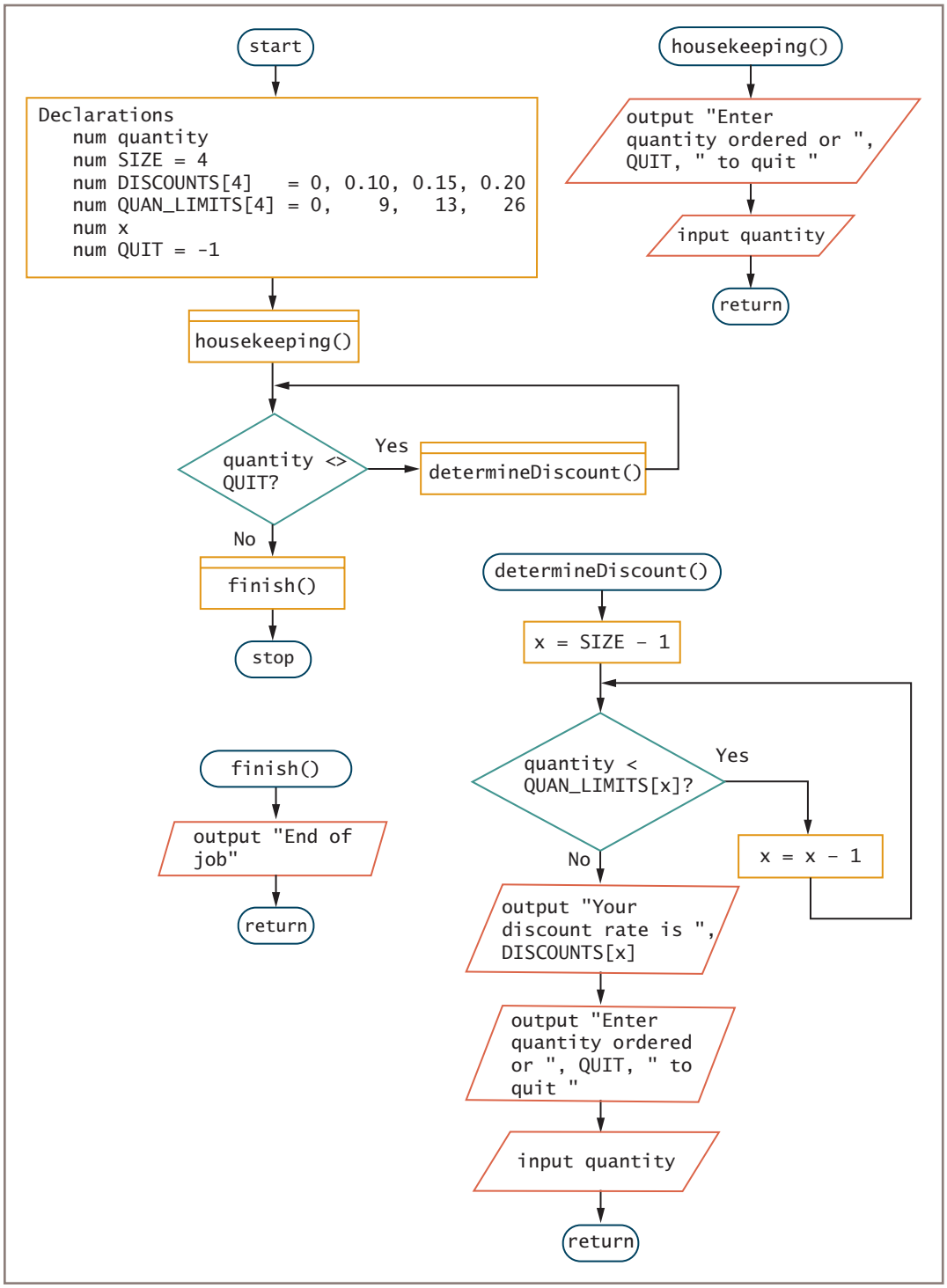


Figure 6-15 Program that determines discount rate *(continues)*

(*continued*)

```
start
   Declarations
      num quantity
      num SIZE = 4
      num DISCOUNTS[4] =   0, 0.10, 0.15, 0.20
      num QUAN_LIMITS[4] = 0,    9,   13,   26
      num x
      num QUIT = -1
   housekeeping()
   while quantity <> QUIT
     determineDiscount()
   endwhile
   finish()
stop

housekeeping()
   output "Enter quantity ordered or ", QUIT, " to quit "
   input quantity
return

determineDiscount()
   x = SIZE - 1
   while quantity < QUAN_LIMITS[x]
      x = x - 1
   endwhile
   output "Your discount rate is ", DISCOUNTS[x]
   output "Enter quantity ordered or ", QUIT, " to quit "
   input quantity
return

finish()
   output "End of job"
return
```

Figure 6-15 Program that determines discount rate

TWO TRUTHS & A LIE

Searching an Array for a Range Match

1. To help locate a range within which a value falls, you can store the highest value in each range in an array.
2. To help locate a range within which a value falls, you can store the lowest value in each range in an array.
3. When using an array to store range limits, you use a series of comparisons that would otherwise require many separate loop structures.

The false statement is #3. When using an array to store range limits, you use a loop to make a series of comparisons that would otherwise require many separate decisions.

Remaining within Array Bounds

To ensure that valid subscripts are used with an array, you must understand two related concepts:

- The array's size
- The bounds of usable subscripts

Understanding Array Size

Every array has a finite size. You can think of an array's size in one of two ways—either by the number of elements in the array or by the number of bytes in the array. Arrays are always composed of elements of the same data type, and elements of the same data type always occupy the same number of bytes of memory, so the number of bytes in an array is always a multiple of the number of elements in an array. For example, in Java, integers occupy 4 bytes of memory, so an array of 10 integers occupies exactly 40 bytes.



For a complete discussion of bytes and how they measure computer memory, read Appendix A.

Understanding Subscript Bounds

In every programming language, when you access data stored in an array you must use a subscript containing a value that accesses memory occupied by the array. If you do, your subscript is **in bounds**; if you do not, your subscript is **out of bounds**.

A subscript accesses an array element using arithmetic. An array name is a memory address and a subscript indicates the value that should be multiplied by the data type size to calculate the subscript's element address. For example, assume that a `prices` array is stored at memory location 4000, as shown in Figure 6-16, and assume that your computer stores numeric variables using four bytes each. As the figure shows, element 0 is at memory location 4000 + 0 * 4, or 4000, element 1 is at memory location 4000 + 1 * 4, or 4004, and element 2 is at memory location 4000 + 2 * 4, or 4008. If you use a subscript that is out of bounds, your program will attempt to access an address that is not part of the array's space.

A common error by beginning programmers is to forget that array subscripts start with 0. If you assume that an array's first subscript is 1, you will always be "off by one" in your array manipulation. For example, if you try to manipulate a 10-element array using subscripts 1 through 10, you will commit two errors: You will fail to access the first element that uses subscript 0, and you will attempt to access an extra element at position 10 when the highest usable subscript is 9.

Examine the program in Figure 6-17. The program accepts a numeric value for `monthNum` and displays the name associated with that month. The logic in Figure 6-17 makes a questionable assumption: that every number entered by the user is a valid month number.

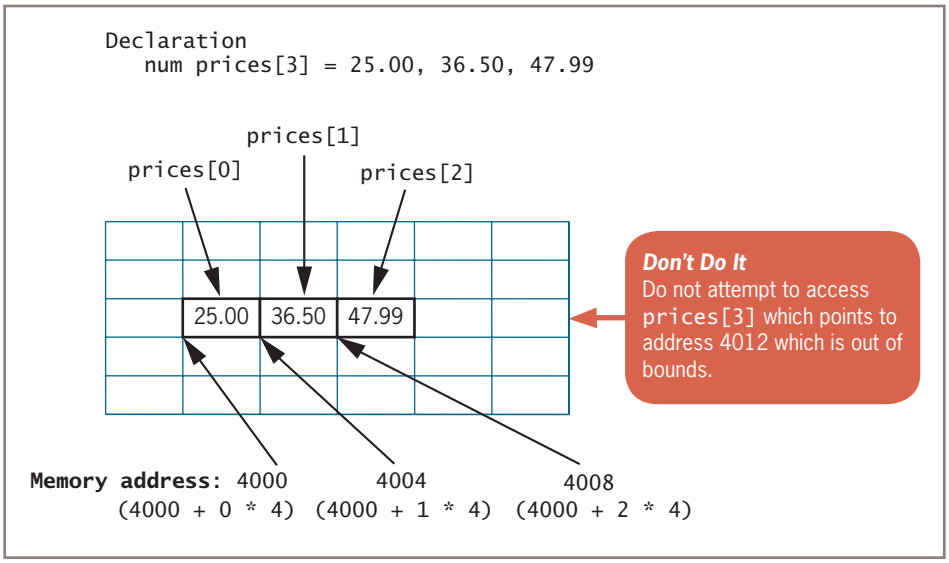


Figure 6-16 An array and its associated memory addresses

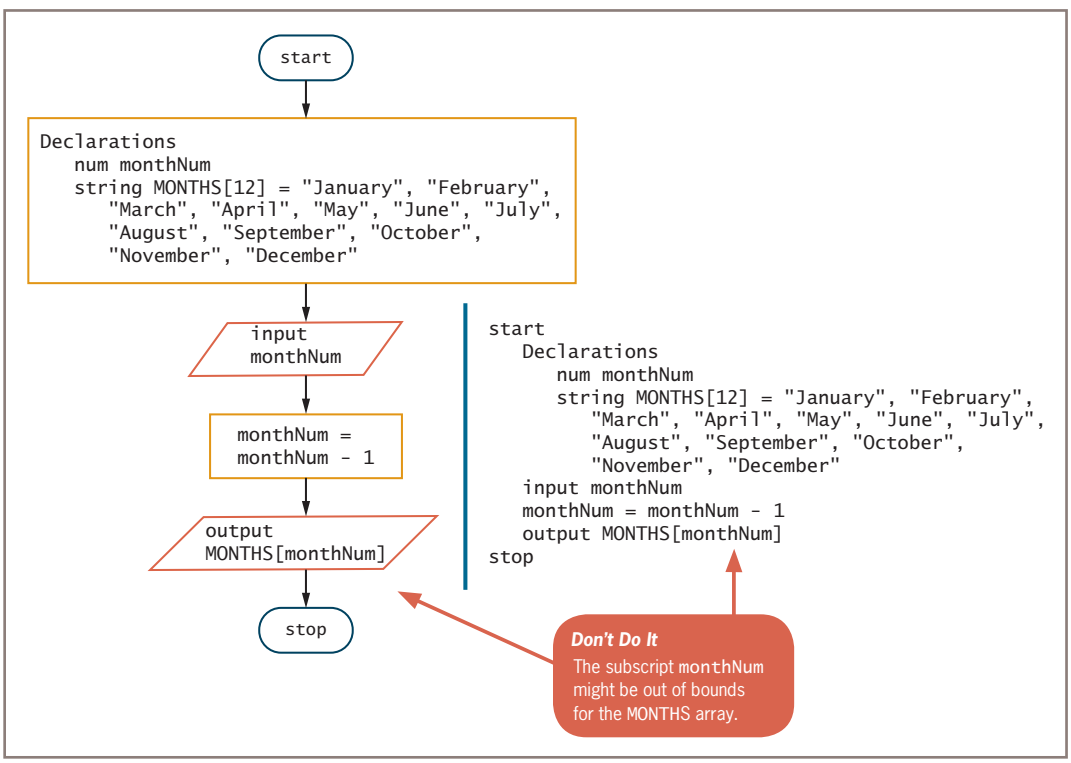


Figure 6-17 Determining the month string from a user's numeric entry



In the program in Figure 6-17, notice that 1 is subtracted from `monthNum` before it is used as a subscript. Although January is the first month in the year, its name occupies the location in the array with the 0 subscript. With values that seem naturally to start with 1, like month numbers, some programmers would prefer to create a 13-element array and simply never use the zero-position element. That way, each "natural" month number would be the correct value to access its data without subtracting. Other programmers dislike wasting memory by creating an extra, unused array element. Although workable programs can be created with or without the extra array element, professional programmers should follow the conventions and preferences of their colleagues and managers.

In Figure 6-17, if the user enters a number that is too small or too large, one of two things will happen depending on the programming language you use. When you use a subscript value that is negative or higher than the highest allowed subscript:

- Some programming languages will stop execution of the program and issue an error message.
- Other programming languages will not issue an error message but will access a value in a memory location that is outside the area occupied by the array. That area might contain garbage, or worse, it accidentally might contain the name of an incorrect month.

Either way, a logical error occurs. Users enter incorrect data frequently; a good program should be able to handle the mistake and not allow the subscript to be out of bounds.



A user might enter an invalid number or might not enter a number at all. In Chapter 5, you learned that many languages have a built-in method with a name like `isNumeric()` that can test for such mistakes.

You can improve the program in Figure 6-17 by adding a test that ensures the subscript used to access the array is within the array bounds. If you find that the input value is not between 1 and 12 inclusive, you might take one of the following approaches:

- Display an error message and end the program.
- Use a default value for the month. For example, when an entered month is invalid, you might want to assume that it is December.
- Continuously reprompt the user for a new value until it is valid.

The way you handle an invalid month depends on the requirements of your program as spelled out by your user, supervisor, or company policy.

TWO TRUTHS & A LIE

Remaining within Array Bounds

1. Elements in an array frequently are different data types, so calculating the amount of memory the array occupies is difficult.
2. If you attempt to access an array with a subscript that is too small, some programming languages will stop execution of the program and issue an error message.
3. If you attempt to access an array with a subscript that is too large, some programming languages access an incorrect memory location outside the array bounds.

The false statement is #1. Array elements are always the same data type, and elements of the same data type always occupy the same number of bytes of memory, so the number of bytes in an array is always a multiple of the number of elements in an array.

Using a `for` Loop to Process an Array

In Chapter 5, you learned about the `for` loop—a loop that, in a single statement, initializes a loop control variable, compares it to a limit, and alters it. The `for` loop is a particularly convenient tool when working with arrays because you frequently need to process every element of an array from beginning to end. As with a `while` loop, when you use a `for` loop, you must be careful to stay within array bounds, remembering that the highest usable array subscript is one less than the size of the array. Figure 6-18 shows a `for` loop that correctly displays all of a company's department names that are stored in an array declared as `DEPTS`. Notice that `dep` is incremented through one less than the number of departments because with a five-item array, the subscripts you can use are 0 through 4.

```
start
   Declarations
      num dep
      num SIZE = 5
      string DEPTS[SIZE] = "Accounting", "Personnel",
         "Technical", "Customer Service", "Marketing"
   for dep = 0 to SIZE - 1 step 1
      output DEPTS[dep]
   endfor
stop
```

Figure 6-18 Pseudocode that uses a `for` loop to display an array of department names

The loop in Figure 6-18 is slightly inefficient because, as it executes five times, the subtraction operation that deducts 1 from `SIZE` occurs each time. Five subtraction operations do not consume much computer power or time, but in a loop that processes thousands or millions of array elements, the program's efficiency would be compromised. Figure 6-19 shows a superior solution. A new constant called `ARRAY_LIMIT` is calculated once, then used repeatedly in the comparison operation to determine when to stop cycling through the array.

```
start
   Declarations
      num dep
      num SIZE = 5
      num ARRAY_LIMIT = SIZE - 1
      string DEPTS[SIZE] = "Accounting", "Personnel",
         "Technical", "Customer Service", "Marketing"
   for dep = 0 to ARRAY_LIMIT step 1
      output DEPTS[dep]
   endfor
stop
```

Figure 6-19 Pseudocode that uses a more efficient `for` loop to output department names

TWO TRUTHS & A LIE

Using a `for` Loop to Process an Array

1. The `for` loop is a particularly convenient tool when working with arrays because initializing, testing, and altering a loop control variable are coded together.
2. You frequently need to process every element of an array from beginning to end in a linear fashion.
3. One advantage to using a `for` loop to process array elements is that you need not be concerned with array bounds.

The false statement is #3. As with a `while` loop, when you use a `for` loop, you must be careful to stay within array bounds.

Chapter Summary

- An array is a named series or list of values in computer memory, all of which have the same data type but are differentiated with subscripts. Each array element occupies an area in memory next to, or contiguous to, the others.
- You often can use a variable as a subscript to an array, which allows you to replace multiple nested decisions with many fewer statements.
- Constants can be used to hold an array's size, to represent its values, or as subscripts. Using named constants can make programs easier to understand and maintain.
- Searching through an array to find a value you need involves initializing a subscript, using a loop to test each array element, and setting a flag when a match is found.
- With parallel arrays, each element in one array is associated with the element in the same relative position in the other array.
- When you need to compare a value to a range of values in an array, you can store either the low- or high-end value of each range for comparison.
- When you access data stored in an array, it is important to use a subscript containing a value that accesses memory within the array bounds.
- The `for` loop is a particularly convenient tool when processing every element of an array sequentially.

Key Terms

An **array** is a series or list of values in computer memory, all of which have the same name but are differentiated with special numbers called subscripts.

An **element** is a single data item in an array.

The **size of the array** is the number of elements it can hold.

A **subscript**, also called an **index** is a number that indicates the position of a particular item within an array.

Populating the array is the act of assigning values to the array elements.

A **linear search** is a search through a list from one end to the other.

A **flag** is a variable that indicates whether some event has occurred.

In **parallel arrays,** each element in one array is associated with the element in the same relative position in the other array(s).

An **indirect relationship** describes the relationship between parallel arrays in which an element in the first array does not directly access its corresponding value in the second array.

A **binary search** is one that compares a target value to a value in the middle of a sorted list, and then determines whether it should continue higher or lower to find the target.

In bounds describes an array subscript that is within the range of acceptable subscripts for its array.

Out of bounds describes an array subscript that is not within the range of acceptable subscripts for its array.

Exercises



Review Questions

1. A subscript is a(n) _____.
   a. element in an array
   b. alternate name for an array
   c. number that represents the highest value stored within an array
   d. number that indicates the position of an array element
2. Each element in an array must have the same _____as the others.
   a. data type
   b. subscript
   c. value
   d. memory location

3. Suppose that you have declared a numeric array named `values` that has 13 elements. Which of the following must be true?

   a. `values[0]` is smaller than `values[1]`
   b. `values[2]` is stored adjacent to `values[4]`
   c. `values[13]` is out of bounds
   d. `values[12]` is the largest value in the array

4. The subscripts of any array are always _____.

   a. integers
   b. fractions
   c. characters
   d. strings of characters

5. Suppose that you have declared a numeric array named `numbers`, and two of its elements are `numbers[1]` and `numbers[4]`. You know that _____.

   a. the two elements hold the same value
   b. the array holds exactly four elements
   c. there are exactly two elements between those two elements
   d. the two elements are at the same memory location

6. Suppose that you have declared a numeric array named numbers, and two of its elements are `numbers[1]` and `numbers[4]`. You know that _____.

   a. `numbers[4]` is larger than `numbers[1]`
   b. the array has at least five elements
   c. the array has been initialized
   d. the two elements are three bytes apart in memory

7. Suppose that you want to write a program that inputs customer data including name, `zipCode`, `balance`, and `regionNum`. At the end of the program, a summary of the number of customers in each of 12 sales regions who owe more than $1,000 each is displayed. The most likely statement during the main processing loop would be _____.

```
a. customerCount[balance] = customerCount[balance] + 1
b. customerCount[regionNum] = customerCount[regionNum] + 1
c. customerCount[regionNum] = regionNum - 1
d. customerCount[balance] = balance + customerCount[regionNum]
```

8. A program contains a seven-element array that holds the names of the days of the week. At the start of the program, you display the day names using a subscript named `dayNum`. You display the same array values again at the end of the program, where you _____ as a subscript to the array.

   a. must use `dayNum`
   b. can use `dayNum`, but can also use another numeric value
   c. must not use `dayNum`
   d. must use a numeric constant instead of a variable

9. Suppose that you have declared an array as follows: `num values[4] = 0, 0, 0, 0`. Which of the following is an allowed operation?

```
a. values[2] = 17
b. input values[0]
c. values[3] = values[0] + 10
```
   d. all of the above

10. Suppose that you have declared an array as follows: `num values[4] = 0, 0, 0, 0`. Which of the following is an allowed operation?

```
a. values[4] = 80
b. values[2] = values[4] - values[0]
c. output values[3]
```
   d. all of the above

11. Filling an array with values during a program's execution is known as_____ the array.

   a. executing
   b. colonizing
   c. populating
   d. declaring

12. A _____is a variable that can be set to indicate whether some event has occurred.

   a. subscript
   b. banner
   c. counter
   d. flag

13. Two arrays in which each element in one array is associated with the element in the same relative position are _____.

   a. cohesive
   b. parallel
   c. hidden
   d. perpendicular

14. In most modern programming languages, the highest subscript you should use with a 12-element array is _____.

    a. 10
    b. 11
    c. 12
    d. 13

15. Parallel arrays _____.

    a. frequently have an indirect relationship
    b. never have an indirect relationship
    c. must be the same data type
    d. must not be the same data type

16. Each element in a seven-element array can hold _____value(s).

    a. one
    b. seven
    c. at least seven
    d. an unlimited number of

17. Suppose that an instructor assigns different letter grades based on points earned on a 10-point quiz, and that all scores have been verified to be between 0 and 10 inclusive. Also suppose that four possible letter grades can be assigned and that 9 or 10 points is an A, 7 or 8 points is a B, 6 points is a C, and 5 or fewer points is an F. To assign letter grades, the most efficient numeric array containing point values would contain _____ elements.

    a. ten
    b. five
    c. four
    d. three

18. When you use a subscript value that is negative or higher than the number of elements in an array, _____.

    a. execution of the program stops and an error message is issued
    b. a value in a memory location that is outside the area occupied by the array will be accessed
    c. a value in a memory location that is outside the area occupied by the array will be accessed, but only if the value is the correct data type
    d. the resulting action depends on the programming language used

19. In every array, a subscript is out of bounds when it is _____.

    a. negative
    b. 0
    c. 1
    d. 999

20. You can access every element of an array using a _____.

    a. `while` loop
    b. `for` loop
    c. either of the above
    d. none of the above

Programming Exercises

1. a. Design the logic for a program that allows a user to enter 20 numbers, then displays them in the reverse order of entry.

   b. Modify the reverse-display program so that the user can enter any amount of numbers up to 20 until a sentinel value is entered.

2. a. Design the logic for a program that allows a user to enter 20 numbers, then displays each number and its difference from the numeric average of the numbers entered.

   b. Modify the program in Exercise 2a so that the user can enter any amount of numbers up to 20 until a sentinel value is entered.

3. a. Design the logic for a program that allows a user to enter 20 numbers, then displays all of the numbers, the largest number, and the smallest.

   b. Modify the program in Exercise 3a so that the user can enter any amount of numbers up to 20 until a sentinel value is entered.

4. Trainers at Tom's Athletic Club are encouraged to enroll new members. Write an application that allows Tom to enter the names of each of his 25 trainers and the number of new members each trainer has enrolled this year. Output is the number of trainers who have enrolled 0 to 5 members, 6 to 12 members, 13 to 20 members, and more than 20 members.

5. a. The Downdog Yoga Studio offers five types of classes, as shown in Table 6-1. Design a program that accepts a number representing a class and then displays the name of the class.

   b. Modify the Downdog Yoga Studio program so that numeric class requests can be entered continuously until a sentinel value is entered. Then display each class number, name, and a count of the number of requests for each class.

Class Number	Class Name
1	Yoga 1
2	Yoga 2
3	Children's Yoga
4	Prenatal Yoga
5	Senior Yoga

Table 6-1 Downdog Yoga Studio classes

6. Search the web to discover the 10 most common user-selected passwords, and store them in an array. Design a program that prompts a user for a password, and continue to prompt the user until the user has not chosen one of the common passwords.

7. The Jumpin' Jive coffee shop charges $2 for a cup of coffee and offers the add-ins shown in Table 6-2.

Product	Price ($)
Whipped cream	0.89
Cinnamon	0.25
Chocolate sauce	0.59
Amaretto	1.50
Irish whiskey	1.75

Table 6-2 Add-in list for Jumpin' Jive coffee shop

Design the logic for an application that allows a user to enter ordered add-ins continuously until a sentinel value is entered. After each item, display its price or the message *Sorry, we do not carry that* as output. After all items have been entered, display the total price for the order.

8. Design the application logic for a company that wants a report containing a breakdown of payroll by department. Input includes each employee's department number, hourly salary, and number of hours worked. The output is a list of the seven departments in the company and the total gross payroll (rate times hours) for each department. The department names are shown in Table 6-3.

Department Number	Department Name
1	Personnel
2	Marketing
3	Manufacturing
4	Computer Services
5	Sales
6	Accounting
7	Shipping

Table 6-3 Department numbers and names

9. Design a program that computes pay for employees. Allow a user to continuously input employees' names until an appropriate sentinel value is entered. Also input each employee's hourly wage and hours worked. Compute each employee's gross pay (hours times rate), withholding tax percentage (based on Table 6-4), withholding tax amount, and net pay (gross pay minus withholding tax). Display all the results for each employee. After the last employee has been entered, display the sum of all the

Weekly Gross Pay ($)	Withholding Percentage (%)
0.00–300.00	10
300.01–550.00	13
550.01–800.00	16
800.01 and up	20

Table 6-4 Withholding percentage based on gross pay

hours worked, the total gross payroll, the total withholding for all employees, and the total net payroll.

10. a. *Daily Life Magazine* wants an analysis of the demographic characteristics of its readers. The marketing department has collected reader survey records containing the age, gender, marital status, and annual income of readers. Design an application that allows a user to enter reader data and, when data entry is complete, produces a count of readers by age groups as follows: younger than 20, 20–29, 30–39, 40–49, and 50 and older.

    b. Modify the *Daily Life Magazine* program so that it produces a count of readers by gender within age group—that is, under-20 females, under-20 males, and so on.

    c. Modify the *Daily Life Magazine* program so that it produces a count of readers by income groups as follows: under $30,000, $30,000–$49,999, $50,000–$69,999, and $70,000 and up.

11. Glen Ross Vacation Property Sales employs seven salespeople, as shown in Table 6-5. When a salesperson makes a sale, a record is created, including the date, time, and dollar amount of the sale. The time is expressed in hours and minutes, based on a 24-hour clock. The sale amount is expressed in whole dollars. Salespeople earn a commission that differs for each sale, based on the rate schedule in Table 6-6.

ID Number	Salesperson Name
103	Darwin
104	Kratz
201	Shulstad
319	Fortune
367	Wickert
388	Miller
435	Vick

Table 6-5 Glen Ross salespeople

Sale Amount ($)	Commission Rate (%)
0–50,999	4
51,000–125,999	5
126,000–200,999	6
201,000 and up	7

Table 6-6 Glen Ross commission schedule

Design an application that produces each of the following:

    a. A list of each salesperson number, name, total sales, and total commissions

    b. A list of each month of the year as both a number and a word (for example, *01 January*), and the total sales for the month for all salespeople

c. A list of total sales as well as total commissions earned by all salespeople for each of the following time frames, based on hour of the day: 00–05, 06–12, 13–18, and 19–23

12. a. Design an application in which the number of days for each month in the year is stored in an array. (For example, January has 31 days, February has 28, and so on. Assume that the year is not a leap year.) Display 12 sentences in the same format for each month; for example, the sentence displayed for January is *Month 1 has 31 days*.

b. Modify the months and days program to contain a parallel array that stores month names. Display 12 sentences in the same format; for example, the first sentence is *January has 31 days*.

c. Modify the months and days program to prompt the user for a month number and display the corresponding sentence, for example, *January has 31 days*.

d. Prompt a user to enter a birth month and day, and continue to prompt until the day entered is in range for the month. Compute the day's numeric position in the year. (For example, February 2 is day 33.) Then, using parallel arrays, find and display the traditional Zodiac sign for the date. For example, the sign for February 2 is Aquarius.



Performing Maintenance

1. A file named MAINTENANCE06-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 6 include DEBUG06-01.txt, DEBUG06-02.txt, and DEBUG06-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 6 include a file named DEBUG06-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.



Game Zone

1. Create the logic for a Magic 8 Ball game in which the user enters a question such as *What does my future hold?* The computer randomly selects one of eight possible vague answers, such as *It remains to be seen*.

2. Create the logic for an application that contains an array of 10 multiple-choice questions related to your favorite hobby. Each question contains three answer choices. Also create a parallel array that holds the correct answer to each question—A, B, or C. Display each question and verify that the user enters only A, B, or C as the answer—if not, keep prompting the user until a valid response is entered. If the user responds to a question correctly, display *Correct!*; otherwise, display *The correct answer is* and the letter of the correct answer. After the user answers all the questions, display the number of correct and incorrect answers.

3. a. Create the logic for a dice game. The application randomly "throws" five dice for the computer and five dice for the player. After each random throw, store the results in an array. The application displays all the values, which can be

from 1 to 6 inclusive for each die. Decide the winner based on the following hierarchy of die values. Any higher combination beats a lower one; for example, five of a kind beats four of a kind.

- Five of a kind
- Four of a kind
- Three of a kind
- A pair

For this game, the numeric dice values do not count. For example, if both players have three of a kind, it's a tie, no matter what the values of the three dice are. Additionally, the game does not recognize a full house (three of a kind plus two of a kind). Figure 6-20 shows how the game might be played in a command-line environment.

b. Improve the dice game so that when both players have the same number of matching dice, the higher value wins. For example, two 6s beats two 5s.

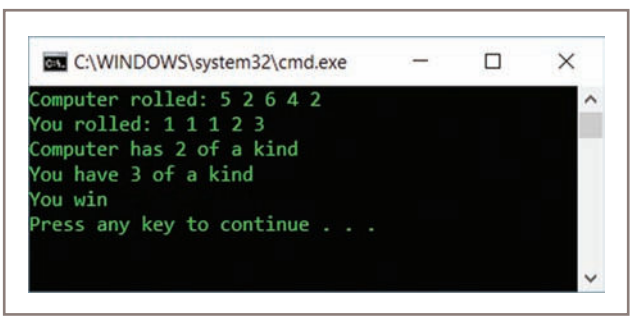


Figure 6-20 Typical execution of the dice game

4. Design the logic for the game Hangman, in which the user guesses letters in a hidden word. Store the letters of a word in an array of characters. Display a dash for each missing letter. Allow the user to continuously guess a letter until all the letters in the word are guessed correctly. As the user enters each guess, display the word again, filling in the guessed letter if it was correct. For example, if the hidden word is *computer*, first display a series of eight dashes: - - - - - - - -. After the user guesses *p*, the display becomes - - -*p*- - - -. Make sure that when a user makes a correct guess, all the matching letters are filled in. For example, if the word is *banana* and the user guesses *a*, all three *a* characters should be filled in.

5. Create two parallel arrays that represent a standard deck of 52 playing cards. One array is numeric and holds the values 1 through 13 (representing Ace, 2 through 10, Jack, Queen, and King). The other array is a string array that holds suits (Clubs, Diamonds, Hearts, and Spades). Create the arrays so that all 52 cards are represented. Then, create a War card game that randomly selects two cards (one for the player and one for the computer) and declares a winner or a tie based on the numeric value of the two cards. The game should last for 26 rounds and use a full deck with no repeated cards. For this game, assume that the lowest card is the Ace. Display the values of the player's and computer's cards, compare their values, and determine the winner. When all the cards in the deck are exhausted, display a count of the number of times the player wins, the number of times the computer wins, and the number of ties. Here are some hints:

   - Start by creating an array of all 52 playing cards.
   - Select a random number for the deck position of the player's first card and assign the card at that array position to the player.
   - Move every higher-positioned card in the deck "down" one to fill in the gap. In other words, if the player's first random number is 49, select the card at position 49 (both the numeric value and the string), move the card that was in position 50 to position 49, and move the card that was in position 51 to position 50. Only 51 cards remain in the deck after the player's first card is dealt, so the available-card array is smaller by one.
   - In the same way, randomly select a card for the computer and "remove" the card from the deck.

CHAPTER 7

File Handling and Applications

Upon completion of this chapter, you will be able to:

- Understand computer files
- Describe the data hierarchy
- Perform file operations
- Describe control break logic
- Merge files
- Describe master and transaction file processing
- Understand random access files

Understanding Computer Files

In Chapter 1, you learned that computer memory, or random access memory (RAM), is volatile, temporary storage. When you write a program that stores a value in a variable, you are using temporary storage; the value you store is lost when the program ends or the computer loses power.

Permanent, nonvolatile storage, on the other hand, is not lost when a computer loses power. Permanent storage is *durable* in the sense that after data items are saved to durable storage, they are available for future use. When you write a program and save it to a disk, you are using a **permanent storage device**. Permanent storage can be a hard drive on your computer or a hard drive on the cloud that you access remotely through the Internet. Other examples include such media as DVDs, USB drives that you insert and remove from your computer, and tape libraries and optical jukeboxes that are accessed by robotic arms.



When discussing computer storage, *temporary* and *permanent* refer to volatility, not length of time. For example, a *temporary* variable might exist for several hours in a very large program or one that runs in an infinite loop, but a *permanent* piece of data might be saved and then deleted by a user within a few seconds. Because you can erase data from files, some programmers prefer the term *persistent storage* to permanent storage. In other words, you can remove data from a file stored on a device such as a disk drive, so it is not technically permanent. However, the data remains in the file even when the computer loses power, so, unlike in RAM, the data persists.

A **computer file** is a collection of data stored on a nonvolatile device in a computer system. Two broad categories of computer files are text files and binary files.

- **Text files** contain data that can be read in a text editor because the data has been encoded using a scheme such as ASCII or Unicode. Text files might include facts and figures used by business programs, such as a payroll file that contains employee numbers, names, and salaries. The programs in this chapter will use text files.
- **Binary files** contain data that has not been encoded as text. Examples include images and music.

Although their contents vary, files have many common characteristics, such as:

- Each has a **filename** that is an identifying name given to a computer file that frequently describes the contents, for example, *JanuaryPayroll* or *PreviousMonthSales*. The name often includes a dot and a **filename extension**, which is a group of characters added to the end of a filename that describes the type of the file. For example, the full name of a file might be *JanuaryPayroll.txt*, in which *.txt* is the filename extension. The extension *.txt* indicates a plain text file, *.dat* is a common extension for a data file, and *.jpg* is used as an extension on image files in Joint Photographic Experts Group format.

A filename's extension frequently designates the software that will be used to open a file; for example, a file that uses the extension *.docx* will usually be opened by Microsoft Word by default.

- Each file has specific times associated with it—for example, its creation time and the time it was last modified.
- Each file occupies space on a section of a storage device; that is, each file has a size. Sizes are measured in bytes. A **byte** is a small unit of storage; for example, in a simple text file, a byte holds only one character. Because a byte is so small, file sizes usually are expressed in **kilobytes** (thousands of bytes), **megabytes** (millions of bytes), **gigabytes** (billions of bytes), or even larger groupings.

Appendix A contains more information about bytes and how file sizes are expressed. Figure 7-1 shows how some files look when you view them in Microsoft Windows.

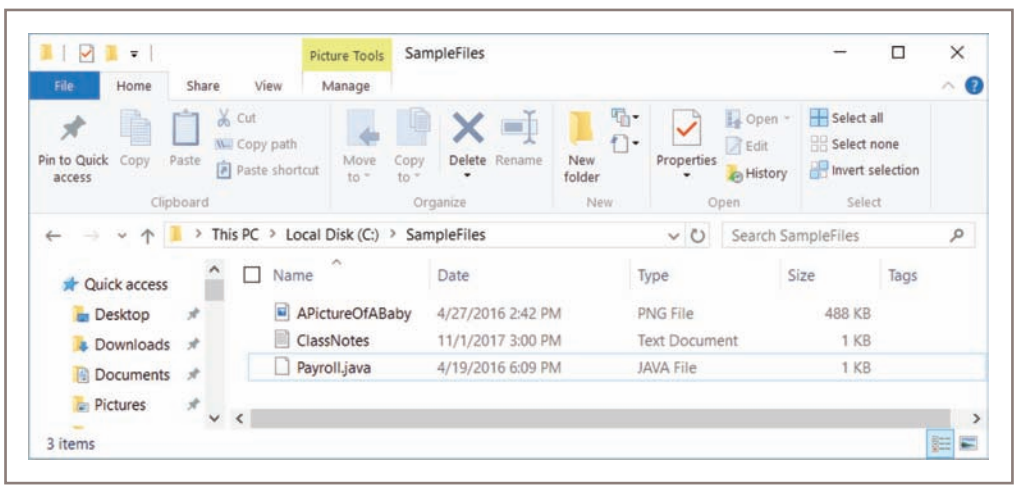


Figure 7-1 Three stored files showing their names, dates of modification, types, and sizes

Watch the video *Understanding Files*.

Organizing Files

Computer files on a storage device are the electronic equivalent of paper files stored in file cabinets. With a paper file, the easiest way to store a document is to toss it into a file cabinet drawer without a folder. However, for better organization, most office clerks place paper documents in folders—and most computer users organize their files into folders or directories. **Directories** and **folders** are organization units on storage devices; each can contain multiple files as well as additional directories. The combination of the disk drive plus the complete hierarchy of directories in which a file resides is its **path**. For example,

in the Windows operating system, the following line would be the complete path for a file named PayrollData.dat on the C drive in a folder named SampleFiles within a folder named Logic:

```
C:\Logic\SampleFiles\PayrollData.dat
```




The terms *directory* and *folder* are used synonymously to mean an entity that organizes files. *Directory* is the more general term; the term *folder* came into use in graphical systems. For example, Microsoft began calling directories *folders* with the introduction of Windows 95.

TWO TRUTHS & A LIE

Understanding Computer Files

1. Temporary storage is volatile.
2. Computer files exist on permanent storage devices, such as RAM.
3. A file's path is the hierarchy of folders in which it is stored.

The false statement is #2. Computer files exist on permanent storage devices, such as hard disks, DVDs, USB drives, and reels of magnetic tape.

Understanding the Data Hierarchy

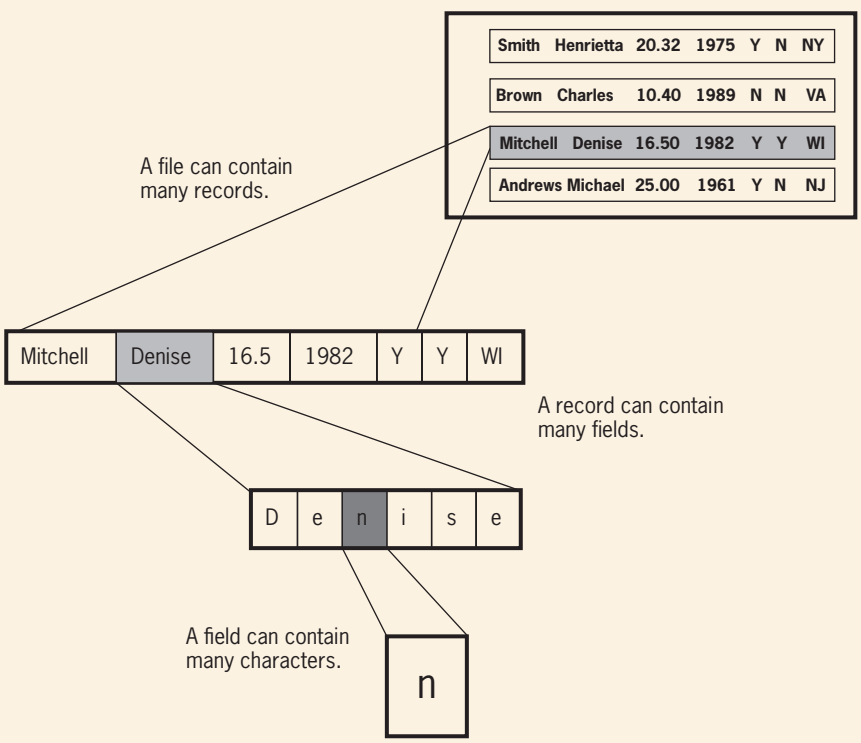
When businesses store data items on computer systems, they often are stored in a framework called the **data hierarchy** that describes the relationships among data components. The data hierarchy consists of the following:

- **Characters** are letters, numbers, and special symbols, such as *A*, 7, and *$*. Anything you can type from the keyboard in one keystroke is a character, including seemingly "empty" characters such as spaces and tabs. Computers also recognize characters you cannot enter from a standard keyboard, such as foreign-alphabet characters such as Ω or Π. Characters are made up of smaller elements called bits, but just as most human beings can use a pencil without caring whether atoms are flying around inside it, most computer users store characters without thinking about these bits.
- **Fields** are data items that each represent a single attribute of a record; fields are composed of one or more characters. Fields include items such as `lastName`, `middleInitial`, `streetAddress`, or `annualSalary`.

- **Records** are groups of fields that go together for some logical reason. A random name, address, and salary aren't very useful, but if they're *your* name, *your* address, and *your* salary, then that's your record. An inventory record might contain fields for item number, color, size, and price; a student record might contain an ID number, grade point average, and major.
- **Files** are groups of related records. The individual records of each student in your class might go together in a file called *Students.dat*. Similarly, records of each person at your company might be in a file called *Personnel.dat*. Some files can have just a few records. For example, a student file for a college seminar might have only 10 records. Others, such as a student file for a university or a file of policy holders for a large insurance company, can contain thousands or even millions of records.

Quick Reference 7-1 provides a concise overview of the components of the data hierarchy.

QUICK REFERENCE 7-1 The Components of the Data Hierarchy





A **database** holds related file data in tables. Database software establishes and maintains relationships between fields in these tables, so that users can pull related data items together in a format that allows businesspeople to make managerial decisions efficiently.

TWO TRUTHS & A LIE

Understanding the Data Hierarchy

1. In the data hierarchy, a field is a single data item, such as `lastName`, `streetAddress`, or `annualSalary`.
2. In the data hierarchy, fields are grouped together to form a record; records are groups of fields that go together for some logical reason.
3. In the data hierarchy, related records are grouped together to form a field.

The false statement is #3. Related records form a file.

Performing File Operations

To use data files in your programs, you need to understand several file operations:

- Declaring a file identifier
- Opening a file
- Reading from a file and processing the data
- Writing to a file
- Closing a file

Declaring a File Identifier

Most languages support several types of files, but one way of categorizing files is by whether they can be used for input or for output. Just as variables and constants have data types such as `num` and `string`, each file has a data type that is defined in the language you are using. For example, a file's type might be `InputFile`. Just like variables and constants, files are declared by giving each a data type and an identifier. As examples, you might declare two files as follows:

```
InputFile employeeData
OutputFile updatedData
```

The `InputFile` and `OutputFile` types are capitalized in this book because their equivalents are capitalized in most programming languages. This approach helps to distinguish these complex types from simple types such as `num` and `string`. The identifiers given to files, such as `employeeData` and `updatedData`, are internal to the program, just as variable names are. To make a program read a file's data from a storage device, you also need to associate the program's internal filename with the operating system's name for the file. Often, this association is accomplished when you open the file.

Opening a File

In most programming languages, before an application can use a data file, it must **open the file**. Opening a file locates it on a storage device and associates a variable name within your program with the file. For example, if the identifier `employeeData` has been declared as type `InputFile`, then you might make a statement similar to the following:

```
open employeeData "EmployeeData.dat"
```

This statement associates the file named EmployeeData.dat on the storage device with the program's internal name `employeeData`. If the data file is not stored in the same directory as the program, you usually specify a more complete path, as in the following:

```
open employeeData "C:\CompanyFiles\CurrentYear\EmployeeData.dat"
```

Reading Data from a File and Processing It

You never use the data values that are stored on a storage device directly. Instead, you use a copy that is transferred into memory. When you copy data from a file on a storage device into RAM, you are **reading from a file**.



When data items are stored on a disk, their location might not be clear to you—data just seems to be "in the computer." To a casual computer user, the lines between permanent storage and temporary memory are often blurred because many newer programs automatically save data for you periodically without asking your permission. However, at any moment in time, the version of a file in memory might differ from the version that was last saved to a storage device.

Once the program's identifier `employeeData` has been associated with the stored file, you can write separate programming statements to input each field, as in the following example:

```
input name from employeeData
input address from employeeData
input payRate from employeeData
```

Most languages also allow you to write a single statement in the following format:

```
input name, address, payRate from employeeData
```

As a further simplification, many programming languages allow you to declare a group item when you declare other variables, as in the following example:

```
EmployeeRecord
string name
string address
num payRate
```

In this example, `EmployeeRecord` is a group name for the values stored in the three fields `name`, `address`, and `payRate`. After the group is defined, you can input all the data items in a record with a single instruction, such as the following:

```
input EmployeeRecord from employeeData
```

You usually do not want to input several items in a single statement when you read data from a keyboard, because you want to prompt the user for each item separately as you input it. However, when you retrieve data from a file, prompts are not needed. Instead, each item is retrieved in sequence and stored in memory at the appropriate named location.

Programming languages have different ways of determining how much data to input when a command is issued to read a variable's value from a file. In many languages, a **delimiter** such as a comma, semicolon, or tab character is stored between data fields. Sometimes, data items are stored in a compact form so that they are not decipherable when the file is opened in a text editor, but sometimes data items are stored in a readable format. For example, Figure 7-2 shows what a readable comma-delimited data file of employee names, addresses, and pay rates might look like in a text reader. The amount of data retrieved depends on the data types of the variables in the input statement. For example, reading a numeric value might imply that four bytes will be read. When you learn to program in a specific language, you will learn how data items are stored and retrieved in that language.

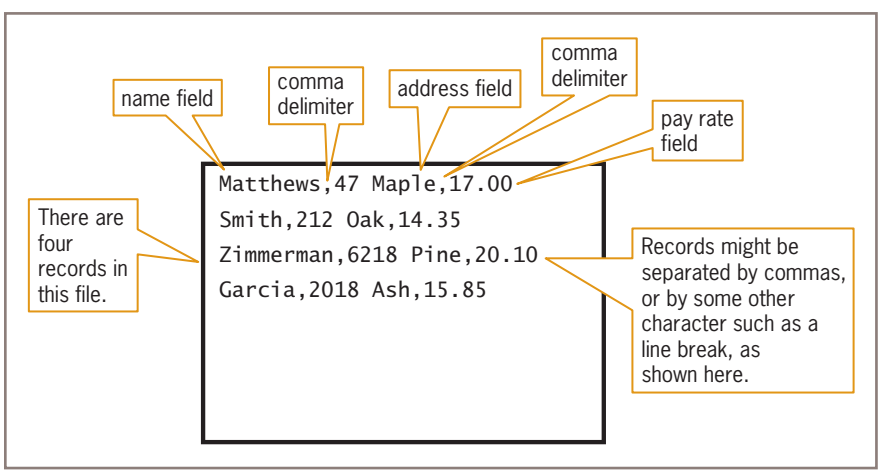


Figure 7-2 How employee data in a readable comma-delimited file might appear in a text reader

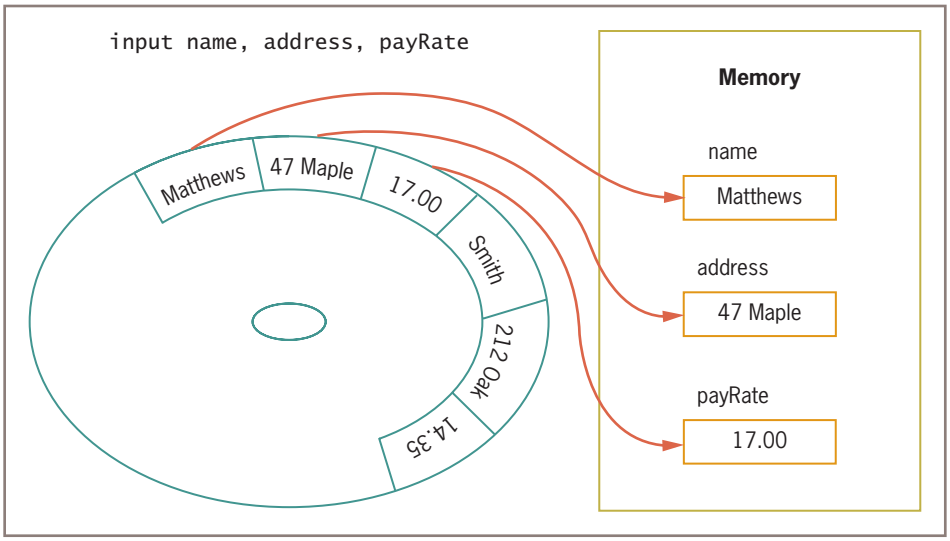


Figure 7-3 Reading three data items from a storage device into memory

Figure 7-3 shows an example of how a statement gets data from a file. When the input statement executes, each field stored in the file is copied and placed in the appropriate variable in computer memory. Nothing on the disk indicates a field name associated with any of the data; the variable names exist within the program only. For example, although this program uses the variable names `name`, `address`, and `payRate`, another program could use the same file as input and call the fields `surname`, `street`, and `salary`.

When you read data from a file, you usually must read all the fields that are stored even though you might not want to use all of them. For example, suppose that you want to read an employee data file that contains names, addresses, and pay rates for each employee, and you want to output a list of names. Even though you are not concerned with the address or pay rate fields for each employee, you typically read them into your program before you can get to the name for the next employee.

A computer program can read records from a file sequentially or randomly. When a program uses a **sequential file**, it reads all the records in the file from beginning to end, processing them one at a time. Frequently, although not always, records in a sequential file have been sorted based on the contents of one or more fields. **Sorting** is the process of placing records in order by the value in a specific field or fields. **Ascending order** describes records sorted in order from lowest to highest values; **descending order** describes records in order from highest to lowest values. Records in a file can be sorted manually before they are entered and saved, or records can be entered in any order and a program can sort them. Later in this chapter, you will learn about random access files, in which the records can be accessed in any order.



You can learn about sorting techniques in Chapter 8 of the comprehensive version of this book. In this chapter, it is assumed that if a file needs to be sorted, the sorting process has already been completed.

Examples of sorted, sequential files include the following:

- A file of employees whose data is stored in order by ID number
- A file of parts for a manufacturing company whose data is stored in order by part number
- A file of customers for a business whose data is stored in alphabetical order by name

After data fields have been read into memory, you can process them. Depending on the application, examples of processing the data might include performing arithmetic with some of the numeric fields or altering some of the string fields by adding or removing characters. Changes made to the fields in memory do not affect the original data stored on the input device. Typically, after the data field contents are processed, you want to output them. If the processed information needs to be visible only to people, you might display it on a screen. If the information will be used by another program, however, then you would want to write the processed information to a file.

Writing Data to a File

When you store data in a computer file on a persistent storage device, you **write to the file**. This means you copy data from RAM to the file. When you write data to a file, you write the contents of the fields using a statement such as the following:

```
output name, address, payRate to employeeData
```

When you write data to a file, you usually do not include explanations that make data easier for humans to interpret; you just write facts and figures. For example, you do not include explanations such as *The pay rate is*, nor do you include commas, dollar signs, or percent signs in numeric values. Those embellishments are appropriate for output on a monitor or on paper, but not for storage.

Closing a File

When you finish using a file, the program should **close the file**—a closed file is no longer available to your application. Failing to close an input file (a file from which you are reading data) usually does not present serious consequences; the data still exists in the file. However, if you fail to close an output file (a file to which you are writing data), the data might not be saved correctly and might become inaccessible. You should always close every file you open, and you should close the file as soon as you no longer need it. When you leave a file open for no reason, you use computer resources, and your computer's performance suffers. Also, within a network, another program might be waiting to use the file.



Although you open and close files on storage devices such as disks, most programming languages allow you to read data from a keyboard or write it to the display monitor without having to issue open or close commands because the keyboard and monitor are the **default input and output devices**, respectively.

A Program that Performs File Operations

Figure 7-4 contains a program that opens two files—an input file and an output file. The program reads each employee record from the input file, alters the employee's pay rate, and writes the updated record to an output file. After all the records have been processed, the program closes the files. The statements that use the files are shaded. When creating flowcharts, some programmers place file open and close statements in a process box (rectangle), but the convention in this book is to place file open and close statements in parallelograms, because they are operations closely related to input and output.

In the program in Figure 7-4, after each employee's pay rate is read into memory, it is increased by $2.00. The value of the pay rate on the input storage device is not altered. When processing is complete, the input file retains the original data for each employee, and the output file contains the revised data. Many organizations would keep the original file as a backup file. A **backup file** is a copy that is kept in case values need to be restored to their original state. The backup copy is called a **parent file** and the newly revised copy is a **child file**.



Logically, the verbs *print*, *write*, and *display* mean the same thing—all produce output. In conversation, however, programmers usually reserve the word *print* for situations in which they mean *produce hard copy output*. Programmers are more likely to use *write* when talking about sending records to a data file and *display* when sending records to a monitor. In some programming languages, there is no difference in the verb used for output regardless of the hardware; you simply assign different output devices (such as printers, monitors, and disk drives) as needed to programmer-named objects that represent them.



Watch the video *File Operations*.

Throughout this book, you have been encouraged to think about input as basically the same process, whether it comes from a user typing interactively at a keyboard or from a stored file on a disk or other media. The concept remains valid for this chapter, which discusses applications that commonly use stored file data.

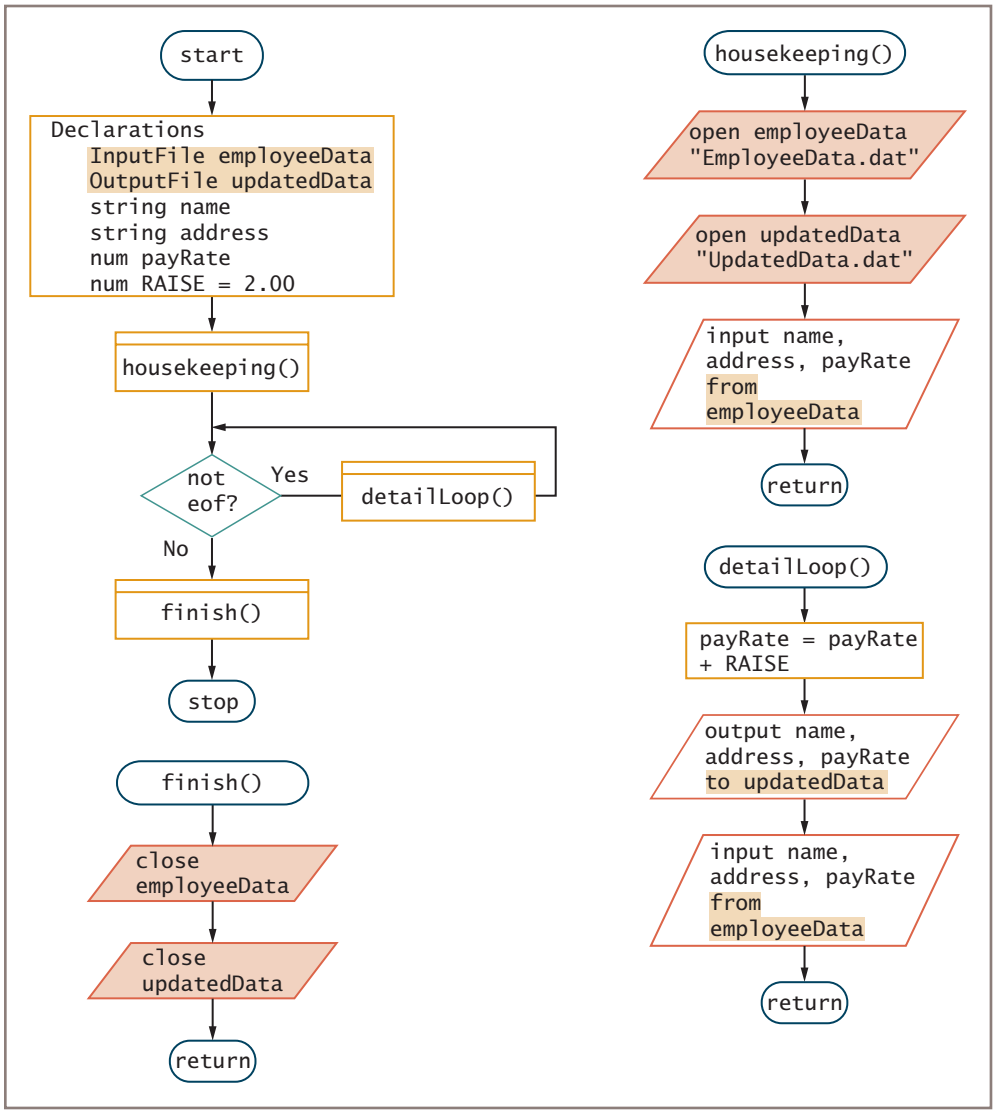


Figure 7-4 Flowchart and pseudocode for program that uses files *(continues)*

(continued)

```
start
   Declarations
      InputFile employeeData
      OutputFile updatedData
      string name
      string address
      num payRate
      num RAISE = 2.00
   housekeeping()
   while not eof
      detailLoop()
   endwhile
   finish()
stop

housekeeping()
   open employeeData "EmployeeData.dat"
   open updatedData "UpdatedData.dat"
   input name, address, payRate from employeeData
return

detailLoop()
   payRate = payRate + RAISE
   output name, address, payRate to updatedData
   input name, address, payRate from employeeData
return

finish()
    close employeeData
    close updatedData
return
```

Figure 7-4 Flowchart and pseudocode for program that uses files

TWO TRUTHS & A LIE

Performing File Operations

1. You give a file an internal name in a program and then associate it with the operating system's name for the file.
2. When you read from a file, you copy values from memory to a storage device.
3. If you fail to close an input file, usually no serious consequences will occur; the data values still exist in the file.

The false statement is #2. When you read from a file, you copy values from a storage device into memory. When you write to a file, you copy values from memory to a storage device.

Understanding Control Break Logic

A **control break** is a temporary detour in the logic of a program. In particular, programmers use a **control break program** to do the following:

- Read in records from a sorted sequential file so that all records that belong to specific groups are stored together in sequence.
- Process each record, checking to determine if it still belongs to the same group as the previous record.
- Pause for special processing whenever a new group of records is encountered.

For example, a control break program might be used to generate a report that lists all company clients in order by state of residence, with a count of clients after each state's client list. See Figure 7-5 for an example of a **control break report** that breaks after each change in state.

Other examples of control break reports produced by control break programs could include:

- All employees listed in order by department number, with a new page started for each department
- All books for sale in a bookstore listed in order by category (such as reference or self-help), with a count following each category of book
- All items sold in order by date of sale, with a different ink color for each new month

Company Clients by State of Residence

Name	City	State	
Albertson	Birmingham	Alabama	
Davis	Birmingham	Alabama	
Lawrence	Montgomery	Alabama	
		Count for Alabama	3
Smith	Anchorage	Alaska	
Young	Anchorage	Alaska	
Davis	Fairbanks	Alaska	
Mitchell	Juneau	Alaska	
Zimmer	Juneau	Alaska	
		Count for Alaska	5
Edwards	Phoenix	Arizona	
		Count for Arizona	1

Figure 7-5 A control break report with totals after each state

Each of these reports shares two traits:

- The records used in a report are listed in order by a specific variable: state, department, category, or date.
- When the value of that variable changes in a new record, the program takes special action: it starts a new page, prints a count or total, or switches ink color.

To generate a control break report, the input records must be organized in sequential order based on the field that will cause the breaks. In other words, to write a program that produces a report of customers by state, such as the one in Figure 7-5, the records must be grouped by state before you begin processing.



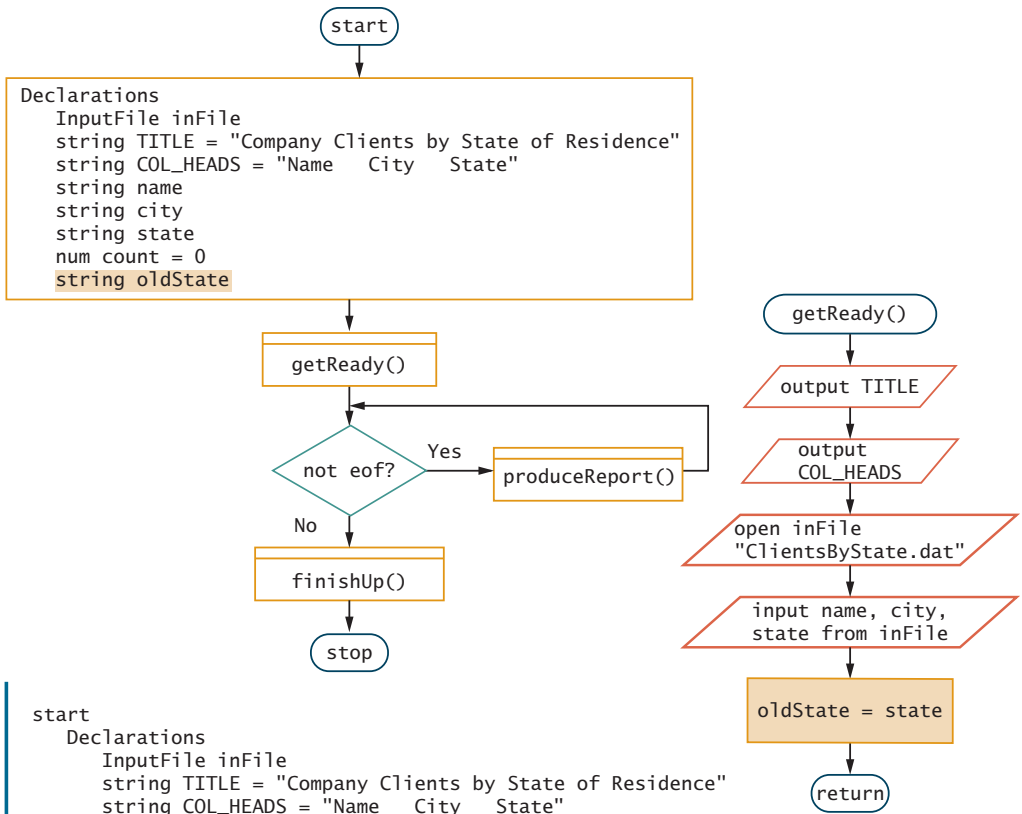
With some languages, such as SQL (Structured Query Language, which is used with database processing), the details of control break processing are handled automatically. Still, understanding how control break programs work improves your competence as a programmer.

Suppose that you have an input file that contains client names, cities, and states, and you want to produce a report like the one in Figure 7-5. The basic logic of the program works like this:

- Each time you read a client's record from the input file, you determine whether the client resides in the same state as the previous client.
- If so, you simply output the client's data, add 1 to a counter, and read another record, without any special processing. If there are 20 clients in a state, these steps are repeated 20 times in a row—read a client's data, count it, and output it.
- Eventually you will read a record for a client who is not in the same state. At that point, before you output the data for the first client in the new state, you must output the count for the previous state. You also must reset the counter to 0 so it is ready to start counting customers in the next state. Then you can proceed to handle client records for the new state, and you continue to do so until the next time you encounter a client from a different state.

This type of program contains a **single-level control break**, a break in the logic of the program (in this case, pausing or detouring to output a count) that is based on the value of a single variable (in this case, the state). The technique you must use to "remember" the old state so you can compare it with each new client's state is to create a special variable, called a **control break field**, to hold the previous state. As you read each new record, comparing the new and old state values determines when it is time to output the count for the previous state.

Figure 7-6 shows the mainline logic and `getReady()` module for a program that produces the report in Figure 7-5. In the mainline logic, the control break variable `oldState` is declared in the shaded statement. In the `getReady()` module, the report headings are




```
start
   Declarations
      InputFile inFile
      string TITLE = "Company Clients by State of Residence"
      string COL_HEADS = "Name   City   State"
      string name
      string city
      string state
      num count = 0
      string oldState
   getReady()
   while not eof
      produceReport()
   endwhile
   finishUp()
stop

getReady()
   output TITLE
   output COL_HEADS
   open inFile "ClientsByState.dat"
   input name, city, state from inFile
   oldState = state
return
```

Figure 7-6 Mainline logic and `getReady()` module for the program that produces clients by state report

output, the file is opened, and the first record is read into memory. Then, the state value in the first record is copied to the `oldState` variable. (See shading.) Note that it would be incorrect to initialize `oldState` when it is declared. When you declare the variables at the beginning of the main program, you have not yet read the first input record; therefore, you don't know what the value of the first state will be. You might assume that it is *Alabama* because that is the first state alphabetically, and you might be right, but perhaps this data set contains no records for *Alabama* and the first state is *Alaska* or even *Wyoming*. You are assured of storing the correct first state value if you copy it from the first input record.

Within the `produceReport()` module in Figure 7-7, the first task is to check whether `state` holds the same value as `oldState`. For the first record, on the first pass through this

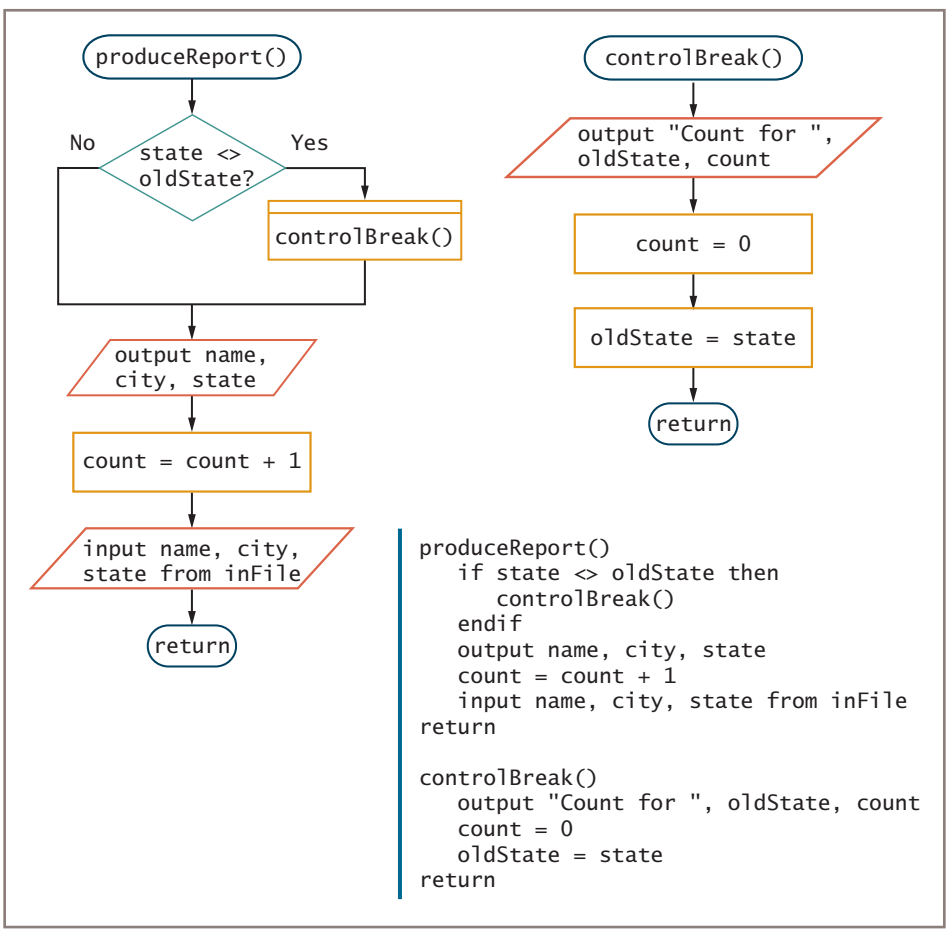


Figure 7-7 The `produceReport()` and `controlBreak()` modules for the program that produces a list of clients by state

method, the values are equal (because you set them to be equal right after getting the first input record in the `getReady()` module). Therefore, you proceed by outputting the first client's data, adding 1 to `count`, and inputting the next record.

As long as each new record holds the same `state` value, you continue outputting, counting, and inputting, never pausing to output the count. Eventually, you will read in a client whose state is different from the previous one. That's when the control break occurs. Whenever a new state differs from the old one, three tasks must be performed:

- The count for the previous state must be output.
- The count must be reset to 0 so it can start counting records for the new state.
- The control break field must be updated to hold the new state.

When the `produceReport()` module receives a client record for which `state` is not the same as `oldState`, you force a break in the normal flow of the program. The new client record must "wait" while the count for the just-finished state is output and `count` and the control break field `oldState` acquire new values.

The `produceReport()` module continues to output client names, cities, and states until the end of the file is reached; then the `finishUp()` module executes. As shown in Figure 7-8, the module that executes after processing the last record in a control break program must complete any required processing for the last group that was handled. In this case, the `finishUp()` module must display the count for the last state that was processed. After the input file is closed, the logic can return to the main program, where the program ends.

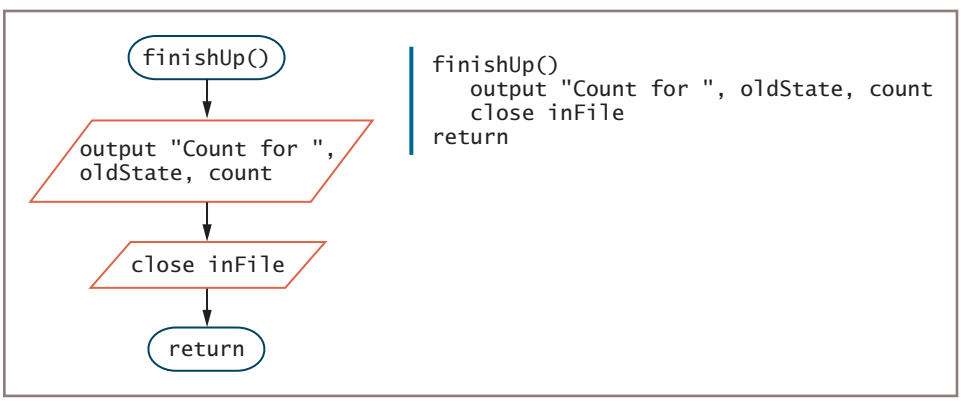


Figure 7-8 The `finishUp()` module for the program that produces clients by state report

Watch the video *Control Break Logic*.

TWO TRUTHS & A LIE

Understanding Control Break Logic

1. In a control break program, a change in the value of a variable initiates special actions or causes unique processing to occur.
2. When a control break variable changes, the program takes special action.
3. To generate a control break report, input records must be organized in sequential order based on the first field in the record.

The false statement is #3. To generate a control break report, input records must be organized in sequential order based on the field that will trigger the break.

Merging Sequential Files

Businesses often need to merge two or more sequential files. **Merging files** involves combining two or more files while maintaining the sequential order of the records. For example:

- Suppose that you have a file of current employees in ID number order and a file of newly hired employees, also in ID number order. You need to merge these two files into one combined file in ID number order before running this week's payroll program.
- Suppose that you have a file of parts manufactured in the Northside factory in part-number order and a file of parts manufactured in the Southside factory, also in part-number order. You want to merge these two files into one combined file, creating a master list of available parts in order by part number.
- Suppose that you have a file that lists last year's customers in alphabetical order and another file that lists this year's customers in alphabetical order. You want to create a mailing list of all customers in order by last name.

Before you can easily merge files, two conditions must be met:

- Each file must contain the same record layout.
- Each file used in the merge must be sorted in the same order based on the same field.

For example, suppose that your business has two locations, one on the East Coast and one on the West Coast, and each location maintains a customer file in alphabetical order by customer name. Each file contains fields for name and customer balance. You can call the

fields in the East Coast file `eastName` and `eastBalance`, and the fields in the West Coast file `westName` and `westBalance`. You want to merge the two files, creating one combined file containing records for all customers. Figure 7-9 shows some sample data for the files; you want to create a merged file like the one shown in Figure 7-10.

East Coast File

eastName	eastBalance
Able	100.00
Brown	50.00
Dougherty	25.00
Hanson	300.00
Ingram	400.00
Johnson	30.00

West Coast File

westName	westBalance
Chen	200.00
Edgar	125.00
Fell	75.00
Grand	100.00

Figure 7-9 Sample data contained in two customer files

Merged File

mergedName	mergedBalance
Able	100.00
Brown	50.00
Chen	200.00
Dougherty	25.00
Edgar	125.00
Fell	75.00
Grand	100.00
Hanson	300.00
Ingram	400.00
Johnson	30.00

Figure 7-10 Merged customer file

The mainline logic for a program that merges two files is similar to the main logic you've used before in other programs: It contains preliminary, housekeeping tasks; a detail module that repeats until the end of the program; and some clean-up, end-of-job tasks. Most programs you have studied, however, processed records until an `eof` condition was met, either because an input data file reached its end or because a user entered a sentinel value in an interactive program. In a program that merges two input files, checking for `eof` in only one of them is insufficient. Instead, the program can't end until both input files are exhausted.

One way to end a file-merging program is to create a string flag variable with a name such as `areBothAtEnd`. You might initialize `areBothAtEnd` to `"N"`, but change its value to `"Y"` after you have encountered `eof` in both input files. (If the language you use supports a Boolean data type, you can use the values `true` and `false` instead of strings.) Figure 7-11 shows the mainline logic for a program that merges the files shown in Figure 7-9. After the `getReady()` module executes, the shaded question that sends the logic to the `finishUp()` module tests the `areBothAtEnd` variable. When it holds `"Y"`, the program ends.

The `getReady()` module for the file merging program is shown in Figure 7-12. It opens three files—the input files for the east and west customers, and an output file in which to place the merged records. The program then reads one record from each input file. If either file has reached its end, the `END_NAME` constant is assigned to the variable that holds the file's

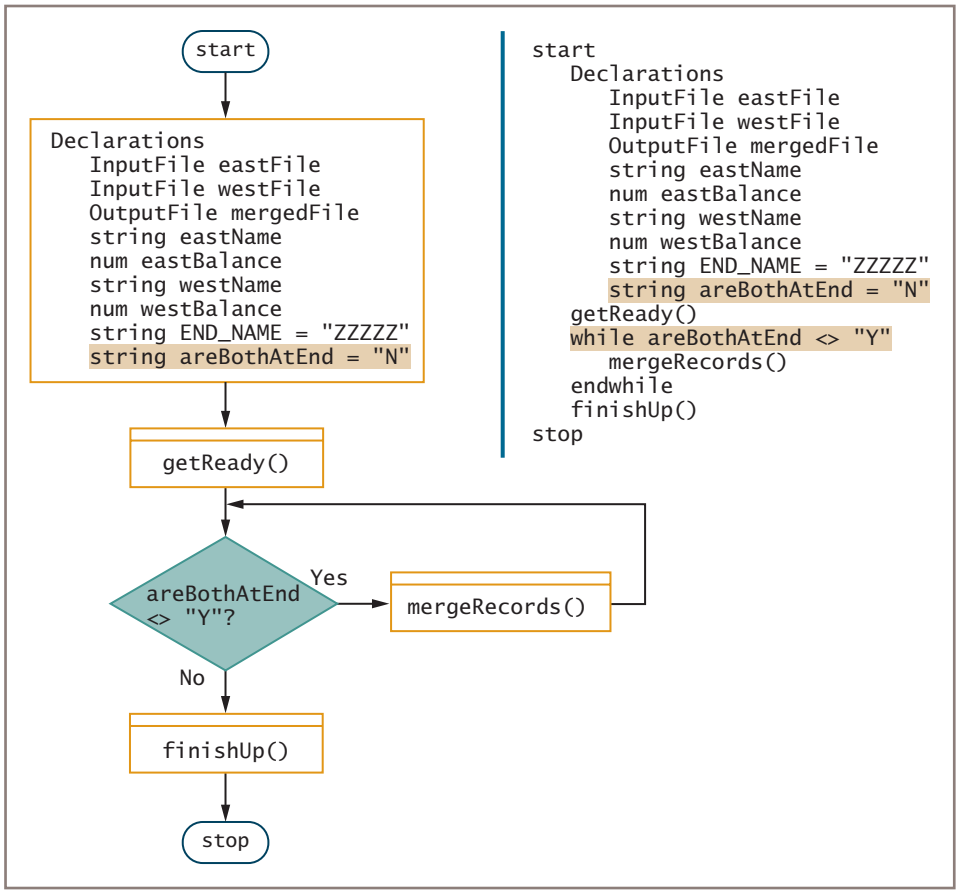


Figure 7-11 Mainline logic of a program that merges files

customer name. The `getReady()` module then checks to see whether both files are finished (admittedly, a rare occurrence in the `getReady()` portion of the program's execution) and sets the `areBothAtEnd` flag variable to `"Y"` if they are. Assuming that at least one record is available, the logic then would enter the `mergeRecords()` module.

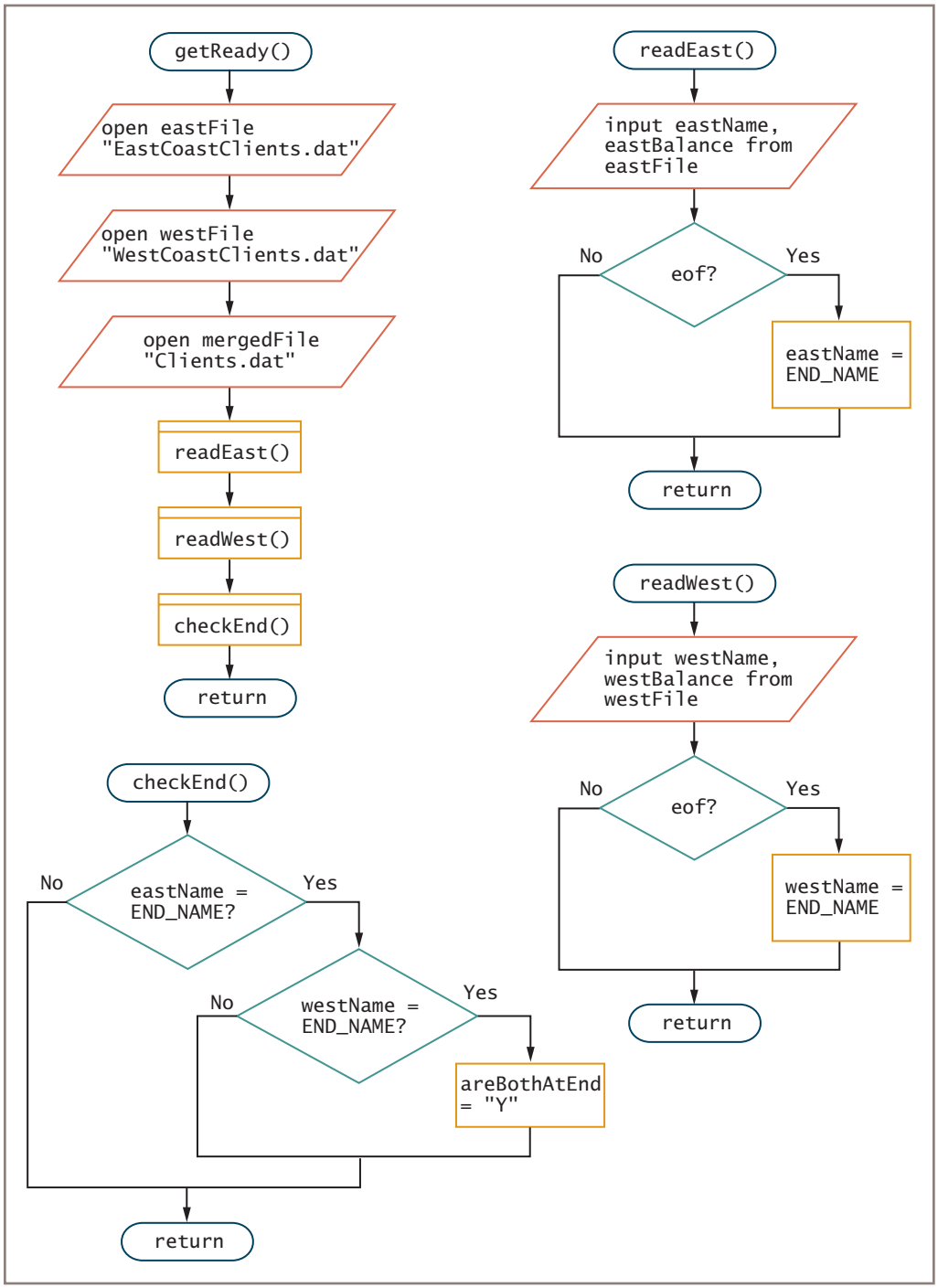


Figure 7-12 The `getReady()` method for a program that merges files, and the methods it calls *(continues)*

(continued)

```
getReady()
   open eastFile "EastCoastClients.dat"
   open westFile "WestCoastClients.dat"
   open mergedFile "Clients.dat"
   readEast()
   readWest()
   checkEnd()
return

readEast()
   input eastName, eastBalance from eastFile
   if eof then
      eastName = END_NAME
   endif
return

readWest()
   input westName, westBalance from westFile
   if eof then
      westName = END_NAME
   endif
return

checkEnd()
   if eastName = END_NAME then
      if westName = END_NAME then
         areBothAtEnd = "Y"
      endif
   endif
return
```

Figure 7-12 The `getReady()` method for a program that merges files, and the methods it calls

When you begin the `mergeRecords()` module in the program using the files shown in Figure 7-9, two records—one from `eastFile` and one from `westFile`—are sitting in the memory of the computer. One of these records needs to be written to the new output file first. Which one? Because the two input files contain records stored in alphabetical order, and you want the new file to store records in alphabetical order, you first output the input record that has the lower alphabetical value in the name field. Therefore, the process begins as shown in Figure 7-13.

Using the sample data from Figure 7-9, you can see that the *Able* record from the East Coast file should be written to the output file, while Chen's record from the West Coast file waits in memory. The `eastName` value *Able* is alphabetically lower than the `westName` value *Chen*.

After you write Able's record, should Chen's record be written to the output file next? Not necessarily. It depends on the next `eastName` following Able's record in `eastFile`. When data records are read into memory from a file, a program typically does not "look ahead" to determine the values stored in the next record. Instead, a program usually reads the record into memory before making decisions about its contents. In this program, you need to read

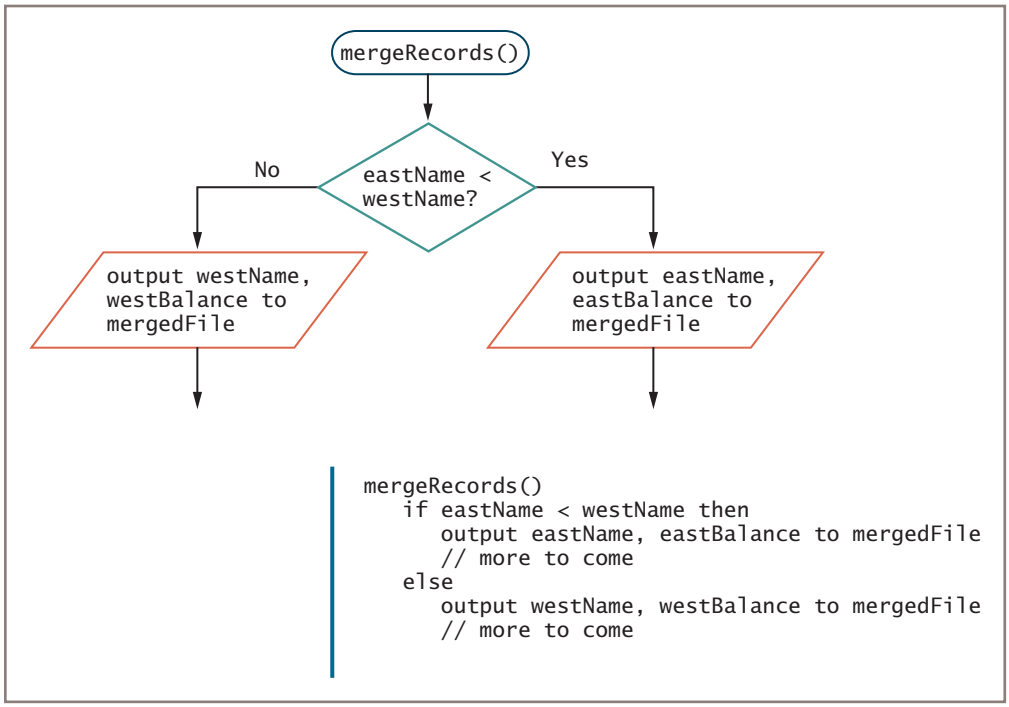


Figure 7-13 Start of merging process

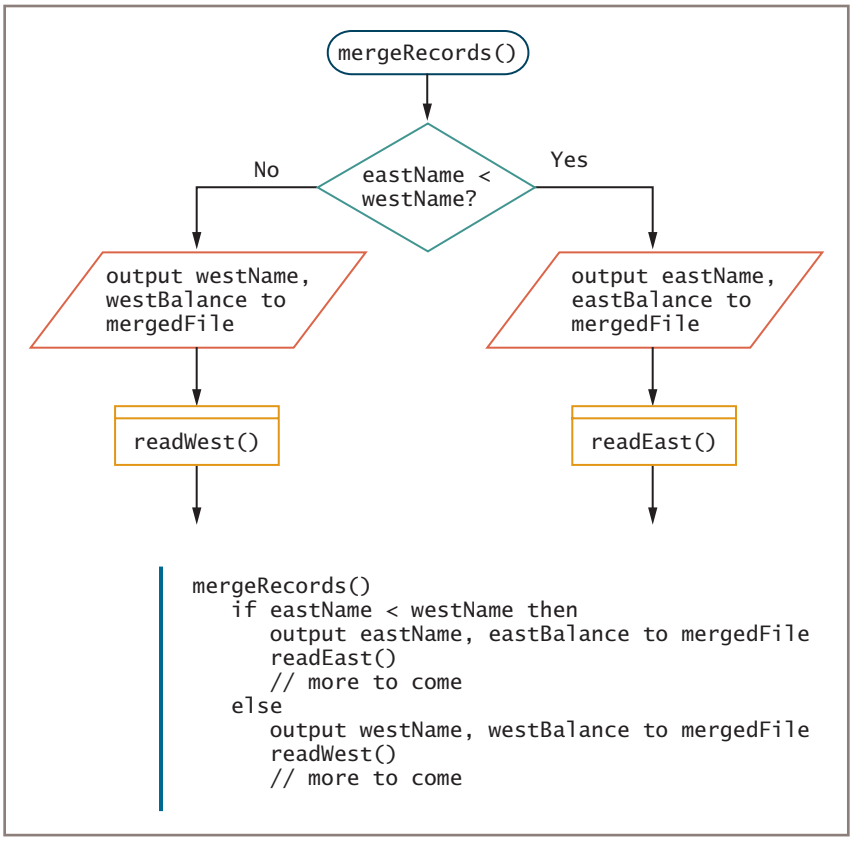
the next `eastFile` record into memory and compare it to *Chen*. Because in this case the next record in `eastFile` contains the name *Brown*, another `eastFile` record is written; no `westFile` records are written yet.

After the first two `eastFile` records, is it Chen's turn to be written now? You really don't know until you read another record from `eastFile` and compare its name value to *Chen*. Because this record contains the name *Dougherty*, it is indeed time to write Chen's record. After Chen's record is written, should you now write Dougherty's record? Until you read the next record from `westFile`, you don't know whether that record should be placed before or after Dougherty's record.

Therefore, the merging method proceeds like this: compare two records, write the record with the lower alphabetical name, and read another record from the *same* input file. See Figure 7-14.

Recall the names from the two original files in Figure 7-9, and walk through the processing steps.

1. Compare *Able* and *Chen*. Write Able's record. Read Brown's record from `eastFile`.
2. Compare *Brown* and *Chen*. Write Brown's record. Read Dougherty's record from `eastFile`.
3. Compare *Dougherty* and *Chen*. Write Chen's record. Read Edgar's record from `westFile`.




```
mergeRecords()
   if eastName < westName then
      output eastName, eastBalance to mergedFile
      readEast()
      // more to come
   else
      output westName, westBalance to mergedFile
      readWest()
      // more to come
```

Figure 7-14 Continuation of merging process

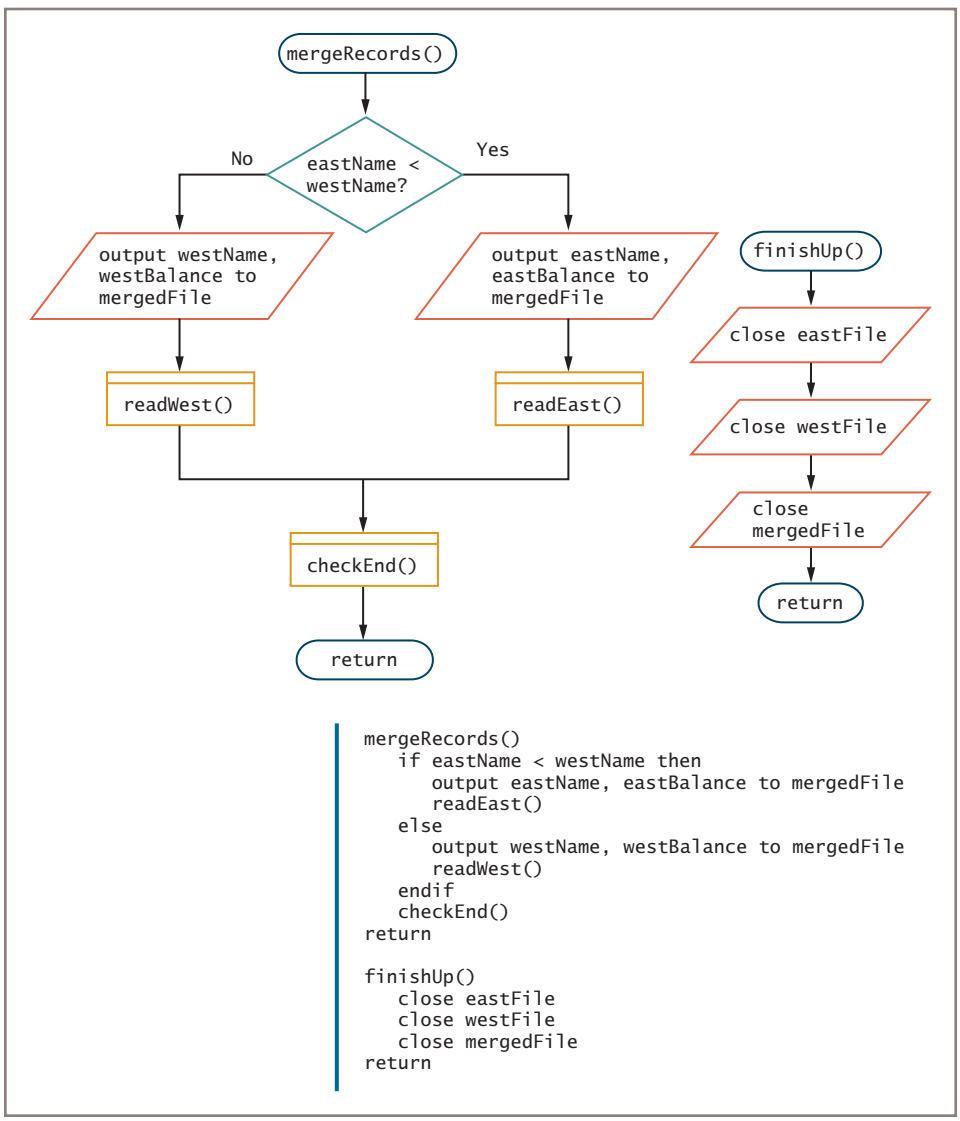
4. Compare *Dougherty* and *Edgar*. Write Dougherty's record. Read Hanson's record from `eastFile`.
5. Compare *Hanson* and *Edgar*. Write Edgar's record. Read Fell's record from `westFile`.
6. Compare *Hanson* and *Fell*. Write Fell's record. Read Grand's record from `westFile`.
7. Compare *Hanson* and *Grand*. Write Grand's record. Read from `westFile`, encountering `eof`. This causes `westName` to be set to `END_NAME`.

What happens when you reach the end of the West Coast file? Is the program over? It shouldn't be because records for *Hanson, Ingram*, and *Johnson* all need to be included in the new output file, and none of them is written yet. Because the `westName` field is set to `END_NAME`, and `END_NAME` has a very high alphabetic value (*ZZZZZ*), each subsequent `eastName` will be lower than the value of `westName`, and the rest of the `eastFile` records will be processed. With a different set of data, the `eastFile` might have ended first. In

that case, `eastName` would be set to `END_NAME`, and each subsequent `westFile` record would be processed. Figure 7-15 shows the complete `mergeRecords()` module and the `finishUp()` module.



As the value for `END_NAME`, you should make sure that the high value you choose is actually higher than any legitimate value. For example, you might choose to use 10 or 20 Zs instead of only five, although it is unlikely that a person will have the last name *ZZZZZ*.




```
mergeRecords()
   if eastName < westName then
      output eastName, eastBalance to mergedFile
      readEast()
   else
      output westName, westBalance to mergedFile
      readWest()
   endif
   checkEnd()
return

finishUp()
   close eastFile
   close westFile
   close mergedFile
return
```

Figure 7-15 The `mergeRecords()` and `finishUp()` modules for the file-merging program

After Grand's record is processed, `westFile` is read and `eof` is encountered, so `westName` gets set to `END_NAME`. Now, when you enter the loop again, `eastName` and `westName` are compared, and `eastName` is still *Hanson*. The `eastName` value (*Hanson*) is lower than the `westName` value (*ZZZZZ*), so the data for `eastName`'s record is written to the output file, and another `eastFile` record (*Ingram*) is read.

The complete run of the file-merging program now executes the first six of the seven steps listed previously, and then proceeds as shown in Figure 7-15 and as follows, starting with a modified Step 7:

7. Compare *Hanson* and *Grand*. Write Grand's record. Read from `westFile`, encountering `eof` and setting `westName` to *ZZZZZ*.
8. Compare *Hanson* and *ZZZZZ*. Write Hanson's record. Read Ingram's record.
9. Compare *Ingram* and *ZZZZZ*. Write Ingram's record. Read Johnson's record.
10. Compare *Johnson* and *ZZZZZ*. Write Johnson's record. Read from `eastFile`, encountering `eof` and setting `eastName` to *ZZZZZ*.
11. Now that both names are *ZZZZZ*, set the flag `areBothAtEnd` equal to `"Y"`.

When the `areBothAtEnd` flag variable equals `"Y"`, the loop is finished, the files are closed, and the program ends.



If two names are equal during the merge process—for example, when there is a Hanson record in each file—then both Hanson records will be included in the final file. When `eastName` and `westName` match, `eastName` is not lower than `westName`, so you write the `westFile` Hanson record. After you read the next `westFile` record, `eastName` will be lower than the next `westName`, and the `eastFile` Hanson record will be output. A more complicated merge program could check another field, such as first name, when last-name values match.

You can merge any number of files. To merge more than two files, the logic is only slightly more complicated; you must compare the key fields from all the files before deciding which file is the next candidate for output.



Watch the video *Merging Files*.

TWO TRUTHS & A LIE

Merging Sequential Files

1. A sequential file is a file in which records are stored one after another in some order. Most frequently, the records are stored based on the contents of one or more fields within each record.
2. Merging files involves combining two or more files while maintaining the sequential order.
3. Before you can easily merge files, each file must contain the same number of records.

The false statement is #3. Before you can easily merge files, each file must contain the same record layout and each file used in the merge must be sorted in the same order based on the same field.

Master and Transaction File Processing

In the last section, you learned how to merge related sequential files in which each record in each file contained the same fields. Some related sequential files, however, do not contain the same fields. Instead, some related files have a master-transaction relationship. A **master file** holds complete and relatively permanent data; a **transaction file** holds more temporary data. For example, a master customer file might hold records that each contain a customer's name, address, phone number, and balance, and a customer transaction file might contain data that describes each customer's most recent purchase.

Commonly, you gather transactions for a period of time, store them in a file, and then use them one by one to update matching records in a master file. You **update the master file** by making appropriate changes to the values in its fields based on the recent transactions. For example, a file containing transaction purchase data for a customer might be used to update each balance due field in a customer record master file.

Here are a few other examples of files that have a master-transaction relationship:

- A library maintains a master file of all patrons and a transaction file with information about each book or other items checked out.
- A college maintains a master file of all students and a transaction file for each course registration.

- A telephone company maintains a master file for every telephone number and a transaction file with information about every call.

When you update a master file, you can take two approaches:

- You can actually change the information in the master file. When you use this approach, the information that existed in the master file prior to the transaction processing is lost.
- You can create a copy of the master file, making the changes in the new version. Then, you can store the previous, parent version of the master file for a period of time, in case there are questions or discrepancies regarding the update process. The updated, child version of the file becomes the new master file used in subsequent processing. This approach is used in a program later in this chapter.



When a child file is updated, it becomes a parent, and its parent becomes a grandparent. Individual organizations create policies concerning the number of generations of backup files they will save before discarding them. The terms *parent* and *child* refer to file backup generations, but they are used for a different purpose in object-oriented programming. You first learned about the concept of object-oriented programming in Chapter 1, and you can learn much more about it in Chapters 10 and 11 of the comprehensive version of this book.

The logic you use to perform a match between master and transaction file records is similar to the logic you use to perform a merge. As with a merge, you must begin with both files sorted in the same order on the same field. Figure 7-16 shows the mainline logic for a program that matches files. The master file contains a customer number, name, and a field that holds the total dollar amount of all purchases the customer has made previously. The transaction file holds data for sales, including a transaction number, the number of the customer who made the transaction, and the amount of the transaction.

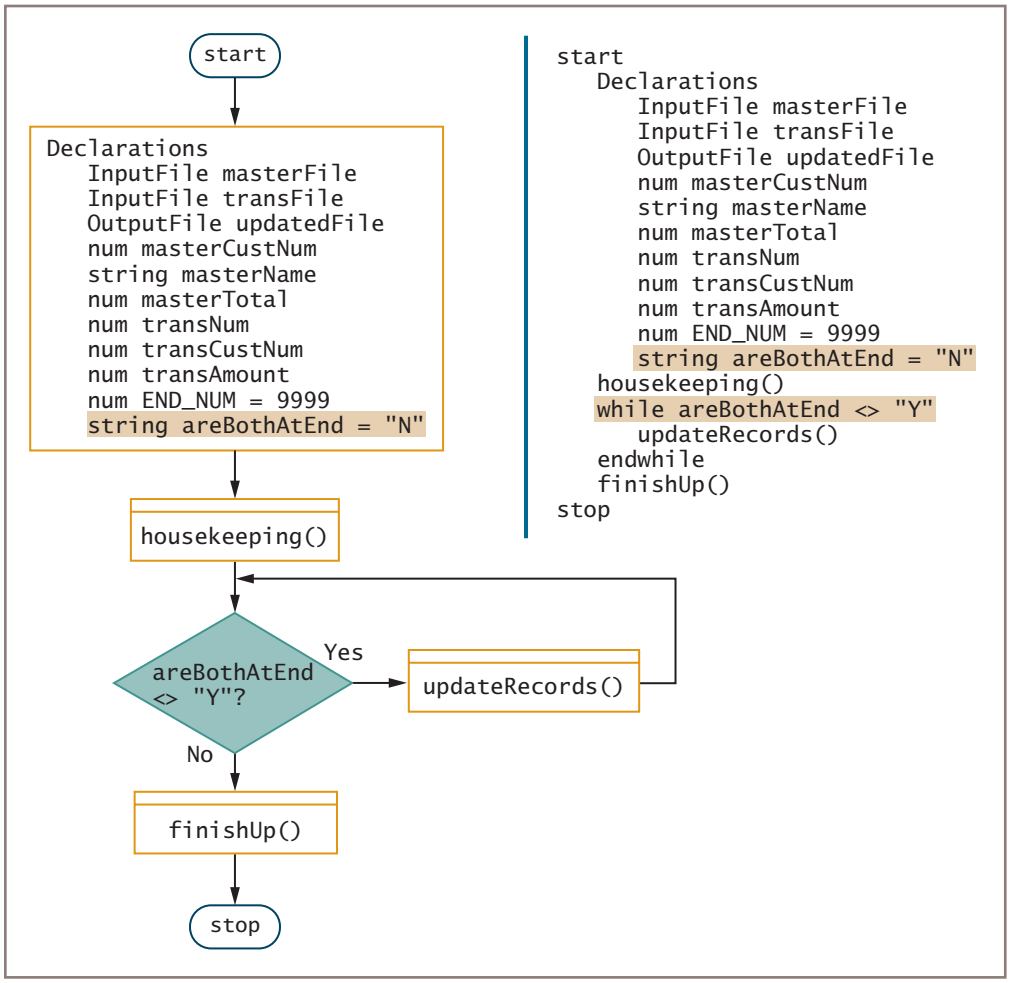


Figure 7-16 Mainline logic for the master-transaction program

Figure 7-17 contains the `housekeeping()` module for the program, and the modules it calls. These modules are very similar to their counterparts in the file-merging program earlier in the chapter. When the program begins, one record is read from each file. When any file ends, the field used for matching is set to a high numeric value, 9999, and when both files end, a flag variable is set so the mainline logic can test for the end of processing. The assumption is that 9999 is higher than any valid customer number.

Imagine that you will update master file records by hand instead of using a computer program, and imagine that each master and transaction record is stored on a separate piece of paper. The easiest way to accomplish the update is to sort all the master records by customer number and place them in a stack, and then sort all the transactions by

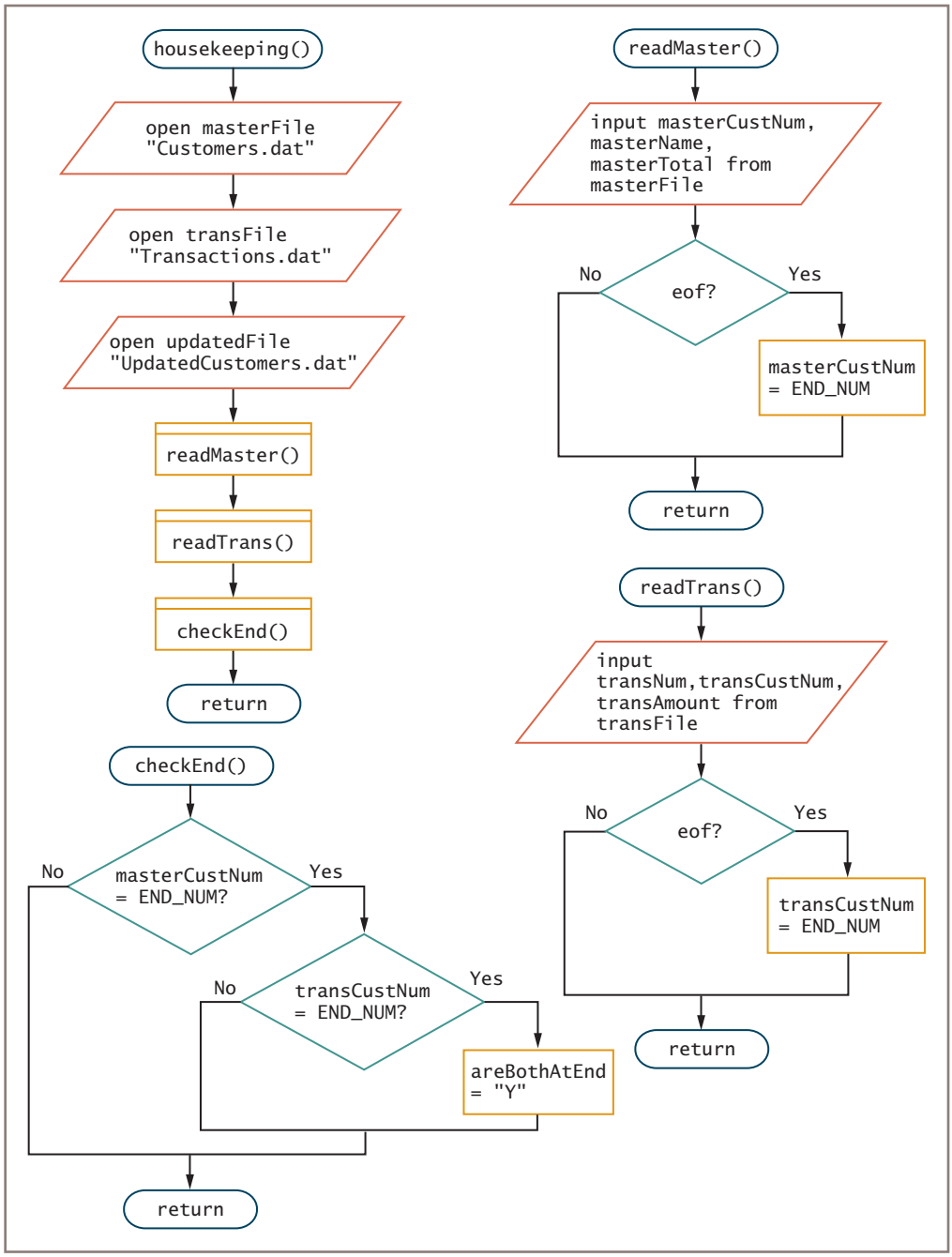


Figure 7-17 The `housekeeping()` module for the master-transaction program, and the modules it calls *(continues)*

(continued)

```
housekeeping()
   open masterFile "Customers.dat"
   open transFile "Transactions.dat"
   open updatedFile "UpdatedCustomers.dat"
   readMaster()
   readTrans()
   checkEnd()
return

readMaster()
   input masterCustNum, masterName, masterTotal from masterFile
   if eof then
      masterCustNum = END_NUM
   endif
return

readTrans()
   input transNum, transCustNum, transAmount from transFile
   if eof then
      transCustNum = END_NUM
   endif
return

checkEnd()
   if masterCustNum = END_NUM then
      if transCustNum = END_NUM then
         areBothAtEnd = "Y"
      endif
   endif
return
```

Figure 7-17 The `housekeeping()` module for the master-transaction program, and the modules it calls

customer number (not transaction number) and place them in another stack. You then would examine the first transaction and look through the master records until you found a match. Any master records without transactions would be placed in a "completed" stack without changes. When a transaction matched a master record, you would correct the master record using the new transaction amount, and then go on to the next transaction. If there was no matching master record for a transaction, then you would realize an error had occurred, and you probably would set the transaction aside before continuing so you could ask someone about it later. The `updateRecords()` module works exactly the same way.

In the file-merging program presented earlier in this chapter, your first action in the program's detail loop was to determine which file held the record with the lower key value; then, you wrote that record. In a matching program, you are trying to determine not only whether one file's comparison field is smaller than another's, but also if they are *equal*.

In this example, you want to update the master file record's `masterTotal` field only if the transaction record field `transCustNum` contains an exact match for the customer number in the master file record. Therefore, you compare `masterCustNum` from the master file and `transCustNum` from the transaction file. Three possibilities exist:

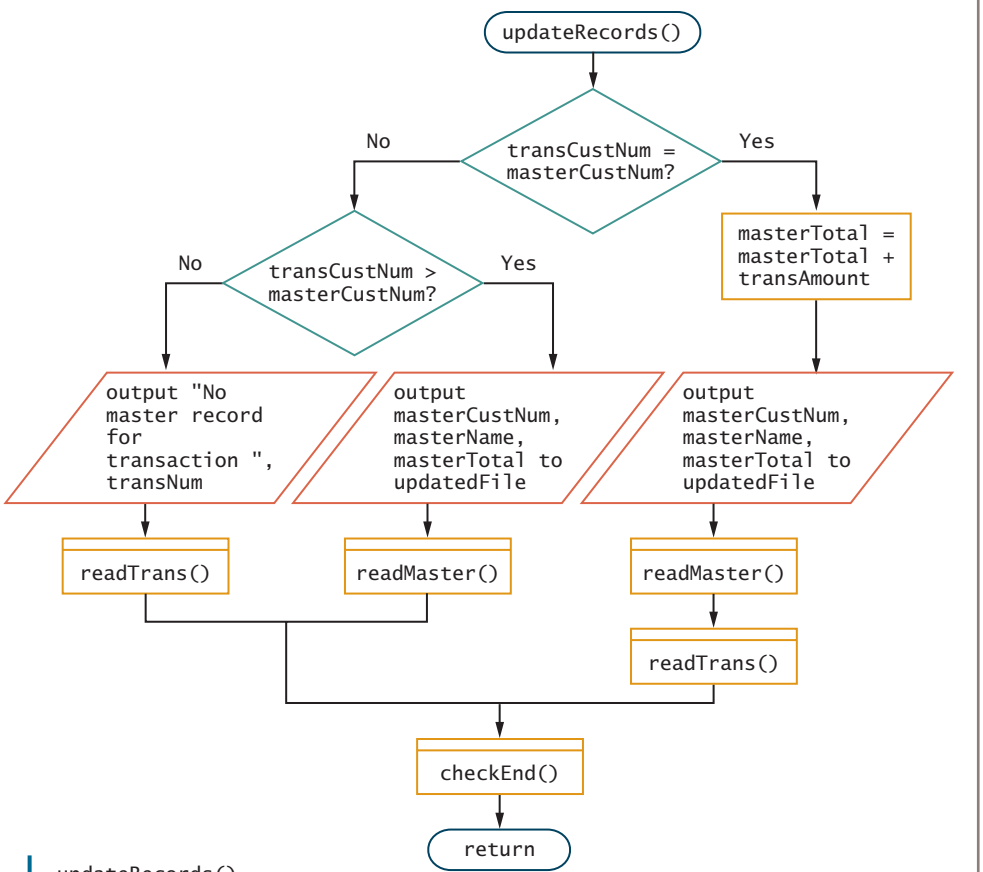
- The `transCustNum` value equals `masterCustNum`. In this case, you add `transAmount` to `masterTotal` and then write the updated master record to the output file. Then, you read in both a new master record and a new transaction record.
- The `transCustNum` value is higher than `masterCustNum`. This means a sale was not recorded for that customer. That's all right; not every customer makes a transaction every period, so you simply write the original customer record with exactly the same information it contained when input. Then, you get the next customer record to see if this customer made the transaction currently under examination.
- The `transCustNum` value is lower than `masterCustNum`. This means you are trying to apply a transaction for which no master record exists, so there must be an error, because a transaction should always have a master record. You could handle this error in a variety of ways; here, you will write an error message to an output device before reading the next transaction record. A human operator can then read the message and take appropriate action.



The logic used here assumes that there can be only one transaction per customer. In the exercises at the end of this chapter, you will develop the logic for a program in which the customer can have multiple transactions.

Whether `transCustNum` was higher than, lower than, or equal to `masterCustNum`, after reading the next transaction or master record (or both), you check whether both `masterCustNum` and `transCustNum` have been set to *9999*, indicating that the end of the file has been reached. When both are 9999, you set the `areBothAtEnd` flag to `"Y"`.

Figure 7-18 shows the `updateRecords()` module that carries out the logic of the file-matching process. Figure 7-19 shows some sample data you can use to walk through the logic for this program.




```
updateRecords()
   if transCustNum = masterCustNum then
      masterTotal = masterTotal + transAmount
      output masterCustNum, masterName, masterTotal to updatedFile
      readMaster()
      readTrans()
   else
      if transCustNum > masterCustNum then
         output masterCustNum, masterName, masterTotal to updatedFile
         readMaster()
      else
         output "No master record for transaction ", transNum
         readTrans()
      endif
   endif
   checkEnd()
return
```

Figure 7-18 The `updateRecords()` module for the master-transaction program

Master File		Transaction File	
masterCustNum	**masterTotal**	**transCustNum**	**transAmount**
100	1000.00	100	400.00
102	50.00	105	700.00
103	500.00	108	100.00
105	75.00	110	400.00
106	5000.00		
109	4000.00		
110	500.00		

Figure 7-19 Sample data for the file-matching program

The program proceeds as follows:

1. Read customer 100 from the master file and customer 100 from the transaction file. Customer numbers are equal, so 400.00 from the transaction file is added to 1000.00 in the master file, and a new master file record is written with a 1400.00 total sales figure. Then, read a new record from each input file.
2. The customer number in the master file is 102 and the customer number in the transaction file is 105, so there are no transactions today for customer 102. Write the master record exactly the way it came in, and read a new master record.
3. Now, the master customer number is 103 and the transaction customer number is still 105. This means customer 103 has no transactions, so you write the master record as is and read a new one.
4. Now, the master customer number is 105 and the transaction number is 105. Because customer 105 had a 75.00 balance and now has a 700.00 transaction, the new total sales figure for the master file is 775.00, and a new master record is written. Read one record from each file.
5. Now, the master number is 106 and the transaction number is 108. Write customer record 106 as is, and read another master.
6. Now, the master number is 109 and the transaction number is 108. An error has occurred. The transaction record indicates that you made a sale to customer 108, but there is no master record for customer number 108. Either the transaction is incorrect (there is an error in the transaction's customer number) or the transaction is correct but you have failed to create a master record. Either way, write an error message so that a clerk is notified and can handle the problem. Then, get a new transaction record.
7. Now, the master number is 109 and the transaction number is 110. Write master record 109 with no changes and read a new one.

8. Now, the master number is 110 and the transaction number is 110. Add the 400.00 transaction to the previous 500.00 balance in the master file, and write a new master record with 900.00 in the `masterTotal` field. Read one record from each file.
9. Because both files are finished, end the job. The result is a new master file in which some records contain exactly the same data they contained going in, but others (for which a transaction has occurred) have been updated with a new total sales figure. The original master and transaction files that were used as input can be saved for a period of time as backups.

Figure 7-20 shows the `finishUp()` module for the program. After all the files are closed, the updated master customer file contains all the customer records it originally contained, and each holds a current total based on the recent group of transactions.

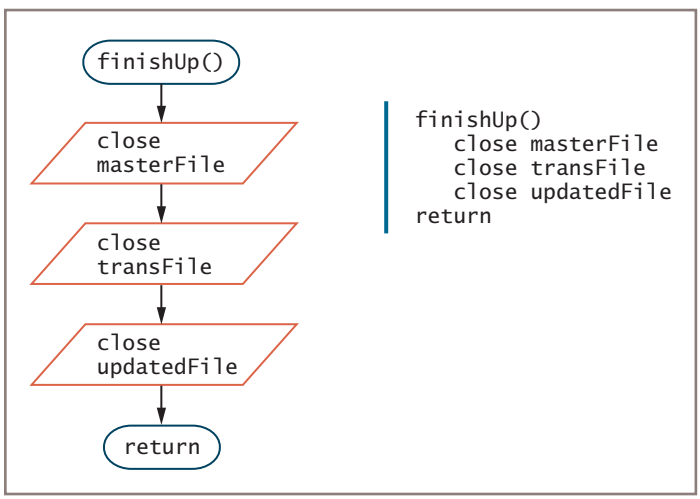


Figure 7-20 The `finishUp()` module for the master-transaction program

TWO TRUTHS & A LIE

Master and Transaction File Processing

1. A master file typically holds temporary data related to transaction file records.
2. A transaction file typically holds data that is used to update a master file.
3. The original version of a master file is the parent file; the updated version is the child file.

The false statement is #1. A master file holds relatively permanent data.

Random Access Files

The files used as examples so far in this chapter are sequential access files, which means that the records are accessed in order from beginning to end. Businesses store data in sequential order when they use the records for **batch processing**, or processing that involves performing the same tasks with many records, one after the other. For example, when a company produces paychecks, the records for the pay period are gathered in a batch and the checks are calculated and printed in sequence. It really doesn't matter whose check is produced first because no checks are distributed to employees until all have been processed. Batch processing usually implies some delay in processing from the time events occur. For example, most companies produce paychecks for time that employees have already worked. Likewise, a customer might order an item but not receive the bill for a month.



Besides indicating a system that works with many records, the term *batch processing* also can mean a system in which many programs can run in sequence without human intervention. Batch processing is an old computing technique, but it is still crucial for such common business tasks as updating records and generating reports.

For many applications, sequential access is inefficient. These applications, known as **real-time** applications, require that a record be accessed immediately while a client is waiting. A program in which the user makes direct requests is an **interactive program**. For example, if a customer telephones a department store with a question about a monthly bill, the customer service representative does not need or want to access every customer account in sequence (perhaps tens of thousands) to find the caller's account. Instead, customer service representatives require **random access files**, files in which records can physically be located in any order. Files in which records must be accessed immediately also are called **instant access files**. Because they enable clients to locate a particular record directly (without reading all of the preceding records), random access files are also called **direct access files**. You can declare a random access file with a statement similar to the following:

```
RandomFile customerFile
```

This statement associates the identifier `customerFile` with a stored file that can be accessed randomly. You can use read, write, and close operations with a random access file just as you can with a sequential file. With random access files, however, you have the additional capability to find a record directly. For example, you might be able to use a statement similar to the following to find customer number 712 on a random access file:

```
seek record 712
```

This feature is particularly useful in real-time, interactive programs. Consider a business with 20,000 customer accounts. When a customer who has the 14,607th record in the file inquires about his balance, it is convenient to access the 14,607th record directly instead of first reading in data for the 14,606 records that precede the requested one. Figure 7-21 illustrates this concept.

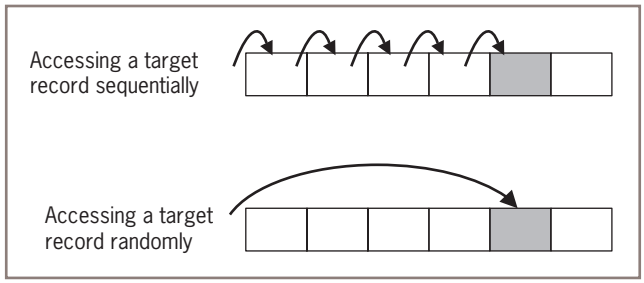


Figure 7-21 Accessing a record in a sequential file and in a random access file

The precise techniques for working with random access files vary among programming languages. You will learn more about this concept when you study a programming language.

TWO TRUTHS & A LIE

Random Access Files

1. A batch program usually uses instant access files.
2. In a real-time application, a record is accessed immediately while a client is waiting.
3. An interactive program usually uses random access files.

The false statement is #1. A batch program usually uses sequential files; interactive programs use random, instant access files.

Chapter Summary

- A computer file is a collection of data stored on a nonvolatile device in a computer system. Although the contents of files differ, each file occupies space on a section of a storage device, and each has a name and specific set of times associated with it. Computer files are organized in directories or folders. A file's complete list of directories is its path.
- Data items in a file usually are stored in a hierarchy. Characters are letters, numbers, and special symbols, such as *A*, 7, and *$*. Fields are data items that each represent a single attribute of a record; they are composed of one or more characters. Records are groups of fields that go together for some logical reason. Files are groups of related records.

- When you use a data file in a program, you must declare the file and open it; opening a file associates an internal program identifier with the name of a physical file on a storage device. A sequential file is a file in which records are stored one after another in some order. When you read from a file, the data is copied into memory. When you write to a file, the data is copied from memory to a storage device. When you are done with a file, you close it.
- A control break program is one that reads a sorted, sequential file and performs special processing based on a change in one or more fields in each record in the file.
- Merging files involves combining two or more files while maintaining the sequential order.
- Some related sequential files are master files that hold relatively permanent data, and transaction files that hold more temporary data. Commonly, you gather transactions for a period of time, store them in a file, and then use them one by one to update matching records in a master file.
- Real-time, interactive applications require random access files in which records can be located in any order. Files in which records must be accessed immediately are also called *instant access files* and *direct access files*.

Key Terms

A **permanent storage device** holds nonvolatile data; examples include hard disks, DVDs, USB drives, and reels of magnetic tape.

A **computer file** is a collection of data stored on a nonvolatile device in a computer system.

Text files contain data that can be read in a text editor.

Binary files contain data that has not been encoded as text.

A **filename** is an identifying name given to a computer file that frequently describes the contents.

A **filename extension** is a group of characters added to the end of a filename that indicates the file type.

A **byte** is a small unit of storage; for example, in a simple text file, a byte holds only one character.

A **kilobyte** is approximately 1,000 bytes.

A **megabyte** is approximately a million bytes.

A **gigabyte** is approximately a billion bytes.

Directories are organization units on storage devices that can contain multiple files as well as additional directories; also called *folders*.

Folders are organization units on storage devices that can contain multiple files as well as additional folders; also called *directories*.

A file's **path** is the combination of its disk drive and the complete hierarchy of directories in which the file resides.

The **data hierarchy** is a framework that describes the relationships between data components. The data hierarchy contains characters, fields, records, and files.

Characters are letters, numbers, and special symbols, such as *A*, *7*, and *$*.

Fields are data items that each represent a single attribute of a record and that are composed of one or more characters.

Records are groups of fields that go together for some logical reason.

Files are groups of related records.

A **database** holds related data items and provides the means for easy retrieval and organization.

Opening a file locates it on a storage device and associates a variable name within your program with the file.

Reading from a file copies data from a file on a storage device into RAM.

A **delimiter** is a character, such as a comma, used to separate fields in a data file.

A **sequential file** is a file in which records are stored one after another in some order.

Sorting is the process of placing records in order by the value in a specific field or fields.

Ascending order describes records placed in order from lowest to highest based on the value in a field.

Descending order describes records placed in order from highest to lowest based on the value in a field.

Writing to a file copies data from RAM to persistent storage.

Closing a file makes it no longer available to an application.

Default input and output devices are those that do not require opening. Usually they are the keyboard and monitor, respectively.

A **backup file** is a copy that is kept in case values need to be restored to their original state.

A **parent file** is a copy of a file before revision.

A **child file** is a copy of a file after revision.

A **control break** is a temporary detour in the logic of a program.

A **control break program** is one in which a change in the value of a variable initiates special actions or causes special or unusual processing to occur.

A **control break report** is a form of output that includes special processing after each group of records.

A **single-level control break** is a break in the logic of a program to perform special processing based on the value of a single variable.

A **control break field** holds a value that causes special processing in a control break program.

Merging files involves combining two or more files while maintaining the sequential order.

A **master file** holds complete and relatively permanent data.

A **transaction file** holds temporary data that is used to update a master file.

To **update a master file** involves making changes to the values in its fields based on transactions.

Batch processing involves performing the same tasks with many records, one after the other.

Real-time applications require that a record be accessed immediately while a client or user is waiting.

An **interactive program** is one in which the user makes direct requests or provides input while a program executes.

A **random access file** is one in which records can be located in any physical order and accessed directly.

Instant access files are random access files in which records can be accessed immediately.

Direct access files are random access files.

Exercises



Review Questions

1. Random access memory is _____.
   a. volatile
   b. permanent
   c. persistent
   d. continual

2. Which is true of text files?

   a. Text files contain data that can be read in a text editor.
   b. Text files commonly contain images and music.
   c. both of the above
   d. none of the above

3. Every file on a storage device has a _____.

   a. name
   b. size
   c. both of the above
   d. none of the above

4. Which of the following is true regarding the data hierarchy?

   a. Fields contain records.
   b. Characters contain fields.
   c. Fields contain files.
   d. Files contain records.

5. The process of _____ a file locates it on a storage device and associates a variable name within your program with the file.

   a. declaring
   b. closing
   c. opening
   d. defining

6. When you write to a file, you _____.

   a. move data from a storage device to memory
   b. copy data from a storage device to memory
   c. move data from memory to a storage device
   d. copy data from memory to a storage device

7. Unlike when you print a report or display information on a screen, when a program's output is a data file, you do not _____.

   a. include explanations or formatting such as dollar signs
   b. open or close the files
   c. output all of the input fields
   d. all of the above

8. When you close a file, it _____.

   a. becomes associated with an internal identifier
   b. cannot be reopened
   c. is no longer available to the program
   d. ceases to exist

9. A file in which records are stored one after another in order based on the contents of a field is a(n) _____ file.

   a. temporal
   b. alphabetical
   c. random
   d. sequential

10. When you combine two or more sorted files while maintaining their sequential order based on a field, you are _____ the files.

    a. tracking
    b. collating
    c. merging
    d. absorbing

11. A control break occurs when a program _____.

    a. pauses to perform special processing based on the value of a field
    b. ends prematurely, before all records have been processed
    c. takes one of two alternate courses of action for every record
    d. passes logical control to a module contained within another program

12. Which of the following is an example of a control break report?

    a. a list of all customers of a business in zip code order, with a count of the number of customers who reside in each zip code
    b. a list of all students in a school, arranged in alphabetical order, with a total count at the end of the report
    c. a list of all employees in a company, with a "Retain" or "Dismiss" message following each employee record
    d. a list of medical clinic patients who have not seen a doctor for at least two years

13. A control break field _____.

    a. always is output prior to any group of records on a control break report
    b. always is output after any group of records on a control break report
    c. never is output on a report
    d. causes special processing to occur

14. Whenever a control break occurs during record processing in any control break program, you must _____.

    a. declare a control break field
    b. set the control break field to 0
    c. update the value in the control break field
    d. output the control break field

15. Assume that you are writing a program to merge two files named FallStudents and SpringStudents. Each file contains a list of students enrolled in a programming logic course during the semester indicated, and each file is sorted in student ID number order. After the program compares two records and subsequently writes a Fall student to output, the next step is to _____.

    a. read a SpringStudents record
    b. read a FallStudents record
    c. write a SpringStudents record
    d. write another FallStudents record

16. When you merge records from two or more sequential files, the usual case is that the records in the files _____.

    a. contain the same data
    b. have the same format
    c. are identical in number
    d. are sorted on different fields

17. A file that holds more permanent data than a transaction file is a _____ file.

    a. master
    b. primary
    c. key
    d. mega-

18. A transaction file is often used to _____ another file.

    a. augment
    b. remove
    c. verify
    d. update

19. The saved version of a file that does not contain the most recently applied transactions is known as a _____ file.

    a. master
    b. child
    c. parent
    d. relative

20. Random access files are used most frequently in all of the following except _____.

    a. interactive programs
    b. batch processing
    c. real-time applications
    d. programs requiring direct access



Programming Exercises

Your downloadable files for Chapter 7 include one or more comma-delimited sample data files for each exercise in this section and the Game Zone section. You might want to use these files in any of several ways:

- You can look at the file contents to better understand the types of data each program uses.
- You can use the file contents as sample data when you desk-check the logic of your flowcharts or pseudocode.
- You can use the files as input files if you implement the solutions in a programming language and write programs that accept file input.
- You can use the data as guides for entering appropriate values if you implement the solutions in a programming language and write interactive programs.

1. Pendant Publishing edits multi-volume manuscripts for many authors. For each volume, they want a label that contains the author's name, the title of the work, and a volume number in the form *Volume 9 of 9*. For example, a set of three volumes requires three labels: *Volume 1 of 3, Volume 2 of 3*, and *Volume*

3 of 3. Design an application that reads records that contain an author's name, the title of the work, and the number of volumes. The application must read the records until `eof` is encountered and produce enough labels for each work.

2. Geraldine's Landscaping Service and Gerard's Lawn Maintenance are merging their businesses and want to merge their customer files. Each file contains a customer number, last name, address, and property area in square feet, and each file is in customer number order. Design the logic for a program that merges the two files into one file containing all customers. Assume there are no identical customer numbers.

3. Laramie Park District has files of participants in its summer and winter programs this year. Each file is in participant ID number order and contains additional fields for first name, last name, age, and class taken (for example, *Beginning Swimming*).

   a. Design the logic for a program that merges the files for summer and winter programs to create a list of the first and last names of all participants for the year in ID number order.
   b. Modify the program so that if a participant has more than one record, the participant's ID number and name are output only once.
   c. Modify the program so that if a participant has more than one record, the ID number and name are output only once along with a count of the total number of classes the participant has taken.

4. The Apgar Medical group keeps a patient file for each doctor in the office. Each record contains the patient's first and last name, home address, and birth year. The records are sorted in ascending birth year order. Two doctors, Dr. Best and Dr. Cushing, have formed a partnership.

   a. Design the logic that produces a merged list of patients' names in ascending order by birth year.
   b. Modify the program so that it does not display patients' names, but only produces a count of the number of patients born each year.

5. Gimme Shelter Roofers maintains a file of past customers, including a customer number, name, address, date of job, and price of job. It also maintains a file of estimates given for jobs not yet performed; this file contains a customer number, name, address, proposed date of job, and proposed price. Each file is in customer number order. Design the logic that merges the two files to produce one combined file of all customers whether past or proposed with no duplicates; when a customer who has been given an estimate is also a past customer, use the proposed data.

6. The Curl Up and Dye Beauty Salon maintains a master file that contains a record for each of its clients. Fields in the master file include the client's ID number, first name, last name, and total amount spent this year. Every week, a transaction file is produced. It contains a customer's ID number, the service received (for example, *Manicure*), and the price paid. Each file is sorted in ID number order.

   a. Design the logic for a program that matches the master and transaction file records and updates the total paid for each client by adding the current week's price paid to the cumulative total. Not all clients purchase services each week. The output is the updated master file and an error report that lists any transaction records for which no master record exists.

   b. Modify the program to output a coupon for a free haircut each time a client exceeds $1,000 in services. The coupon, which contains the client's name and an appropriate congratulatory message, is output during the execution of the update program when a client total surpasses $1,000. Make sure that only one coupon is printed per client, even if the client has purchased multiple services to pass the $1,000 cutoff value.

7. The Timely Talent Temporary Help Agency maintains an employee master file that contains an employee ID number, last name, first name, address, and hourly rate for each temporary worker. The file has been sorted in employee ID number order. Each week, a transaction file is created with a job number, address, customer name, employee ID, and hours worked for every job filled by Timely Talent workers. The transaction file is also sorted in employee ID order.

   a. Design the logic for a program that matches the current week's transaction file records to the master file and outputs one line for each transaction, indicating job number, employee ID number, hours worked, hourly rate, and gross pay. Assume that each temporary worker works at most one job per week. Output one line for each worker, even if the worker has completed no jobs during the current week.

   b. Modify the help agency program to output lines only for workers who have completed at least one job during the current week.

   c. Modify the help agency program so that any temporary worker can work any number of separate jobs during the week. Output one line for each job that week.

   d. Modify the help agency program so that it accumulates the worker's total pay for all jobs in a week and outputs one line per worker.

Performing Maintenance

1. A file named MAINTENANCE07-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.

Find the Bugs

1. Your downloadable files for Chapter 7 include DEBUG07-01.txt, DEBUG07-02.txt, and DEBUG07-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 7 include a file named DEBUG07-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.

Game Zone

1. The International Rock Paper Scissors Society holds regional and national championships. Each region holds a semifinal competition in which contestants play 500 games of Rock Paper Scissors. The top 20 competitors in each region are invited to the national finals. Assume that you are provided with files for the East, Midwest, and Western regions. Each file contains the following fields for the top 20 competitors: last name, first name, and number of games won. The records in each file are sorted in alphabetical order. Merge the three files to create a file of the top 60 competitors who will compete in the national championship.

2. In the Game Zone section of Chapter 5, you designed a guessing game in which the application generates a random number and the player tries to guess it. After each guess, you displayed a message indicating whether the player's guess was correct, too high, or too low. When the player eventually guessed the correct number, you displayed a score that represented a count of the number of required guesses. Modify the game so that when it starts, the player enters his or her name. After a player plays the game exactly five times, save the best (lowest) score from the five games to a file. If the player's name already exists in the file, update the record with the new lowest score. If the player's name does not already exist in the file, create a new record for the player. After the file is updated, display all the best scores stored in the file.

CHAPTER 8

Advanced Data Handling Concepts

Upon completion of this chapter, you will be able to:

- Understand the need for sorting data
- Describe the bubble sort algorithm
- Appreciate the nuances of sorting multifield records
- Describe the insertion sort algorithm
- Use multidimensional arrays
- Understand indexed files and linked lists

Understanding the Need for Sorting Data

Stored data records exist in some type of order; that is, one record is first, another second, and so on. When records are in **sequential order**, they are arranged one after another on the basis of the value in a particular field. Examples include employee records stored in numeric order by Social Security number or department number, or in alphabetical order by last name or department name. Even if the records do not seem to be stored in any particular order—for example, if they are in the order in which a clerk felt like entering them—they still exist one after the other, although probably not in the order desired for processing or viewing. Data records that are stored randomly, or that are not in the order needed for a particular application, need to be sorted. As you learned in Chapter 7, sorting records is the process of placing them in order based on the contents of one or more fields. You can sort data either in ascending order, arranging records from lowest to highest value within a field, or in descending order, arranging records from highest to lowest value.



The sorting process usually is reserved for a relatively small number of data items. If thousands of customer records are stored, and they frequently need to be accessed in order based on different fields (alphabetical order by customer name one day, zip code order the next), the records would probably not be sorted at all, but would be indexed or linked. You learn about indexing and linking later in this chapter.

Examples of occasions when you would need to sort records might include:

- A college stores student records in ascending order by student ID number, but the registrar wants to view the data in descending order by credit hours earned so he can contact students who are close to graduation.
- A department store maintains customer records in ascending order by customer number, but at the end of a billing period, the credit manager wants to contact customers whose balances are 90 or more days overdue. The manager wants to list these overdue customers in descending order by the amount owed, so the customers with the largest debt can be contacted first.
- A sales manager keeps records for her salespeople in alphabetical order by last name, but she needs to list the annual sales figure for each salesperson so she can determine the median annual sale amount.



The **median** value in a list is the value of the middle item when the values are listed in order. (If the list contains an even number of values, the median is halfway between the two middle values.) The median is not the same as the arithmetic average, or **mean**. The median is often used as a statistic because it represents a more typical case—half the values are below it and half are above it. Unlike the median, the mean is skewed by a few very high or low values.

- A store manager wants to create a control break report in which individual sales are listed in order in groups by their department. As you learned in Chapter 7, when you create a control break report, the records must have been sorted in order by the control break field.

When computers sort data, they always use numeric values to make comparisons between values. This is clear when you sort records by fields such as a numeric customer ID or balance due. However, even alphabetic sorts are numeric, because computer data items are stored as numbers using a series of 0s and 1s. Ordinary computer users seldom think about the numeric codes behind the letters, numbers, and punctuation marks they enter from their keyboards or see on a monitor. However, they see the consequence of the values behind letters when they see data sorted in alphabetical order. In every popular computer coding scheme, *B* is numerically one greater than *A*, and *y* is numerically one less than *z*. Because *A* is always less than *B*, *B* is always less than *C*, and so on, alphabetic sorts are ascending sorts. Unfortunately, your system dictates whether an uppercase *A* is represented by a number that is greater or smaller than the number representing a lowercase *a*. Therefore, if data items are not input using consistent capitalization, then the programmer must make conversions before sorting them.



The most popular coding schemes include ASCII, Unicode, and EBCDIC. In each code, a number represents a specific computer character. Appendix A contains additional information about these codes.

As a professional programmer, you might never have to write a program that sorts data, because of one or more of the following reasons:

- Programmers in your organization might already have created a sorting method you can use.
- Your organization might have purchased a generic, "canned" sorting program.
- You might be using a language for which the compiler includes built-in methods that can sort data for you.

However, even if you never have to create code that sorts data, it is beneficial to understand the sorting process so that you can write a special-purpose sort when needed. Understanding the sorting process also improves your array-manipulating skills.

TWO TRUTHS & A LIE

Understanding the Need for Sorting Data

1. When you sort data in ascending order, you arrange records from lowest to highest based on the value in a specific field.
2. Normal alphabetical order, in which *A* precedes *B*, is descending order.
3. When computers sort data, they use numeric values to make comparisons, even when string values are compared.

The false statement is #2. Normal alphabetical order is ascending.

Using the Bubble Sort Algorithm

When you learn a method such as sorting, programmers say you are learning an algorithm. An **algorithm** is a list of instructions that accomplish a task. In this section, you will learn about the bubble sort algorithm for sorting a list of simple values; later in this chapter, you will learn more about how multifield records are sorted.

One of the simplest sorting techniques to understand is a **bubble sort**, in which items in a list are compared with each other in pairs. You can use a bubble sort to arrange data items in either ascending or descending order. A bubble sort examines items in a list, and when an item is out of order, it trades places, or is swapped, with the item below it. With an ascending bubble sort, after each adjacent pair of items in a list has been compared once, the largest item in the list will have "sunk" to the bottom. After many passes through the list, the smallest items rise to the top like bubbles in a carbonated drink. A bubble sort is sometimes called a *sinking sort*. To understand the bubble sort algorithm, you first must learn about swapping values.

Understanding Swapping Values

A concept central to many sorting algorithms, including the bubble sort, is the idea of swapping values. When you **swap values** stored in two variables, you exchange their values; you set the first variable equal to the value of the second, and the second variable equal to the value of the first. However, there is a trick to swapping any two values. Assume that you have declared two variables as follows:

```
num score1 = 90
num score2 = 85
```

You want to swap the values so that `score1` is 85 and `score2` is 90. If you first assign `score1` to `score2` using a statement such as `score2 = score1`, both `score1` and `score2` hold 90, and the value 85 is lost. Similarly, if you first assign `score2` to `score1` using a statement such as `score1 = score2`, both variables hold 85, and the value 90 is lost.

To correctly swap two values, you create a temporary variable to hold a copy of one of the scores so it doesn't get lost. Then, you can accomplish the swap as shown in Figure 8-1. First, the value in `score2`, 85, is assigned to a temporary holding variable named `temp`. Then, the `score1` value, 90, is assigned to `score2`. At this point, both `score1` and `score2` hold 90. Then, the 85 in `temp` is assigned to `score1`. Therefore, after the swap process, `score1` holds 85 and `score2` holds 90.

In Figure 8-1, you could accomplish identical results by assigning `score1` to `temp`, assigning `score2` to `score1`, and finally assigning `temp` to `score2`.



Watch the video *Swapping Values*.

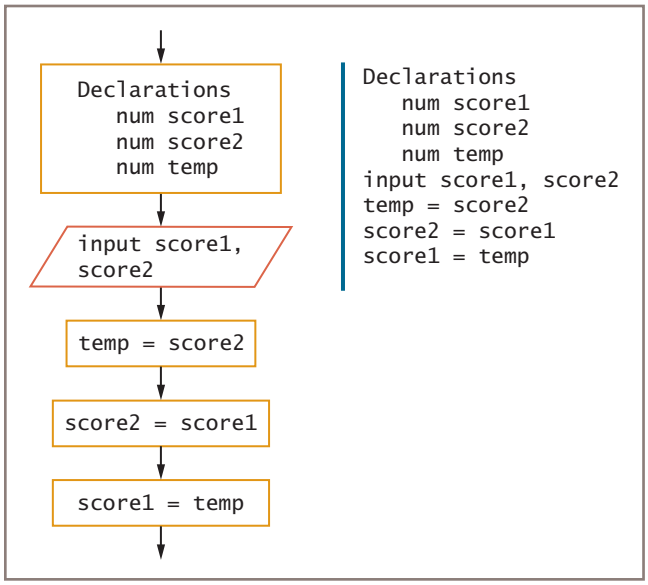


Figure 8-1 Program segment that swaps two values

Understanding the Bubble Sort

Assume that you want to sort five student test scores in ascending order. Figure 8-2 shows a program in which a constant is declared to hold an array's size, and then the array is declared to hold five scores. (The other variables and constants, which are shaded in the figure, will be discussed in the next paragraphs when they are used.) The program calls three main procedures—one to input the five scores, one to sort them, and the final one to display the sorted result.



In Chapter 6, you learned that many modern languages allow you to use a built-in value as an array size, which relieves you of the requirement to declare a constant for the size. Because the name of the built-in value varies among programming languages, the examples in this chapter use a declared, named constant.

Figure 8-3 shows the `fillArray()` method. Within the method, a subscript, `x`, is initialized to 0 and each array element is filled in turn. After a user enters five scores, control returns to the main program.

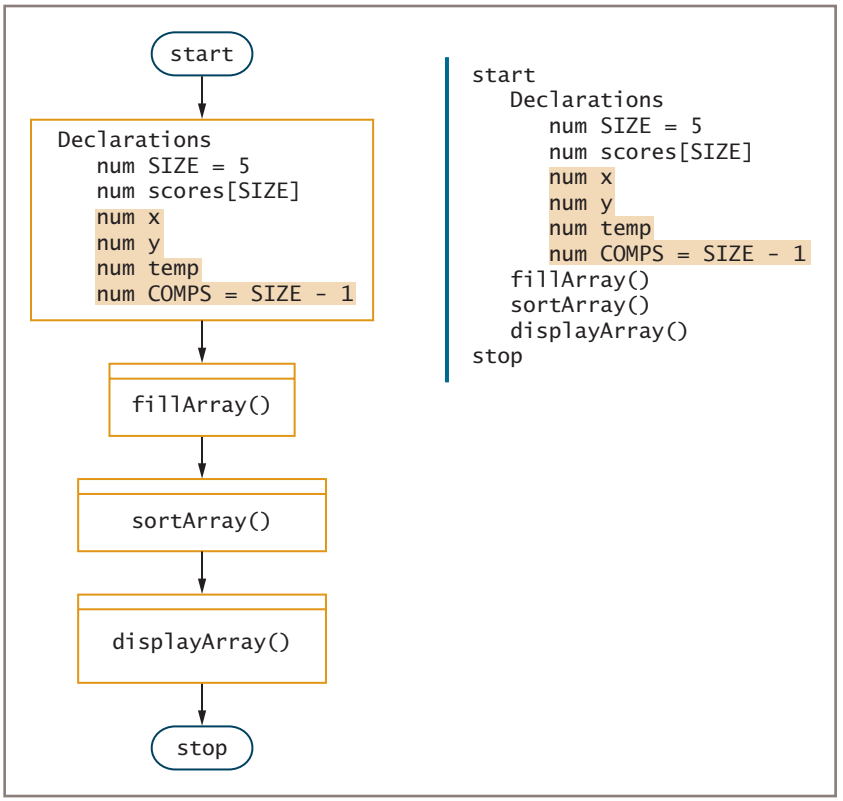


Figure 8-2 Mainline logic for program that accepts, sorts, and displays scores

The `sortArray()` method in Figure 8-4 sorts the array elements by making a series of comparisons of adjacent element values and swapping them if they are out of order. To begin sorting this list of scores, you compare the first two scores, `scores[0]` and `scores[1]`. If they are out of order—that is, if `scores[0]` is larger than `scores[1]`—you want to reverse their positions, or swap their values.

For example, assume that the five entered scores are:

```
scores[0] = 90
scores[1] = 85
scores[2] = 65
scores[3] = 95
scores[4] = 75
```

In this list, `scores[0]` is 90 and `scores[1]` is 85; you want to exchange the values of the two elements so that the smaller value ends up earlier in the array. You call the `swap()` method, which places the scores in slightly better order than they were originally. Figure 8-5 shows the `swap()` method. This module switches any two adjacent elements in the `scores` array.

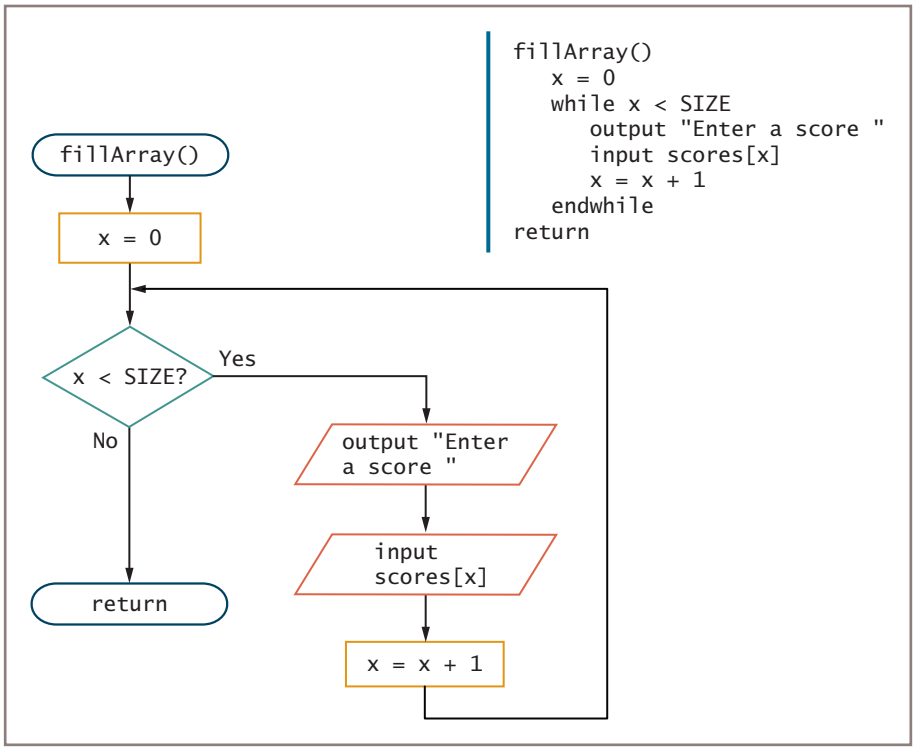


Figure 8-3 The `fillArray()` method

In Figure 8-4, the number of comparisons made is based on the value of the constant named `COMPS`, which was initialized to the value of `SIZE - 1`. That is, for an array of size 5, the `COMPS` constant will be 4. Therefore, the following comparisons are made:

```
scores[0] > scores[1]
scores[1] > scores[2]
scores[2] > scores[3]
scores[3] > scores[4]
```

Each element in the array is compared to the element that follows it. When `x` becomes `COMPS`, the `while` loop ends. If the loop continued when `x` became equal to `COMPS`, then the next comparison would use `scores[4]` and `scores[5]`. This would cause an error because the highest allowed subscript in a five-element array is 4. You must evaluate the expression `scores[x] > scores[x + 1]` four times—when `x` is 0, 1, 2, and 3.

For an ascending sort, you need to perform the `swap()` method whenever any given element of the `scores` array has a value greater than the next element. For any `x`, if the *x*th element is not greater than the element at position `x + 1`, the swap should not take place. For example, when `scores[x]` is 90 and `scores[x + 1]` is 85, a swap should occur. On the other hand, when `scores[x]` is 65 and `scores[x + 1]` is 95, then no swap should occur.

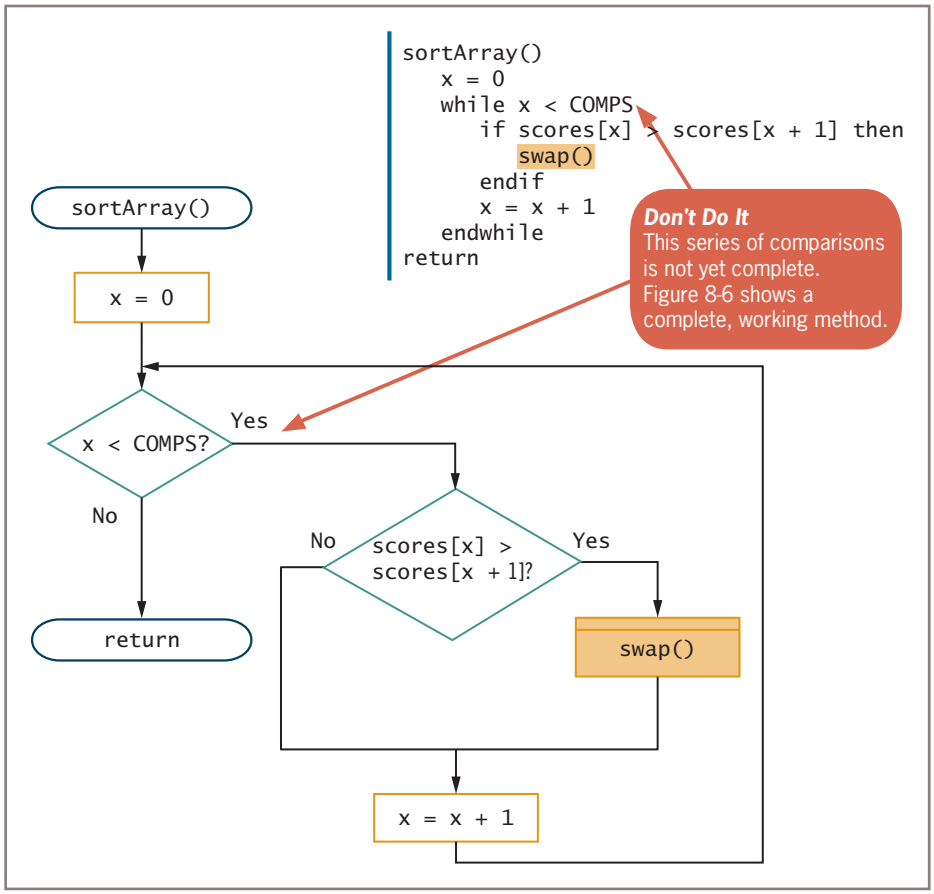


Figure 8-4 The incomplete `sortArray()` method

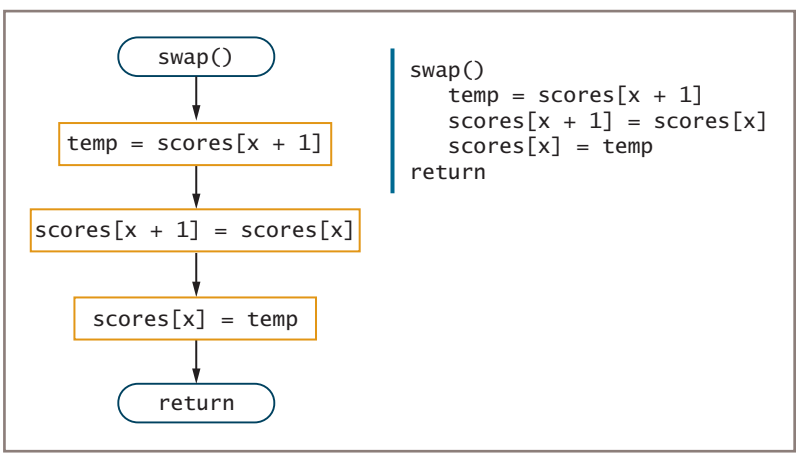


Figure 8-5 The `swap()` method

For a descending sort in which you want to end up with the highest value first, you would write the decision so that you perform the switch when `scores[x]` is *less than* `scores[x + 1]`.

As a complete example of how this application works using an ascending sort, suppose that you have these original scores:

```
scores[0] = 90
scores[1] = 85
scores[2] = 65
scores[3] = 95
scores[4] = 75
```

The logic of the `sortArray()` method proceeds like this:

1. Set `x` to 0.
2. The value of `x` is less than 4 (`COMPS`), so enter the loop.
3. Compare `scores[x]`, 90, to `scores[x + 1]`, 85. The two scores are out of order, so they are swapped.

   The list is now:

   ```
   scores[0] = 85
   scores[1] = 90
   scores[2] = 65
   scores[3] = 95
   scores[4] = 75
   ```

4. After the swap, add 1 to `x`, so `x` is 1.
5. Return to the top of the loop. The value of `x` is less than 4, so enter the loop a second time.
6. Compare `scores[x]`, 90, to `scores[x + 1]`, 65. These two values are out of order, so swap them.

   Now the result is:

   ```
   scores[0] = 85
   scores[1] = 65
   scores[2] = 90
   scores[3] = 95
   scores[4] = 75
   ```

7. Add 1 to `x`, so `x` is now 2.
8. Return to the top of the loop. The value of `x` is less than 4, so enter the loop.
9. Compare `scores[x]`, 90, to `scores[x + 1]`, 95. These values are in order, so no swap is necessary.
10. Add 1 to `x`, making it 3.
11. Return to the top of the loop. The value of `x` is less than 4, so enter the loop.

12. Compare `scores[x]`, 95, to `scores[x + 1]`, 75. These two values are out of order, so swap them.

    Now the list is as follows:

```
scores[0] = 85
scores[1] = 65
scores[2] = 90
scores[3] = 75
scores[4] = 95
```

13. Add 1 to `x`, making it 4.

14. Return to the top of the loop. The value of `x` is 4, so do not enter the loop again.

When `x` reaches 4, every element in the list has been compared with the one adjacent to it. The highest score, 95, has "sunk" to the bottom of the list. However, the scores still are not in order. They are in slightly better ascending order than they were when the process began, because the largest value is at the bottom of the list, but they are still out of order. You need to repeat the entire procedure so that 85 and 65 (the current `scores[0]` and `scores[1]` values) can switch places, and 90 and 75 (the current `scores[2]` and `scores[3]` values) can switch places. Then, the scores will be 65, 85, 75, 90, and 95. You will have to go through the list yet again to swap 85 and 75.

As a matter of fact, if the scores had started in the worst possible order (95, 90, 85, 75, 65), the comparison process would have to take place four times. In other words, you would have to pass through the list of values four times, making appropriate swaps, before the numbers would appear in perfect ascending order. You need to place the loop in Figure 8-4 within another loop that executes four times.

Figure 8-6 shows the complete logic for the `sortArray()` module. The module uses a loop control variable named `y` to cycle through the list of scores four times. (The initialization, comparison, and alteration of this loop control variable are shaded in the figure.) With an array of five elements, it takes four comparisons to work through the array once, comparing each pair, and it takes four sets of those comparisons to ensure that every element in the entire array is in sorted order. In the `sortArray()` method in Figure 8-6, `x` must be reset to 0 for each new value of `y` so that the comparisons always start at the top of the list.

When you sort the elements in an array this way, you use nested loops—an inner loop that swaps out-of-order pairs, and an outer loop that goes through the list multiple times. The general rules for making comparisons with the bubble sort are:

- The greatest number of pair comparisons you need to make during each loop is *one less* than the number of elements in the array. You use an inner loop to make the pair comparisons.
- The number of times you need to process the list of values is *one less* than the number of elements in the array. You use an outer loop to control the number of times you walk through the list.

As an example, if you want to sort a 10-element array, you make nine pair comparisons on each of nine iterations through the loop, executing a total of 81 score comparison statements.

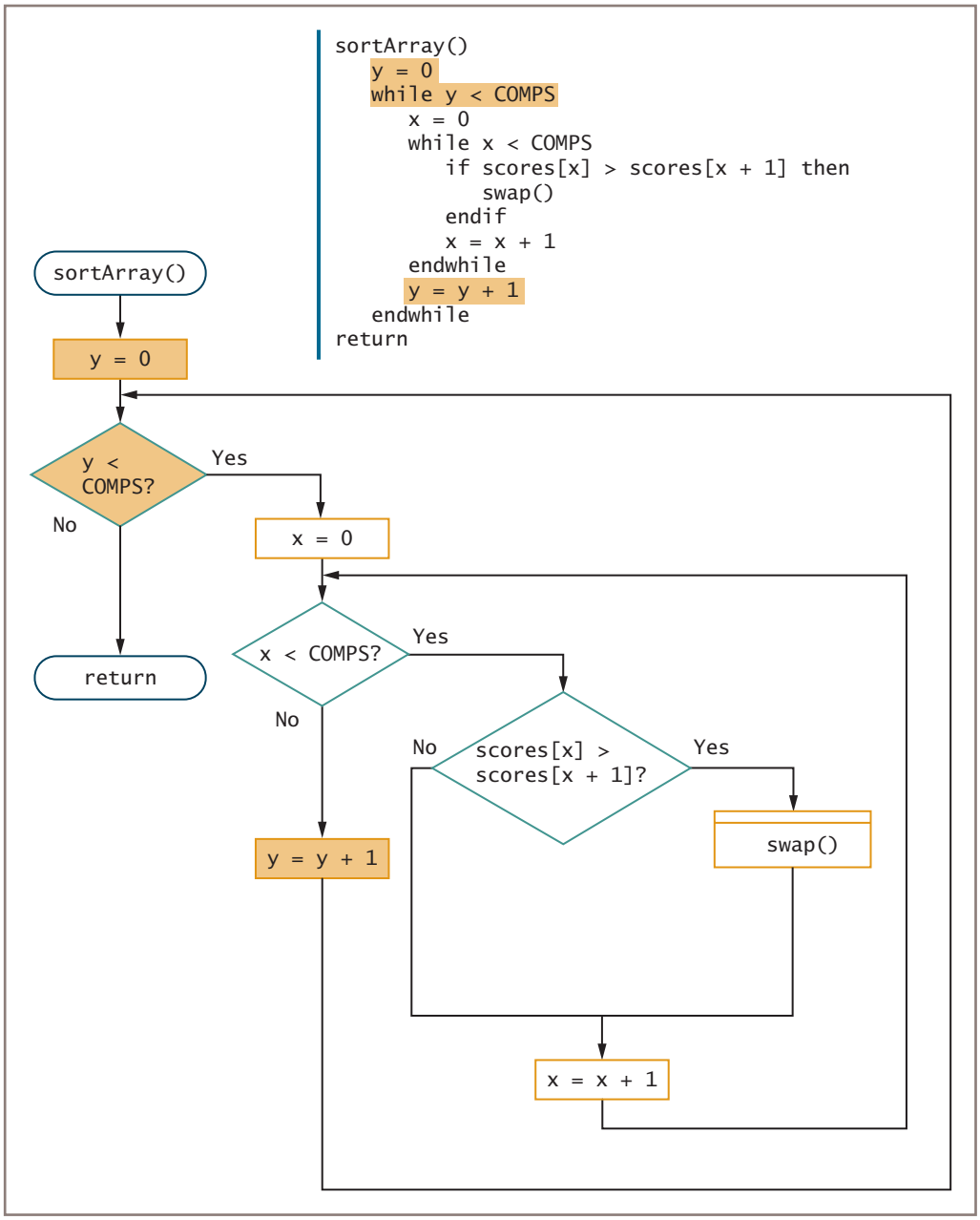


Figure 8-6 The completed `sortArray()` method

The last method called by the score-sorting program in Figure 8-2 is the one that displays the sorted array contents. Figure 8-7 shows this method.

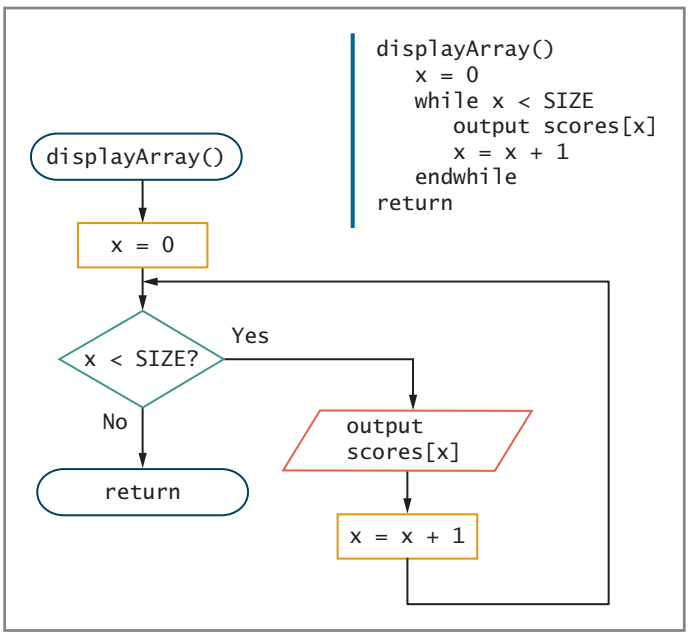


Figure 8-7 The `displayArray()` method

Watch the video *The Bubble Sort*.

Sorting a list of variable size

In the score-sorting program in the previous section, a `SIZE` constant was initialized to the number of elements to be sorted at the start of the program. At times, however, you don't want to create such a value because you might not know how many array elements will hold valid values. For example, on one program run you might want to sort only three or four scores, and on another run you might want to sort 20. In other words, the size of the list to be sorted might vary.

To keep track of the number of elements stored in an array, you can count the data items and create an application similar to the one shown in Figure 8-8 which can sort up to 100 items. As in the original version of the program, you call the `fillArray()` method, and when you input each score, you increase `x` by 1 to place each new score into a successive element of the `scores` array. After you input one value and place it in the first element of the `scores` array, `x` is 1. After a second score is input and placed in `scores[1]`, `x` is 2, and so on. After you reach the end of input, `x` holds the number of scores that have been placed in the array, so you can store `x` in `numberOfEls`, and compute `comparisons` as

numberOfEls - 1. With this approach, it doesn't matter if there are not enough values to fill the scores array. The sortArray() and displayArray() methods use comparisons and numberOfEls instead of COMPS and SIZE to process the array. For example, if 35 scores are input, numberOfEls will be set to 35 in the fillArray() module, and when the program sorts, it will use 34 as a cutoff point for the number of pair comparisons to make. The sorting program will never make pair comparisons on array elements 36 through 100—those elements will just "sit there," never being involved in a comparison or swap.



In the fillArray() method in Figure 8-8, notice that a priming read has been added to the method. If the user enters the QUIT value at the first input, then the number of elements to be sorted will be 0.

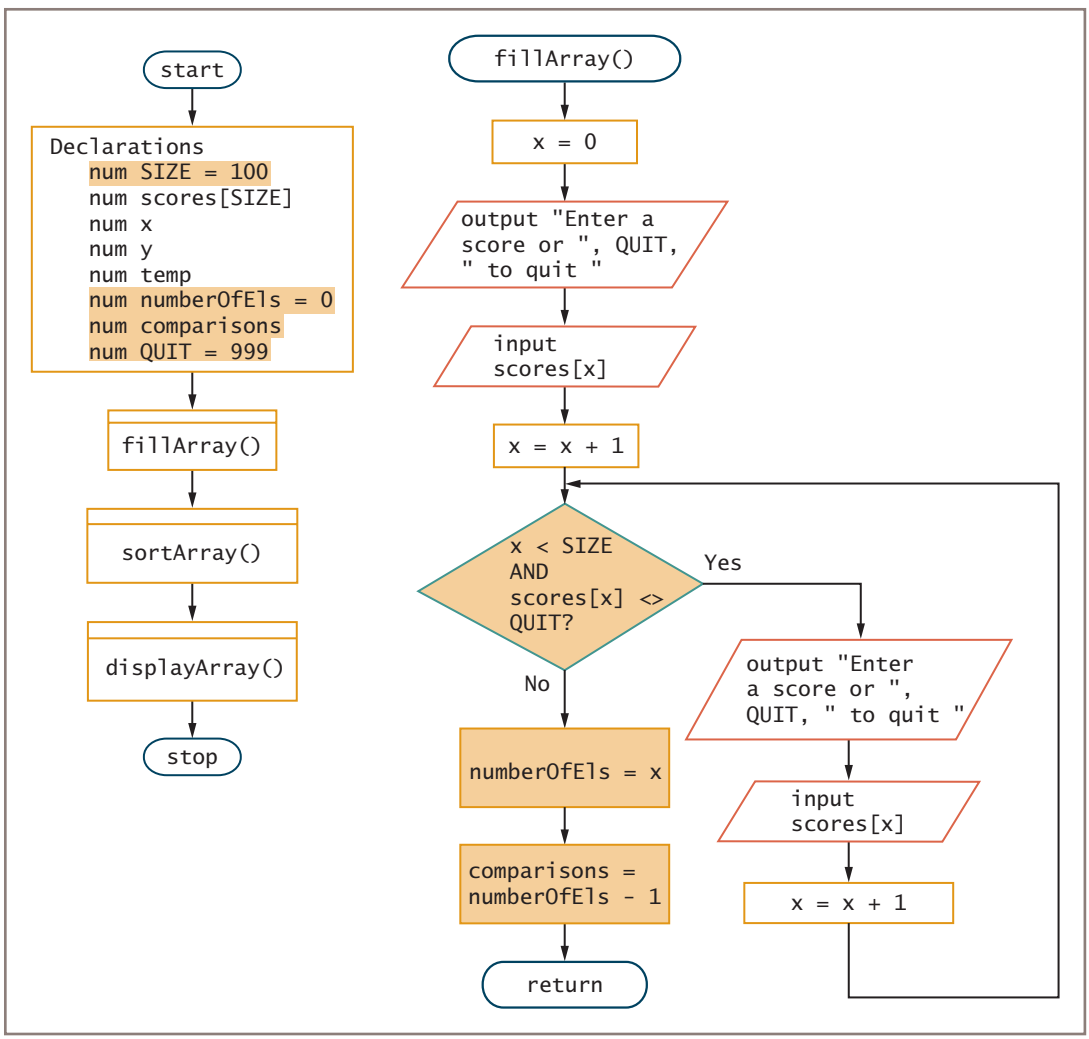


Figure 8-8 Score-sorting application in which number of elements to sort can vary *(continues)*

(continued)

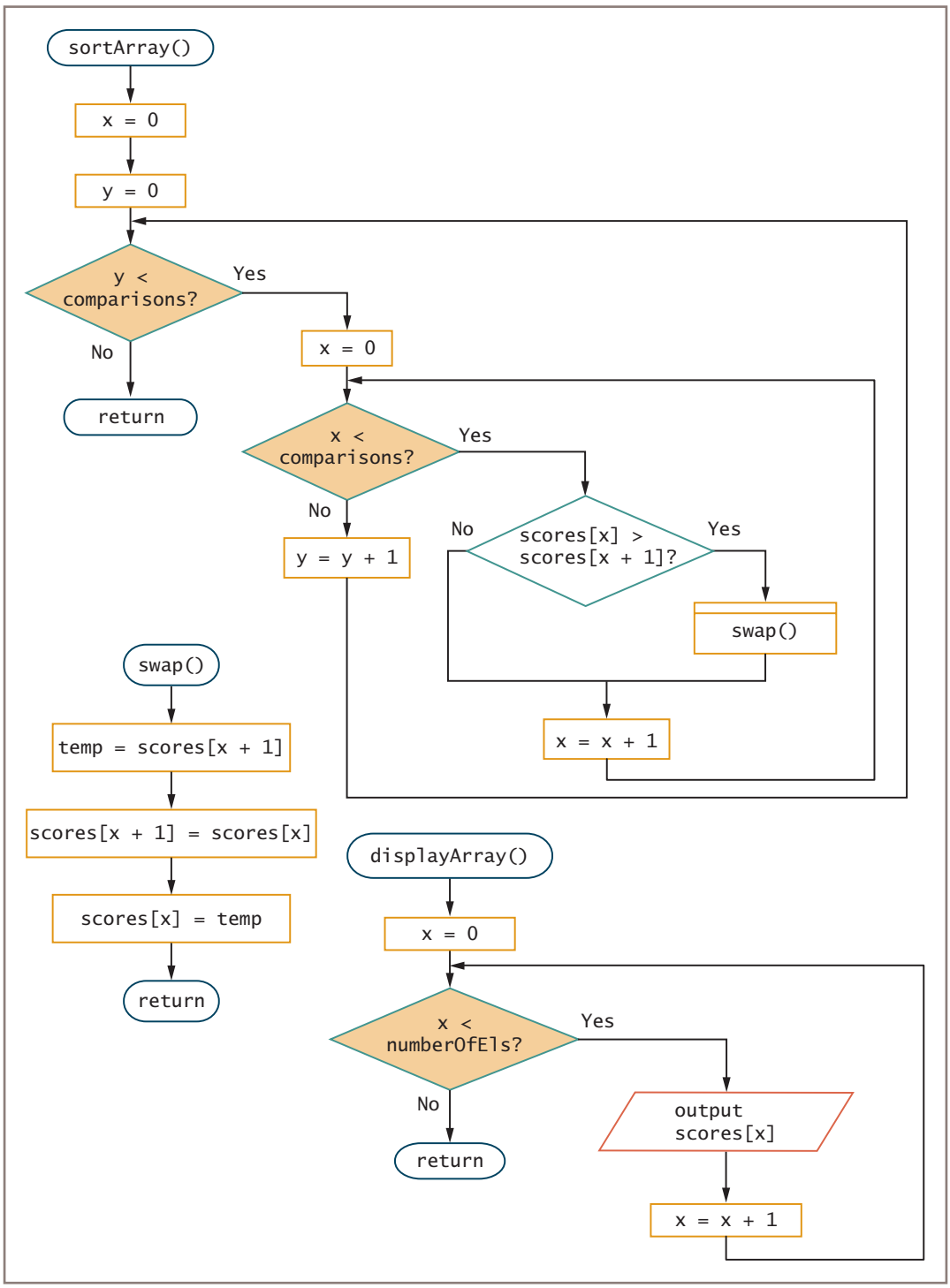


Figure 8-8 Score-sorting application in which number of elements to sort can vary *(continues)*

(continued)

```
start
   Declarations
      num SIZE = 100
      num scores[SIZE]
      num x
      num y
      num temp
      num numberOfEls = 0
      num comparisons
      num QUIT = 999
   fillArray()
   sortArray()
   displayArray()
stop

fillArray()
   x = 0
   output "Enter a score or ", QUIT, " to quit "
   input scores[x]
   x = x + 1
   while x < SIZE AND scores[x] <> QUIT
      output "Enter a score or ", QUIT, " to quit "
      input scores[x]
      x = x + 1
   endwhile
   numberOfEls = x
   comparisons = numberOfEls - 1
return

sortArray()
   x = 0
   y = 0
   while y < comparisons
      x = 0
      while x < comparisons
         if scores[x] > scores[x + 1] then
            swap()
         endif
         x = x + 1
      endwhile
      y = y + 1
   endwhile
return

swap()
   temp = scores[x + 1]
   scores[x + 1] = scores[x]
   scores[x] = temp
return

displayArray()
   x = 0
   while x < numberOfEls
      output scores[x]
      x = x + 1
   endwhile
return
```

Figure 8-8 Score-sorting application in which number of elements to sort can vary

When you count the input values and use the `numberOfEls` variable, it does not matter if there are not enough scores to fill the array. However, an error occurs if you attempt to store more values than the array can hold. When you don't know how many elements will be stored in an array, you must overestimate the number of elements you declare.

Refining the bubble sort to reduce unnecessary comparisons

You can make some improvements to the bubble sort created in the previous sections. As illustrated in Figure 8-8, when you perform the sorting module for a bubble sort, you pass through a list, making comparisons and swapping values if two adjacent values are out of order. If you are performing an ascending sort and you have made one pass through the list, the largest value is guaranteed to be in its correct final position at the bottom of the list. Similarly, the second-largest element is guaranteed to be in its correct second-to-last position after the second pass through the list, and so on. If you continue to compare every element pair on every pass through the list, you are comparing elements that are already guaranteed to be in their final correct position. In other words, after the first pass through the list, you no longer need to check the bottom element; after the second pass, you don't need to check the two bottom elements.

On each pass through the array, you can afford to stop your pair comparisons one element sooner. You can avoid comparing the values that are already in place by creating a new variable, `pairsToCompare`, and setting its initial value to `numberOfEls - 1`. On the first pass through the list, every pair of elements is compared, so `pairsToCompare` *should* equal `numberOfEls - 1`. In other words, with five array elements to sort, four pairs are compared, and with 50 elements to sort, 49 pairs are compared. On each subsequent pass through the list, `pairsToCompare` should be reduced by 1; for example, after the first pass is completed, it is not necessary to check the bottom element. See Figure 8-9 to examine the use of the `pairsToCompare` variable.

Refining the bubble sort to eliminate unnecessary passes

You could also improve the bubble sort module in Figure 8-9 by reducing the number of passes through the array when possible. If array elements are badly out of order, many passes through the list are required to place it in order; it takes one fewer pass than the value in `numberOfEls` to complete all the comparisons and swaps needed to sort the list. However, when the array elements are in order or nearly in order to start, all the elements might be correctly arranged after only a few passes through the list. All subsequent passes result in no swaps. For example, assume that the original scores are as follows:

```
scores[0] = 65
scores[1] = 75
scores[2] = 85
scores[3] = 90
scores[4] = 95
```

The original bubble sort module in Figure 8-9 would pass through the array list four times, making four sets of pair comparisons. It would always find that each `scores[x]` is *not*

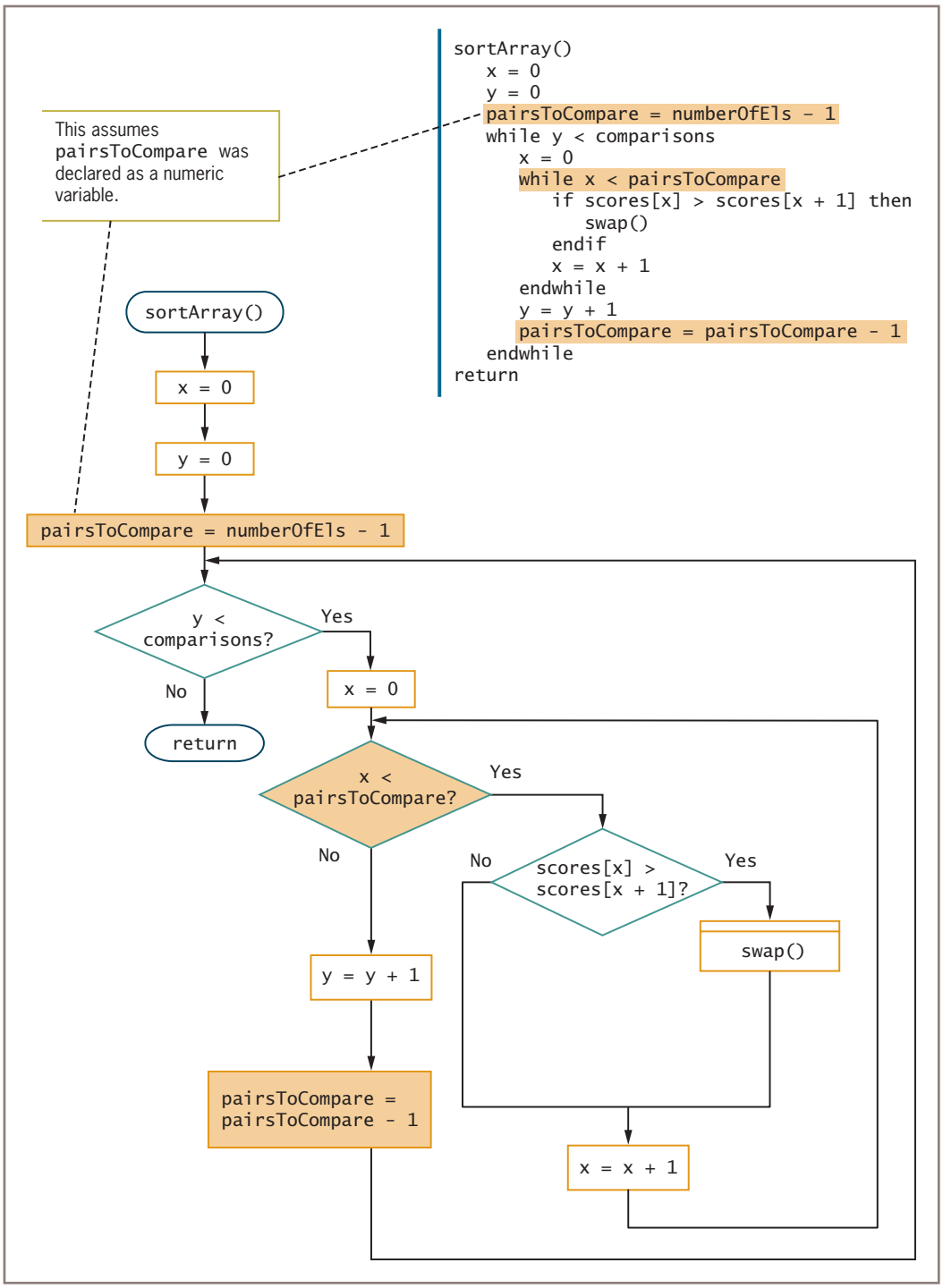


Figure 8-9 Flowchart and pseudocode for `sortArray()` method using `pairsToCompare` variable

greater than the corresponding `scores[x + 1]`, so no switches would ever be made. The scores would end up in the proper order, but they *were* in the proper order in the first place; therefore, a lot of time would be wasted.

A possible remedy is to add a flag variable set to a "continue" value on any pass through the list in which any pair of elements is swapped (even if just one pair), and which holds a different "finished" value when no swaps are made—that is, when all elements in the list are already in the correct order. For example, you can create a variable named `didSwap` and set it to `"No"` at the start of each pass through the list. You can change its value to `"Yes"` each time the `swap()` module is performed (that is, each time a switch is necessary).

If you make it through the entire list of pairs without making a switch, the `didSwap` flag will *not* have been set to `"Yes"`, meaning that no swap has occurred and that the array elements must already be in the correct order. This situation might occur on the first or second pass through the array list, or, depending on the values stored, it might not occur until a much later pass. Once the array elements are in the correct order, you can stop making passes through the list.

Figure 8-10 illustrates a module that sorts scores and uses a `didSwap` flag. At the beginning of the `sortArray()` module, initialize `didSwap` to `"Yes"` before entering the comparison loop the first time. Then, immediately set `didSwap` to `"No"`. When a switch occurs—that is, when the `swap()` module executes—set `didSwap` to `"Yes"`.



With the addition of the flag variable in Figure 8-10, you no longer need the variable `y`, which was keeping track of the number of passes through the list. Instead, you keep going through the list until you can make a complete pass without any swaps.

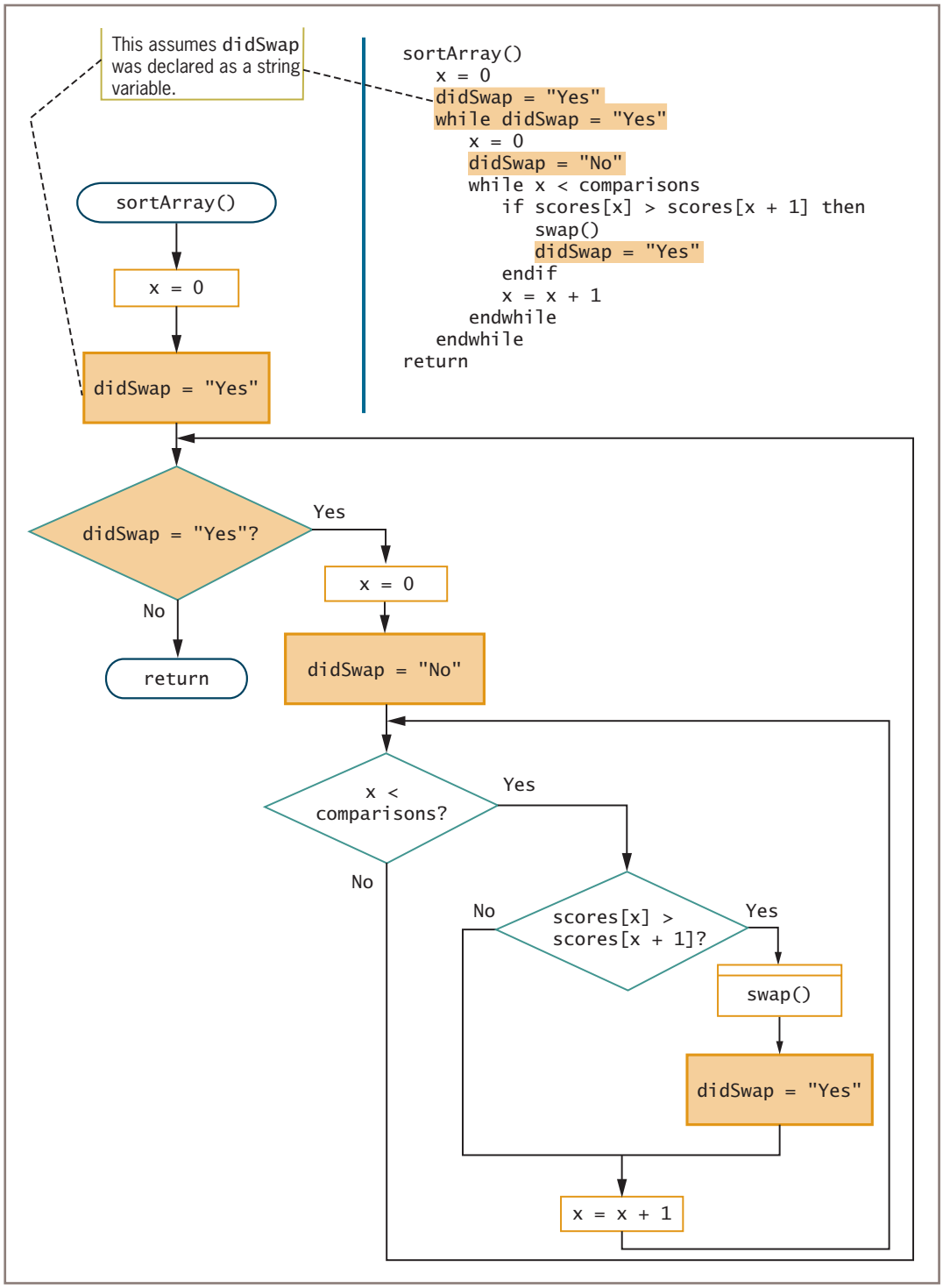


Figure 8-10 Flowchart and pseudocode for `sortArray()` method using `didSwap` variable

TWO TRUTHS & A LIE

Using the Bubble Sort Algorithm

1. You can use a bubble sort to arrange records in ascending or descending order.
2. In a bubble sort, items in a list are compared with each other in pairs, and when an item is out of order, it swaps values with the item below it.
3. With any bubble sort, after each adjacent pair of items in a list has been compared once, the largest item in the list will have "sunk" to the bottom.

The false statement is #3. Statement #3 is true of an ascending bubble sort. However, with a descending bubble sort, the smallest item in the list will have "sunk" to the bottom after each adjacent pair of items has been compared once.

Sorting Multifield Records

The bubble sort algorithm is useful for sorting a list of values, such as a list of test scores in ascending order or a list of names in alphabetical order. Records, however, are most frequently composed of multiple fields. When you want to sort records, you need to make sure data that belongs together stays together. When you sort records, two approaches you can take are to place related data items in parallel arrays and to sort records as a whole.

Sorting Data Stored in Parallel Arrays

Suppose that you have parallel arrays containing student names and test scores, like the arrays shown in Figure 8-11. Each student's name appears in the same relative position in the `names` array as his or her test score appears in the `scores` array. Further suppose that you want to sort the student names and their scores in alphabetical order. If you use a sort algorithm on the

names[0]	Cody	scores[0]	95
names[1]	Emma	scores[1]	90
names[2]	Brad	scores[2]	60
names[3]	Anna	scores[3]	85
names[4]	Doug	scores[4]	67

Figure 8-11 Appearance of `names` and `scores` arrays in memory

names array to place the names in alphabetical order, the name that starts in position 3, *Anna*, should end up in position 0. If you also neglect to rearrange the scores array, Anna's name will no longer be in the same relative position as her score, which is 85. Notice that you don't want to sort the values in the scores array. If you did, scores[2], 60, would move to position 0, and that is not Anna's score. Instead, when you sort the names, you want to make sure that each corresponding score is moved to the same position as the name to which it belongs.

Figure 8-12 shows the swap() module for a program that sorts names array values in alphabetical order and moves scores array values correspondingly. This version of the swap() module uses two temporary variables—a string named tempName and a numeric variable named tempScore. The swap() method executes whenever two names in positions x and x + 1 are out of order. Besides swapping the names in positions x and x + 1, the module also swaps the scores in the same positions. Therefore, each student's score always moves along with its student's name.

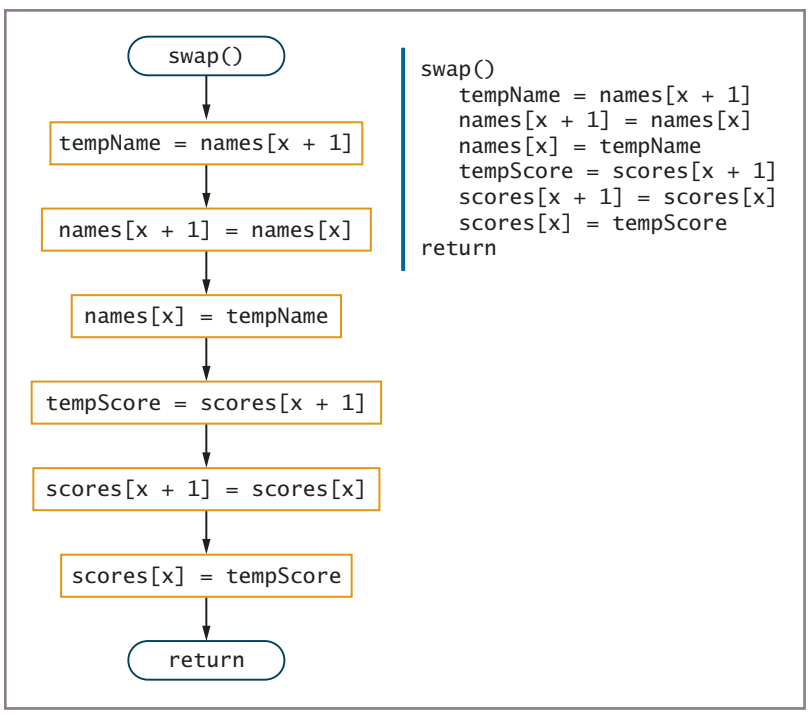


Figure 8-12 The swap() method for a program that sorts student names and retains their correct scores

Sorting Records as a Whole

In most modern programming languages, you can create group items that can be manipulated more easily than single data items. (You first learned about such group names in Chapter 7.) Creating a group name for a set of related data fields is beneficial when you

want to move related data items together, as when you sort records with multiple fields. These group items are sometimes called *structures*, but more frequently are created as *classes*. Chapters 10 and 11 provide much more detail about creating classes, but for now, understand that you can create a group item with syntax similar to the following:

```
class StudentRecord
  string name
  num score
endClass
```

To sort student records using the group name, you could do the following:

- Define a class named `StudentRecord`, as shown in the preceding code.
- Define what *greater than* means for a `StudentRecord`. For example, to sort records by student name, you would define *greater than* to compare `names` values, not `scores` values. The process for creating this definition varies among programming languages.
- Use a sort algorithm that swaps `StudentRecord` items, including both `names` and `scores`, whenever two `StudentRecords` are out of order.

TWO TRUTHS & A LIE

Sorting Multifield Records

1. To sort related parallel arrays, you must sort each in the same order—either ascending or descending.
2. When you sort related parallel arrays and swap values in one array, you must make sure that all associated arrays make the same relative swap.
3. Most modern programming languages allow you to create a group name for associated fields in a record.

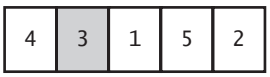
The false statement is #1. To sort related parallel arrays successfully, you must make sure all items in each array are swapped in a synchronized manner. You do not want to sort the arrays separately.

Other Sorting Algorithms

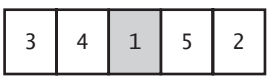
The bubble sort works well and is relatively easy to understand and manipulate, but many other sorting algorithms have been developed.

When you use an **insertion sort**, you look at each list element one at a time. If an element is out of order relative to any of the items earlier in the list, you move each earlier item down one position and then insert the tested element. The insertion sort is similar to the technique you would most likely use to sort a group of objects manually. Figure 8-13 shows how an ascending insertion sort would work for a list that starts out containing the values 4, 3, 1, 5, and 2.

The insertion sort algorithm



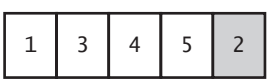
Start with the second element. Compare 3 and 4. They are out of order, so swap them.



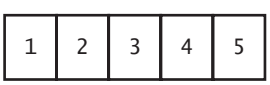
Compare 1 and 3. They are out of order. So save 1 in a temporary variable, move 4 and 3 down, and insert 1 where 3 was.



Compare 5 and 1. They are in order, so do nothing. Compare 5 and 3. They are in order, so do nothing. Compare 5 and 4. They are in order, so do nothing.



Compare 2 and 1. They are in order so do nothing. Compare 2 and 3, they are out of order, so save 2 in a temporary variable, move 3,4, and 5 down, and insert 2 where 3 used to be.



The sort is complete.

Figure 8-13 The insertion sort algorithm

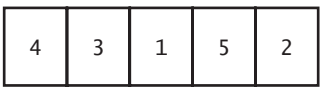


Watch the video *The Insertion Sort*.

When you use a **selection sort**, you picture an array of values divided into two sublists—values already sorted and values not yet sorted. To start, the already-sorted sorted list has zero elements. For an ascending sort, you find the smallest value in the unsorted sublist and swap its position with the leftmost value in the unsorted list. Then you move the rightmost boundary of the sorted sublist one element to the right. Figure 8-14 shows how an ascending selection sort would work for a list that starts out containing the values 4, 3, 1, 5, and 2.

The selection sort algorithm

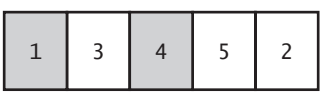
Sorted (0) Unsorted (5)



At first, the sorted sublist has zero elements and the unsorted sublist has five elements.

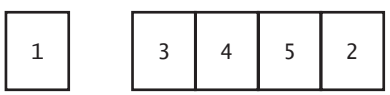
Find the smallest value in the unsorted list and swap it with the leftmost value in the unsorted list.

Sorted (0) Unsorted (5)



Then move the right edge of the sublist one element to the right.

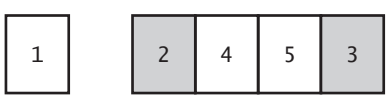
Sorted (1) Unsorted (4)



Now the sorted sublist has one element and the unsorted sublist has four.

Find the smallest value in the unsorted list and swap it with the leftmost in the unsorted list.

Sorted (1) Unsorted (4)



Then move the right edge of the sorted sublist one element to the right.

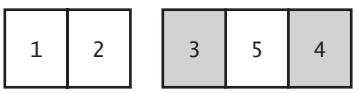
Sorted (2) Unsorted (3)



Now the sorted sublist has two elements and the unsorted sublist has three.

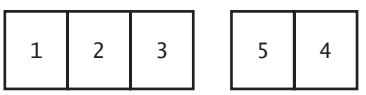
Find the smallest value in the unsorted list and swap it with the leftmost value in the unsorted list.

Sorted (2) Unsorted (3)



Then move the right edge of the sorted sublist one element to the right.

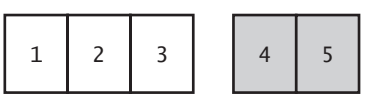
Sorted (3) Unsorted (2)



Now the sorted sublist has three elements and the unsorted sublist has two.

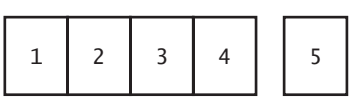
Find the smallest value in the unsorted list and swap it with the leftmost value in the unsorted list.

Sorted (3) Unsorted (2)



Then move the right edge of the sorted sublist one element to the right.

Sorted (4) Unsorted (1)



When the unsorted list has only one element, the sort is complete.

Figure 8-14 The selection sort algorithm



You might want to investigate the logic used by other sorting algorithms such as the *cocktail sort*, *gnome sort*, and *quick sort*.

TWO TRUTHS & A LIE

Other Sorting Algorithms

1. The insertion sort algorithm moves list elements down when a tested item should be inserted before them.
2. Insertion sorts are used for ascending sorts and selection sorts are used for descending ones.
3. The selection sort algorithm uses two sublists.

The false statement is #2. Both insertion and selection sorts can be used both for ascending and descending orders.

Using Multidimensional Arrays

In Chapter 6, you learned that an array is a series or list of values in computer memory, all of which have the same name and data type but are differentiated with subscripts. Usually, all the values in an array have something in common; for example, they might represent a list of employee ID numbers or a list of prices for items sold in a store. A subscript, also called an index, is a number that indicates the position of a particular item within an array.

An array whose elements you can access using a single subscript is a **one-dimensional** or **single-dimensional array**. The array has only one dimension because its data can be stored in a table that has just one dimension—height. If you know the vertical position of a one-dimensional array's element, you can find its value.

For example, suppose that you own an apartment building and charge five different rent amounts for apartments on different floors (including floor 0, the basement), as shown in Table 8-1.

Floor	Rent ($)
0	350
1	400
2	475
3	600
4	1000

Table 8-1 Rent schedule based on floor

You could declare the following array to hold the rent values shown in Table 8-1:

```
num RENTS_BY_FLR[5] = 350, 400, 475, 600, 1000
```

The location of any rent value in Table 8-1 depends on only a single variable—the floor of the building. So, when you create a single-dimensional array to hold rent values, you need just one subscript to identify the row.

Sometimes, however, locating a value in an array depends on more than one variable. An array that requires more than one subscript to access an element is a **multidimensional array**.

For example, if you must represent values in a table or grid that contains rows and columns instead of a single list, then you might want to use a **two-dimensional array**. A two-dimensional array contains two dimensions: height and width. That is, the location of any element depends on two factors. For example, if an apartment's rent depends on two variables—both the floor of the building and the number of bedrooms—then you want to create a two-dimensional array.

As an example of how useful two-dimensional arrays can be, assume that you own an apartment building with five floors, and that each of the floors has studio apartments (with no bedroom) and one- and two-bedroom apartments. Table 8-2 shows the rental amounts.

Floor	Studio Apartment	1-bedroom Apartment	2-bedroom Apartment
0	350	390	435
1	400	440	480
2	475	530	575
3	600	650	700
4	1000	1075	1150

Table 8-2 Rent schedule based on floor and number of bedrooms

To determine a tenant's rent, you need to know two pieces of information: the floor where the tenant lives and the number of bedrooms in the apartment. Each element in a two-dimensional array requires two subscripts to reference it—one subscript to determine the row and a second to determine the column. Thus, the 15 rent values for a two-dimensional array based on Table 8-2 would be arranged in five rows and three columns and defined as follows:

```
num RENTS_BY_FLR_AND_BDRMS[5][3]= {350, 390, 435},
                                  {400, 440, 480},
                                  {475, 530, 575},
                                  {600, 650, 700},
                                  {1000, 1075, 1150}
```

Figure 8-15 shows how the one- and two-dimensional rent arrays might appear in computer memory.

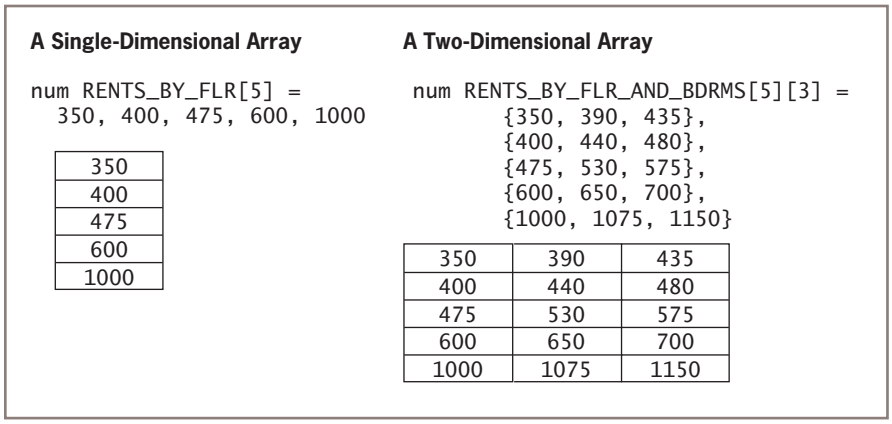


Figure 8-15 One- and two-dimensional arrays in memory

When a one-dimensional array has been declared in this book, a set of square brackets follows the array type and name. To declare a two-dimensional array, many languages require you to use two sets of brackets after the array type and name. For each element in the array, the first set of square brackets holds the number of rows and the second set holds the number of columns. In other words, the two dimensions represent the array's height and its width.



Instead of two sets of brackets to indicate a position in a two-dimensional array, some languages use a single set of brackets but separate the subscripts with commas. Therefore, the elements in row 1, column 2 would be `RENTS_BY_FLR_AND_BDRMS[1, 2]`.

In the `RENTS_BY_FLR_AND_BDRMS` array declaration, the values that are assigned to each row are enclosed in curly braces to help you picture the placement of each number in the array. The first row of the array holds the three rent values 350, 390, and 435 for floor 0; the second row holds 400, 440, and 480 for floor 1; and so on.

You access a two-dimensional array value using two subscripts, in which the first subscript represents the row and the second one represents the column. The legal values for each of the dimensions include 0 through one less than the size of the array.

For example, some of the values in the two-dimensional rents array are as follows:

- `RENTS_BY_FLR_AND_BDRMS[0][0]` is 350
- `RENTS_BY_FLR_AND_BDRMS[0][1]` is 390
- `RENTS_BY_FLR_AND_BDRMS[0][2]` is 435

- RENTS_BY_FLR_AND_BDRMS[4][0] is 1000
- RENTS_BY_FLR_AND_BDRMS[4][1] is 1075
- RENTS_BY_FLR_AND_BDRMS[4][2] is 1150

If you declare two variables to hold the floor number and bedroom count as `num floor` and `num bedrooms`, any tenant's rent is `RENTS_BY_FLR_AND_BDRMS[floor][bedrooms]`.



When mathematicians use a two-dimensional array, they often call it a **matrix** or a **table**. You may have used a spreadsheet, which is a two-dimensional array in which you need to know a row number and a column letter to access a specific cell.

Figure 8-16 shows a program that continuously displays rents for apartments based on renter requests for floor location and number of bedrooms. Notice that although significant setup is required to provide all the values for the rents, the basic program is extremely brief and easy to follow. (You could improve the program in Figure 8-16 by making sure the values for `floor` and `bedrooms` are within range before using them as array subscripts.)



Watch the video *Two-Dimensional Arrays*.

Two-dimensional arrays are never actually *required* in order to achieve a useful program. The same 15 categories of rent information in Figure 8-16 could be stored in three separate single-dimensional arrays of five elements each, and you could use a decision to determine which array to access. Of course, don't forget that even one-dimensional arrays are never required to solve a problem. You could also declare 15 separate rent variables and make 15 separate decisions to determine the rent.

Besides one- and two-dimensional arrays, many programming languages also support **three-dimensional arrays**. For example, if you own a multistory apartment building with different numbers of bedrooms available in apartments on each floor, you can use a two-dimensional array to store the rental fees, but if you own several apartment buildings, you might want to employ a third dimension to store the building number. For example, if a three-dimensional array is stored on paper, you might need to know an element's row, column, and page to access it, as shown in Figure 8-17.

If you declare a three-dimensional array named `RENTS_BY_3_FACTORS`, then you can use an expression such as `RENTS_BY_3_FACTORS[floor][bedrooms][building]`, which refers to a specific rent figure for an apartment whose floor and bedroom numbers are stored in the `floor` and `bedrooms` variables, and whose `building` number is stored in the `building` variable. Specifically, `RENTS_BY_3_FACTORS[0][1][2]` refers to a basement (floor 0) one-bedroom apartment in building 2 (which is the third building).

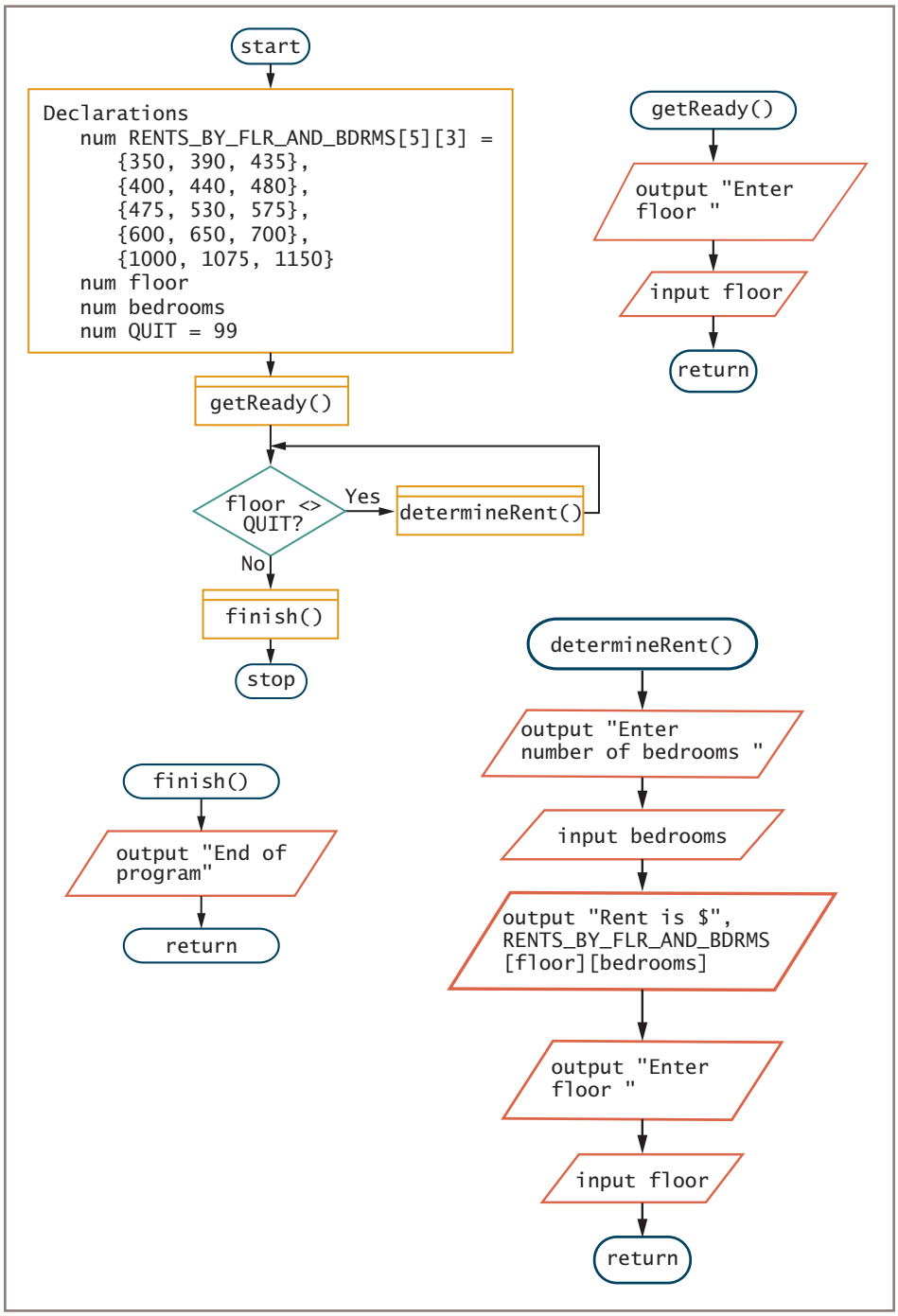


Figure 8-16 A program that determines rents (*continues*)

(continued)

```
start
   Declarations
      num RENTS_BY_FLR_AND_BDRMS[5][3] = {350, 390, 435},
                                         {400, 440, 480},
                                         {475, 530, 575},
                                         {600, 650, 700},
                                         {1000, 1075, 1150}
      num floor
      num bedrooms
      num QUIT = 99
   getReady()
   while floor <> QUIT
      determineRent()
   endwhile
   finish()
stop

getReady()
   output "Enter floor "
   input floor
return

determineRent()
   output "Enter number of bedrooms "
   input bedrooms
   output "Rent is $", RENTS_BY_FLR_AND_BDRMS[floor][bedrooms]
   output "Enter floor "
   input floor
return

finish()
   output "End of program"
return
```

Figure 8-16 A program that determines rents



Both two- and three-dimensional arrays are examples of multidimensional arrays. Some languages allow many more dimensions. For example, in C# and Visual Basic, an array can have 32 dimensions. However, it's usually hard for people to keep track of more than three dimensions.

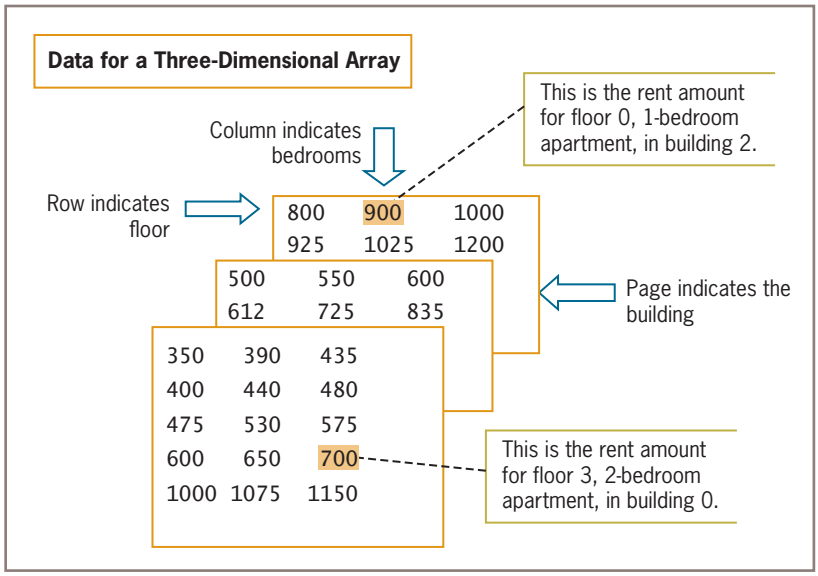


Figure 8-17 Picturing a three-dimensional array

TWO TRUTHS & A LIE

Using Multidimensional Arrays

1. In every multidimensional array, the location of any element depends on two factors.
2. For each element in a two-dimensional array, the first subscript represents the row number and the second one represents the column.
3. Multidimensional arrays are never actually *required* in order to achieve a useful program.

The false statement is #1. In a two-dimensional array, the location of any element depends on two factors, but multidimensional arrays include those that depend on any number of factors.

Using Indexed Files and Linked Lists

Sorting a list of five or even 100 scores does not require significant computer resources. However, many data files contain thousands of records, and each record might contain dozens of data fields. Sorting large numbers of data records requires considerable time and computer memory. When a large data file needs to be processed in ascending or descending order based on a particular field, the most efficient approach is usually to access records

based on their logical order rather than sorting and accessing them in their physical order. **Physical order** refers to a "real" order for storage; an example would be writing the names of 10 friends, each one on a separate index card. You can arrange the cards alphabetically by the friends' last names, chronologically by length of the friendship, or randomly by throwing the cards in the air and picking them up as you find them. Whichever way you do it, the records still follow each other in *some* order. In addition to their current physical order, you can think of the cards as having a **logical order**; that is, a virtual order, based on any criterion you choose—from the tallest friend to the shortest, from the one who lives farthest away to the closest, and so on. Sorting the cards in a new physical order can take a lot of time; using the cards in their logical order without physically rearranging them is often more efficient.

Using Indexed Files

A common method of accessing records in logical order requires using an index. Using an index involves identifying a key field for each record. A record's **key field** is the field whose contents make the record unique among all records in a file. For example, multiple employees can have the same last name, first name, salary, or street address, but each employee possesses a unique employee identification number, so an ID number field might make a good key field for a personnel file. Similarly, a product number makes a good key field in an inventory file.

As pages in a book have numbers, computer memory and storage locations have **addresses**. In Chapter 1, you learned that every variable has a numeric address in computer memory; likewise, every data record on a disk has a numeric address where it is stored. You can store records in any physical order on the disk, but when you **index** records, you store a list of key fields paired with the storage address for the corresponding data record. Then you can use the index to find the records in order based on their addresses.

When you use an index, you can store records on a **random-access storage device**, such as a disk, from which records can be accessed in any order. Each record can be placed in any physical location on the disk, and you can use the index as you would use an index in the back of a book. If you pick up a 600-page American history book because you need some facts about Betsy Ross, you do not want to start on page 1 and work your way through the book. Instead, you turn to the index, discover that Betsy Ross is mentioned on page 418, and go directly to that page. As a programmer, you do not need to determine a record's exact physical address in order to use it. A computer's operating system takes care of locating available storage for your records.



Chapter 7 contains a discussion of random access files and how they differ from sequential files.

You can picture an index based on ID numbers by looking at the index in Figure 8-18. The index is stored on a portion of the disk. The address in the index refers to other scattered locations on the disk.

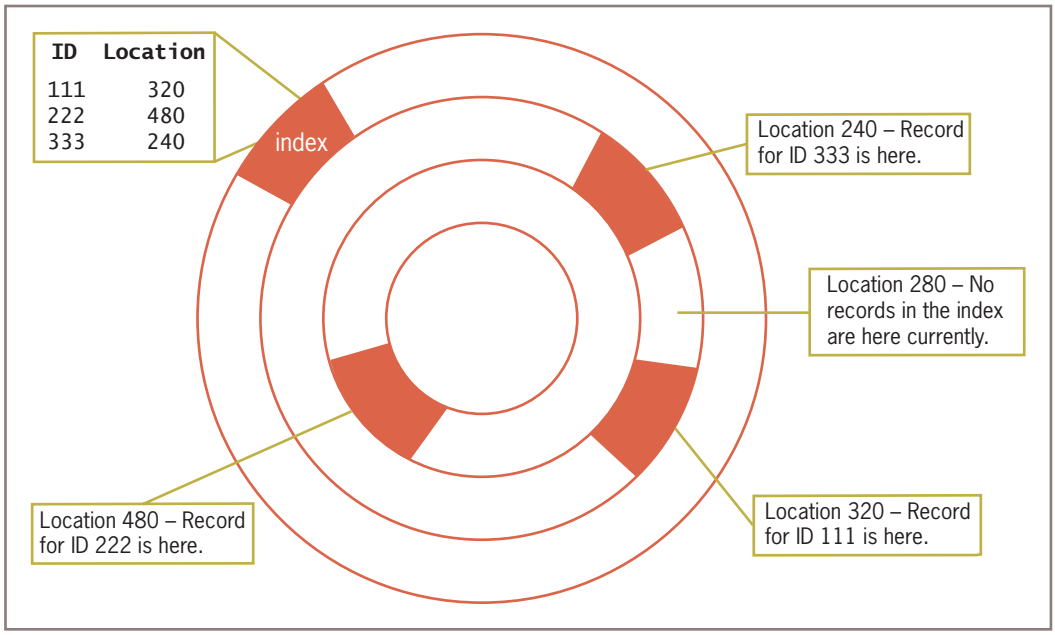


Figure 8-18 An index on a disk that associates ID numbers with disk addresses

When you want to access the data for employee 333, you tell your computer to look through the ID numbers in the index, find a match, and then proceed to the memory location specified. Similarly, when you want to process records in order based on ID number, you tell your system to retrieve records at the locations in the index in sequence. Thus, employee 111 might have been hired last and the record might be stored at the highest physical address on the disk, but if the employee record has the lowest ID number, it will be accessed first during any ordered processing.

When a record is removed from an indexed file, it does not have to be physically removed. Its reference can simply be deleted from the index, and then it will not be part of any further processing.

Watch the video *Using an Indexed File*.

Using Linked Lists

Another way to access records in a desired order, even though they might not be physically stored in that order, is to create a linked list. In its simplest form, creating a **linked list** involves creating one extra field in every record of stored data. This extra field holds the

physical address of the next logical record. For example, a record that holds a customer's ID, name, and phone number might contain the following fields:

```
idNum
name
phoneNum
nextCustAddress
```

Every time you use a record, you access the next record based on the address held in the `nextCustAddress` field.

Every time you add a new record to a linked list, you search through the list for the correct logical location of the new record. For example, assume that customer records are stored at the addresses shown in Table 8-3 and that they are linked in customer ID order. Notice that the addresses of the records are not shown in sequential order. The records are shown in their logical order by `idNum`.

Address	idNum	name	phoneNum	nextCustAddress
0000	111	Baker	234-5676	7200
7200	222	Vincent	456-2345	4400
4400	333	Silvers	543-0912	6000
6000	444	Donovan	328-8744	eof

Table 8-3 Sample linked customer list

You can see from Table 8-3 that each customer record contains a `nextCustAddress` field that stores the address of the next customer who follows in customer ID number order (and not necessarily in address order). For any individual customer, the next logical customer's address might be physically distant.

Examine the file shown in Table 8-3, and suppose that a new customer is acquired with number *245* and the name *Newberg*. Also suppose that the computer operating system finds an available storage location for Newberg's data at address 8400. In this case, the procedure to add Newberg to the list is:

1. Create a variable named `currentAddress` to hold the address of the record in the list you are examining. Store the address of the first record in the list, 0000, in this variable.
2. Compare the new customer Newberg's ID, 245, with the current (first) record's ID, 111 (in other words, the ID at address 0000). The value 245 is higher than 111, so you save the first customer's address—0000, the address you are currently examining—in a variable you can name `saveAddress`. The `saveAddress` variable always holds the address you just finished examining. The first customer record

contains a link to the address of the next logical customer—7200. Store 7200 in the `currentAddress` variable.

3. Examine the second customer record, the one that physically exists at the address 7200, which is currently held in the `currentAddress` variable.
4. Compare Newberg's ID, 245, with the ID stored in the record at `currentAddress`, 222. The value 245 is higher, so save the current address, 7200, in `saveAddress` and store its `nextCustAddress` address field, 4400, in the `currentAddress` variable.
5. Compare Newberg's ID, 245, with 333, which is the ID at `currentAddress` (4400). Up to this point, 245 had been higher than each ID tested, but this time the value 245 is lower, so customer 245 should logically precede customer 333. Set the `nextCustAddress` field in Newberg's record (customer 245) to 4400, which is the address of customer 333 and the address stored in `currentAddress`. This means that in any future processing, Newberg's record will logically be followed by the record containing 333. Also set the `nextCustAddress` field of the record located at `saveAddress` (7200, customer 222, Vincent, who logically preceded Newberg) to the new customer Newberg's address, 8400. The updated list appears in Table 8-4.

Address	idNum	name	phoneNum	nextCustAddress
0000	111	Baker	234-5676	7200
7200	222	Vincent	456-2345	8400
8400	245	Newberg	222-9876	4400
4400	333	Silvers	543-0912	6000
6000	444	Donovan	328-8744	eof

Table 8-4 Updated customer list

As with indexing, when removing records from a linked list, the records do not need to be physically deleted from the medium on which they are stored. If you need to remove customer 333 from the preceding list, all you need to do is change Newberg's `nextCustAddress` field to the value in Silvers' `nextCustAddress` field, which is Donovan's address: 6000. In other words, the value of 6000 is obtained not by knowing to which record Newberg should point, but by knowing to which record Silvers previously pointed. When Newberg's record points to Donovan, Silvers' record is then bypassed during any further processing that uses the links to travel from one record to the next.

More sophisticated linked lists are doubly linked—they store *two* additional fields with each record. One field stores the address of the next record, and the other field stores the address of the *previous* record so that the list can be accessed either forward or backward.



Watch the video *Using a Linked List*.

TWO TRUTHS & A LIE

Using Indexed Files and Linked Lists

1. When a large data file needs to be processed in order based on a particular field, the most efficient approach is usually to sort the records.
2. A record's key field contains a value that makes the record unique among all records in a file.
3. Creating a linked list requires you to create one extra field for every record; this extra field holds the physical address of the next logical record.

The false statement is #1. The most efficient approach is usually to store and access records based on their logical order rather than sorting and accessing them in their physical order.

Chapter Summary

- Frequently, data items need to be sorted. When you sort data, you can sort either in ascending order, arranging records from lowest to highest value, or in descending order, arranging records from highest to lowest value.
- In a bubble sort, items in a list are compared with each other in pairs, and appropriate swaps are made. With an ascending bubble sort, after each adjacent pair of items in a list has been compared once, the largest item in the list will have "sunk" to the bottom; after many passes through the list, the smallest items rise to the top. The bubble sort algorithm can be improved to sort varying numbers of values and to eliminate unnecessary comparisons.
- When you sort records, two possible approaches are to place related data items in parallel arrays and to sort records as a whole.
- When you use an insertion sort, you look at each list element, and when an element is out of order relative to any of the items earlier in the list, you move each earlier item down one position and then insert the tested element. When you use a selection sort, you use two sublists—values already sorted and values not yet sorted. You repeatedly look for the smallest value in the unsorted sublist, swap it with the item at the beginning of the unsorted list, and then add that element to the end of the sorted sublist.

- Two-dimensional arrays have both rows and columns of values. You must use two subscripts when you access an element in a two-dimensional array. Many languages support arrays with even more dimensions.
- You can use an index or linked list to access data records in a logical order that differs from their physical order. Using an index involves identifying a physical address and key field for each record. Creating a linked list involves creating an extra field within every record to hold the physical address of the next logical record.

Key Terms

Sequential order describes the arrangement of records when they are stored one after another on the basis of the value in a particular field.

The **median** value in a list is the value in the middle position when the values are sorted; when the list contains an even number of values, the median is the mean of the values in the two middle positions.

The **mean** value in a list is the arithmetic average.

An **algorithm** is a list of instructions that accomplish a task.

A **bubble sort** is a sorting algorithm in which a list of elements is arranged by comparing items in pairs; when an item is out of order, it is swapped with the item below it.

To **swap values** is to exchange the values of two variables.

An **insertion sort** is a sorting algorithm in which list elements are examined and if an element is out of order relative to any of the items earlier in the list, each earlier item is moved down one position and then the tested element is inserted.

A **selection sort** is a sorting algorithm in which list values are divided into sorted and unsorted sublists; then the unsorted sublist is repeatedly examined for its smallest value which is then moved to the end of the sorted sublist.

A **one-dimensional** or **single-dimensional array** is a list accessed using a single subscript.

Multidimensional arrays are lists with more than one dimension; more than one subscript is required to access an element.

Two-dimensional arrays have both rows and columns of values; you must use two subscripts when you access an element in a two-dimensional array.

Matrix and **table** are terms used by mathematicians to describe a two-dimensional array.

Three-dimensional arrays are arrays in which each element is accessed using three subscripts.

A list's **physical order** is the order in which its elements are actually stored.

A list's **logical order** is the order in which it is used, even though its elements are not necessarily stored in that physical order.

The **key field** of a record contains a value that makes the record unique among all records in a file.

Addresses identify computer memory and storage locations.

When you **index** records, you store a list of key fields paired with the storage address for the corresponding data record.

A **random-access storage device**, such as a disk, is one from which records can be accessed in any order.

A **linked list** contains an extra field in every record of stored data; this extra field holds the physical address of the next logical record.

Exercises



Review Questions

1. Employee records stored in order from highest-paid to lowest-paid have been sorted in _____ order.

   a. descending
   b. ascending
   c. staggered
   d. recursive

2. Student records stored in alphabetical order by last name have been sorted in _____ order.

   a. descending
   b. ascending
   c. staggered
   d. recursive

3. When computers sort data, they always _____.

   a. place items in ascending order
   b. use a bubble sort
   c. use numeric values when making comparisons
   d. begin the process by locating the position of the lowest value

4. Which of the following code segments correctly swaps the values of variables named x and y?

   a.
   ```
   x = y
   y = temp
   x = temp
   ```
   b.
   ```
   temp = x
   x = y
   y = temp
   ```
   c.
   ```
   x = y
   temp = x
   y = temp
   ```
   d.
   ```
   temp = x
   y = x
   x = temp
   ```

5. Which type of sort compares list items in pairs, swapping any two adjacent values that are out of order?

   a. insertion sort
   b. indexed sort
   c. bubble sort
   d. selection sort

6. To sort a list of 15 values using a bubble sort, the greatest number of times you would have to pass through the list making comparisons is_____.

   a. 15
   b. 14
   c. 13
   d. 12

7. To completely sort a list of 10 values using a bubble sort, the greatest possible number of required pair comparisons is _____.

   a. 9
   b. 10
   c. 81
   d. 100

8. When you do not know how many items need to be sorted in a program, you can create an array that has _____.

   a. variable-sized elements
   b. at least as many elements as the number you predict you will need
   c. at least one element less than the number you predict you will need
   d. You cannot sort items if you do not know the number of items when you write the program.

9. In a bubble sort, on each pass through the list that must be sorted, you can stop making pair comparisons _____.

   a. one comparison sooner
   b. two comparisons sooner
   c. one comparison later
   d. two comparisons later

10. When performing a bubble sort on a list of 10 values, you can stop making passes through the list of values as soon as _____ on a single pass through the list.

    a. no swaps are made
    b. exactly one swap is made
    c. no more than nine swaps are made
    d. no more than 10 swaps are made

11. The bubble sort is _____.

    a. the most efficient sort
    b. a relatively fast sort compared to others
    c. a relatively easy sort to understand
    d. all of the above

12. Data stored in a table that can be accessed using row and column numbers is stored as a _____ array.

    a. single-dimensional
    b. two-dimensional
    c. three-dimensional
    d. nondimensional

13. A two-dimensional array declared as `num myArray[6][7]` has _____ columns.

    a. 5
    b. 6
    c. 7
    d. 8

14. In a two-dimensional array declared as `num myArray[6][7]`, the highest row number is _____.

    a. 5
    b. 6
    c. 7
    d. 8

15. If you access a two-dimensional array with the expression `output myArray[2][5]`, the output value will be _____.

    a. 0
    b. 2
    c. 5
    d. impossible to tell from the information given

16. Three-dimensional arrays _____.

    a. are supported in many modern programming languages
    b. always contain at least nine elements
    c. are used only in object-oriented languages
    d. all of the above

17. Student records are stored in ID number order, but accessed by grade-point average for a report. Grade-point average order is a(n) _____ order.

    a. imaginary
    b. physical
    c. logical
    d. illogical

18. When you store a list of key fields paired with the storage address for the corresponding data record, you are creating _____.

    a. a directory
    b. a three-dimensional array
    c. a linked list
    d. an index

19. When a record in an indexed file is not needed for further processing, _____.

    a. its first character must be replaced with a special character, indicating it is a deleted record
    b. its position must be retained, but its fields must be replaced with blanks
    c. it must be physically removed from the file
    d. the record can stay in place physically, but its reference is removed from the index

20. With a linked list, every record _____.

    a. is stored in sequential order
    b. contains a field that holds the address of another record
    c. contains a code that indicates the record's position in an imaginary list
    d. is stored in a physical location that corresponds to a key field



Programming Exercises



Several of the programming exercises in this section ask you to find the mean and median value in a list. Recall that the mean is the arithmetic average and that that median is the middle value in an ordered list. When a list contains an odd number of values, the median is the value in the middle position. When a list contains an even number of values, the median is the mean of the values in the two middle positions. Chapter 2 described the remainder operator which you can use to determine whether a number is even or odd. When a number is divided by 2, the remainder is 0 if the number is even.

1. Design an application that accepts 10 numbers and displays them in descending order.

2. Design an application that accepts 15 words and displays them in alphabetical order.

3. a. Professor Zak allows students to drop the four lowest scores on the ten 100-point quizzes she gives during the semester. Design an application that accepts a student name and 10 quiz scores. Output the student's name and total points for the student's six highest-scoring quizzes.

   b. Modify the application in Exercise 3a so that the student's mean and median scores on the six best quizzes are displayed.

4. Girl Scout Troop 815 has 18 members. Write a program in which the troop leader can enter the number of boxes of cookies sold by each scout, and output the total number of boxes sold and the mean and median values.

5. The village of Marengo conducted a census and collected records that contain household data, including the number of occupants in each household. The exact number of household records has not yet been determined, but you know that Marengo has fewer than 500 households. Develop the logic for a program that allows a user to enter each household size and determine the mean and median household size in Marengo.

6. a. Three Strikes Bowling Lanes hosts an annual tournament for 12 teams Design a program that accepts each team's name and total score for the tournament and stores them in parallel arrays. Display the names of top three teams.

   b. Modify the bowling tournament program so that, instead of the team's total score, the program accepts the score of each of the four team members. Display the names of the five top scorers in the tournament as well as their team names.

7. a. *The Daily Trumpet* newspaper accepts classified advertisements in 15 categories such as *Apartments for Rent* and *Pets for Sale*. Develop the logic for a program that accepts classified advertising data, including a category code (an integer 1 through 15) and the number of words in the ad. Store these values in parallel arrays. Then sort the arrays so that records are sorted in ascending order by category. The output lists each category number, the number of ads in the category, and the total number of words in the ads in the category.

   b. Modify the newspaper advertising program in Exercise 7a to display a descriptive string with each category. For example, Category 1 might be *Apartments for Rent*.

8. Le Chef Heureux Restaurant has 20 tables that can be reserved at 5 p.m., 7 p.m., or 9 p.m. Design a program that accepts reservations for specific tables at specific times; the user enters the number of customers, the table number, and the time. Do not allow more than four guests per table or invalid table numbers or times. If an attempt is made to reserve a table already taken, reprompt the user. Continue to accept reservations until the user enters a sentinel value or all slots are filled. Then display all empty tables in each time slot.

9. Building Block Day Care Center charges varying weekly rates depending on the age of the child and the number of days per week the child attends, as shown in Table 8-5. Develop the logic for a program that continuously accepts child care data and displays the appropriate weekly rate.

	Days Per Week				
Age in Years	**1**	**2**	**3**	**4**	**5**
0	30.00	60.00	88.00	115.00	140.00
1	26.00	52.00	70.00	96.00	120.00
2	24.00	46.00	67.00	89.00	110.00
3	22.00	40.00	60.00	75.00	88.00
4 or more	20.00	35.00	50.00	66.00	84.00

Table 8-5 Day care rates

10. Executive Training School offers typing classes. Each final exam evaluates a student's typing speed and the number of typing errors made. Develop the logic for a program that produces a summary table of each examination's results. Each row represents the number of students whose typing speed falls within the following ranges of words per minute: 0–19, 20–39, 40–69, and 70 or more. Each column represents the number of students who made different numbers of typing errors—0 through 6 or more.

11. HappyTunes is an application for downloading music files. Each time a file is purchased, a transaction record is created that includes the music genre and price paid. The available genres are *Classical, Easy Listening, Jazz, Pop, Rock*, and *Other*. Develop an application that accepts input data for each transaction and displays a report that lists each of the music genres, along with a count of the number of downloads in each of the following price categories:

    - Over $10.00
    - $6.00 through $9.99
    - $3.00 through $5.99
    - Under $3.00



Performing Maintenance

1. A file named MAINTENANCE08-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.

Find the Bugs

1. Your downloadable files for Chapter 8 include DEBUG08-01.txt, DEBUG08-02.txt, and DEBUG08-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 8 include a file named DEBUG08-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.

Game Zone

1. In the Game Zone section of Chapter 6, you designed the logic for a quiz that contains multiple-choice questions about a topic of your choice. (Each question had three answer options.) Now, modify the program so it allows the user to retake the quiz up to four additional times or until the user achieves a perfect score, whichever comes first. At the end of all the quiz attempts, display a recap of the user's scores.

2. In the Game Zone section of Chapter 5, you designed a guessing game in which the application generates a random number and the player tries to guess it. After each guess, you displayed a message indicating whether the player's guess was correct, too high, or too low. When the player eventually guessed the correct number, you displayed a score that represented a count of the number of required guesses. Now, modify that program to allow a player to replay the game as many times as he likes, up to 20 times. When the player is done, display the scores from highest to lowest, and display the mean and median scores.

3. a. Create a TicTacToe game. In this game, two players alternate placing Xs and Os into a grid until one player has three matching symbols in a row, either horizontally, vertically, or diagonally. Create a game that displays a three-by-three grid containing the digits 1 through 9, similar to the first window shown in Figure 8-19. When the user chooses a position by typing a number, place an X in the appropriate spot. For example, after the user chooses 3, the screen looks like the second window in Figure 8-19. Generate a random number for the position where the computer will place an O. Do not allow the player or the computer to place a symbol where one has already been placed. When either the player or computer has three symbols in a

row, declare a winner. If all positions have been used and no one has three symbols in a row, declare a tie.

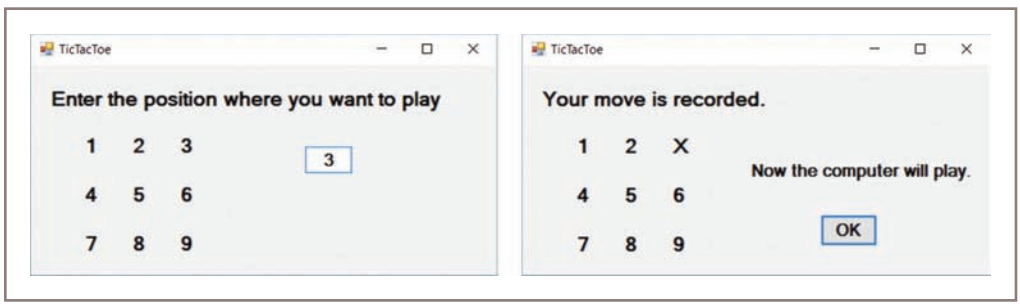


Figure 8-19 A TicTacToe game

b. In the TicTacToe game in Exercise 3a, the computer's selection is chosen randomly. Improve the game so that when the computer has two Os in any row, column, or diagonal, it selects the winning position for its next move rather than selecting a position randomly.

CHAPTER 9

Advanced Modularization Techniques

Upon completion of this chapter, you will be able to:

- Name the parts of a method
- Design methods with no parameters
- Design methods that require parameters
- Design methods that return a value
- Pass arrays to methods
- Overload methods
- Use predefined methods
- Appreciate method design issues, including implementation hiding, cohesion, and coupling
- Describe recursion

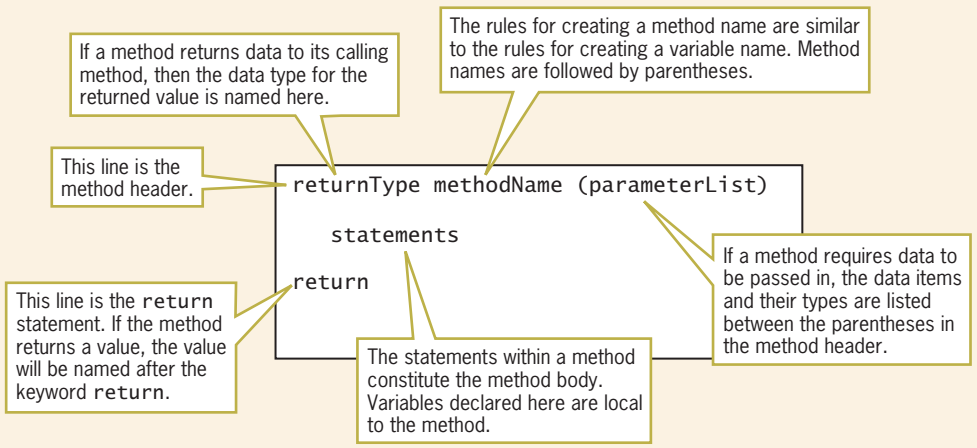
The Parts of a Method

In object-oriented programming languages such as Java and C#, modules are most often called *methods*. In Chapter 2, you learned about many features of methods and much of the vocabulary associated with them. For example:

- A **method** is a program module that contains a series of statements that carry out a task; you can invoke or call a method from another program or method. The calling program or method is the called method's **client**.
- Any program can contain an unlimited number of methods, and each method can be called an unlimited number of times.
- The rules for naming methods are different in every programming language, but they often are similar to the language's rules for variable names. In this text, method names are followed by a set of parentheses.
- A method must include a **method header** (sometimes also called the *method declaration*), which contains identifying information about the method.
- A method includes a **method body**. The body contains the method's **implementation**—the statements that carry out the method's tasks.
- A **method `return` statement** returns control to the calling method after a method executes. Although methods with multiple `return` statements are allowed in many programming languages, that practice is not recommended. Structured programming requires that a method must contain a single entry point and a single exit point. Therefore, a method should have only one `return` statement, and it should be the last statement. In many programming languages, control is returned to the calling method even if the `return` statement is omitted, but for clarity, this book will include a `return` statement at the end of each method.
- Variables and constants can be declared within a method. A data item declared in a method is **local** to that method, meaning it is in scope, or recognized only within that method.
- The opposite of local is global. When a data item is known to all of a program's methods or modules, it is a **global** data item. In general, programmers prefer local data items because when data is contained within the method that uses it, the method is more portable and less prone to error. In Chapter 2, you learned that when a method is described as *portable*, it can easily be moved to another application and used there.
- Methods can have parameter lists that provide details about data passed into methods.
- Methods have return types that provide information about data the method returns. In some languages, like C++, a default return type is implied if no return type is listed.

Quick Reference 9-1 shows important parts of a method. You learn more about these parts in the rest of this chapter.

QUICK REFERENCE 9-1 The Anatomy of a Method



TWO TRUTHS & A LIE

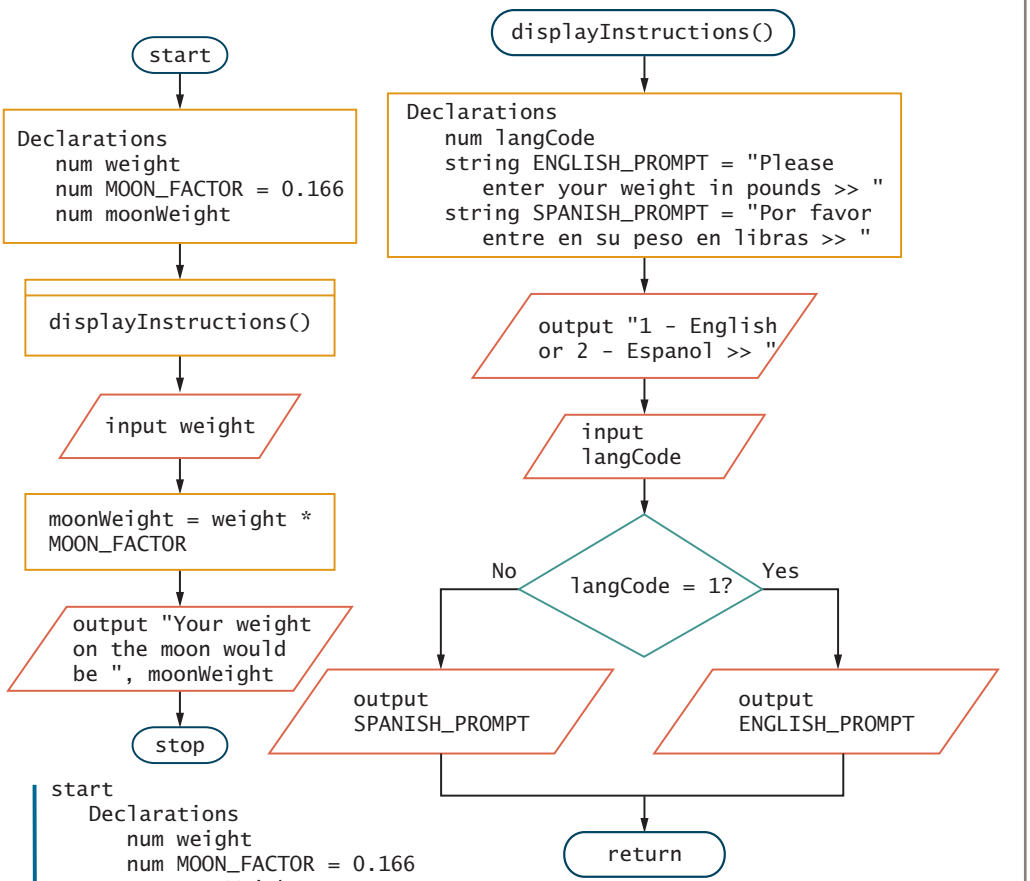
The Parts of a Method

1. A program can contain an unlimited number of methods, but each method can be called only once.
2. A method includes a header and a body.
3. Variables and constants are in scope within, or local to, only the method in which they are declared.

The false statement is #1. Each method can be called an unlimited number of times.

Using Methods with no Parameters

Figure 9-1 shows a program that allows a user to enter a preferred language (English or Spanish) and then, using the chosen language, asks the user to enter his or her weight. The program then calculates the user's weight on the moon as 16.6 percent of the user's weight on Earth. The main program contains declarations for two variables and a constant. The program calls the `displayInstructions()` method, which contains its own local variable and constants that are invisible (and therefore not available) to the main program. The method prompts the user for a language indicator and displays a prompt in the selected language. Figure 9-2 shows a typical program execution in a command-line environment.




```
start
   Declarations
      num weight
      num MOON_FACTOR = 0.166
      num moonWeight
   displayInstructions()
   input weight
   moonWeight = weight * MOON_FACTOR
   output "Your weight on the moon would be ", moonWeight
stop

displayInstructions()
   Declarations
      num langCode
      string ENGLISH_PROMPT = "Please enter your weight in pounds >> "
      string SPANISH_PROMPT = "Por favor entre en su peso en libras >> "
   output "1 - English or 2 - Espanol >> "
   input langCode
   if langCode = 1 then
      output ENGLISH_PROMPT
   else
      output SPANISH_PROMPT
   endif
return
```

Figure 9-1 A program that calculates the user's weight on the moon



In Chapter 2, you learned that this book uses a rectangle with a horizontal stripe across the top to represent a method call statement in a flowchart. Some programmers prefer to use a rectangle with two vertical stripes at the sides, and you should use that convention if your organization prefers it. This book reserves the shape with two vertical stripes to represent a method from a library that is external to the program.

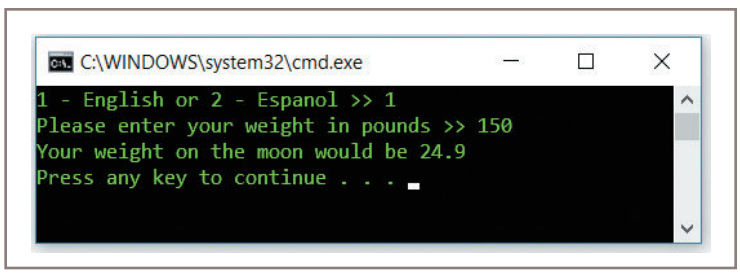


Figure 9-2 Output of moon weight calculator program in Figure 9-1

In Figure 9-1, the main program and the called method each contain only data items that are needed at the local level. The main program does not know about or have access to the variables and constants `langCode`, `ENGLISH_PROMPT`, and `SPANISH_PROMPT` declared locally within the method. Similarly, in modern programming languages, the `displayInstructions()` method does not have knowledge about or access to `weight`, `MOON_FACTOR`, or `moonWeight`, which are declared in the main program. In this program, there is no need for either method to know about the data in the other. However, sometimes two or more parts of a program require access to the same data. When methods must share data, you can pass the data into methods and return data out of them.

In this chapter, you learn how to pass data into and receive data from called methods. When you call a method from a program or other method, you should know four things:

- What the method does in general—in other words, you should know why you are calling the method
- The name of the called method
- What type of information to send to the method, if any
- What type of return data to expect from the method, if any

TWO TRUTHS & A LIE

Using Methods with no Parameters

1. When a method contains variable declarations, those variables are local to the method.
2. The values of variables declared locally within a method can be used by a calling method, but not by other methods.
3. In modern languages, the usual methodology is to declare variables locally in their methods and pass their values to other methods as needed.

The false statement is #2. When a variable is declared locally within a method, its value cannot be used by other methods. If its value is needed by a calling method, the value must be returned from the method.

Creating Methods that Require Parameters

Some methods require information to be sent in from the outside. When a program passes a data item to a method, the data item is an **argument to the method**, or more simply, an *argument*. When the method receives the data item, it is a **parameter to the method**, or more simply, a *parameter*. *Parameter* and *argument* are closely related terms. A calling method sends an argument to a called method. A called method accepts the value as its parameter.

If a method could not receive parameters, you would have to use global variables or write an infinite number of methods to cover every possible situation. As a real-life example, when you make a restaurant reservation, you do not need to employ a different method for every date of the year at every possible time of day. Rather, you can supply the date and time as information to the person who carries out the method. The method that records the reservation is carried out in the same manner, no matter what date and time are supplied.

As a programming example, if you design a `square()` method that multiplies a numeric value by itself, you should supply the method with a parameter that represents the value to be squared, rather than developing a `square1()` method that squares the value 1, a `square2()` method that squares the value 2, and so on. To call a `square()` method that accepts a parameter, you might write a statement that uses a constant, like `square(17)` or `square(86)`, or one that uses a variable, like `square(inputValue)`. The method uses the value of whatever argument you send.

When you write the declaration for a method that can receive a parameter, you must provide a **parameter list** that includes the following items for the parameter within the method declaration's parentheses:

- The type of the parameter
- A local name for the parameter.

Later in this chapter, you learn to create parameter lists with more than one parameter. A method's name and parameter list constitute the method's **signature**.

For example, suppose that you decide to improve the moon weight program in Figure 9-1 by making the final output more user-friendly and adding the explanatory text in the chosen language. It makes sense that if the user can request a prompt in a specific language, the user would want to see the output explanation in the same language. However, in Figure 9-1, the `langCode` variable is local to the `displayInstructions()` method and therefore cannot be used in the main program. You could rewrite the program by taking several approaches:

- You could rewrite the program without including any methods. That way, you could prompt the user for a language preference and display the prompt and the result in the appropriate language. This approach works, but you would not be taking advantage of the benefits provided by modularization. Those benefits include making the main program more streamlined and abstract, and making the `displayInstructions()`method a self-contained unit that can easily be transported to other programs—for example, applications that might determine a user's weight on Saturn or Mars.
- You could retain the `displayInstructions()` method, but make at least the `langCode` variable global by declaring it outside of any methods. If you took this approach, you would lose some of the portability of the `displayInstructions()` method because everything it used would no longer be contained within the method.
- You could retain the `displayInstructions()` method as is with its own local declarations, but add a section to the main program that also asks the user for a preferred language to display the result. The disadvantage to this approach is that the user must answer the same question twice during one execution of the program.
- You could store the variable that holds the language code in the main program so that it could be used to determine the result language. You could also retain the `displayInstructions()` method, but pass the language code to it so the prompt would appear in the appropriate language. This is the best choice because it employs modularity, which keeps the main program simpler and creates a portable method. A program that uses the method is shown in Figure 9-3, and a typical execution appears in Figure 9-4.

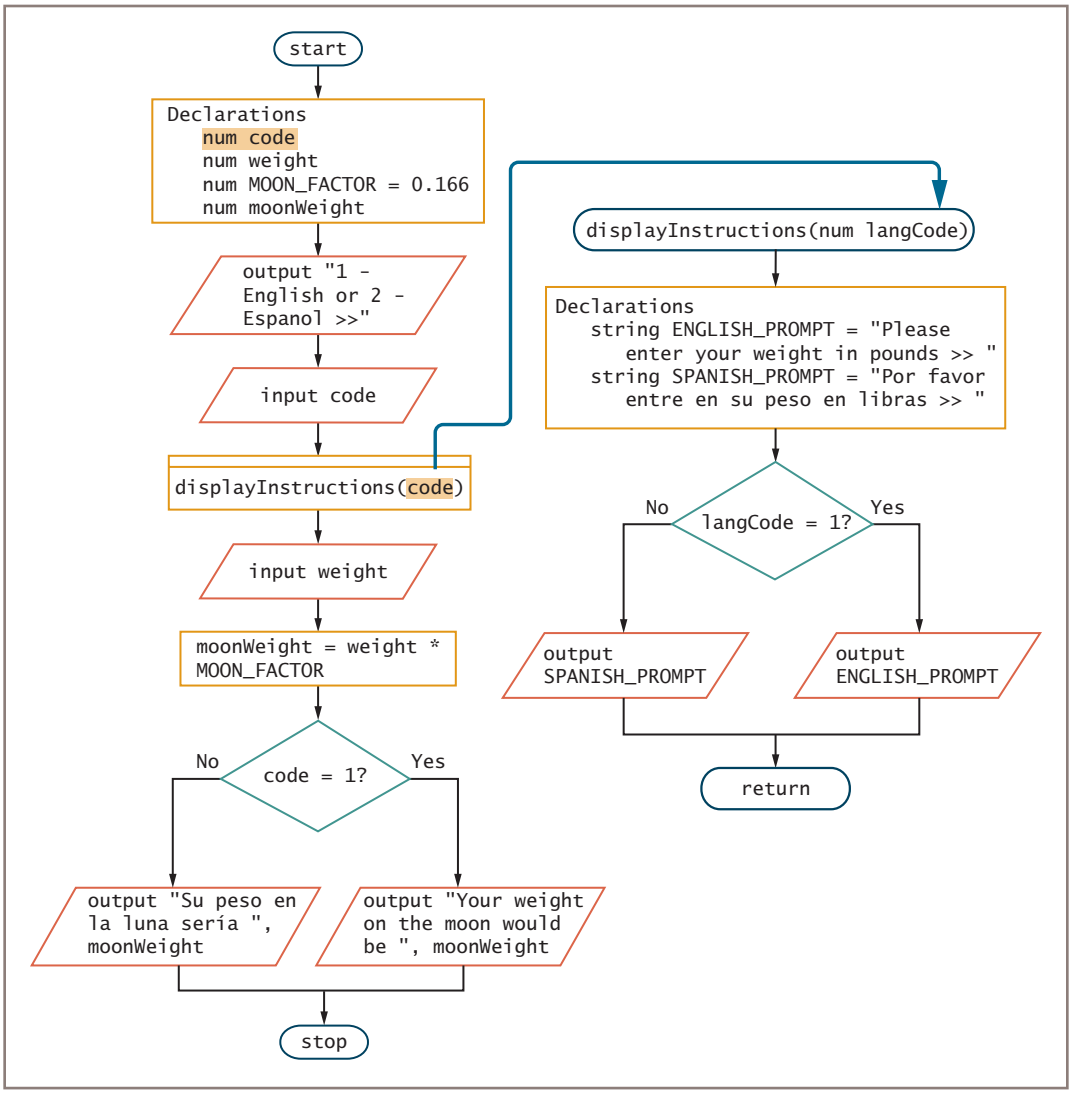


Figure 9-3 Moon weight program that passes an argument to a method *(continues)*

(*continued*)

```
start
   Declarations
      num code
      num weight
      num MOON_FACTOR = 0.166
      num moonWeight
   output "1 - English or 2 - Espanol >>"
   input code
   displayInstructions(code)
   input weight
   moonWeight = weight * MOON_FACTOR
   if code = 1 then
      output "Your weight on the moon would be ", moonWeight
   else
      output "Su peso en la luna sería ", moonWeight
   endif
stop

displayInstructions(num langCode)
   Declarations
      string ENGLISH_PROMPT = "Please enter your weight in pounds >> "
      string SPANISH_PROMPT = "Por favor entre en su peso en libras >> "
   if langCode = 1 then
      output ENGLISH_PROMPT
   else
      output SPANISH_PROMPT
   endif
return
```

Figure 9-3 Moon weight program that passes an argument to a method

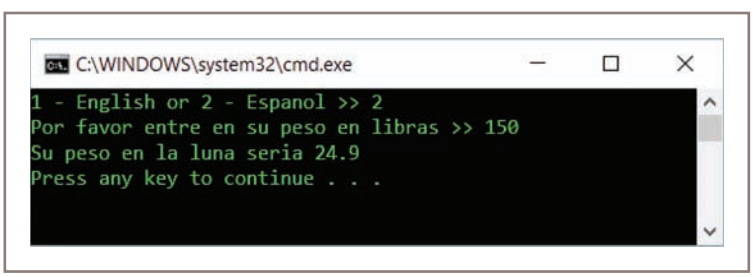


Figure 9-4 Typical execution of moon weight program in Figure 9-3

In the main program in Figure 9-3, a numeric variable named `code` is declared and the user is prompted for a value. The value then is passed to the `displayInstructions()` method. The value of the language code is stored in two places in memory:

- The main method stores the code in the variable `code` and passes it to `displayInstructions()` as an argument.

- The `displayInstructions()` method accepts the value of the argument `code` as the value of the parameter `langCode`. In other words, within the method, `langCode` takes on the value that `code` had in the main program.

You can think of the parentheses in a method declaration as a funnel into the method; parameters listed there hold values that are "dropped in" to the method. A variable passed into a method is **passed by value**; that is, a copy of its value is sent to the method and stored in a new memory location accessible to the method. The `displayInstructions()` method could be called using any numeric value as an argument, whether it is a variable, a named constant, a literal constant, or even an arithmetic expression. In other words, suppose that the main program contains the following declarations:

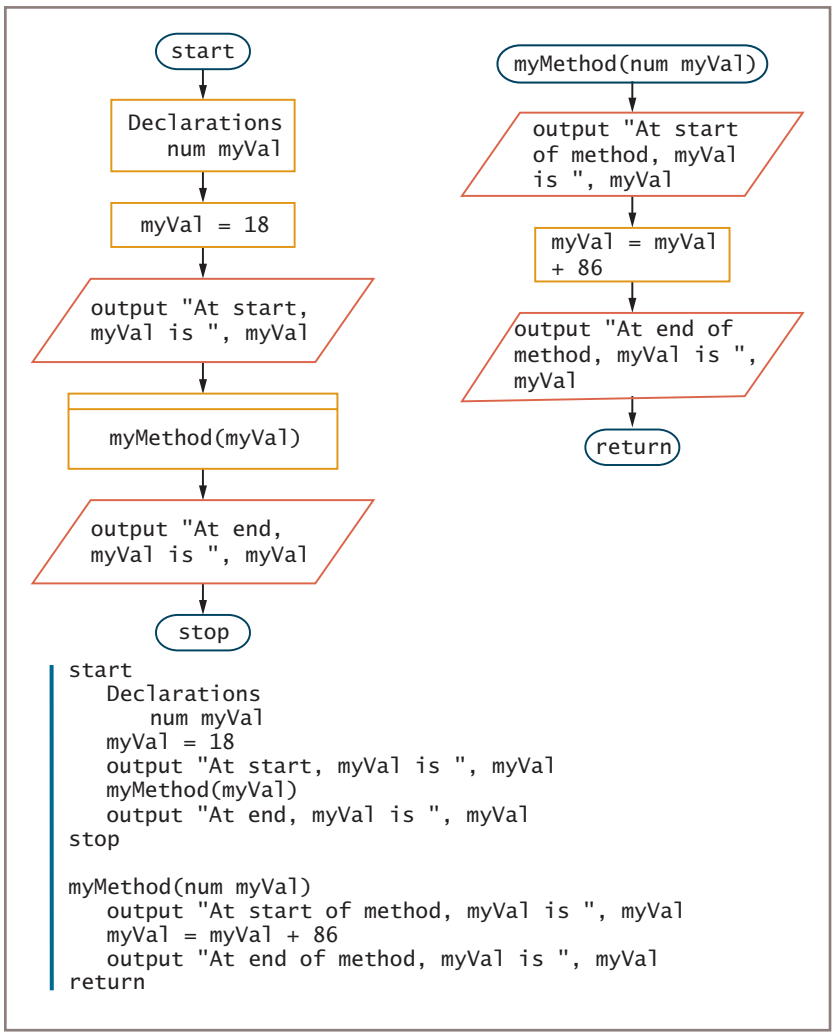
```
num x = 2
num langCode = 2
num SPANISH = 2
```

Then any of the following statements would work to call the `displayInstructions()` method:

- `displayInstructions(x)`, using a variable
- `displayInstructions(langCode)`, using a different variable
- `displayInstructions(SPANISH)`, using a named constant
- `displayInstructions(2)`, using a literal constant

If the value used as an argument in the method call is a variable or named constant, it might possess the same identifier as the parameter declared in the method header, or it might possess a different identifier. Within a method, the passed variable or named constant is simply a temporary placeholder; it makes no difference what name the variable or constant "goes by" in the calling program.

Each time a method executes, any parameters listed in the method header are redeclared—that is, new memory locations are reserved and named. When the method ends at the `return` statement, the locally declared parameter variables cease to exist. For example, Figure 9-5 shows a program that declares a variable, assigns a value to it, displays it, and sends it to a method. Within the method, the parameter is displayed, altered, and displayed again. When control returns to the main program, the original variable is displayed one last time. As the execution in Figure 9-6 shows, even though the variable in the method was altered, the original variable in the main program retains its starting value because it never was altered; it occupies a different memory address from the variable in the method.




```
start
   Declarations
      num myVal
   myVal = 18
   output "At start, myVal is ", myVal
   myMethod(myVal)
   output "At end, myVal is ", myVal
stop

myMethod(num myVal)
   output "At start of method, myVal is ", myVal
   myVal = myVal + 86
   output "At end of method, myVal is ", myVal
return
```

Figure 9-5 A program that calls a method in which the argument and parameter have the same identifier

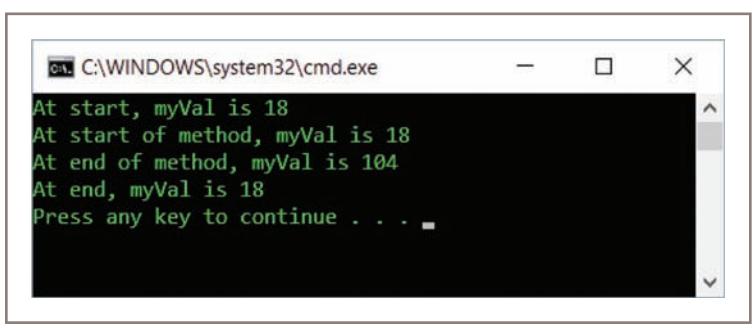


Figure 9-6 Execution of the program in Figure 9-5



Watch the video *Methods with a Parameter*.

Creating Methods that Require Multiple Parameters

You create and use a method with multiple parameters by doing the following:

- You list the arguments within the method call, separated by commas.
- You list a data type and local identifier for each parameter within the method header's parentheses, separating each declaration with a comma. Even if multiple parameters are the same data type, the type must be repeated with each parameter.



The arguments sent to a method in a method call are also called its *actual parameters*. The variables in the method declaration that accept the values from the actual parameters are *formal parameters*.

For example, suppose that you want to create a `computeTax()` method that calculates a tax on any value passed into it. You can create a method to which you pass two values—the amount to be taxed as well as a rate by which to tax it. Figure 9-7 shows a method that accepts two such parameters.

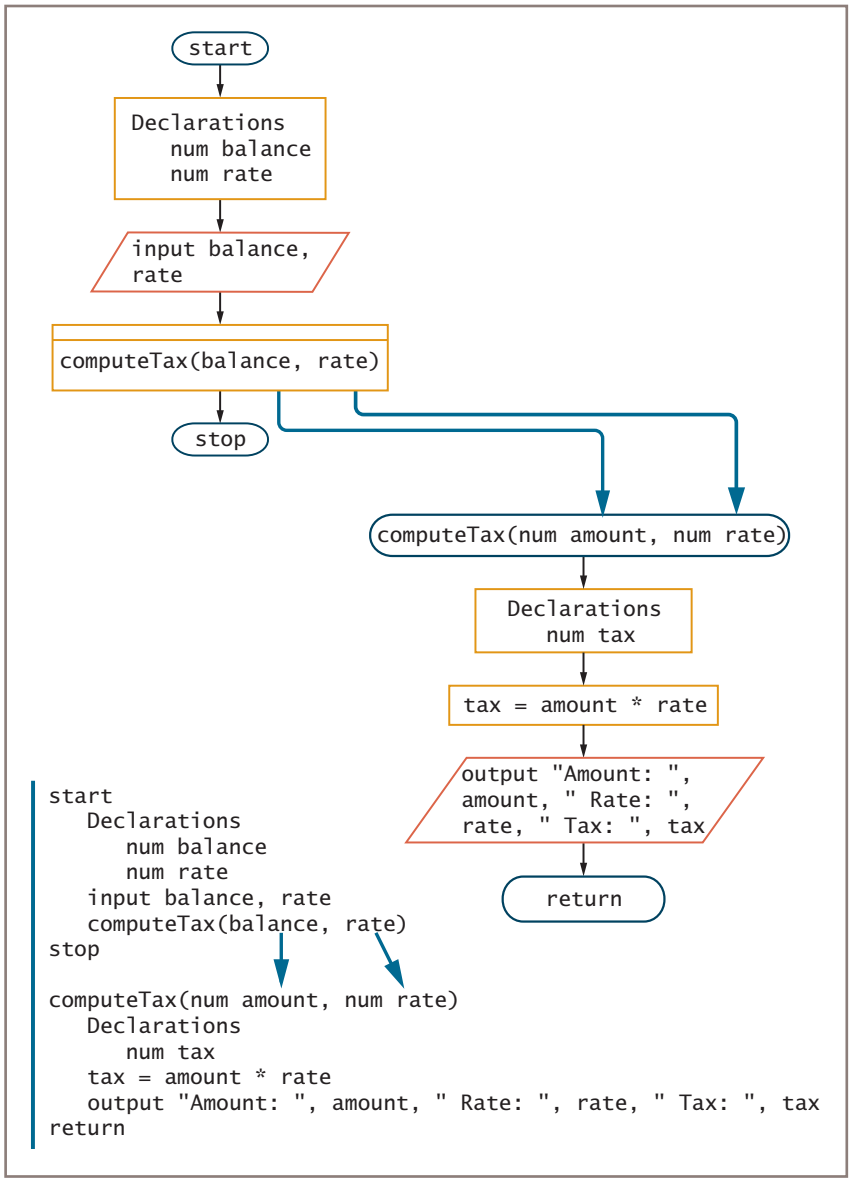


Figure 9-7 A program that calls a `computeTax()` method that requires two parameters



In Figure 9-7, notice that one of the arguments to the method has the same name as the corresponding method parameter, and the other has a different name from its corresponding parameter. Each could have the same identifier as its counterpart, or all could be different. Each identifier is local to its own method.

In Figure 9-7, two parameters (`num amount` and `num rate`) appear within the parentheses in the method header. A comma separates each parameter, and each requires its own declared

data type (in this case, both are numeric) as well as its own identifier. When multiple values are passed to a method, they are accepted into the parameters in the order in which they are passed. You can write a method so that it takes any number of parameters in any order. However, when you call a method, the arguments you send to the method must match in order—both in number and in type—the parameters listed in the method declaration. A call of `computeTax(rate, balance)` instead of `computeTax(balance, rate)` would result in incorrect values being displayed in the output statement. If method arguments are the same type—for example, two numeric arguments—passing them to a method in the wrong order results in a logical error. The program will compile and execute, but will produce incorrect results in most cases. If a method expects arguments of diverse types—for example, a number and a string—then passing arguments in the wrong order is a syntax error, and the program will not compile.



Watch the video *Methods with Multiple Parameters*.

TWO TRUTHS & A LIE

Creating Methods that Require Parameters

1. A value sent to a method from a calling program is a parameter.
2. When you write the declaration for a method that can receive parameters, you must include a data type for each parameter even if multiple parameters are the same type.
3. When a variable is used as an argument in a method call, it can have the same identifier as the parameter in the method header.

The false statement is #1. A calling method sends an argument; when the method receives the value, it is a parameter.

Creating Methods that Return a Value

A variable declared within a method ceases to exist when the method ends—it goes out of scope. When you want to retain a value that exists when a method ends, you can return the value from the method to the calling method. When a method returns a value, the method must have a **return type** that matches the data type of the returned value. A return type can be any type, which includes `num` and `string`, as well as other types specific to the programming language you are using. A method can also return nothing, in which case the return type is `void`, and the method is a **void method**. (The term *void* means "nothing" or "empty.") A method's return type is known more succinctly as a **method's type**, and it is listed in front of the method name when the method is defined. Previously, this book has

not included return types for methods because all the methods have been void. From this point forward, a return type is included with every method header.



Along with an identifier and parameter list, a return type is part of a method's declaration. A method's return type is not part of its signature, although you might hear some programmers claim that it is. Only the method name and parameter list constitute the signature.

For example, a method that returns the number of hours an employee has worked might have the following header:

```
num getHoursWorked()
```

This method returns a numeric value, so its type is `num`.

When a method returns a value, you usually want to use the returned value in the calling method, although this is not required. For example, Figure 9-8 shows how a program might use the value returned by the `getHoursWorked()` method. A variable named `hours` is declared in the main program. The `getHoursWorked()` method call is part of an assignment statement. When the method is called, the logical control is transferred to the `getHoursWorked()` method, which contains a variable named `workHours`. A value is obtained for this variable, which is returned to the main program where it is assigned to `hours`. After logical control returns to the main program from the `getHoursWorked()` method, the method's local variable `workHours` no longer exists. However, its value has been stored in the main program where, as `hours`, it can be displayed and used in a calculation.



As an example of when you might call a method but not use its returned value, consider a method that gets a character from the keyboard and returns its value to the calling program. In some applications, you would want to use the value of the returned characters. However, in other applications, you might want to tell the user to press any key. Then, you could call the method to accept the character from the keyboard, but you would not care which key was pressed or which key value was returned.

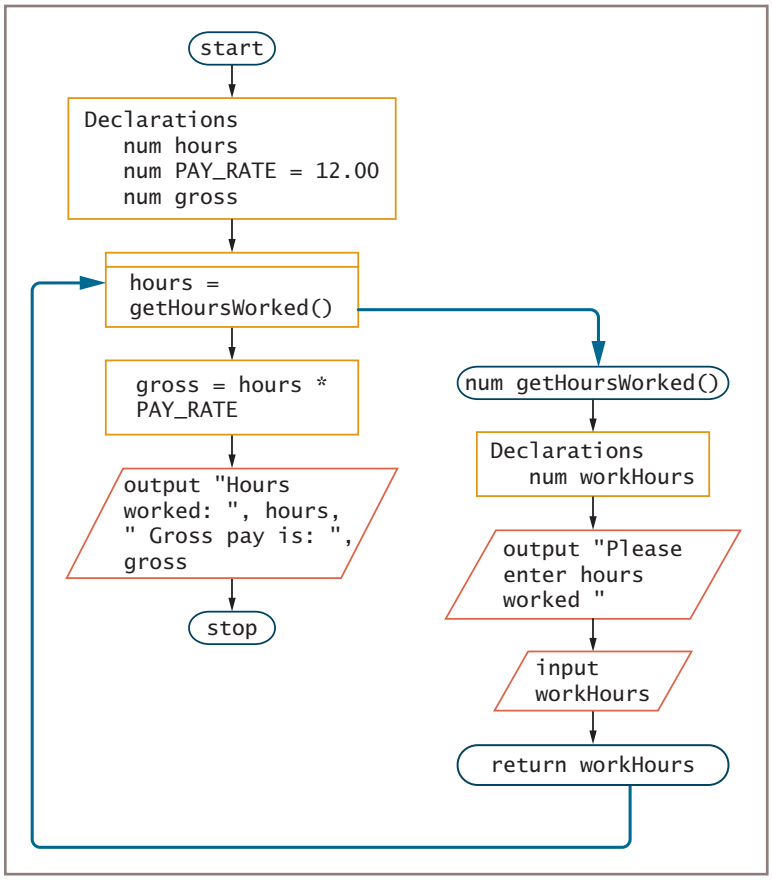


Figure 9-8 A payroll program that calls a method that returns a value

In Figure 9-8, notice the return type `num` that precedes the method name in the `getHoursWorked()` method header. A method's declared return type must match the type of the value used in the `return` statement; if it does not, the program will not compile. In Figure 9-8, a numeric value is correctly included in the `return` statement—the last statement in the `getHoursWorked()` method. When you place a value in a `return` statement, the value is sent from the called method back to the calling method.

A method's `return` statement can return one value at most. The returned value can be a variable or a constant. The value can be a simple or complex data type. For example, in Chapter 10 you will learn to create objects, which are more complex data types.

You are not required to assign a method's return value to a variable to use the value. Instead, you can use a method's returned value directly, without storing it. You use a method's value in the same way you would use any variable of the same type. For example, you can output a return value in a statement such as the following:

```
output "Hours worked is ", getHoursWorked()
```

Because `getHoursWorked()` returns a numeric value, you can use the method call `getHoursWorked()` in the same way that you would use any simple numeric value. Figure 9-9 shows an example of a program that uses a method's return value directly without storing it. The value of the shaded `workHours` variable returned from the method is used directly in the calculation of `gross` in the main program.

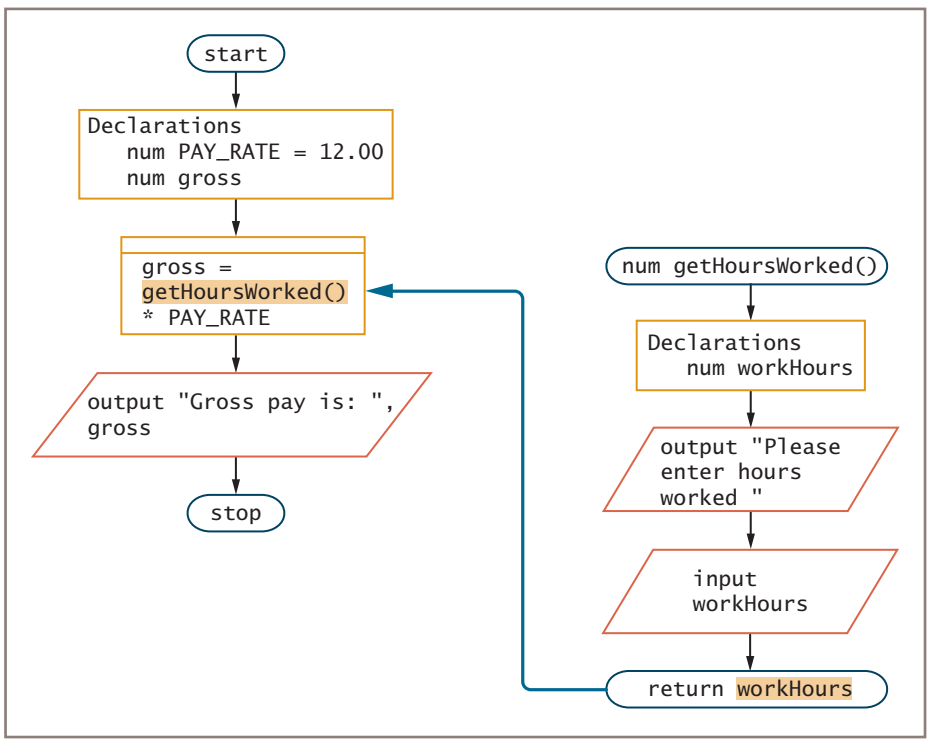


Figure 9-9 A program that uses a method's returned value without storing it



When a program needs to use a method's returned value in more than one place, it makes sense to store the returned value in a variable instead of calling the method multiple times. A program statement that calls a method requires more computer time and resources than a statement that does not call any outside methods. Programmers use the term **overhead** to describe any extra time and resources required by an operation.

As mentioned earlier, in most programming languages you technically are allowed to include multiple `return` statements in a method, but this book does not recommend the practice for most business programs. For example, consider the `findLargest()` method in Figure 9-10. The method accepts three parameters and returns the largest of the values. Although this method works correctly and you might see this technique used, its style is awkward and not structured. In Chapter 3, you learned that structured

logic requires each structure to contain one entry point and one exit point. The `return` statements in Figure 9-10 violate this convention by leaving decision structures before they are complete. Figure 9-11 shows the superior and recommended way to handle the problem. In Figure 9-11, the largest value is stored in a variable. Then, when the nested decision structure is complete, the stored value is returned in the last method statement.

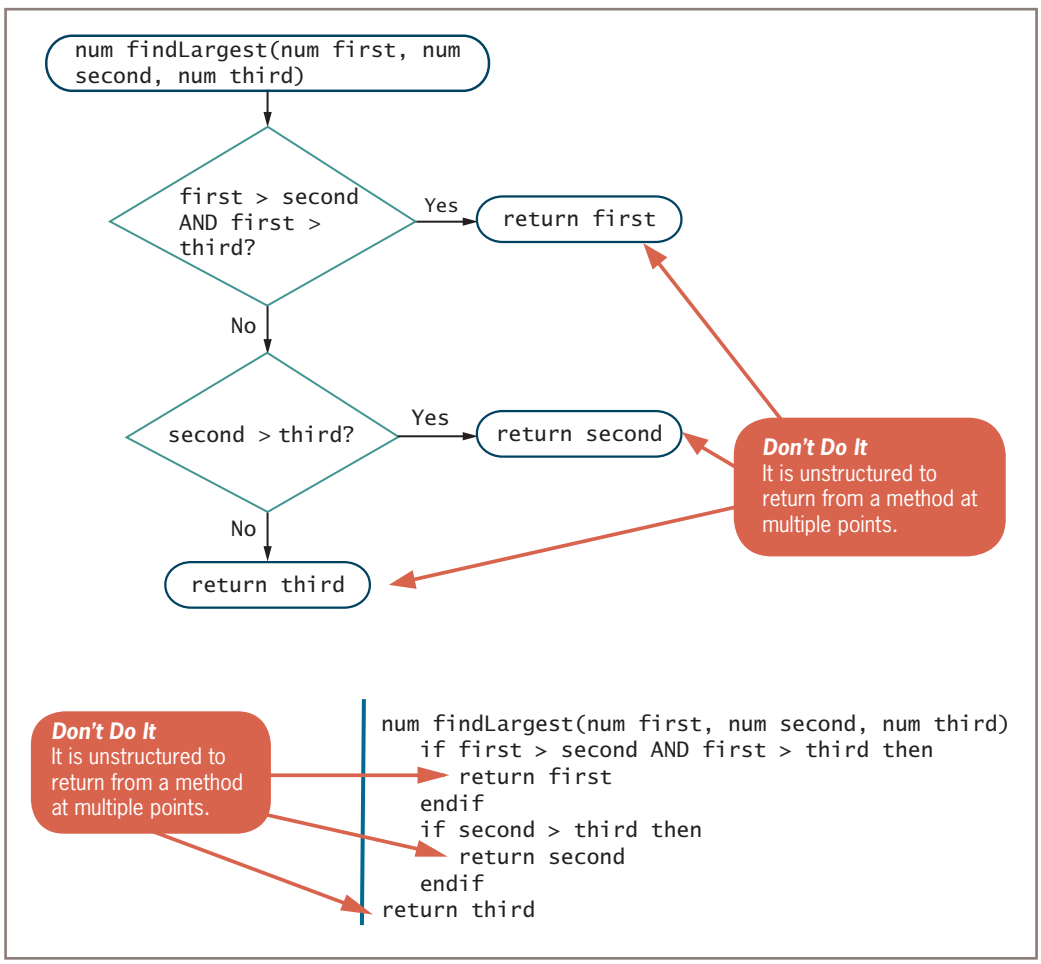
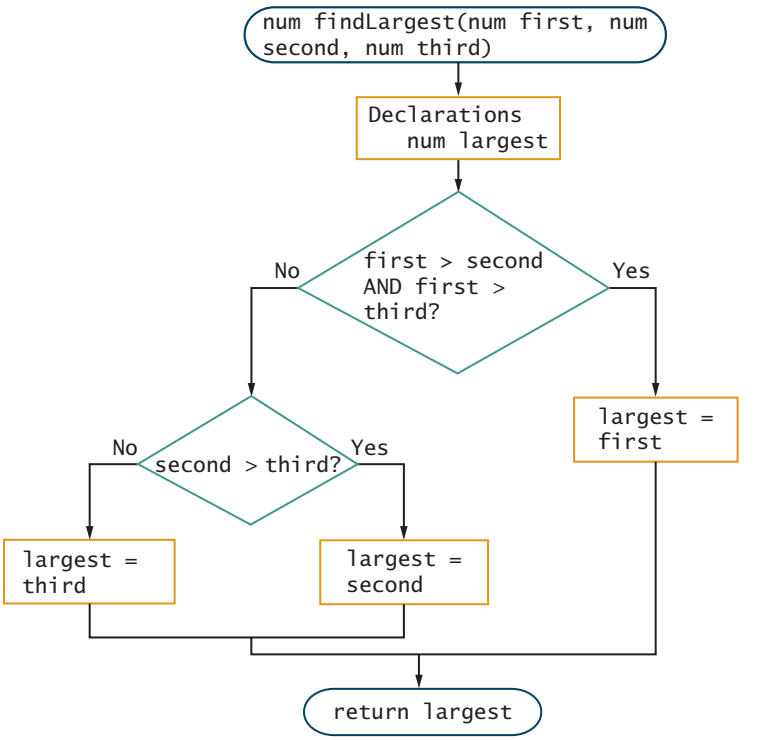


Figure 9-10 Unstructured approach to returning one of several values




```
num findLargest(num first, num second, num third)
   Declarations
      num largest
   if first > second AND first > third then
      largest = first
   else
      if second > third then
         largest = second
      else
         largest = third
      endif
   endif
return largest
```

Figure 9-11 Recommended, structured approach to returning one of several values

Using an IPO Chart

When designing methods to use within larger programs, some programmers find it helpful to use an **IPO chart**, a tool that identifies and categorizes each item needed within the method as pertaining to input, processing, or output. For example, consider a method that finds the smallest of three numeric values. When you think about designing this method,

you can start by placing each of its components in one of the three processing categories, as shown in Figure 9-12.

Input	Processing	Output
First value Second value Third value	If the firstvalue is smaller than each of the other two,save it as the smallest value; otherwise, if the second value is smaller than the third, save it as the smallest value; otherwise, save the third value as the smallest value	Smallest value

Figure 9-12 IPO chart for the method that finds the smallest of three numeric values

The IPO chart in Figure 9-12 provides an overview of the processing steps involved in the method. Like a flowchart or pseudocode, an IPO chart is just another tool to help you plan the logic of your programs. Many programmers create an IPO chart only for specific methods in their programs and as an alternative to flowcharting or writing pseudocode. IPO charts provide an overview of input to the method, the processing steps that must occur, and the resulting output. This book emphasizes creating flowcharts and pseudocode, but you can find many more examples of IPO charts on the Web.

TWO TRUTHS & A LIE

Creating Methods that Return a Value

1. The return type for a method can be any type, which includes numeric, character, and string, as well as other more specific types that exist in the programming language you are using.
2. A method's return type must be the same type as one of the method's parameters.
3. You are not required to use a method's returned value.

The false statement is #2. The return type of a method can be any type. The return type must match the type of value in the method's return statement. A method's return type is not required to match any of the method's parameters.

Passing an Array to a Method

In Chapter 6, you learned that you can declare an array to create a list of elements, and that you can use any individual array element in the same manner you would use any single variable of the same type. For example, suppose that you declare a numeric array as follows:

```
num someNums[12]
```

You can subsequently output `someNums[0]` or perform arithmetic with `someNums[11]`, just as you would for any simple variable that is not part of an array. Similarly, you can pass a single array element to a method in exactly the same manner you would pass a variable or constant.

Consider the program shown in Figure 9-13. This program creates an array of four numeric values and then outputs them. Next, the program calls a method named `tripleTheValue()` four times, passing each of the array elements in turn. The method outputs the passed value, multiplies it by 3, and outputs it again. Finally, back in the calling program, the four numbers are output again. Figure 9-14 shows an execution of this program in a command-line environment.

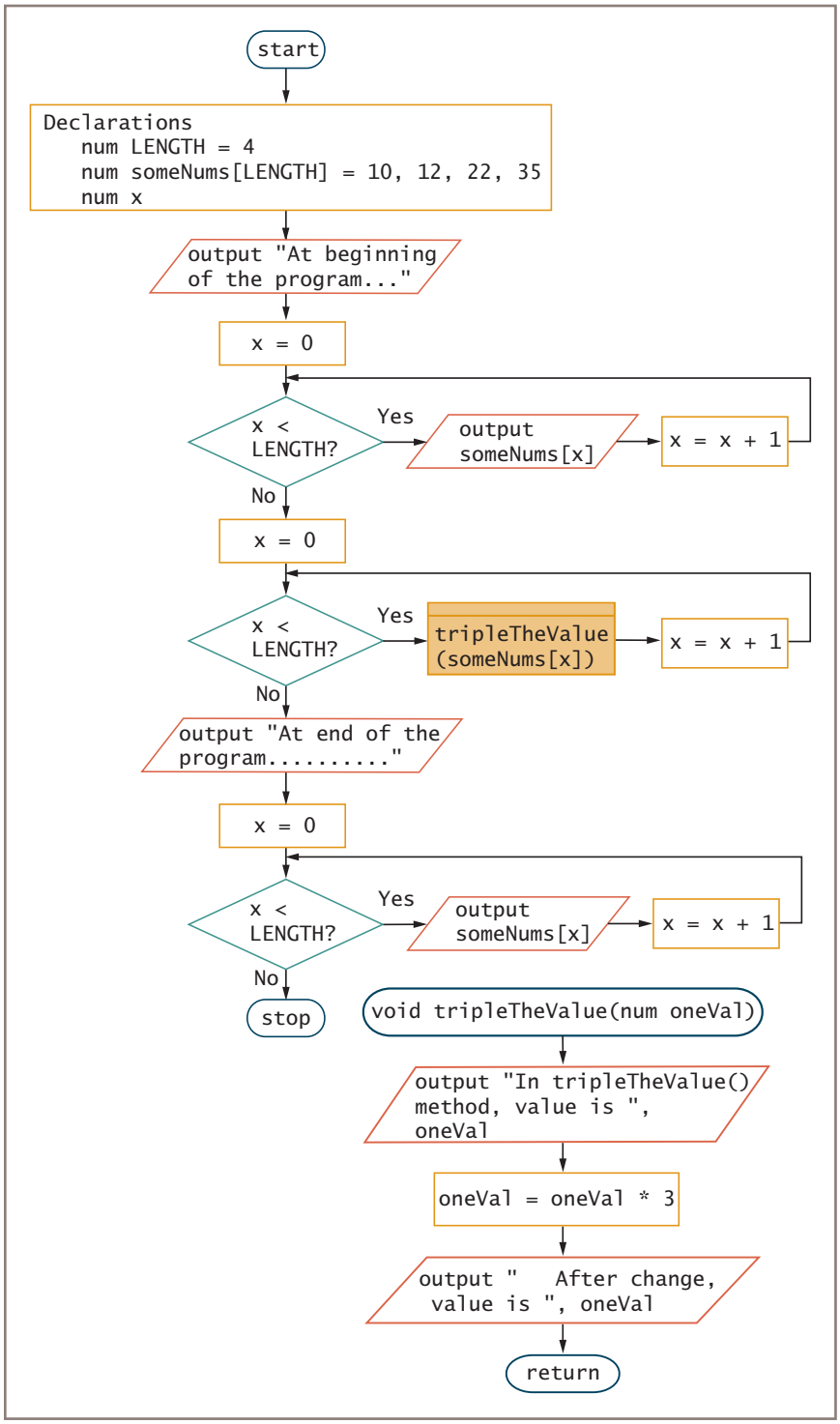


Figure 9-13 `PassArrayElement` program *(continues)*

(*continued*)

```
start
   Declarations
      num LENGTH = 4
      num someNums[LENGTH] = 10, 12, 22, 35
      num x
   output "At beginning of the program..."
   x = 0
   while x < LENGTH
      output someNums[x]
      x = x + 1
   endwhile
   x = 0
   while x < LENGTH
      tripleTheValue(someNums[x])
      x = x + 1
   endwhile
   output "At end of the program........."
   x = 0
   while x < LENGTH
      output someNums[x]
      x = x + 1
   endwhile
stop

void tripleTheValue(num oneVal)
   output "In tripleTheValue() method, value is ", oneVal
   oneVal = oneVal * 3
   output "     After change, value is ", oneVal
return
```

Figure 9-13 `PassArrayElement` program

```
C:\WINDOWS\system32\cmd.exe
At beginning of the program...    10    12    22    35
In tripleTheValue() method, value is 10
    After change, value is 30
In tripleTheValue() method, value is 12
    After change, value is 36
In tripleTheValue() method, value is 22
    After change, value is 66
In tripleTheValue() method, value is 35
    After change, value is 105
At end of the program.........   10    12    22    35
Press any key to continue . . .
```

Figure 9-14 Output of the `PassArrayElement` program

As you can see in Figure 9-14, the program displays the four original values, then passes each value to the `tripleTheValue()` method, where it is displayed, multiplied by 3, and displayed again. After the method executes four times, the logic returns to the main program where the four values are displayed again, showing that they are unchanged by the statements within `tripleTheValue()`. The `oneVal` variable is local to the `tripleTheValue()` method; therefore, any changes to it are not permanent and are not reflected in the array declared in the main program. The `oneVal` variable in the `tripleTheValue()` method holds only a copy of each array element passed into the method, and although `oneVal` is altered in the method, each newly assigned, larger value exists only while the `tripleTheValue()` method is executing. In all respects, a single array element acts just like any single variable of the same type would.

Instead of passing a single array element to a method, you can pass an entire array as an argument. You can indicate that a method parameter must be an array by using the convention of placing square brackets after the data type in the method's parameter list. Arrays, unlike simple built-in types, are **passed by reference**; the method receives the actual memory address of the array and has access to the actual values in the array elements. The name of an array represents a memory address, and the subscript used with an array name represents an offset from that address. Therefore, when you pass an array to a method, changes you make to array elements within the method are reflected in the original array that was sent to the method.



Some languages, such as Visual Basic, use parentheses after an identifier to indicate an array as a parameter to a method. Many other languages, including Java, C++, and C#, use square brackets after the data type. Because this book uses parentheses following method names, it uses brackets to indicate arrays.

The program shown in Figure 9-15 creates an array of four numeric values. After the numbers are output, the entire array is passed to a method named `quadrupleTheValues()`. Within the method header, the parameter is declared as an array by using square brackets after the parameter type. Within the method, the numbers are output, which shows that they retain their values from the main program upon entering the method. Then the array values are multiplied by 4. Even though `quadrupleTheValues()` returns nothing to the calling program, when the program displays the array for the last time within the mainline logic, all of the values have been changed to their new quadrupled values. Figure 9-16 shows an execution of the program. Because arrays are passed by reference, the `quadrupleTheValues()` method "knows" the address of the array declared in the calling program and makes its changes directly to the original array that was declared in the calling program.

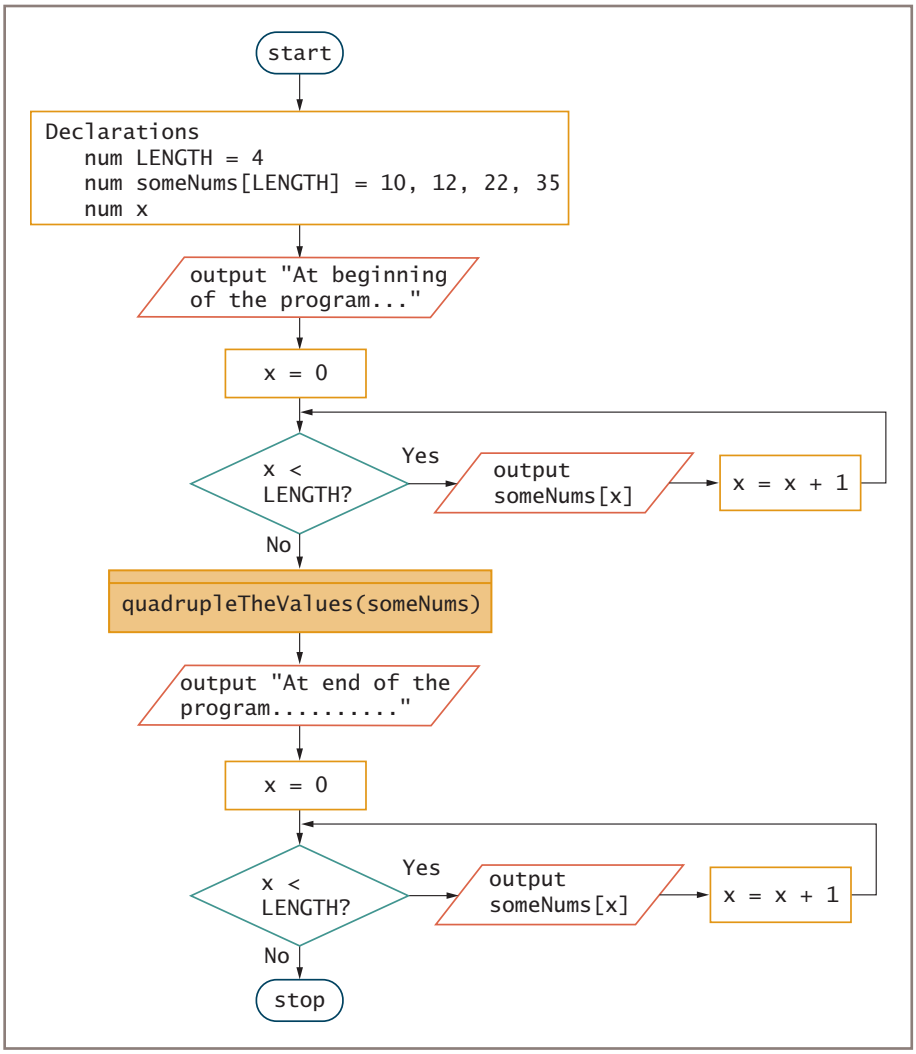


Figure 9-15 `PassEntireArray` program *(continues)*

(*continued*)

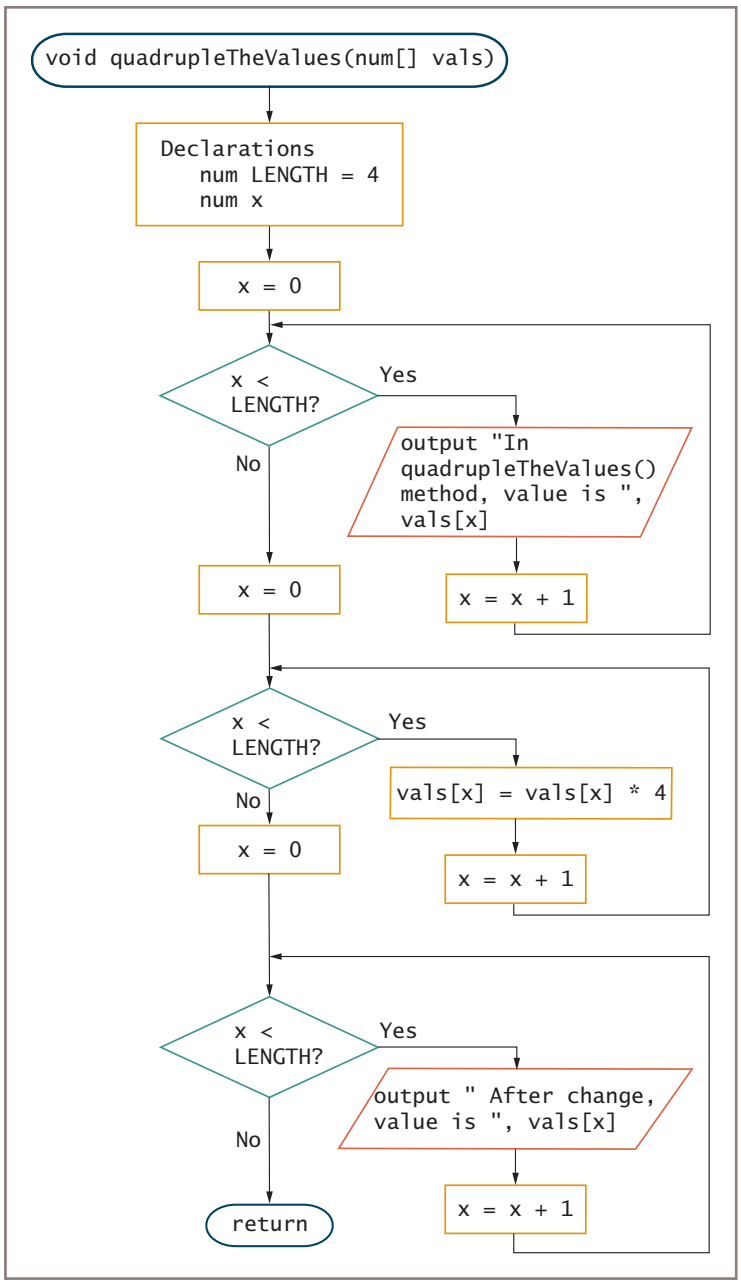


Figure 9-15 `PassEntireArray` program *(continues)*

(*continued*)

```
start
   Declarations
      num LENGTH = 4
      num someNums[LENGTH] = 10, 12, 22, 35
      num x
   output "At beginning of the program..."
   x = 0
   while x < LENGTH
      output someNums[x]
      x = x + 1
   endwhile
   quadrupleTheValues(someNums)
   output "At end of the program.........."
   x = 0
   while x < LENGTH
      output someNums[x]
      x = x + 1
   endwhile
stop

void quadrupleTheValues(num[] vals)
   Declarations
      num LENGTH = 4
      num x
   x = 0
   while x < LENGTH
      output "In quadrupleTheValues() method, value is ", vals[x]
      x = x + 1
   endwhile
   x = 0
   while x < LENGTH
      vals[x] = vals[x] * 4
      x = x + 1
   endwhile
   x = 0
   while x < LENGTH
      output "     After change, value is ", vals[x]
      x = x + 1
   endwhile
return
```

Figure 9-15 PassEntireArray program

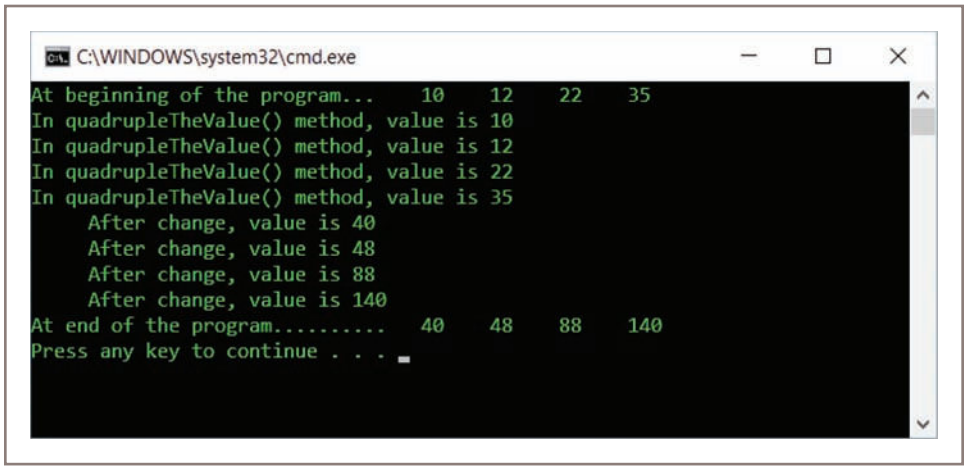


Figure 9-16 Output of the `PassEntireArray` program

When an array is a method parameter, the square brackets in the method header remain empty and do not hold a size. The array name that is passed is a memory address that indicates the start of the array. Depending on the language you are working in, you can control the values you use for a subscript to the array in different ways. In some languages, you might also want to pass a constant that indicates the array size to the method. In other languages, you can access the automatically created length field for the array. Either way, the array size itself is never implied when you use the array name. The array name indicates only the starting point from which subscripts will be used.

TWO TRUTHS & A LIE

Passing an Array to a Method

1. You can pass an entire array as a method's argument.
2. You can indicate that a method parameter must be an array by placing square brackets after the data type in the method's parameter list.
3. Arrays, unlike simple built-in types, are passed by value; the method receives a copy of the original array.

The false statement is #3. Arrays, unlike simple built-in types, are passed by reference; the method receives the actual memory address of the array and has access to the actual values in the array elements.

Overloading Methods

In programming, **overloading** involves supplying diverse meanings for a single identifier. When you use the English language, you frequently overload words. When you say *break a window, break bread, break the bank*, and *take a break*, you describe four very different actions that use different methods and produce different results. However, anyone who speaks English well comprehends your meaning because *break* is understood in the context of the discussion.



In most programming languages, some operators are overloaded. For example, a + between two values indicates addition, but a single + to the left of a value means the value is positive. The + sign has different meanings based on the arguments used with it.



Overloading a method is an example of **polymorphism**—the ability of a method to act appropriately according to the context. Literally, *polymorphism* means "many forms."

When you **overload a method**, you write multiple methods with a shared name but different parameter lists. When you call an overloaded method, the language translator understands which version of the method to use based on the arguments used. For example, suppose that you create a method to output a message and the amount due on a customer bill, as shown in Figure 9-17. The method receives a numeric parameter that represents the customer's balance and produces two lines of output. Assume that you also need a method that is similar to `printBill()`, except the new method applies a discount to the customer bill. One solution to this problem would be to write a new method with a different name—for example, `printBillWithDiscount()`. A downside to this approach is that a programmer who uses your methods must remember the names of each slightly different version. It is more natural for your methods' clients to use a single well-designed method name for the task of printing bills, but to be able to provide different arguments as appropriate. In this case, you can overload the `printBill()` method so that, in addition to the version that takes a single numeric argument, you can create a version that takes two numeric arguments—one that represents the balance and one that represents the discount rate. Figure 9-17 shows the two versions of the `printBill()` method.

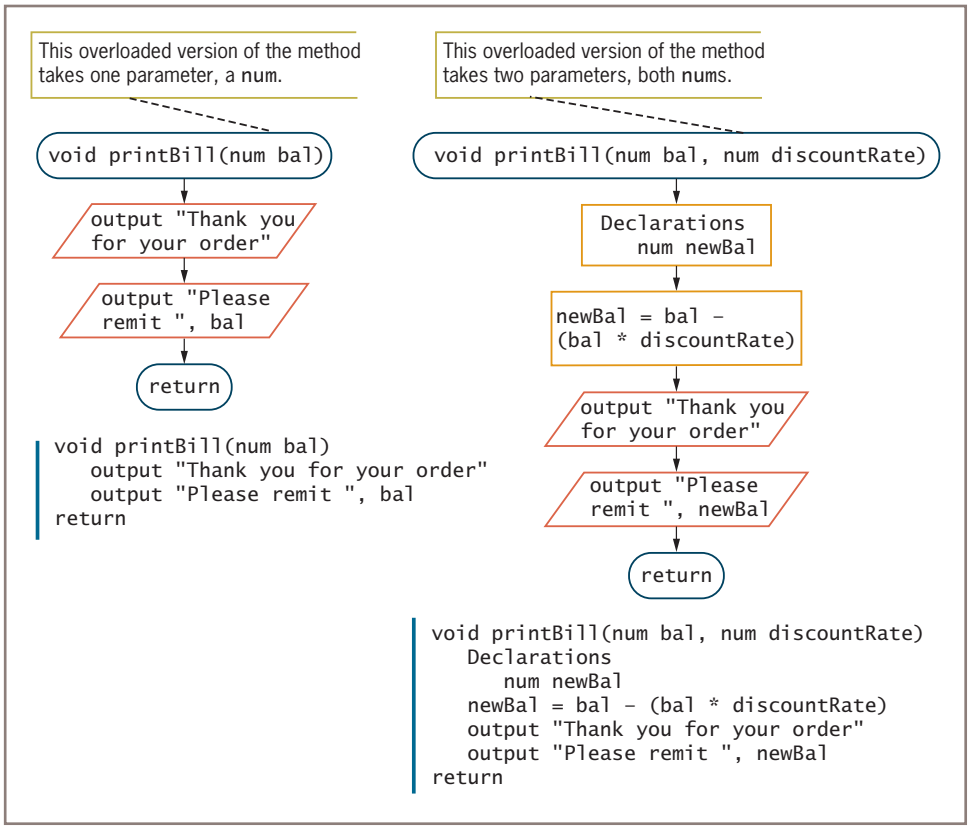


Figure 9-17 Two overloaded versions of the `printBill()` method

If both versions of `printBill()` are included in a program and you call the method using a single numeric argument, as in `printBill(custBalance)`, the first version of the method in Figure 9-17 executes. If you use two numeric arguments in the call, as in `printBill (custBalance, rate)`, the second version of the method executes.

If it suited your needs, you could provide more versions of the `printBill()` method, as shown in Figure 9-18. The first version accepts a numeric parameter that holds the customer's balance, and a string parameter that holds an additional message that can be customized for the bill recipient and displayed on the bill. For example, if a program makes a method call such as the following, the first version of `printBill()` in the figure will execute:

```
printBill(custBal, "Due in 10 days")
```

The second version of the method in Figure 9-18 accepts three parameters, providing a balance, discount rate, and customized message. For example, the following method call would use this second version of the method in the figure:

```
printBill(balanceDue, discountRate, specialMessage)
```

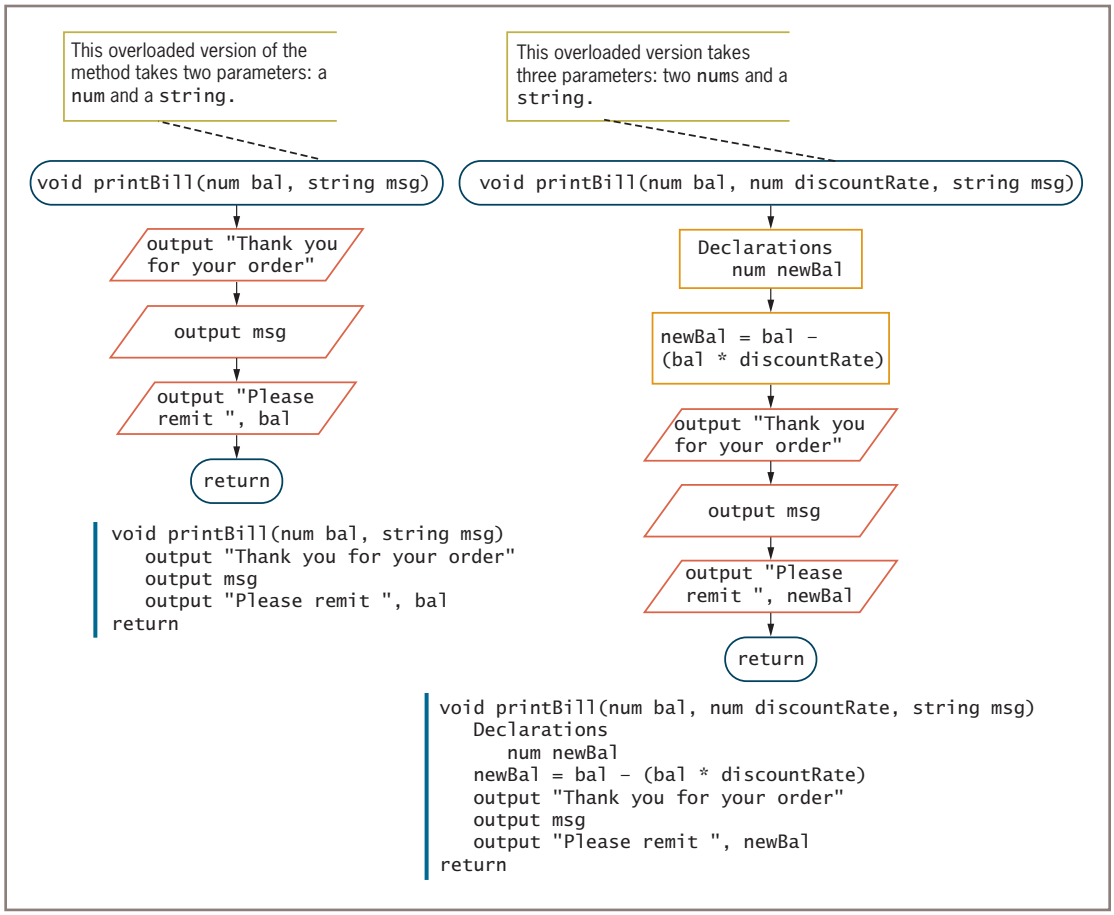


Figure 9-18 Two additional overloaded versions of the `printBill()` method

Overloading methods is never required in a program. Instead, you could create multiple methods with unique identifiers such as `printBill()` and `printBillWithDiscountAndMessage()`. Overloading methods does not reduce your work when creating a program; you need to write each method individually. The advantage is provided to your method's clients; those who use your methods need to remember just one appropriate name for all related tasks.



In many programming languages, the `output` statement is actually an overloaded method that you call. Using a single name such as `output`, whether you want to output a number, a `string`, or any combination of the two, is convenient.

Even if you write two or more overloaded versions of a method, many program clients will use just one version. For example, suppose that you develop a bill-creating program that contains all four versions of the `printBill()` method just discussed, and then sell it

to different companies. An organization that adopts your program and its methods might want to use only one or two versions of the method. You probably own many devices for which only some of the features are meaningful to you; for example, some people who own microwave ovens use only the *Popcorn* button or never use *Defrost*.

Avoiding Ambiguous Methods

When you overload a method, you run the risk of creating **ambiguous methods**—a situation in which the compiler cannot determine which method to use. Every time you call a method, the compiler decides whether a suitable method exists; if so, the method executes, and if not, you receive an error message. For example, suppose that you write two versions of a `printBill()` method, as shown in the program in Figure 9-19. One version of the method is intended to accept a customer balance and a discount rate, and the other is intended to accept a customer balance and a discount amount expressed in dollars.

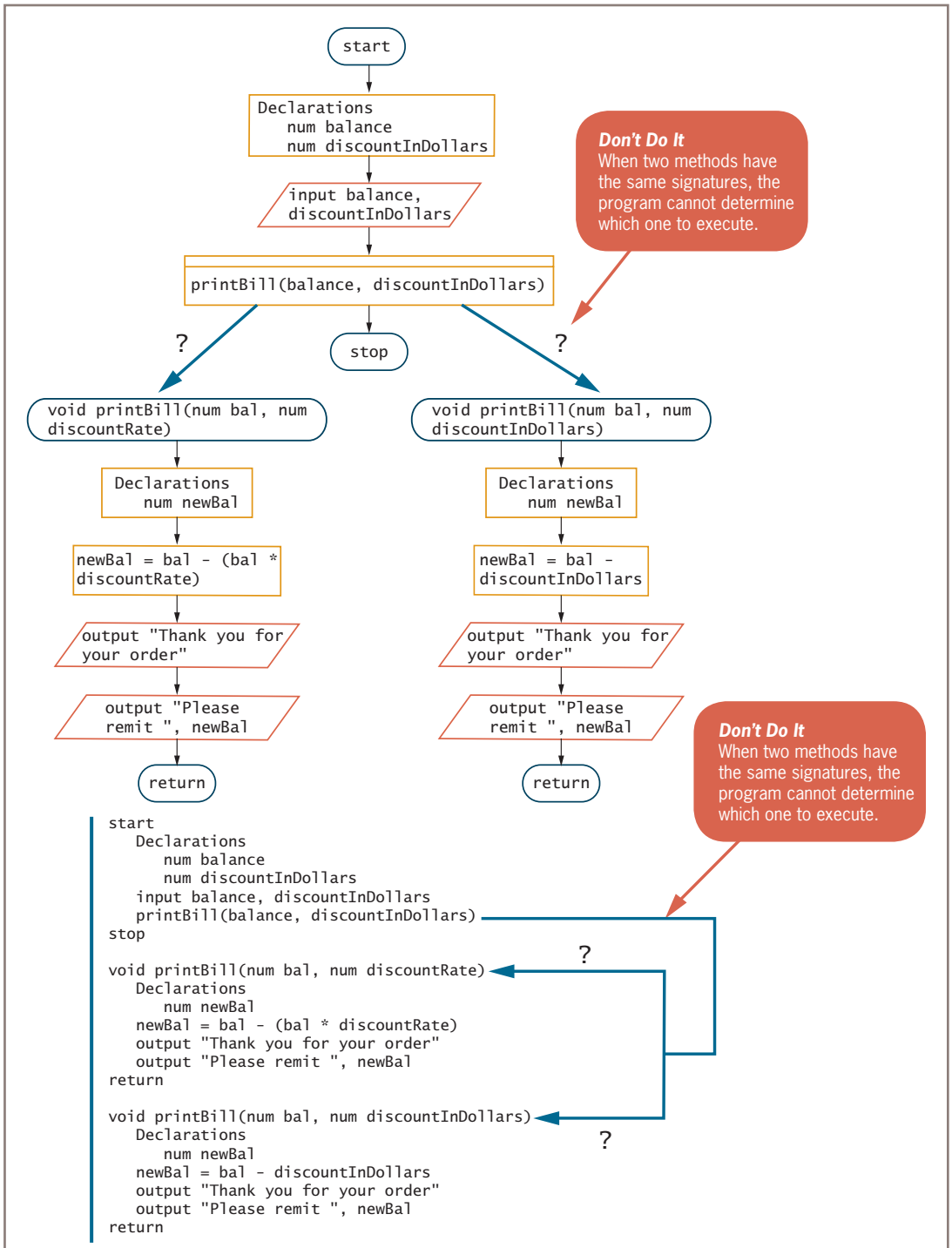


Figure 9-19 Program that contains ambiguous method call

Each of the two versions of `printBill()` in Figure 9-19 is a valid method on its own. However, when the two versions exist in the same program, a problem arises. When the main program calls `printBill()` using two numeric arguments, the compiler cannot determine which version to call. You might think that the version of the method with a parameter named `discountInDollars` would execute, because the method call uses the identifier `discountInDollars`. However, the compiler determines which version of a method to call based on argument data types only, not their identifiers. Because both versions of the `printBill()` method could accept two numeric parameters, the compiler cannot determine which version to execute, so an error occurs and program compilation stops.



An overloaded method is not ambiguous on its own—it becomes ambiguous only if you make a method call that matches multiple method signatures. In many languages, a program with potentially ambiguous methods will run without problems if you don't make any method calls that match more than one method.

Methods can be overloaded correctly by providing different parameter lists for methods with the same name. Methods with identical names that have identical parameter lists but different return types are not overloaded—they are ambiguous. For example, the following two method headers create ambiguity:

```
string aMethod(num x)
num aMethod(num y)
```

The compiler determines which version of a method to call based on parameter lists, not return types. When the method call `aMethod(17)` is made, the compiler will not know which of the two methods to execute because both possible choices take a numeric argument.



All the popular object-oriented programming languages support multiple numeric data types. For example, Java, C#, C++, and Visual Basic all support integer (whole number) data types that are different from floating-point (decimal place) data types. Many languages have even more specialized numeric types, such as signed and unsigned. Methods that accept different specific types are correctly overloaded.



Watch the video *Overloading Methods*.

TWO TRUTHS & A LIE

Overloading Methods

1. In programming, overloading involves supplying diverse meanings for a single identifier.
2. When you overload a method, you write multiple methods with different names but identical parameter lists.
3. Methods can be overloaded correctly by providing different parameter lists for methods with the same name.

The false statement is #2. When you overload a method, you write methods with a shared name but different parameter lists.

Using Predefined Methods

All modern programming languages contain many methods that have already been written for programmers. Predefined methods might originate from several sources:

- Some prewritten methods are built into a language. For example, methods that perform input and output are usually predefined.
- When you work on a program in a team, each programmer might be assigned specific methods to create, and your methods will interact with methods written by others.
- If you work for a company, many standard methods might already have been written and you will be required to use them. For example, the company might have a standard method that displays its logo.

Predefined methods save you time and effort. For example, in most languages, displaying a message on the screen involves using a built-in method. When you want to display *Hello* on the command prompt screen in C#, you write the following:

```
Console.WriteLine("Hello");
```

In Java, you write:

```
System.out.println("Hello");
```

In these statements, you can recognize `WriteLine()` and `println()` as method names because they are followed by parentheses; the parentheses hold an argument that represents the message to display. If these methods were not prewritten, you would have to know the low-level details of how to manipulate pixels on a screen to display the characters. Instead, by using the prewritten methods, you can concentrate on the higher-level task of displaying a useful and appropriate message.



In C#, the convention is to begin method names with an uppercase letter. In Java, method names conventionally begin with a lowercase letter. `The WriteLine()` and `println()` methods follow their respective language's convention. The `WriteLine()` and `println()` methods are both overloaded in their respective languages. For example, if you pass a string to either method, the version of the method that accepts a string parameter executes, but if you pass a number, another version that accepts a numeric parameter executes.

Most programming languages also contain a variety of mathematical methods, such as those that compute the square root or absolute value of a number. Other methods retrieve the current date and time from the operating system or select a random number to use in a game application. These methods were written as a convenience for you—computing a square root and generating random numbers are complicated tasks, so it is convenient to have methods already written, tested, and available when you need them. The names of the methods that perform these functions differ among programming languages, so you need to research the language's documentation to use them. For example, many of a language's methods are described in introductory programming language textbooks, and you can also find language documentation online.

Whether you want to use a predefined method or any other method, you should know the following four details:

- What the method does in general—for example, compute a square root.
- The method's name—for example, it might be `sqrt()`.
- The method's required parameters—for example, a square root method might require a single numeric parameter. There might be multiple overloaded versions of the method from which you can choose. For example, one method version might accept an integer and another version might accept a floating-point number.
- The method's return type—for example, a square root method most likely returns a numeric value that is the square root of the argument passed to the method.

You do not need to know how the method is implemented—that is, how the instruction statements are written within it. Like all methods, you can use built-in methods without worrying about their low-level implementation details.

TWO TRUTHS & A LIE

Using Predefined Methods

1. The name of a method that performs a specific function (such as generating a random number) is likely to be the same in various programming languages.
2. When you want to use a predefined method, you should know what the method does in general, along with its name, required parameters, and return type.
3. When you want to use a predefined method, you do not need to know how the method works internally to be able to use the method effectively.

The false statement is #1. Methods that perform standard functions are likely to have different names in various languages.

Method Design Issues: Implementation Hiding, Cohesion, and Coupling

To design effective methods, you should consider several program qualities:

- You should employ implementation hiding.
- You should strive to increase cohesion.
- You should strive to reduce coupling.

Understanding Implementation Hiding

An important principle of modularization is the notion of **implementation hiding**, the encapsulation of method details. That is, when a program makes a request to a method, it doesn't know the details of how the method is executed. For example, when you make a restaurant reservation, you do not need to know how the reservation is actually recorded at the restaurant—perhaps it is written in a book, marked on a large chalkboard, or entered into a computerized database. The implementation details don't concern you as a patron, and if the restaurant changes its methods from one year to the next, the change does not affect your use of the reservation method—you still call and provide your name, a date, and a time. With well-written methods, using implementation hiding means that a method that calls another must know only the following:

- The name of the called method
- What type of information to send to the method
- What type of return data to expect from the method.

In other words, the calling method needs to understand only the **interface to the method** that is called. The interface is the only part of a method with which the method's client (or method's caller) interacts. The program does *not* need to know how the method works internally. Additionally, if you substitute a new, improved method implementation but the interface to the method does not change, you won't need to make changes in any methods that call the altered method. Programmers refer to hidden implementation details as existing in a **black box**—you can examine what goes in and what comes out, but not the details of how the method works inside.

Increasing Cohesion

When you begin to design computer programs, it is difficult to decide how much to put into a method. For example, a process that requires 40 instructions can be contained in a single 40-instruction method, two 20-instruction methods, five 8-instruction methods, or many other combinations. In most programming languages, any of these combinations is allowed; you can write a program that executes and produces correct results no matter how you divide the individual steps into methods. However, placing too many or too few instructions in a single method makes a program harder to follow and reduces flexibility.

To help determine the appropriate division of tasks among methods, you want to analyze each method's **cohesion**, which refers to how the internal statements of a method serve to accomplish the method's purpose. In highly cohesive methods, all the operations are related, or "go together." Such methods demonstrate **functional cohesion**—all their operations contribute to the performance of a single task. Functionally cohesive methods usually are more reliable than those that have low cohesion; they are considered stronger, and they make programs easier to write, read, and maintain.

For example, consider a method that calculates gross pay. The method receives parameters that define a worker's pay rate and number of hours worked. The method computes gross pay and displays it. The cohesion of this method is high because each of its instructions contributes to one task—computing gross pay. If you can write a sentence describing what a method does using only two words—for example, *Compute gross*, *Cube value*, or *Display record*—the method is probably functionally cohesive.

You might work in a programming environment that has a rule such as *No method will be longer than can be printed on one page* or *No method will have more than 30 lines of code*. The rule maker is trying to achieve more cohesion, but such rules are arbitrary. A two-line method could have low cohesion and a 40-line method might have high cohesion. Because good, functionally cohesive methods perform only one task, they tend to be short. However, the issue is not size. If it takes 20 statements to perform one task within a method, the method is still cohesive.

Most programmers do not consciously make decisions about cohesiveness for each method they write. Rather, they develop a "feel" for what types of tasks belong together, and for which subsets of tasks should be diverted to their own methods.

Reducing Coupling

Coupling is a measure of the strength of the connection between two program methods; it expresses the extent to which information is exchanged by methods. Coupling is either tight or loose, depending on how much one method relies on information from another. **Tight coupling,** which occurs when methods depend on each other excessively, makes programs more prone to errors. With tight coupling, you have many data paths to keep track of, many chances for bad data to pass from one method to another, and many chances for one method to alter information needed by another method. **Loose coupling** occurs when methods do not depend on others. In general, you want to reduce coupling as much as possible because connections between methods make them more difficult to write, maintain, and reuse.

Imagine four cooks wandering in and out of a kitchen while preparing a stew. If each is allowed to add seasonings at will without the knowledge of the other cooks, you could end up with a culinary disaster. Similarly, if four payroll program methods can alter your gross pay without the "knowledge" of the other methods, you could end up with a financial disaster. A program in which several methods have access to your gross pay figure has methods that are tightly coupled. A superior program would control access to the payroll value by passing it only to methods that need it.

You can evaluate whether coupling between methods is loose or tight by examining how methods share data.

- Tight coupling occurs when methods have access to the same globally defined variables. When one method changes the value stored in a variable, other methods are affected. You should avoid tight coupling, but be aware that you might see it in programs written by others.
- Loose coupling occurs when a copy of data that must be shared is passed from one method to another. That way, the sharing of data is always purposeful—variables must be explicitly passed to and from methods that use them. The loosest (best) methods pass single arguments if possible, rather than many variables or entire records.

Additionally, there is a time and a place for shortcuts. If a memo must go out in five minutes, you don't have time to change fonts or add clip art with your word processor. Similarly, if you need a quick programming result, you might very well use cryptic variable names, tight coupling, and minimal cohesion. When you create a professional application, however, you should keep professional guidelines in mind.

TWO TRUTHS & A LIE

Method Design Issues: Implementation Hiding, Cohesion, and Coupling

1. A calling method must know the interface to any method it calls.
2. You should try to avoid loose coupling, which occurs when methods do not depend on others.
3. Functional cohesion occurs when all operations in a method contribute to the performance of only one task.

The false statement is #2. You should aim for loose coupling so that methods are independent.

Understanding Recursion

Recursion occurs when a method is defined in terms of itself. A method that calls itself is a **recursive method**. Some programming languages do not allow a method to call itself, but those that do can be used to create recursive methods that produce interesting effects. Figure 9-20 shows a simple example of recursion. The program calls an `infinity()` method, which displays *Help!* and calls itself again (see the shaded statement). The second call to `infinity()` displays *Help!* and generates a third call. The result is a large number of repetitions of the `infinity()` method. The output is shown in Figure 9-21.

```
start
   infinity()
stop

infinity()
   output "Help! "
   infinity()
return
```

Figure 9-20 A program that calls a recursive method

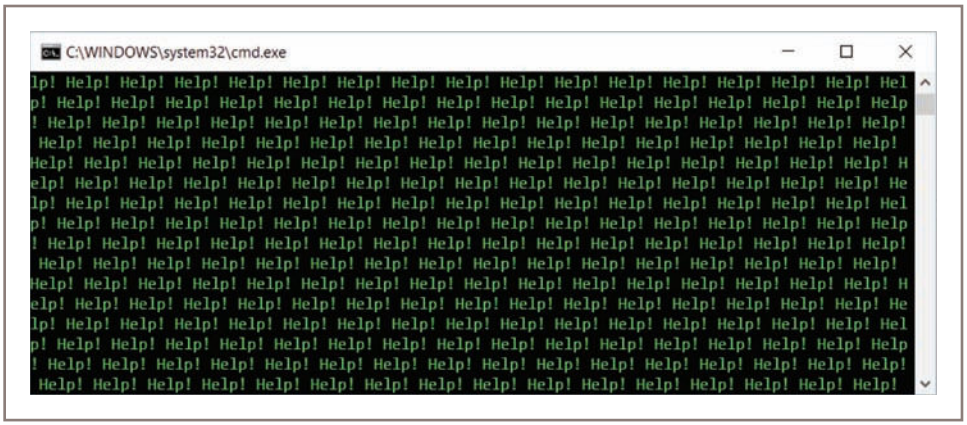


Figure 9-21 Output of the program in Figure 9-20

Every time you call a method, the address to which the program should return at the completion of the method is stored in a memory location called the **stack**. When a method ends, the address is retrieved from the stack and the program returns to the location where the method call was made, then proceeds to the next instruction. For example, suppose that a program calls `methodA()` and that `methodA()` calls `methodB()`. When the program calls `methodA()`, a return address is stored in the stack, and then `methodA()` begins execution. When `methodA()` calls `methodB()`, a return address in `methodA()` is stored in the stack and `methodB()` begins execution. When `methodB()` ends, the last entered address is retrieved from the stack and program control returns to complete `methodA()`. When `methodA()` ends, the remaining address is retrieved from the stack and program control returns to the main program method to continue execution.

Like all computer memory, the stack has a finite size. When the program in Figure 9-20 calls the `infinity()` method, the stack receives so many return addresses that it eventually overflows. The recursive calls will end after an excessive number of repetitions and the program issues an error message.

Of course, there is no practical use for an infinitely recursive program. Just as you must be careful not to create endless loops, when you write useful recursive methods you must provide a way for the recursion to stop eventually. The input values that cause a method to recur are called the **recursive cases**, and the input value that makes the recursion stop is called the **base case** or **terminating case**.



Using recursion successfully is easier if you thoroughly understand looping. You learned about loops in Chapter 5. An everyday example of recursion is printed on shampoo bottles: *Lather, rinse, repeat.*

Figure 9-22 shows an application that uses recursion productively. The program calls a recursive method that computes the sum of every integer from 1 up to and including the method's argument value. For example, the sum of every integer up to and including 3 is 1+2+3, or 6, and the sum of every integer up to and including 4 is 1+2+3+4, or 10.


```
start
   Declarations
      num LIMIT = 10
      num number
   number = 1
   while number <= LIMIT
      output "When number is ", number,
         " then cumulativeSum(number) is ",
         cumulativeSum(number)
      number = number + 1
   endwhile
return

num cumulativeSum(num number)
   Declarations
      num returnVal
   if number = 1 then
      returnVal = number
   else
      returnVal = number + cumulativeSum(number - 1)
   endif
return returnVal
```

Figure 9-22 Program that uses a recursive `cumulativeSum()` method

When thinking about cumulative summing relationships, remember that the sum of all the integers up to and including any number is that number plus the sum of the integers for the next lower number. In other words, consider the following:

- The sum of the digits from 1, up to and including 1, is simply 1.
- The sum of the digits from 1 through 2 is the previous sum, plus 2.
- The sum of the digits from 1 through 3 is the previous sum, plus 3.
- The sum of the digits from 1 through 4 is the previous sum, plus 4.
- And so on.

The recursive `cumulativeSum()` method in Figure 9-22 uses this knowledge. For each `number`, its cumulative sum consists of the value of the number itself plus the cumulative sum of all the previous lesser numbers. The program in Figure 9-22 calls the

`cumulativeSum()` method 10 times in a loop to show the cumulative sum of every integer from 1 through 10. Figure 9-23 shows the output.

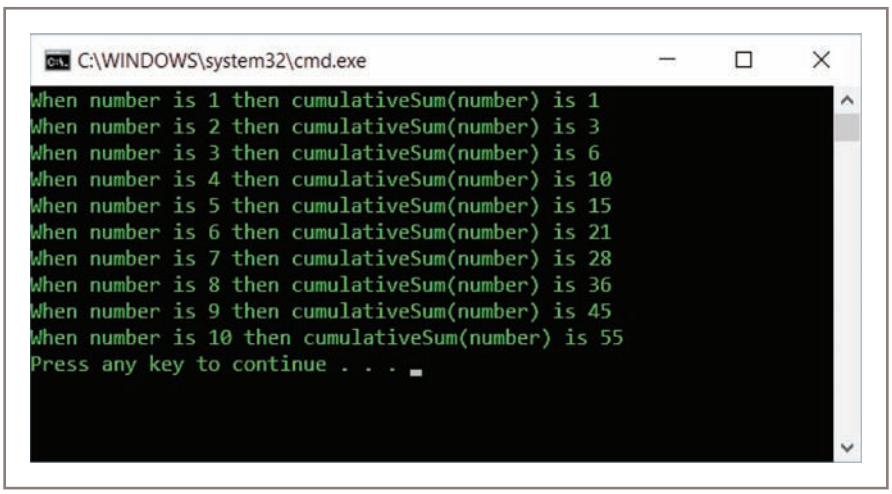


Figure 9-23 Output of the program in Figure 9-22

If you examine Figures 9-22 and 9-23 together, you can see the following:

- When 1 is passed to the `cumulativeSum()` method, the `if` statement within the method determines that the argument is equal to 1, `returnVal` becomes 1, and 1 is returned for output. (The input value 1 is the base case or terminating case.)
- On the next pass through the loop, 2 is passed to the `cumulativeSum()` method. When the method receives 2 as an argument, the `if` statement within the method is false, and `returnVal` is set to 2 plus the value of `cumulativeSum(1)`. (The input value 2 is a recursive case.) This second call to `cumulativeSum()` using 1 as an argument returns a 1, so when the method ends, it returns 2+1, or 3.
- On the third pass through the loop within the calling program, 3 is passed to the `cumulativeSum()` method. When the method receives 3 as an argument, the `if` statement within the method is false and the method returns 3 plus the value of `cumulativeSum(2)`. (The input value 3, like 2, is a recursive case.) The value of this call is 2 plus `cumulativeSum(1)`. The value of `cumulativeSum(1)` is 1. Ultimately, `cumulativeSum(3)` is 3+2+1.

Many sophisticated programs that operate on lists of items use recursive processing. However, following the logic of a recursive method can be difficult, and programs that use recursion are sometimes error-prone and hard to debug. Because such programs also can be hard for others to maintain, some business organizations forbid their programmers from using recursive logic in company programs. Many of the problems solved by recursive methods can be solved using loops. For example, examine the program in Figure 9-24. This program produces the same result as the previous recursive program, but in a more straightforward fashion.

```
start
   Declarations
      num number
      num total
      num LIMIT = 10
   total = 0
   number = 1
   while number <= LIMIT
      total = total + number
      output "When number is ", number,
         " then the cumulative sum of 1 through",
         number, " is ", total
      number = number + 1
   endwhile
stop
```

Figure 9-24 Nonrecursive program that computes cumulative sums



A humorous illustration of recursion is found in this sentence: "In order to understand recursion, you must first understand recursion." A humorous dictionary entry is "Recursion: See Recursion." These examples contain an element of truth, but useful recursive algorithms always have a point at which the infinite loop is exited. In other words, when recursion is implemented correctly, the base case or terminating case is always reached at some point.



Watch the video *Recursion*.

TWO TRUTHS & A LIE

Understanding Recursion

1. A method that calls itself is a recursive method.
2. Every time you call a method, the address to which the program should return at the completion of the method is stored in a memory location called the stack.
3. Following the logic of a recursive method is usually much easier than following the logic of an ordinary program, so recursion makes debugging easier.

The false statement is #3. Following the logic of a recursive method is difficult, and programs that use recursion are sometimes error-prone and hard to debug.

Chapter Summary

- A method is a program module. Any program can contain an unlimited number of methods, and each method can be called an unlimited number of times. A method must include a header, a body, and a `return` statement that marks the end of the method.
- Variables and constants are in scope within, or local to, only the method within which they are declared.
- When you pass a data item into a method, it is an argument to the method. When the method receives the data item, it is called a parameter. When you write the declaration for a method that can receive parameters, you must include the data type and a local name for each parameter within the method declaration's parentheses. You can pass multiple arguments to a called method by listing the arguments within the method call and separating them with commas. When you call a method, the arguments you send to the method must match in order—both in number and in type—the parameters listed in the method declaration.
- A method's return type indicates the data type of the value that the method will send back to the location where the method call was made. The return type also is known as a method's type, and is placed in front of the method name when the method is defined. When a method returns a value, you usually want to use the returned value in the calling method, although this is not required.
- You can pass a single array element to a method in exactly the same manner you would pass a variable or constant. You can indicate that a method parameter is an array by placing square brackets after the data type in the method's parameter list. When you pass an array to a method, it is passed by reference; that is, the method receives the actual memory address of the array and has access to the actual values in the array elements.
- When you overload a method, you write multiple methods with a shared name but different parameter lists. The compiler understands your meaning based on the arguments you use when calling the method. Overloading a method introduces the risk of creating ambiguous methods—a situation in which the compiler cannot determine which version of a method to use.
- All modern programming languages contain many built-in, prewritten methods to save you time and effort.
- With well-written methods, the implementation is hidden, cohesion is high, and coupling is loose.
- Recursion occurs when a method is defined in terms of itself. Following the logic of a recursive method can be difficult, and programs that use recursion are sometimes error-prone and hard to debug.

Key Terms

A **method** is a program module that contains a series of statements that carry out a task.

A method's **client** is a program or other method that uses the method.

A **method header** precedes a method body; it includes the method identifier and possibly other necessary identifying information, such as a return type and parameter list.

A **method body** contains all the statements in the method.

Implementation describes the body of a method or the statements that carry out the tasks of a method.

A **method `return` statement** marks the end of a method and identifies the point at which control returns to the calling method.

Local describes data items that are only known to the method in which they are declared.

Global describes data items that are known to all the methods in a program.

An **argument to a method** is a value passed to a method in the call to the method.

A **parameter to a method** is a data item defined in a method header that accepts data passed into the method from the outside.

A **parameter list** is all the data types and parameter names that appear in a method header.

A method's **signature** includes its name and parameter list.

A variable passed into a method is **passed by value**; that is, a copy of its value is sent to the method and stored in a new memory location accessible to the method.

A method's **return type** indicates the data type of the value that the method will send back to the location where the method call was made.

A **void method** returns no value.

A **method's type** is the type of its return value.

Overhead refers to all the resources and time required by an operation.

An **IPO chart** identifies and categorizes each item needed within the method as pertaining to input, processing, or output.

Passed by reference describes how values are accepted into a method when the method receives the actual memory address item. Arrays are passed by reference.

Overloading involves supplying diverse meanings for a single identifier.

Polymorphism is the ability of a method to act appropriately according to the context.

To **overload a method** is to create multiple versions with the same name but different parameter lists.

Ambiguous methods are those methods that the compiler cannot distinguish between because they have the same name and parameter types.

Implementation hiding is a programming principle that describes the encapsulation of method details.

The **interface to a method** includes the method's return type, name, and arguments; it is the part that a client sees and uses.

A **black box** is the analogy programmers use to refer to hidden method implementation details.

Cohesion is a measure of how the internal statements of a method serve to accomplish the method's purpose.

Functional cohesion occurs when all operations in a method contribute to the performance of only one task; it is the highest and most desirable level of cohesion.

Coupling is a measure of the strength of the connection between two program methods.

Tight coupling occurs when methods excessively depend on each other; it makes programs more prone to errors.

Loose coupling occurs when methods do not depend on others.

Recursion occurs when a method is defined in terms of itself.

A **recursive method** is a method that calls itself.

The **stack** is a memory area that holds addresses to which methods should return.

Recursive cases describe the input values that cause a recursive method to execute again.

The **base case** or **terminating case** of a recursive method describes the value that ends the repetition.

Exercises



Review Questions

1. Which of the following is true?
   a. A program can call one method at most.
   b. A method can contain one or more other methods.
   c. A program can contain a method that calls another method.
   d. All of the above are true.

2. Which of the following must every method have?

   a. a parameter list
   b. a header
   c. a return value
   d. all of the above

3. Which of the following is most closely related to the concept of *local*?

   a. in scope
   b. object-oriented
   c. program level
   d. abstract

4. Although the terms *parameter* and *argument* are closely related, the difference is that *argument* refers to _____.

   a. a value in a method call
   b. a passed constant
   c. a formal parameter
   d. a variable that is local to a method

5. A method's interface is its _____.

   a. parameter list
   b. return type
   c. identifier
   d. all of the above

6. When you write the declaration for a method that can receive a parameter, which of the following must be included in the method declaration?

   a. the name of the argument that will be used to call the method
   b. a local name for the parameter
   c. the data type of the parameter
   d. two of the above

7. When you use a variable name in a method call, it _____ as the variable in the method header.

   a. can have the same name
   b. cannot have the same name
   c. must have the same name
   d. cannot have the same data type

8. Assume that you have written a method with the header `void myMethod(num a, string b)`. Which of the following is a correct method call?

   a. `myMethod(12)`
   b. `myMethod("Goodbye")`
   c. `myMethod(12, "Hello")`
   d. It is impossible to tell.

9. Assume that you have written a method with the header `num yourMethod(string name, num code)`. The method's type is_____.

   a. `num`
   b. `string`
   c. `num` and `string`
   d. `void`

10. Assume that you have written a method with the header `string myMethod(num score, string grade)`. Also assume that you have declared a numeric variable named `test`. Which of the following is a correct method call?

    a. `myMethod()`
    b. `myMethod(test)`
    c. `myMethod(test, test)`
    d. `myMethod(test, "A")`

11. The value used in a method's `return` statement must _____.

    a. be numeric
    b. be a variable
    c. match the data type used before the method name in the header
    d. two of the above

12. When a method receives a copy of the value stored in an argument used in the method call, it means the variable was _____.

    a. unnamed
    b. passed by value
    c. passed by reference
    d. assigned its original value when it was declared

13. A void method _____.

    a. contains no statements
    b. requires no parameters
    c. returns nothing
    d. has no name

14. When an array is passed to a method, it is _____.

    a. passed by reference
    b. passed by value
    c. unnamed in the method
    d. unalterable in the method

15. When you overload a method, you write multiple methods with the same _____.

    a. name
    b. parameter list
    c. number of parameters
    d. return type

16. A program contains a method with the header `num calculateTaxes(num amount, string name)`. Which of the following methods can coexist in the same program with no possible ambiguity?

    a. `num calculateTaxes(string name, num amount)`
    b. `string calculateTaxes(num money, string taxpayer)`
    c. `num calculateTaxes(num annualPay, string taxpayerId)`
    d. All of these can coexist without ambiguity.

17. Methods in the same program with identical names and identical parameter lists are _____.

    a. overloaded
    b. overworked
    c. overwhelmed
    d. ambiguous

18. Methods in different programs with identical names and identical parameter lists are _____.

    a. overloaded
    b. illegal
    c. both of the above
    d. none of the above

19. The notion of _____ most closely describes the way a calling method is not aware of the statements within a called method.

    a. abstraction
    b. object-oriented
    c. implementation hiding
    d. encapsulation

20. Programmers should strive to _____.

    a. increase coupling
    b. increase cohesion
    c. both of the above
    d. neither a nor b



Programming Exercises

1. Create an IPO chart for each of the following methods:

   a. The method that calculates the amount owed on a restaurant check, including tip

   b. The method that calculates the cost to drive your car a mile

   c. The method that calculates your annual medical expenses after the insurance company has made its payments

2. a. Create the logic for a program that calculates and displays the amount of money you would have if you invested $5000 at 2 percent simple interest for one year. Create a separate method to do the calculation and return the result to be displayed.

   b. Modify the program in Exercise 2a so that the main program prompts the user for the amount of money and passes it to the interest-calculating method.

   c. Modify the program in Exercise 2b so that the main program also prompts the user for the interest rate and passes both the amount of money and the interest rate to the interest-calculating method.

3. Create the logic for a program that accepts a user's birth month and year and passes them to a method that calculates the user's age in the current month and returns the value to the main program to be be displayed.

4. a. Create the logic for a program that performs arithmetic functions. Design the program to contain two numeric variables, and prompt the user for values for the variables. Pass both variables to methods named `sum()` and `difference()`. Create the logic for the methods `sum()` and `difference()`; they compute the sum of and difference between the values of two arguments, respectively. Each method should perform the appropriate computation and display the results.

   b. Modify the program in Exercise 4a so that the two entered values are passed to a method named `getChoice()`. The `getChoice()` method asks the user whether addition or subtraction should be performed and then passes the two values to the appropriate method, where the result is displayed.

5. Create the logic for a program that continuously prompts a user for a numeric value until the user enters 0. The application passes the value in turn to the following methods:

   - A method that displays all whole numbers from 1 up to and including the entered number
   - A method that computes the sum of all the whole numbers from 1 up to and including the entered number
   - A method that computes the product of all the whole numbers from 1 up to and including the entered number

6. Create the logic for a program that calls a method that computes and returns a homeowner's profit from the home's sale. Arguments passed to the method include the sale price and the following which must be deducted from the sale price: mortgage payoff, realtor's commission, title insurance fee, and transfer tax amount.

7. Create the logic for a program that continuously prompts the user for three numeric values that represent the length, width, and depth in inches of a proposed patio. Include two overloaded methods that compute the cost of construction. One method accepts all three parameters and calculates the cost at $0.12 per cubic inch. The other takes two numeric parameters that represent length and width and uses a default depth of 4 inches. Accept input and respond as follows:

   - When the user enters zero for the length value, end the program.
   - If the user enters a negative number for any value, continue to reprompt the user until the value is not negative.

- If all numbers entered are greater than 0, call the method version that accepts three.
- If the depth value is zero, call the version of the method that uses the default depth.

8. a. Plan the logic for an insurance company program to determine policy premiums. The program continuously prompts the user for an insurance policy number. When the user enters an appropriate sentinel value, end the program. Call a method that prompts each user for the type of policy needed—health or auto. While the user's response does not indicate health or auto, continue to prompt the user. When the value is valid, return it from the method. Pass the user's response to a new method where the premium is set and returned—$550 for a health policy or $225 for an auto policy. Display the results for each policy.

   b. Modify Exercise 8a so that the premium-setting method calls one of two additional methods—one that determines the health premium or one that determines the auto premium. The health insurance method asks users whether they smoke; the premium is $550 for smokers and $345 for nonsmokers. The auto insurance method asks users to enter the number of traffic tickets they have received in the last three years. The premium is $225 for drivers with three or more tickets, $190 for those with one or two tickets, and $110 for those with no tickets. Each of these two methods returns the premium amount to the calling method, which returns the amount to be displayed.

9. Create the logic for a program that prompts the user for numeric values for a month, day, and year. Then pass the three variables to the following methods:

   a. A method that displays the date with dashes in month-day-year order, as it is often represented in the United States—for example, *6-24-2015*

   b. A method that displays the date with dashes in day-month-year order, as it is often represented in the United Kingdom—for example, *24-6-2015*

   c. A method that displays the date with dashes in year-month-day order, as it is represented in the International Standard—for example, *2015-6-24*

   d. A method that prompts the user for the desired format ("US", "UK", or "IS") and then passes the three values to one of the methods described in parts a, b, and c of this exercise

10. Create the logic for a program that computes hotel guest rates at Cornwall's Country Inn. Include two overloaded methods named `computeRate()`. One version accepts a number of days and calculates the rate at $99.99 per day. The other accepts a number of days and a code for a meal plan. If the code is *A*, three meals per day are included, and the price is $169.00 per day. If the code is *C*, breakfast is included, and the price is $112.00 per day. All other codes are invalid.

Each method returns the rate to the calling program where it is displayed. The main program asks the user for the number of days in a stay and whether meals should be included; then, based on the user's response, the program either calls the first method or prompts for a meal plan code and calls the second method.

11. Create the logic for a program that prompts a user for 12 numbers and stores them in an array. Pass the array to a method that reverses the order of the numbers. Display the reversed numbers in the main program.

12. Create the logic for a program that prompts a user for 20 numbers and stores them in an array. Pass the array to a method that calculates the arithmetic average of the numbers and returns the value to the calling program. Display each number and how far it is from the arithmetic average. Continue to prompt the user for additional sets of 20 numbers until the user wants to quit.

13. Each of the programs in Figure 9-25 uses a recursive method. Try to determine the output in each case.

a.

```
start
   output recursiveA(0)
stop
num recursiveA(num x)
   num result
   if x = 0 then
      result = x
   else
      result = x *
         (recursiveA(x - 1))
   endif
return result
```

b.

```
start
   output recursiveB(2)
stop
num recursiveB(num x)
   num result
   if x = 0 then
      result = x
   else
      result = x *
         (recursiveB(x - 1))
   endif
return result
```

c.

```
start
   output recursiveC(2)
stop
num recursiveC(num x)
   num result
   if x = 1 then
      result = x
   else
      result = x *
         (recursiveC(x - 1))
   endif
return result
```

Figure 9-25 Problems for Exercise 13



Performing Maintenance

1. A file named MAINTENANCE09-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 9 include DEBUG09-01.txt, DEBUG09-02.txt, and DEBUG09-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 9 include a file named DEBUG09-04.jpg that contains a flowchart with syntax and/or logical errors. Examine the flowchart, and then find and correct all the bugs.



Game Zone

1. In the Game Zone sections of Chapters 6 and 8, you designed the logic for a quiz that contains questions about a topic of your choice. Now, modify the program so it contains an array of five multiple-choice quiz questions related to the topic of your choice. Each question contains four answer choices. Also, create a parallel array that holds the correct answer to each question—A, B, C, or D. In turn, pass each question to a method that displays the question and accepts the player's answer. If the player does not enter a valid answer choice, force the player to reenter the choice. Return the user's valid (but not necessarily correct) answer to the main program. After the user's answer is returned to the main program, pass it and the correct answer to a method that determines whether the values are equal and displays an appropriate message. After the user answers all five questions, display the number of correct and incorrect answers that the user chose.

2. In the Game Zone section of Chapter 6, you designed the logic for the game Hangman, in which the user guesses letters in a hidden word. Improve the game to store an array of 10 words. One at a time, pass each word to a method that allows the user to guess letters continuously until the game is solved. The method returns the number of guesses it took to complete the word. Store the number in an array before returning to the method for the next word. After all 10 words have been guessed, display a summary of the number of guesses required for each word, as well as the average number of guesses per word.

CHAPTER 10

Object-Oriented Programming

Upon completion of this chapter, you will be able to:

- Describe the principles of object-oriented programming
- Understand classes
- Understand public and private access
- Appreciate ways to organize classes
- Use instance methods
- Use static methods
- Use objects

Principles of Object-Oriented Programming

Object-oriented programming (OOP) is a programming model that focuses on an application's components and the data and methods the components use. With OOP, you consider the items that a program will manipulate—for example, a customer invoice, a loan application, a button that a user clicks, or a menu from which a user selects an option. These items are called *objects*, and when you program, you define their characteristics, functions, and capabilities.

OOP uses all of the familiar concepts of modular, procedural programming, such as variables, methods, and passing arguments. Methods in object-oriented programs continue to use sequence, selection, and looping structures and make use of arrays. However, OOP adds several new concepts to programming and requires that you learn new vocabulary to describe those concepts.

Five important features of object-oriented languages are:

- Classes
- Objects
- Polymorphism
- Inheritance
- Encapsulation

Classes and Objects

In object-oriented terminology, a **class** describes a group or collection of objects with common attributes. An **object** is one **instance** (or one **instantiation**) of a class. When a program creates an object, it **instantiates** the object.

For example, a `Car` class might describe all the general features of an automobile. My `redChevroletAutomobileWithTheDent` is an instance of the class, as is your `brandNewBlackPorsche`. As another example, Figure 10-1 depicts a `Dog` class and two instances of it.

Objects both in the real world and in object-oriented programming can contain attributes and methods. **Attributes** are the characteristics of an object. For example, some of a `Car`'s attributes are its make, model, year, and purchase price. These attributes don't change during each object's life. Examples of attributes that might change frequently include whether the automobile is currently running, its gear, its speed, and whether it is dirty. All `Car` objects possess the same attributes, but not the same values for those attributes. Similarly, your `Dog` has attributes that include its breed, name, age, and whether its shots are current.

Methods are the actions that can be taken on an object; often they alter, use, or retrieve the attributes. For example, a `Car` has methods for changing and viewing its speed, and a `Dog` has methods for changing and viewing its shot status.

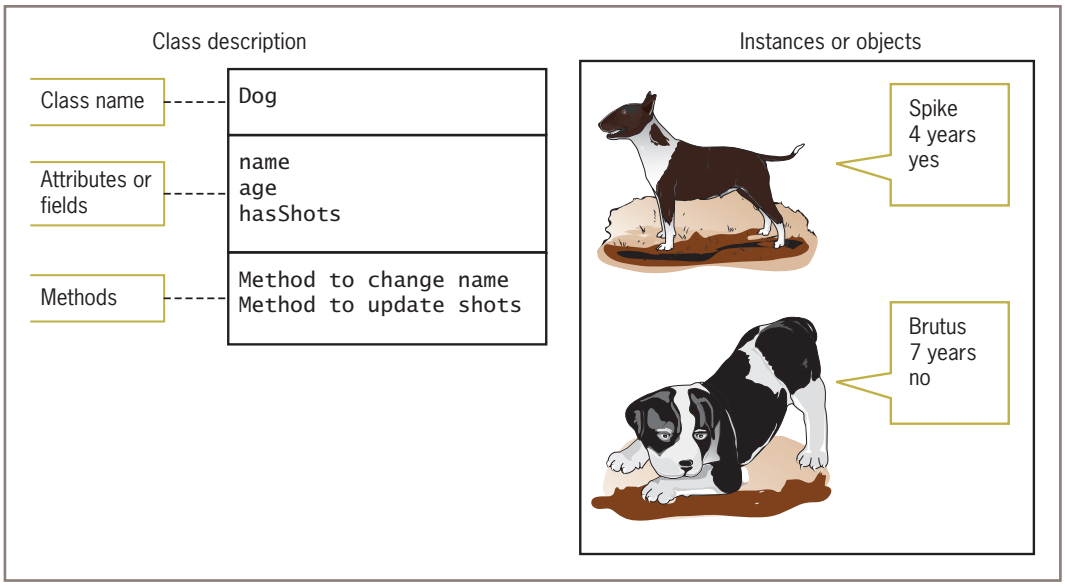


Figure 10-1 A `Dog` class and two instances

Thinking of items as instances of a class allows you to apply your general knowledge of the class to the individual objects created from it. You know what attributes an object has when you know what class defines it. For example, if your friend purchases a `Car`, you know it has a model name, and if your friend gets a `Dog`, you know the dog has a breed. You might not know the current status of your friend's `Car`, such as its current speed, or the status of her `Dog`'s shots, but you do know what attributes exist for objects of the `Car` and `Dog` classes, which allows you to imagine these objects reasonably well before you see them. You know enough to ask the `Car`'s model and not its breed; you know enough to ask the `Dog`'s name and not its engine size. As another example, when you use a new application on your computer, you expect each component to have specific, consistent attributes, such as a button being clickable or a window being closable. Each component gains these attributes as an instance of the general class of GUI (graphical user interface) components.



Most programmers employ a naming convention in which class names begin with an uppercase letter and multiple-word identifiers are run together, such as `SavingsAccount` or `TemporaryWorker`. Each new word within the identifier starts with an uppercase letter. In Chapter 2, you learned that this convention is known as *Pascal casing*.

Much of your understanding of the world comes from your ability to categorize objects and events into classes. As a young child, you learned the concept of *animal* long before you knew the word. Your first encounter with an animal might have been with the family dog, a neighbor's cat, or a goat at a petting zoo. As you developed speech, you might have used the same term for all of these creatures, gleefully shouting, "Doggie!" as your parents pointed out cows, horses, and sheep in picture books or along the roadside on drives in the country.

As you grew more sophisticated, you learned to distinguish dogs from cows; still later, you learned to distinguish breeds. Your understanding of the class `Animal` helps you see the similarities between dogs and cows, and your understanding of the class `Dog` helps you see both the differences between a `Dog` and other `Animals` as well as the similarities between a `GreatDane` and a `Chihuahua`. Understanding classes gives you a framework for categorizing new experiences. You might not know the term *okapi*, but when you learn it's an `Animal`, you begin to develop a concept of what an okapi might be like.

When you think in an object-oriented manner, everything is an object. You can think of any inanimate physical item as an object—your desk, your computer, and your house are all called *objects* in everyday conversation. You can think of living things as objects, too—your houseplant, your pet goldfish, and your sister are objects. Events also are objects—the stock purchase you made, the mortgage closing you attended, and your graduation party are all objects.

Everything is an object, and every object is an instance of a more general class. Your desk is an instance of the class that includes all desks, and your pet goldfish is an instance of the class that contains all fish. These statements represent **is-a relationships** because you can say, "My oak desk with the scratch on top *is a* `Desk` and my goldfish named Moby *is a* `Fish`." Your goldfish, my guppy, and the zoo's shark each constitute one instance of the `Fish` class.



Object-oriented programmers also use the term *is-a* when describing inheritance. You will learn about inheritance later in this chapter and in Chapter 11.

The concept of a class is useful because of its reusability. For example, if you invite me to a graduation party, I automatically know many things about the party object. I assume that there will be attributes such as a starting time, a number of guests, some quantity of food, and gifts. I understand parties because of my previous knowledge of the `Party` class, of which all parties are tangible examples or instances. I might not know the number of guests or the date or time of this particular party, but I understand that because all parties have a date and time, then this one must as well. Similarly, even though every stock purchase is unique, each must have a dollar amount and a number of shares. All objects have predictable attributes because they are instantiated from specific classes.

The data components of a class that belong to every instantiated object are the class's **instance variables**. Instance variables often are called **fields** to help distinguish them from other variables you might use. The set of all the values or contents of an object's instance variables is known as its **state**. For example, the current state of a particular party might be *8 p.m.* and *Friday*; the state of a particular stock purchase might be *$10* and *five shares*.

In addition to their attributes, classes have methods associated with them, and every object instantiated from a given class possesses the same methods. For example, at some point you might want to issue invitations for a party. You might name the method `issueInvitations()`, and it might display some text as well as the values of the party's date and time fields. Your graduation party, then, might be named `myGraduationParty`.

As an object of the `Party` class, it would have data members for the date and time, like all parties, and it would have a method to issue invitations. When you use the method, you might want to be able to send an argument to `issueInvitations()` that indicates how many copies to create. When you think about an object and its methods, it's as though you can send a message to the object to direct it to accomplish a particular task—you can tell the party object named `myGraduationParty` to create the number of invitations you request. Even though `yourAnniversaryParty` also is an instance of the `Party` class, and even though it also has access to the `issueInvitations()` method, you will send a different argument value to `yourAnniversaryParty`'s `issueInvitations()` method than I send to `myGraduationParty`'s corresponding method. Within an object-oriented program, you continuously make requests to an object's methods, often including arguments as part of those requests.



In grammar, a noun is equivalent to an object and the values of a class's attributes are adjectives—they describe the characteristics of the objects. An object also can have methods, which are equivalent to verbs.

When you program in object-oriented languages, you frequently create classes from which objects will be instantiated. You also write applications to use the objects, along with their data and methods. Often, you will write programs that use classes created by others; at other times, you might create a class that other programmers will use to instantiate objects within their own programs. A program or class that instantiates objects of another prewritten class is a **class client** or **class user**.

For example, your organization might already have a class named `Customer` that contains attributes such as `name`, `address`, and `phoneNumber`, and you might create clients that include arrays of thousands of `Customers`. Similarly, in a GUI operating environment, you might write applications that include prewritten components from classes with names like `Window` and `Button`.

Polymorphism

The real world is full of objects. Consider a door. A door is an object that needs to be opened and closed. You open a door with an easy-to-use interface known as a doorknob. Object-oriented programmers would say you are *passing a message* to the door when you tell it to open by turning its knob. The same message (turning a knob) has a different result when applied to your radio than when applied to a door. As depicted in Figure 10-2, the procedure you use to open something—call it the "open" procedure—works differently on a door than it does on a desk drawer, a bank account, a computer file, or your eyes. However, even though these procedures operate differently using the various objects, you can call each of these procedures "open." In Chapter 9, you learned that this concept is called *polymorphism*.

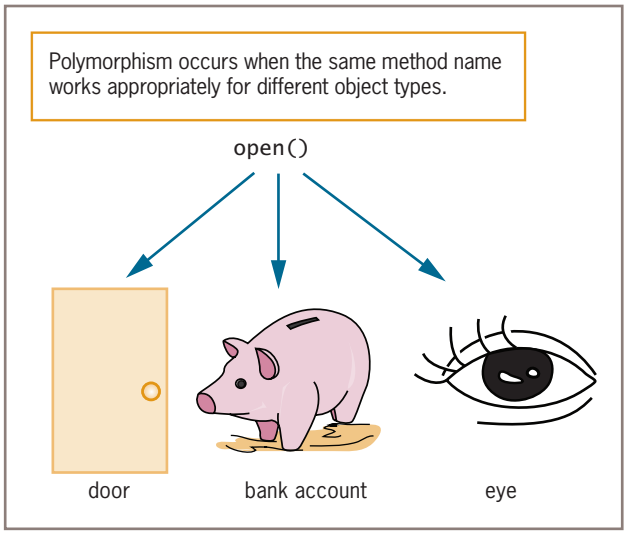


Figure 10-2 Examples of polymorphism

Within classes in object-oriented programs, you can create multiple methods with the same name, which will act differently and appropriately when used with different types of objects. For example, you might use a method named `print()` to print a customer invoice, loan application, or envelope. Because you use the same method name to describe the different actions needed to print these diverse objects, you can write statements in object-oriented programming languages that are more like English; you can use the same method name to describe the same type of action, no matter what type of object is being acted upon. Using the method name `print()` is easier than remembering `printInvoice()`, `printLoanApplication()`, and so on. Object-oriented languages understand verbs in context, just as people do.

As another example of the advantages to using one name for a variety of objects, consider a screen you might design for a user to enter data into an application you are writing. Suppose that the screen contains a variety of objects—some forms, buttons, scroll bars, dialog boxes, and so on. Suppose also that you decide to make all the objects blue. Instead of having to memorize the method names that these objects use to change color—perhaps `changeFormColor()`, `changeButtonColor()`, and so on—your job would be easier if the creators of all those objects had developed a `changeColor()` method that works appropriately with each type of object.



Purists find a subtle difference between *overloading* (which you learned about in Chapter 9) and *polymorphism* Although there are subtle differences, both terms refer to the ability to use a single name to communicate multiple meanings.

Inheritance

Inheritance is the process of acquiring the traits of one's predecessors. In the real world, a new door with a stained glass window inherits most of its traits from a standard door. It has the same purpose, it opens and closes in the same way, and it has the same knob and hinges. As Figure 10-3 shows, the door with the stained glass window simply has one additional trait—its window. Even if you have never seen a door with a stained glass window, you know what it is and how to use it because you understand the characteristics of all doors. Inheritance is an important concept in object-oriented programming because once you create a class, you can develop new classes whose objects possess all the traits of objects of the original class plus any new traits the new class needs.

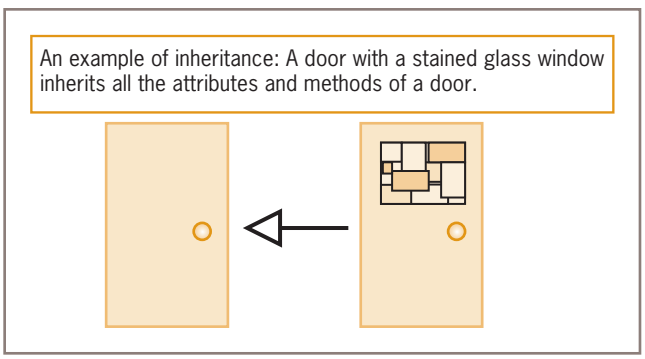


Figure 10-3 An example of inheritance

For example, if you develop a `CustomerBill` class of objects, there is no need to develop an `OverdueCustomerBill` class from scratch. You can create the new class to contain all the characteristics of the already developed one, and simply add necessary new characteristics. This not only reduces the work involved in creating new classes, it makes them easier to understand because they possess most of the characteristics of already-developed classes.

Watch the video *An Introduction to Object-Oriented Programming*.

Encapsulation

Real-world objects often employ encapsulation and information hiding.

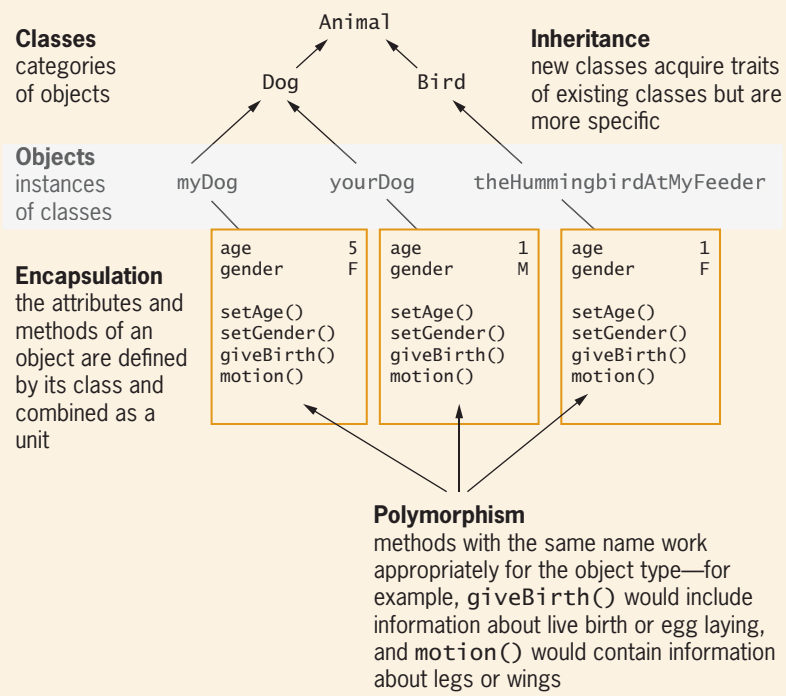
- **Encapsulation** is the process of combining all of an object's attributes and methods into a single package; the package includes data items that are frequently hidden from outside classes as well as methods that often are available to outside classes to access and alter the data.

- **Information hiding** is the concept that other classes should not alter an object's attributes—only the methods of an object's own class should have that privilege. (The concept is also called **data hiding**.) Outside classes should only be allowed to make a request that an attribute be altered; then it is up to the class's methods to determine whether the request is appropriate.

For example, when you use a door, you usually are unconcerned with the latch or hinge construction, and you don't have access to the interior workings of the knob. Those features are hidden, and the casual user cannot change them. You care only about the functionality and the interface to the door—the user-friendly boundary between the user and the internal mechanisms. When you turn a door's knob, you are interacting appropriately with the interface. Banging on the knob would be an inappropriate interaction, as would talking to the knob, so the door would not respond. Similarly, the detailed workings of objects you create within object-oriented programs can be hidden from outside programs and modules if necessary, and the methods you write can control how the objects operate. When the details are hidden, programmers can focus on the functionality and the interface, as people do with real-life objects.

In summary, understanding object-oriented programming means that you must consider five of its integral components: classes, objects, polymorphism, inheritance, and encapsulation. Quick Reference 10-1 illustrates these components.

QUICK REFERENCE 10-1 Components of Object-Oriented Programming



TWO TRUTHS & A LIE

Principles of Object-Oriented Programming

1. Learning about object-oriented programming is difficult because it does not use the concepts you already know, such as declaring variables and using modules.
2. In object-oriented terminology, a class describes a group or collection of objects with common attributes; an instance of a class is an existing object of a class.
3. A program or class that instantiates objects of another prewritten class is a class client or class user.

The false statement is #1. Object-oriented programming uses many features of procedural programming, including declaring variables and using modules.

Defining Classes and Creating Class Diagrams

A class is a category of things; an object is a specific instance of a class. A **class definition** is a set of program statements that lists the characteristics of each object and the methods each object can use.

A class definition can contain three parts:

- Every class has a name.
- Most classes contain data, although this is not required.
- Most classes contain methods, although this is not required.

For example, you can create a class named `Employee`. Each `Employee` object will represent one employee who works for an organization. Data fields, or attributes of the `Employee` class, include fields such as `lastName`, `hourlyWage`, and `weeklyPay`.

The methods of a class include all the actions you want to perform with the class. Appropriate methods for an `Employee` class might include `setHourlyWage()`, `getHourlyWage()`, and `calculateWeeklyPay()`. The job of `setHourlyWage()` is to provide values for an `Employee`'s wage data field, the purpose of `getHourlyWage()` is to retrieve the wage value so it can, for example, be displayed or used in a calculation, and the purpose of `calculateWeeklyPay()` is to multiply the `Employee`'s `hourlyWage` by the number of hours in a workweek to calculate a weekly salary. With object-oriented languages, you think of the class name, data, and methods as a single encapsulated unit.

Declaring a class does not create actual objects. A class is just an abstract description of what an object will be if any objects are actually instantiated. Just as you might understand all the characteristics of an item you intend to manufacture before the first item rolls off

the assembly line, you can create a class with fields and methods long before you instantiate objects from it.

When you declare a simple variable that is a built-in data type, you write a statement such as one of the following:

```
num money
string name
```

When you write a program that declares an object that is a class data type, you write a statement such as the following:

```
Employee myAssistant
```



In some object-oriented programming languages, you need to add more to the declaration statement to actually create an `Employee` object. For example, in Java you would write:

```
Employee myAssistant = new Employee();
```

You will understand more about the format of this statement when you learn about constructors in Chapter 11.

When you declare the `myAssistant` object, it contains all the data fields within its class and has access to all the methods. In other words, a larger section of memory is set aside than when you declare a simple variable, because an `Employee` contains several fields. You can use any of an `Employee`'s methods with the `myAssistant` object. The usual syntax is to provide an object name, a dot (period), and a method name with parentheses and a possible argument list. For example, you can write a program that contains statements such as those shown in Figure 10-4.

```
start
   Declarations
      Employee myAssistant
   myAssistant.setLastName("Reynolds")
   myAssistant.setHourlyWage(16.75)
   output "My assistant makes ",
      myAssistant.getHourlyWage(), " per hour"
stop
```

Figure 10-4 Application that declares and uses an `Employee` object



The program segment in Figure 10-4 is very short. In a more useful real-life program, you might read employee data from a data file before assigning it to the object's fields, each `Employee` might contain dozens of fields, and your application might create hundreds or thousands of objects.



Besides referring to `Employee` as a class, many programmers would refer to it as a **user-defined type**, but a more accurate term is **programmer-defined type**. A class from which objects are instantiated is the data type of its objects. Object-oriented programmers typically refer to a class like `Employee` as an **abstract data type** (ADT); this term implies that the type's data is private and can be accessed only through methods. You will learn about private data later in this chapter.

When you write a statement such as `myAssistant.setHourlyWage(16.75)`, you are making a call to a method that is contained within the `Employee` class. Because `myAssistant` is an `Employee` object, it is allowed to use the `setHourlyWage()` method that is part of its class. You can tell from the method call that `setHourlyWage()` must have been written to accept a numeric parameter.

When you write the application in Figure 10-4, you do not need to know what statements are written within the `Employee` class methods, although you could make an educated guess based on the method names. Before you could execute the application in Figure 10-4, someone would have to write appropriate statements within the `Employee` class methods. If you wrote the methods, of course you would know their contents, but if another programmer has already written the methods, you could use the application without knowing the details contained in the methods. To use the methods, you need only to know their names, parameter lists, and return types.

In Chapter 9, you learned that the ability to use methods as a black box without knowing their contents is a feature of encapsulation. The real world is full of many black-box devices. For example, you can use your television and microwave oven without knowing how they work internally—all you need to understand is the interface. Similarly, with well-written methods that belong to classes you use, you need not understand how they work internally to be able to use them; you need only understand the ultimate result when you use them.

In the client program segment in Figure 10-4, the focus is on the object—the `Employee` named `myAssistant`—and the methods you can use with that object. This is the essence of object-oriented programming.



In older object-oriented programming languages, simple numbers and characters are said to be **primitive data types**; this distinguishes them from objects that are class types. In the newest programming languages, every item you name, even one that is a numeric or string type, is an object created from a class that defines both data and methods.



When you instantiate objects, their data fields are stored at separate memory locations. However, all objects of the same class share one copy of the class's methods. You will learn more about this concept later in this chapter.

Creating Class Diagrams

A **class diagram** consists of a rectangle divided into three sections, as shown in Figure 10-5. The top section contains the name of the class, the middle section contains the names and data types of the attributes, and the bottom section contains the methods. This generic class diagram shows two attributes and three methods, but a given class might have any number of attributes or methods, including none. Programmers often use a class diagram to plan or illustrate class features. Class diagrams also are useful for describing a class to nonprogrammers.

Figure 10-6 shows the class diagram for the `Employee` class. By convention, a class diagram lists the names of the data items first; each name is followed by a colon and the data type.

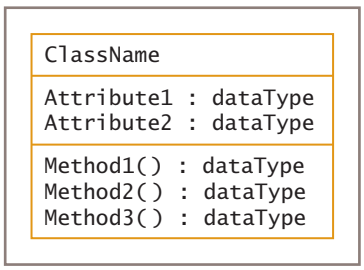


Figure 10-5 Generic class diagram

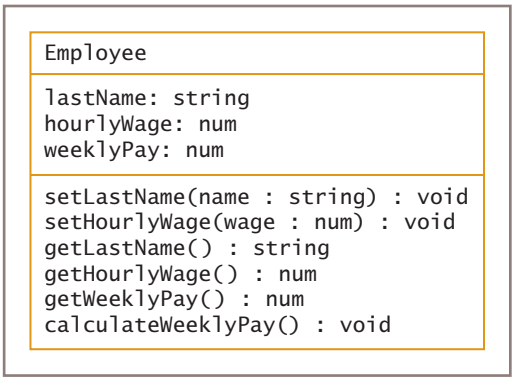


Figure 10-6 `Employee` class diagram



Some developers prefer to insert the word `void` within the parentheses of methods listed in class diagrams when the methods do not have parameter lists. For example, `void` could be inserted between the parentheses in `getLastName()` in the class diagram in Figure 10-6. Inserting `void` instead of writing nothing shows that a parameter list was not inadvertently omitted. You should follow the conventions of your organization.

Method names are listed next, and each is followed by its data type (return type). Listing the names first and the data types last emphasizes the purposes of the fields and methods.

Figures 10-5 and 10-6 both show that a class diagram is intended to be only an overview of class attributes and methods. A class diagram shows *what* data items and methods the class will use, not the details of the methods nor *when* they will be used. It is a design tool that helps you see the big picture in terms of class requirements. Figure 10-6 shows the `Employee` class containing three data fields that represent an employee's name, hourly pay rate, and weekly pay amount. Every `Employee` object created in a program that uses this class will contain these three data fields. In other words, when you declare an `Employee` object, the single declaration statement allocates enough memory to hold all three fields.

Figure 10-6 also shows that the `Employee` class contains six methods. For example, the first method is defined as follows:

```
setLastName(name : string) : void
```

This notation means that the method name is `setLastName()`, that it takes a single `string` parameter named `name`, and that it returns nothing.



Various books, websites, and organizations use class diagrams that describe methods in different ways. For example, some developers use the method name only, and others omit parameter lists. This book takes the approach of being as complete as possible, so the class diagrams you see here will contain each method's identifier, parameter list with types, and return type.

The Employee class diagram shows that two of the six methods—setLastName() and setHourlyWage()—take parameters. The diagram also shows the return type for each method—three void methods, two numeric methods, and one string method. The class diagram does not indicate what takes place inside the method, although you might be able to make an educated guess. Later, when you write the code that actually creates the Employee class, you include method implementation details. For example, Figure 10-7 shows some pseudocode you can use to list the details for the methods in the Employee class.

```
class Employee
   Declarations
      string lastName
      num hourlyWage
      num weeklyPay

   void setLastName(string name)
      lastName = name
   return

   void setHourlyWage(num wage)
      hourlyWage = wage
      calculateWeeklyPay()
   return

   string getLastName()
   return lastName

   num getHourlyWage()
   return hourlyWage

   num getWeeklyPay()
   return weeklyPay

   void calculateWeeklyPay()
      Declarations
         num WORK_WEEK_HOURS = 40
      weeklyPay = hourlyWage * WORK_WEEK_HOURS
   return
endClass
```

Figure 10-7 Pseudocode for Employee class described in the class diagram in Figure 10-6

In Figure 10-7, the Employee class attributes are identified with a data type and a field name. In addition to listing the required data fields, the figure shows the complete methods for the Employee class. The purposes of the methods can be divided into three categories:

- Two of the methods accept values from a client and assign them to data fields; these methods, by convention, have the prefix *set*.

- Three of the methods send data to a client; these methods, by convention, have the prefix *get*.
- One method performs work within the class; this method is named `calculateWeeklyPay()`. This method does not communicate with any client; its purpose is to multiply `hourlyWage` by the number of hours in a week.

The Set Methods

In Figure 10-7, two methods begin with the word *set*; they are `setLastName()` and `setHourlyWage()`. The purpose of a **set method**, or **mutator method**, is to set or change the values of data fields defined within the class. There is no requirement that such method names start with *set*; the prefix is merely conventional and clarifies the intention of the methods. In the `setLastName()` method, a string `name` is passed in as a parameter and assigned to the field `lastName`. Because `lastName` is contained in the same class as this method, the method has access to the field and can alter it.

Similarly, the method `setHourlyWage()` accepts a numeric parameter named `wage` and assigns it to the class field `hourlyWage`. This method also calls the `calculateWeeklyPay()` method, which sets `weeklyPay` based on `hourlyWage`. By writing the `setHourlyWage()` method to call the `calculateWeeklyPay()` method automatically, you guarantee that the `weeklyPay` field is updated any time `hourlyWage` changes.

When you create an `Employee` object with a statement such as `Employee mySecretary`, you can use statements such as the following:

```
mySecretary.setLastName("Johnson")
mySecretary.setHourlyWage(20.00)
```

Instead of literal constants, you could pass variables or named constants to the methods as long as they were the correct data type. For example, if you write a program in which you make the following declaration, then the assignment in the next statement is valid.

```
Declarations
   num PAY_RATE_TO_START = 15.00
mySecretary.setHourlyWage(PAY_RATE_TO_START)
```



In some languages—for example, Visual Basic and C#—you can create a **property** instead of creating a set method. Using a property provides a way to set a field value using a simpler syntax. By convention, if a class field is `hourlyWage`, its property would be `HourlyWage`, and in a program you could make a statement similar to `mySecretary.HourlyWage = PAY_RATE_TO_START`. The implementation of the property `HourlyWage` (with an uppercase initial letter) would be written in a format very similar to that of the `setHourlyWage()` method.

Like other methods, the methods that manipulate fields within a class can contain any statements you need. For example, a more complicated version of the `setHourlyWage()`

method that validates input might be written as shown in Figure 10-8. In this version, the wage passed to the method is tested against minimum and maximum values, and is assigned to the class field `hourlyWage` only if it falls within the prescribed limits. If the wage is too low, the `MINWAGE` value is substituted, and if the wage is too high, the `MAXWAGE` value is substituted.


```
void setHourlyWage(num wage)
   Declarations
      num MINWAGE = 14.50
      num MAXWAGE = 70.00
   if wage < MINWAGE then
      hourlyWage = MINWAGE
   else
      if wage > MAXWAGE then
         hourlyWage = MAXWAGE
      else
         hourlyWage = wage
      endif
   endif
   calculateWeeklyPay()
return
```

Figure 10-8 A version of the `setHourlyWage()` method including validation

Similarly, if the set methods in a class required them, the methods could contain output statements, loops, array declarations, or any other legal programming statements. However, if the main purpose of a method is *not* to set a field value, then for clarity the method should not be named with the *set* prefix.

The Get Methods

The purpose of a **get method**, or **accessor method**, is to return a value from the class to a client. In the `Employee` class in Figure 10-7, the three get methods have the prefix *get*: `getLastName()`, `getHourlyWage()`, and `getWeeklyPay()`. Like with set methods, the prefix *get* is not required for a get method, but it is conventional and clarifies the method's purpose. Each of the get methods in the `Employee` class in Figure 10-7 contains only a `return` statement that simply returns the value in the field associated with the method name. Like set methods, any of these get methods could also contain more complex statements as needed. For example, in a more complicated class, you might return the hourly wage of an employee only if the user had also passed an appropriate security access code to the method, or you might return the weekly pay value as a string with a dollar sign and appropriate commas inserted instead of as a numeric value. When you declare

an Employee object such as Employee mySecretary, you can then make statements in a program similar to the following:

```
Declarations
   string employeeName
employeeName = mySecretary.getLastName()
output "Wage is ", mySecretary.getHourlyWage()
output "Pay for half a week is ",
mySecretary.getWeeklyPay() * 0.5
```

In other words, the value returned from a get method can be used as any other variable of its type would be used. You can assign the value to another variable, display it, perform arithmetic with it if it is numeric, or make any other statement that works correctly with the returned data type.

Work Methods

The Employee class in Figure 10-7 contains one method that is neither a get nor a set method. This method, calculateWeeklyPay(), is a **work method** within the class. A work method is also known as a **help method** or **facilitator**. The calculateWeeklyPay() method contains a locally named constant that represents the hours in a standard workweek, and it computes the weeklyPay field value by multiplying hourlyWage by the named constant.

From the implementation of the calculateWeeklyPay() method shown in Figure 10-7, you can see that no values need to be passed into this method, and no value is returned from it. The calculateWeeklyPay() method does not communicate with any client of the Employee class. Instead, this method is called only from another method in the same class (the setHourlyWage() method), and that method is called from a client. Each time a program uses the setHourlyWage() method to alter an Employee's hourlyWage field, calculateWeeklyPay() is called to recalculate the weeklyPay field. No setWeeklyPay() method is included in this Employee class to directly assign a value to the weeklyPay field because the intention is that weeklyPay is set only inside the calculateWeeklyPay() method each time the setHourlyWage() method calls it. If you wanted programs to be able to set the weeklyPay field directly, you would have to write a method to allow it.



Programmers who are new to class creation often want to pass the hourlyWage value into the calculateWeeklyPay() method so that it can use the value in its calculation. Although this technique would work, it is not required. The calculateWeeklyPay() method has direct access to the hourlyWage field by virtue of being a member of the same class.

For example, Figure 10-9 shows a program that declares an Employee object and sets the hourly wage value. The program displays the weeklyPay value. Then a new value is assigned to hourlyWage, and weeklyPay is displayed again. As you can see from the output in Figure 10-10, the weeklyPay value has been recalculated even though it was never set directly by the client program.

```
start
   Declarations
      num LOW = 9.00
      num HIGH = 14.65
      Employee myGardener
   myGardener.setLastName("Greene")
   myGardener.setHourlyWage(LOW)
   output "My gardener makes ",
     myGardener.getWeeklyPay(), " per week"
   myGardener.setHourlyWage(HIGH)
   output "My gardener makes ",
     myGardener.getWeeklyPay(), " per week"
stop
```

Figure 10-9 Program that sets and displays `Employee` data two times

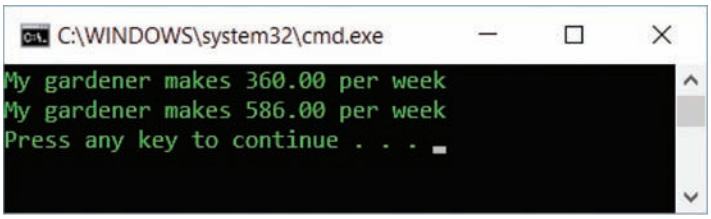


Figure 10-10 Execution of program in Figure 10-9

TWO TRUTHS & A LIE

Defining Classes and Creating Class Diagrams

1. Every class has a name, data, and methods.
2. After an object has been instantiated, its methods can be accessed using the object's identifier, a dot, and a method call.
3. A class diagram consists of a rectangle divided into three sections; the top section contains the name of the class, the middle section contains the names and data types of the attributes, and the bottom section contains the methods.

The false statement is #1. Most classes contain data and methods, although neither is required.

Understanding Public and Private Access

When you buy a new product, one of the usual conditions of its warranty is that the manufacturer must perform all repair work. For example, if your computer has a warranty and something goes wrong with its operation, you cannot open the system unit yourself, remove and replace parts, and then expect to get your money back for a device that does not work properly. Instead, when something goes wrong, you must take the computer to an approved technician. The manufacturer guarantees that your machine will work properly only if the manufacturer can control how the computer's internal mechanisms are modified.

Similarly, in object-oriented design, you do not want outside programs or methods to alter your class's data fields unless you have control over the process. For example, you might design a class that performs complicated statistical analysis on some data, and you would not want others to be able to alter your carefully crafted result. Or, you might design a graphic and not want anyone to alter the dimensions of your artistic design. To prevent outsiders from changing your data fields in ways you do not endorse, you force other programs and methods to use a method that is part of your class to alter data. (Earlier in this chapter, you learned that the principle of keeping data private and inaccessible to outside classes is called *information hiding* or *data hiding*.)

To prevent unauthorized field modifications, object-oriented programmers usually specify that their data fields will have **private access**—the data cannot be accessed by any method that is not part of the class. The methods themselves, like `setHourlyWage()` in the `Employee` class, support public access. When methods have **public access**, other programs and methods can use the methods, often to get access to the private data.

Figure 10-11 shows a complete `Employee` class to which access specifiers have been added to describe each attribute and method. An **access specifier** is the adjective that defines the type of access (`public` or `private`) outside classes will have to the attribute or method. In the figure, each access specifier is shaded.



In many object-oriented programming languages, if you do not declare an access specifier for a data field or method, then it is private by default. For clarity, this book will follow the convention of explicitly specifying access for every class member.

In Figure 10-11, each of the data fields is private, which means each field is inaccessible to an object declared in a client program. In other words, if a program declares an `Employee` object, such as `Employee myAssistant`, then the following statement is illegal:

```
myAssistant.hourlyWage = 15.00
```

Don't Do It
You cannot directly assign a value to a private data field from outside its class.

Instead, `hourlyWage` can be assigned only through a public method as follows:

```
myAssistant.setHourlyWage(15.00)
```

```
class Employee
   Declarations
      private string lastName
      private num hourlyWage
      private num weeklyPay

   public void setLastName(string name)
      lastName = name
   return

   public void setHourlyWage(num wage)
      hourlyWage = wage
      calculateWeeklyPay()
   return

   public string getLastName()
   return lastName

   public num getHourlyWage()
   return hourlyWage

   public num getWeeklyPay()
   return weeklyPay

   private void calculateWeeklyPay()
      Declarations
         num WORK_WEEK_HOURS = 40
      weeklyPay = hourlyWage * WORK_WEEK_HOURS
   return
endClass
```

Figure 10-11 `Employee` class including `public` and `private` access specifiers

If you made `hourlyWage` public instead of private, then a direct assignment statement would work, but you would violate the important OOP principle of data hiding using encapsulation. Data fields should usually be private, and a client application should be able to access them only through the public interfaces—in other words, through the class's public methods. That way, if you have restrictions on the value of `hourlyWage`, those restrictions will be enforced by the public method that acts as an interface to the private data field. Similarly, a public get method might control how a private value is retrieved. Perhaps you do not want clients to have access to an `Employee`'s `hourlyWage` if it is more than a specific value, or maybe you want to return the wage to the client as a string with a dollar sign attached. Even when a field has no data value requirements or restrictions, making data private and providing public set and get methods establishes a framework that makes such modifications easier in the future.

In the `Employee` class in Figure 10-11, all of the methods are public except one—the `calculateWeeklyPay()` method is private. That means if you write a program and declare

an `Employee` object such as `Employee myAssistant`, then the following statement is not permitted:

```
myAssistant.calculateWeeklyPay()
```

Don't Do It
The `calculateWeeklyPay()` method is not accessible outside the class.

Because it is private, the only way to call the `calculateWeeklyPay()` method is from another method that already belongs to the class. In this example, it is called from the `setHourlyWage()` method. This prevents a client program from setting `hourlyWage` to one value while setting `weeklyPay` to an incompatible value. By making the `calculateWeeklyPay()` method private, you ensure that the class retains full control over when and how it is used.

Classes usually contain private data and public methods, but as you have just seen, they can contain private methods. Classes can contain public data items as well. For example, an `Employee` class might contain a public constant data field named `MINIMUM_WAGE`; outside programs then would be able to access that value without using a method. Public data fields are not required to be named constants, but they frequently are.



In some object-oriented programming languages, such as C++, you can label a set of data fields or methods as public or private using the access specifier name just once, then follow it with a list of the items in that category. In other languages, such as Java, you use the specifier *public* or *private* with each field or method. For clarity, this book will label each field and method as public or private.

Many programmers like to specify in class diagrams whether each component in a class is public or private. Figure 10-12 shows the conventions that are typically used. A minus sign (−) precedes the items that are private (less accessible); a plus sign (+) precedes those that are public (more accessible).

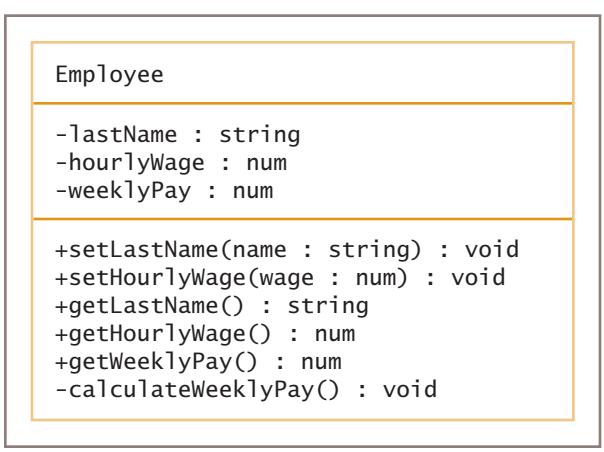


Figure 10-12 `Employee` class diagram with public and private access specifiers

When you learn more about inheritance in Chapter 11, you will learn about an additional access specifier—the protected access specifier. In a class diagram, you use an octothorpe, also called a pound sign or number sign (#), to indicate protected access.

In object-oriented programming languages, the main program is most often written as a method named `main()` or `Main()`, and that method is virtually always defined as public.

Watch the video *Creating a Class*.

TWO TRUTHS & A LIE

Understanding Public and Private Access

1. Object-oriented programmers usually specify that their data fields will have private access.
2. Object-oriented programmers usually specify that their methods will have private access.
3. In a class diagram, a minus sign (−) precedes the items that are private; a plus sign (+) precedes those that are public.

The false statement is #2. Object-oriented programmers usually specify that their methods will have public access.

Organizing Classes

The `Employee` class in Figure 10-12 contains just three data fields and six methods; most classes you create for professional applications will have many more. For example, in addition to a last name and pay information, real employees require an employee number, a first name, address, phone number, hire date, and so on, as well as methods to set and get those fields. As classes grow in complexity, deciding how to organize them becomes increasingly important.

Although it is not required, most programmers place data fields in some logical order at the beginning of a class. For example, an ID number is most likely used as a unique identifier for each employee, so it makes sense to list the employee ID number first in the class. An employee's last name and first name "go together," so it makes sense to store the two components adjacently. Despite these common-sense rules, in most languages you have considerable flexibility when positioning your data fields within a class. For example, depending on the class, you might choose to store the data fields alphabetically, or you

might group together all the fields that are the same data type. Alternatively, you might choose to store all public data items first, followed by private ones, or vice versa.

In some languages, you can organize data fields and methods in any order within a class. For example, you could place all the methods first, followed by all the data fields, or you could organize the class so that data fields are followed by methods that use them. This book will follow the convention of placing all data fields first so that you can see their names and data types before reading the methods that use them. This format also echoes the way data and methods appear in standard class diagrams.

For ease in locating a class's methods, some programmers store them in alphabetical order. Other programmers arrange them in pairs of get and set methods, in the same order as the data fields are defined. Another option is to list all accessor (get) methods together and all mutator (set) methods together. Depending on the class, you might decide to create other logically functional groupings. Of course, if your company distributes guidelines for organizing class components, you must follow those rules.

TWO TRUTHS & A LIE

Organizing Classes

1. As classes grow in complexity, deciding how to organize them becomes increasingly important.
2. You have a considerable amount of flexibility in how you organize data fields within a class.
3. In a class, methods must be stored in the order in which they are used.

The false statement is #3. Methods can be stored in alphabetical order, in pairs of get and set methods, in the same order as the data fields are defined, or in any other logically functional groupings.

Understanding Instance Methods

Classes contain data and methods, and every instance of a class possesses the same data and has access to the same methods. For example, Figure 10-13 shows a class diagram for a simple `Student` class that contains just one private data field for a student's grade point average. The class also contains get and set methods for the field. Figure 10-14 shows the pseudocode for the `Student` class. This class becomes the model for a new data type named `Student`; when `Student` objects are created eventually, each will have its own `gradePointAverage` field and have access to methods to get and set it.

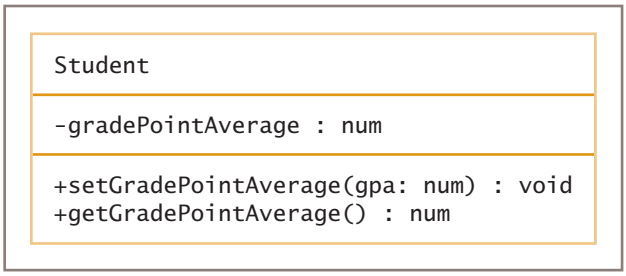


Figure 10-13 Class diagram for `Student` class

```
class Student
   Declarations
      private num gradePointAverage

   public void setGradePointAverage(num gpa)
      gradePointAverage = gpa
   return

   public num getGradePointAverage()
   return gradePointAverage
endClass
```

Figure 10-14 Pseudocode for the `Student` class

If you create multiple `Student` objects using the class in Figure 10-14, you need a separate storage location in computer memory to store each `Student`'s unique grade point average. For example, Figure 10-15 shows a client program that creates three `Student` objects and assigns values to their `gradePointAverage` fields. It also shows how the `Student` objects look in memory after the values have been assigned.

```
start
   Declarations
      Student oneSophomore
      Student oneJunior
      Student oneSenior
   oneSophomore.setGradePointAverage(2.6)
   oneJunior.setGradePointAverage(3.8)
   oneSenior.setGradePointAverage(3.4)
stop
```

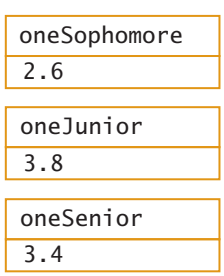


Figure 10-15 Program that creates three `Student` objects and picture of how they look in memory

It makes sense for each Student object in Figure 10-15 to have its own gradePointAverage field, but it does not make sense for each Student to have its own copy of the methods that get and set gradePointAverage. Creating identical copies of a method for each instance would be inefficient. Instead, even though every Student has its own gradePointAverage field, only one copy of each of the methods getGradePointAverage() and setGradePointAverage() is stored in memory. Each instantiated object of the class can use the single method copy. A method that works appropriately with different objects is an **instance method**.

Because only one copy of each instance method is stored no matter how many Student objects are created, the program needs a way to determine which gradePointAverage is being set or retrieved when one of the methods is called. The mechanism that handles this problem is illustrated in Figure 10-16. When a method call such as oneSophomore.setGradePointAverage(2.6) is made, the true method call, which is invisible and automatically constructed, includes the memory address of the oneSophomore object. (These method calls are represented by the three narrow boxes in the center of Figure 10-16.)

Within the setGradePointAverage() method in the Student class, an invisible and automatically created parameter is added to the list. (For illustration purposes, this parameter is named aStudentAddress and is shaded in the Student class definition in Figure 10-16. In fact, no parameter is created with that name.) This parameter accepts the address of a Student object because the instance method belongs to the Student class; if this method belonged to another class—Employee, for example—then the method would accept an address for that type of object. The shaded addresses are not written as code in any program—they are "secretly" sent and received behind the scenes. The address variable in Figure 10-16 is called a this reference. A **this reference** is an automatically created variable that holds the address of an object that is passed to an instance method whenever the method is called. It is called a this reference because it refers to "this particular object" that is using the method at the moment. In other words, an instance method receives a this reference to a specific class instance.

In the application in Figure 10-16, when oneSophomore uses the setGradePointAverage() method, the address of the oneSophomore object is contained in the this reference. Later in the program, when the oneJunior object uses the setGradePointAverage() method, the this reference will hold the address of that Student object.

Figure 10-16 shows each place the this reference is used in the Student class. It is implicitly passed as a parameter to each instance method. You never explicitly refer to the this reference when you write the method header for an instance method; Figure 10-16 just shows where it implicitly exists. Within each instance method, the this reference is implied any time you refer to one of the class data fields. For example, when you call setGradePointAverage() using a oneSophomore object, the gradePointAverage assigned within the method is the *this* gradePointAverage, or the one that belongs to the oneSophomore object. The phrase "this gradePointAverage"

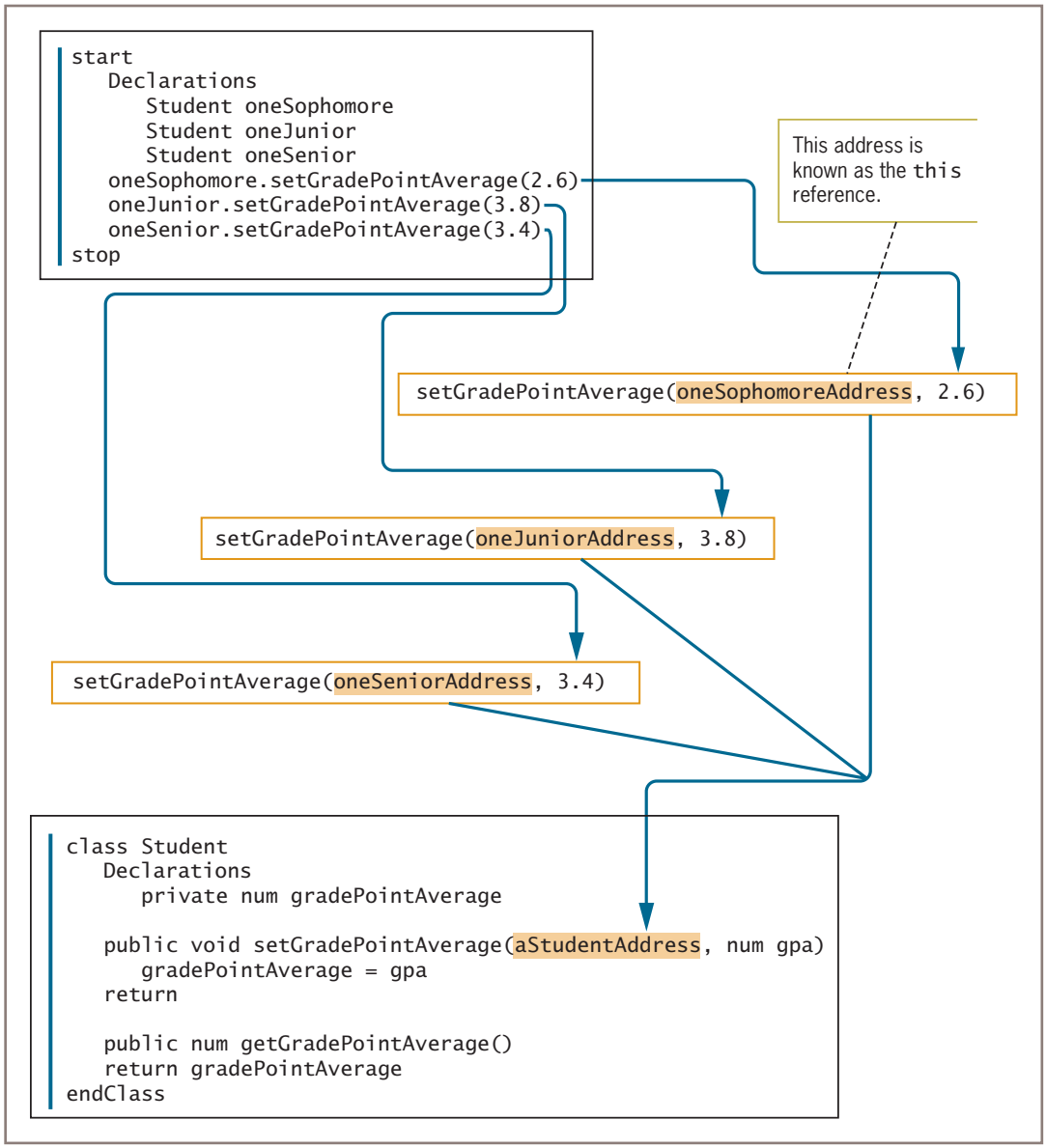


Figure 10-16 How `Student` object memory addresses are passed from an application to an instance method of the `Student` class

usually is written as `this`, followed by a dot, followed by the field name—`this.gradePointAverage`.

The `this` reference exists throughout every instance method. You can explicitly use the `this` reference with data fields, but it is not required. Figure 10-17 shows two locations

where the `this` reference can be used implicitly, or where you can (but do not have to) use it explicitly. Within an instance method, the following two identifiers mean exactly the same thing:

- any field name defined in the class
- `this`, followed by a dot, followed by the same field name.

For example, within the `setGradePointAverage()` method of the `Student` class, `gradePointAverage` and `this.gradePointAverage` refer to exactly the same memory location.

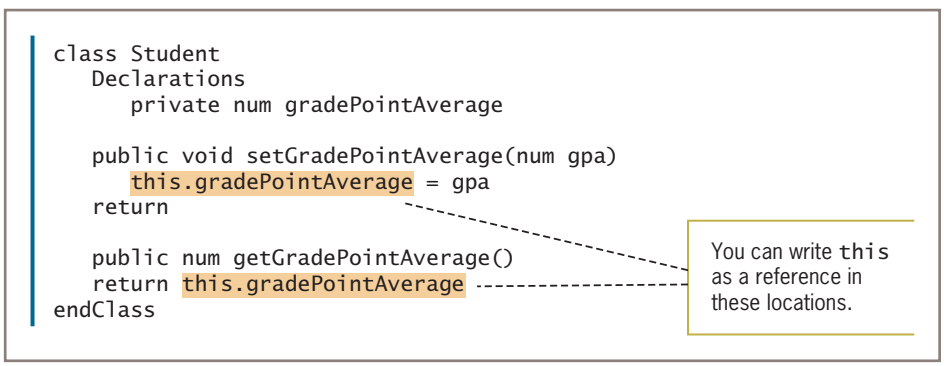


Figure 10-17 Explicitly using `this` in the `Student` class

The `this` reference can be used only with identifiers that are part of the class definition—that is, field names or instance methods. You cannot use it with local variables that are parameters to instance methods or declared within the method bodies. For example, in Figure 10-17 you can refer to `this.gradePointAverage`, but you cannot refer to `this.gpa` because `gpa` is not a class field—it is only a local variable.



The syntax for using `this` differs among programming languages. For example, within a class in C++, you can refer to the `Student` class `gradePointAverage` value as `this->gradePointAverage` or `(*this).gradePointAverage`, but in Java you refer to it as `this.gradePointAverage`. In Visual Basic, the `this` reference is named `Me`, so the variable would be `Me.gradePointAverage`.

Usually you do not need to use the `this` reference explicitly within the methods you write, but the `this` reference is always there, working behind the scenes, accessing the data field for the correct object.



Your organization might prefer that you explicitly use the `this` reference for clarity even though it is not required to create a workable program. It is the programmer's responsibility to follow the conventions established at work or by clients.

As an example of when you might use the `this` reference explicitly, consider the following `setGradePointAverage()` method and compare it to the version in the `Student` class in Figure 10-17.

```
public void setGradePointAverage(num gradePointAverage)
   this.gradePointAverage = gradePointAverage
return
```

In this version of the method, the programmer has used the identifier `gradePointAverage` both as the parameter to the method and as the instance field within the class. Therefore, `gradePointAverage` is the name of a local variable within the method whose value is received by passing; it also is the name of a class field. To differentiate the two, you explicitly use the `this` reference with the copy of `gradePointAverage` that is a member of the class. Omitting the `this` reference in this case would result in the local parameter `gradePointAverage` being assigned to itself, and the class's instance variable would not be set. Any time a local variable in a method has the same identifier as a field, the field is hidden; you must use a `this` reference to distinguish the field from the local variable.



Watch the video *The this Reference*.

TWO TRUTHS & A LIE

Understanding Instance Methods

1. An instance method operates correctly yet differently for each separate instance of a class.
2. A `this` reference is a variable you must explicitly declare with each class you create.
3. When you write an instance method in a class, the following two identifiers within the method always mean exactly the same thing: any field name or `this` followed by a dot, followed by the same field name.

The false statement is #2. A `this` reference is an automatically created variable that holds the address of an object and passes it to an instance method whenever the method is called. You do not declare it explicitly.

Understanding Static Methods

Some methods do not require a `this` reference because it makes no sense for them either implicitly or explicitly. For example, the `displayStudentMotto()` method in Figure 10-18 could be added to the `Student` class. Its purpose is to display a motto that all `Student` objects use in the same way. The method does not use any data fields from the `Student` class, so it does not matter which `Student` object calls it. If you write a program in which you declare 100 `Student` objects, the `displayStudentMotto()` method executes in exactly the same way for each of them; it does not need to know whose motto is displayed and it does not need to access any specific object addresses. As a matter of fact, you might want to display the `Student` motto without instantiating *any* `Student` objects. Therefore, the `displayStudentMotto()` method can be written as a static method instead of an instance method.

```
public static void displayStudentMotto()
   output "Every student is an individual"
   output "in the pursuit of knowledge."
   output "Every student strives to be"
   output "a literate, responsible citizen."
return
```

Figure 10-18 `Student` class `displayStudentMotto()` method

When you write a class, you can indicate two types of methods:

- **Static methods**, also called **class methods**, are those for which no object needs to exist, like the `displayStudentMotto()` method in Figure 10-18. Static methods do not receive a `this` reference as an implicit parameter. Typically, static methods include the word `static` in the method header, as shown shaded in Figure 10-18. (Java, C#, and C++ use the keyword `static`. In Visual Basic, the keyword `Shared` is used in place of `static`.)
- **Nonstatic methods** are methods that exist to be used with an object. These instance methods receive a `this` reference to a specific object. In most programming languages, you use the word `static` when you want to declare a static class member, but you do not use a special word when you want a class member to be nonstatic. In other words, methods in a class are nonstatic instance methods by default.



In everyday language, the word *static* means "stationary"; it is the opposite of *dynamic*, which means "changing." In other words, static methods are always the same for every instance of a class, whereas nonstatic methods act differently depending on the object used to call them.

In most programming languages, you use a static method with the class name (but not an object name), as in the following:

```
Student.displayStudentMotto()
```

In other words, no object is necessary with a static method.



In some languages, notably C++, besides using a static method with the class name, you also can use a static method with any object of the class, as in `oneSophomore.displayStudentMotto()`.

TWO TRUTHS & A LIE

Understanding Static Methods

1. Class methods do not receive a `this` reference.
2. Static methods do not receive a `this` reference.
3. Nonstatic methods do not receive a `this` reference.

The false statement is #3. Nonstatic methods receive a `this` reference automatically.

Using Objects

A class is a complex data type defined by a programmer, but in many ways you can use its instances as you use items of simpler data types. For example, you can pass an object to a method, return an object from a method, or use arrays of objects.

Consider the `InventoryItem` class in Figure 10-19. The class represents items that a company manufactures and holds in inventory. Each item has a number, description, and price. The class contains a get and set method for each of the three fields.

```
class InventoryItem
   Declarations
      private string inventoryNumber
      private string description
      private num price

   public void setInventoryNumber(string number)
      inventoryNumber = number
   return

   public void setDescription(string description)
      this.description = description
   return

   public void setPrice(num price)
      if(price < 0)
         this.price = 0
      else
         this.price = price
      endif
   return

   public string getInventoryNumber()
   return inventoryNumber

   public string getDescription()
   return description

   public num getPrice()
   return price

endClass
```

Notice the uses of the `this` reference to differentiate between the method parameter and the class field.

Figure 10-19 `InventoryItem` class

Passing an Object to a Method

You can pass an object to a method in the same way you can pass a simple numeric or string variable. For example, Figure 10-20 shows a program that declares an `InventoryItem` object and passes it to a method for display. The `InventoryItem` is declared in the main program and assigned values. Then the completed item is passed to a method, where it is displayed. Figure 10-21 shows the execution of the program.

The `InventoryItem` declared in the main program in Figure 10-20 is passed to the `displayItem()` method in much the same way a numeric or string variable would be. The method receives a copy of the `InventoryItem` that is known locally by the identifier `item`. Within the method, the field values of the local item can be retrieved, displayed, and used in arithmetic statements in the same way they could have been in the main program where the `InventoryItem` was originally declared.

```
start
   Declarations
      InventoryItem oneItem
   oneItem.setInventoryNumber("1276")
   oneItem.setDescription("Mahogany chest")
   oneItem.setPrice(450.00)
   displayItem(oneItem)
stop

public static void displayItem(InventoryItem item)
   Declarations
      num TAX_RATE = 0.06
      num tax
      num pr
      num total
   output "Item #", item.getInventoryNumber()
   output item.getDescription()
   pr = item.getPrice()
   tax = pr * TAX_RATE
   total = pr + tax
   output "Price is $", pr, " plus $", tax, " tax"
   output "Total is $", total
return
```

Figure 10-20 Application that declares and uses an `InventoryItem` object

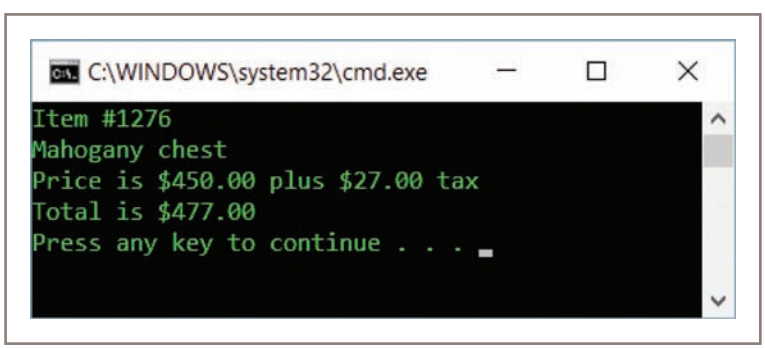


Figure 10-21 Execution of application in Figure 10-20

Returning an Object from a Method

Figure 10-22 shows a more realistic application that uses `InventoryItem` objects. In the main program, an `InventoryItem` is declared and the user is prompted for a number. As long as the user does not enter the `QUIT` value, a loop is executed in which the entered inventory item number is passed to the `getItemValues()` method. Within that method, a local `InventoryItem` object is declared. This local object gathers and holds the user's input values. The user is prompted for a description and price, and then the passed item number

```
start
   Declarations
      InventoryItem oneItem
      string itemNum
      string QUIT = "0"
   output "Enter item number or ", QUIT, " to quit… "
   input itemNum
   while itemNum <> QUIT
      oneItem = getItemValues(itemNum)
      displayItem(oneItem)
      output "Enter next item number or ", QUIT, " to quit… "
      input itemNum
   endwhile
stop

public static InventoryItem getItemValues(string number)
   Declarations
      InventoryItem inItem
      string desc
      num price
   output "Enter description… "
   input desc
   output "Enter price… "
   input price
   inItem.setInventoryNumber(number)
   inItem.setDescription(desc)
   inItem.setPrice(price)
return inItem

public static void displayItem(InventoryItem item)
   Declarations
      num TAX_RATE = 0.06
      num tax
      num pr
      num total
   output "Item #", item.getInventoryNumber()
   output item.getDescription()
   pr = item.getPrice()
   tax = pr * TAX_RATE
   total = pr + tax
   output "Price is $", pr, " plus $", tax, " tax"
   output "Total is $", total
return
```

Figure 10-22 Application that uses `InventoryItem` objects

and newly obtained description and price are assigned to the local `InventoryItem` object via its set methods. The completed object is returned to the program, where it is assigned to the `InventoryItem` object that is then passed to the `displayItem()` method. As in the previous example, the method calculates tax and displays results. Figure 10-23 shows a typical execution.

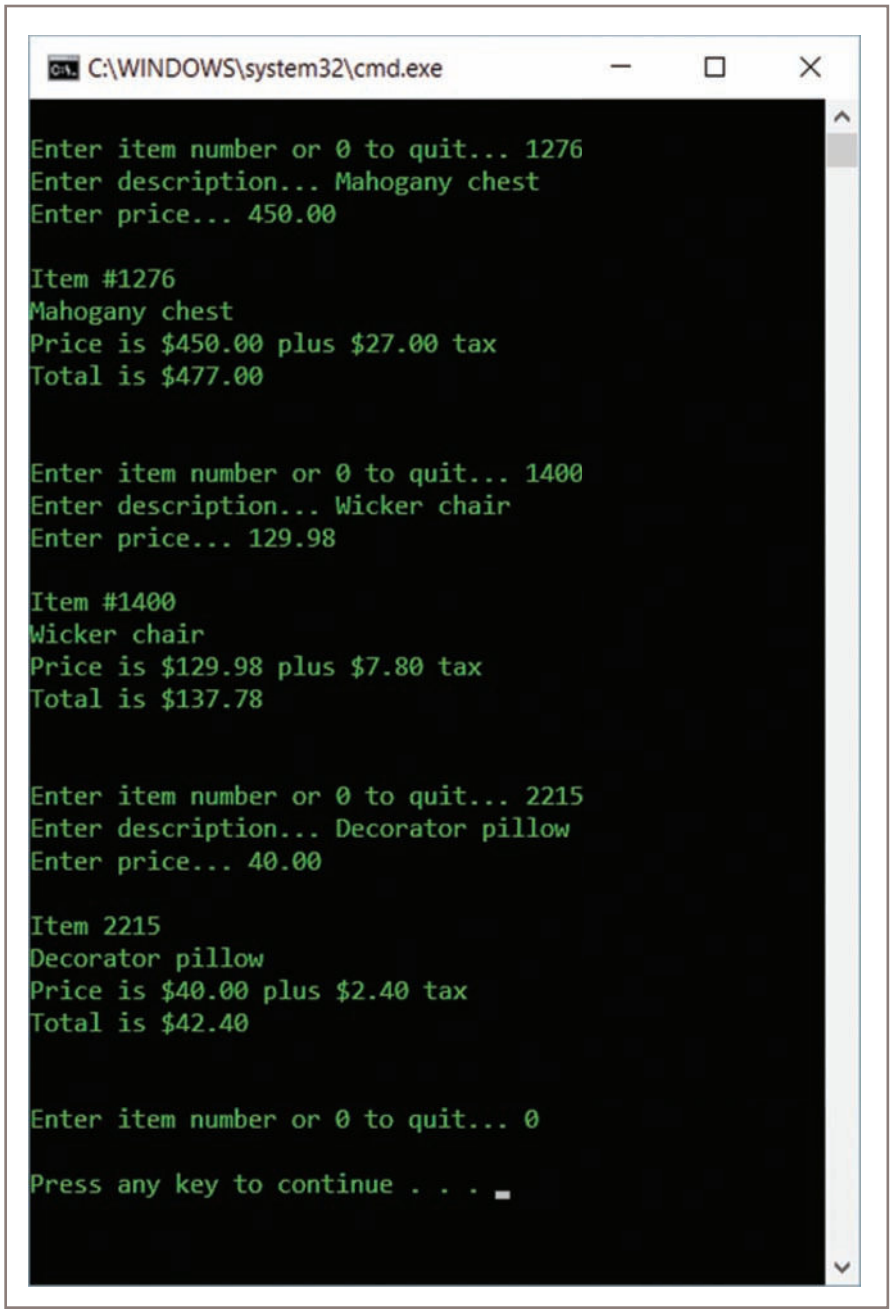


Figure 10-23 Typical execution of program in Figure 10-22

In Figure 10-22, notice that the return type for the `getItemValues()` method is `InventoryItem`. A method can return only a single value. Therefore, it is convenient that the `getItemValues()` method can encapsulate two strings and a number in a single `InventoryItem` object that it returns to the main program.

Using Arrays of Objects

In Chapter 6, you learned that when you declare an array, you use the data type, an identifier, and a size contained in brackets. For example, the following statements declare `num` and `string` arrays, respectively:

```
num scores[10]
string names[5]
```

You can use similar syntax to declare object arrays: a data type (class), an identifier, and a size in brackets. For example, you could declare an array of seven `InventoryItem` objects as follows:

```
InventoryItem items[7]
```

Any individual `items` array element could be used in the same way as any single object of the `InventoryItem` type. For example, the third element in the array could be passed to the `displayItem()` method in Figure 10-22 using the following statement:

```
displayItem(items[2])
```

The entire array can be passed to a method that defines an array of the correct type as a parameter. For example, the statement `displayArray(items)` can be used to call a method with the following header:

```
public static void displayArray(InventoryItem[] list)
```

Within this method, the array would be known as `list`.

Any public member of the `InventoryItem` class can be used with any object in the array by using a subscript to identify the element. For example, the *x*th element in the `items` array can use the public `setInventoryNumber()` method of the `InventoryItem` class by using the following statement:

```
items[x].setInventoryNumber(34);
```

Figure 10-24 shows a complete program that declares seven `InventoryItem` objects, sets their values, and displays them.

In the program in Figure 10-24, a constant is declared for the size of the array, and then the array is declared. The program uses a `while` loop to call a method named `getItemValues()` seven times. The method accepts no parameters. It declares an `InventoryItem` object, and then it prompts the user for an item number, description, and price. Those values are assigned to the `InventoryItem` object using its public methods. The completed object is then returned to the main program, which passes the array and its size to a method named `displayItems()` that displays data for all the items.

```
start
   Declarations
      num SIZE = 7
      InventoryItem items[SIZE]
      num sub
   sub = 0
   while sub < SIZE
      items[sub] = getItemValues()
      sub = sub + 1
   endwhile
   displayItems(items, SIZE)
stop

public static InventoryItem getItemValues()
   Declarations
      InventoryItem item
      num itemNum
      string desc
      num price
   output "Enter item number … "
   input itemNum
   output "Enter description… "
   input desc
   output "Enter price… "
   input price
   item.setInventoryNumber(number)
   item.setDescription(desc)
   item.setPrice(price)
return item

public static void displayItems(InventoryItem[] items, num SIZE)
   Declarations
      num TAX_RATE = 0.06
      num tax
      num pr
      num total
      int x
   x = 0
   while x < SIZE
      output "Item number #", items[x].getInventoryNumber()
      output items[x].getDescription()
      pr = items[x].getPrice()
      tax = pr * TAX_RATE
      total = pr + tax
      output "Price is $", pr, " plus $", tax, " tax"
      output "Total is $", total
      x = x + 1
   endwhile
return
```

Figure 10-24 Application that uses an array of `InventoryItem` objects

The program in Figure 10-24 could have been written using different techniques:

- The program calls the method to get values seven times. Instead, it could have been written to call the method just once, but passing the entire array. Then a loop could have been used within the method.
- The program in Figure 10-24 sends the entire array to the display method. Instead, it could have been written so that the display method accepts only one `InventoryItem`, and then it would have been necessary to call the method seven times in a loop.

As you have learned throughout this book, there are often multiple ways to accomplish the same goal. One method in the program in Figure 10-24 was used to get values, and a different one was used to display values to demonstrate how both approaches work.

TWO TRUTHS & A LIE

Using Objects

1. You can pass an object to a method.
2. Because only one value can be returned from a method, you cannot return an object that holds more than one field.
3. You can declare an object locally within a method.

The false statement is #2. An object can be returned from a method.

Chapter Summary

- Classes are the basic building blocks of object-oriented programming. A class describes a collection of objects; each object is an instance of a class. A class's fields, or instance variables, hold its data, and every object that is an instance of a class has access to the same methods. A program or class that instantiates objects of another prewritten class is a class client or class user. In addition to classes and objects, three important features of object-oriented languages are polymorphism, inheritance, and encapsulation.
- A class definition is a set of program statements that lists the fields and methods each object can use. A class definition can contain a name, data, and methods. Programmers often use a class diagram to illustrate class features. Many methods contained in a class can be divided into three categories: set methods, get methods, and work methods.

- Object-oriented programmers usually specify that their data fields will have private access—that is, the data cannot be accessed by any method that is not part of the class. The methods frequently support public access, which means that other programs and methods can use the methods that control access to the private data. In a class diagram, a minus sign (−) precedes each item that is private; a plus sign (+) precedes each item that is public.
- As classes grow in complexity, deciding how to organize them becomes increasingly important. Depending on the class, you might choose to store the data fields by listing a key field first. You also might list fields alphabetically, by data type, or by accessibility. Methods might be stored in alphabetical order or in pairs of get and set methods.
- An instance method operates correctly yet differently for every object instantiated from a class. When an instance method is called, a `this` reference that holds the object's memory address is automatically and implicitly passed to the method.
- A class can contain two types of methods: static methods, which are also known as class methods and do not receive a `this` reference as an implicit parameter; and nonstatic methods, which are instance methods and do receive a `this` reference implicitly.
- You can use objects in many of the same ways you use items of simpler data types, such as passing them to and from methods and storing them in arrays.

Key Terms

Object-oriented programming (OOP) is a programming model that focuses on an application's components and data and the methods you need to manipulate them.

A **class** describes a group or collection of objects with common attributes.

An **object** is one tangible example of a class; it is an instance of a class.

An **instance** is one tangible example of a class; it is an object.

An **instantiation** of a class is an instance or object.

To **instantiate** an object is to create it.

Attributes are the characteristics that define an object.

An **is-a relationship** exists between an object and its class.

A class's **instance variables** are the data components that belong to every instantiated object.

Fields are object attributes or data.

The **state** of an object is the set of all the values or contents of its instance variables.

A **class client** or **class user** is a program or class that instantiates objects of another prewritten class.

Inheritance is the process of acquiring the traits of one's predecessors.

Encapsulation is the process of combining all of an object's attributes and methods into a single package.

Information hiding (or **data hiding**) is the concept that other classes should not alter an object's attributes—only the methods of an object's own class should have that privilege.

A **class definition** is a set of program statements that define the fields and methods for a class.

A **user-defined type**, or **programmer-defined type**, is a type that is not built into a language but is created by an application's programmer.

An **abstract data type** (ADT) is a programmer-defined type, such as a class.

Primitive data types are simple numbers and characters that are not class types.

A **class diagram** consists of a rectangle divided into three sections that show the name, data, and methods of a class.

A **set method**, or **mutator method**, is an instance method that sets or changes the value of a data field defined in a class.

A **property** provides methods that allow you to get and set a class field value using a simple syntax.

A **get method**, or **accessor method**, is an instance method that returns a value from a field defined in a class.

A **work method**, **help method**, or **facilitator** performs tasks within a class.

Private access specifies that data or methods cannot be used by any method that is not part of the same class.

Public access specifies that other programs and methods can use the specified data or methods within a class.

An **access specifier** is the adjective that defines the type of access outside classes will have to the attribute or method.

An **instance method** operates correctly yet differently for each object. An instance method is nonstatic and implicitly receives a `this` reference.

A **`this` reference** is an automatically created variable that holds the address of an object and passes it to an instance method whenever the method is called.

Static methods are those for which no object needs to exist; they are not instance methods, and they do not receive a `this` reference.

A **class method** is a static method; it is not an instance method, and it does not receive a `this` reference.

Nonstatic methods are methods that exist to be used with an object; they are instance methods and receive a `this` reference.

Exercises



Review Questions

1. Which of the following means the same as *object*?

   a. instance
   b. class
   c. field
   d. category

2. Which of the following means the same as *instance variable*?

   a. class
   b. field
   c. category
   d. record

3. A program that instantiates objects of another prewritten class is a(n)_____ .

   a. instance
   b. object
   c. client
   d. GUI

4. The relationship between an instance and a class is a(n)_____ relationship.

   a. has-a
   b. hostile
   c. polymorphic
   d. is-a

5. Which of these does not belong with the others?

   a. instance variable
   b. attribute
   c. object
   d. field

6. The process of acquiring the traits of one's predecessors is _____ .

   a. polymorphism
   b. encapsulation
   c. inheritance
   d. orientation

7. When discussing classes and objects, *encapsulation* means that _____ .

   a. all the fields belong to the same object
   b. all the fields are private
   c. all the fields and methods are grouped together
   d. all the methods are public

8. Every class definition must contain _____ .

   a. a name
   b. data
   c. methods
   d. all of the above

9. Assume that a working program contains the following statement:

   ```
   myDog.setName("Bowser")
   ```

   Which of the following do you know?

   a. `setName()` is a public method.
   b. `setName()` accepts a string parameter.
   c. Both of the above.
   d. None of the above.

10. Assume that a working program contains the following statement:

    ```
    name = myDog.getName()
    ```

    Which of the following do you know?

    a. `getName()` returns a string.
    b. `getName()` returns a value that is the same data type as `name`.
    c. Both of the above.
    d. None of the above.

11. A class diagram _____ .

    a. provides an overview of a class's data and methods
    b. provides method implementation details
    c. is never used by nonprogrammers because it is too technical
    d. is all of the above

12. Which of the following is the most likely scenario for a specific class?

    a. Its data is private, and its methods are public.
    b. Its data is public, and its methods are private.
    c. Its data and methods are both public.
    d. Its data and methods are both private.

13. An instance method _____ .

    a. is static
    b. receives a `this` reference
    c. Both of the above
    d. None of the above

14. Assume that you have created a class named `Dog` that contains a data field named `weight` and an instance method named `setWeight()`. Further assume that the `setWeight()` method accepts a numeric parameter named `weight`. Which of the following statements correctly sets a `Dog`'s weight within the `setWeight()` method?

    a. `weight = weight`
    b. `this.weight = this.weight`
    c. `weight = this.weight`
    d. `this.weight = weight`

15. A static method is also known as a(n) _____ method.

    a. instance
    b. class
    c. private
    d. public

16. By default, methods contained in a class are _____ methods.

    a. static
    b. nonstatic
    c. class
    d. public

17. Assume that you have created a class named `MyClass`, and that a working program contains the following statement:

```
output MyClass.numberOfStudents
```

    Which of the following do you know?

    a. `numberOfStudents` is a numeric field.
    b. `numberOfStudents` is a static field.
    c. `numberOfStudents` is an instance variable.
    d. All of the above.

18. Assume that you have created an object named `myObject` and that a working program contains the following statement:

```
output myObject.getSize()
```

    Which of the following do you know?

    a. `getSize()` is a static method.
    b. `getSize()` returns a number.
    c. `getSize()` receives a `this` reference.
    d. All of the above.

19. Assume that you have created a class that contains a private field named `myField` and a nonstatic public method named `myMethod()`. Which of the following is true?

    a. `myMethod()` has access to `myField` and can use it.
    b. `myMethod()` does not have access to `myField` and cannot use it.
    c. `myMethod()` can use `myField` but cannot pass it to other methods.
    d. `myMethod()` can use `myField` only if it is passed to `myMethod()` as a parameter.

20. An object can be _____ .

    a. stored in an array
    b. passed to a method
    c. returned from a method
    d. all of the above



Programming Exercises

1. Identify three objects that might belong to each of the following classes:

    a. `Author`
    b. `RaceHorse`
    c. `Country`
    d. `RetailPurchase`

2. Identify three different classes that might contain each of these objects:

    a. `myBlueDenimShirt`
    b. `presidentOfTheUnitedStates`
    c. `myPetCat`
    d. `myCousinLindsey`

3. Design a class named `TermPaper` that holds an author's name, the subject of the paper, and an assigned letter grade. Include methods to set the values for each data field and display the values for each data field. Create the class diagram and write the pseudocode that defines the class.

4. Design a class named `Computer` that holds the make, model, and amount of memory of a computer. Include methods to set the values for each data field, and include a method that displays all the values for each field. Create the class diagram and write the pseudocode that defines the class.

5. Complete the following tasks:

    a. Design a class named `AutomobileLoan` that holds a loan number, make and model of automobile, and balance. Include methods to set values for each data field and a method that displays all the loan information. Create the class diagram and write the pseudocode that defines the class.

    b. Design an application that declares two `AutomobileLoan` objects and sets and displays their values.

    c. Design an application that declares an array of 20 `AutomobileLoan` objects. Prompt the user for data for each object, and then display all the values.

    d. Design an application that declares an array of 20 `AutomobileLoan` objects. Prompt the user for data for each object, and then pass the array to a method that determines the sum of the balances.

6. Complete the following tasks:

   a. Design a class named `StockTransaction` that holds a stock symbol (typically one to four characters), stock name, and price per share. Include methods to set and get the values for each data field. Create the class diagram and write the pseudocode that defines the class.

   b. Design an application that declares two `StockTransaction` objects and sets and displays their values.

   c. Design an application that declares an array of 15 `StockTransaction` objects. Prompt the user for data for each object, and then display all the values.

   d. Design an application that declares an array of 15 `StockTransaction` objects. Prompt the user for data for each object, and then pass the array to a method that determines and displays the two stocks with the highest and lowest price per share.

7. Complete the following tasks:

   a. Design a class named `Cake`. Data fields include two string fields for cake flavor and icing flavor and numeric fields for diameter in inches and price. Include methods to get and set values for each of these fields. Create the class diagram and write the pseudocode that defines the class.

   b. Design an application that declares two `Cake` objects and sets and displays their values.

   c. Design an application that declares an array of 250 `Cake` objects. Prompt the user for data for each `Cake`, then display all the values.

   d. Design an application that declares an array of 25 `Cake` objects. Prompt the user for a topping and diameter for each `Cake`, and pass each object to a method that computes the price and returns the complete `Cake` object to the main program. Then display all the `Cake` values. An 8-inch cake is $19.99, a 9-inch cake is $22.99, and a 10-inch cake is $25.99. Any other entered size is invalid and should cause the price to be set to 0.

8. Complete the following tasks:

   a. Design a class named `BaseballGame` that has fields for two team names and a final score for each team. Include methods to set and get the values for each data field. Create the class diagram and write the pseudocode that defines the class.

   b. Design an application that declares three `BaseballGame` objects and sets and displays their values.

   c. Design an application that declares an array of 12 `BaseballGame` objects. Prompt the user for data for each object, and display all the values. Then pass each object to a method that displays the name of the winning team or "Tie" if the score is a tie.



Performing Maintenance

1. A file named MAINTENANCE10-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable student files for Chapter 10 include DEBUG10-01.txt, DEBUG10-02.txt, and DEBUG10-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.

2. Your downloadable files for Chapter 10 include a file named DEBUG10-04.jpg that contains a class diagram with syntax and/or logical errors. Examine the class diagram, and then find and correct all the bugs.



Game Zone

1. a. Playing cards are used in many computer games, including versions of such classics as Solitaire, Hearts, and Poker. Design a `Card` class that contains a string data field to hold a suit (spades, hearts, diamonds, or clubs) and a numeric data field for a value from 1 to 13. Include get and set methods for each field. Write an application that randomly selects two playing cards and displays their values.

   b. Using two `Card` objects, design an application that plays a simple version of the card game War. Deal two `Cards`—one for the computer and one for the player. Determine the higher card, then display a message indicating whether the cards are equal, the computer won, or the player won. (Playing cards are considered equal when they have the same value, no matter what their suit is.) For this game, assume that the Ace (value 1) is low. Make sure that the two `Cards` dealt are not the same `Card`. For example, a deck cannot contain more than one Queen of Spades.

CHAPTER 11

More Object-Oriented Programming Concepts

Upon completion of this chapter, you will be able to:

- Create constructors
- Create destructors
- Understand composition
- Describe inheritance
- Understand GUI objects
- Describe exception handling
- Appreciate the advantages of object-oriented programming

Understanding Constructors

In Chapter 10, you learned that you can create classes to encapsulate data and methods, and that you can instantiate objects from the classes you define. For example, you can create an `Employee` class that contains fields such as `lastName`, `hourlyWage`, and `weeklyPay`, and methods that set and return values for those fields. When you instantiate an object with a statement that uses the class type and an object identifier, such as `Employee chauffeur`, you are actually calling a method named `Employee()`. A method that has the same name as a class and that establishes an object is a constructor method, or more simply, a **constructor**. Constructors fall into two broad categories:

- A **default constructor** is one that requires no arguments.
- A **non-default constructor** or a **parameterized constructor** requires arguments.

All non-default constructors are written by a programmer, but there are two categories of default constructors:

- An automatically-created default constructor exists in a class in which the programmer has not explicitly written any constructors.
- A programmer-written default constructor can reside in any class and replaces the automatically-created one.

In object-oriented programming (OOP) languages, you can write both default and non-default constructors for a class; if so, the constructors are overloaded.



In some programming languages, such as Visual Basic and C++, you do not need to use the constructor name when declaring an object, but the constructor is called nevertheless. In other languages, such as Java and C#, you include the constructor name in the declaration.

A constructor for a class named `Employee` class instantiates one `Employee` object. If the class contains no programmer-written constructors, then it contains an automatically-supplied default constructor. Depending on the programming language, the automatically-supplied constructor might provide initial values for the object's data fields; for example, in many languages, all numeric fields are set to zero by default. If you do not want an object's fields to hold default values, or if you want to perform additional tasks when you create an instance of a class, you can write your own constructor. Any constructor you write must have the same name as the class in which it is defined, and it does not have a return type. Normally, you declare constructors to be public so that other classes can instantiate objects that belong to the class. You can create constructors for a class with or without parameters. Once you write a constructor, regardless of whether you include parameters, the automatically-supplied default constructor no longer exists.

When you write a constructor that does not include parameters, your constructor becomes the default constructor for the class. In other words, a class can have three types of constructors:

- A class can contain a default (parameterless) constructor that is created automatically. When a class contains an automatically-created default constructor, it means that no constructors have been explicitly written for the class.
- A class can contain a default (parameterless) constructor that you create explicitly. A class with an explicitly-created default constructor no longer contains the automatically-supplied version, but it can coexist with one or more non-default constructors that you write.
- A class can contain a non-default constructor (with one or more parameters), which must be explicitly created. A class with a non-default constructor no longer contains the automatically-supplied default version, but it can coexist with an explicitly-created default constructor and also with other non-default constructors.

Default Constructors

As an example, suppose you create the `Employee` class that appears in Figure 11-1 and that you want every `Employee` object to have a starting hourly wage of $10.00 as well as the correct weekly pay for that wage, then you could write the default constructor that is shaded in the figure. Any `Employee` object instantiated will have an `hourlyWage` field equal to 10.00 and a `weeklyPay` field equal to 400.00. Because the `lastName` field is not assigned in the constructor, each object will hold the default value for strings in the programming language in which this class is implemented.

The `Employee` constructor in Figure 11-1 calls the `calculateWeeklyPay()` method. You can write any statement you want in a constructor, including calling other methods, accepting input, declaring local variables, and so on. You can place a constructor anywhere inside the class, outside of any other method. Often, programmers list constructors first among the methods, because a constructor is the first method called when an object is created.

Figure 11-2 shows a program in which two `Employee` objects are declared and their `hourlyWage` values are displayed. In the output in Figure 11-3, you can see that even though the `setHourlyWage()` method is never called directly in the program, the `Employees` possess valid hourly wages as set by their constructors.

The `Employee` class in Figure 11-1 sets an `Employee`'s hourly wage to 10.00 at construction, but the class also contains a `setHourlyWage()` method that a client application could use later to change the initial `hourlyWage` value. A superior way to write the `Employee` class constructor is shown in Figure 11-4. In this version of the constructor, a named constant

```
class Employee
   Declarations
      private string lastName
      private num hourlyWage
      private num weeklyPay

   public Employee()
      hourlyWage = 10.00
      calculateWeeklyPay()
   return

   public void setLastName(string name)
      lastName = name
   return

   public void setHourlyWage(num wage)
      hourlyWage = wage
      calculateWeeklyPay()
   return

   public string getLastName()
   return lastName

   public num getHourlyWage()
   return hourlyWage

   public num getWeeklyPay()
   return weeklyPay

   private void calculateWeeklyPay()
      Declarations
         num WORK_WEEK_HOURS = 40
      weeklyPay = hourlyWage * WORK_WEEK_HOURS
   return
endClass
```

Figure 11-1 `Employee` class with a default constructor that sets `hourlyWage` and `weeklyPay`

```
start
   Declarations
      Employee myPersonalTrainer
      Employee myInteriorDecorator
   output "Trainer's wage: ",
      myPersonalTrainer.getHourlyWage()
   output "Decorator's wage: ",
      myInteriorDecorator.getHourlyWage()
stop
```

Figure 11-2 Program that declares `Employee` objects using class in Figure 11-1

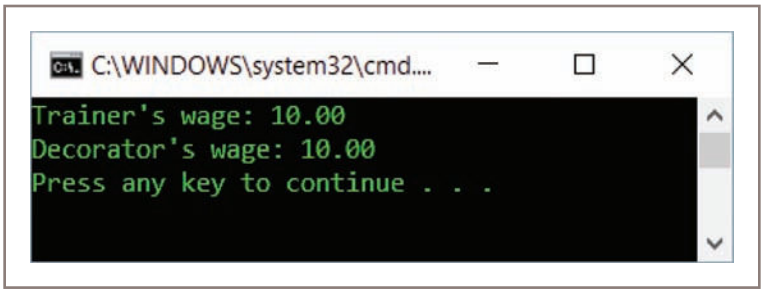


Figure 11-3 Output of program in Figure 11-2

```
public Employee()
   Declarations
      num DEFAULT_WAGE = 10.00
   setHourlyWage(DEFAULT_WAGE)
return
```

Figure 11-4 Improved version of the `Employee` class constructor

with the value 10.00 is passed to `setHourlyWage()`. Using this technique provides several advantages:

- The statement to call `calculateWeeklyPay()` is no longer required in the constructor because the constructor calls `setHourlyWage()`, which, in turn, calls `calculateWeeklyPay()`.
- In the future, if restrictions should be imposed on `hourlyWage`, the code will need to be altered in only one location. For example, if the `setHourlyWage()` method is modified to add decisions that disallow rates that are too high and too low, the code will change only in the `setHourlyWage()` method and will not also have to be modified in the constructor. This reduces the amount of work required and reduces the possibility for error.

Of course, if different `hourlyWage` requirements are needed at initialization from those that are required when the value is set after construction, then different statements will be written in the constructor from those written in the `setHourlyWage()` method.

Non-default Constructors

Non-default constructors accept one or more parameters. You can write one or more non-default constructors for a class. When you create any constructor, whether it is default or non-default, the automatically supplied default constructor is no longer accessible.

You could explicitly create a default (parameterless) constructor in addition to a non-default one, in which case the constructors would be overloaded.

For example, instead of forcing every `Employee` to be constructed with the same initial values, you might choose to create `Employee` objects that each are initialized with a unique `hourlyWage` by passing a numeric value for the wage to the constructor. Figure 11-5 shows an `Employee` constructor that receives an argument. When you declare an object using the `Employee` class that contains this constructor, you pass an argument to the constructor using a declaration similar to one of the following:

- `Employee partTimeWorker(8.81)`, using an unnamed, literal constant
- `Employee partTimeWorker(BASE_PAY)`, using a named constant
- `Employee partTimeWorker(valueEnteredByUser)`, using a variable.

In each of these cases, when the constructor executes, the numeric value within the constructor call is passed to `Employee()`, where the parameter `rate` takes on the value of the argument. The value is assigned to `hourlyWage` within the constructor.

```
public Employee(num rate)
   setHourlyWage(rate)
return
```

Figure 11-5 `Employee` constructor that accepts a parameter

If you create an `Employee` class with a constructor such as the one shown in Figure 11-5, and it is the only constructor in the class, then every `Employee` object you create must use a numeric argument in its declaration. In other words, with this new version of the class that contains a single non-default constructor, the following declaration no longer works:

```
Employee partTimeWorker
```

However, if the class also contains a default constructor, then the constructors are overloaded, and you can use either version when an object is instantiated.

Overloading Instance Methods and Constructors

In Chapter 9, you learned that you can overload methods by writing multiple versions of a method with the same name but different parameter lists. You can overload a class's instance methods and constructors in the same way. For example, Figure 11-6 shows a version of the `Employee` class that contains two constructors.

Recall that a method's signature is its name and list of argument types. The constructors in Figure 11-6 have different signatures—one version requires no argument and the other requires a numeric argument. In other words, this version of the class contains both a default constructor and a non-default constructor.

```
class Employee
   Declarations
      private string lastName
      private num hourlyWage
      private num weeklyPay

   public Employee()
      Declarations
         num DEFAULT_WAGE = 10.00
      setHourlyWage(DEFAULT_WAGE)
   return

   public Employee(num rate)
      setHourlyWage(rate)
   return

   public void setLastName(string name)
      lastName = name
   return

   public void setHourlyWage(num wage)
      hourlyWage = wage
      calculateWeeklyPay()
   return

   public string getLastName()
   return lastName

   public num getHourlyWage()
   return hourlyWage

   public num getWeeklyPay()
   return weeklyPay

   private void calculateWeeklyPay()
      Declarations
         num WORK_WEEK_HOURS = 40
      weeklyPay = hourlyWage * WORK_WEEK_HOURS
   return
endClass
```

Default constructor

Nondefault constructor

Figure 11-6 `Employee` class with overloaded constructors

When you use the version of the class shown in Figure 11-6, then you can make statements such as the following:

- `Employee deliveryPerson`—When you declare an `Employee` using this statement, an `hourlyWage` of 10.00 is automatically set because the statement uses the parameterless version of the constructor.
- `Employee myButler(25.85)`—When you declare an `Employee` using this statement, `hourlyWage` is set to the passed value.

Any method or constructor in a class can be overloaded, and you can provide as many versions as you want, provided that each version has a unique signature. (In the next section, you learn about destructors, which cannot be overloaded.)

As an example, you could add a third constructor to the `Employee` class. Figure 11-7 shows a version that can coexist with the other two constructors because the parameter list is different from either existing version. With this version you can specify the hourly rate for the `Employee` as well as a name. If an application makes a statement similar to the following, then this two-parameter version would execute:

```
Employee myHousekeeper(22.50, "Parker")
```

```
public Employee(num rate, string name)
   lastName = name
   setHourlyWage(rate)
return
```

Figure 11-7 A third possible `Employee` class constructor

You might create an `Employee` class with several constructor versions to provide flexibility for client programs. For example, a particular client program might use only one version, a different client might use another, and a third client might use them all.



Watch the video *Constructors*.

TWO TRUTHS & A LIE

Understanding Constructors

1. A constructor is a method that establishes an object.
2. A default constructor is defined as one that is created automatically.
3. Depending on the programming language, a default constructor might provide initial values for the object's data fields.

The false statement is #2. A default constructor is one that takes no arguments. Although the automatically created constructor for a class is a default constructor, not all default constructors are created automatically.

Understanding Destructors

A **destructor** contains the actions you require when an instance of a class is destroyed. Most often, an instance of a class is destroyed when the object goes out of scope. For example, when an object is declared within a method, the object goes out of scope when the method ends. As with constructors, if you do not explicitly create a destructor for a class, one is provided automatically.

The most common way to declare a destructor explicitly is to use an identifier that consists of a tilde (~) followed by the class name. You cannot provide parameters to a destructor; it must have an empty parameter list. As a consequence, destructors cannot be overloaded; a class can have only one destructor. Like a constructor, a destructor has no return type.



The rules for creating and naming destructors vary among programming languages. For example, in Visual Basic classes, the destructor for a class is called `Finalize`.

Figure 11-8 shows an `Employee` class that contains only one field `(idNumber)`, a constructor, and a shaded destructor. Although it is unusual for a constructor or destructor to output anything, these display messages so you can see when the objects are created and destroyed. When you execute the client program in Figure 11-9, you instantiate two `Employee` objects, each with its own `idNumber` value. When the program ends, the two `Employee` objects go out of scope, and the destructor for each object is called automatically. Figure 11-10 shows the output.

```
class Employee
   Declarations
      private string idNumber
   public Employee(string empID)
      idNumber = empId
      output "Employee ", idNumber, " is created"
   return
   public ~Employee()
      output "Employee ", idNumber, " is destroyed"
   return
endClass
```

Figure 11-8 `Employee` class with destructor

```
start
   Declarations
      Employee aWorker("101")
      Employee anotherWorker("202")
stop
```

Figure 11-9 Program that declares two `Employee` objects

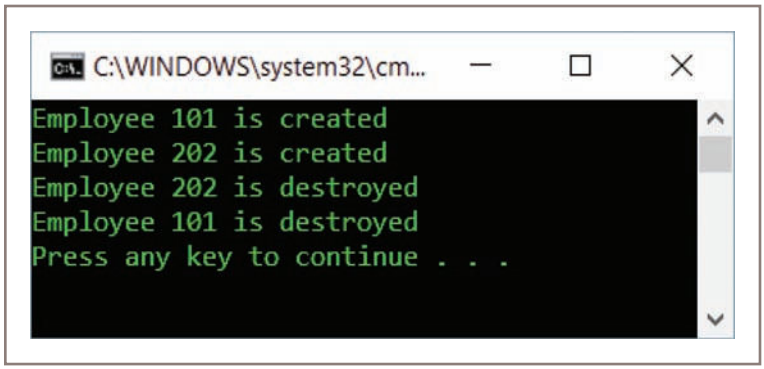


Figure 11-10 Output of program in Figure 11-9

The program in Figure 11-9 never explicitly calls the `Employee` class destructor, yet you can see from the output that the destructor executes twice. Destructors are invoked automatically; you usually do not explicitly call one, although in some languages you can. Interestingly, you can see from the output in Figure 11-10 that the last object created is the first object destroyed; the same relationship would hold true no matter how many objects the program instantiated if the objects went out of scope at the same time.



An instance of a class becomes *eligible* for destruction when it is no longer possible for any code to use it—that is, when it goes out of scope. In many languages, the actual execution of an object's destructor might occur at any time after the object becomes eligible for destruction.

For now, you have little reason to create a destructor except to demonstrate how it is called automatically. Later, when you write more sophisticated programs that work with files, databases, or large quantities of computer memory, you might want to perform specific cleanup or close-down tasks when an object goes out of scope. If you develop a game in which the player attempts to eliminate monsters or pop bubbles, you might want to perform an action such as adding points to the player's score when an object in the game is destroyed. In these examples, you could place appropriate instructions within a destructor.

TWO TRUTHS & A LIE

Understanding Destructors

1. Unlike constructors, you must explicitly create a destructor if you want one for a class.
2. A destructor must have an empty parameter list.
3. Destructors cannot be overloaded; a class can have only one destructor.

The false statement is #1. As with constructors, if you do not explicitly create a destructor for a class, one is provided automatically.

Understanding Composition

A class can contain simple variables as data fields, and it can contain objects of another class as data fields. For example, you might create a class named `Date` that contains a month, day, and year, and add two `Date` fields to an `Employee` class to hold the `Employee`'s birth date and hire date. Then you might create a class named `Department` that represents every department in a company, and create the `Department` class to contain a supervisor, who is an `Employee`. Figure 11-11 contains a diagram of these relationships. When a class contains objects of another class, the relationship is called a **whole-part relationship** or **composition**. The relationship created is also called a **has-a relationship** because one class "has an" instance of another—for example, a `Department` has an `Employee` and an `Employee` has a hire `Date`.

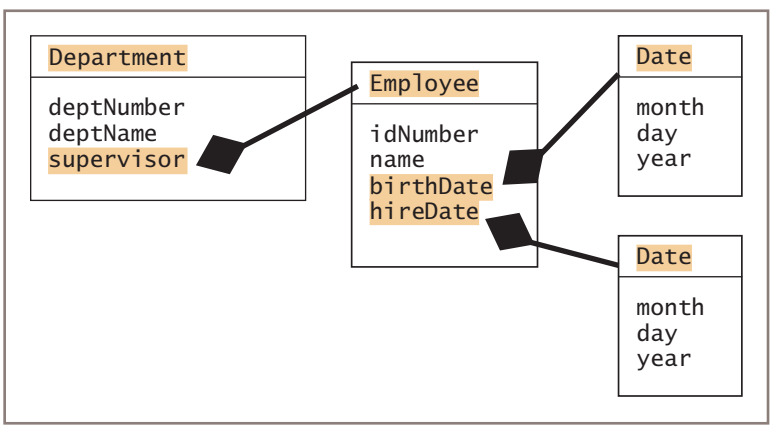


Figure 11-11 Diagram of typical composition relationships



Placing one or more objects within another object is often known as *composition* when the parts cease to exist if the whole ceases to exist, and *aggregation* when the parts can exist without the whole. For example, the relationship of a `Business` to its `Departments` is *composition* because if the `Business` ceases to exist, so do its `Departments`. However, the relationship of a `Department` to its `Employees` might be called *aggregation* because the `Employees` continue to exist even if their `Department` does not.

When your classes contain objects that are members of other classes, your programming task becomes increasingly complex. For example, you sometimes must refer to a method by a very long name. Suppose you create a `Department` class that contains an array of `Employee` objects (those who work in the department), and a method named `getHighestPaidEmployee()` that returns a single `Employee` object. The `Employee` class contains a method named `getHireDate()`that returns a `Date` object—an `Employee`'s hire date. Further suppose the `Date` class contains a method that returns the year portion of the `Date`, and that you create a `Department` object named `sales`. An application might contain a statement such as the following, which outputs the year that the highest-paid employee in the sales department was hired:

```
output sales.getHighestPaidEmployee().getHireDate() .getYear()
```

Additionally, when classes contain objects that are members of other classes, all the corresponding constructors and destructors execute in a specific order. As you work with object-oriented programming languages, you will learn to manage these complex issues.

TWO TRUTHS & A LIE

Understanding Composition

1. A class can contain objects of another class as data fields.
2. Composition occurs when you use an object as a field within another class.
3. Composition is called an *is-a* relationship because one class "is an" instance of another.

The false statement is #3. Composition is called a *has-a* relationship because one class "has an" instance of another.

Understanding Inheritance

Understanding classes helps you organize objects in real life. Understanding inheritance helps you organize them more precisely. Inheritance enables you to apply your knowledge of a general category to more specific objects. When you use the term *inheritance*, you might think of genetic inheritance. You know from biology that your blood type and eye color are determined by inherited genes. You might choose to own plants and animals based on their inherited attributes. You plant impatiens next to your house because they thrive in the shade; you adopt a poodle because you know poodles don't shed. Every plant and pet has slightly different characteristics, but within a species, you can count on many consistent inherited attributes and behaviors. In other words, you can reuse the knowledge you gain about general categories and apply it to more specific categories.

Similarly, the classes you create in object-oriented programming languages can inherit data and methods from existing classes. When you create a class by making it inherit from another class, the new class contains fields and methods automatically, allowing you to reuse fields and methods that are already written and tested.

You already know how to create classes and how to instantiate objects that are members of those classes.

For example, consider the `Employee` class in Figure 11-12. The class contains two data fields, `empNum` and `weeklySalary`, as well as methods that get and set each field.

Suppose that you hire a new type of `Employee` who earns a commission as well as a weekly salary. You could create a class with a name such as `CommissionEmployee`, and provide this class with three fields (`empNum`, `weeklySalary`, and `commissionRate`) and six methods (to get and set each of the three fields). However, this would duplicate much of the work that you already have done when creating the `Employee` class. The wise and efficient alternative is to create the `CommissionEmployee` class so it inherits all the attributes and methods of `Employee`. Then, you can add just the single field and two methods (the get and set methods for the new field) that are needed to complete the new class. Figure 11-13 depicts these relationships. The complete `CommissionEmployee` class is shown in Figure 11-14.

```
class Employee
   Declarations
      private string empNum
      private num weeklySalary

   public void setEmpNum(string number)
      empNum = number
   return

   public string getEmpNum()
   return empNum

   public void setWeeklySalary(num salary)
      weeklySalary = salary
   return

   public num getWeeklySalary()
   return weeklySalary
endClass
```

Figure 11-12 An `Employee` class

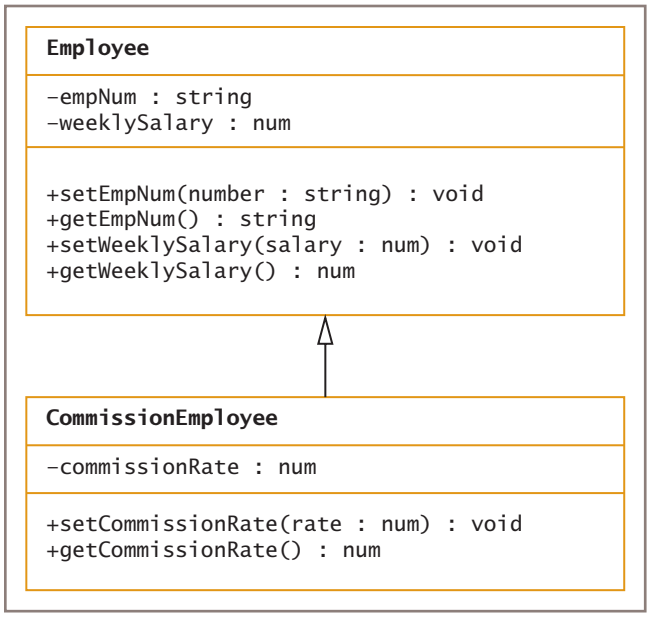


Figure 11-13 CommissionEmployee inherits from Employee

Recall that a plus sign in a class diagram indicates public access and a minus sign indicates private access.

```
class CommissionEmployee inheritsFrom Employee
   Declarations
      private num commissionRate

   public void setCommissionRate(num rate)
      commissionRate = rate
   return

   public num getCommissionRate()
   return commissionRate
endClass
```

Figure 11-14 CommissionEmployee class

The class in Figure 11-14 uses the phrase `inheritsFrom Employee` (see shading) to indicate inheritance. Each programming language uses its own syntax. For example, in Java you would write `extends`, in Visual Basic you would write `Inherits`, and in C++ and C# you would use a colon between the new class name and the one from which it inherits.

When you use inheritance to create the `CommissionEmployee` class, you acquire the following benefits:

- You save time, because you need not re-create the `Employee` fields and methods.
- You reduce the chance of errors, because the `Employee` methods have already been written and tested.
- You make it easier for anyone who has used the `Employee` class to understand the `CommissionEmployee` class, because these users can concentrate on the new features only.
- You reduce the chance for errors and inconsistencies in shared fields. For example, if your company decides to change employee ID numbers from four digits to five, and you have code in the `Employee` class constructor that ensures valid ID numbers, then you can simply change the code in the `Employee` class; every `CommissionEmployee` object will automatically acquire the change. Without inheritance, not only would you have to make the change in multiple places, but the likelihood would increase that you would forget to make the change in one of the classes or introduce an inconsistency when making a change in one of the classes.

Imagine that besides `CommissionEmployee`, you want to create several other more specific `Employee` classes (perhaps `PartTimeEmployee`, including a field for hours worked, or `DismissedEmployee`, including a reason for dismissal). By using inheritance, you can develop each new class correctly and more quickly. The ability to use inheritance makes programs easier to write, easier to understand, and less prone to errors.



In part, the concept of class inheritance is useful because it makes class code reusable. However, you do not use inheritance simply to save work. When properly used, inheritance always involves general-to-specific relationships that make logical sense.

Understanding Inheritance Terminology

A class that is used as a basis for inheritance, like `Employee`, is called a **base class**. When you create a class that inherits from a base class (such as `CommissionEmployee`), it is a **derived class** or **extended class**. When two classes have a base-derived relationship, you can distinguish the classes by using them in a sentence with the phrase *is a*. A derived class always "is-a" case or instance of the more general base class. For example, a `Tree` class might be a base class to an `Evergreen` class. Every `Evergreen` *is a* `Tree`; however, it is not true that every `Tree` is an `Evergreen`. Thus, `Tree` is the base class and `Evergreen` is the derived class. Similarly, a `CommissionEmployee` *is an* `Employee`—not always the other way around—so `Employee` is the base class and `CommissionEmployee` is derived.

You can use the terms **superclass** and **subclass** as synonyms for base class and derived class. Thus, `Evergreen` can be called a subclass of the `Tree` superclass. You also can use the terms **parent class** and **child class**. A `CommissionEmployee` is a child to the `Employee` parent.

As an alternative way to discover which of two classes is the base class and which is the derived class, you can try saying the two class names together, although this technique

might not work with every base-subclass pair. When people say their names in the English language, they state the more specific name before the all-encompassing family name, such as *Mary Johnson*. Similarly, with classes, the order that "makes more sense" is the child-parent order. Thus, because "Evergreen Tree" makes more sense than "Tree Evergreen," you can deduce that `Evergreen` is the child class. It also is convenient to think of a derived class as building upon its base class by providing the "adjectives" or additional descriptive terms for the "noun." Frequently, the names of derived classes are formed in this way, as in `CommissionEmployee` or `EvergreenTree`.

Finally, you usually can distinguish base classes from their derived classes by size. Although it is not required, a derived class is generally larger than its base class, in the sense that it usually has additional fields and methods. A subclass description might look small, but any subclass contains all of its base class's fields and methods as well as its own more specific fields and methods.



Do not think of a subclass as a *subset* of another class—in other words, as possessing only parts of its base class. In fact, a derived class usually contains more than its parent.

A derived class can be further extended. In other words, a subclass can have a child of its own. For example, after you create a `Tree` class and derive `Evergreen`, you might derive a `Spruce` class from `Evergreen`. Similarly, a `Poodle` class might derive from `Dog`, `Dog` from `DomesticPet`, and `DomesticPet` from `Animal`. The entire list of parent classes from which a child class is derived constitutes the **ancestors** of the subclass.



After you create the `Spruce` class, you might be ready to create `Spruce` objects. For example, you might create `theTreeInMyBackYard`, or you might create an array of 1000 `Spruce` objects for a tree farm. On the other hand, before you are ready to create objects, you might first want to create even more specific child classes such as `ColoradoSpruce` and `NorwaySpruce`.

A child inherits all the data fields and methods of all its ancestors. For example, when you declare a `Spruce` object, it contains all the attributes and methods of both an `Evergreen` and a `Tree`, and a `CommissionEmployee` contains all the attributes and methods of an `Employee`. In other words, the components of `Employee` and `CommissionEmployee` are as follows:

- `Employee` contains two fields and four methods, as shown in Figure 11-12.
- `CommissionEmployee` contains three fields and six methods, even though you do not see all of them in Figure 11-14. Two of its fields and four of its methods are defined in its parent's class.

Although a child class contains all the data fields and methods of its parent, a parent class does not gain any child class data or methods. Therefore, when `Employee` and `CommissionEmployee` classes are defined as in Figures 11-12 and 11-14, the statements in Figure 11-15 are all valid in an application. The `salesperson` object can use all the methods of its parent, and it can use its own `setCommissionRate()` and `getCommissionRate()` methods. Figure 11-16 shows the output of the program as it would appear in a command-line environment.

```
start
   Declarations
      Employee manager
      CommissionEmployee salesperson
   manager.setEmpNum("111")
   manager.setWeeklySalary(700.00)
   salesperson.setEmpNum("222")
   salesperson.setWeeklySalary(300.00)
   salesperson.setCommissionRate(0.12)
   output "Manager ", manager.getEmpNum(), manager.getWeeklySalary()
   output "Salesperson ", salesperson.getEmpNum(),
      salesperson.getWeeklySalary(), salesperson.getCommissionRate()
stop
```

Figure 11-15 `EmployeeDemo` application that declares two `Employee` objects

The following statements would not be allowed in the `EmployeeDemo` application in Figure 11-15 because `manager`, as an `Employee` object, does not have access to the methods of the `CommissionEmployee` child class:

```
manager.setCommissionRate(0.08)
output manager.getCommissionRate()
```

Don't Do It
A base class object cannot use methods that belong to its child class.

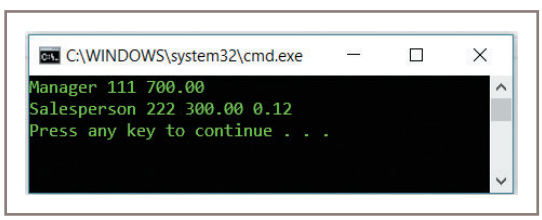


Figure 11-16 Output of the program in Figure 11-15



When you create your own inheritance chains, you want to place fields and methods at their most general level. In other words, a method named `grow()` rightfully belongs in a `Tree` class, whereas it would be appropriate to place a `leavesTurnColor()` method in a `DeciduousTree` class rather than separately within `Oak` or `Maple` child classes.

It makes sense that a parent class object does not have access to its child's data and methods. When you create the parent class, you do not know how many future child classes might be created, or what their data or methods might look like. In addition, derived classes are more specific, so parent class objects cannot use them. For example, a `Cardiologist` class and an `Obstetrician` class are children of a `Doctor` class. You do not expect all members of the general parent class `Doctor` to have the `Cardiologist`'s `repairHeartValve()` method or the `Obstetrician`'s `performCaesarianSection()` method. However, `Cardiologist` and `Obstetrician` objects have access to the more general `Doctor` methods `takeBloodPressure()` and `billPatients()`. As with specialization of doctors, it

is convenient to think of derived classes as *specialists*. That is, their fields and methods are more specialized than those of the parent class.



In some programming languages, such as C#, Visual Basic, and Java, every class you create is a child of one ultimate base class, often called the `Object` class. The `Object` class usually provides basic functionality that is inherited by all the classes you create—for example, the ability to show its name or an object's memory location.

Accessing Private Fields and Methods of a Parent Class

In Chapter 10, you learned that when you create classes, the most common scenario is for methods to be public but for data to be private. Making data private is an important concept in object-oriented programming. By making data fields private and allowing access to them only through a class's methods, you control the ways in which data items can be altered and used.

When a data field within a class is private, no outside class can use it—including a child class. The principle of data hiding would be lost if you could access a class's private data merely by creating a child class. However, it can be inconvenient when the methods of a child class cannot directly access the data fields it inherits.



Watch the video *Inheritance*.

For example, suppose that some employees do not earn a weekly salary as defined in the `Employee` class, but are paid by the hour. You might create an `HourlyEmployee` class that descends from `Employee`, as shown in Figure 11-17. The class contains two new fields, `hoursWorked` and `hourlyRate`, and a get and set method for each.

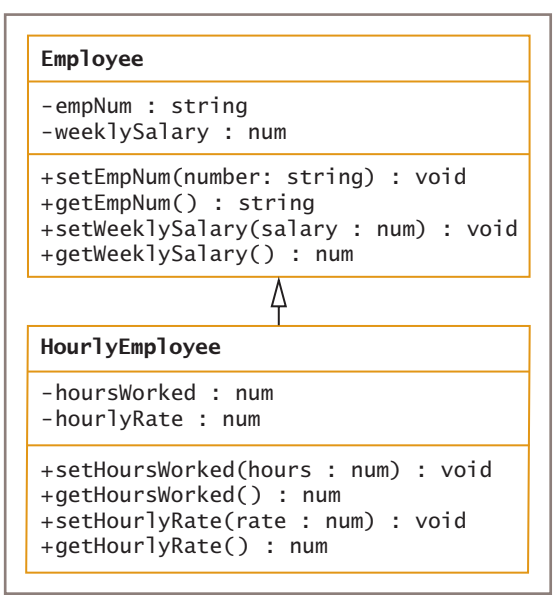


Figure 11-17 Class diagram for `HourlyEmployee` class

You can implement the new class as shown in Figure 11-18. Whenever you set either `hoursWorked` or `hourlyRate`, you want to recalculate `weeklySalary` using the newly modified values for the hours and rate. The logic makes sense, but when you write the code in a programming language, it does not compile. The two shaded statements show that the `HourlyEmployee` class is attempting to modify the `weeklySalary` field. Although every `HourlyEmployee` *has* a `weeklySalary` field by virtue of being a child of `Employee`, the `HourlyEmployee` class methods do not have access to the `weeklySalary` field, because `weeklySalary` is private within the `Employee` class. In this case, the private `weeklySalary` field is **inaccessible** to any class other than the one in which it is defined.

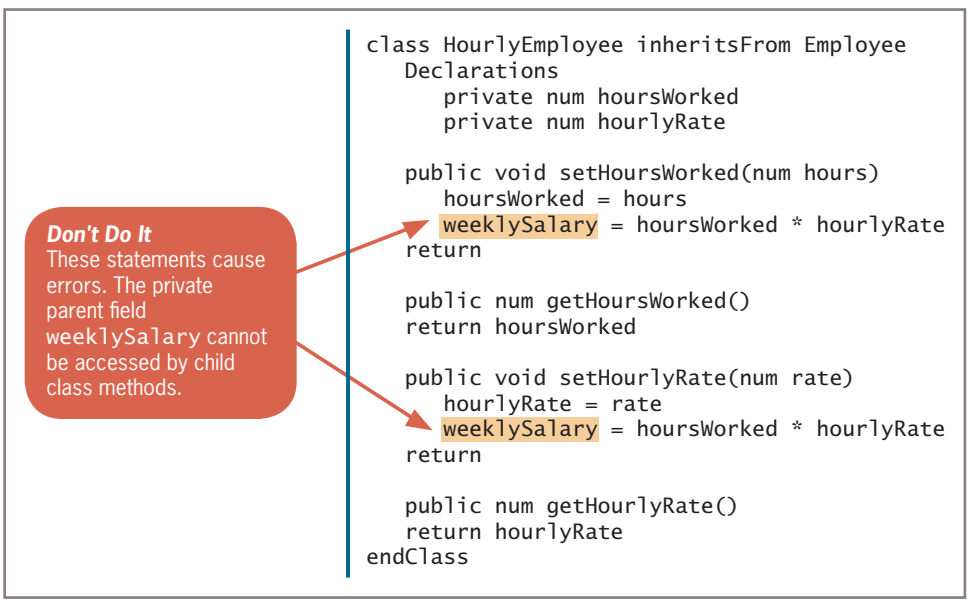


Figure 11-18 Implementation of `HourlyEmployee` class that attempts to access `weeklySalary`

One solution to this problem would be to make `weeklySalary` public in the parent `Employee` class. Then the child class could use it. However, that action would violate the important object-oriented principle of data hiding. Good object-oriented style dictates that your data should be altered only by the methods you choose and only in ways that you can control. If outside classes could alter an `Employee`'s private fields, then the fields could be assigned values that the `Employee` class could not control. In such a case, the principle of data hiding would be destroyed, causing the behavior of the object to be unpredictable.

Therefore, OOP languages allow a medium-security access specifier that is more restrictive than public but less restrictive than private. The `protected` **access specifier** is used when you want no outside classes to be able to use a data field, except classes that are children of the original class. Figure 11-19 shows a rewritten `Employee` class that uses

the `protected` access specifier on one of its data fields (see shading). When this modified class is used as a base class for another class, such as `HourlyEmployee`, the child class's methods will be able to access any protected items (fields or methods) originally defined in the parent class. When the `Employee` class is defined with a protected `weeklySalary` field, as shown in Figure 11-19, the code in the `HourlyEmployee` class in Figure 11-18 works correctly.

```
class Employee
   Declarations
      private string empNum
      protected num weeklySalary

   public void setEmpNum(string number)
      empNum = number
   return

   public string getEmpNum()
   return empNum

   public void setWeeklySalary(num salary)
      weeklySalary = salary
   return

   public num getWeeklySalary()
   return weeklySalary
endClass
```

Figure 11-19 `Employee` class with a protected field

Figure 11-20 contains the class diagram for the version of the `Employee` class shown in Figure 11-19. Notice that the `weeklySalary` field is preceded with an octothorpe (#)—the character that conventionally is used in class diagrams to indicate protected class members.

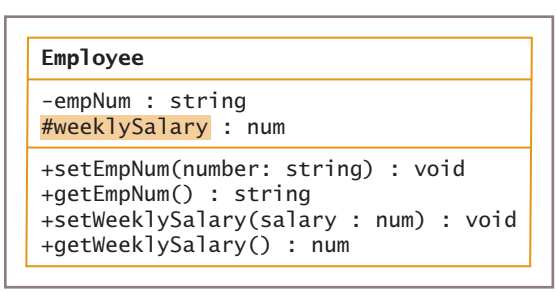


Figure 11-20 `Employee` class with protected member

If `weeklySalary` is defined as protected instead of private in the `Employee` class, then either the creator of the class knew that a child class would want to access the field or the class was revised after it became known the child class would need access to the field.

If the `Employee` class's creator did not foresee that a field would need to be accessible, or if it is not preferable to revise the class, then `weeklySalary` will remain private. It is still possible to correctly set an `HourlyEmployee`'s weekly pay—the `HourlyEmployee` is just required to use the same means as any other class would. That is, the `HourlyEmployee` class can use the public method `setWeeklySalary()` that already exists in the parent class. Any class, including a child, can use a public field or method of the base class. So, assuming that `weeklySalary` remains private in `Employee`, Figure 11-21 shows how `HourlyEmployee` could be written to correctly set `weeklySalary`.

```
class HourlyEmployee inheritsFrom Employee
   Declarations
      private num hoursWorked
      private num hourlyRate

   public void setHoursWorked(num hours)
      hoursWorked = hours
      setWeeklySalary(hoursWorked * hourlyRate)
   return

   public num getHoursWorked()
   return hoursWorked

   public void setHourlyRate(num rate)
      hourlyRate = rate
      setWeeklySalary(hoursWorked * hourlyRate)
   return

   public num getHourlyRate()
   return hourlyRate
endClass
```

Figure 11-21 The `HourlyEmployee` class when `weeklySalary` remains private

In the version of `HourlyEmployee` in Figure 11-21, the shaded statements within `setHoursWorked()` and `setHourlyRate()` assign a value to the corresponding child class field (`hoursWorked` or `hourlyRate`, respectively). Each method then calls the public parent class method `setWeeklySalary()`. In this example, no `protected` access specifiers are needed for any fields in the parent class, and the creators of the parent class did not have to foresee that a child class would eventually need to access any of its fields. Instead, any child classes of `Employee` simply follow the same access rules as any other outside class would. As an added benefit, if the parent class method `setWeeklySalary()` contained additional code (for example, to require a minimum base weekly pay for all employees), then that code would be enforced even for `HourlyEmployees`.

So, in summary, when you create a child class that must access a private field of its parent's class, you can take one of several approaches:

- If you are the developer of the parent class, you can modify the parent class to make the field public. Usually, this is not advised because it violates the principle of data hiding.
- If you are the developer of the parent class, you can modify the parent class to make the field protected so that child classes have access to it, but other outside classes do not. This approach is necessary if you do not want public methods to be able to access the parent class field. Be aware that some programmers oppose making any data fields non-private. They feel that public methods should always control data access, even by a class's children.
- The child class can use a public method within the parent class that modifies the field, just as any other outside class would. This is frequently, but not always, the best option.

Using the `protected` access specifier for a field can be convenient, and it improves program performance a little by using a field directly instead of "going through" another method. Also, using the `protected` access specifier is occasionally necessary when no existing public method accesses a field in a way required by the child class. However, protected data members should be used sparingly. Whenever possible, the principle of data hiding should be observed, and even child classes should have to go through methods to "get to" their parent's private data.

The likelihood of future errors increases when child classes are allowed direct access to a parent's fields. For example, if the company decides to add a bonus to every `Employee`'s weekly salary, you might make a change in the `setWeeklySalary()` method. If a child class is allowed direct access to the `Employee` field `weeklySalary` without using the `setWeeklySalary()` method, then any child class objects will not receive the bonus. Classes that depend on field names from parent classes are said to be **fragile** because they are prone to errors—that is, they are easy to "break."



Some OOP languages, such as C++, allow a subclass to inherit from more than one parent class. For example, you might create an `InsuredItem` class that contains data fields such as value and purchase date for each insured possession, and an `Automobile` class with appropriate data fields (for example, vehicle identification number, make, model, and year). When you create an `InsuredAutomobile` class for a car rental agency, you might want to include information and methods for `Automobiles` and `InsuredItems`, so you might want to inherit from both. The capability to inherit from more than one class is called **multiple inheritance**.



Sometimes, a parent class is so general that you never intend to create any specific instances of the class. For example, you might never create an object that is "just" an `Employee`; each `Employee` is more specifically a `SalariedEmployee`, `HourlyEmployee`, or `ContractEmployee`. A class such as `Employee` that you create only to extend from, but not to instantiate objects from, is an abstract class. An **abstract class** is one from which you cannot create any concrete objects, but from which you can inherit.

Overriding Parent Class Methods in a Child Class

Overriding is the mechanism by which a child class method is used by default when a parent class contains a method with the same signature. That is, by default, the child class version of the method is used with any child class object.

For example, suppose that an `Employee` class contains a `setEmpNum()` method that accepts a string parameter and assigns it to a private data field named `empNum`. If you create a `CommissionEmployee` class that is a child of `Employee`, and if the `CommissionEmployee` class does not contain its own `setEmpNum()` method that accepts a string parameter, the parent class method executes when you use the method name with a child class object.

However, suppose that `CommissionEmployee` is a child of `Employee` and that it also contains a `setEmpNum()` method that accepts a string parameter, but concatenates a "C" to the value before assigning it to `empNum`. When you declare an `Employee` object and use the method name with it, the parent class version of the method executes. When you declare a `CommissionEmployee` object and use the method name with it, the child class version of the method overrides the parent class version and executes appropriately.

Using Inheritance to Achieve Good Software Design

When an automobile company designs a new car model, it does not build every component of the new car from scratch. The company might design a new feature; for example, at some point a carmaker designed the first air bag. However, many of a new car's features are simply modifications of existing features. The manufacturer might create a larger gas tank or more comfortable seats, but even these new features still possess many properties of their predecessors in the older models. Most features of new car models are not even modified; instead, existing components such as air filters and windshield wipers are included on the new model without any changes.

Similarly, you can create powerful computer programs more easily if many of their components are used either "as is" or with slight modifications. Inheritance makes your job easier because you don't have to create every part of a new class from scratch. Professional programmers constantly create new class libraries for use with OOP languages. Having classes in these libraries that are available to use and extend makes programming large systems more manageable. When you create a useful, extendable superclass, you and other future programmers gain several advantages:

- Subclass creators save development time because much of the code needed for the class has already been written.
- Subclass creators save testing time because the superclass code has already been tested and probably used in a variety of situations. In other words, the superclass code is reliable.
- Programmers who create or use new subclasses already understand how the superclass works, so the time it takes to learn the new class features is reduced.
- When you create a new subclass, neither the superclass source code nor the translated superclass code is changed. The superclass maintains its integrity.

When you consider classes, you must think about their commonalities, and then you can create superclasses from which to inherit. You might be rewarded professionally when you see your own superclasses extended by others in the future.

TWO TRUTHS & A LIE

Understanding Inheritance

1. When you create a class by making it inherit from another class, you save time because you need not re-create the base class fields and methods.
2. A class that is used as a basis for inheritance is called a base class, derived class, or extended class.
3. When a data field within a class is private, no outside class can use it—including a child class.

The false statement is #2. A class that is used as a basis for inheritance is called a base class, superclass, or parent class. A derived class is a subclass, extended class, or child class.

An Example of Using Predefined Classes: Creating GUI Objects

When you purchase or download a compiler for an object-oriented programming language, it comes packaged with many predefined, built-in classes. The classes are stored in **libraries** or **packages**—collections of classes that serve related purposes. Some of the most helpful are the classes you can use to create graphical user interface (GUI) objects such as frames, buttons, labels, and text boxes. You place these GUI components within interactive programs so that users can manipulate them by typing on a keyboard, clicking a mouse, or touching or swiping a screen. For example, if you want to place a clickable button on the screen using a language that supports GUI applications, you instantiate an object that belongs to an existing class with a name similar to `Button`. You then create objects with names such as `yesButton` or `buyProductNowButton`. The `Button` class contains private data fields such as `text` and `height` and public methods such as `setText()` and `setHeight()` that allow you to alter the objects' fields. For example, you might write a statement such as the following to change the text on a `Button` object:

```
buyProductNowButton.setText("Click here to buy now")
```

If no predefined GUI object classes existed, you could create your own. However, this would present several disadvantages:

- It would be a lot of work. Creating graphical objects requires a substantial amount of code and at least a modicum of artistic talent.

- It would be repetitious work. Almost all GUI programs require standard components such as buttons and labels. If each programmer created the classes for these components from scratch, much of this work would be repeated unnecessarily.
- The components would look different in various applications. If each programmer created his or her own component classes, objects such as buttons would look different and operate in slightly different ways. Users prefer standardization in their components—title bars on windows that are a uniform height, buttons that appear to be pressed when clicked, frames and windows that contain maximize and minimize buttons in predictable locations, and so on. By using standard component classes, programmers are assured that the GUI components in their programs have the same look and feel as those in other programs.

Programming languages that supply existing GUI classes often provide a **visual development environment** in which you can create programs by dragging components such as buttons and labels onto a screen and arranging them visually. (In several languages, the visual development environment is known by the acronym **IDE**, which stands for *integrated development environment*.) Then you write programming statements to control the actions that take place when a user manipulates the controls—by clicking them using a mouse, for example. Many programmers never create classes of their own from which they will instantiate objects, but only write application classes that use built-in GUI component classes. Some languages—for example, Visual Basic and C#—lend themselves very well to this type of programming. In Chapter 12, you will learn more about creating programs that use GUI objects.

TWO TRUTHS & A LIE

An Example of Using Predefined Classes: Creating GUI Objects

1. Collections of classes that serve related purposes are called annals.
2. GUI components are placed within interactive programs so that users can manipulate them using input devices.
3. By using standard component classes, programmers are assured that the GUI components in their programs have the same look and feel as those in other programs.

The false statement is #1. Collections of classes that serve related purposes are stored in libraries or packages.

Understanding Exception Handling

A great deal of the effort that goes into writing programs involves checking data items to make sure they are valid and reasonable. Professional data-entry operators who create the files used in business applications spend their entire working day entering facts and figures,

so operators can and do make typing errors. When programs depend on data entered by average users who are not trained typists, the likelihood of errors is even greater.



Programmers use the acronym *GIGO* to describe what happens when worthless or invalid input causes inaccurate or unrealistic results. GIGO is an acronym for "garbage in, garbage out."

In procedural programs, programmers handled errors in various ways that were effective, but the techniques had some drawbacks. The introduction of object-oriented programming has led to a new model called exception handling.

Drawbacks to Traditional Error-Handling Techniques

In traditional programming, probably the most common error-handling outcome was to terminate the program, or at least to terminate the method in which the offending statement executed. For example, suppose that a program prompts a user to enter an insurance premium type from the keyboard, and that the entered value should be *A* or *H* for *Auto* or *Health*. Figure 11-22 shows a segment of pseudocode that causes the `determinePremium()` method to end if `policyType` is invalid; in the shaded `if` statement, the method ends abruptly when `policyType` is not *A* or *H*. This method of handling an error is not only unforgiving, it isn't even structured. Recall that a structured method should have exactly one entry point and exactly one exit point. The method in Figure 11-22 contains two exit points at the two `return` statements.

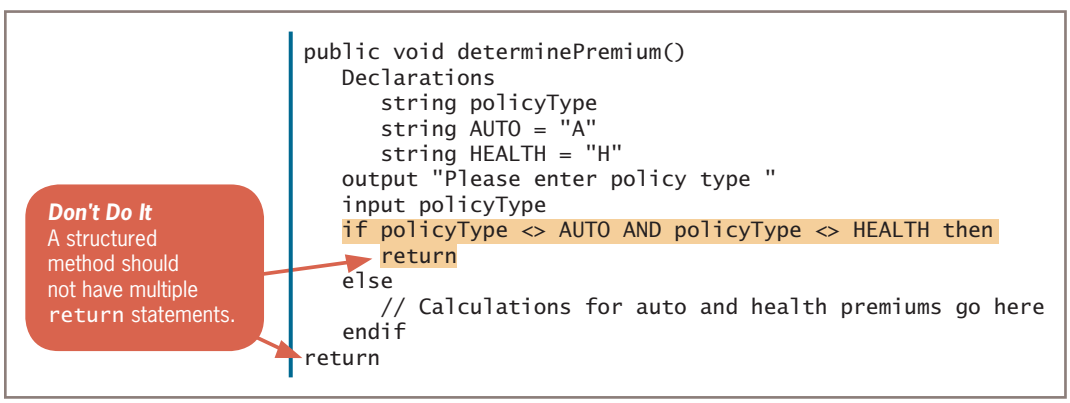


Figure 11-22 A method that handles an error in an unstructured manner

In the example in Figure 11-22, if `policyType` contains an invalid value, the method in which the code appears is terminated. The client program might continue with an invalid value or it might stop working. If the program that contains this method is part of a business program or a game, the user might be annoyed. However, an early termination in a program that monitors a hospital patient's vital signs or navigates an airplane might have far more serious consequences.

Rather than ending a method prematurely just because it encounters a piece of invalid data, a more elegant solution involves repeating data entry in a loop until the data item becomes valid, as shown in the highlighted portion of Figure 11-23. As long as the value of `policyType` is invalid, the user is prompted continuously to enter a new value. Only when `policyType` is *A* or *H* does the method continue.

```
public void determinePremium()
   Declarations
      string policyType
      string AUTO = "A"
      string HEALTH = "H"
   output "Please enter policy type "
   input policyType
   while policyType <> AUTO AND policyType <> HEALTH
      output "You must enter ", AUTO, " or ", HEALTH
      input policyType
   endwhile
   // Calculations for auto and health premiums go here
return
```

Figure 11-23 A method that handles an error using a loop

The error-handling logic shown in Figure 11-23 has at least two shortcomings:

- The method is not as reusable as it could be.
- The method is not as flexible as it might be.

One of the principles of modular and object-oriented programming is reusability. The method in Figure 11-23 is reusable only under limited conditions. The `determinePremium()` method allows the user to reenter policy data any number of times, but other programs in the insurance system might need to limit the number of chances the user gets to enter correct data, or might allow no second chance at all. A more flexible `determinePremium()` method would be able to detect an error and then notify the calling program or method that an error has occurred. Each client that uses the `determinePremium()` method then could handle the mistake appropriately for the current application.

The other drawback to forcing the user to reenter data is that the technique works only with interactive programs. A more flexible program accepts any kind of input, including data stored in a file. Program errors can occur as a result of many factors—for example, a disk drive might not be ready, a file might not exist in the specified location, or stored data items might be invalid. You cannot continue to reprompt a storage device for valid data the way you can reprompt a user in an interactive program; if stored data is invalid, it remains invalid.

In the next section, you will learn object-oriented exception-handling techniques that overcome the limitations of traditional error handling.

The Object-Oriented Exception-Handling Model

Object-oriented programs employ a group of techniques for handling errors called **exception handling**. The generic name used for errors in object-oriented languages is **exceptions** because errors are not usual occurrences; they are the "exceptions to the rule."

In object-oriented terminology, you **try** some code that might **throw an exception**. If an exception is thrown, it is passed to a block of code that can **catch the exception**, which means to receive it in a way similar to how a parameter is received by a method. In some languages, the exception object that is thrown can be any data type—a number, a string, or a programmer-created object. However, even when a language permits any data type to be thrown, most programmers throw an object of the built-in class `Exception`, or they use inheritance techniques to derive a class from a built-in `Exception` class.

For example, Figure 11-24 shows a `determinePremium()` method that throws an exception only if `policyType` is neither *H* nor *A*. If `policyType` is invalid, an object of type `Exception` named `mistake` is instantiated and thrown from the method by a `throw` statement. A **`throw` statement** is one that sends an `Exception` object out of the current code block or method so it can be handled elsewhere. If `policyType` is *H* or *A*, the method continues, the premium is calculated, and the method ends normally.

```
public void determinePremium()
   Declarations
      string policyType
      string AUTO = "A"
      string HEALTH = "H"
   output "Please enter policy type "
   input policyType
   if policyType <> AUTO AND policyType <> HEALTH then
      Declarations
         Exception mistake
      throw mistake
   else
      // Calculations for auto and health premiums go here
   endif
return
```

Figure 11-24 A method that creates and throws an `Exception` object

When you create a segment of code in which something might go wrong, you place the code in a **`try` block**, which is a block of code you attempt to execute while acknowledging that an exception might occur. A `try` block consists of the keyword `try` followed by any number of statements, including some that might cause an exception to be thrown. If a statement in the block causes an exception, the remaining statements in the `try` block do not execute and the `try` block is abandoned. For pseudocode purposes, you can end a `try` block with a sentinel keyword such as `endtry`.

You almost always code at least one `catch` block immediately following a `try` block. A **`catch` block** is a segment of code written to handle an exception that might be thrown by the `try` block that precedes it. Each `catch` block "catches" one type of exception that it handles—in many languages the caught object must be of type `Exception` or one of its child classes. You create a `catch` block using the following pseudocode elements:

- The keyword `catch`, followed by parentheses that contain an `Exception` type and an identifier
- Statements that take action to handle the error condition
- An `endcatch` keyword to indicate the end of the `catch` block in the pseudocode.

Figure 11-25 shows a client program that calls the `determinePremium()` method. Because `determinePremium()` has the potential to throw an exception, the call to the method is contained in a `try` block. If `determinePremium()` throws an exception, the `catch` block in the program executes; if all goes well and `determinePremium()` does not throw an exception, the `catch` block is bypassed. A `catch` block looks like a method named `catch()`, which takes an argument that is some type of `Exception`. However, it is not a method; it has no `return` type, and you can't call it directly.

```
start
   try
      determinePremium()
   endtry
   catch(Exception mistake)
      output "A mistake occurred"
   endcatch
   // Other statements that would execute whether
   // or not the exception was thrown could go here
stop
```

Figure 11-25 A program that contains a `try...catch` pair

In the program in Figure 11-25, a message is displayed when the exception is thrown. Another application might take different actions. For example, you might write an application in which the `catch` block forces the `policyType` to *H* or to *A*, or reprompts the user for a valid value. Various programs can use the `determinePremium()` method and handle an error in the way that is considered most appropriate.



In the method in Figure 11-25, the variable `mistake` in the `catch` block is an object of type `Exception`. The object is not used within the `catch` block, but it could be. For example, depending on the language, the `Exception` class might contain a method named `getMessage()` that returns a string explaining the cause of the error. In that case, you could place a statement such as `output mistake.getMessage()` in the `catch` block.



Even when a program uses a method that throws an exception, the exceptions are created and thrown only occasionally, when something goes wrong. Programmers sometimes refer to the more common situation in which nothing goes wrong as the **sunny day case**.

The general principle of exception handling in object-oriented programming is that a method that uses data should be able to detect errors, but not be required to handle them. The handling should be left to the application that uses the object, so that each application can use each method appropriately.



Watch the video *Exception Handling*.

Using Built-in Exceptions and Creating Your Own Exceptions

Many OOP languages provide built-in types that are subclasses of the language's basic `Exception` type. For example, Java, Visual Basic, and C# each provide dozens of categories of automatically created exception types with names like `ArrayOutOfBoundsException`, which is thrown when you attempt to use an invalid subscript with an array; and `DivideByZeroException`, which is thrown when a program attempts to divide a number by zero.

Although some actions, such as dividing by zero, are errors in all programming situations, the built-in `Exceptions` in a programming language cannot cover *every* condition that might be an `Exception` in your applications. For example, you might want to declare an `Exception` when your bank balance is negative or when an outside party attempts to access your email account. Most organizations have specific rules for exceptional data; for example, an employee number must not exceed three digits, or an hourly salary must not be less than the legal minimum wage. You can check for each of these potential error situations with `if` statements, and create and throw `Exceptions` if needed. Then, the methods that catch your `Exceptions` can react appropriately for their application.

To create your own throwable `Exception`, you usually extend a built-in `Exception` class. For example, you might create a class named `NegativeBankBalanceException` or `EmployeeNumberTooLargeException`. (When you create a class that derives from `Exception`, it is conventional, but not required, to use `Exception` in the name.) By inheriting from the `Exception` class, you gain access to methods contained in the parent class, such as those that display a default message describing the `Exception`. Depending on the language you are using, you might be able to extend from other throwable classes as well as `Exception`.

When you use built-in `Exception` types, they derive from the general `Exception` class. When you create specialized `Exception` types of your own, you also frequently derive them from the general `Exception` class. Either way, all the types can be caught by a `catch` block that is written to catch the general `Exception` type. In most object-oriented programming languages, a method can throw any number of exceptions. Either a general or more specific `catch` block must be available for each type of exception that is thrown; otherwise, the program terminates and issues an error message.

TWO TRUTHS & A LIE

Understanding Exception Handling

1. In object-oriented terminology, you try some code that might throw an exception, and the exception can then be caught and handled.
2. A `catch` block is a segment of code that can handle an exception that might be thrown by the `try` block preceding it.
3. The general principle of exception handling in object-oriented programming is that a method that uses data should be able both to detect and to handle most common errors.

The false statement is #3. The general principle of exception handling in object-oriented programming is that a method that uses data should be able to detect errors, but not be required to handle them.

Reviewing the Advantages of Object-Oriented Programming

In Chapter 10, and this chapter, you have been exposed to many concepts and features of object-oriented programming, which provide extensive benefits as you develop programs. Whether you instantiate objects from classes you have created or from those created by others, you save development time because each object automatically includes appropriate, reliable methods and attributes. When using inheritance, you can develop new classes more quickly by extending classes that already exist and work; you need to concentrate only on new features added by the new class. When using existing classes, you need to concentrate only on the interface to those classes, not on the internal instructions that make them work. By using polymorphism, you can use reasonable, easy-to-remember names for methods and concentrate on their purpose rather than on memorizing different method names.

TWO TRUTHS & A LIE

Reviewing the Advantages of Object-Oriented Programming

1. When you instantiate objects in programs, you save development time because each object automatically includes appropriate, reliable methods and attributes.
2. When using inheritance, you can develop new classes more quickly by extending existing classes that already work.
3. By using polymorphism, you can avoid the strict rules of procedural programming and take advantage of more flexible object-oriented methods.

The false statement is #3. By using polymorphism, you can use reasonable, easy-to-remember names for methods and concentrate on their purpose rather than on memorizing different method names.

Chapter Summary

- A constructor is a method that instantiates an object. A default constructor is one that requires no arguments; in OOP languages, a default constructor is created automatically by the compiler for every class you write. If you want to perform specific tasks when you create an instance of a class, then you can write your own constructor. In most programming languages, a constructor has the same name as its class, and cannot have a return type. Once you write a constructor for a class, you no longer receive the automatically-written default constructor.
- A destructor contains the actions you require when an instance of a class is destroyed, most often when the object goes out of scope. As with constructors, if you do not explicitly create a destructor for a class, one is automatically provided. The most common way to declare a destructor explicitly is to use an identifier that consists of a tilde (~) followed by the class name. You cannot provide parameters to a destructor; as a consequence, destructors cannot be overloaded. Like a constructor, a destructor has no return type.
- A class can contain objects of another class as data fields. Creating whole-part relationships is known as composition or aggregation (has-a relationship).
- When you create a class by making it inherit from another class, you are provided with prewritten and tested data fields and methods automatically. Using inheritance helps you save time, reduces the chance of errors and inconsistencies, and makes it easier for readers to understand your classes. A class that is used as a basis for inheritance is called a base class. A class that inherits from a base class is a derived class or extended class. The terms *superclass* and *parent class* are synonyms for *base class*. The terms *subclass* and *child class* are synonyms for *derived class*.
- Some of the most useful classes packaged in language libraries are used to create graphical user interface (GUI) objects such as frames, buttons, labels, and text boxes. Programming languages that supply existing GUI classes often provide a visual development environment in which you can create programs by dragging components such as buttons and labels onto a screen and arranging them visually.
- Exception-handling techniques are used to handle errors in object-oriented programs. When you try a block of code, you attempt to use it, and if an exception occurs, it is thrown. A `catch` block of the correct type can receive the thrown exception and handle it. Many OOP languages provide built-in `Exception` types, and you can create your own types by extending the `Exception` class.
- When you use object-oriented programming techniques, you save development time because each object automatically includes appropriate, reliable methods and attributes. Efficiency is achieved through both inheritance and polymorphism.

Key Terms

A **constructor** is an automatically called method that instantiates an object.

A **default constructor** is one that requires no arguments.

A **non-default constructor** or a **parameterized constructor** requires at least one argument.

A **destructor** is an automatically called method that contains the actions you require when an instance of a class is destroyed.

A **whole-part relationship** is a relationship in which an object of one class is contained within an object of another class.

Composition is the technique of placing an object within an object of another class.

A **has-a relationship** describes a whole-part relationship.

A **base class** is one that is used as a basis for inheritance.

A **derived class** or **extended class** is one that is extended from a base class.

Superclass and **parent class** are synonyms for *base class*.

Subclass and **child class** are synonyms for *derived class*.

The **ancestors** of a derived class are the entire list of parent classes from which the class is derived.

Inaccessible describes any field or method that cannot be reached.

The **`protected` access specifier** is used when you want no outside classes to be able to use a data field, except classes that are children of the original class.

Fragile describes classes that depend on field names from parent classes. They are prone to errors—that is, they are easy to "break."

Multiple inheritance is the capability to inherit from more than one class.

An **abstract class** is one from which you cannot instantiate concrete objects, but from which you can inherit.

Overriding is the mechanism by which a child class method is used by default when a parent class contains a method with the same signature.

Libraries are stored collections of classes that serve related purposes.

Packages are another name for libraries in some languages.

A **visual development environment** is one in which you can create programs by dragging components such as buttons and labels onto a screen and arranging them visually.

IDE is the acronym for *integrated development environment*, which is the visual development environment in some programming languages.

Exception handling is a set of object-oriented techniques for handling errors.

Exception is the generic term used for an error in object-oriented languages.

To **try** a statement or block of statements is to allow the statement(s) to possibly throw an exception.

To **throw an exception** is to pass it out of a block where it occurs, usually to a block that can handle it.

To **catch an exception** is to receive it from a throw so it can be handled.

A **`throw` statement** is one that sends an `Exception` object out of a block or method so it can be handled elsewhere.

A **`try` block** is a block of code you attempt to execute while acknowledging that an exception might occur; it consists of the keyword `try` followed by any number of statements, including some that might cause an exception to be thrown.

A **`catch` block** is a segment of code written to handle an exception that might be thrown by the `try` block that precedes it.

A **sunny day case** is a case in which nothing goes wrong.

Exercises



Review Questions

1. When you instantiate an object, the automatically created method that is called is a(n) _____ .

   a. creator
   b. initiator
   c. constructor
   d. architect

2. Every class has _____.

   a. exactly one constructor
   b. at least one constructor
   c. at least two constructors
   d. a default constructor and a programmer-written constructor

3. Which of the following can be overloaded?

   a. constructors
   b. instance methods
   c. both of the above
   d. none of the above

4. Every default constructor _____ .

   a. requires no parameter
   b. sets a default value for every field in a class
   c. is created automatically
   d. is the only constructor that is explicitly written in a class

5. When you write a constructor that receives a parameter, _____ .

   a. the automatically-created default constructor no longer exists
   b. the parameter must be used to set a data field
   c. it becomes the default constructor
   d. the constructor body must be empty

6. When you write a constructor that receives no parameters, _____ .

   a. the automatically-created constructor no longer exists
   b. it becomes known as the default constructor
   c. both of the above
   d. none of the above

7. Most often, a destructor is called when _____ .

   a. an explicit call is made to it
   b. an object is instantiated
   c. an object goes out of scope
   d. a value is returned from a class method

8. Which of the following is *not* a similarity between constructors and destructors?

   a. Both can be called automatically.
   b. Both can be overloaded.
   c. Both have the same name as their class.
   d. Both have no return type.

9. Which of the following is *not* an advantage of creating a class that inherits from another class?

   a. You make it easier for anyone who has used the original class to understand the new class.
   b. You save time because you need not re-create the fields and methods in the original class.
   c. You reduce the chance of errors because the original class's methods have already been used and tested.
   d. You save time because subclasses are created automatically from those that come built in as part of a programming language.

10. Employing inheritance reduces errors because _____ .

    a. the new classes have access to fewer data fields
    b. the new classes have access to fewer methods
    c. you can copy and paste methods that you already created
    d. many of the methods you need have already been used and tested

11. A class that is used as a basis for inheritance is called a _____ .

    a. derived class
    b. subclass
    c. base class
    d. child class

12. Which of the following is another name for a derived class?

    a. base class
    b. child class
    c. superclass
    d. parent class

13. Which of the following is *not* another name for a derived class?

    a. extended class
    b. subclass
    c. child class
    d. superclass

14. Which of the following is true?

    a. A base class usually has more fields than its descendent.
    b. A child class can also be a parent class.
    c. A class's ancestors consist of its entire list of children.
    d. To be considered object oriented, a class must have a child.

15. A derived class inherits _____ data and methods of its ancestors.

    a. all
    b. only the public
    c. only the private
    d. no

16. Which of the following is true?

    a. A class's methods usually are public.
    b. A class's data fields usually are public.
    c. both of the above
    d. none of the above

17. A _____ is a collection of predefined, built-in classes that you can use when writing programs.

    a. vault
    b. black box
    c. library
    d. store

18. An environment in which you can develop GUI programs by dragging components to their desired positions is a(n) _____ .

    a. visual development environment
    b. integrated compiler
    c. text-based editor
    d. GUI formatter

19. In object-oriented programs, errors are known as _____ .

    a. faults
    b. gaffes
    c. exceptions
    d. omissions

20. The general principle of exception handling in object-oriented programming is that a method that uses data should _____ .

    a. be able to detect errors, but not be required to handle them
    b. be able to handle errors, but not detect them
    c. be able to handle and detect errors
    d. not be able to detect or handle errors



Programming Exercises

1. Complete the following tasks:

    a. Design a class named `Circle` with fields named `radius`, `area`, and `diameter`. Include a constructor that sets the radius to 1. Include get methods for each field, but include a set method only for the radius. When the radius is set, do not allow it to be zero or a negative number. When the radius is set, calculate the diameter (twice the radius) and the area (the radius squared times pi, which is approximately 3.14). Create the class diagram and write the pseudocode that defines the class.

    b. Design an application that declares two `Circles`. Set the radius of one manually, but allow the other to use the default value supplied by the constructor. Then, display each `Circle`'s values.

2. Complete the following tasks:

    a. Design a class named `PhoneCall` with four fields: two strings that hold the 10-digit phone numbers that originated and received the call, and two numeric fields that hold the length of the call in minutes and the cost of the call. Include a constructor that sets the phone numbers to *X*s and the numeric fields to 0. Include get and set methods for the phone number and call length fields, but do not include a set method for the cost field. When the

call length is set, calculate the cost of the call at three cents per minute for the first 10 minutes, and two cents per subsequent minute. Create the class diagram and write the pseudocode that defines the class.

b. Design an application that declares three `PhoneCall`s. Set the length of one `PhoneCall` to 10 minutes, another to 11 minutes, and allow the third object to use the default value supplied by the constructor. Then, display each `PhoneCall`'s values.

c. Create a child class named `InternationalPhoneCall` that inherits from `PhoneCall`. Override the parent class method that sets the call length to calculate the cost of the call at 40 cents per minute.

d. Create the logic for an application that instantiates a `PhoneCall` object and an `InternationalPhoneCall` object and displays the costs for each.

3. Complete the following tasks:

a. Design a class named `ItemForSale` that holds data about items placed for sale on Carlos's List, a classified advertising website. Fields include an ad number, item description, asking price, and phone number. Include get and set methods for each field. Include a static method that displays the website's motto ("Sell Stuff Locally!"). Include two overloaded constructors as follows:

- A default constructor that sets the ad number to 101, the asking price to $1, and the item description and phone number both to *XXX*
- A constructor that allows you to pass values for all four fields

Create the class diagram and write the pseudocode that defines the class.

b. Design an application that declares two `ItemForSale` objects using a different constructor version with each object. Display each `ItemForSale`'s values and then display the motto.

4. Complete the following tasks:

a. Create a class named `Meal` that includes a string variable for the meal's description, an array of strings that holds up to five of the `Meal`'s components (for example, "roasted chicken", "mashed potatoes", and "green beans"), and a numeric variable that holds the calorie count. Include a method that prompts the user for a value for each field. Also create two overloaded methods named `display()`. The first method takes no parameters and displays the `Meal` details. The second takes a numeric parameter that indicates how many of the `Meal`'s components to display, or an error message if the parameter value is less than 0 or more than 5.

b. Create an application that declares two `Meal` objects, sets their values, and demonstrates how both method versions can be called.

5. Complete the following tasks:

   a. Create a class named `Apartment` that includes an apartment number, number of bedrooms, number of baths, and monthly rent. Include two overloaded constructors. The default constructor sets each field to 0. The non-default constructor accepts four parameters—one for each field. Include two overloaded `display()` methods. The parameterless version displays all the `Apartment` details. The second version accepts a value that represents a maximum rent and displays the `Apartment` details only if the `Apartment`'s rent is no greater than the parameter.

   b. Create an application that instantiates several `Apartment` objects and demonstrates all the methods.

6. Complete the following tasks:

   a. Design a class named `Book` that holds a stock number, author, title, price, and number of pages. Include methods to set and get the values for each data field. Also include a `displayInfo()` method that displays each of the `Book`'s data fields with explanations.

   b. Design a class named `TextBook` that is a child class of `Book`. Include a new data field for the grade level of the book. Override the `Book` class `displayInfo()` method to accommodate the new grade-level field.

   c. Design an application that instantiates an object of each type and demonstrates all the methods.

7. Complete the following tasks:

   a. Design a class named `Player` that holds a player number and name for a sports team participant. Include methods to set the values for each data field and output the values for each data field.

   b. Design two classes named `BaseballPlayer` and `BasketballPlayer` that are child classes of `Player`. Include a new data field in each class for the player's position. Include an additional field in the `BaseballPlayer` class for batting average. Include a new field in the `BasketballPlayer` class for free-throw percentage. Override the `Player` class methods that set and output the data so that you accommodate the new fields.

   c. Design an application that instantiates an object of each type and demonstrates all the methods.

8. Complete the following tasks:

   a. Create a class for a cell phone service named `Message` that includes a field for the price of the message. Create get and set methods for the field.

   b. Derive three subclasses—`VoiceMessage`, `TextMessage`, and `PictureMessage`. The `VoiceMessage` class includes a numeric field to hold the length of the message in minutes and a get and set method for the field. When a `VoiceMessage`'s length value is set, the price is calculated at 4 cents per minute. The `TextMessage` class includes a numeric field to hold the length of the message in words and a get and set method for the field. When a `TextMessage`'s length value is set, the price is calculated at 2 cents per word. The `PictureMessage` class includes a numeric field that holds the size of the picture in kilobytes and get and set methods for the field. When a `PictureMessage`'s length value is set, the price is calculated at 1 cent per kilobyte.

   c. Design a program that instantiates one object of each of the three classes, and demonstrate using all the methods defined for each class.

9. Complete the following tasks:

   a. Create a class named `Order` that performs order processing of a single item. The class has four fields: customer name, customer number, quantity ordered, and unit price. Include set and get methods for each field. The set methods prompt the user for values for each field. This class also needs a `computePrice()` method to compute the total price (quantity multiplied by unit price) and a method to display the field values.

   b. Create a subclass named `ShippedOrder` that overrides `computePrice()` by adding a shipping and handling charge of $4.00.

   c. Create the logic for an application that instantiates an object of each of these two classes. Prompt the user for data for the `Order` object and display the results; then prompt the user for data for the `ShippedOrder` object and display the results.

   d. Create the logic for an application that continuously prompts for order information until the user enters *ZZZ* for the customer name or 10 orders have been taken, whichever comes first. Ask the user whether each order will be shipped, and create an `Order` or a `ShippedOrder` accordingly. Store each order in an array element. When the user finishes entering data, display all the order information taken as well as the total price that was computed for each order.

10. Complete the following tasks:

    a. Design a method that calculates the cost of boarding a horse at Delmar Stables. The method accepts a code for the type of boarding, *S* for *self-care*, which provides just shelter and costs $200 per month, or *F* for *full service*, which provides all care, including feeding and grooming, and costs $700 per month. The method should throw an exception if the boarding code is invalid.

b. Write a method that calls the method designed in Exercise 10a. If the method throws an exception, force the price of boarding to 0.

c. Write a method that calls the method designed in Exercise 10a. If the method throws an exception, require the user to reenter the code.

d. Write a method that calls the method designed in Exercise 10a. If the method throws an exception, force the code to *F* and the price to $700.

11. Design a method that calculates the monthly cost to rent a roadside billboard. Variables include the size of the billboard (*S, M,* or *L* for small, medium, or large) and its location (*H*, *M*, or *L* for high-, medium-, or low-traffic areas). The method should throw an exception if the size or location code is invalid. The monthly rental cost is shown in Table 11-1.

	High Traffic	Medium Traffic	Low Traffic
Small size	900	500	200
Medium size	1600	1200	600
Large size	2000	1500	800

Table 11-1 Monthly billboard rental rates



Performing Maintenance

1. A file named MAINTENANCE11-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 11 include DEBUG11-01.txt, DEBUG11-02.txt, and DEBUG11-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.
2. Your downloadable files for Chapter 11 include a file named DEBUG11-04.jpg that contains a class diagram with syntax and/or logical errors. Examine the class diagram, and then find and correct all the bugs.

Game Zone

1. a. Computer games often contain different characters or creatures. For example, you might design a game in which alien beings possess specific characteristics such as color, number of eyes, or number of lives. Create an `Alien` class. Include at least three data fields of your choice. Include a constructor that requires a value for each data field and a method named `toString()` that returns a string containing a complete description of the `Alien`.

   b. Create two classes—`Martian` and `Jupiterian`—that descend from `Alien`. Supply each with a constructor that sets the `Alien` data fields with values you choose. For example, you can decide that a `Martian` has four eyes but a `Jupiterian` has only two.

   c. Create an application that instantiates one `Martian` and one `Jupiterian`. Call the `toString()` method with each object and display the results.

2. In Chapter 2, you learned that in many programming languages you can generate a random number between 1 and a limiting value named `LIMIT` by using a statement similar to `randomNumber = random(LIMIT)`. In Chapters 4 and 5, you created and fine-tuned the logic for a guessing game in which the application generates a random number and the player tries to guess it. As written, the game should work as long as the player enters numeric guesses. However, if the player enters a letter or other nonnumeric character, the game throws an automatically generated exception. Improve the game by handling any exception so that the user is informed of the error and allowed to enter data again.

3. a. In Chapter 10, you developed a `Card` class that contains a string data field to hold a suit and a numeric data field for a value from 1 to 13. Now extend the class to create a class called `BlackjackCard`. In the game of Blackjack, each card has a point value as well as a face value. These two values match for cards with values of 2 through 10, and the point value is 10 for jacks, queens, and kings (face values 11 through 13). For a simplified version of the game, assume that the value of the ace is 11. (In the official version of Blackjack, the player chooses whether each ace is worth 1 or 11 points.)

   b. Randomly assign values to 10 `BlackjackCard` objects, then design an application that plays a modified version of Blackjack. The objective is to accumulate cards whose total value equals 21, or whose value is closer to 21 than the opponent's total value without exceeding 21. Deal five `BlackjackCards` each to the player and the computer. Make sure that each `BlackjackCard` is unique. For example, a deck cannot contain more than one queen of spades.

Determine the winner as follows:

- If the player's first two, first three, first four, or all five cards have a total value of exactly 21, the player wins, even if the computer also achieves a total of 21.
- If the player's first two cards do not total exactly 21, sum as many as needed to achieve the highest possible total that does not exceed 21. For example, suppose that the player's five cards are valued as follows: 10, 4, 5, 9, 2. In that case, the player's total for the first three cards is 19; counting the fourth card would cause the total to exceed 21.
- After you have determined the player's total, sum the computer's cards in sequence. For example, suppose that the computer's cards are 10, 10, 5, 6, 7. The first two cards total 20; you would not use the third card because it would cause the total to exceed 21.
- The winner has the highest total among the cards used. For example, if the player's total using the first three cards is 19 and the computer's total using the first two cards is 20, the computer wins.

Display a message that indicates whether the game ended in a tie, the computer won, or the player won.

CHAPTER 12

Event-Driven GUI Programming, Multithreading, and Animation

Upon completion of this chapter, you will be able to:

- Understand the principles of event-driven programming
- Describe user-initiated actions and GUI components
- Appreciate design issues in graphical user interfaces
- Develop an event-driven application
- Describe threads and multithreading
- Understand how to create animation

Understanding Event-Driven Programming

From the 1950s, when businesses began to use computers, through the 1980s, almost all interactive dialogues between people and computers took place at the command prompt. (Programmers also call the command prompt the *command line*, and users of the Disk Operating System often call the command line the **DOS prompt**.) In Chapter 1, you learned that the command line is used to type entries to communicate with the computer's **operating system**—the software that runs a computer and manages its resources. In the early days of computing, interacting with an operating system was difficult because users had to know the exact syntax to use when typing commands, and they had to spell and type those commands accurately. Figure 12-1 shows the command prompt in the Windows 10 operating system.

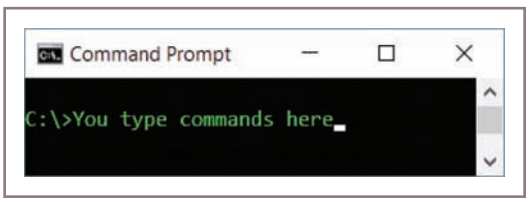


Figure 12-1 Command prompt screen



You can access the command prompt in a variety of ways, depending on the operating system you are using. In most systems you can use a menu option or icon to get to the command prompt.

Fortunately for today's computer users, operating system software allows them to use their fingers, a mouse, or another pointing device to select screen controls, such as buttons and scroll bars or pictures (also called **icons**). As you learned in Chapter 1, this type of environment is a graphical user interface, or GUI. Computer users can expect to see a standard interface in GUI programs. Rather than memorizing difficult commands that must be typed at a command line, GUI users can select options from menus and click buttons to make their preferences known to a program. The icons used on buttons and other components are best understood when they follow convention—for example, an "X" button is expected to mean "Close." Sometimes, users can select icons that look like their real-world counterparts and get the expected results. For example, users can select an icon that looks like a pencil when they want to write a memo, or they can drag an icon shaped like a folder to a recycling bin icon to delete the folder and its contents. Figure 12-2 shows a Windows program named Paint in which icons representing paintbrushes and other objects appear on clickable buttons.

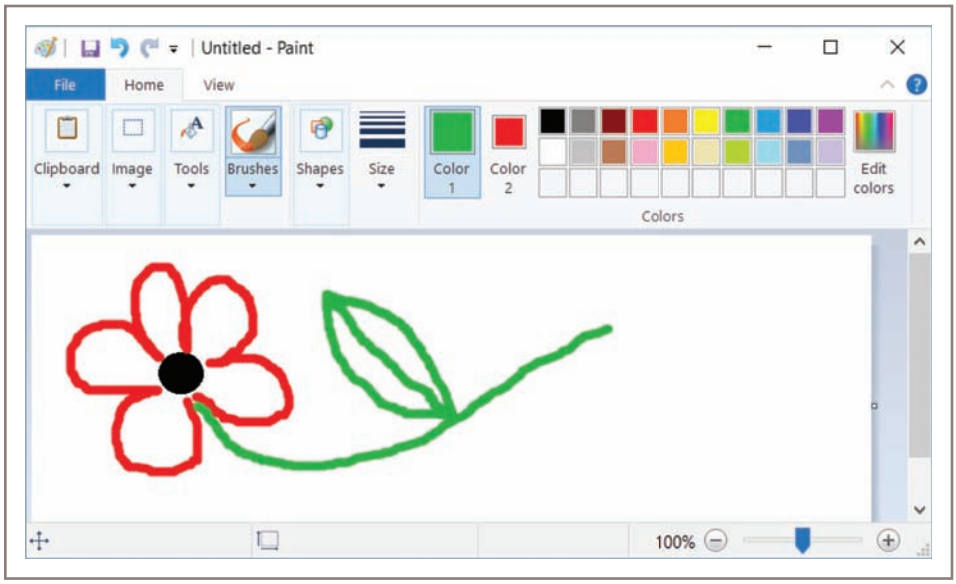


Figure 12-2 A GUI application that contains buttons and icons

Performing an operation on a control or using an icon (for example, clicking or dragging it) causes an **event**—an occurrence that generates a message sent to an object. GUI programs frequently are called **event-driven** or **event-based** because actions occur in response to user-initiated events such as tapping a screen or clicking a mouse button. When you program with event-driven languages, the emphasis is on objects that users can manipulate, such as text boxes, buttons, and menus, and on events that users can initiate with those objects, such as typing, pointing, clicking, or double-clicking. The programmer writes instructions within modules that execute in response to each type of event.

Throughout this book, the program logic you have developed has been procedural, and not event-driven; each step occurs in the order the programmer determines. In a procedural application, if you write statements that display a prompt and accept a user's response, the processing stops after the prompt is displayed, and the program goes no further until input is received. When you write a procedural program, you have complete control over the order in which all the statements will execute. If you call `moduleA()` before calling `moduleB()`, `moduleB()` does not execute until `moduleA()` is finished.

In contrast, with most event-driven programs, the user might initiate any number of events in any order. For example, when you use an event-driven word-processing program, you have dozens of choices at your disposal at any moment. You can type words, select text with the mouse, click a button to change the text style to bold or italics, choose a menu item such as *Save* or *Print*, and so on. With each word-processing document you create, the program must be ready to respond to any event you initiate. The programmers who created

the word processor are not guaranteed that you will select *Bold* before you select *Italics*, or that you will select *Save* before you select *Quit*, so they must write programs that are more flexible than their procedural counterparts.

Within an event-driven program, a component from which an event is generated is the **source of the event**. A button that users can click to cause an action is an example of a source; a text box in which users enter typed characters is another source. An object that is "interested in" an event to which you want it to respond is a **listener**. It "listens for" events so it knows when to respond. Not all objects can receive all events—you probably have used programs in which clicking many areas of the screen has no effect. If you want an object such as a button to be a listener for an event such as a mouse click, you must write two types of appropriate program statements. You write the statements that define the object as a listener and also write the statements that constitute the event.

Although event-driven programming is newer than procedural programming, the instructions that programmers write to respond to events are still simply sequences, selections, and loops. Event-driven programs still have methods that declare variables, use arrays, and contain all the attributes of their procedural-program ancestors. The user's screen in an event-driven program might contain buttons or check boxes with labels such as *Sort Records*, *Merge Files*, or *Total Transactions*, but each of these processes represents a method that uses the same logic you have learned throughout this book for programs that did not have a graphical interface. In object-oriented languages, the procedural modules that depend on user-initiated events are often called **scripts**. Writing event-driven programs involves thinking of possible events, writing scripts to execute actions, and writing the statements that link user-initiated events to the scripts.

TWO TRUTHS & A LIE

Understanding Event-Driven Programming

1. GUI programs are called event-driven or event-based because actions occur in response to user-initiated events such as clicking a mouse button.
2. With event-driven programs, the user might initiate any number of events in any order.
3. Within an event-driven program, a component from which an event is generated, such as a button, is a listener. An object that is "interested in" an event is the source of the event.

The false statement is #3. Within an event-driven program, a component from which an event is generated is the source of the event, and an object that is "interested in" an event to which you want it to respond is a listener.

User-Initiated Actions and GUI Components

To understand GUI programming, you need to have a clear picture of the possible events a user can initiate. A partial list is shown in Table 12-1. Most languages allow you to distinguish between many additional events. For example, you might be able to initiate three different events when (1) a mouse key is pressed, (2) during the time it is held down, and (3) when it is released.

Event	Description of User's Action
Tap	Tapping on the screen
Swipe	Quickly dragging a finger across the screen
Zoom	Dragging across the screen with two fingers slightly apart to zoom out, or closer together to zoom in
Key press	Pressing a key on the keyboard
Mouse point or mouse over	Placing the mouse pointer over an area on the screen
Mouse click or left mouse click	Pressing the left mouse button
Right mouse click	Pressing the right mouse button
Mouse double-click	Pressing the left mouse button twice in rapid sequence
Mouse drag	Holding down the left mouse button while moving the mouse over the desk surface

Table 12-1 Common user-initiated events

You also need to be able to picture common GUI components. Table 12-2 describes some common GUI components, and Figure 12-3 shows how they look on a screen.

Component	Description
Label	A rectangular area that displays text
Text box	A rectangular area into which the user can type text
Check box	A label placed beside a small square; you can click the square to display or remove a check mark, which selects or deselects an option
Option buttons	A group of controls that are similar to check boxes. When the controls are square, users typically can select any number of them; they are called a *check box group*. When the controls are round, they are often mutually exclusive and are called *radio buttons*.
List box	When the user clicks a list box, a menu of items appears. Depending on the options the programmer sets, you might be able to make only one selection, or you might be able to make multiple selections.
Button	A rectangular control you can click; when you do, its appearance usually changes to look pressed

Table 12-2 Common GUI components

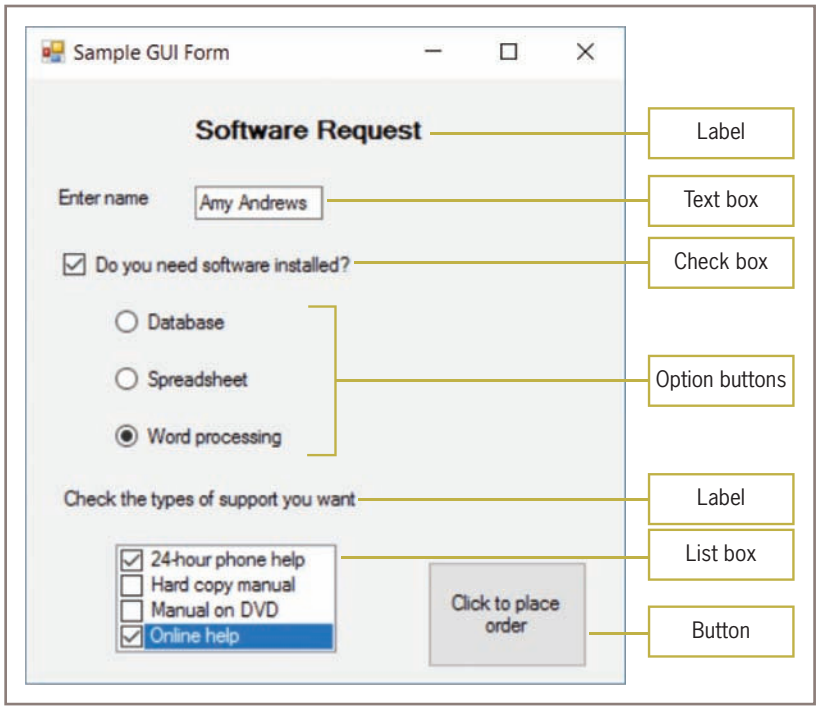


Figure 12-3 Common GUI components

When you write a program that uses GUI components, you do not create them from scratch. Instead, you call prewritten methods that draw the GUI components on the screen for you. The components themselves are created using existing classes complete with names, attributes, and methods. In some programming languages, you can work in a text environment and write statements that instantiate GUI objects. In other languages, you can work in a graphical environment, drag GUI objects onto your screen from a toolbox, and arrange them appropriately for your application. Some languages offer both options. Either way, you do not think about the details of creating the components. Instead, you concentrate on the actions that should occur when a user initiates an event from one of the components. Thus, GUI components are excellent examples of the best principles of object-oriented programming—they represent objects with attributes and methods that operate like black boxes, making them easy for you to use.

When you use existing GUI components, you instantiate objects, each of which belongs to a prewritten class. For example, you might use a `Button` object when you want the user to click a button to make a selection. Depending on the programming language, the `Button` class might contain attributes or properties such as the text on the `Button` and its position on the screen. The class might also contain methods such as `setText()` and `setPosition()`. For example, Figure 12-4 shows how a built-in `Button` class might be written.

```
class Button
   Declarations
      private string text
      private num x_position
      private num y_position

   public void setText(string messageOnButton)
      text = messageOnButton
   return

   public void setPosition(num x, num y)
      x_position = x
      y_position = y
   return
endClass
```

Figure 12-4 `Button` class



The `x_position` and `y_position` of the `Button` object in Figure 12-4 refer to horizontal and vertical coordinates where the `Button` appears on an object, such as a window that appears on the screen during program execution. A **pixel** is one of the tiny dots of light that form a grid on your screen. The term *pixel* derives from combining the first syllables of *picture* and *element*. You will use x- and y-positions again when you learn about animation later in this chapter.



Watch the video *GUI Components*.

The `Button` class shown in Figure 12-4 is an abbreviated version so you can easily see its similarity to a class such as `Employee`, which you read about in Chapter 11. The figure shows three fields and two set methods. A complete, working `Button` class in most programming languages would contain many more fields and methods. For example, a full-blown class also might contain get methods for the text and position, and other fields and methods to manipulate a `Button`'s font, color, size, and so on.

To create a `Button` object in a client program, you would write a statement similar to the following:

```
Button myProgramButton
```

In this statement, Button is the data type and `myProgramButton` is the identifier for the object created. To use a `Button`'s methods, you would write statements such as the following:

```
myProgramButton.setText("Click here")
myProgramButton.setPosition(10, 30)
```

Different GUI classes support different attributes and methods. For example, a `CheckBox` class might contain a method named `getCheckedStatus()` that returns true or false, indicating whether the `CheckBox` object has been checked. A `Button`, however, would have no need for such a method.



An important advantage of using GUI data-entry objects is that you often can control what users enter by limiting their options. When you provide a finite set of buttons to click or a limited number of menu items, the user cannot make unexpected, illegal, or bizarre choices. For example, if you provide only two buttons so the user must click Yes or No, you can eliminate writing code to handle invalid entries.

TWO TRUTHS & A LIE

User-Initiated Actions and GUI Components

1. In a GUI program, a key press is a common user-initiated event and a check box is a typical GUI component.
2. When you program in a language that supports event-driven logic, you call prewritten methods that draw GUI components on the screen for you.
3. An advantage of using GUI objects is that each class you use to create the objects supports identical methods and attributes.

The false statement is #3. Different GUI classes support different attributes and methods.

Designing Graphical User Interfaces

You should consider several general design principles when creating a program that will use a GUI:

- The interface should be natural and predictable.
- The interface should be attractive, easy to read, and nondistracting.
- To some extent, it's helpful if the user can customize your applications.
- The program should be forgiving.
- The GUI is only a means to an end.

The Interface Should Be Natural and Predictable

The GUI program interface should represent objects like their real-world counterparts. In other words, it makes sense to use an icon that looks like a recycling bin to let a user drag files or other components to the bin and delete them. Using a recycling bin icon is "natural" in that people use one in real life when they want to discard actual items. Using a recycling bin for discarded items is also predictable, because users are already familiar with the icon in other programs. Some icons might be natural, but if they are not predictable as well, then they are not as effective. An icon that depicts a recycling truck might seem just as natural as one that depicts a bin, but because other programs do not use such imagery, it is not as predictable.

GUIs should also be predictable in their layout. For example, when a user must enter personal information in text boxes, the street address is expected to come before the city and state. If you arrange an input screen so that the user enters the city and state first, users will generate a lot of errors. Also, a menu bar appears at the top of the screen in most GUI programs, and the first menu item in many applications is *File*. If you design a program interface in which the menu runs vertically down the right side of the screen, or in which *File* is the last menu option instead of the first, you will confuse users. Either they will make mistakes when using your program or they might give up using it entirely. It doesn't matter if you can prove that your layout plan is more efficient than the standard one—if you do not use a predictable layout, your program likely will be rejected in the marketplace.



Many studies have proven that the Dvorak keyboard layout is more efficient for typists than the QWERTY keyboard layout that most of us use. The QWERTY keyboard layout gets its name from the first six letter keys in the top row. With the Dvorak layout, which is named for its inventor, the most frequently used keys are in the home row, allowing typists to complete many more keystrokes per minute. However, the Dvorak keyboard has not caught on because it is not predictable to users who know the QWERTY keyboard.



Stovetops often have an unnatural interface, making unfamiliar stoves more difficult for you to use. Most stovetops have four burners arranged in two rows, but the knobs that control the burners frequently are placed in a single horizontal row. Because there is not a natural correlation between the placement of a burner and its control, you are likely to select the wrong knob when adjusting the burner's flame or heating element.

The Interface Should Be Attractive, Easy to Read, and Nondistracting

If your interface is attractive, people are more likely to use it. If it is easy to read, users are less likely to make mistakes. When it comes to GUI design, fancy fonts and weird color combinations are the signs of amateur designers. In addition, you should make sure that unavailable screen options are either dimmed (also called *grayed*) or removed, so the user does not waste time clicking components that aren't functional. An excellent way to learn about good GUI design is to pay attention to the design features used in popular applications and in websites you visit. Notice that the designs you like to use feel more "natural."

Screen designs should not be distracting. When a screen has too many components, users can't find what they're looking for. When a component is no longer needed, it should be removed from the interface. GUI programmers sometimes refer to screen space as *real estate*. Just as a plot of land becomes unattractive when it supports no open space, your screen becomes unattractive when you fill the limited space with too many components.

You also want to avoid distracting users with overly creative design elements. When users click a button to open a file, they might be amused the first time a filename dances across the screen or the speakers play a tune. However, after one or two experiences with your creative additions, users find that intruding design elements hamper the actual work of the program. Also, creative embellishments might consume extensive memory and CPU time, slowing an application's performance.

To Some Extent, It's Helpful If the User Can Customize Your Applications

All users work in their own way. If you are designing an application that will use numerous menus and toolbars, it's helpful if users can position components in the order that's easiest for them. Users appreciate being able to change features like color schemes. Allowing a user to change the background color in your application might seem frivolous to you, but to users who are color blind or visually impaired, it might make the difference in whether they use your application at all. Making programs easier to use for people with physical limitations is known as enhancing **accessibility**.

Don't forget that many programs are used internationally. If you can allow the user to work with a choice of languages, you might be able to market your program more successfully in other countries. If you can allow the user to convert prices to multiple currencies, you might be able to make sales in more markets.

The Program Should Be Forgiving

Perhaps you have had the inconvenience of accessing a voice mail system in which you selected several sequential options, only to find yourself at a dead end with no recourse but to hang up and redial the number. Good program design avoids similar problems. You should always provide an escape route to accommodate users who make bad choices or change their minds. By providing a Back button or a working Escape key, you provide more functionality to your users. It also can be helpful to include an option for the user to revert to the default settings after making changes. Some users might be afraid to alter an application's features if they are not sure they can easily return to the original settings.

Users also appreciate being able to perform tasks in a variety of ways. For example, you might allow a user to select a word on a screen by highlighting it using a mouse, by touching it on the screen, or by holding down the Ctrl and Shift keys while pressing the right arrow key. A particular technique might be easier for people with disabilities, and it might be the only one available after the mouse batteries fail or the user accidentally disables the keyboard by spilling coffee on it.

The GUI Is Only a Means to an End

The most important principle of GUI design is to remember that a GUI is only an interface. Using a mouse to click items and drag them around is not the point of any business programs except those that train people how to use a mouse. Instead, the point of a graphical interface is to help people be more productive. To that end, the design should help the user see what options are available, allow the use of components in the ordinary way, and not force the user to concentrate on how to interact with your application. The real work of a GUI program—making decisions, performing calculations, sorting records, and so on—is done after the user clicks a button or makes a list box selection.

TWO TRUTHS & A LIE

Designing Graphical User Interfaces

1. To keep the user's attention, a well-designed GUI interface should contain unique and creative controls.
2. To be most useful, a GUI interface should be attractive, easy to read, and nondistracting.
3. To avoid frustrating users, a well-designed program should be forgiving.

The false statement is #1. A GUI interface should be natural and predictable.

Developing an Event-Driven Application

In Chapter 1, you first learned the steps to developing a procedural computer program:

1. Understanding the problem
2. Planning the logic
3. Coding the program
4. Translating the program into machine language
5. Testing the program
6. Putting the program into production
7. Maintaining the program

When you develop an event-driven application, you expand on Step 2 (planning the logic) and you might include four new substeps as follows:

2a. Creating wireframes

2b. Creating storyboards

2c. Defining the objects

2d. Defining the connections between the screens the user will see

For example, suppose that you want to create a simple, interactive program that determines premiums for prospective insurance customers. A graphical interface will allow users to select a policy type—health or auto. Next, users answer pertinent questions about their age, driving record, and whether they smoke. Although most insurance premiums would be based on more characteristics than these, assume that policy rates are determined using the factors shown in Table 12-3. The final output of the program is a second screen that shows the semiannual premium amount for the chosen policy.

Health Policy Premiums	Auto Policy Premiums
Base rate: $500	Base rate: $750
Add $100 if over age 50	Add $400 if more than 2 tickets
Add $250 if smoker	Subtract $200 if over age 50

Table 12-3 Insurance premiums based on customer characteristics

Creating Wireframes

A **wireframe** is a picture or sketch of a screen the user will see when running a program. A wireframe, also called a **page schematic** or **screen blueprint**, is a visual guide that helps developers and their user clients decide on the basic features of an interactive program or website. Wireframes can be pencil sketches or they can be produced by software applications. Typically, they do not contain graphics or show the final font styles that will be used; instead, they focus on the functionality of an application. Figure 12-5 shows two wireframes for the program that determines insurance premiums. They represent the introductory screen, at which the user selects a premium type and answers questions, and the final screen, which displays the semiannual premium.

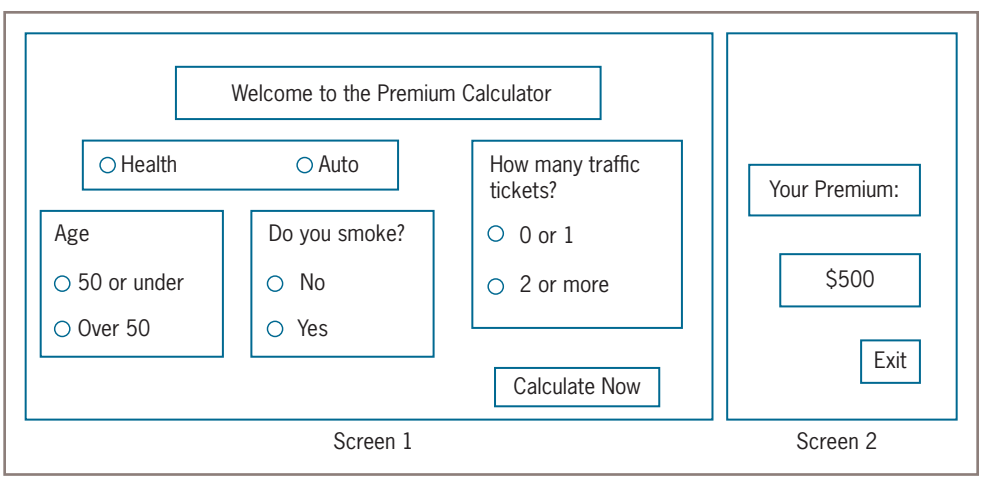


Figure 12-5 Storyboard for the insurance program, which is composed of two wireframes

Creating Storyboards

A **storyboard** contains a series of wireframes that represent a user's experience with proposed software. Filmmakers have long used storyboards to illustrate key moments in the plots they are developing; similarly, GUI storyboards represent "snapshot" views of the screens the user will encounter during the run of a program. If the user could view up to 10 screens during the insurance premium program, then you would draw 10 storyboard cells, or wireframes. Sometimes, developers will enhance wireframes with color

and graphics when incorporating them into a storyboard. The two wireframes shown in Figure 12-5 represent the insurance application's storyboard.

Defining the Storyboard Objects in an Object Dictionary

An event-driven program can contain dozens or even hundreds of objects. To keep track of them, programmers often use an object dictionary. An **object dictionary** is a list of the objects used in a program, the screens where the objects are used, and any associated code (script).

Figure 12-6 shows an object dictionary for the insurance premium program. The type and name of each object to be placed on a screen are listed in the columns on the left. The third column shows the screen number on which the object appears. The fourth column names any variables that are affected by an action on the object. The last column indicates whether any code or script is associated with the object. For example, the label named `welcomeLabel` appears on the first screen. It has no associated actions—nothing a user does to it executes any methods or changes any variables; it is just a label. When a user clicks the `calcButton`, however, a method named `calcRoutine()` is called. This method calculates the semiannual premium amount and stores it in the `premiumAmount` variable. Depending on the programming language, you might need to name `calcRoutine()` something similar to `calcButton.click()`. In languages that use this format, a standard method named `click()` holds the statements that execute when the user clicks the `calcButton`.

Object Type	Name	Screen Number	Variables Affected	Script?
Label	welcomeLabel	1	none	none
RadioButton	healthRadioButton	1	premiumAmount	none
RadioButton	autoRadioButton	1	premiumAmount	none
Label	ageLabel	1	none	none
RadioButton	lowAgeRadioButton	1	premiumAmount	none
RadioButton	highAgeRadioButton	1	premiumAmount	none
Label	smokeLabel	1	none	none
RadioButton	smokeNoRadioButton	1	premiumAmount	none
RadioButton	smokeYesRadioButton	1	premiumAmount	none
Label	ticketsLabel	1	none	none
RadioButton	lowTicketsRadioButton	1	premiumAmount	none
RadioButton	highTicketsRadioButton	1	premiumAmount	none
Button	calcButton	1	premiumAmount	calcRoutine()
Label	premiumLabel	2	none	none
Label	premAmtLabel	2	none	none
Button	exitButton	2	none	exitRoutine()

Figure 12-6 Object dictionary for insurance premium program

Defining Connections Between the User Screens

The insurance premium program is small, but with larger programs you might need to draw connections between the screens to show how they interact. Figure 12-7 shows an interactivity diagram for the screens used in the insurance premium program. An **interactivity diagram** shows the relationship between screens in an interactive GUI program. Figure 12-7 shows that the first screen calls the second screen, and the program ends.

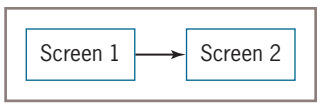


Figure 12-7 Interactivity diagram for insurance premium program

Figure 12-8 shows how a diagram might look for a more complicated program in which the user has several options available at Screens 1, 2, and 3. Notice how each of these screens can lead to different screens, depending on the options the user selects.

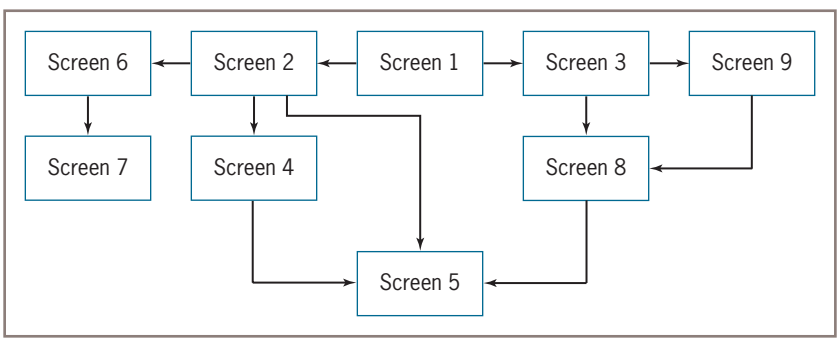


Figure 12-8 Interactivity diagram for a complicated program

Planning the Logic

In an event-driven program, you design the screens, define the objects, and define how the screens will connect. Then you can start to plan the client program. For example, following the storyboard plan for the insurance program (see Figure 12-5), you need to create the first screen, which contains four labels, four sets of radio buttons, and a button. Figure 12-9 shows the pseudocode that creates these components.

When you use an integrated development environment to create applications, you can drag components like those in Figure 12-9 onto a screen without explicitly writing all the statements shown in the pseudocode. In that case, the coding statements will be generated for you. It's beneficial to understand these statements so that you can more easily modify and debug your programs.

```
Declarations
   Label welcomeLabel
   RadioButton healthRadioButton
   RadioButton autoRadioButton
   Label ageLabel
   RadioButton lowAgeRadioButton
   RadioButton highAgeRadioButton
   Label smokeLabel
   RadioButton smokeNoRadioButton
   RadioButton smokeYesRadioButton
   Label ticketsLabel
   RadioButton lowTicketsRadioButton
   RadioButton highTicketsRadioButton
   Button calcButton

welcomeLabel.setText("Welcome to the Premium Calculator")
welcomeLabel.setPosition(30, 10)

healthRadioButton.setText("Health")
healthRadioButton.setPosition(15, 40)

autoRadioButton.setText("Auto")
autoRadioButton.setPosition(50, 40)

ageLabel.setText("Age")
ageLabel.setPosition(5, 60)

lowAgeRadioButton.setText("50 or under")
lowAgeRadioButton.setPosition(5, 70)

highAgeRadioButton.setText("Over 50")
highAgeRadioButton.setPosition(5, 80)

smokeLabel.setText("Do you smoke?")
smokeLabel.setPosition(40, 60)

smokeNoRadioButton.setText("No")
smokeNoRadioButton.setPosition(40, 70)

smokeYesRadioButton.setText("Yes")
smokeYesRadioButton.setPosition(40, 80)

ticketsLabel.setText("How many traffic tickets?")
ticketsLabel.setPosition(60, 50)

lowTicketsRadioButton.setText("0 or 1")
lowTicketsRadioButton.setPosition(60, 70)

highTicketsRadioButton.setText("2 or more")
highTicketsRadioButton.setPosition(60, 90)

calcButton.setText("Calculate Now")
calcButton.setPosition(60, 100)
calcButton.registerListener(calcRoutine())
```

Figure 12-9 Component definitions for first screen of insurance program

In Figure 12-9, the following statement specifies that the method `calcRoutine()` executes when a user clicks the `calcButton`:

```
calcButton.registerListener(calcRoutine())
```

The syntax of this statement varies among programming languages. With most object-oriented programming (OOP) languages, you must **register components**, or sign them up so that they can react to events initiated by other components. The details vary among languages, but the basic process is to write a statement that links the appropriate method (such as the `calcRoutine()` or `exitRoutine()` method) with an event such as a user's button click. In many development environments, the statement that registers a component to react to a user-initiated event is written for you automatically when you click components while designing your screen.



In reality, you might generate more code than what is shown in Figure 12-9 when you create the insurance program components. For example, each component might require a color and font. You also might want to initialize some components with default values to indicate they are selected. For example, you might want one radio button in a group to be selected already, which requires the user to click a different option only if he does not want the default selection.

You also need to create the component that holds all the GUI elements in Figure 12-9. Depending on the programming language, you might use a class with a name such as `Screen`, `Form`, or `Window`. Each of these is a **container**, or a class of objects whose main purpose is to hold other elements. The container class contains methods that allow you to set physical properties such as height and width, as well as methods that allow you to add the appropriate components to a container. Figure 12-10 shows how you would define a `Screen` object, set its size, and add the necessary components.

```
Declarations
   Screen screen1
screen1.setSize(150, 150)
screen1.add(welcomeLabel)
screen1.add(healthRadioButton)
screen1.add(autoRadioButton)
screen1.add(ageLabel)
screen1.add(lowAgeRadioButton)
screen1.add(highAgeRadioButton)
screen1.add(smokeLabel)
screen1.add(smokeNoRadioButton)
screen1.add(smokeYesRadioButton)
screen1.add(ticketsLabel)
screen1.add(lowTicketsRadioButton)
screen1.add(highTicketsRadioButton)
screen1.add(calcButton)
```

Figure 12-10 Statements that create `screen1`

Similarly, Figure 12-11 shows how you can create and define the components for the second screen in the insurance program and how to add the components to the container. Notice the label that holds the user's insurance premium is not filled with text, because the amount is not known until the user makes all the selections on the first screen.


```
Declarations
   Screen screen2
   Label premiumLabel
   Label premAmtLabel
   Button exitButton

screen2.setSize(100, 100)

premiumLabel.setText("Your Premium:")
premiumLabel.setPosition(5, 30)

premAmtLabel.setPosition(20, 50)

exitButton.setText("Exit")
exitButton.setPosition(60, 80)
exitButton.registerListener(exitRoutine())

screen2.add(premiumLabel)
screen2.add(premAmtLabel)
screen2.add(exitButton)
```

Figure 12-11 Statements that define and create `screen2` and its components

After the GUI components are designed and arranged, you can plan the logic for each of the methods or scripts that the program will use. For example, given the program requirements shown earlier in Table 12-3, you can write the pseudocode for the `calcRoutine()` method of the insurance premium program, as shown in Figure 12-12. The `calcRoutine()` method does not execute until the user clicks the `calcButton`. At that point, the user's choices are sent to the method and used to calculate the premium amount.

The pseudocode in Figure 12-12 should look very familiar to you—it declares numeric constants and a variable and uses decision-making logic you have used since the early chapters of this book. After the premium is calculated based on the user's choices, it is

placed in the label that appears on the second screen. The basic control structures of sequence, selection, and loop will continue to serve you well, whether you are programming in a procedural or event-driven environment.

The last two statements in the `calcRoutine()` method indicate that after the insurance premium is calculated and placed in its label, the first screen is removed and the second screen is displayed. Screen removal and display are accomplished differently in different languages; this example assumes that the appropriate methods are named `remove()` and `display()`.

```
public void calcRoutine()
   Declarations
      num HEALTH_AMT = 500
      num HIGH_AGE = 100
      num SMOKER = 250
      num AUTO_AMT = 750
      num HIGH_TICKETS = 400
      num HIGH_AGE_DRIVER_DISCOUNT = 200
      num premiumAmount
   if healthRadioButton.getChecked() then
      premiumAmount = HEALTH_AMT
      if highAgeRadioButton.getChecked() then
         premiumAmount = premiumAmount + HIGH_AGE
      endif
      if smokeYesRadioButton.getChecked() then
         premiumAmount = premiumAmount + SMOKER
      endif
   else
      premiumAmount = AUTO_AMT
      if highTicketsRadioButton.getChecked() then
         premiumAmount = premiumAmount + HIGH_TICKETS
      endif
      if highAgeRadioButton.getChecked() then
         premiumAmount = premiumAmount - HIGH_AGE_DRIVER_DISCOUNT
      endif
   endif
   premAmtLabel.setText(premiumAmount)
   screen1.remove()
   screen2.display()
return
```

Figure 12-12 Pseudocode for `calcRoutine()` method of insurance premium program

Two more program segments are needed to complete the insurance premium program. These segments include the main program that executes when the program starts and the last method that executes when the program ends. In many GUI languages, the process is slightly more complicated, but the general logic appears in Figure 12-13. The final method in the program is associated with the `exitButton` object on `screen2`. In Figure 12-13, this method is called `exitRoutine()`. In this example, the main program sets up the first screen and the last method removes the last screen.

```
start
   screen1.display()
stop

public void exitRoutine()
   screen2.remove()
return
```

Figure 12-13 The main program and `exitRoutine()` method for the insurance program

TWO TRUTHS & A LIE

Developing an Event-Driven Application

1. A storyboard represents a diagram of the logic used in an interactive program.
2. An object dictionary is a list of the objects used in a program, the screens where the objects are used, and any associated code.
3. An interactivity diagram shows the relationship between screens in an interactive GUI program.

The false statement is #1. A storyboard represents a picture or sketch of the series of screens the user will see when running a program.

Understanding Threads and Multithreading

A **thread** is the flow of execution of one set of program statements. When you execute a program statement by statement, from beginning to end, you are following a thread. Many applications follow a single thread; this means that the application executes only a single program statement at a time. For example, Figure 12-14 shows how three tasks might execute in a single thread in a computer with a single CPU. Each task must end before the next task starts.

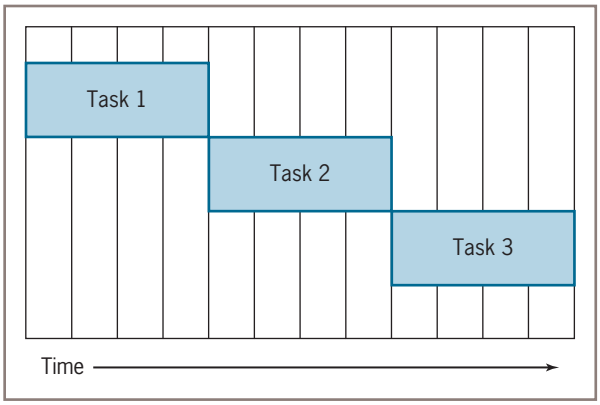


Figure 12-14 Executing multiple tasks as single threads in a single-processor system

Even if the computer has only one CPU, all major OOP languages allow you to launch, or start, multiple threads of execution by using a technique known as **multithreading**. With multithreading, threads share the CPU's time, as shown in Figure 12-15. The CPU devotes a small amount of time to one task, and then devotes a small amount of time to another. The CPU never actually performs two tasks at the same instant. Instead, it performs a piece of one task and then part of another. The CPU performs so quickly that each task seems to execute without interruption.

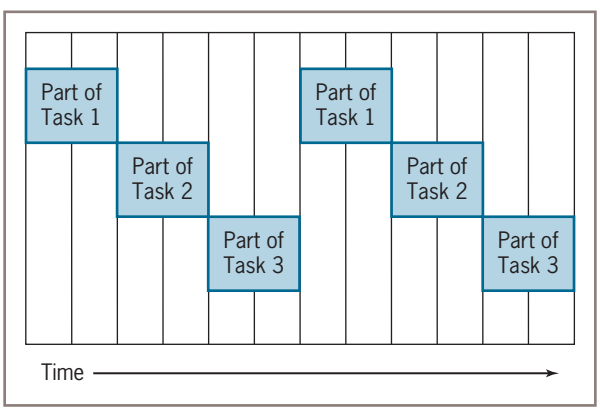


Figure 12-15 Executing multiple threads in a single-processor system

Perhaps you have seen an expert chess player participate in games with several opponents at once. The expert makes a move on the first chess board, and then moves to a second board against a second opponent while the first opponent analyzes his next move. The master can move to a third board, make a move, and return to the first board before the first opponent is even ready to respond. To the first opponent, it might seem as though the expert is devoting all of his time to the first game. Because the expert is so fast, he can play

other opponents while the first opponent contemplates his next move. Executing multiple threads on a single CPU is a similar process. The CPU transfers its attention from thread to thread so quickly that the tasks don't even "miss" the CPU's attention.

You use multithreading to improve the performance of your programs. Multithreaded programs often run faster, but more importantly, they are more user-friendly. With a multithreaded program, a user can continue to make choices by clicking buttons while the program is reading a data file. An animated figure can appear on one part of the screen while the user makes menu selections elsewhere on the screen. When you use the Internet, the benefits of multithreading increase. For example, you can begin to read a long text file, watch a video, or listen to an audio file while the file is still downloading. Web users are likely to abandon a site if they cannot use it before a lengthy downloading process completes. When a website employs multithreading to perform concurrent tasks, visitors are less likely to abandon the website—this is particularly important if the site sells a product or service or relies on advertising income that is based on the number and duration of user visits.



Programmers sometimes describe thread execution as a *lightweight process* because it is not a full-blown program. Rather, a thread must run within the context of a full, heavyweight program.



If a computer has more than one central processing unit (CPU), then each can execute a thread at the same time. However, if a computer has a single CPU and the system only supports single threading, then tasks must occur one at a time.

Writing good code to execute multithreading requires skill. Without careful coding, problems such as deadlock and starvation can arise. **Deadlock** occurs when two or more threads wait for each other to execute, and **starvation** occurs when a thread is abandoned because other threads occupy all the computer's resources.

When threads share an object, special care is needed to avoid unwanted results. For example, consider a customer order program in which two clerks are allowed to fill orders concurrently. Imagine the following scenario:

- The first clerk accesses an inventory file and tells a customer that only one item is available.
- A second clerk accesses the file and tells a different customer that only one item (the same item) is available.
- The first customer places an order, and inventory is reduced to 0.
- The second customer places an order, and inventory is reduced to –1.

Two items have been ordered, but only one exists, and the inventory file is now incorrect. There will be confusion in the warehouse, problems in the accounting department, and

one unsatisfied customer. Similar problems can occur in programs that reserve airline seats or concert tickets. OOP languages provide sophisticated techniques, known as **thread synchronization**, that help avoid these potential problems.

Object-oriented languages often contain a built-in `Thread` class that contains methods to help handle and synchronize multiple threads. For example, a `sleep()` method is sometimes used to pause program execution for a specified amount of time, perhaps a few seconds. Computer processing speed is so rapid that sometimes you have to slow down processing for human consumption. The next section describes one application that frequently requires using a `sleep()` method—computer animation.



Watch the video *Threads and Multithreading*.

TWO TRUTHS & A LIE

Understanding Threads and Multithreading

1. In the last few years, few programs that follow a single thread have been written.
2. Single-thread programs contain statements that execute in very rapid sequence, but only one statement executes at a time.
3. When you use a computer with multiple CPUs, the computer can execute multiple instructions simultaneously.

The false statement is #1. Many applications follow a single thread; this means that at any one time the application executes only a single program statement.

Creating Animation

Animation is the rapid display of still images, each slightly different from the previous one, that produces the illusion of movement. Cartoonists create animated films by drawing a sequence of frames or cells. These individual drawings are shown to the audience in rapid succession to create the sense of natural movement. You create computer animation using the same techniques. If you display computer images as fast as your CPU can process them, you might not be able to see anything. Most computer animation employs a `Thread` class `sleep()` method to pause for short intervals between the display of animation cells so the human brain has time to absorb each image's content.

Many object-oriented languages offer built-in classes that contain methods you can use to draw geometric figures. The methods typically have names like `drawLine()`, `drawCircle()`, `drawRectangle()`, and so on. You place figures on the screen based on a

graphing coordinate system. Each component has a horizontal, or **x-axis**, position as well as a vertical, or **y-axis**, position on the screen. The upper-left corner of a display is position 0, 0. The first, or **x-coordinate**, value increases as you travel from left to right across the window. The second, or **y-coordinate**, value increases as you travel from top to bottom. Figure 12-16 shows four screen coordinate positions.

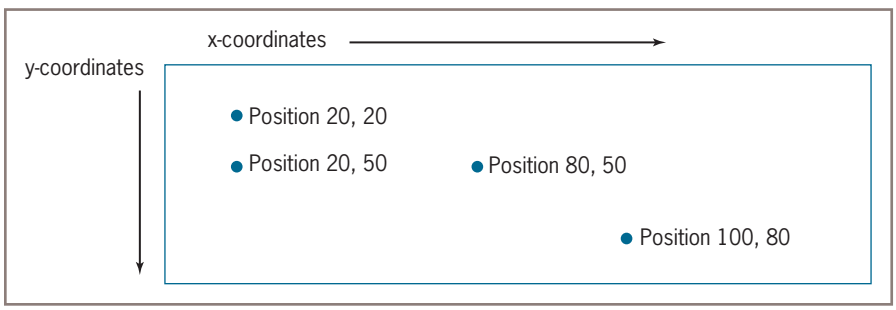


Figure 12-16 Selected screen coordinate positions

Artists often spend a great deal of time creating the exact images they want to use in an animation sequence. As a simple example, Figure 12-17 shows pseudocode for a `MovingCircle` class. As its name implies, the class moves a circle across the screen. The class contains data fields to hold x- and y-coordinates that identify the location at which a circle appears. The constants `SIZE` and `INCREASE` define the size of the first circle drawn and the relative increase in size and position of each subsequent circle. The `MovingCircle` class assumes that you are working with a language that provides a `drawCircle()` method, which creates a circle when given parameters for horizontal and vertical positions and radius. Assuming you are working with a language that provides a `sleep()` method to accept a pause time in milliseconds, the `SLEEP_TIME` constant provides a 100-millisecond gap before the production of each new circle. For simplicity, the class also assumes that you are working with a language in which no error occurs when the circles eventually move off the screen. You might want to provide statements to stop the drawing when the circle size and position exceed predetermined limits.

```
public class MovingCircle
   Declarations
      private num x = 20
      private num y = 20
      private num SIZE = 40
      private num INCREASE = SIZE / 10
      private num SLEEP_TIME = 100

   public void main()
      while true
         repaintScreen()
      endwhile
   return

   public void repaintScreen()
      drawCircle(x, y, SIZE)
      x = x + INCREASE
      y = y + INCREASE
      SIZE = SIZE + INCREASE
      Thread.sleep(SLEEP_TIME)
   return
endClass
```

Figure 12-17 The `MovingCircle` class

In most object-oriented languages, a method named `main()` executes automatically when a class object is created. The `main()` method in the `MovingCircle` class executes a continuous loop. A similar technique is used in many languages that support GUI interfaces. Program execution will cease only when the user quits the application—by clicking a window's Close button, for example. In the `repaintScreen()` method of the `MovingCircle` class, a circle is drawn at the `x, y` position, then `x, y,` and the circle size are increased. The application sleeps for one-tenth of a second (the `SLEEP_TIME` value), and then the `repaintScreen()` method draws a new circle more to the right, further down, and a little larger. The effect is a moving circle that leaves a trail of smaller circles behind as it moves diagonally across the screen. Figure 12-18 shows the output as a Java version of the application executes and after execution is complete.

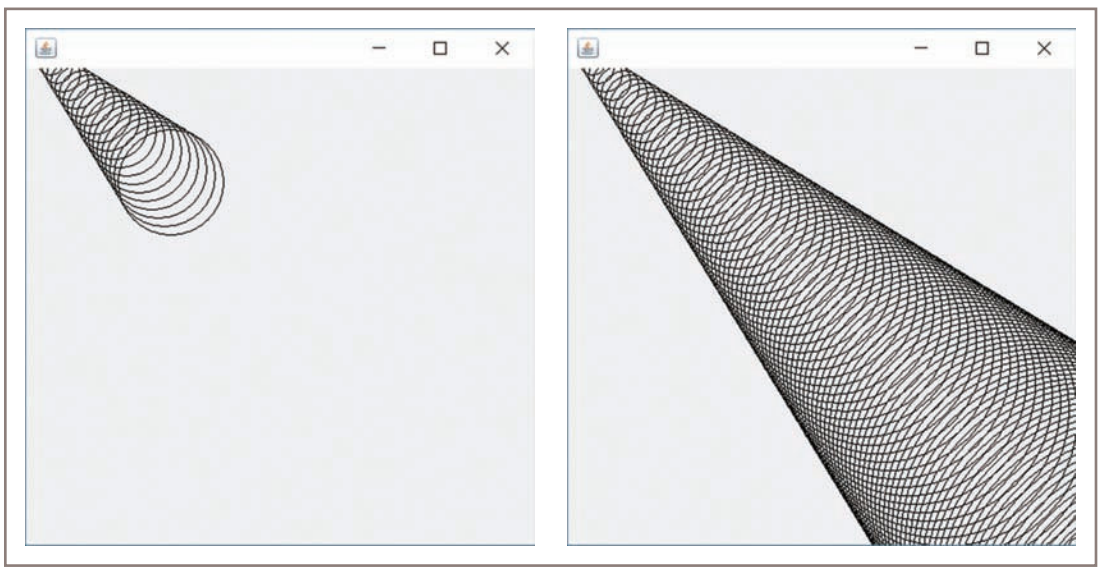


Figure 12-18 Output of the `MovingCircle` application at two points in time

Although an object-oriented language might make it easy to draw geometric shapes, you also can substitute a variety of more sophisticated, predrawn animated images to achieve the graphic effects you want within your programs. An image is loaded in a separate thread of execution, which allows program execution to continue while the image loads. This is a significant advantage because loading a large image can be time-consuming.



Many animated images are available on the Web for you to use freely. Use your search engine and keywords such as *gif files*, *jpeg files*, and *animation* to find sources for free files.

TWO TRUTHS & A LIE

Creating Animation

1. Each component you place on a screen has a horizontal, or x-axis, position as well as a vertical, or y-axis, position.
2. The x-coordinate value increases as you travel from left to right across a window.
3. You almost always want to display animation cells as fast as your processor can handle them.

The false statement is #3. If you display computer images as fast as your CPU can process them, you might not be able to see anything. Most computer animation employs a method to pause for short periods of time between animation cells.

Chapter Summary

- Interacting with a computer operating system from the command line is difficult; it is easier to use an event-driven graphical user interface (GUI), in which users manipulate objects such as buttons and menus. Within an event-driven program, a component from which an event is generated is the source of the event, and an object that is "interested in" an event listens for it.
- A user can initiate many events, such as tapping a screen or clicking a mouse. Common GUI components include labels, text boxes, buttons, check boxes, check box groups, option buttons, and list boxes. GUI components are examples of the best principles of object-oriented programming; they represent objects with attributes and methods that operate like black boxes.
- When you create a program that uses a GUI, the interface should be natural, predictable, attractive, easy to read, and nondistracting. It's helpful if the user can customize your applications. The program should be forgiving, and you should not forget that the GUI is only a means to an end.
- Developing event-driven applications requires more steps than developing procedural programs. The steps include creating wireframes, creating storyboards, defining objects and dictionaries for them, and defining the connections between the screens the user will see.

- A thread is the flow of execution of one set of program statements. Many applications follow a single thread; others use multithreading so that diverse tasks can execute concurrently.
- Animation is the rapid sequence of still images that produces the illusion of movement. Many object-oriented languages contain built-in classes that contain methods you can use to draw geometric figures on the screen. Each component has a horizontal, or x-axis, position as well as a vertical, or y-axis, position on the screen.

Key Terms

The **DOS prompt** is the command line in the DOS operating system.

An **operating system** is the software that you use to run a computer and manage its resources.

Icons are small pictures on the screen that help the user navigate a system.

An **event** is an occurrence that generates a message sent to an object.

In **event-driven** or **event-based** programs, actions occur in response to user-initiated events such as tapping a screen or clicking a mouse.

The **source of an event** is the component from which the event is generated.

A **listener** is an object that is "interested in" an event and responds to it.

A **script** is a term in object-oriented programming used to describe procedural modules that depend on user-initiated events.

A **pixel** is a picture element, or one of the tiny dots of light that form a grid on your screen.

Accessibility describes the screen design concerns that make programs easier to use for people with physical limitations.

A **wireframe** is a picture or sketch of a screen the user will see when running a program.

A **page schematic** is a wireframe.

A **screen blueprint** is a wireframe.

A **storyboard** contains a series of wireframes that represent a user's experience with proposed software.

An **object dictionary** is a list of the objects used in a program, the screens where the objects are used, and any associated code (script).

An **interactivity diagram** shows the relationship between screens in an interactive GUI program.

Registering components is the act of signing them up so they can react to events initiated by other components.

A **container** is a class of objects whose main purpose is to hold other elements—for example, a window.

A **thread** is the flow of execution of one set of program statements.

Multithreading is using multiple threads of program execution.

Deadlock is a flaw in multithreaded programs in which two or more threads wait for each other to finish executing.

Starvation is a flaw in multithreaded programs in which a thread is unable to complete because other threads occupy all the computer's resources.

Thread synchronization is a set of techniques that coordinates threads of execution to help avoid potential multithreading problems.

Animation is the rapid display of still images, each slightly different from the previous one, that produces the illusion of movement.

The **x-axis** represents horizontal positions in a screen window.

The **y-axis** represents vertical positions in a screen window.

The **x-coordinate** value increases as you travel from left to right across a screen window.

The **y-coordinate** value increases as you travel from top to bottom across a screen window.

Exercises



Review Questions

1. Compared to using a command line, an advantage to using an operating system that employs a GUI is _____ .

   a. you can interact directly with the operating system
   b. you do not have to deal with confusing icons
   c. you do not have to memorize complicated commands
   d. all of the above

2. When users can initiate actions by clicking the mouse on an icon, the program is _____-driven.

   a. command
   b. event
   c. prompt
   d. incident

3. A component from which an event is generated is the _____ of the event.

   a. base
   b. icon
   c. listener
   d. source

4. An object that responds to an event is a _____ .

   a. listener
   b. snooper
   c. transponder
   d. source

5. All of the following are user-initiated events except a _____ .

   a. key press
   b. key drag
   c. right mouse click
   d. mouse drag

6. All of the following are typical GUI components except a _____ .

   a. button
   b. text box
   c. list box
   d. handle

7. GUI components operate most like _____ .

   a. looping structures
   b. procedural functions
   c. black boxes
   d. command lines

8. Which of the following is *not* a principle of good GUI design?

   a. The interface should be predictable.
   b. The fancier the screen design, the better.
   c. The program should be forgiving.
   d. The user should be able to customize applications.

9. Which of the following aspects of a GUI layout is most predictable and natural for the user?

   a. A menu bar runs down the right side of the screen.
   b. *Help* is the first option on a menu.
   c. A dollar sign icon represents saving a file.
   d. Pressing *Esc* allows the user to cancel a selection.

10. In most GUI programming environments, the programmer can change all of the following attributes of most components except their _____ .

    a. color
    b. screen location
    c. size
    d. The programmer can change all of these attributes.

11. Depending on the programming language, you might _____ to change a screen component's attributes.

    a. use an assignment statement
    b. call a module
    c. enter a value into a list of properties
    d. all of the above

12. When you create an event-driven application, which of the following must be done before defining objects?

    a. Translate the program.
    b. Create wireframes and storyboards.
    c. Test the program.
    d. Code the program.

13. A _____ is a sketch of a screen the user will see when running a program.

    a. flowchart
    b. hierarchy chart
    c. storyboard
    d. tale timber

14. An object _____ is a list of objects used in a program.

    a. thesaurus
    b. dictionary
    c. index
    d. glossary

15. A(n) _____ diagram shows the connections between the various screens a user might see during a program's execution.

    a. interactivity
    b. help
    c. cooperation
    d. communication

16. The flow of execution of one set of program statements is a _____ .

    a. thread
    b. string
    c. path
    d. route

17. When a computer contains a single CPU, it can execute _____ computer instruction(s) at a time.

    a. one
    b. several
    c. an unlimited number of
    d. from several to thousands of

18. Multithreaded programs usually _____ than their procedural counterparts.

    a. run faster
    b. are harder to use
    c. are older
    d. all of the above

19. An object's horizontal position on the computer screen is its _____ .

    a. a-coordinate
    b. h-coordinate
    c. x-coordinate
    d. y-coordinate

20. You create computer animation by _____ .

    a. drawing an image and setting its animation property to true
    b. drawing a single image and executing it on a multiprocessor system
    c. drawing a sequence of frames that are shown in rapid succession
    d. Animation is not used in computer applications.



Programming Exercises

1. Take a critical look at three GUI applications you have used—for example, a spreadsheet, a word-processing program, and a game. Describe how well each conforms to the GUI design guidelines listed in this chapter.

2. Select one element of poor GUI design in a program you have used. Describe how you would improve the design.

3. Select a GUI program that you have never used before. Describe how well it conforms to the GUI design guidelines listed in this chapter.

4. Design the wireframes and storyboard, interactivity diagram, object dictionary, and any necessary scripts for an interactive program for customers of Sanderson's Ice Cream Sundaes.

   Allow customers the option of choosing a three-scoop, two-scoop, or one-scoop creation at a base price of $4.00, $3.00, or $2.20, respectively. Let the customer choose chocolate, strawberry, or vanilla as the primary flavor. If the customer adds nuts, whipped cream, or cherries to the order, add $0.50 for each to the base price. After the customer clicks an Order Now button, display the price of the order.

5. Design the wireframes and storyboard, interactivity diagram, object dictionary, and any necessary scripts for an interactive program for customers of Fortune's Vacation Resort.

   Allow customers the option of choosing a studio, one-bedroom, or two-bedroom cabin, each of which costs a different weekly rental amount. Also allow the option of lake view, which increases the rental fee. The total fee is displayed when the user clicks a Reserve Cabin button.

6. Design the wireframes and storyboard, interactivity diagram, object dictionary, and any necessary scripts for an interactive program for customers of the Natural Munches Sandwich Shop.

   Allow customers the option of choosing one of three types of bread and any number of six filler items for a sandwich. After the customer clicks an Order Now button, display the price of the sandwich.

7. Design the wireframes and storyboard, interactivity diagram, object dictionary, and any necessary scripts for an interactive program for customers of the Friar Farm Market.

   Allow customers the option of choosing tomatoes ($3.00 per pound), peppers ($2.50 per pound), or onions ($2.25 per pound). The customer enters a weight in pounds in a text box. After the customer clicks a Select button, display the price of the order.

8. Design the wireframes and storyboard, interactivity diagram, object dictionary, and any necessary scripts for an interactive program for clients of Larry's Lawn Service.

   Allow clients to choose the size of their yard so they can be charged accordingly. For example, a lot that covers less than one-third of an acre costs $50 per service call; a lot that covers one-third to two-thirds of an acre costs $72.50 per service call; and a lot that covers more than two-thirds of an acre costs $84 per service call. Also, allow clients to choose a schedule of weekly or semiweekly lawn maintenance. After the customer clicks a Select button, display the price of the service per week. Note that semiweekly service comes with a 10 percent discount.



Performing Maintenance

1. A file named MAINTENANCE12-01.txt is included with your downloadable student files. Assume that this program is a working program in your organization and that it needs modifications as described in the comments (lines that begin with two slashes) at the beginning of the file. Your job is to alter the program to meet the new specifications.



Find the Bugs

1. Your downloadable files for Chapter 12 include DEBUG12-01.txt, DEBUG12-02.txt, and DEBUG12-03.txt. Each file starts with some comments that describe the problem. Comments are lines that begin with two slashes (//). Following the comments, each file contains pseudocode that has one or more bugs you must find and correct.
2. Your downloadable files for Chapter 12 include a file named DEBUG12-04.jpg that contains a storyboard that contains logical errors; examine the storyboard, and then find and correct all the bugs.



Game Zone

1. Design the wireframes and storyboard, interactivity diagram, object dictionary, and any necessary scripts for an interactive program that allows a user to play a card game named Lucky Seven. In real life, the game can be played with seven cards, each containing a number from 1 through 7, that are shuffled and dealt number-side down. To start the game, a player turns over any card. The exposed

number on the card determines the position (reading from left to right) of the next card that must be turned over. For example, if the player turns over the first card and its number is 7, the next card turned must be the seventh card (counting from left to right). If the player turns over a card whose number denotes a position that was already turned, the player loses the game. If the player succeeds in turning over all seven cards, the player wins.

Instead of cards, you will use seven buttons labeled 1 through 7 from left to right. Randomly associate one of the seven values 1 through 7 with each button. (In other words, the associated value might or might not be equivalent to the button's labeled value.) When the player clicks a button, reveal the associated hidden value. If the value represents the position of a button already clicked, the player loses. If the revealed number represents an available button, force the user to click it—that is, do not take any action until the user clicks the correct button. After a player clicks a button, remove the button from play.

For example, a player might click Button 7, revealing a 4. Then the player clicks Button 4, revealing a 2. Then the player clicks Button 2, revealing a 7. The player loses because Button 7 is already "used."

2. In the Game Zone sections of Chapters 6 and 9, you designed and fine-tuned the logic for the game Hangman, in which the user guesses letters in a series of hidden words. Design the wireframes and storyboard, interactivity diagram, object dictionary, and any necessary scripts for a version of the game in which the user clicks lettered buttons to fill in the secret words. Draw a "hanged" person piece by piece with each missed letter. For example, when the user chooses a correct letter, place it in the appropriate position or positions in the word, but the first time the user chooses a letter that is not in the target word, draw a head for the "hanged" man. The second time the user makes an incorrect guess, add a torso. Continue with arms and legs. If the complete body is drawn before the user has guessed all the letters in the word, display a message indicating that the player has lost the game. If the user completes the word before all the body parts are drawn, display a message that the player has won. Assume that you can use built-in methods named `drawCircle()` and `drawLine()`. The `drawCircle()` method requires three parameters—the x- and y-coordinates of the center, and a radius size. The `drawLine()` method requires four parameters—the x- and y-coordinates of the start of the line, and the x- and y-coordinates of the end of the line.

APPENDIX A

Understanding Numbering Systems and Computer Codes

The numbering system you know best is the **decimal numbering system**: the system based on 10 digits, 0 through 9. Mathematicians call decimal-system numbers **base 10** numbers. When you use the decimal system, only the value symbols 0 through 9 are available; if you want to express a value larger than 9, you must use multiple digits from the same pool of 10, placing them in columns.

When you use the decimal system, you analyze a multicolumn number by mentally assigning place values to each column. The value of the far right column is 1, the value of the next column to the left is 10, the next column is 100, and so on; the column values are multiplied by 10 as you move to the left. There is no limit to the number of columns you can use; you simply keep adding columns to the left as you need to express higher values. For example, Figure A-1 shows how the value 305 is represented in the decimal system. You simply multiply each digit by the value of its column, and then sum the results.

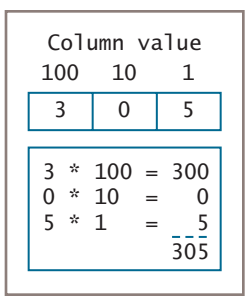


Figure A-1 Representing 305 in the decimal system

The **binary numbering system** works in the same way as the decimal numbering system, except that it uses only two digits, 0 and 1. When you use the binary system, you must use multiple columns if you want to express a value greater than 1 because no single symbol is available that represents any value other than 0 or 1. However, instead of each new column to the left being 10 times greater than the previous column, each new column in the binary system is only two times the value of the previous column. For example, Figure A-2 shows how the numbers 9 and 305 are represented in the binary system. Notice that in both the binary system and the decimal system, it is perfectly acceptable, and often necessary, to create numbers with 0 in one or more columns. Mathematicians call numbers expressed in binary **base 2** numbers. As with the decimal system, there

is no limit to the number of columns used in a binary number; you can use as many as it takes to express a value.

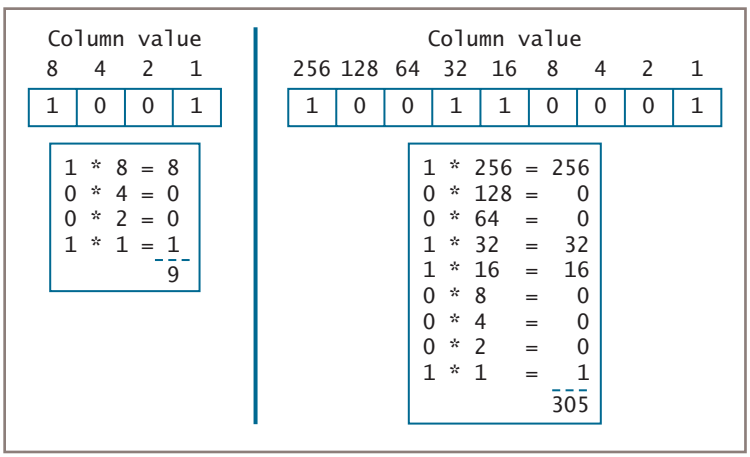


Figure A-2 Representing decimal values 9 and 305 in the binary system

A computer stores every piece of data it uses as a set of 0s and 1s. Each 0 or 1 is known as a **bit**, which is short for *binary digit*. Every computer uses 0s and 1s because all values in a computer are stored as electronic signals that are either on or off. This two-state system is most easily represented using just two digits.

Computers use a set of binary digits to represent stored characters. If computers used only one binary digit to represent characters, then only two different characters could be represented, because the single bit could be only 0 or 1. If computers used only two digits to represent characters, then only four characters could be represented—the four codes 00, 01, 10, and 11, which in decimal values are 0, 1, 2, and 3, respectively. Many computers use sets of eight binary digits to represent each character they store, because using eight binary digits provides 256 different combinations. A set of eight bits is a **byte**. One byte's combination of bits can represent an *A*, another a *B*, still others *a* and *b*, and so on. Two hundred fifty-six combinations are enough so that each capital letter, lowercase letter, digit, and punctuation mark used in English has its own code; even a space has a code. For example, in the system named the **American Standard Code for Information Interchange (ASCII)**, 01000001 represents the character *A*. The binary number 01000001 has a decimal value of 65, but this numeric value is not important to ordinary computer users; it is simply a code that stands for *A*.

The ASCII code is not the only computer code, but it is typical, and is the one used in most personal computers. The **Extended Binary Coded Decimal Interchange Code (EBCDIC)**, is an eight-bit code that is used in IBM mainframe and midrange computers. In these computers, the principle is the same—every character is stored in a byte as a series of binary digits. However, the actual values used are different. For example, in EBCDIC, an *A* is 11000001, or 193. Another code used by languages such as Java and C# is **Unicode**; with this code, 16 bits are used to represent each character. The character *A* in Unicode has the same decimal value as the ASCII *A*, 65, but it is stored as 0000000001000001.

Using two bytes provides many more possible combinations than using only eight bits—65,536 to be exact. With Unicode, enough codes are available to represent all English letters and digits, as well as characters from many international alphabets.

Ordinary computer users seldom think about the numeric codes behind the letters, numbers, and punctuation marks they enter from their keyboards or see displayed on a monitor. However, they see the consequence of the values behind letters when they see data sorted in alphabetical order. When you sort a list of names, *Andrea* comes before *Brian*, and *Caroline* comes after *Brian* because the numeric code for *A* is lower than the code for *B*, and the numeric code for *C* is higher than the code for *B*, no matter whether you use ASCII, EBCDIC, or Unicode.

Table A-1 shows the decimal and binary values behind the most commonly used characters in the ASCII character set—the letters, numbers, and punctuation marks you can enter from your keyboard using a single key press. Other values not shown in Table A-1 also have specific purposes. For example, when you display the character that holds the decimal value 7, nothing appears on the screen, but a bell sounds. Programmers often use this character when they want to alert a user to an error or some other unusual condition.



Each binary number in Table A-1 is shown containing two sets of four digits; this convention makes the eight-digit numbers easier to read. Four digits, or a half byte, is a **nibble**.

Decimal Number	Binary Number	ASCII Character	Decimal Number	Binary Number	ASCII Character
32	0010 0000	Space	40	0010 1000	(Left parenthesis
33	0010 0001	! Exclamation point	41	0010 1001	) Right parenthesis
34	0010 0010	" Quotation mark, or double quote	42	0010 1010	* Asterisk
35	0010 0011	# Number sign, also called an octothorpe or a pound sign	43	0010 1011	+ Plus sign
			44	0010 1100	, Comma
			45	0010 1101	- Hyphen or minus sign
36	0010 0100	$ Dollar sign	46	0010 1110	. Period or decimal point
37	0010 0101	% Percent			
38	0010 0110	& Ampersand	47	0010 1111	/ Slash or front
39	0010 0111	' Apostrophe, single quote	48	0011 0000	0

Table A-1 Decimal and binary values for common ASCII characters *(continues)*

(continued)

Decimal Number	Binary Number	ASCII Character	Decimal Number	Binary Number	ASCII Character
49	0011 0001	1	72	0100 1000	H
50	0011 0010	2	73	0100 1001	I
51	0011 0011	3	74	0100 1010	J
52	0011 0100	4	75	0100 1011	K
53	0011 0101	5	76	0100 1100	L
54	0011 0110	6	77	0100 1101	M
55	0011 0111	7	78	0100 1110	N
56	0011 1000	8	79	0100 1111	O
57	0011 1001	9	80	0101 0000	P
58	0011 1010	: Colon	81	0101 0001	Q
59	0011 1011	; Semicolon	82	0101 0010	R
60	0011 1100	< Less-than sign	83	0101 0011	S
61	0011 1101	= Equal sign	84	0101 0100	T
62	0011 1110	> Greater-than sign	85	0101 0101	U
63	0011 1111	? Question mark	86	0101 0110	V
64	0100 0000	@ At sign	87	0101 0111	W
65	0100 0001	A	88	0101 1000	X
66	0100 0010	B	89	0101 1001	Y
67	0100 0011	C	90	0101 1010	Z
68	0100 0100	D	91	0101 1011	[Opening or left bracket
69	0100 0101	E	92	0101 1100	\ Backslash
70	0100 0110	F	93	0101 1101	] Closing or right bracket
71	0100 0111	G			

Table A-1 Decimal and binary values for common ASCII characters *(continues)*

(continued)

Decimal Number	Binary Number	ASCII Character
94	0101 1110	^ Caret
95	0101 1111	_ Underline or underscore
96	0110 0000	` Grave accent
97	0110 0001	a
98	0110 0010	b
99	0110 0011	c
100	0110 0100	d
101	0110 0101	e
102	0110 0110	f
103	0110 0111	g
104	0110 1000	h
105	0110 1001	i
106	0110 1010	j
107	0110 1011	k
108	0110 1100	l
109	0110 1101	m
110	0110 1110	n
111	0110 1111	o
112	0111 0000	p
113	0111 0001	q
114	0111 0010	r
115	0111 0011	s
116	0111 0100	t
117	0111 0101	u
118	0111 0110	v
119	0111 0111	w
120	0111 1000	x
121	0111 1001	y
122	0111 1010	z
123	0111 1011	{ Opening or left brace
124	0111 1100	| Vertical line or pipe
125	0111 1101	} Closing or right brace
126	0111 1110	~ Tilde

Table A-1 Decimal and binary values for common ASCII characters

The Hexadecimal System

The **hexadecimal numbering system** is the **base 16** system; it uses 16 digits. As shown in Table A-2, the digits are 0 through 9 and A through F. Computer professionals often use the hexadecimal system to express addresses and instructions as they are stored in computer memory because hexadecimal provides convenient shorthand expressions for groups of binary values. In Table A-2, each hexadecimal value represents one of the 16 possible combinations of four-digit binary values. Therefore, instead of referencing memory contents as a 16-digit binary value, for example, programmers can use a 4-digit hexadecimal value.

Decimal Value	Hexadecimal Value	Binary Value (shown using four digits)	Decimal Value	Hexadecimal Value	Binary Value (shown using four digits)
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	10	A	1010
3	3	0011	11	B	1011
4	4	0100	12	C	1100
5	5	0101	13	D	1101
6	6	0110	14	E	1110
7	7	0111	15	F	1111

Table A-2 Values in the decimal and hexadecimal systems

In the hexadecimal system, each column is 16 times the value of the column to its right. Therefore, column values from right to left are 1, 16, 256, 4096, and so on. Figure A-3 shows how 78, 171, and 305 are expressed in hexadecimal.

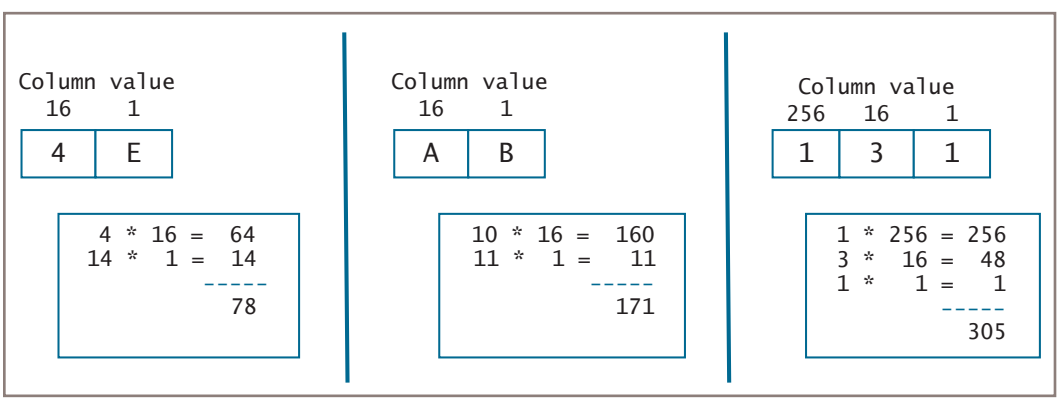


Figure A-3 Representing decimal values 78, 171, and 305 in the hexadecimal system

Measuring Storage

In computer systems, both internal memory and external storage are measured in bits and bytes. Eight bits make a byte, and a byte frequently holds a single character (in ASCII or EBCDIC) or half a character (in Unicode). Because a byte is such a small unit of storage, the size of memory and files is often expressed in thousands or millions of bytes. Table A-3 describes some commonly used terms for storage measurement.

Term	Abbreviation	Number of Bytes Using Binary System	Number of Bytes Using Decimal System	Example
Kilobyte	KB or kB	1,024	one thousand	In Microsoft Word, this appendix occupies about 70 kB on a hard disk.
Megabyte	MB	1,048,576 (1,024 × 1,024 kilobytes)	one million	One megabyte can hold an average book in text format.
Gigabyte	GB	1,073,741,824 (1,024 megabytes)	one billion	A gigabyte can hold a symphony recording in high fidelity or a movie at TV quality.
Terabyte	TB	1,024 gigabytes	one trillion	Some hard drives are 1 terabyte. The Web archive data of the Library of Congress occupied about 500 terabytes when this book was published.
Petabyte	PB	1,024 terabytes	one quadrillion	The Google Web site processes about 24 petabytes per day.
Exabyte	EB	1,024 petabytes	one quintillion	A popular expression claims that all words ever spoken by humans could be stored in text form in 5 exabytes.
Zettabyte	ZB	1,024 exabytes	one sextillion	A popular expression claims that all words ever spoken by humans could be stored in audio form in 42 zettabytes.
Yottabyte	YB	1,024 zettabytes	one septillion (a 1 followed by 24 zeros)	The combined space on all hard drives in the world is less than 1 yottabyte.

Table A-3 Commonly used terms for computer storage

In the metric system, the prefix *kilo* means 1,000. However, in Table A-3, notice that a kilobyte is 1,024 bytes. The discrepancy occurs because everything stored in a computer is based on the binary system, so multiples of two are used in most measurements. If you multiply 2 by itself 10 times, the result is 1,024, which is a little over 1000. Similarly, a gigabyte is 1,073,741,824 bytes, which is more than a billion.

Confusion arises because many hard-drive manufacturers use the decimal system instead of the binary system to describe storage. For example, if you buy a hard drive that holds 100 gigabytes, it holds exactly 100 billion bytes. However, in the binary system, 100 GB is more than 107 billion bytes, so when you check your hard drive's capacity, your computer will report that you don't quite have 100 GB, but only a little more than 93 GB.

Key Terms

The **decimal numbering system** is the numbering system based on 10 digits in which column values are multiples of 10.

Base 10 describes numbers created using the decimal numbering system.

The **binary numbering system** is the numbering system based on two digits in which column values are multiples of 2.

Base 2 describes numbers created using the binary numbering system.

A **bit** is a binary digit; it is a unit of storage equal to one-eighth of a byte.

A **byte** is a storage measurement equal to eight bits.

American Standard Code for Information Interchange (ASCII) is an eight-bit character-coding scheme used on many personal computers.

Extended Binary Coded Decimal Interchange Code (EBCDIC) is an eight-bit character-coding scheme used on many larger computers.

Unicode is a 16-bit character-coding scheme.

A **nibble** is a storage measurement equal to four bits, or half a byte.

The **hexadecimal numbering system** is the numbering system based on 16 digits in which column values are multiples of 16.

Base 16 describes numbers created using the hexadecimal numbering system.

APPENDIX B

Solving Difficult Structuring Problems

In Chapter 3, you learned that you can solve any logical problem using only the three standard structures—sequence, selection, and loop. Modifying an unstructured program to make it adhere to structured rules is often a simple matter. Sometimes, however, structuring a more complicated program can be challenging. Still, no matter how complicated, large, or poorly structured a problem is, the same tasks can *always* be accomplished in a structured manner.

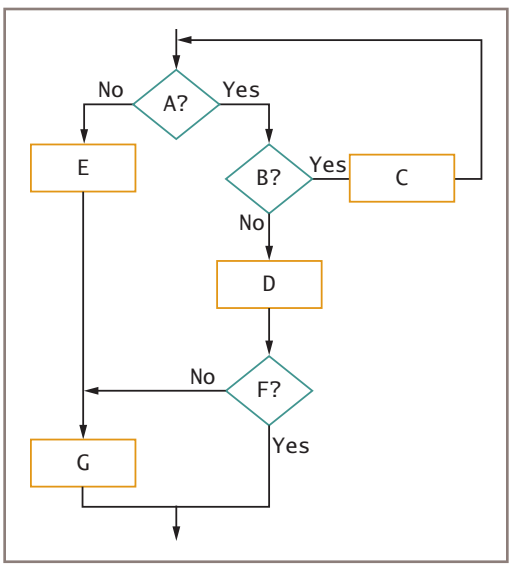


Figure B-1 Unstructured flowchart segment

Consider the flowchart segment in Figure B-1. Is it structured?

No, it is not structured. To straighten out the flowchart segment, thereby making it structured, you can use the "spaghetti" method. Untangle each path of the flowchart as if you were attempting to untangle strands of spaghetti in a bowl. The objective is to create a new flowchart segment that performs exactly the same tasks as the first, but using only the three structures—sequence, selection, and loop.

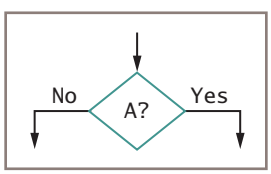


Figure B-2 Structuring, Step 1

To begin to untangle the unstructured flowchart segment, you start at the beginning with the evaluation labeled A, shown in Figure B-2. This step must represent

the beginning of either a selection or a loop, because a sequence would not contain any possibility of branching.

If you follow the logic on the *No* side, or the left side, of the question in the original flowchart, you can imagine pulling up on the left branch of the evaluation. You encounter process E, followed by G, followed by the end, as shown in Figure B-3. Compare the *No* actions after the evaluation of A in the first flowchart (Figure B-1) with the actions after A in Figure B-3; they are identical.

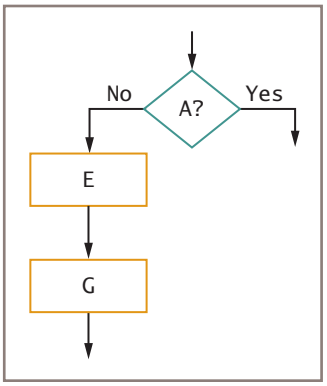


Figure B-3 Structuring, Step 2

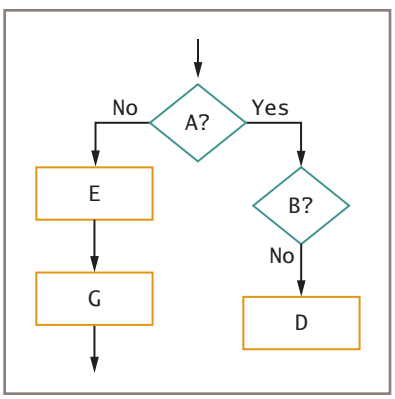


Figure B-4 Structuring, Step 3

Now continue on the right side, or the *Yes* side, of the A evaluation in Figure B-1. When you follow the flowline, you encounter a decision symbol labeled B. Imagine pulling on B's left side, and a process, D, comes up next. See Figure B-4.

After Step D in the original diagram, an evaluation, labeled F, comes up. Pull on its left, or *No*, side and you get a process, G, and then the end. When you pull on F's right side, or *Yes* side, in the original flowchart, you simply reach the end, as shown in Figure B-5. Notice in Figure B-5 that the G process now appears in two locations. When you improve unstructured flowcharts, so that they become structured, you often must repeat steps to eliminate crossed lines and spaghetti logic, which is difficult to follow.

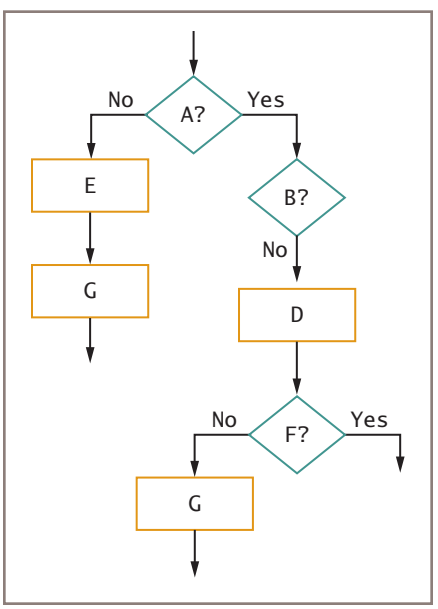


Figure B-5 Structuring, Step 4

The biggest problem in structuring the original flowchart segment from Figure B-1 follows the right, or *Yes*, side of the B evaluation. When the evaluation of B is *Yes*, you encounter process C, as shown in Figures B-1 and B-6. The structure that begins with the evaluation of C looks like a loop because it doubles back, up to A. However, a structured loop must have the appearance as shown in Figure B-7: a question followed by a structure, returning right back to the question. In Figure B-1, if the path coming from C returned directly to B, there would be no problem; it would be a simple, structured loop. However, as it is, Question A must be repeated. The spaghetti technique requires that if lines of logic are tangled up, you must start repeating the steps in question. So, you repeat an A evaluation after C, as Figure B-6 shows.

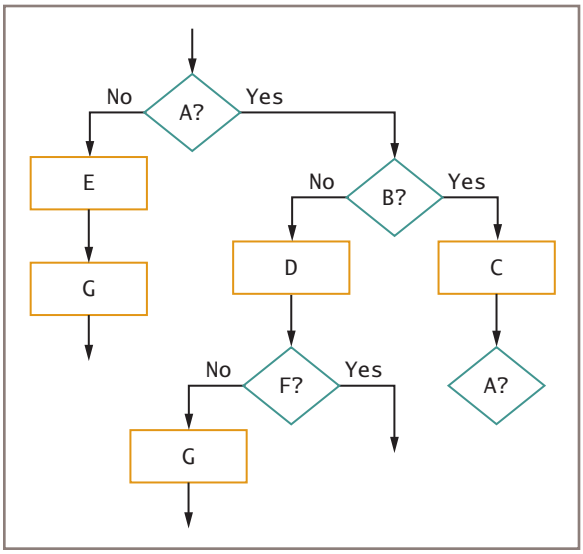


Figure B-6 Structuring, Step 5

In the original flowchart segment in Figure B-1, when A is *Yes*, Question B always follows. So, in Figure B-8, after A is *Yes* and B is *Yes*, Step C executes, and A is asked again; when A is *Yes*, B repeats. In the original, when B is *Yes*, C executes, so in Figure B-8, on the right side of B, C repeats. After C, A occurs. On the right side of A, B occurs. On the right side of B, C occurs. After C, A should occur again, and so on. Soon you should realize that, to follow the steps in the same order as in the original flowchart segment, you will repeat these same steps forever.

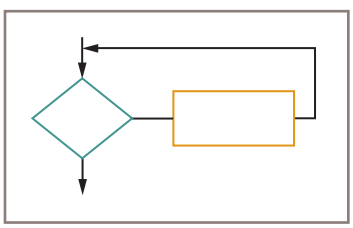


Figure B-7 A structured loop

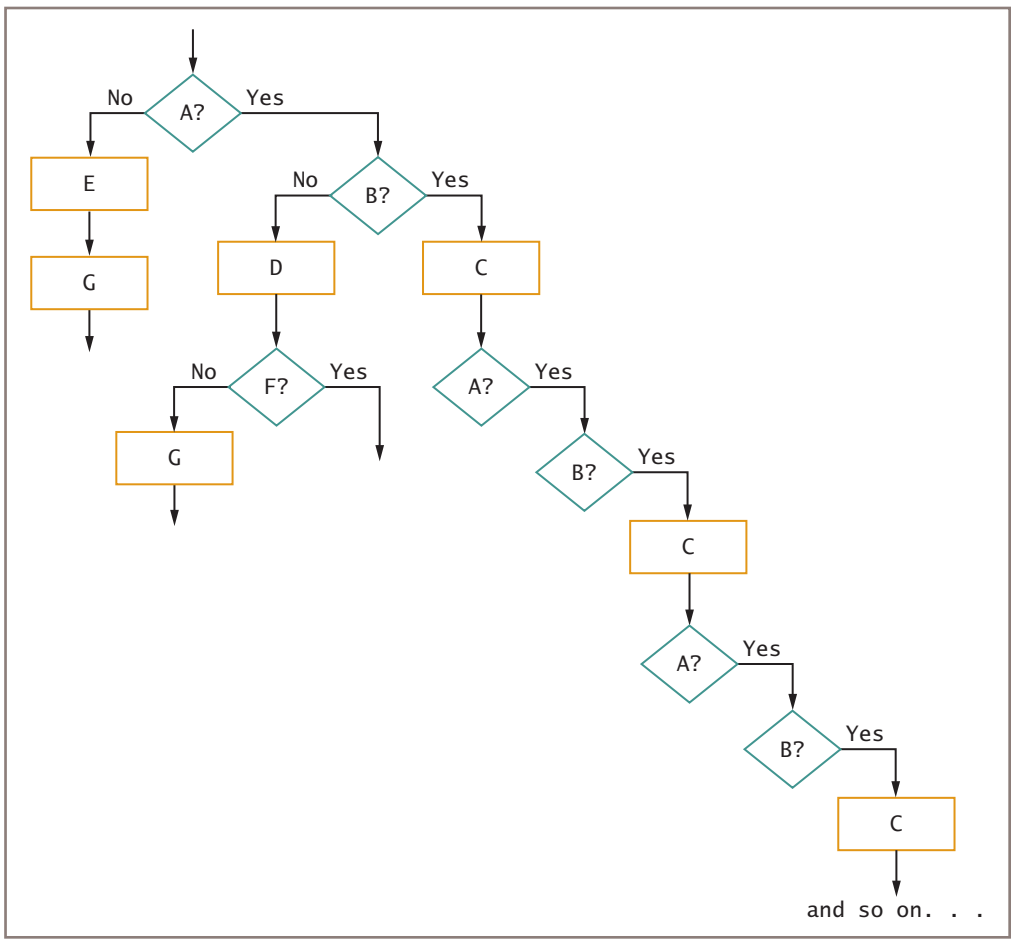


Figure B-8 Structuring, Step 6

If you continue with Figure B-8, you will never be able to finish the flowchart; every C is always followed by another A, B, and C. Sometimes, to make a program segment structured, you have to add an extra flag variable to get out of an infinite mess. A flag is a variable that you set to indicate a true or false state. Typically, a variable is called a flag when its only purpose is to tell you whether some event has occurred. You can create a flag variable named `shouldRepeat` and set its value to *Yes* or *No*, depending on whether it is appropriate to repeat the evaluation of A. When A is *No*, the `shouldRepeat` flag should be set to *No* because, in this situation, you never want to repeat Question A again. See Figure B-9.

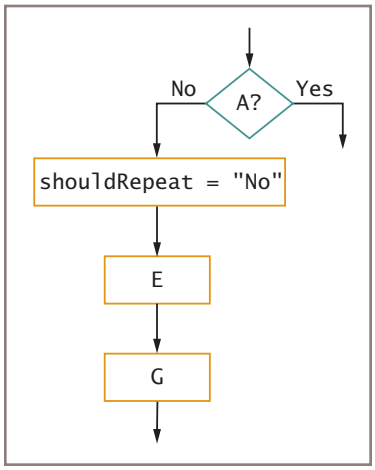


Figure B-9 Adding a flag to the flowchart

Similarly, after A is *Yes*, but when B is *No*, you never want to repeat Question A again. Figure B-10 shows that you set `shouldRepeat` to *No*, when the B decision evaluates to *No*. Then you continue with D, and the F evaluation that executes G when F is *No*.

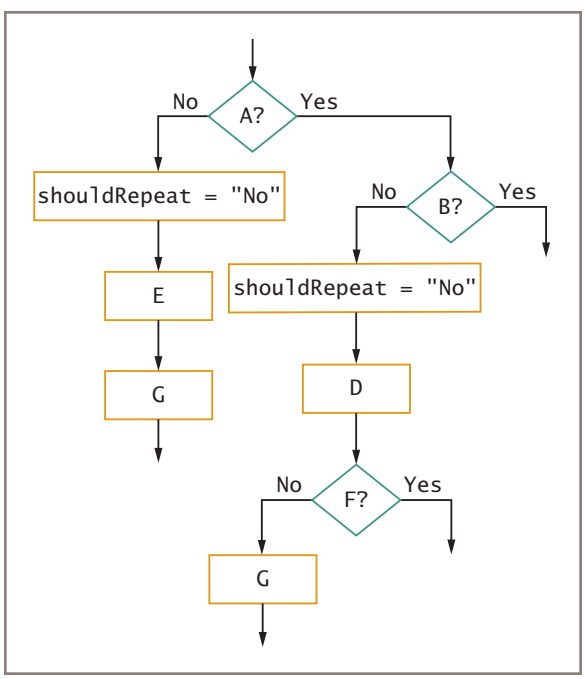


Figure B-10 Adding a flag to a second path in the flowchart

However, in the original flowchart segment in Figure B-1, when the B evaluation result is *Yes*, you *do* want to repeat A. So, when B is *Yes*, perform the process for C and set the `shouldRepeat` flag equal to *Yes*, as shown in Figure B-11.

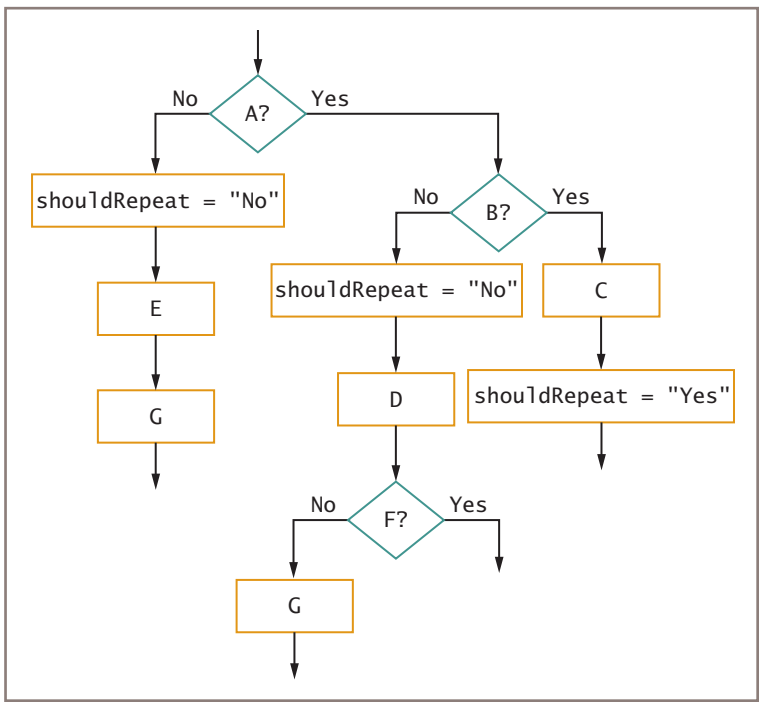


Figure B-11 Adding a flag to a third path in the flowchart

Now all paths of the flowchart can join together at the bottom with one final question: Is `shouldRepeat` equal to *Yes*? If it isn't, exit; but if it is, extend the flowline to go back to repeat Question A. See Figure B-12. Take a moment to verify that the steps that would execute following Figure B-12 are the same steps that would execute following Figure B-1.

- When A is *No*, E and G always execute.
- When A is *Yes* and B is *No*, D and evaluation F always execute.
- When A is *Yes* and B is *Yes*, C always executes and A repeats.

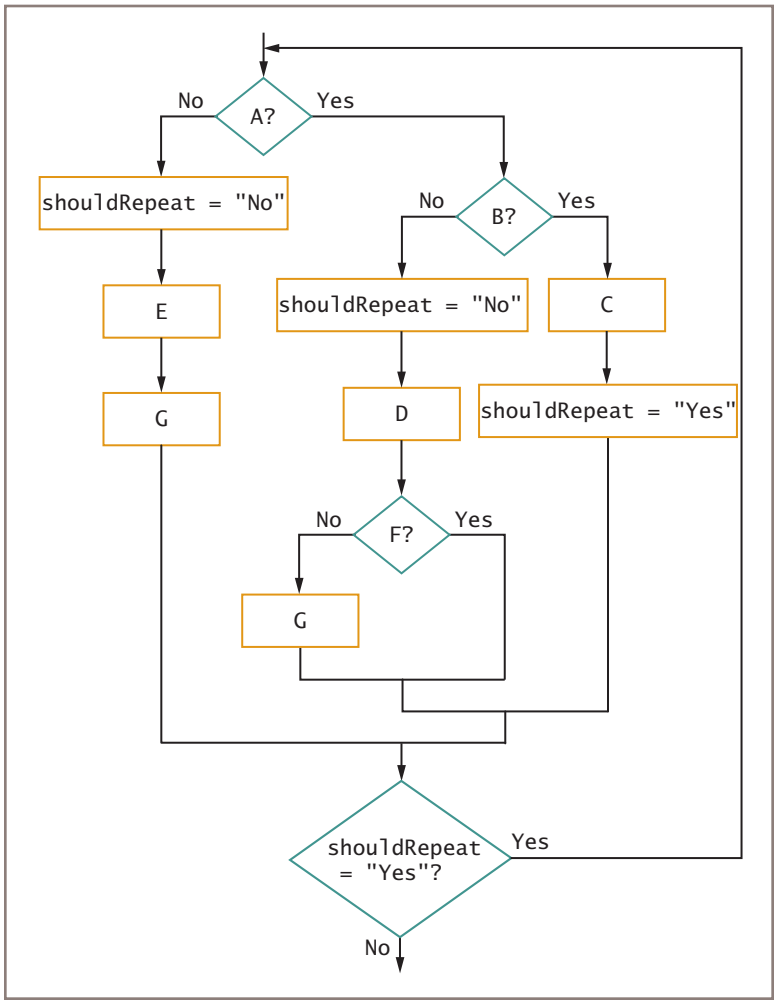


Figure B-12 Tying up the loose ends

Figure B-12 contains three nested selection structures. Notice how the F evaluation begins a complete selection structure whose *Yes* and *No* paths join together when the structure ends. This F selection structure is within one path of the B decision structure; the B evaluation begins a complete selection structure whose *Yes* and *No* paths join at the bottom. Likewise, the B selection structure resides entirely within one path of the A selection structure.

The flowchart segment in Figure B-12 performs identically to the original spaghetti version in Figure B-1. However, is this new flowchart segment structured? There are so many steps in the diagram, it is hard to tell. You may be able to see the structure more clearly if you create a module named `aThroughG()`. If you create the module shown in Figure B-13, then the original flowchart segment can be drawn as in Figure B-14.

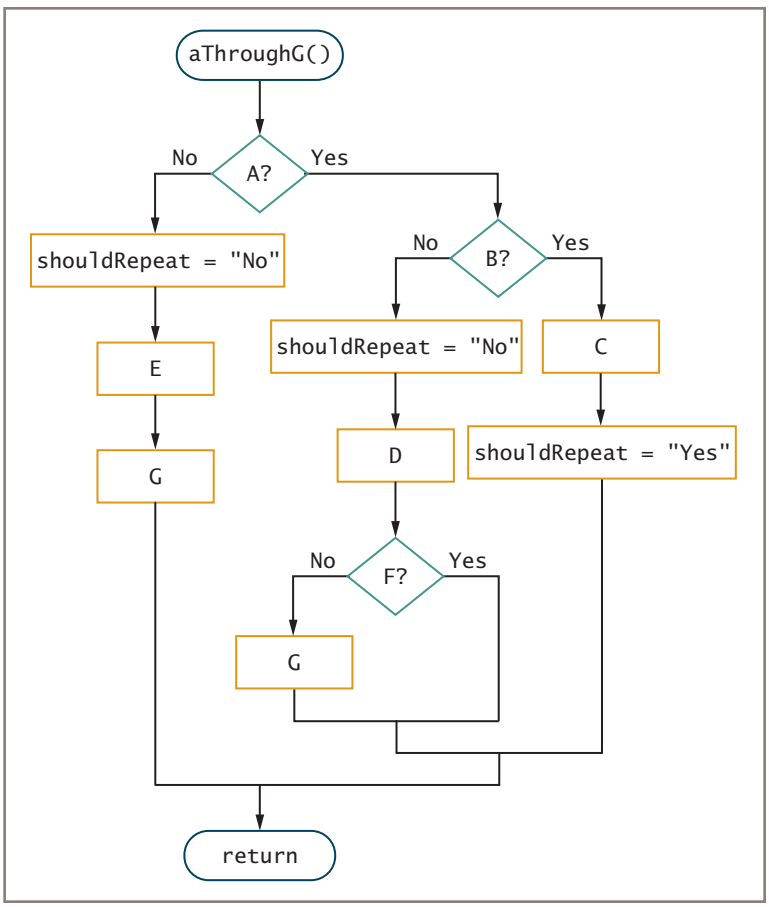


Figure B-13 The `aThroughG()` module

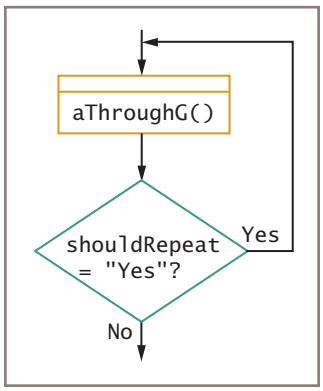


Figure B-14 Logic in Figure B-12, substituting a module for steps A through G

Now you can see that the completed flowchart segment in Figure B-14 is a `do-until` loop. If you prefer to use a `while` loop, you can redraw Figure B-14 to perform a sequence followed by a `while` loop, as shown in Figure B-15.

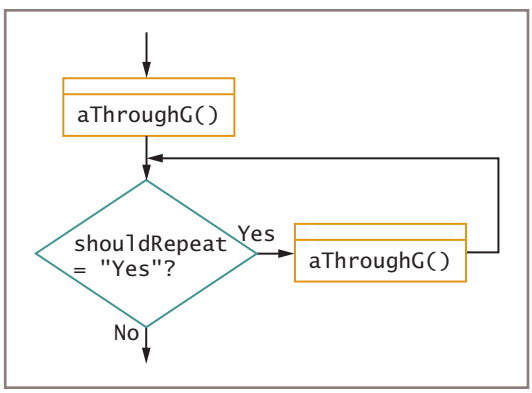


Figure B-15 Logic in Figure B-14, substituting a sequence and `while` loop for the `do-until` loop

It has taken some extra effort, including repeating specific steps and using some flag variables, but every logical problem can be solved and made to conform to structured rules by using the three structures: sequence, selection, and loop.

Glossary

A

abstract class—A class from which concrete objects cannot be instantiated, but which can serve as a basis for inheritance.

abstract data type (ADT)—A programmer-defined type, such as a class, as opposed to a built-in data type.

abstraction—The process of paying attention to important properties while ignoring nonessential details.

access specifier—The adjective that defines the type of access outside classes will have to an attribute or method.

accessibility—Describes screen design issues that make programs easier to use for people with physical limitations.

accessor method—An instance method that returns a value from a field defined in a class. See also *get method*.

accumulator—A variable used to gather or accumulate values.

addresses—Numbers that identify computer memory and storage locations.

algorithm—The sequence of steps necessary to solve any problem.

alphanumeric values—The set of values that includes alphabetic characters, numbers, and punctuation.

ambiguous methods—Methods between which the compiler cannot distinguish because they have the same name and parameter types.

American Standard Code for Information Interchange (ASCII)—An eight-bit character-coding scheme used on many personal computers.

ancestors—The entire list of parent classes from which a class is derived.

AND decision—A decision in which two conditions must both be true for an action to take place.

animation—The rapid display of still images, each slightly different from the previous one, that produces the illusion of movement.

annotation symbol—A flowchart symbol used to hold comments; it is most often represented by a three-sided box connected with a dashed line to the step it explains.

app—A piece of application software; the term is frequently used for applications on mobile devices.

application software—All the programs that help users with tasks (for example, accounting

or word processing), in contrast to *system software*.

argument to a method—A value passed to a method in the method call.

array—A series or list of variables in computer memory, all of which have the same name but are differentiated with subscripts.

ascending order—Describes the arrangement of data items from lowest to highest.

assignment operator—The equal sign; it always requires the name of a memory location on its left side.

assignment statement—A statement that stores a value on its right side to the named location on its left side.

attribute—One field in an object; an instance variable.

B

backup file—A copy that is kept in case values need to be restored to their original state.

base 2—Describes numbers created using the binary numbering system.

base 10—Describes numbers created using the decimal numbering system.

base 16—Describes numbers created using the hexadecimal numbering system.

base case—Describes the input that halts a recursive method; also called a *terminating case*.

base class—A class that is used as a basis for inheritance.

batch processing—Processing that performs the same tasks with many records in sequence.

binary files—Files that contain data that has not been encoded as text.

binary language—A computer language represented using a series of 0s and 1s.

binary numbering system—The numbering system based on two digits in which column values are multiples of 2.

binary operator—An operator that requires two operands—one on each side.

binary search—A search that compares a target value to an element in the middle of a sorted list, and then determines whether it should continue higher or lower to find the target.

bit—A binary digit; a unit of storage equal to one-eighth of a byte.

black box—The analogy that programmers use to refer to the details of hidden methods.

block—A group of statements that execute as a single unit.

Boolean expression—An expression that represents only one of two states, usually expressed as true or false.

bubble sort—A sorting algorithm in which list elements are arranged in ascending or descending order by comparing items in pairs and swapping them when they are out of order.

byte—A storage measurement equal to eight bits.

C

call a module—To use a module's name to invoke it, causing it to execute.

camel casing—A naming convention in which the initial letter is lowercase, multiple-word names are run together, and each new word within the name begins with an uppercase letter.

cascading `if` statement—A series of nested `if` statements.

case structure—A structure that tests a single variable against multiple values, providing separate actions for each logical path.

catch an exception—To receive an exception from a throw so it can be handled.

`catch` block—A segment of code written to handle an exception that might be thrown by the `try` block that precedes it.

central processing unit (CPU)—The computer hardware component that processes data.

character—A letter, number, or special symbol such as *A*, *7*, or *$*.

child class—A derived class.

child file—A copy of a file after revision.

class—A group or collection of objects with common attributes.

class client or **class user**—A program or class that instantiates objects of another prewritten class.

class definition—A set of program statements that define the fields and methods of a class.

class diagram—A tool for describing a class that consists of a rectangle divided into three sections that show the name, data, and methods of a class.

class method—A static method; class methods are not instance methods and they do not receive a `this` reference.

client—A program or other method that uses a method.

closing a file—An action that makes a file no longer available to an application.

cloud—Remote computers accessed through the Internet.

coding the program—The act of writing the statements of a program in a programming language.

cohesion—A measure of how a method's internal statements are focused to accomplish the method's purpose.

command line—The location on a computer screen where entries are typed to communicate with the computer's operating system.

compiler—Software that translates a high-level language into machine language and identifies syntax errors. A compiler is similar to an interpreter; however, a compiler translates all the statements in a program prior to executing them.

composition—The technique of placing an object within an object of another class; also known as a *whole-part relationship*.

compound condition—An evaluation with multiple parts.

computer file—A collection of data stored on a nonvolatile device in a computer system.

computer memory—The temporary, internal storage within a computer.

computer system—A combination of all the components required to process and store data using a computer.

conditional `AND` operator—A symbol used to combine decisions so that two or more conditions must be true for an action to occur. Also called an *AND operator*.

conditional `OR` operator—A symbol used to combine decisions when any one condition can be true for an action to occur. Also called an *OR operator*.

constructor—An automatically called method that instantiates an object.

container—One of a class of objects whose main purpose is to hold other elements—for example, a window.

control break—A temporary detour in the logic of a program for special group processing.

control break field—A variable that holds the value that signals a special processing break in a program.

control break program—A program in which a change in the value of a variable initiates special actions or processing.

control break report—A report that lists items in groups. Frequently, each group is followed by a subtotal.

conversion—The set of actions an organization must take in order to switch over to using a new program or system.

counter—Any numeric variable used to count the number of times an event has occurred.

counted loop—A loop whose repetitions are managed by a counter; also called a *counter-controlled loop*.

coupling—A measure of the strength of the connection between two program methods.

D

data dictionary—A list of every variable name used in a program, along with its type, size, and description.

data hierarchy—Represents the relationship between databases, files, records, fields, and characters.

data items—All the text, numbers, and other information processed by the computer.

data type—The characteristic of a program value; a variable or constant's data type describes the kind of values it can hold and the types of operations that can be performed with it.

database—A logical container that holds related data that can easily be retrieved to serve the information needs of an organization.

dead path—A logical path that can never be traveled; also called *unreachable path*.

deadlock—A flaw in multithreaded programs in which two or more threads wait for each other to execute.

debugging—The process of finding and correcting program errors.

decimal numbering system—The numbering system based on 10 digits in which column values are multiples of 10.

decision structure—A program structure in which a condition is tested, and, depending on the result, one of two courses of action is taken. Then, no matter which path is followed, the paths join and the next task executes.

decision symbol—A symbol that represents a decision in a flowchart; it is shaped like a diamond.

declaration—A statement that names a variable and its data type.

decrement—To change a variable by decreasing it by a constant value, frequently 1.

default constructor—A constructor that requires no arguments.

default input and output devices—Hardware devices that do not require opening; usually they are the keyboard and monitor, respectively.

defensive programming—A technique in which programmers try to prepare for all possible errors before they occur.

definite loop—A loop for which the number of repetitions is a predetermined value.

delimiter—a character, such as a comma, used to separate fields in a data file.

derived class—An extended class.

descending order—Describes the arrangement of data items from highest to lowest.

desk-checking—The process of walking through a program solution on paper.

destructor—An automatically called method that contains the actions required when an instance of a class is destroyed.

detail loop tasks—The steps that are repeated for each set of input data.

direct access files—Random access files.

directories—Organization units on storage devices that can contain multiple files as well as additional directories; also called *folders*.

documentation—All of the supporting material that goes with a program.

DOS prompt—The command line in the DOS operating system.

`do-until` loop—A posttest loop that iterates until its controlling condition is false.

`do-while` loop—A posttest loop in which the body executes before the loop control variable is tested.

dual-alternative `if` or **dual-alternative selection**—A selection structure that defines one action to be taken when the tested condition is true, and another action to be taken when it is false.

dummy value—A preselected value that stops the execution of a program.

E

echoing input—The act of repeating input back to a user either in a subsequent prompt or in output.

element—A separate array variable.

`else` clause—A part of a selection structure that holds the action or actions that execute only when the tested Boolean expression is false.

encapsulation—The act of containing a task's instructions and data in the same method.

end-of-job task—A step at the end of a program to finish an application.

end-structure statement—A statement that designates the end of a pseudocode structure.

`eof`—An end-of-data file marker, short for *end of file*.

event—An occurrence that generates a message sent to an object.

event-driven or **event-based**—Describes programs and actions that occur in response to user-initiated events such as clicking a mouse button.

exception—The generic term used for an error in object-oriented languages.

exception-handling techniques—The object-oriented techniques for managing errors.

execute—To have a computer use a written and compiled program; also called *run*.

Extended Binary Coded Decimal Interchange Code (EBCDIC)—An eight-bit character-coding scheme used on many larger computers.

extended class—A class that is extended from a base class; also called a *derived class*.

external documentation—All of the external material that programmers develop to support a program; in contrast to *program comments*, which are internal program documentation.

F

facilitator—A work method.

field—A single data item that represents a single attribute of a record or class, such as `lastName`, `streetAddress`, or `annualSalary`.

file—A group of records that go together for some logical reason.

filename—an identifying name given to a computer file that frequently describes the contents.

filename extension—a group of characters added to the end of a filename that indicates the file type.

flag—A variable that indicates whether some event has occurred.

floating-point value—A fractional, numeric variable that contains a decimal point.

flowchart—A pictorial representation of the logical steps it takes to solve a problem.

flowline—An arrow that connects the steps in a flowchart.

folders—Organization units on storage devices that can contain multiple files as well as additional folders; also called *directories*.

`for` statement—A statement that can be used to code definite loops; also called a *for loop*. The statement contains a loop control variable that it automatically initializes, evaluates, and alters.

forcing data—Overriding a data value by setting it to a specific, default value.

formal parameters—The variables in a method declaration that accept values from the actual parameters.

fragile—Describes classes that depend on field names from parent classes and are prone to errors.

functional cohesion—The extent to which all operations in a method contribute to the performance of only one task.

functional decomposition—The act of reducing a large program into more manageable modules.

G

garbage—Describes the unknown value stored in an unassigned variable.

get method—An instance method that returns a value from a field defined in a class. See also *accessor method*.

gigabyte—A billion bytes.

GIGO—Acronym for *garbage in, garbage out*; it means that if input is incorrect, output is worthless.

global—Describes variables that are known to an entire program.

goto-less programming—A name to describe structured programming, because structured programmers do not use a "go to" statement.

graphical user interface (GUI)—A program interface that uses screens to display program output and allows users to interact with a program in a graphical environment.

H

hardware—The equipment of a computer system.

has-a relationship—A whole-part relationship; the type of relationship that exists when using composition.

help method—A work method.

hexadecimal numbering system—The numbering system based on 16 digits in which column values are multiples of 16.

hierarchy chart—A diagram that illustrates program modules' relationships to each other.

high-level programming language—A programming language that is English-like, as opposed to a low-level programming language.

housekeeping tasks—Tasks that must be performed at the beginning of a program to prepare for the rest of the program.

Hungarian notation—A naming convention that stores a data type or other information as part of a name.

I

I/O symbol—An input/output symbol.

icons—Small pictures on a screen that help a user navigate a system.

identifier—A program component's name.

IDE—The acronym for Integrated Development Environment, which is the visual development environment in some programming languages.

`if-then`—A single-alternative selection structure; it contains a tested Boolean expression and an action that is taken only when the expression is true.

`if-then` clause—The part of a decision that holds the resulting action when the Boolean expression in a selection structure is true.

`if-then-else`—Another name for a dual-alternative selection structure.

implementation—The body of a method; the statements that carry out the tasks of a method.

implementation hiding—A programming principle that describes the encapsulation of method details.

in bounds—Describes an array subscript that is within the range of acceptable subscripts for its array.

in scope—The characteristic of variables and constants that describes the extent to which they are available for use.

inaccessible—Describes any field or method that cannot be reached.

increment—To change a variable by adding a constant value to it, frequently 1.

indefinite loop—A loop for which the number of executions cannot be predicted when the program is written.

index—A term used to describe storage in which a list of key fields is paired with the storage address for the corresponding data record. With arrays, the term is also sometimes used as a synonym for *subscript*.

indirect relationship—The relationship between parallel arrays in which an element in the first array does not directly access its corresponding value in the second array.

infinite loop—A repeating flow of logic without an ending.

information—Processed data.

information hiding or **data hiding**—The concept that other classes should not alter an object's attributes—only the methods of an object's own class should have that privilege.

inheritance—The process of acquiring the traits of one's predecessors.

initializing a variable—The act of assigning the first value to a variable, often at the same time that the variable is created.

inner loop—When loops are nested, the loop that is contained within the other loop.

input—Describes the entry of data items into computer memory using hardware devices such as keyboards and mice.

input symbol—A symbol that indicates an input operation and is represented in flowcharts as a parallelogram.

input/output symbol—A parallelogram used within flowcharts to indicate input or output operations; also called an *I/O symbol*.

insertion sort—A sorting algorithm in which each list element is examined one at a time; if an element is out of order relative to any of the items earlier in the list, each earlier item is moved down one position and then the tested element is inserted.

instance—An existing object or tangible example of a class.

instance method—A method that operates correctly yet differently for each class object; an instance method is nonstatic and receives a `this` reference.

instance variables—The data components that belong to every instantiated object.

instant access files—Random access files in which records must be accessed immediately.

instantiate—To create an object.

instantiation—An instance of a class; an object.

integer—A whole number.

integrated development environment (IDE)—A software package that provides an editor, compiler, and other programming tools.

interactive program—A program in which a user makes direct requests or provides input while a program executes.

interactivity diagram—A diagram that shows the relationship between screens in an interactive GUI program.

interface to a method—A method's return type, name, and arguments; the part of a method that a client sees and uses.

internal documentation—Documentation within a program. See also *program comments*.

interpreter—Software that translates a high-level language into machine language and identifies syntax errors. See *compiler*.

IPO chart—A program development tool that delineates input, processing, and output tasks.

is-a relationship—The relationship between an object and each of the classes in its ancestry.

iteration—The action of repeating.

K

kebob case—A term sometimes used to describe the naming convention in which dashes separate parts of a name.

key field—The field whose contents make a record unique among all records in a file.

keywords—The limited word set that is reserved in a language.

kilobyte—Approximately 1,000 bytes.

L

left-to-right associativity—Describes operators that evaluate the expression to the left first.

libraries—Stored collections of classes that serve related purposes.

linear search—A search through a list from one end to the other.

linked list—A list that contains an extra field in every record that holds the physical address of the next logical record.

listener—In object-oriented programming, an object that is interested in an event and responds to it.

local—Describes variables that are declared within the method that uses them.

logic—The complete sequence of instructions that lead to a problem solution.

logical error—An error that occurs when incorrect instructions are performed, or when instructions are performed in the wrong order.

logical `NOT` operator—A symbol that reverses the meaning of a Boolean expression.

logical order—The order in which a list is used, even though it is not necessarily stored in that physical order.

loop—A structure that repeats actions while a condition continues.

loop body—The set of actions or statements within a loop.

loop control variable—A variable that determines whether a loop will continue.

loop structure—A structure that repeats actions while a test condition remains true.

loose coupling—A relationship that occurs when methods do not depend on others.

low-level language—A programming language not far removed from machine language, as opposed to a high-level programming language.

lower camel casing—Another name for the camel casing naming convention.

lvalue—The memory address identifier to the left of an assignment operator.

M

machine language—A computer's on/off circuitry language; the low-level language made up of 1s and 0s that the computer understands.

magic number—An unnamed numeric constant whose purpose is not immediately apparent.

main program—A program that runs from start to stop and calls other modules; also called a *main program method*.

mainline logic—The overall logic in a program's main module.

maintenance—All the improvements and corrections made to a program after it is in production.

making declarations—The process of naming program variables and assigning a type to them.

master file—A file that holds complete and relatively-permanent data.

matrix—A term sometimes used by mathematicians to describe a two-dimensional array.

mean—The arithmetic average.

median—The value in the middle position of a list when the values are sorted.

megabyte—A million bytes.

merging files—The act of combining two or more files while maintaining the sequential order.

method—A program module that contains a series of statements that carry out a task.

method body—The set of all the statements in a method.

method header—A program component that precedes a method body; the header includes the method identifier and, possibly, other necessary information, such as a return type and parameter list.

method `return` statement—A statement that marks the end of the method and identifies the point at which control returns to the calling method.

method type—The data type of a method's return value.

mixed case with underscores—A naming convention similar to snake casing, in which words are separated with underscores, but new words start with an uppercase letter.

modularization—The process of breaking down a program into modules.

module—A program unit, also called *subroutine*, *procedure*, *function*, or *method*.

module body—The part of a module that contains all of its statements, as opposed to the *module header*.

module header—The part of a module that includes the module identifier and, possibly, other necessary identifying information, as opposed to the *module body*.

module `return` statement—The part of a module that marks its end and identifies the point at which control returns to the program or module that called the module.

multidimensional arrays—Lists with more than one dimension; more than one subscript is required to access an element.

multiple inheritance—The ability to inherit from more than one class.

multithreading—Using multiple threads of execution.

mutator method—An instance method that sets or changes the values of a data field within a class object. See also *set method*.

N

named constant—A named memory location whose value never changes after assignment; conventionally, constants are named using all capital letters.

nested decision—A decision within the `if-then` or `else` clause of another decision; also called a *nested if*.

nested loop—A loop structure within another loop structure.

nesting structures—Placing a structure within another structure.

nibble—A storage measurement equal to four bits, or half a byte.

nondefault constructor—A constructor that requires at least one argument. See also *parameterized constructor*.

nonstatic methods—Methods that exist to be used with an object; they are instance methods and they receive a `this` reference.

nonvolatile—Describes storage that retains its contents when power is lost.

NOT operator—A symbol that reverses the meaning of a Boolean expression.

null case—The branch of a decision in which no action is taken; also called *null branch*.

numeric—Describes data that consists of numbers and with which numeric operations can be performed.

numeric constant—A specific numeric value; also called *literal numeric constant*.

numeric variable—A variable that holds numeric values.

O

object—One tangible example of a class; an instance of a class.

object code—Program statements that have been translated into machine language.

object dictionary—A list of the objects used in a program, including which screens they are used on and whether any code, or script, is associated with them.

object-oriented programming (OOP)—A programming model that focuses on components and data items (objects) and describes their attributes and behaviors, as opposed to procedural programming.

one-dimensional array—A list accessed using a single subscript; also called a *single-dimensional array*.

opening a file—The process of locating a file on a storage device, physically preparing it for reading, and associating it with an identifier inside a program.

operand—A value used by an operator.

operating system—The software that runs a computer and manages its resources.

OR decision—A decision that contains two or more conditions; if at least one condition is met, then the resulting action takes place.

order of operations—Describes the rules of precedence.

out of bounds—Describes an array subscript that is not within the range of acceptable subscripts.

outer loop—The loop that contains a nested loop.

output—Describes the operation of retrieving information from memory and sending it to a device, such as a monitor or printer, so that people can view, interpret, and work with the results.

output symbol—A symbol that indicates an output operation and is represented as a parallelogram in flowcharts.

overhead—All the resources and time required by an operation.

overload a method—To create multiple methods with the same name but different parameter lists.

overloading—Supplying diverse meanings for a single identifier.

overriding—The mechanism by which a child class method is used by default when a parent class contains a method with the same signature.

P

packages—Another name for libraries in some languages.

page schematic—A wireframe.

parallel arrays—Two or more arrays in which each element in one array is associated with the element in the same relative position in the other array or arrays.

parameter list—All the data types and parameter names that appear in a method header.

parameter to a method—A data item defined in a method header that accepts data passed into the method from the outside.

parameterized constructor—A constructor that requires at least one argument. See also *nondefault constructor*.

parent class—A base class.

parent file—A copy of a file before revision.

Pascal casing—A naming convention in which the initial letter is uppercase, multiple-word names are run together, and each new word within the name begins with an uppercase letter.

passed by reference—Describes a method parameter that represents the item's memory address.

passed by value—Describes a variable that has a copy of its value sent to a method and stored in a new memory location accessible to the method.

path—The combination of a file's disk drive and the complete hierarchy of directories in which a file resides.

permanent storage device—A hardware device that holds nonvolatile data; examples include hard disks, DVDs, Zip disks, USB drives, and reels of magnetic tape.

physical order—The order in which a list is actually stored even though it might be accessed in a different logical order.

pixel—A picture element; one of the tiny dots of light that form a grid on a monitor.

polymorphism—The ability of a method to act appropriately depending on the context.

populate an array—To assign values to array elements.

portable—Describes a module that can more easily be reused in multiple programs.

posttest loop—A loop that tests its controlling condition after each iteration, meaning that the loop body executes at least one time.

precedence—The quality of an operation that determines the order in which it is evaluated.

pretest loop—A loop that tests its controlling condition before each iteration, meaning that the loop body might never execute.

priming input or **priming read**—The statement that reads the first input data record prior to starting a structured loop.

primitive data types—In a programming language, simple number and character types that are not class types.

private access—A privilege of class members in which data or methods cannot be used by any method that is not part of the same class.

procedural programming—A programming technique that focuses on procedures or actions, as opposed to object-oriented programming.

processing—To organize data items, check them for accuracy, or perform mathematical operations on them.

processing symbol—A symbol represented as a rectangle in flowcharts.

program—Sets of instructions for a computer.

program code—The set of instructions a programmer writes in a programming language.

program comment—A nonexecuting statement that programmers place within code to explain program statements in English. See also *internal documentation*.

program development cycle—The steps that occur during a program's lifetime, including planning, coding, translating, testing, producing, and maintaining the program.

program level—The level at which global variables are declared.

programming—The act of developing and writing programs.

programming language—A language, such as Visual Basic, C#, C++, Java, or COBOL, used to write programs.

prompt—A message that is displayed on a monitor, asking the user for a response.

property—A method that gets and sets a field value using simple syntax.

`protected` access specifier—A specifier used when outside classes should not be able to use a data field unless they are children of the original class.

pseudocode—An English-like representation of the logical steps it takes to solve a problem.

public access—A privilege of class members in which other programs and methods can use the specified data or methods within a class.

pure polymorphism—The situation in which one method implementation can be used with a variety of arguments in object-oriented programming.

R

random access files—Files that contain records that can be located in any physical order and accessed directly.

random access memory (RAM)—Temporary, internal computer storage.

random-access storage device—A storage device, such as a disk, from which records can be accessed in any order.

range check—The comparison of a variable to a series of values that mark the limiting ends of ranges.

reading from a file—The act of copying data from a file on a storage device into RAM.

real numbers—Floating-point numbers.

real-time—Describes applications that require a record to be accessed immediately while a client is waiting.

records—Groups of fields that go together for some logical reason.

recursion—A programming event that occurs when a method is defined in terms of itself.

recursive cases—Describe the input values that cause a recursive method to execute again.

recursive method—A method that calls itself.

registering components—The act of signing up components so they can react to events initiated by other components.

relational comparison operator—A symbol that expresses Boolean comparisons. Examples include: =, >, <, >=, <=, and <>.

reliability—The feature of modular programs that ensures a module has been tested and proven to function correctly.

remainder operator—An arithmetic operator used in some programming languages; when its operands are divided, it provides the remainder.

repetition—Another name for a loop structure.

return type—The data type for any value a method returns.

reusability—The feature of modular programs that allows individual modules to be used in a variety of applications.

right-associativity and **right-to-left associativity**—Descriptions of operators that evaluate the expression to the right first.

rules of precedence—Rules that dictate the order in which operations in the same statement are carried out.

run—To have a computer use a written and compiled program; also called *execute*.

S

screen blueprint—A wireframe.

script—A procedural module that depends on user-initiated events in object-oriented programs.

scripting language—A language, such as Python, Lua, Perl, or PHP, used to write programs that are typed directly from a keyboard and are stored as text, rather than as binary executable files. Also called *scripting programming languages* or *script languages*.

selection sort—A sorting algorithm in which list values are divided into sorted and unsorted sublists; the unsorted sublist is repeatedly examined for its smallest value, which is then moved to the end of the sorted sublist.

selection structure—A program structure that tests a condition and takes one of two courses of action depending on the result. Then, no matter which path is followed, the paths join and the next task executes.

self-documenting—Describes programs that contain meaningful and descriptive identifiers.

semantic error—An error that occurs when a correct word is used in an incorrect context.

sentinel value—A value that represents an entry or exit point.

sequence structure—A program structure that contains steps that execute in order. A sequence can contain any number of tasks, but there is no chance to branch off and skip any of the tasks.

sequential file—A file in which records are stored one after another in some order.

sequential order—The arrangement of records when they are stored one after another on the basis of the value in a particular field.

set method—An instance method that sets or changes the values of a data field within an object. See also *mutator method*.

short-circuit evaluation—A logical feature in which each part of a larger expression is evaluated only as far as necessary to determine the final outcome.

signature—A method's name and parameter list.

single-alternative `if` or single-alternative selection—A selection structure in which action is required for only one branch of the decision, also called an `if-then` structure.

single-dimensional array—A list accessed using a single subscript; also called *one-dimensional array*.

single-level control break—A break in the logic of a program based on the value of a single variable.

sinking sort—A bubble sort.

size of the array—The number of elements an array can hold.

snake casing—A naming convention in which parts of a name are separated by underscores.

software—Programs that tell the computer what to do.

sorting—The process of placing records in order by the value in a specific field or fields.

source code—The readable statements of a program, written in a programming language; they are later translated into object code.

source of an event—The component (for example, a button) from which an event (such as a click) is generated.

spaghetti code—Snarled, unstructured program logic.

stack—A memory location that holds the memory addresses to which a program's logic should return after a method executes.

stacking structures—To attach program structures end to end.

starvation—A flaw in multithreaded programs in which a thread is abandoned because other threads occupy all the computer's resources.

state—The set of all the values or contents of a class's instance variables.

static methods—Methods for which no object needs to exist; static methods are not instance methods and they do not receive a `this` reference.

step value—A number used to alter a loop control variable on each pass through a loop.

storage device—A hardware apparatus that holds information for later retrieval.

storyboard—A picture or sketch of screens the user will see when running a program.

string—Describes data that is nonnumeric.

string constant or **literal string constant**—A specific group of characters enclosed within quotation marks.

string variable—A variable that can hold text that includes letters, digits, and special characters, such as punctuation marks.

structure—A basic unit of programming logic; each structure is a sequence, selection, or loop.

structured programs—Programs that follow the rules of structured logic.

subclass—A derived class.

subscript—A number that indicates the position of an element within an array; also called an *index*.

summary report—A report that lists only totals, without individual detail records.

sunny day case—A program execution in which nothing goes wrong.

superclass—A base class.

swap values—To exchange the values of two variables.

syntax—The rules of a language.

syntax error—An error in language or grammar.

system software—The programs that manage a computer, in contrast to *application software*.

T

table—A term sometimes used by mathematicians to describe a two-dimensional array.

temporary variable—A working variable that holds intermediate results during a program's execution; also called *work variable*.

terminal symbol—A symbol used at each end of a flowchart; its shape is a lozenge; also called a *start/stop symbol*.

terminating case—Describes the input that halts a recursive method; also called the *base case*.

text editor—A program used to create simple text files; it is similar to a word processor, but without as many features.

text files—Files that contain data that can be read in a text editor.

`this` reference—An automatically created variable that holds the address of an object and passes it to an instance method whenever the method is called.

thread—The flow of execution of one set of program statements.

thread synchronization—A set of techniques that coordinates threads of execution to help avoid potential multithreading problems.

three-dimensional arrays—Arrays in which each element is accessed using three subscripts.

throw an exception—To pass an exception out of a block where it occurs, usually to a block that can handle it.

`throw` statement—An object-oriented programming statement that sends an `Exception` object out of a method or code block to be handled elsewhere.

tight coupling—A problem that occurs when methods depend excessively on each other; it makes programs more prone to errors.

transaction file—A file that holds temporary data used to update a master file.

trivial expression—An expression that always evaluates to the same value.

truth table—A diagram used in mathematics and logic to help describe the truth of an entire expression based on the truth of its parts.

try—To execute code that might throw an exception.

`try` block—A block of code that attempts to execute while acknowledging that an exception might occur.

two-dimensional arrays—Arrays that have rows and columns of values accessed using two subscripts.

type-safety—The feature of programming languages that prevents assigning values of an incorrect data type.

U

unary operator—An operator that uses only one operand.

Unicode—A 16-bit character-coding scheme.

unnamed constant—A literal numeric or string value.

unstructured programs—Programs that do *not* follow the rules of structured logic.

update a master file—To modify the values in a master file based on transaction records.

upper camel casing—Another name for the Pascal casing naming convention.

user-defined type or **programmer-defined type**—A type that is not built into a language but is created by an application's programmer.

users or **end users**—People who work with, and benefit from, computer programs.

V

validate data—The process of ensuring that data items are meaningful and useful; are correct data types, fall within an acceptable range, or are reasonable.

variable—A named memory location of a specific data type, whose contents can vary or differ over time.

visible—A characteristic of data items that means they "can be seen" only within the method in which they are declared.

visual development environment—A programming environment in which programs are created by dragging components such as buttons and labels onto a screen and arranging them visually.

void method—A method that returns no value.

volatile—A characteristic of internal memory in which its contents are lost every time the computer loses power.

W

`while loop` or **`while`…`do` loop**—A loop in which a process continues while some condition continues to be true.

whole-part relationship—A relationship in which an object of one class is contained within an object of another class; also known as *composition*.

wireframe—A picture or sketch of a screen the user will see when running a program.

work method—A method that performs tasks within a class.

writing to a file—The act of copying data from RAM into persistent storage.

X

x-axis—An imaginary line that represents horizontal positions in a screen window.

x-coordinate—A position value that increases from left to right across a screen window.

Y

y-axis—An imaginary line that represents vertical positions in a screen window.

y-coordinate—A position value that increases from top to bottom across a screen window.

Index

Note: Page numbers in **boldface** type indicate where key terms are defined.

Special Characters

* (asterisk), 48
{} (curly braces), 200
= (equal sign), 45
! (exclamation point), 132, 154
> (greater than operator), 130
>= (greater-than-or-equal-to operator), 132–131
< (less than operator), 130
<= (less-than-or-equal-to operator), 130
- (minus sign), 47, 50, 439
<> (not-equal-to operator), 130
() (parentheses), 48, 49, 51
% (percent sign), 51
+ (plus sign), 47, 50, 439
; (semicolon), 200
/ (slash), 48, 50
~ (tilde), 472

A

abbreviations, caution about use, 69
abstract classes, **485**
abstract data types (ADTs), **429**
abstraction, **52**, 52–53
accessibility, **516**
accessor methods, **434**, 434–435
access specifiers, **437**, 439–440
  private and public, **437**, 437–440
  protected, **482**, 482–485
accumulators, **205**, 206, 207
actions, user-initiated, 511
addition operator (+), 47, 50
addresses, **352**
ADTs. *See* abstract data types (ADTs)
advanced data handling concepts
  bubble sort algorithm, 324–340
  indexed files and linked lists, 351–356
  multidimensional arrays, 345–351
  multifield records sorting, 340–342
  sorting data, 322–323
advanced modularization techniques
  method design issues, 402–405
  methods, 367–368
  overloading methods, **394**, 394–400
  predefined methods, 400–402
  recursion, **405**, 405–409
  requiring parameters, creating methods, 371–379
  returning value, creating methods, 379–385
algorithms, **10**, **324**
alphanumeric values, **40**
ambiguous methods, **397**

ancestors, **479**
AND decisions, 137–138
  avoiding common errors, 141–143
  nested, 136, 138–143
`AND` operator, **139**, 139–141
  combining with OR operator, precedence, 160–163
animation, **528**, 528–530
  free images, 530
annotation symbol, **67**
application software (app), **2**
arguments (argument to the method), **371**
arithmetic operations, 47–51
arrays, **228**
  constants, 239
  objects, 453–455
  parallel, 246–250, 252
  populating, 229
  processing using `for` loops, 260
  remaining within array bounds, 255–258
  replacing nested decisions, 231–238
  searching for exact matches, 240–243
  searching for range matches, 250–254
ascending order, **280**
assignment operators, **45**, 48
assignment statements, **45**
asterisk (*), multiplication operator, 48
attributes
  objects, **421**

B

backup files, 282
  generations, 300
base 16, 7
base case/terminating case, **406**
base classes, **478**, 478–479
batch processing, **308**
binary files, **273**
binary language, **4**
binary operators, **45**
binary searches, **249**
black box, **403**
blocks, **94**
body, modules, 54
boolean expressions, 125. *See also* AND decisions; OR decisions
  completeness, 142, 147–148
  inadvertently trivial, avoiding, 142–143
  structured selections, 164
bounds, subscripts, 255–258
bubble sort algorithm, **324**
  mainline logic, 326
  procedures, 325
  reducing unnecessary comparisons, 336
  refining, to eliminate unnecessary passes, 336–340
  sorting, variable size list, 332–336
  swapping values, 324–325
built-in exceptions, 493
`Button` class, 512–513
button, GUIs, 511, 512
bytes, **274**

C

calling a module, 51
camel casing, **42**, 70
cascading if statements, **134**
`case` structure, 112, 163–165
`catch` blocks, **492**
catching the exception, 491
central processing unit (CPU), **2**
  multithreading, 525–528
characters, **275**
check boxes, GUIs, 511, 512
child classes, **478**, 479
  overriding parent class methods in child classes, 486
child files, 282, 300

clarity
  statements, 71–72
  structure use, 106
class(es), **421**, 421–424
  abstract, **485**
  base, **478**, 478–479
  `Button`, 512–513
  clients (class users), **424**
  definitions, **428**, 428–430
    defining, 428–430
  derived (extended), **478**, 478–479
  diagrams, **430**, 430–433
  fragile, **485**
  instances, **421**
  methods, **447**, 447–448
  organization, 440–441
  predefined, 487–488
  subclasses, **478**, 479
  superclasses, **478**
clients
  class, **424**
closing files, **281**
cloud, **3**
code
  object, 4
  program, 3
  pseudocode, 14–16
  reliable, 486
  source, 4
  spaghetti, 88–90
  trying, **491**
coding the program, 3, 10–11
cohesion, **403**
  functional, 58
command line, **25**, 508
comments, 67–69
compilers, **4**
composition, **474**, 474–475
compound conditions, **134**
computer files, **273**, 273–275, 281
  backup, 282, 300
  binary, 273
  child, 282, 307
  closing, 281
  data hierarchy, 277
  declaring file identifiers, 277–278
  direct access, 308
  grandparent, 300
  instant access, 308
  master, 299–307
  merging, 290–299
  opening, 278
  organizing, 274–275
  parent, 282, 300
  program performing file operations, 282–284
  random access, 308–309
  reading and processing data from, 278–281
  sequential, 280, 290–299
  size, 274
  text, 273
  transaction, 299
  writing data, 281
computer memory, 4, 7
  RAM, **4**
  temporary, 278
computer systems, 2, **2**
conditional operators
  logical operators *vs.*, 141
  `AND` operator, **139**
  `OR` operator, **147**
consistency of data, validating, 213–214
constants
  as array element values, 239
  as array subscripts, 239
  declaring within modules, 58–60
  global, 60
  local, 60
  naming, 69–70

constants (*continued*)
numeric (literal numeric), 39
as size of array, 239–240
string (literal string), 39
unnamed, **40**, 239
constructors, **465**, 465–471
default, **465**, 466–468
nondefault (parameterized), **465**, 468–469
overloading, 469–471
containers, **522**
control break(s), **286**
fields, **286**
programs, **285**
reports, 285, **285**, 286, 290
single-level, 286
conversion, **14**
counted loops, **179**
counter(s), **181**
counter-controlled loops, **179**
coupling, **404**
curly braces (`{}`), statements, 200

D

databases, **277**
data dictionaries, **70**
data hiding, **427**, 437
data hierarchy, **275**, 275–277
data items, **2**
visible (in scope), 59
data types, **39**, 41
abstract, 429
integers, 50–51
primitive, **430**
validating, 212–213
variables, 45–46
deadlock, **527**
dead paths, **157**
debugging, **13**
decision making process, 232, 233, 235
decision structure, **91**, 91–92, 101, 137. *See also* selection structure
decision symbol, **21**
declarations, **41**
decrementing, **181**
default constructors, **465**, 466–468
default input and output devices, **282**
defensive programming, **209**
definite loops, **181**
derived classes, **478**, 478–479
descending order, **280**
desk-checking, **10**, 66
destructors, **472**, 472–473
detail loop tasks, **61**
dimmed screen options, 515
direct access files, **310**
directories, **274**, 275
displaying, definition, 281
division operator (/), 48, 49
documentation, **9**
external, 67
internal, 67
`do` loop, 112
DOS prompt, 508
doubly linked list, 355
`do-until` loop, 204
`do-while` loop, **201**
drawing, 17–18
symbols, 15–20, 55, 67, 201
dual-alternative ifs (dual-alternative selections), **92**
dummy values, **22**

E

echoing input, **73**, 75
efficiency
AND decisions, 137–138
OR decisions, 145–146
searching arrays, 250–252
structure use, 106
elements
constants as values, 239
`else` clauses, **126**

encapsulation, **58**, **426**, 426–427
end-of-job tasks, **61**
end-structure statement, **91**, 92
end users, **8**
`eof`, **22**
equal sign (=)
  assignment operator, **45**
  equivalency operator, 130
event(s), 509
  source, **510**
event-driven (event-based) program(s), 508–510, **509**, 517–525
  defining connections between user screens, 520
  planning logic, 520–525
  storyboards, 518–519
  wireframes (page schematics; screen blueprints), 518
exception(s), **491**
  built-in, 493
  catching, **491**
  throwing, **491**
  user-created, 493
exception handling, 488–494, **491**
  built-in and user-created exceptions, 493
  drawbacks to traditional techniques, 489–490
  object-oriented model, 491–493
exclamation point (!), `NOT` operator, 154
executing a program, **4**
extended classes, **478**, 478–479
external documentation, **67**

F

facilitators, **435**, 435–436
fields, **275**, 423, **423**
  inaccessible, **482**
  private, parent classes, accessing, 481–485
flags, **243**
floating-point numeric variables, **39**
flowcharts, **15**
flowlines, **17**
folders, **274**, 275
forcing, **211**
`for` loops, **199**, 199–201
  processing arrays, 258–260
`for` statement. *See* `for` loop
fragile classes, **485**
functional cohesion, **58**, **403**

G

garbage, **46**
garbage in, garbage out (GIGO), **209**, 489
generations, backup files, 300
get methods, **434**, 434–435
gigabytes, **274**
GIGO. *See* garbage in, garbage out (GIGO)
global data item, **367**
global variables and constants, **60**
goto-less programming, **106**
grandparent files, 300
graphical user interfaces (GUIs), **25**, 25–26
  accessibility, **516**
  attractiveness, readability, and nondistracting nature, 515
  components, 511–514
  design, 514–516
  forgiving nature, 516
  naturalness and predictability, 514–515
  objects, creating, 487–488
  principle of, 516
  user customization, 516
grayed screen options, 515
greater-than operator (>), 130
greater-than-or-equal-to operator (>=), 130, 131
GUIs. *See* graphical user interfaces (GUIs)

H

hardware, **2**
has-a relationships, 474–475
headers, modules, 54

help methods, **435**, 435–436
hierarchy charts, **64**, 64–66
high-level programming languages, **11**
housekeeping tasks, **60**
Hungarian notation, **43**

I

icons, **508**
identifiers, 40, 69–71
`if-then` clauses, **126**
`if-then-else`, **91**
if-then selections, 125
implementation, **367**
implementation hiding, **402**, 402–403
inaccessible fields, **482**
in bounds, **255**
incrementing, **181**
indefinite loops, **181**, 181–183
index, **352**
indexed files
  addresses, **352**
  ID numbers with disk addresses, 353
  key field, **352**
indirect relationships, **248**
infinite loops, **19**
information, **3**
information hiding, **427**, 437
inheritance, **426**, 475–487
  accessing private fields and methods of parent classes, 481–485
  multiple, **485**
  overriding parent class methods in child classes, 486
  terminology, 478–481
  using to achieve good software design, 486–487
initializing the variable, **45**
  loop control variables, 191–192
inner loop, **185**
input(s), **2**
  echoing, **73**
  priming, 99–106
input devices, default, 281
input/output (I/O) symbol, **17**
input symbol, **17**
in-scope data items, **59**
insertion sort, **342**, 343
instance(s), classes, **421**
instance methods, 441–446, **443**
instance variables, 423, **423**
instant access files, **310**
instantiation, 421
instructions, repeating, 19–20
integer(s)
  data types, 50–51
  numeric variables, **39**
integrated development environments (IDE), **24**, 24–25, **488**
interactive programs, **308**, 310
interactivity diagrams, **520**
interface, to method, **403**
interpreters, **4**
IPO chart, **384**, 384–385
is-a relationships, **423**
iteration, **92**

K

kebob case, **43**
key field, **352**
key press event, 511
kilobytes, **274**

L

labels, GUIs, 511, 512
left mouse click event, 511
left-to-right associativity, **49**
less-than operator (<), 130
less-than-or-equal-to operator (<=), 130
libraries, **487**
linear searches, **241**
line breaks, confusing, avoiding, 71
linked lists, **353**, 353–356

list boxes, GUIs, 511, 512
listeners, **510**
literal numeric constants, **39**
literal string constants, **39**
local data item, **367**
local variables and constants, 60
logic, 5–7
  event-driven programs, planning, 520–525
  mainline, 54, 183–185
  planning, 10
  understanding loops in, 183–185
  unstructured, structuring and modularizing, 110–115
logical errors, **5**
logical operators
  conditional operators *vs.,* 141
  NOT operator, **153**, 153–154
logical order, **352**
loop(s), **19**, 101, 176–219
  accumulating totals, 205–208
  advantages, 177–179
  avoiding common mistakes, 190–198
  counted, 179
  counter-controlled, 179
  definite, 179–181
  `do`, 112
  `do-until`, 204
  `do-while`, 201
  `for`, 199–201, 258–260
  including statements in loop body thatábelong outside loop, 194–198
  indefinite, 181–183
  infinite, 19
  inner, 185
  loop control variables, 179–185
  mainline logic, 183–185
  nested, 185–190
  outer, 185
  posttest, 201
  pretest, 203
  reprompting, limiting, 209–211
  selections compared, 213–216
  validating data types, 211–212
  validating data using, 207–209
  validating reasonableness and consistency of data, 212–213
  `while`, 92–93, 112
loop body, **92**
loop control variables, **179**, 179–185
  decrementing, 181
  definite loops with counters, 179–181
  failing to initialize, 190–191
  incrementing, 181
  indefinite loops with sentinel values, 181–183
  neglecting to alter, 191–192
  program's mainline logic, 183–185
  using wrong type of comparison when testing, 192–194
loop structure, **92**, 92–93
loose coupling, **404**
Lovelace, Ada Byron, 27
low-level machine language, **11**
lvalues, **45**

M

machine language, **4**
  translating programs into, 11–12
magic numbers, **46**, 238
mainline logic, **54**
main program, **54**
maintenance, 14, **14**
  structure use, 106
making a decision, **27**
master files, **299**, 299–307
  updating, 299–307
matrix, 348
mean, **322**
median, **322**
megabytes, **274**

merging files, **290**, 290–299
methods, **367**
  body, **367**
  cohesion, **403**
  coupling reduction, 404
  get (accessor), **434**, 434–435
  header, **367**
  implementation, **367**
  implementation hiding, **402**, 402–403
  instance, 441–446, **443**
  nonstatic, **447**
  with no parameters, 368–371
  parent classes, accessing, 481–485
  passing objects to, 449–450
  require multiple parameters, 377–379
  requiring parameters, 371–379
  return a value, creating, 379–385
  returning objects from, 450–453
  return statement, **367**
  set (mutator), **433**, 433–434
  `sleep()`, 528, 529
  static (class), **447**, 447–448
  type, **379**
  work (help), **435**, 435–436
Microsoft Visual Studio IDE, **24**
minus sign (-)
  private access specifier, 439
  subtraction operator, 47, 50
mixed case with underscores, **43**
modularization, 51–58, **52**
  abstraction, **52**, 52–53
  declaring variables and constants within modules, 58–60
  mainline logic, 60–66
  multiple programmers, 53
  reusing work, 53–54
  unstructured logic, 110–115
module(s), **51**
  abstraction, **52**
  calling, **52**
  declaring variables and constants within, 58–60
  hierarchy charts, 64–66
  naming, 55
  reuse, 58
  structure use, 106
module body, **54**
module header, **54**
module return statement, **54**
modulo (modulus) operator (%), 51
mouse click event, 511
mouse double-click event, 511
mouse drag event, 511
mouse over event, 511
mouse point event, 511
multidimensional arrays, **346**
  one-dimensional/single-dimensional array, **345**
  three-dimensional arrays, **348**, 351
  two-dimensional array, **346**
multified records sorting
  sorting data, parallel arrays, 340–341
  sorting records as a whole, 341–342
multiple inheritance, **485**
multiplication operator (*), 48
multithreading, **526**, 526–527
mutator methods, **433**, 433–434

N

named constants, **46**
  declaring, 46–47
name(s), self-documenting, **69**
naming
  constants, 70
  improving efficiency, 248–250
  modules, 55
  subscripts, 248

variables, 42–43, 69
nested decisions, **134**, 136–139
replacing with arrays, 231–238
nested `ifs`, **134**, 136, 140
nested loops, **186**, 186–191
nesting structures, **94**, 94–95
nondefault constructors, **465**, 468–469
nonstatic methods, **447**
nonvolatile storage, **4**
`not eof`? question, 101
not-equal-to operator (<>), 130
NOT operator, **153**, 153–154
null case (null branch), 92
numeric constants, **39**
numeric data, **39**
numeric variables, 45

O

object(s), 421, 448–455
arrays, 453–455
attributes, **421**
passing to methods, 449–450
returning from methods, 450–453
object code, **4**
object dictionary, **519**
object-oriented programming (OOP), **27**, 421–456, 465–495
advantages, 494
classes, **421**, 421–424
composition, 474–475
constructors. *See* constructors
destructors, **472**, 472–473
encapsulation, **426**, 426–427
objects, **421**, 421–424
polymorphism, 424–425
predefined classes, 487–488
principles, 421–428
thread, 528
one-dimensional array, **345**
opening files, **278**
operands, **45**
operating systems, **508**
option buttons, GUIs, 511, 512
OR decisions, **143**, 146, 150, 152, 163
order of operations, **48**, 49
`OR` operator, **149**
combining with `AND` operator, precedence, 163–166
OR selection
avoiding common errors, 147–152
outer loop, **186**
out of bounds, 255
output, **3**
output devices, default, 282
output symbol, **17**
overhead, **47**, 58, **382**
overloading, **394**
constructors, 469–471
methods, **394**, 394–400, 469–471
overriding, **486**

P

packages, **487**
page schematics, **518**
parallel arrays, **244**, 244–250
parameterized constructors, **465**, 468–469
parameter list, **371**
parameter to the method, **371**
parent classes, **478**, 479
accessing private fields and methods, 481–485
overriding parent class methods in child classes, **486**
parent files, **282**, 300
parentheses (()), variable names within, 55
Pascal casing, **43**
passed by reference, **389**
passed by value, **375**
passing array, to method, 386–393

passing objects to methods, 449–450
paths, **274**
  dead (unreachable), 157
percent sign (%), remainder (modulo, modulus) operator, 51
permanent storage, devices, 273, **273**, 275
physical order, **352**
pixels, 513
planning, logic, 10
plus sign (+)
  addition operator, 47, 50
  public access specifier, 439
polymorphism, **394**, 424–425
populating the array, **229**
portable modules, **60**
posttest loops, **201**
precedence, combining `AND` and `OR` operators, 163
predefined classes, 487–488
priming inputs (priming reads), 99–106, **103**
primitive data types, **430**
printing, definition, 282
private access, **437**, 437–440
procedural programming, **27**
processing, 2, **2**
  symbol, **17**
production, putting programs into, 13–14
professionalism, structure use, 106
program(s), **2**
  coding, 3, 10–11
  ending with sentinel values, 20–23
  executing (running), 4
  good design features, 66–74
  interactive, 310
  main, 54
  structured, 88
  translating into machine language, 10–11
  unstructured, 88–90
program code, **3**
program comments, **67**, 67–69
program development cycle, 8–14
  coding step, 10
  maintenance step, 14
  planning logic step, 10
  production step, 13–14
  testing step, 12–13
  translating program into machine language, 11–12
  understanding problem step, 8–10
program-ending test, 105
program logic, 5–7
programmer-defined types, **429**
programming, **2**
  defensive, 207, 209
  environments, 23–25
  event-driven, 508–510
  good habits, 74
  multiple programmers, 54
  object-oriented, 27
  procedural, 27
  reusing work, 53–54
programming language, **3**
programming models, evolution, 27–28
prompts, 72–74
  clear, 72–74
properties, **433**
protected access specifier, **482**, 482–485
pseudocode, **15**, 15–16
  writing, 15–16
public access, **437**, 437–440

R

random access files, **308**, 308–309
random access memory (RAM), **4**
random-access storage device, **352**

range checks, **155**, 157–160
avoiding common errors, 157–160
reading from files, **278**, 278–281
read(s), priming, 99–106
real estate, 515
real numbers, **39**
real-time applications, **309**
reasonableness of data, validating, 213–214
records, **276**
recursion, **405**, 405–409
recursive cases, **406**
recursive method, **405**
refining bubble sort, to eliminate unnecessary passes, 336–340
register components, **522**
relational comparison operators, **129**, 129–134
relationships
has-a, **474**, 474–475
is-a, **423**
whole-part, **474**, 474–475
reliability, **54**
reliable code, 486
remainder operator, **51**
repetition, **92**
reports
control break, 286, 287, 289, 290
summary, **208**
returning value, creating methods, 379–385
`return` statement, 381
modules, **54**
return type, **379**
reusability, **54**
right-associativity, **45**
right mouse click event, 511
right-to-left associativity, **45**
rules of precedence, 48, **48**, 49
running a program, 4

S

score-sorting application, 333–335
screen blueprints, **518**
script(s), **510**
scripting languages (scripting programming languages; script languages), **5**
searches
binary, **249**
linear, **241**
searching arrays
exact matches, 240–243
improving efficiency, 248–250
parallel arrays, 244–250
range matches, 250–254
selection(s)
loops compared, 215
within ranges, 155–160
selection sort, **343**, 344
selection structure, **91**, 91–92, 101, 125–129
self-documenting names, **69**
semicolon (;), separating actions, 200
sentinel symbols, flowcharts, 55
sentinel values, **22**
ending programs, 20–23
sequence structure, **90**
sequential files, **280**
sequential order, **322**
set methods, **433**, 433–434
short-circuit evaluation, **139**
signature, **372**
single-alternative `if`s (single-alternative selections), **92**
single-dimensional array, **345**. *See also* one-dimensional array
single-level control breaks, **286**
size of the array
constants as, 238
slash (/), division operator, 48

`sleep()` method, 528, 529
snake casing, **43**
software, **2**
  design, advantages of inheritance, 486–487
  translating programs into machine language, 11–12
sorting, data, **280**
  numeric values, 323
  sequential order, 322
source code, **4**
source of the event, **510**
spaghetti code, **88**, 88–90
stack, **58**, **406**
stacking structures, **93**, 94
starvation, **527**
state, **423**
statements
  clarity, 71–72
  long, temporary variables to clarify, 71–72
  throw, **491**
static methods, **447**, 447–448
step values, **199**
storage
  devices, **3**
  temporary and permanent, 273
storyboards, **518**
  object dictionary, 519
string constants, **40**
string data, **39**
string variables, **41**
strongly-typed languages, 41
structure(s), 87–115, **90**
  combining, 93–99
  priming input to structure programs, 99–106
  reasons for, 106–107
  recognizing, 107–110
  selection, 91–92, 147, 149
  sequence, **90**
  spaghetti code, 88–90
  structuring and modularizing unstructured logic, 110–115
structure charts, 64
structured programs, **88**, 88–90
subclasses, **478**, 479
  ancestors, **479**
subprocedures, 51
subroutines, 51. *See also* module(s)
subscripts
  bounds, 255–258
  constants, 239
  naming, 250
subtraction operator (-), 47, 50
summary reports, **208**
sunny day case, **493**
superclasses, **478**
swap values, **324**, 324–325
swipe event, 511
symbols, flowcharts, 15–20, 67, 201
syntax, **3**, 11
syntax errors, 3, 4, 11
system software, **2**

T

tables, **277**, **348**
tap event, 511
temporary storage, 273, 275
temporary variables, **71**
  clarifying long statements, 71–72
terminal symbols, **17**
testing
  loop control variables, 192–194
  programs, 12–13
text boxes, GUIs, 511, 512
text editors, 23
text files, **273**

this references, 443, **443**, 444–445
thread(s), **525**, 525–528
`Thread` class, 528
thread synchronization, **528**
three-dimensional arrays, **348**
throwing an exception, **491**
`throw` statements, **491**
tight coupling, **404**
tilde (~), destructors, 472
totals, accumulating using loops, 205–208
transaction files, **299**, 299–307
translating programs into machine language, 11–12
trivial expressions, **131**
truncation, 50
truth tables, **139**
try blocks, **491**
trying code, **491**
Turing, Alan, 27
two-dimensional array, **346**
type-safety, **41**

U

unary operators, **153**
uninitialized variables, 206
unnamed constants, **39**
unreachable paths, **157**
unstructured programs, **88**, 88–90
updating a master file, **299**
upper camel casing, **43**
user(s), **8**
  class, **424**
user-created exceptions, 493
user-defined types, **429**
user environments, 25–26
user-initiated actions, 511
user screens, defining connections between, 520

V

validating data, **209**
  data types, 212–213
  reasonableness and consistency of data, 213–214
values, assigning to variables, 45–46
variables, **6**, 40–41
  assigning values, 45–46
  data dictionaries, 70
  data types, 45–46
  declaring within modules, 58–60
  decrementing, 181
  global, **60**
  incrementing, 181
  initializing, 45–46
  instance, 423, **423**
  local, **60**
  naming, 42–44, 69
  numeric, 45
  parentheses around names, 55
  string, **41**
  uninitialized, 206
variable `workhours`, 382
visible, data items, **60**
visual development environments, **488**
void method, **379**
volatile storage, **4**

W

`while`, `do` loops, **93**
`while` loops, **92**, 92–93, 199–201
whole-part relationships, 474–475
wireframes, **518**
work methods, **435**, 435–436
work variables, **71**
writing, definition, 281
writing to files, 281

X

x-axis, **529**
x-coordinate, **529**

Y

y-axis, **529**
y-coordinate, **529**

Z

zero, uninitialized variables, 206
zoom event, 511